

A PRACTICAL HANDBOOK FOR SENIOR ENGINEERS

Advanced Rust

*Ownership, concurrency, performance, and production
systems — 56 chapters across eight parts.*

O. Iakushkin

Contents

Part I · Getting Your Bearings in Rust

1	Why Rust Feels Different	5
2	The Rust Mental Model	9
3	Project Structure and Tooling	12

Part II · Memory, Ownership, and Data Layout

4	Ownership, Borrowing, and Lifetimes	21
5	Pointers, References, and Ownership Inside Structs	30
6	Pointers, References, and Ownership Inside Vectors	39
7	Copying Data vs Cloning Data	50
8	Undefined Behavior and Unsafe Rust	60
9	Smart Pointers and Pinning	71
10	Arrays, Slices, and Vectors	81
11	Hash Maps and Sets	89
12	Matrices and Multidimensional Data	98
13	Arena Allocation and Region-Based Memory	107

Part III · Designing Types and Abstractions

14	Interfaces in Rust: Traits	117
15	OOP Models in Rust	126
16	Domain-Driven Design in Rust	137
17	Refactoring Toward Idiomatic Rust	149
18	Generics Instead of Templates	157
19	Serialization and Data Contracts	164
20	Metaprogramming	180
21	Reflection and Type Introspection	191

Part IV · Concurrency, Async, and Systems IO

22	Multithreading in Rust	202
23	Synchronization Primitives	212
24	Coroutines, Futures, and Async Rust	223
25	Tokio	235
26	Task Libraries and Parallel Execution	244
27	IO Tricks and Systems Programming Patterns	254

Part V · Integration, Distributed Systems, and HPC

28	C++ Integration	267
29	JavaScript and C++ Integration for WASM	279
30	AMQP and Message Brokers	288
31	Distributed Task Execution	297
32	MPI and High-Performance Computing	309
33	Performance-Oriented Rust	320
34	Memory Profiling	331
35	Performance Profiling	342
36	Distributed Tasks Profiling	354
37	CUDA and GPU Acceleration	366
38	Merkle Tree Games and Challenges	379
39	Graph Search Games	390
40	Matrix Optimization Games	402

Part VI · Engineering Systems for Production

41	Error Handling in Large Systems	417
42	Testing Advanced Rust Systems	431
43	Observability	442
44	Packaging and Deployment	453
45	Capstone: A Distributed Rust System	462

Part VII · Networked Services and Secure Boundaries

46	FastAPI-Style Web Apps, Swagger, and OpenAPI Codegen	476
47	gRPC Services with Protobuf and Service API Codegen	491
48	WebSockets and Long-Lived Connections	504
49	HTTPS, TLS, and Secure Service Boundaries	516
50	libp2p and Peer-to-Peer Rust Systems	527

Part VIII · Frontier Specializations

51	Zero-Knowledge Proofs for Rust Engineers	541
52	ZoKrates Workflows and Ethereum Verifiers	551
53	EZKL, Verifiable LLM Inference, and GPU-Aware ZKML	563
54	ONNX Model Inference in Rust	575
55	Smart Contracts in Rust: Solana, Sei, and Beyond	584
56	no-std Rust and Constrained Runtime Derivatives	594

PART I

Getting Your Bearings in Rust

Recalibrate your mental model and stand up a working toolchain before you fight the borrow checker.

-
- 1** Why Rust Feels Different
 - 2** The Rust Mental Model
 - 3** Project Structure and Tooling
-

CHAPTER 1 · WHY RUST FEELS DIFFERENT

Why Rust Feels Different

Control without unsafety by default, zero-cost abstractions, and compiler-guided design

Production Rust is valuable because ownership, failure handling, concurrency, and deployment rules become explicit contracts that teams can review before release.

Opening scenario

A payments platform is moving latency-sensitive services to Rust after repeated incidents in resource ownership, failure recovery, and concurrent state updates. The business requirement is direct: make ownership, mutation authority, and fallibility explicit enough for release review before the system grows.

What Rust is asking you to state up front

- Name the owner of the data or resource.
- Say whether the next step only borrows, mutates exclusively, or takes ownership outright.
- Make fallibility and concurrency visible where the boundary actually changes.

A reading checklist

Mental model

Rust is not “C++ with a stricter compiler,” “C# without a GC,” or “Go with harder syntax.” The operational model is different: values have owners, borrows describe temporary access, and APIs must state whether they transfer responsibility, share read access, or require exclusive mutation.

- Ownership says who is responsible for cleanup.
- Borrowing says who may access a value temporarily.
- Traits describe behavior; generics usually compile via monomorphization rather than dynamic dispatch.
- Thread boundaries carry trait requirements like Send and Sync.

Core concepts

Rust’s core promise: control without unsafety by default

Rust wants C-like control over layout, allocation, and destruction while keeping memory unsafety out of ordinary code. Safe Rust rules out use-after-free, double-free, and data races in the safe subset. When low-level work truly needs escape hatches, you use unsafe in small, auditable regions and document the invariants there.

Zero-cost abstractions in practice

“Zero-cost” does not mean “free magic.” It means abstractions do not inherently add overhead relative to a hand-written equivalent. Iterator pipelines, enums, and generics often compile to code close to an explicit

loop. Traits become dynamic only when you opt into trait objects; otherwise, generic code is usually monomorphized.

The senior-developer Rust learning curve

The learning curve is mostly architectural, not syntactic. Rust pushes design choices earlier: where is ownership transferred, which references may outlive which scopes, which types are allowed across threads, and where should cloning be explicit? Later chapters go deeper on borrowing, traits, and async, but this chapter's goal is to make the early discomfort interpretable.

Rust compared with C++, C#, and Go

The most useful comparison is not syntax. It is where each language places trust: C++ trusts discipline around aliasing, C# trusts a managed runtime, Go trusts a runtime plus simplified concurrency primitives, and Rust spends more compile time to reduce what must be trusted in production.

what changes by background

Why this feels slower before it feels faster

Compile-time guarantees vs runtime checks

- Move-after-use and many lifetime mistakes
- Overlapping mutable borrows
- Thread-boundary safety via trait bounds
- Missing exhaustiveness in pattern matching
- Allocation, syscalls, and network failures
- Bounds checks when data-dependent
- Parsing invalid input and I/O availability
- Contention, cache misses, and real performance costs

Reading compiler errors productively

1. Read the violated rule first, not just the red text.
2. Find the origin span: where was the move, borrow, or trait bound introduced?
3. Change ownership shape before changing syntax.
4. Only clone or heap-share after deciding that duplication or sharing is actually correct.

Typical repair:

Production patterns

Pitfalls and tradeoffs

A borrow-checker error is not an obstacle to work around. It is exposing an ownership protocol you have not stated clearly enough yet. Treat that as design feedback, especially in production code.

Examples

Example 1: iterator pipeline with no extra abstraction tax

This reads like high-level code, but the abstraction can still compile down close to an ordinary loop.

Example 1-1. `zero_cost_pipeline.rs` — *Rust pipeline*

```
fn critical_count(readings: &[i32], threshold: i32) -> usize {
    readings
        .iter()
        .copied()
        .filter(|reading| *reading >= threshold)
        .count()
}

fn main() {
    let readings = vec![42, 87, 91, 63, 99];
    let count = critical_count(&readings, 80);
    println!("critical count = {}", count);
}
```

OUTPUT

```
critical count = 3
```

Quick check: press Run first to confirm the baseline output, then change the threshold or input data and see how the pipeline stays readable while remaining executable.

Iterator adapters, closure types, and the final return type describe structure. In ordinary generic code, that structure does not imply a heap allocation or a vtable lookup.

The CPU still walks the slice, evaluates the predicate, branches, and counts matches. Zero-cost abstractions remove abstraction tax, not the cost of real work.

Example 2: ownership handoff into a worker thread

Moving the job into the closure makes the lifetime and thread boundary explicit.

Example 1-2. `ownership_thread_handoff.rs` — *Thread handoff*

```
use std::thread;

fn main() {
    let job = String::from("rebuild-search-index");

    let handle = thread::spawn(move || {
        println!("worker started: {}", job);
    });

    handle.join().unwrap();
}
```

OUTPUT

```
worker started: rebuild-search-index
```

Quick check: reproduce the expected output, then change the job name or move more data into the closure to explore how ownership makes the thread boundary explicit.

The move closure consumes captured non-Copy data so the worker owns what it needs. That makes the handoff explicit instead of leaving lifetime questions to convention.

Queues, threads, and async tasks are ownership boundaries. Borrow locally when you can, but hand off owned work items when execution may outlive the current scope.

Next chapter

Chapter 02 turns this intuition into an operational model: values, bindings, moves, drops, stack versus heap-backed ownership, and why lifetimes are about borrowed references rather than object survival.

Summary

- Rust feels different because it makes resource protocols part of the type-checked API instead of part of team folklore.
- The goal is not abstract safety rhetoric. It is earlier feedback on ownership, mutation, and thread-boundary mistakes.
- Zero-cost abstractions remove abstraction tax, not the real cost of parsing, allocation, synchronization, or IO.
- Once you can read compiler errors as design notes, later chapters get much easier to use well.

CHAPTER 2 · THE RUST MENTAL MODEL

The Rust Mental Model

Values, bindings, moves, drops, stack versus heap, expressions, RAII, and lifetimes

Reliable Rust services depend on a precise model of values, ownership transfer, drops, stack and heap storage, expressions, and references. This chapter defines that model for code review and debugging.

Builds on Chapter 01

Chapter 01 explained why Rust feels different at a high level. This chapter turns that intuition into an operational model: who owns a value, what actually moves, what gets dropped, and why lifetimes are proofs about references rather than a runtime memory system.

Opening scenario

A message-ingestion service parses inbound bytes, enriches records, and dispatches owned jobs to workers. The business requirement is a precise value lifecycle: where data is created, which component owns it, where heap storage is used, and when cleanup occurs.

Operational checklist

Mental model

Core concepts

translating prior instincts

Two corrections that remove a lot of confusion

The useful model is simpler: ownership transferred. For small fixed-size `Copy` values the data is copied. For non-`Copy` types, think of the old binding as no longer owning the value. That is the rule that matters when reading APIs and diagnostics.

Owned values live until their owner is dropped. Lifetimes constrain references that point at those values. If the owner is dropped, no annotation can keep a reference to it valid.

Expressions, statements, and blocks in one pass

A statement performs work in the enclosing scope. It does not become the value of that scope.

An expression produces a value. In Rust, `if`, `match`, and blocks are often value-producing tools, not only control-flow syntax.

The last line without a semicolon becomes the block's value. Add a trailing semicolon and the block evaluates to `()` instead.

Lifetime repair rule of thumb

When a lifetime error appears, repair ownership before you reach for annotations. Usually the real choice is one of two shapes: borrow from caller-owned input that already lives long enough, or return an owned value so the function transfers data instead of a reference. Randomly adding 'a to a signature rarely fixes the model because lifetimes describe valid borrowing relationships; they do not extend how long an owner lives.

Production patterns

Pitfalls and tradeoffs

Deterministic destruction is powerful, but it is not magic. Destructors should release resources and maintain invariants, not hide slow network calls, lock acquisition chains, or critical business logic that is hard to reason about during unwinding and shutdown.

Examples

Example 1: move a value and observe deterministic drop

The binding in `main` stops owning the value after the function call, and cleanup runs when the new owner leaves scope.

Example 2-1. `move_and_drop_timeline.rs` — *Move drop*

```
struct FileHandle {
    name: String,
}

impl Drop for FileHandle {
    fn drop(&mut self) {
        println!("drop {}", self.name);
    }
}

fn ship(handle: FileHandle) {
    println!("shipping {}", handle.name);
}

fn main() {
    let handle = FileHandle {
        name: String::from("audit.log"),
    };

    println!("before move");
    ship(handle);
    println!("after ship");
}
```

OUTPUT

```
before move
shipping audit.log
```

```
drop audit.log
after ship
```

Quick check: run the baseline once, then rename the file or move more fields through the handoff to make the ownership timeline concrete.

Example 2: use block expressions to build owned heap data with narrow mutation

The outer bindings stay immutable while a short inner block uses mutation to assemble the final value.

Example 2-2. `expression_blocks_and_heap.rs` — *Blocks heap*

```
fn main() {
    let queue_depth = 2048;

    let status = if queue_depth > 1024 { "hot" } else { "steady" };

    let labels = {
        let mut labels = Vec::with_capacity(8);
        labels.push(String::from("ingest"));
        labels.push(String::from("priority"));
        labels
    };

    let capacity = {
        let heap_slots = labels.capacity();
        heap_slots
    };

    println!("status = {}", status);
    println!("capacity = {}", capacity);
}
```

OUTPUT

```
status = hot
capacity = 8
```

Quick check: confirm the expected output, then experiment with the block contents and see how narrow mutation still produces an owned final value.

Summary

- Rust gets calmer when you treat bindings as names and ownership as the cleanup contract.
- Stack and heap explain storage placement; ownership explains who is responsible for the value that uses it.
- Immutability by default and expression-oriented blocks push mutation into narrower, more reviewable scopes.
- RAII in Rust is explicit and deterministic, which is exactly why `Drop` should stay small and unsurprising.
- Lifetimes talk only about borrowed references. They never extend the life of owned data.

CHAPTER 3 · PROJECT STRUCTURE AND TOOLING

Project Structure and Tooling

Cargo, crates, workspaces, modules, features, dependencies, and a practical workflow

Large Rust repositories need predictable crate boundaries, module visibility, feature flags, dependency policy, and CI commands. This chapter sets the project structure that keeps builds repeatable and maintainable.

Repository note

The current repository is a web book template, not a live Cargo workspace. To stay aligned with the existing template, this chapter uses standalone Rust example files under `examples/ch03_project_structure_and_tooling/` and explains the Cargo layouts you would create in a real Rust repository.

Opening scenario

A product repository must deliver an API service, a worker, a support CLI, and a shared domain model from one Rust workspace. The business requirement is clear package ownership, crate boundaries that match release and dependency policy, module visibility that protects invariants, and repeatable Cargo commands in CI.

Mental model

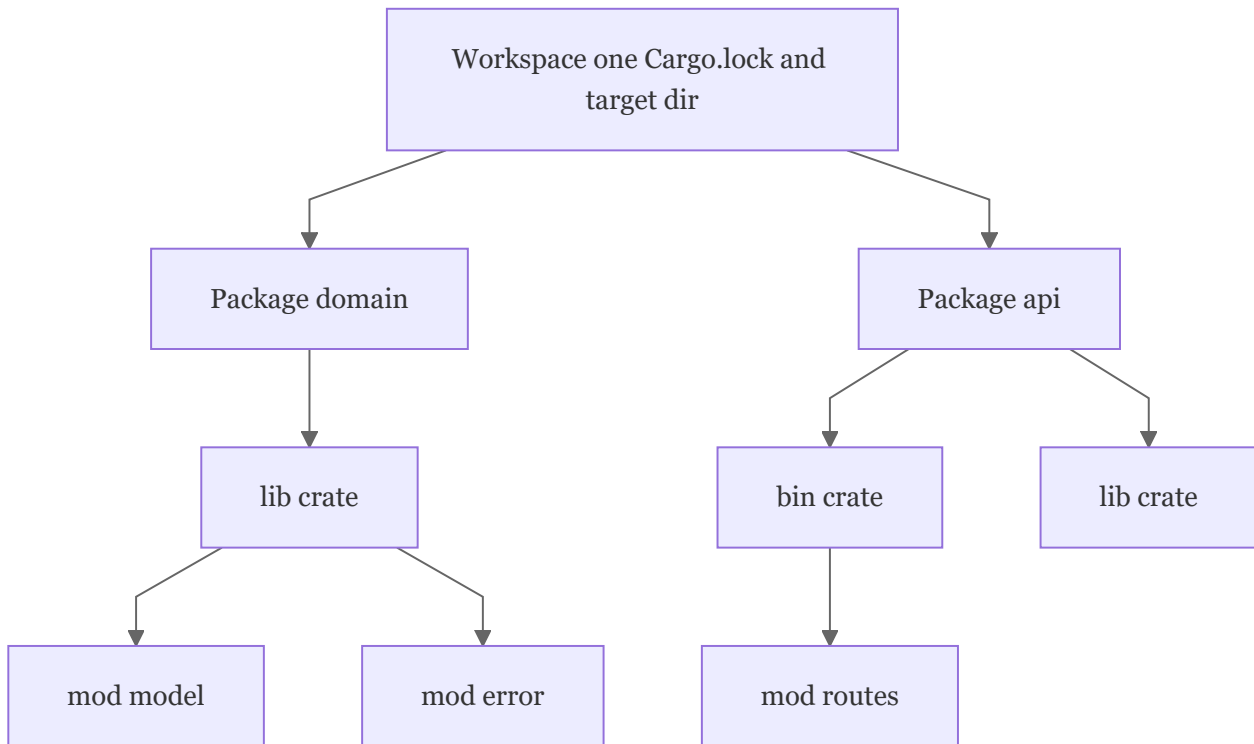
Core concepts

Cargo, crates, packages, and workspaces

The easiest mistake is to treat these words as synonyms. They are not. A package is what Cargo manages from one `Cargo.toml`. A crate is what the compiler builds. A workspace groups packages so the repository can share one `Cargo.lock`, one target directory, and one top-level command set.

In practice, start with fewer crates than your instincts may suggest. A new crate is justified when it gives you a real boundary: dependency isolation, compile-time separation, reuse with a stable surface, or a clean public/private split. If the change is only organizational, a module is often the cheaper unit.

The relationship is strictly nested, and the diagram is worth fixing in memory before anything else: a workspace contains packages, each package is defined by exactly one `Cargo.toml` and produces one or more crates, and each crate contains a tree of modules. Note where the units do and do not line up: one package can yield several crates, but a module never escapes its crate.



Workspace contains packages, a package can build several crates, and modules live inside one crate. Cargo manages the package; the compiler manages the crate.

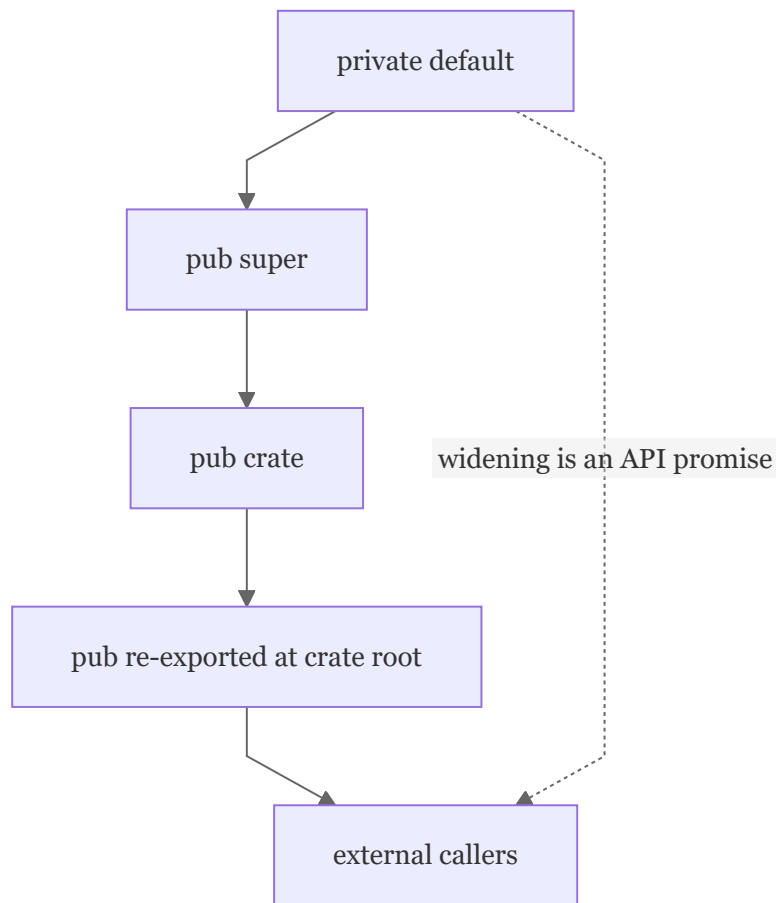
- The dependency graph truly differs and you want to keep heavyweight crates out of the core.
- The package exports a stable API consumed by multiple binaries or external users.
- The team wants a separate release cadence or a separately testable integration layer.
- You need a compile boundary, not just a folder boundary.

Modules and visibility

Modules are how you hide detail inside a crate. The default visibility is private, which is a strong default to keep. Reach first for `pub(crate)`, then `pub(super)`, and only then full `pub` when you truly mean external callers.

The crate root is where you shape the surface. Internal modules may stay deep and specific while the public API becomes smaller and calmer through deliberate `pub` use re-exports. That keeps callers stable while giving you room to refactor internals.

What to look at: visibility is a ladder, not a switch. Each rung widens who can reach an item, and each rung up is harder to walk back later because more callers can depend on it. Climb only as far as a real caller forces you to, and shape the public path with a deliberate `pub` use at the crate root.



Each step widens the audience for an item. Default private is the cheapest to change; a name re-exported at the crate root is the hardest, because outside code can now depend on it.

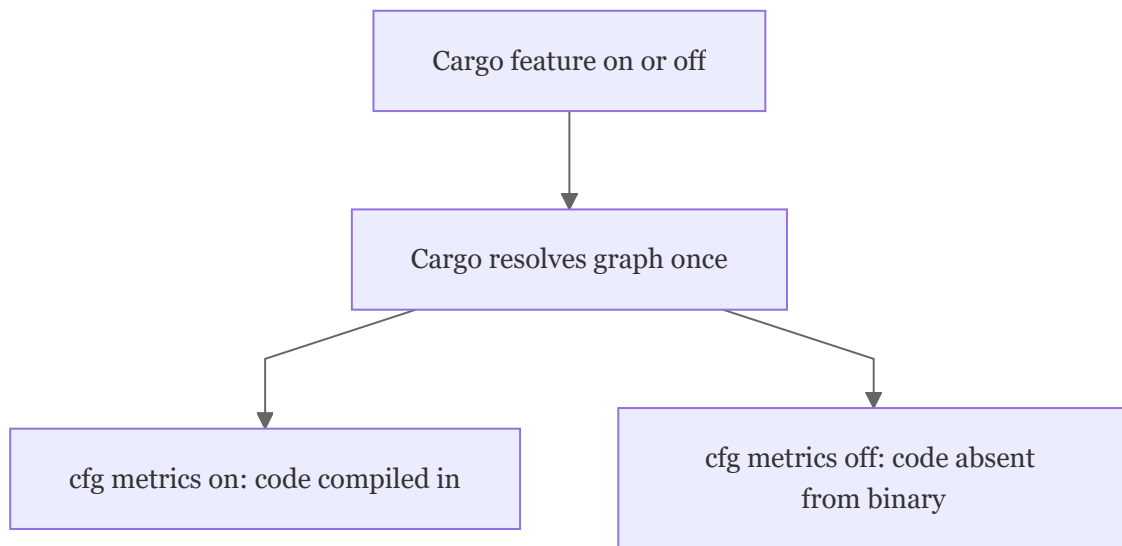
- Fields stay private until callers truly need direct structural access.
- Constructors and accessors are often cheaper than exposing layout forever.
- Re-export names at the crate root when you want a stable path but flexible internals.

Features and conditional compilation

Cargo features work best as additive capability switches: optional integrations, platform-specific plumbing, or extra instrumentation. The real tools are compile-time conditions such as `#[cfg(feature = "metrics")]`, `#[cfg(test)]`, and optional dependencies.

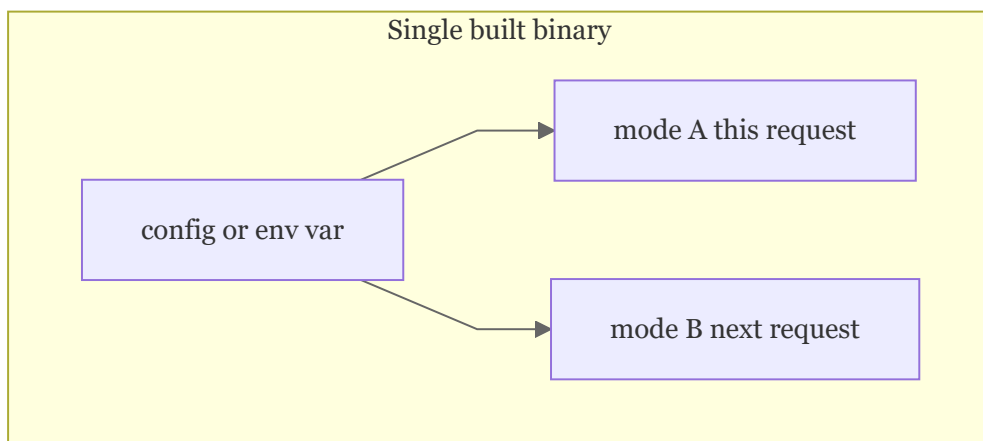
The trap is to encode runtime mode systems as feature matrices. Features are resolved once for the whole build graph. If the question is “what mode is this service in today?”, a runtime config file or environment variable is usually the better abstraction.

What to look at in the diagram: the feature is decided once, on the left, before the compiler runs, and everything downstream is fixed for that whole build. A runtime decision sits on the opposite side — it branches per request inside a single binary. Choosing the wrong side is the classic feature-flag mistake.



Compile-time side: a feature is a fork resolved once, before the binary exists. The code is either compiled in or absent from the shipped binary.

The runtime side is the opposite: one already-built binary that branches per request.



Runtime side: a mode is a branch inside the one binary you shipped, chosen per request. Capability belongs on the compile-time side; operational state belongs here.

Keep feature design boring. Additive features compose. Mutually exclusive features multiply test matrices, documentation burden, and integration surprises.

Dependency management

Be strict about direct dependencies. Each direct crate becomes part of your maintenance surface, your security review surface, and sometimes your public API surface. If several packages share the same policy, centralize it with workspace-level dependency declarations such as `workspace.dependencies` instead of duplicating versions.

Commit `Cargo.lock` for binaries and services so production resolution stays reproducible. Library crates also keep a lockfile in the repository for CI sanity, but downstream consumers still resolve their own graph.

- Prefer fewer transitive surfaces over convenient one-off crates.
- Audit default features before accepting them blindly.

- Ask whether a dependency belongs in the core crate or at an outer adapter layer.
- Review version and feature changes the way you review schema and protocol changes.

Testing, benchmarking, linting, and formatting

The repository also includes `examples/cho3_project_structure_and_tooling/testing_layers.rs` as a small, standalone example of colocated tests.

A recommended Rust workflow

1. Sketch the crate boundary first: core domain, adapters, binaries, and external integration points.
2. Start with modules inside one crate, then split crates only where the dependency or API boundary is real.
3. Run fast structural checks early and often: check, test, format, then Clippy.
4. Keep feature flags additive and dependency additions reviewed, not casual.
5. Only benchmark after the boundary and ownership model are stable enough to interpret the numbers.

How prior instincts translate

Almost every confusion in this chapter comes from carrying a structural assumption from another language and quietly assuming Rust agrees. It usually does not. The cards below name the one shift to make for each background before any of the concrete layout advice will land.

Production patterns

Pitfalls and tradeoffs

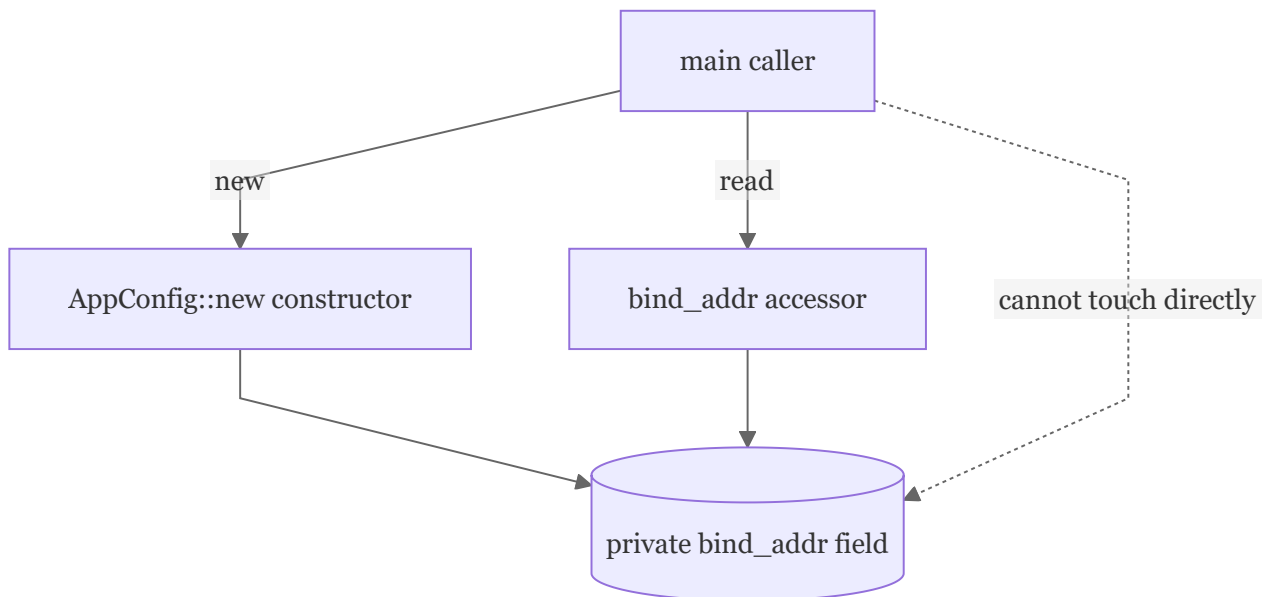
Repository structure is architecture. In Rust, visibility, crate boundaries, and feature design are not cleanup chores after the code works. They are part of how the code keeps working.

Examples

Example 1: shape a narrow module surface

Keep layout private where possible and expose behavior or focused accessors instead of raw structure.

What to look at: `service_name` is public, but `bind_addr` is a private field reached only through a method. Callers get the value without getting the right to depend on how it is stored, which is exactly the seam the diagram traces.



The caller reaches the private field only through the constructor and accessor. The dashed edge is the path the compiler refuses, which is what frees you to change the field's storage later.

Example 3-1. module_visibility_service.rs — Module visibility

```

mod config {
    pub struct AppConfig {
        pub service_name: String,
        bind_addr: String,
    }

    impl AppConfig {
        pub fn new(service_name: &str, bind_addr: &str) -> Self {
            Self {
                service_name: service_name.to_string(),
                bind_addr: bind_addr.to_string(),
            }
        }

        pub fn bind_addr(&self) -> &str {
            &self.bind_addr
        }
    }
}

fn main() {
    let config = config::AppConfig::new("api", "127.0.0.1:8080");
    println!("{}", config.service_name, config.bind_addr());
}

```

OUTPUT

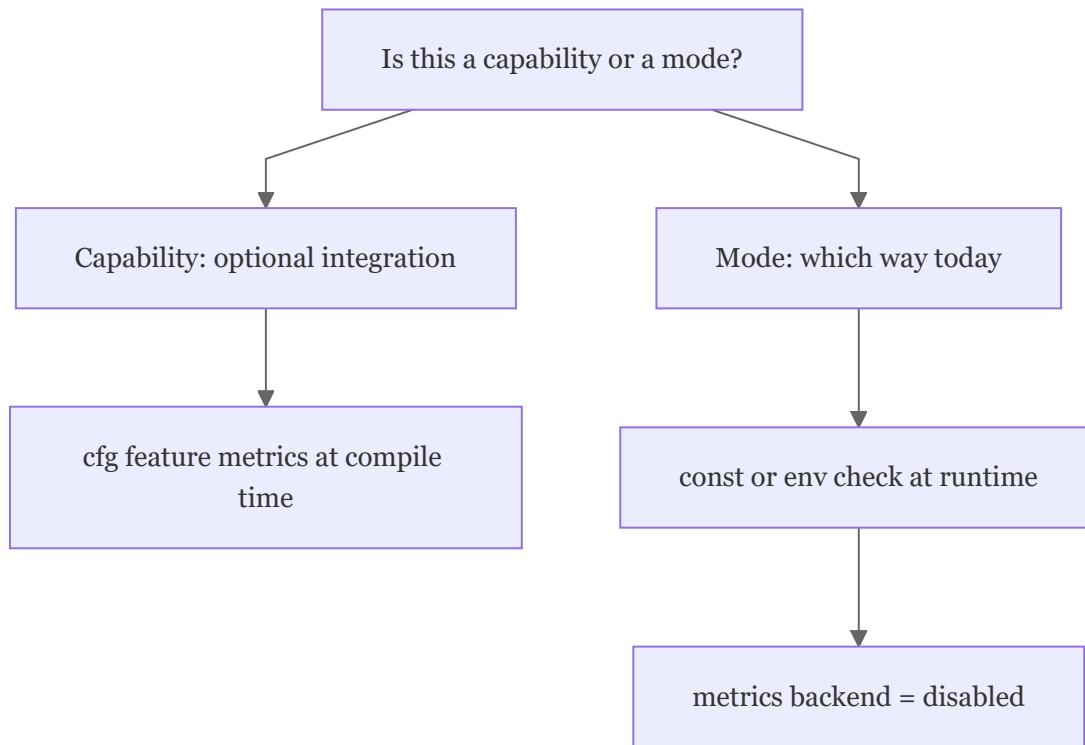
```
api@127.0.0.1:8080
```

Quick check: run the baseline, then change the constructor arguments and keep the private field private. The point is not accessor boilerplate; it is preserving the right to change internal layout later.

Example 2: model an optional capability cleanly

The editor uses a simple constant so you can run the idea in one file. The repository example `feature_gated_metrics.rs` shows the real `#[cfg(feature = "metrics")]` shape.

What to look at: the example uses a plain `const` so it runs in one file, but read it as the right-hand runtime branch in the diagram below. The exercise is to decide whether this choice should instead move left, to a compile-time feature, so the unused backend never enters the binary at all.



The decision tree behind the example: a true optional capability belongs in a Cargo feature, while a 'which mode right now' question belongs at runtime. The `const` here stands in for the runtime branch.

Example 3-2. `feature_gated_metrics.rs` — Feature flags

```

// In a real crate, a Cargo feature would usually select this at compile time.
const METRICS_ENABLED: bool = false;

fn metrics_backend() -> &'static str {
    if METRICS_ENABLED {
        "prometheus"
    } else {
        "disabled"
    }
}

fn main() {
    println!("metrics backend = {}", metrics_backend());
}
  
```

OUTPUT

```
metrics backend = disabled
```

Quick check: toggle `METRICS_ENABLED` between false and true. Then ask whether the decision really belongs at runtime or should move to a compile-time feature.

Summary

- Package, crate, module, and workspace are different units with different jobs. Keep the distinctions crisp.
- Split crates only where the architectural boundary is real; use modules for cheaper internal structure.
- Visibility is design. Narrow it by default and shape public API intentionally at the crate root.
- Features work best as additive compile-time capability switches, not as a runtime mode system in disguise.
- A strong Rust workflow is ordinary and repeatable: check, test, format, lint, and benchmark deliberately.

PART II

Memory, Ownership, and Data Layout

Ownership, borrowing, lifetimes, smart pointers, and the containers you reach for daily — the part that makes every later API make sense.

-
- 4** Ownership, Borrowing, and Lifetimes
 - 5** Pointers, References, and Ownership Inside Structs
 - 6** Pointers, References, and Ownership Inside Vectors
 - 7** Copying Data vs Cloning Data
 - 8** Undefined Behavior and Unsafe Rust
 - 9** Smart Pointers and Pinning
 - 10** Arrays, Slices, and Vectors
 - 11** Hash Maps and Sets
 - 12** Matrices and Multidimensional Data
 - 13** Arena Allocation and Region-Based Memory
-

CHAPTER 4 · OWNERSHIP, BORROWING, AND LIFETIMES

Ownership, Borrowing, and Lifetimes

Ownership as the core design tool, borrowing rules, lifetimes, elision, and ownership-shaped APIs

Ownership, borrowing, and lifetimes define who may read, mutate, retain, or transfer data at every API boundary. This chapter applies those rules to production interfaces and resource management.

Builds on Chapters 02 and 03

Chapter 02 established values, moves, drops, and lifetimes as reference constraints. Chapter 03 showed that repository boundaries are also ownership boundaries. This chapter connects those models into the day-to-day rules you will feel in function signatures, method design, worker handoff, and API shape.

Opening scenario

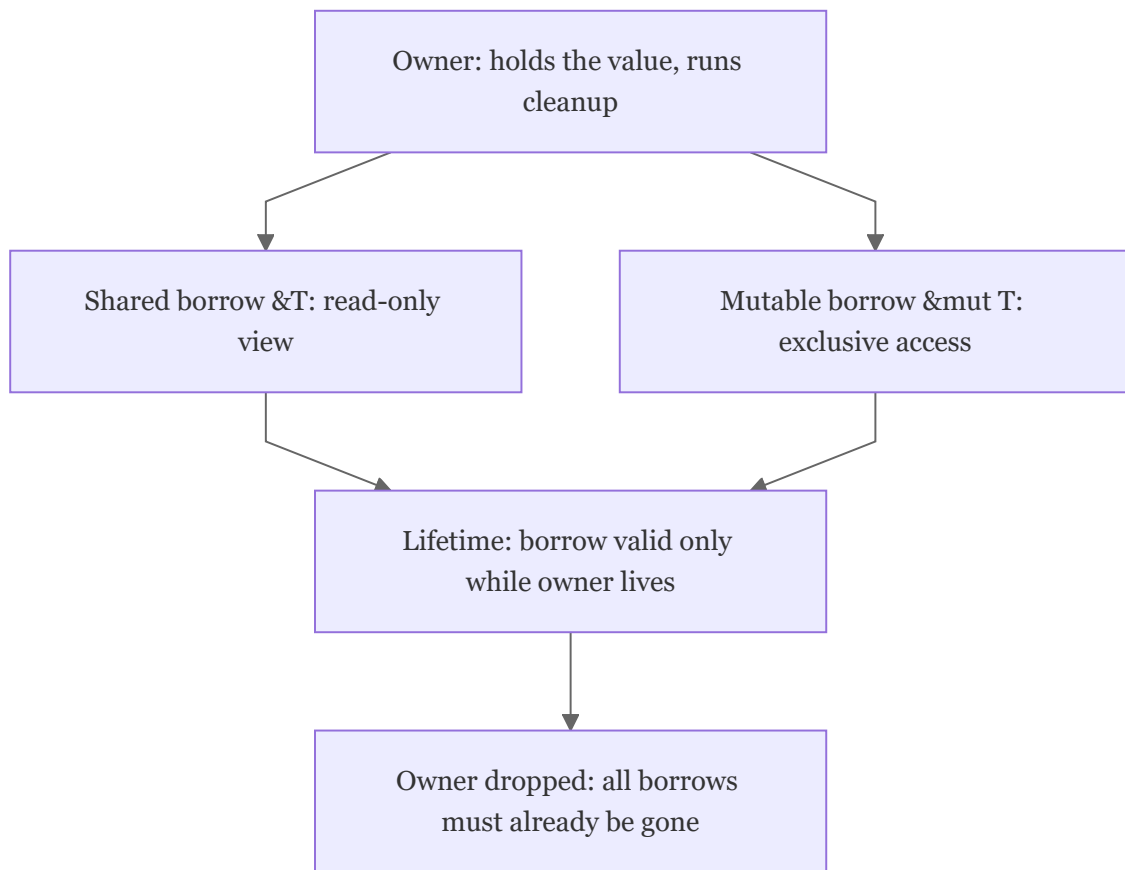
An ingress service parses HTTP data, derives routing keys, emits audit labels, and queues work for downstream processors. The business requirement is explicit data ownership at each layer: request bytes stay borrowed only while local, and derived outputs become owned when they cross queues, caches, or worker boundaries.

A reliable order for asking the questions

1. Who owns this value right now?
2. Can the next operation borrow it, or must it take ownership?
3. If a reference is returned, which input owner keeps that data alive?
4. If the answer is awkward, should this API return owned data instead?

Mental model

The three ideas stack in one direction. Ownership is the foundation; a borrow is a temporary permission granted by an owner; a lifetime is just the compiler's name for the span during which that borrow stays valid. Read the diagram top to bottom and notice that nothing below the owner can outlive it.



Ownership sits underneath; borrows are permissions the owner grants; the lifetime is the window in which a borrow is usable. The owner cannot be dropped while any borrow is still live.

Core concepts

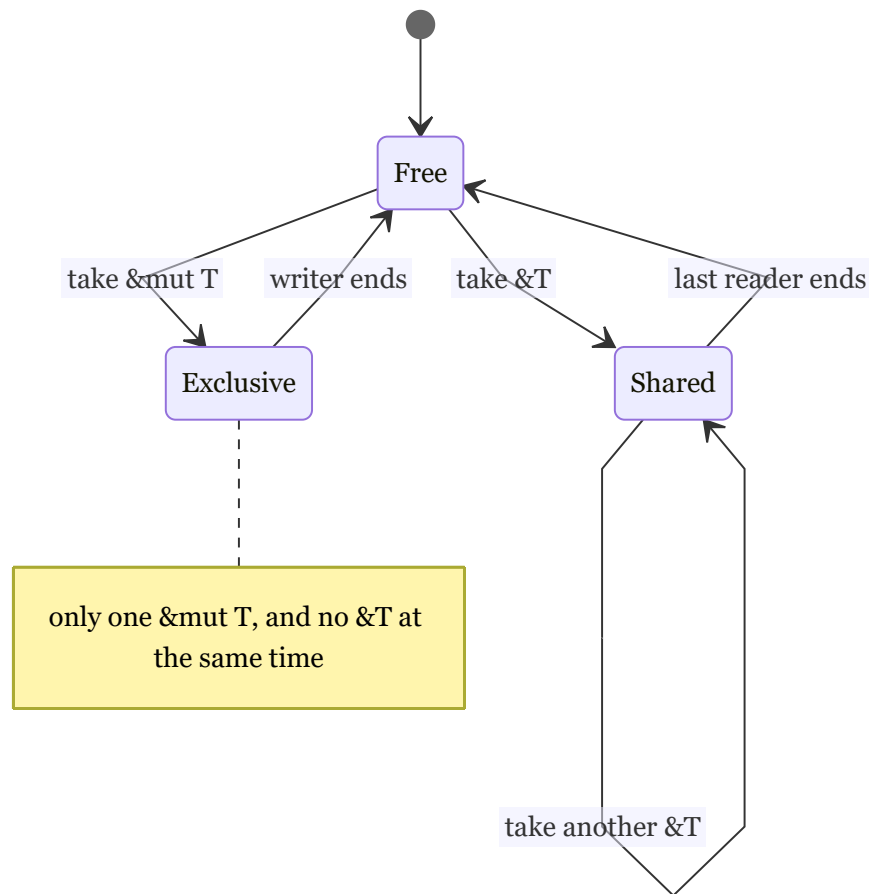
Ownership as Rust's central abstraction

Ownership is the reason moves, drops, borrows, thread handoff, and many API decisions line up into one model. The owner is the place where cleanup responsibility lives. Once that is clear, borrowing becomes a temporary permission story instead of a second memory-management model.

This is why Rust often feels strict at subsystem edges. Queues, async tasks, caches, and stored structs are all ownership decisions. They force you to state whether the next stage only observes data or must outlive the current owner.

Borrowing rules

The whole borrow checker reduces to one exclusivity rule, and it is easier to hold as a small state machine than as a paragraph. At any moment a value is in exactly one of three states: nobody is borrowing it, several readers are, or one writer is. The forbidden combination is the one the diagram never lets you reach.



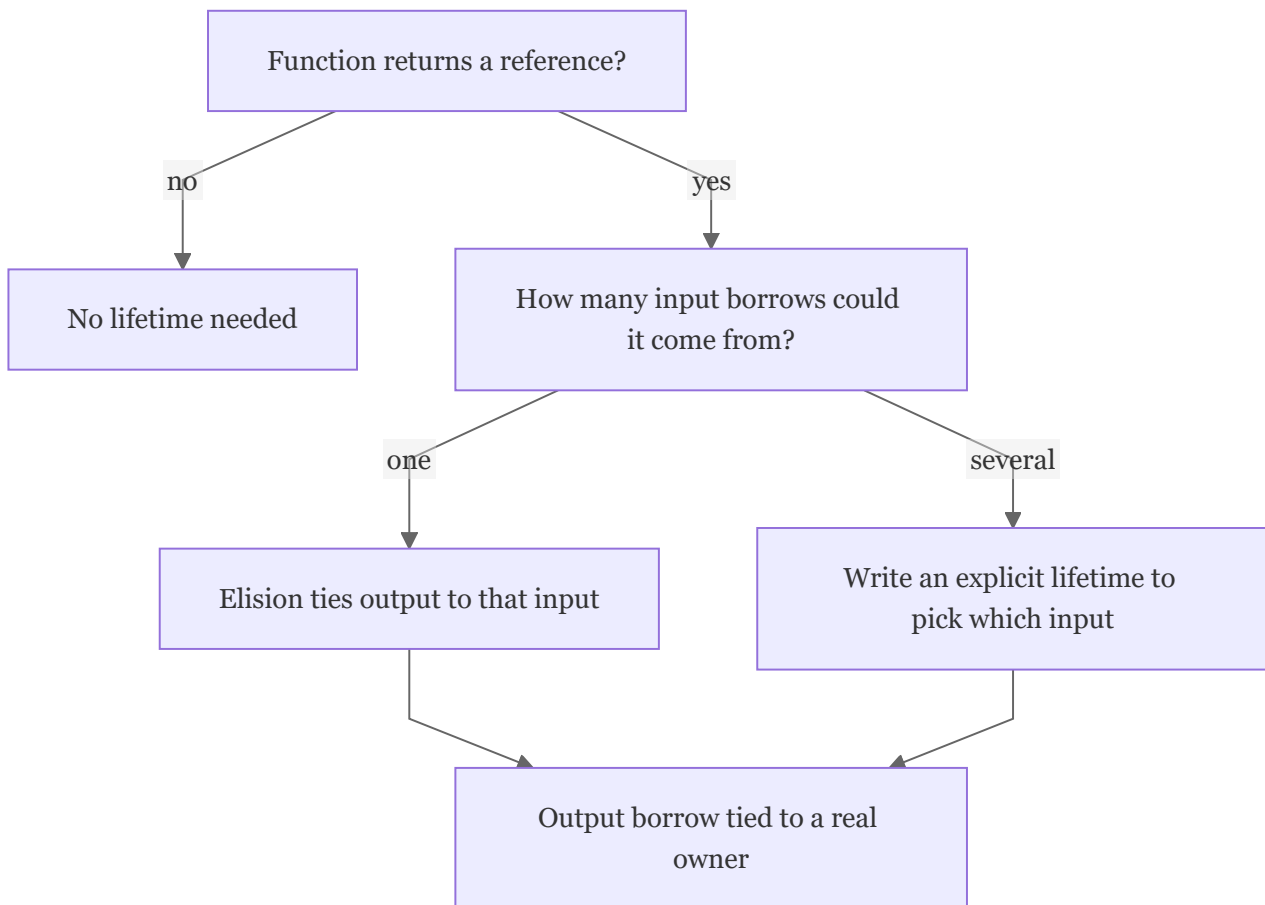
Any number of shared readers, or exactly one exclusive writer, never both. Every borrow error is the compiler keeping you out of the missing fourth state: a writer plus a reader at once.

Use `&T` when the callee only needs to inspect data. Many readers are allowed at once because no one is mutating through those aliases.

Use `&mut T` when the callee needs to change the value. Rust requires exclusivity here because mutation plus aliasing is one of the places memory and concurrency bugs are born.

Lifetimes in function signatures

A lifetime parameter answers one question for a function that returns a reference: which input does the output borrow from? When there is only one candidate input, the answer is obvious and Rust writes it for you; when there are several, you have to point at the right one. The diagram traces that decision.



The lifetime is not extending anything; it only records which input owner the returned reference is tied to. One input means elision can do it; several means you choose.

A lifetime annotation matters when the compiler must understand how output references relate to input references. A classic example is choosing one of two input references and returning it. The function is not extending lifetime; it is documenting which borrowing relationship must hold.

When a function borrows from one input and returns a slice of that same input, the relationship is often simple enough that Rust can infer it. This is where lifetime elision helps keep common signatures small without changing the underlying model.

Lifetime elision

Elision is convenience, not a second ruleset. Rust still reasons about lifetimes; it simply fills in common cases for you. The useful operational reading is below.

The practical takeaway is simple: if there is one input borrow, elision often works. If there are several candidate input borrows and one output borrow, the relationship usually needs to be written explicitly.

Common lifetime traps for C++ and Go developers

Designing APIs around ownership

A strong default for service code is this: accept borrowed input for local inspection, parsing, and validation; return owned output when the result should be stored, queued, retried, or otherwise decoupled from the caller's input buffer.

Coming from another language

The mechanics of borrowing are not the hard part. The hard part is that ownership and aliasing rules you used to enforce by habit, by convention, or by trusting a runtime are now checked by the compiler. Each of these cards names the single mental-model shift that trips people up most when they bring instincts from a specific language.

Production patterns

Pitfalls and tradeoffs

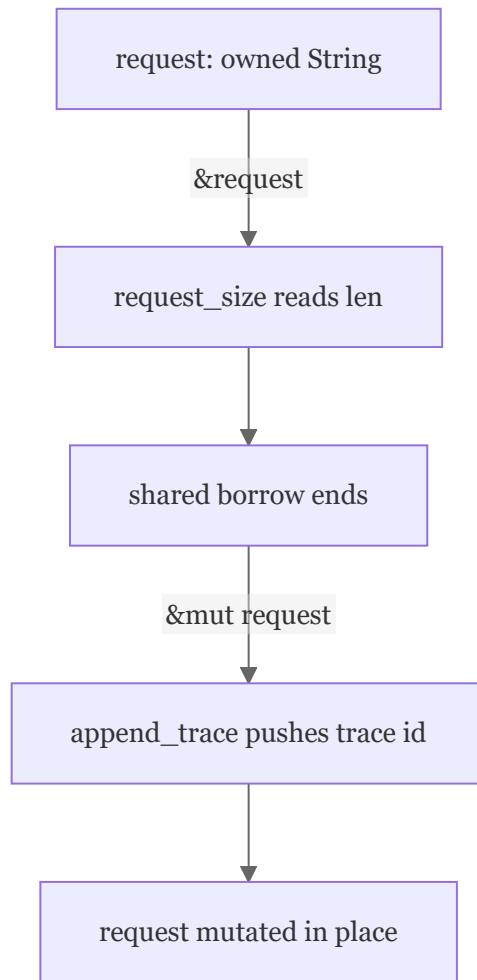
The wrong repair for a lifetime problem is often “add more lifetimes.” The right repair is usually one of three moves: shorten the borrow, change the owner, or return owned data.

Examples

Example 1: shared borrow first, mutable borrow second

Read through a shared reference, then take an exclusive mutable borrow only for the short mutation step.

What to look at: the shared borrow for `request_size` is created, used, and finished before `append_trace` ever asks for its mutable borrow. The two borrows do not overlap in time, which is exactly why the code compiles.



Shared borrow first, then it ends, then the exclusive mutable borrow. Sequencing the two so they never coexist is the design move, not cloning.

Example 4-1. `shared_and_mutable_borrows.rs` — *Shared mutable*

```

fn request_size(line: &str) -> usize {
    line.len()
}

fn append_trace(line: &mut String, trace_id: &str) {
    line.push_str(" trace=");
    line.push_str(trace_id);
}

fn main() {
    let mut request = String::from("GET /ready");
    let len_before = request_size(&request);
    append_trace(&mut request, "abc-123");

    println!("len before = {}", len_before);
    println!("request = {}", request);
}

```

OUTPUT

```
len before = 10
```

```
request = GET /ready trace=abc-123
```

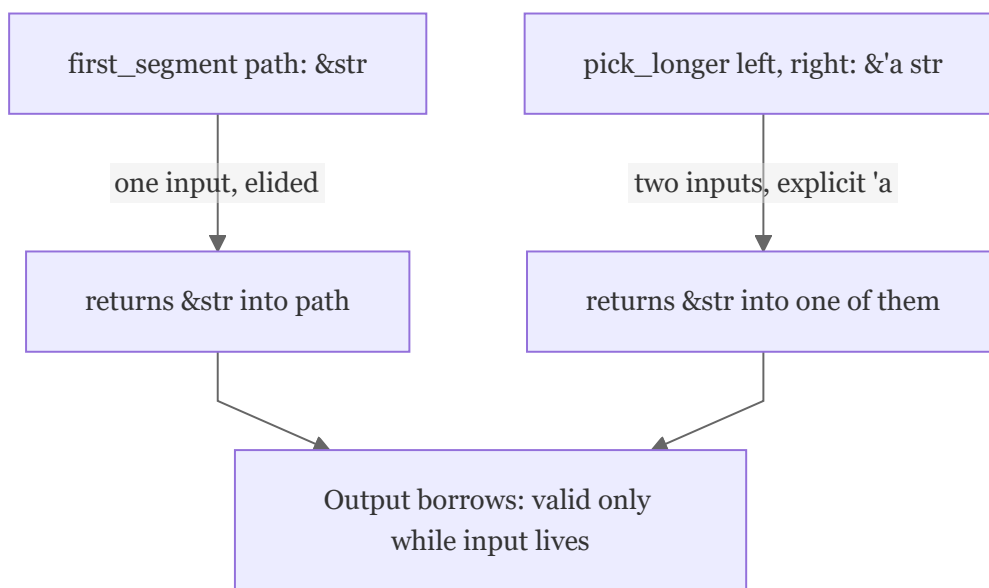
The immutable borrow used for `request_size` ends before the mutable borrow begins. That sequencing is the real design tool.

Inspect first, mutate second, and keep the exclusive borrow narrow. This pattern resolves a surprising number of logging, parsing, and request-enrichment borrow errors.

Example 2: explicit lifetime where needed, owned output where cleaner

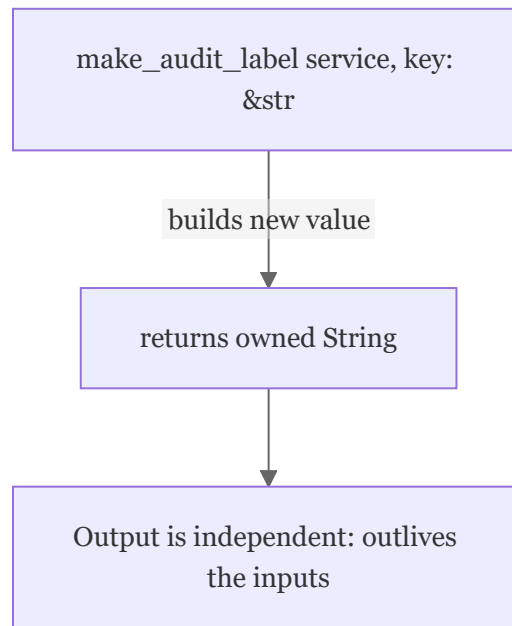
One helper uses lifetime elision, one requires an explicit relationship, and one deliberately returns an owned string.

What to look at: compare where each function's result gets its validity. Two of them return a slice that borrows from an input, so their output lives only as long as that input; the third builds a brand-new String it owns outright. The diagram groups the three by that distinction.



The two functions that return slices: `first_segment` uses elision because it has one input; `pick_longer` needs an explicit 'a to pick which of two inputs the result borrows from. Either way the output is valid only while that input lives.

The third function takes the opposite path: instead of borrowing from an input, it builds and returns a value it owns.



make_audit_label returns an owned String that survives on its own. Returning owned data is the deliberate escape from a borrow relationship.

Example 4-2. lifetimes_and_owned_boundaries.rs — *Borrowing lifetimes*

```

fn first_segment(path: &str) -> &str {
    path.split('/').find(|segment| !segment.is_empty()).unwrap_or("root")
}

fn pick_longer<'a>(left: &'a str, right: &'a str) -> &'a str {
    if left.len() >= right.len() { left } else { right }
}

fn make_audit_label(service: &str, key: &str) -> String {
    format!("{}", service, key)
}

fn main() {
    let route = String::from("/api/health");
    let segment = first_segment(&route);
    let longer = pick_longer("job", "request-id");
    let label = make_audit_label("gateway", segment);

    println!("segment = {}", segment);
    println!("longer = {}", longer);
    println!("label = {}", label);
}
  
```

OUTPUT

```

segment = api
longer = request-id
label = gateway::api
  
```

first_segment borrows from one input, so elision keeps the signature compact.

pick_longer chooses between two inputs, so the returned reference relationship must be written.

`make_audit_label` returns `String` because the label should outlive the borrowed inputs cleanly.

Summary

- Ownership is the main abstraction. Borrowing and lifetimes refine it; they do not replace it.
- Shared references permit many readers. Mutable references require exclusive access for the duration of mutation.
- Lifetimes describe valid reference relationships. They do not extend object lifetime or keep a dropped owner's data alive.
- Elision handles common cases, but multiple input borrows often require an explicit output relationship.
- Good production APIs often accept borrowed input and return owned data at subsystem boundaries.

CHAPTER 5 · POINTERS, REFERENCES, AND OWNERSHIP INSIDE STRUCTS

Pointers, References, and Ownership Inside Structs

Owned fields, borrowed fields, Box/Rc/Arc, owned buffers with views, and safe alternatives to self-references

Struct design fixes ownership policy in public data models. This chapter covers owned fields, borrowed views, shared pointers, and stable representations for systems that must expose safe state without hidden aliasing.

Builds on Chapters 02 and 04

Chapter 02 established values, moves, drops, and heap-backed ownership. Chapter 04 explained borrowing and lifetime relationships. This chapter moves those rules into struct design, where production Rust either becomes clear or becomes accidentally lifetime-heavy.

Opening scenario

A command-processing service converts request text into normalized records, stores selected data, and sends background work to processors. The business requirement is to classify every struct as a transient view or owned state, so lifetimes stay local and durable data remains easy to store, test, and move.

A durable design order

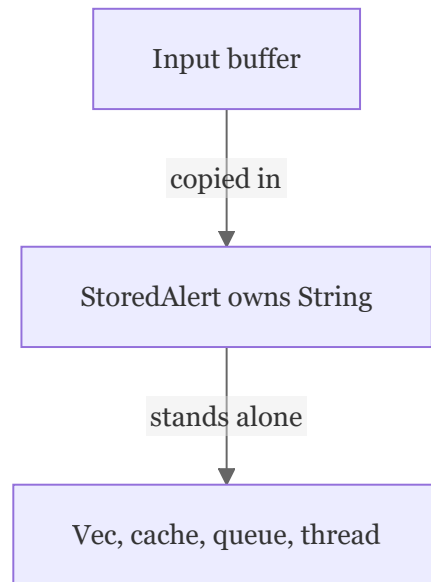
1. Decide whether the struct is a view or an owner.
2. If it borrows, say which outer owner keeps it alive.
3. If it owns, decide whether the owned fields are inline, boxed, or shared.
4. If you are tempted by self-reference, change the representation instead.

Mental model

Core concepts

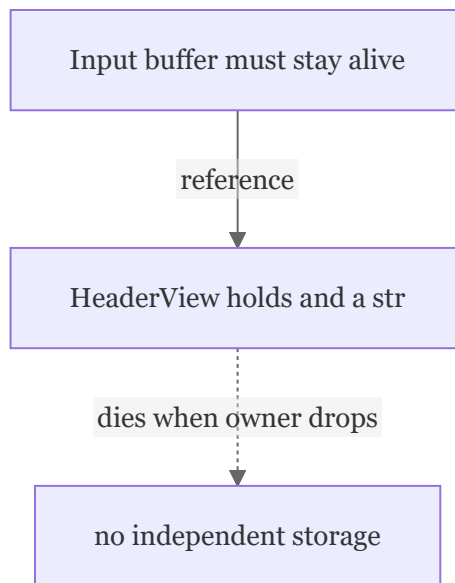
Owned fields vs borrowed fields

The choice is not about which is faster to type. It is about which direction the dependency points. An owned field copies the data inside the struct, so the struct stands alone. A borrowed field stores a reference back to data someone else holds, so the struct can only live as long as that owner does. Trace both arrows before you read the signatures.



Owned field: the data is copied inside the struct, which then carries it forward into long-lived storage on its own.

The borrowed case points the dependency the other way:

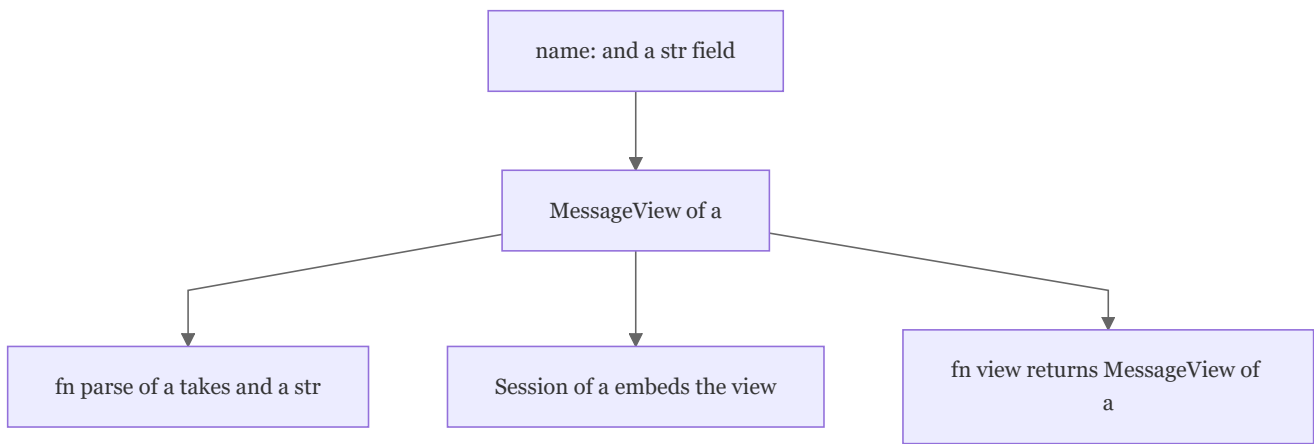


Borrowed field: the struct only points back at an owner that must outlive it, so it has no independent storage.

A borrowed-field struct is usually a temporary lens over someone else's data. An owned-field struct is usually a value the rest of the system may keep, move, test, queue, and reason about independently.

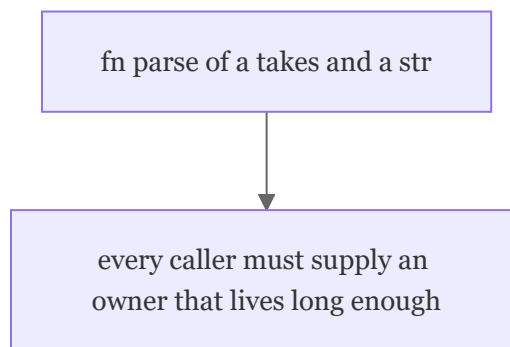
Structs containing references and lifetime parameters on structs

The thing to watch is contagion. The moment one field is a reference, the struct gains a lifetime parameter, and that parameter has to be named everywhere the struct appears: in the functions that build it, in the fields of any struct that holds it, in the return types that hand it back. The diagram shows the same lifetime 'a' threading through each of those positions.



One reference field forces a lifetime parameter, and that single 'a has to be named in the builder, in any holder, and in the return type.

That parameter does not stop at the type definition. It reaches all the way out to call sites:

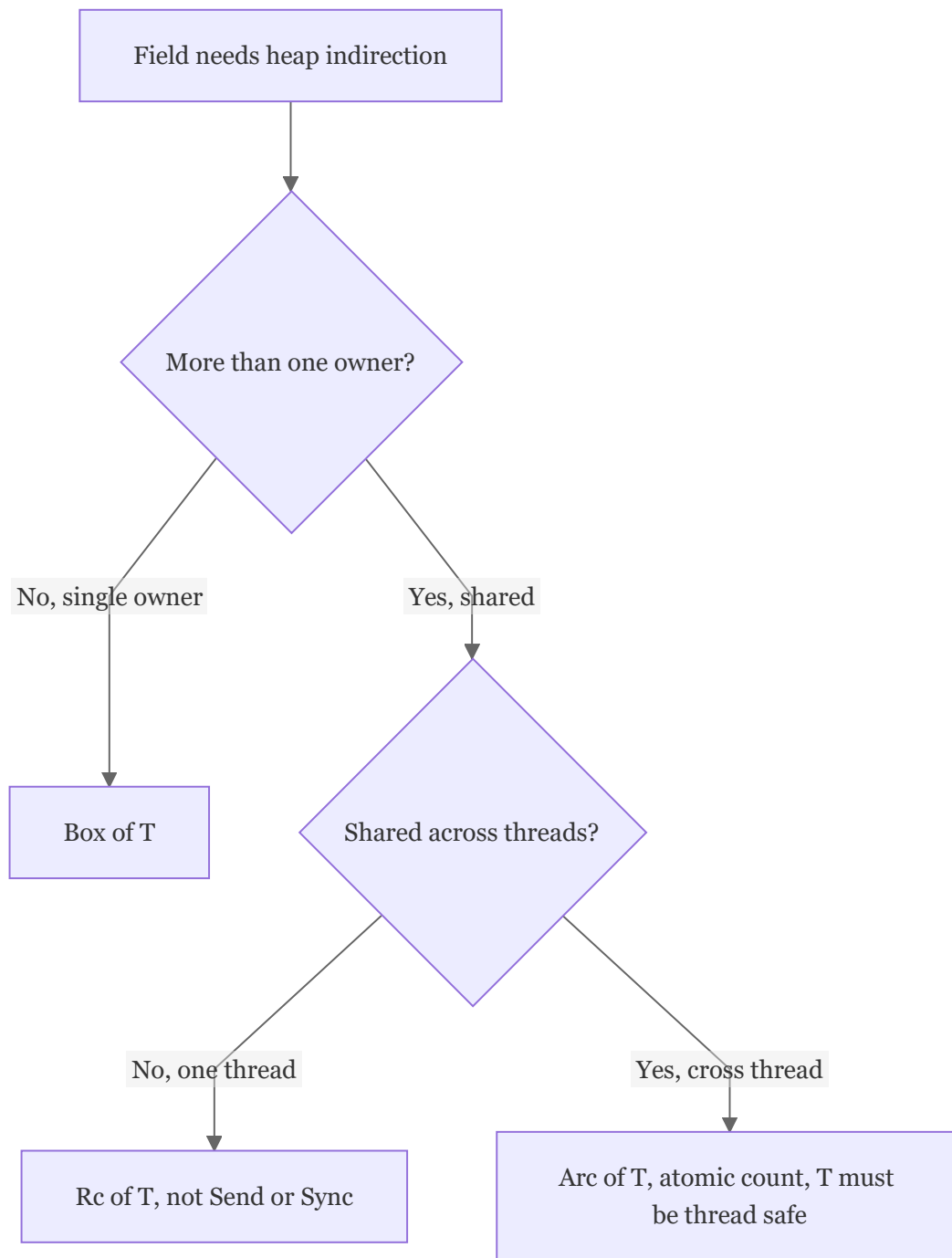


The builder propagates the same 'a outward, so ultimately every caller must supply an owner that lives long enough.

If this struct ended up inside a `Vec`, a cache, a worker message, or a test fixture, would you still want it tied to some external owner's lifetime? If the answer is no, give the struct owned fields.

Interior ownership using `Box`, `Rc`, and `Arc`

These three smart pointers all put a field behind heap indirection, but they answer different questions. The deciding factor is how many owners the value has and whether any of them live on another thread. Walk the two questions in the diagram and each pointer falls out of the answer.



Two questions decide the pointer: how many owners, and whether ownership crosses a thread. Box for one owner, Rc for shared on one thread, Arc for shared across threads.

The pointer type is part of the domain model. `Box<T>` means one owner plus indirection. `Rc<T>` means shared ownership in one thread. `Arc<T>` means shared ownership across threads, provided `T` is itself safe there.

Structs that own buffers and expose views

This is one of the most Rust-native patterns in systems and service code. Let the struct own the buffer. Then derive borrowed slices from `&self` as needed. The struct manages lifetime. Callers get safe read-only views.

The repository example `examples/ch05_ownership_inside_structs/owned_buffer_views.rs` mirrors the first example below.

Self-referential struct problems

The tempting shape is something like `struct Parsed<'a> {" raw: String, first: &'a str "}`. You cannot construct it in safe code: filling `first` requires borrowing `self.raw`, which conflicts with moving `raw` into the same value, and the lifetime `'a` is supplied by the caller — no lifetime can name the struct's own field. The deeper soundness reason self-references generally need `Pin` is that for address-sensitive inline data, moving the struct moves the storage that the internal pointer aims at. For `String` or `Vec` the bytes live on the heap and survive moves, so it is the construction rule, not the move, that rules this layout out.

The `repository` example `examples/ch05_ownership_inside_structs/stable_indices_instead_of_self_reference.rs` shows the most practical repair: own the buffer, store ranges, and derive views when needed.

Translating from C++, C#, and Go

Production patterns

Pitfalls and tradeoffs

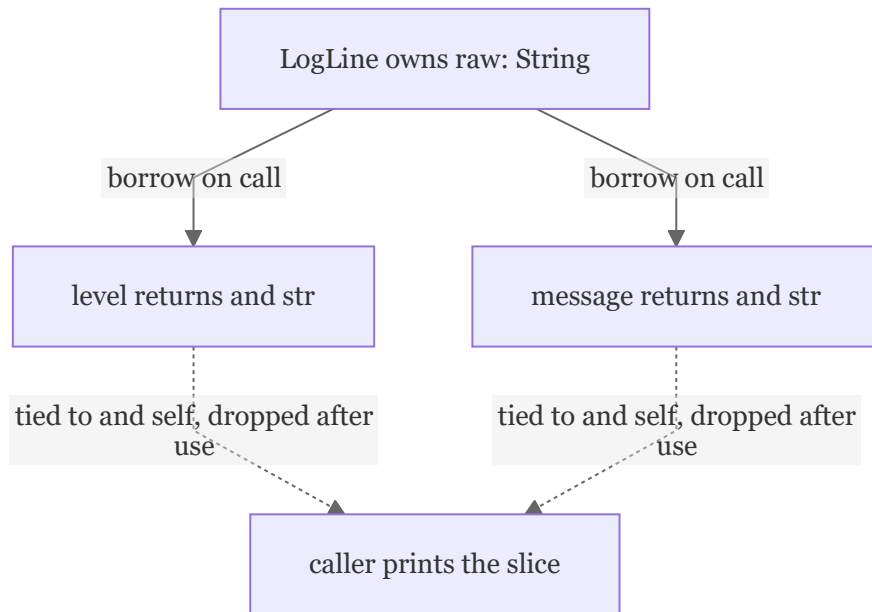
The cheapest-looking field type is not always the cheapest design. Saving one allocation by borrowing can cost far more if the whole object graph becomes lifetime-coupled and hard to store, test, or hand off.

Examples

Example 1: own the buffer, borrow the view

The struct owns the raw line. Methods derive lightweight views from that owned buffer.

What to look at: the struct holds exactly one owned field, the raw line. Every accessor returns a `&str` that is sliced out of that field on demand, so the borrows live and die with each call instead of being stored. Follow the single owner down to the short-lived views in the diagram, then read it in the code.



One owned buffer, many transient borrows. The returned slices are bound to &self, so the struct stays movable and storable.

Example 5-1. owned_buffer_views.rs — Owned views

```

struct LogLine {
    raw: String,
}

impl LogLine {
    fn new(raw: &str) -> Self {
        Self {
            raw: raw.to_string(),
        }
    }

    fn level(&self) -> &str {
        self.raw.split(':').next().unwrap_or("UNKNOWN")
    }

    fn message(&self) -> &str {
        self.raw
            .split_once(':')
            .map(|(_, message)| message.trim())
            .unwrap_or("")
    }
}

fn main() {
    let line = LogLine::new("INFO: user signed in");
    println!("level = {}", line.level());
    println!("message = {}", line.message());
}

```

OUTPUT

```
level = INFO
message = user signed in
```

Quick check: run the baseline, then change the raw line passed to `LogLine::new`. The output changes, but the ownership stays simple: one owner, many short borrows.

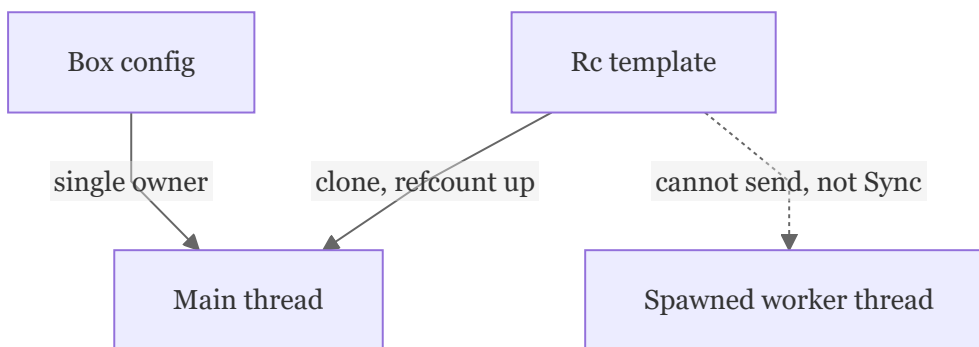
The returned `&str` values are tied to `&self`, not stored inside the struct. That keeps the representation movable and ordinary.

This pattern shows up in log parsing, protocol headers, request lines, and binary message frames: own once, slice many times, store offsets only when repeated lookup deserves it.

Example 2: choose Box, Rc, and Arc by ownership semantics

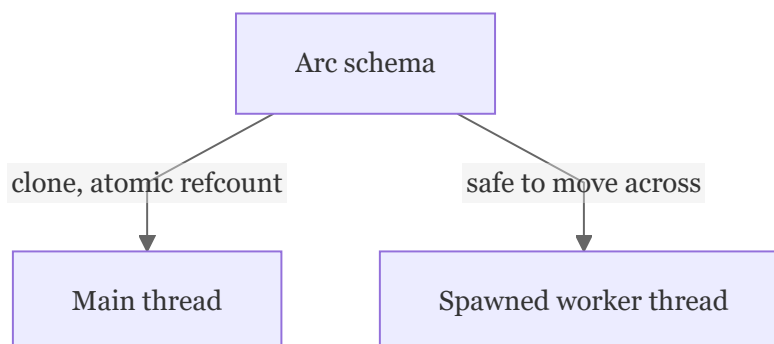
One field needs single-owner indirection, one needs single-thread shared ownership, and one must cross a thread boundary.

What to look at: read the three field types, not the strings they hold. Each field encodes a different ownership story, and the code downstream is forced to respect it. The boxed config is consumed by one owner, the `Rc` template is cloned and used only on the main thread, and the `Arc` schema is the one value that is allowed to move into the spawned worker. The diagram lines up each field with the boundary it may cross.



Box keeps one owner on the main thread, and Rc can be cloned there but is barred from crossing into the worker.

The `Arc`-wrapped schema is the one field allowed to cross the thread boundary:



Only the Arc-wrapped schema uses atomic counting, so it is the field that can safely move into the worker.

Example 5-2. `pointer_choices_inside_structs.rs` — *Pointer choices*

```

use std::rc::Rc;
use std::sync::Arc;
use std::thread;

struct ParserConfig {
    delimiter: Box<str>,
}

#[derive(Clone)]
struct UiTemplate {
    name: Rc<str>,
}

#[derive(Clone)]
struct SharedSchema {
    version: Arc<str>,
}

fn main() {
    let parser = ParserConfig {
        delimiter: "=>".into(),
    };
    let template = UiTemplate {
        name: Rc::from("invoice"),
    };
    let template_copy = template.clone();
    let schema = SharedSchema {
        version: Arc::from("v2"),
    };
    let schema_for_worker = schema.clone();

    let handle = thread::spawn(move || {
        format!("thread schema = {}", schema_for_worker.version)
    });

    println!("box delimiter = {}", parser.delimiter);
    println!("rc clones = {}", Rc::strong_count(&template_copy.name));
    println!("{}", handle.join().unwrap());
}

```

OUTPUT

```

box delimiter = =>
rc clones = 2
thread schema = v2

```

Quick check: keep the three field types distinct, then change the string literals and rerun. The point is not the output itself. The point is that the field type communicates the ownership model.

The parser config still has one owner. The box gives indirection without changing ownership count.

The UI template is shared on one thread only. That is why the example clones it locally and never sends it to the worker.

The shared schema crosses a thread boundary, so the reference count must be atomic and the inner type must remain thread-safe.

Summary

- Struct field types are ownership decisions, not mere storage details.
- Borrowed fields are useful for short-lived views, but they spread lifetime coupling through the type.
- `Box<T>`, `Rc<T>`, and `Arc<T>` solve different ownership problems. Pick by semantics, not habit.
- Owning a buffer and deriving views from `&self` is a strong Rust-native pattern for parsers and protocols.
- Most self-referential designs should be rewritten as offsets, ranges, stable IDs, or separate owner/view types.

CHAPTER 6 · POINTERS, REFERENCES, AND OWNERSHIP INSIDE VECTORS

Pointers, References, and Ownership Inside Vectors

Vec<T> ownership, reference invalidation, reallocation hazards, index handles, stable-address patterns, slices, and safe mutation while iterating

A vector owns one resizable buffer, and growth can move every element it holds. This chapter is about choosing the right handle into that buffer: when a borrowed reference is safe, when an index or ID is the durable choice, and when the model genuinely needs stable addresses from boxed or arena-style storage.

Builds on Chapters 04 and 05

Chapter 04 explained borrowing and lifetime relationships. Chapter 05 moved those rules into struct field design. This chapter applies the same discipline to collections, where growth, reordering, and temporary views make ownership mistakes especially easy to import from C++, C#, or Go.

Opening scenario

A worker registry tracks live connections, appends new workers during scaling, scans readiness, and removes unhealthy entries. The business requirement is a stable handle policy: use short borrows only while the vector shape is fixed, and use indices, IDs, boxed storage, or arena handles when growth and reordering are normal.

A practical review order

1. Will the vector's length, capacity, or ordering change while this handle is alive?
2. If yes, should the handle be an index or ID instead of a reference?
3. If address stability truly matters, should each element be separately allocated?
4. If the algorithm needs both inspection and reshaping, can it run in two phases?

Mental model

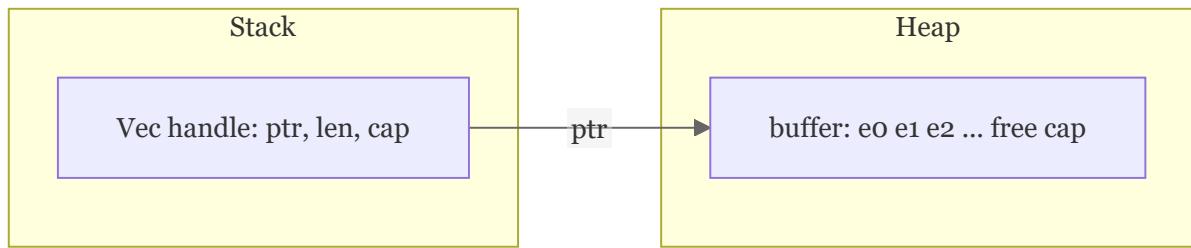
Core concepts

What a vector actually is

A vector is one owner of one resizable buffer. The value stored in a local binding is only the control block: pointer, length, and capacity. The elements live in heap-backed contiguous storage managed by that owner.

Moving the `Vec<T>` moves ownership of the buffer. Growing beyond capacity may move the elements themselves into a new buffer. That second fact is the one that makes element references delicate.

The thing to picture is the split between the small fixed-size handle on the stack and the heap buffer it points at. The handle stays put; the buffer underneath it can be replaced.



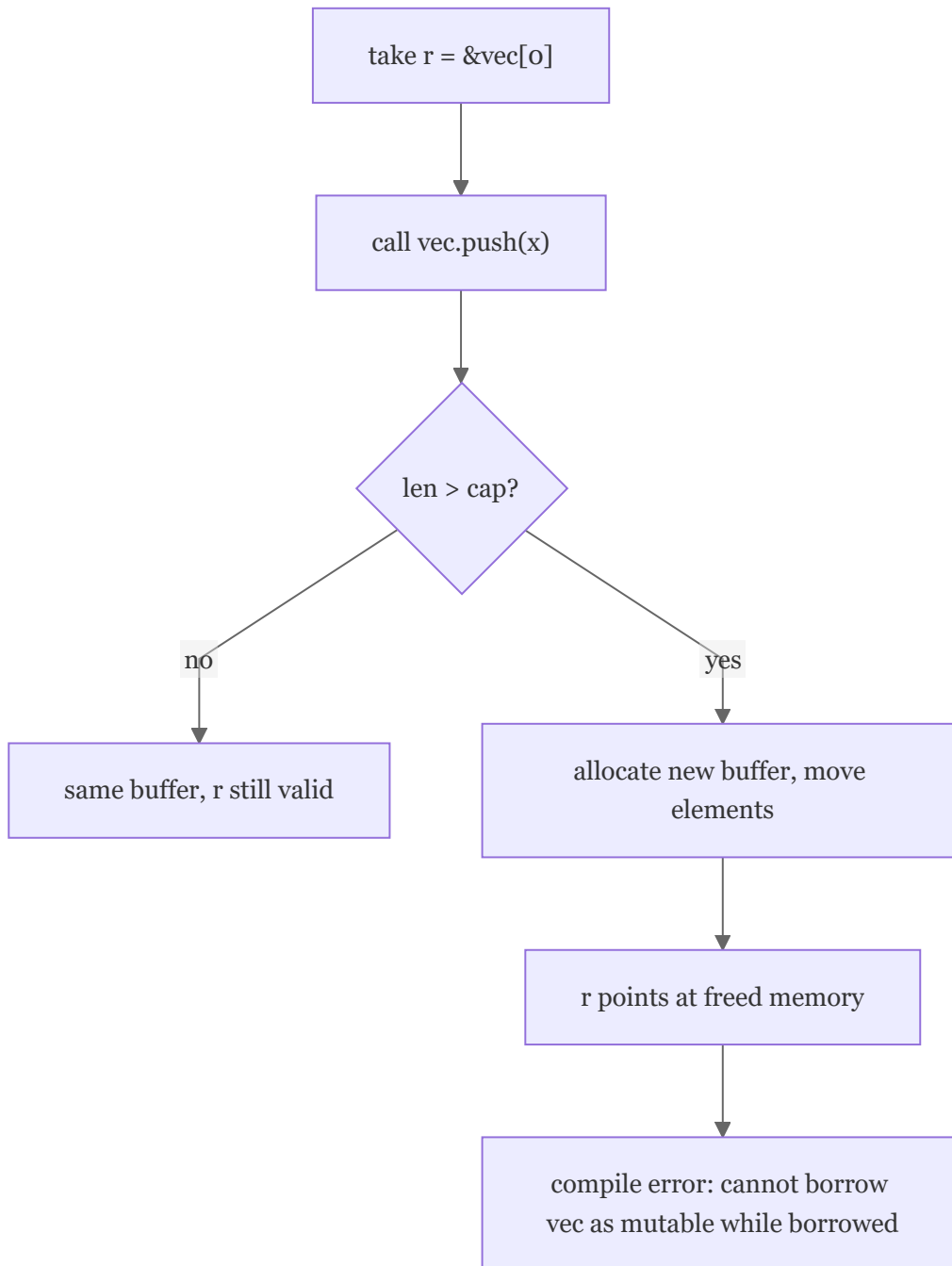
The Vec value on the stack is just a pointer, length, and capacity. The elements live in a separate heap buffer the vector owns.

References into vectors and invalidation

A borrowed element such as `&jobs[i]` is tied to the current buffer and the current aliasing rules. It says, in effect, "while this borrow lives, nobody may perform conflicting access that would make it invalid or ambiguous."

Rust therefore rejects many patterns that would be merely dangerous in other languages: take a borrow, then append, reserve, insert, or otherwise reshape the same vector while still using the borrowed element later.

The hazard is easiest to see as a sequence. A borrow names a slot in the current buffer; a growth past capacity allocates a new buffer and moves the elements; the old borrow would now point into freed memory. Rust rejects the code at the second step rather than letting the third one happen at runtime.



*Growth past capacity relocates the buffer, which would dangle the earlier borrow.
Rust forbids the conflicting mutable access at compile time.*

- push, extend, and insert may move the buffer when length grows past capacity.
- reserve is an explicit request for more capacity and may also relocate storage.
- remove, swap_remove, sorting, and compaction may shift element positions even when the buffer does not relocate.

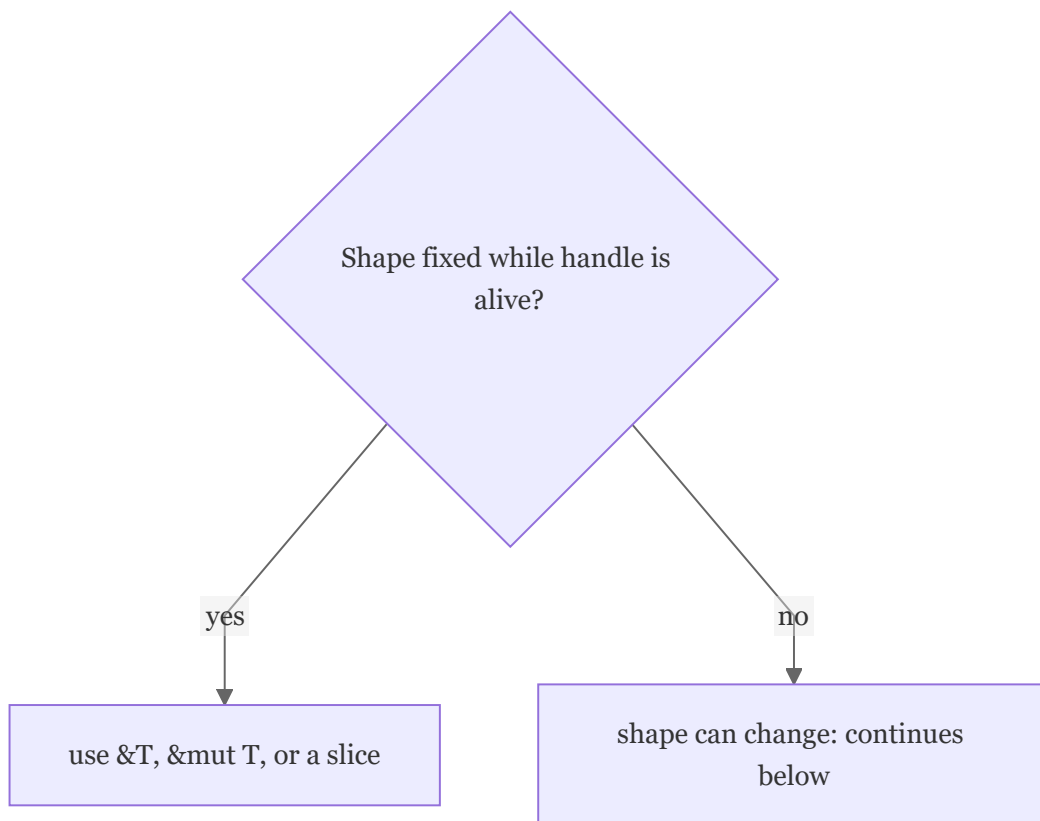
Index-based designs

Indices are the default durable handle for a plain vector. You store the position, mutate the vector as needed, and then reborrow from the vector when you are ready to use the element again.

This works well when you control reordering policy. If the code uses `swap_remove`, sorting, or compaction, the index is a location, not an identity. Production code often wraps indices in typed IDs or adds generation checks when stale handles matter.

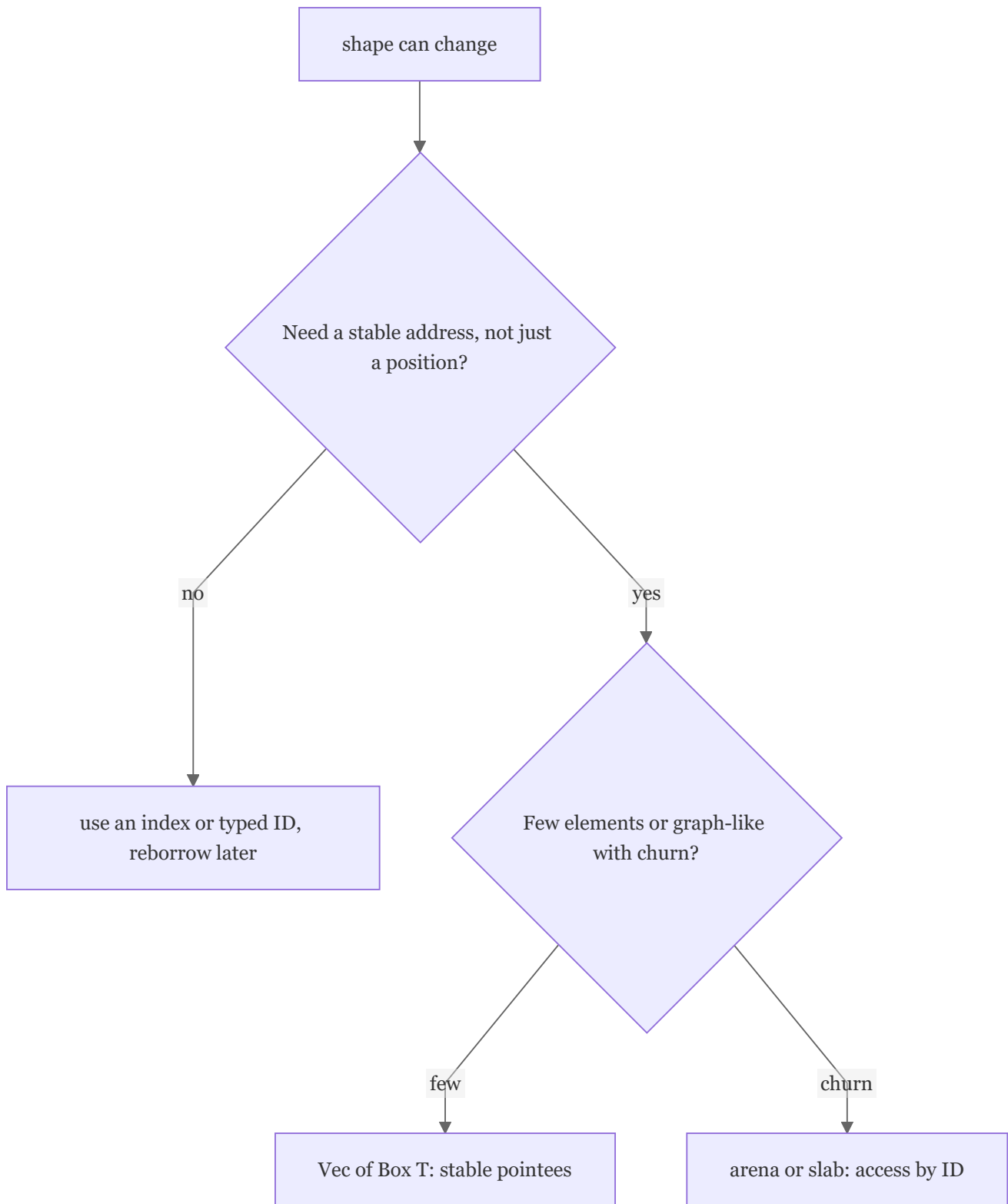
Choosing a handle

Before reaching for any representation, the decision worth making explicit is what kind of handle the code needs. The first question is whether the vector's shape is fixed for the lifetime of the handle. If it is, borrows and slices are ideal. If it is not, the choice is between a location handle (an index or ID) and a representation that gives genuinely stable addresses.



First question: if the vector cannot grow, shrink, or reorder while the handle lives, a short borrow or slice is ideal.

When the shape can change, the choice is between a location handle and genuinely stable addresses:



If shape changes, prefer an index for plain growth; reach for boxed or arena storage only when an address itself must stay put.

Stable-address allocation patterns

Sometimes the correct requirement really is stable addressability. A parser cache, intrusive structure, or FFI boundary may need each element to stay put even if the collection grows. That is when you change the representation, not when you try to stretch a plain borrowed element reference farther than it should go.

A precise note on Pin: pinning can matter for address-sensitive pointees such as certain async or self-referential low-level machinery, but it does not make references into a plain `Vec<T>` stable on its own. For ordinary collection design, indices, boxes, or arena-style handles are the usual tools.

`Vec<Box<T>>`, arenas, and slab allocation

`Vec<Box<T>>` is a tactical choice: indexing stays simple, and each element lives in its own heap allocation. Growing the vector moves boxes, not pointees.

Arena and slab-style stores solve a different problem. They usually give you stable logical handles, efficient reuse, and explicit deletion policy. The ecosystem offers several variants, but the shared idea is more important than any one crate: store by owner-managed pool, access by ID.

Borrowing slices from vectors

How this lands by background

Every language with a growable array has the same underlying reallocation behavior. What differs is whether you normally hold a reference into a slot, and whether anything stops you from using it after the array moves. The mental shift below is the trap most likely to surprise an engineer coming from each language.

Production patterns

Pitfalls and tradeoffs

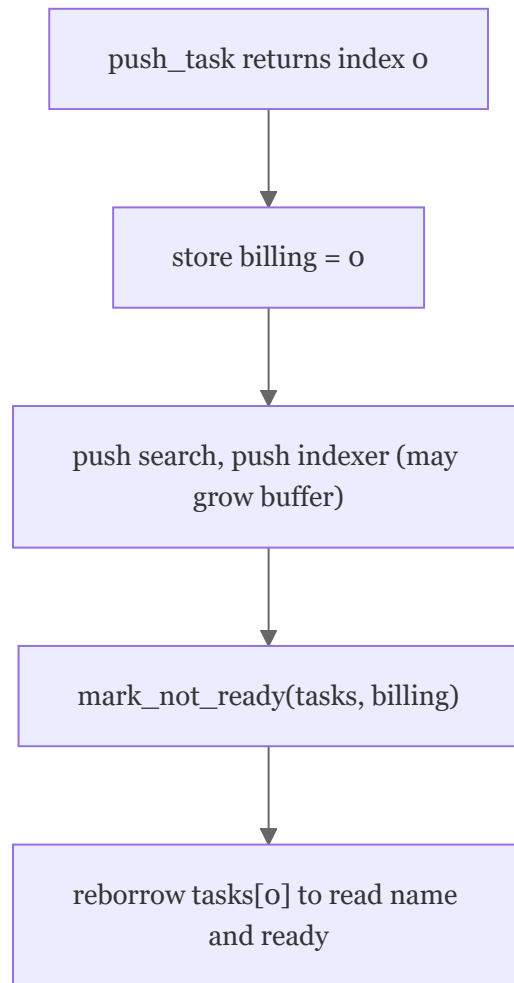
The cheapest-looking handle is often the most expensive long-term design. A borrowed element reference is wonderful when the vector shape is fixed. It is a trap when growth and reshaping are part of the normal lifecycle.

Examples

Example 1: index handles instead of borrowed elements

The vector may grow, so the durable handle is the index. The code reborrows from the vector only when it actually needs the element.

What to look at: `push_task` returns the index it just wrote, not a reference. Later pushes are free to reallocate the buffer, and the code only reborrows through `tasks[billing]` after all growth is done. The index is the handle that survives the mutation.



The index stays valid across growth. Each access reborrows the current buffer instead of holding a reference through the pushes.

Example 6-1. `index_handles_instead_of_refs.rs` — Vectors indices

```

#[derive(Debug)]
struct Task {
    name: String,
    ready: bool,
}

fn push_task(tasks: &mut Vec<Task>, name: &str) -> usize {
    let index = tasks.len();
    tasks.push(Task {
        name: name.to_string(),
        ready: true,
    });
    index
}

fn mark_not_ready(tasks: &mut [Task], index: usize) {
    tasks[index].ready = false;
}

fn main() {
    let mut tasks = Vec::with_capacity(1);
  
```

```
let billing = push_task(&mut tasks, "billing");
push_task(&mut tasks, "search");
push_task(&mut tasks, "indexer");
mark_not_ready(&mut tasks, billing);

println!("task = {}", tasks[billing].name);
println!("ready = {}", tasks[billing].ready);
println!("total = {}", tasks.len());
}
```

OUTPUT

```
task = billing
ready = false
total = 3
```

Quick check: keep the handle as an index, then change task names or add more pushes. The design stays correct because the vector is allowed to grow without carrying old element borrows through the mutation.

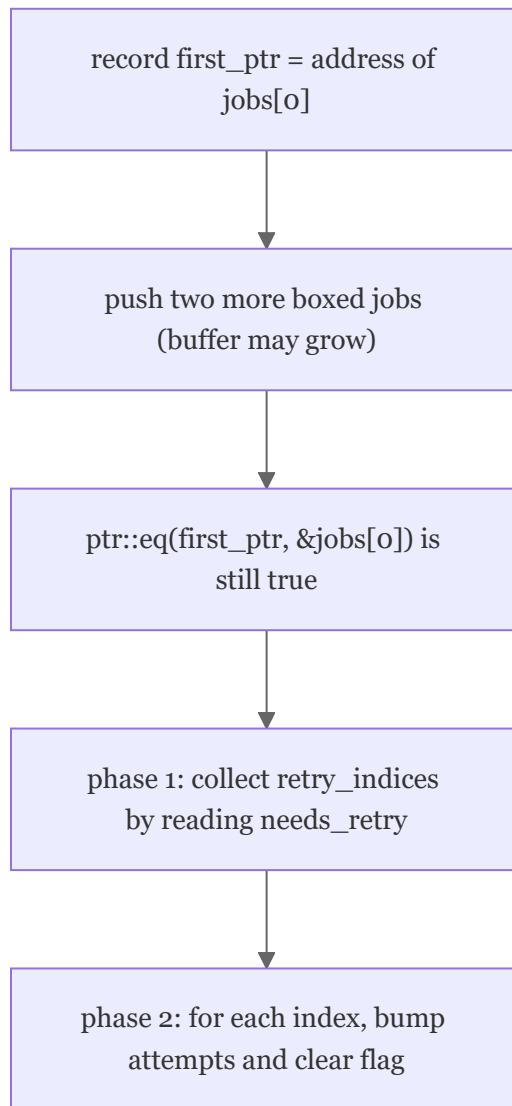
If the collection may reallocate, keep a location handle such as an index and borrow later.

Worker tables, scheduler queues, and parser token streams often become easier once durable handles stop being raw references into resizable storage.

Example 2: stable pointees plus two-phase mutation

Each job lives in its own box, and shape-changing logic is split from inspection logic.

What to look at: `first_ptr` records the heap address of the first job before two more pushes. Because each job is its own `Box`, growing the vector relocates the boxes but not the jobs, so the address still matches afterward. The retry update then runs in two phases: collect indices first, mutate second, so no borrow is held across a shape change.



Boxes move when the vector grows; the boxed jobs keep their addresses. Inspection (phase 1) is fully separated from mutation (phase 2).

Example 6-2. `vec_box_stable_addresses.rs` — *Boxed stable*

```

use std::ptr;

#[derive(Debug)]
struct Job {
    name: String,
    attempts: usize,
    needs_retry: bool,
}

fn main() {
    let mut jobs: Vec<Box<Job>> = Vec::new();

    jobs.push(Box::new(Job {
        name: String::from("billing"),
        attempts: 1,
        needs_retry: true,
    }));
  
```

```

let first_ptr: *const Job = &*jobs[0];

jobs.push(Box::new(Job {
    name: String::from("search"),
    attempts: 1,
    needs_retry: false,
}));
jobs.push(Box::new(Job {
    name: String::from("indexer"),
    attempts: 1,
    needs_retry: false,
}));

let same_address = ptr::eq(first_ptr, &*jobs[0]);

let retry_indices: Vec<usize> = jobs
    .iter()
    .enumerate()
    .filter_map(|(index, job)| job.needs_retry.then_some(index))
    .collect();

for index in retry_indices {
    jobs[index].attempts += 1;
    jobs[index].needs_retry = false;
}

println!("same address = {}", same_address);
println!("attempts = {}", jobs[0].attempts);
println!("retry pending = {}", jobs.iter().filter(|job| job.needs_retry).count());
}

```

OUTPUT

```

same address = true
attempts = 2
retry pending = 0

```

Quick check: keep the boxed representation, then add more jobs or change the retry flag. The point is not boxing by habit. The point is that stable pointees and two-phase updates are explicit, reviewable choices.

The vector may move boxes around, but the boxed job itself stays at one heap address.

The first pass records retry targets. The second pass performs mutation. Inspection and reshaping stay separate.

Extra allocation buys stronger address stability. Use it when the model needs it, not by reflex.

Summary

- A vector owns a resizable contiguous buffer, and growth may move every element.
- References and slices are short-lived views into the current buffer, not durable handles across shape changes.
- Indices are the default stable handle for plain vectors, provided the code manages reordering policy explicitly.

- Stable-address needs usually call for a different representation such as `Vec<Box<T>>`, arenas, or slab-style storage.
- When mutation changes vector shape, inspection-first and mutation-second is often the cleanest production pattern.

CHAPTER 7 · COPYING DATA VS CLONING DATA

Copying Data vs Cloning Data

Move semantics, Copy vs Clone, clone-on-write, Rc/Arc cloning costs, allocation discipline, and receiver choices

Whether a value moves, copies, or clones decides latency, memory use, and what an API promises about ownership. This chapter separates those events so that a reader, and a code reviewer, can always tell which one is happening and why the cost was chosen on purpose.

Builds on Chapters 04, 05, and 06

Chapter 04 explained ownership and borrowing. Chapter 05 moved those rules into struct design. Chapter 06 showed why resizable collections make handle choice matter. This chapter adds the duplication model on top: when values move, when they copy, when they clone, and how to keep that decision visible in production APIs.

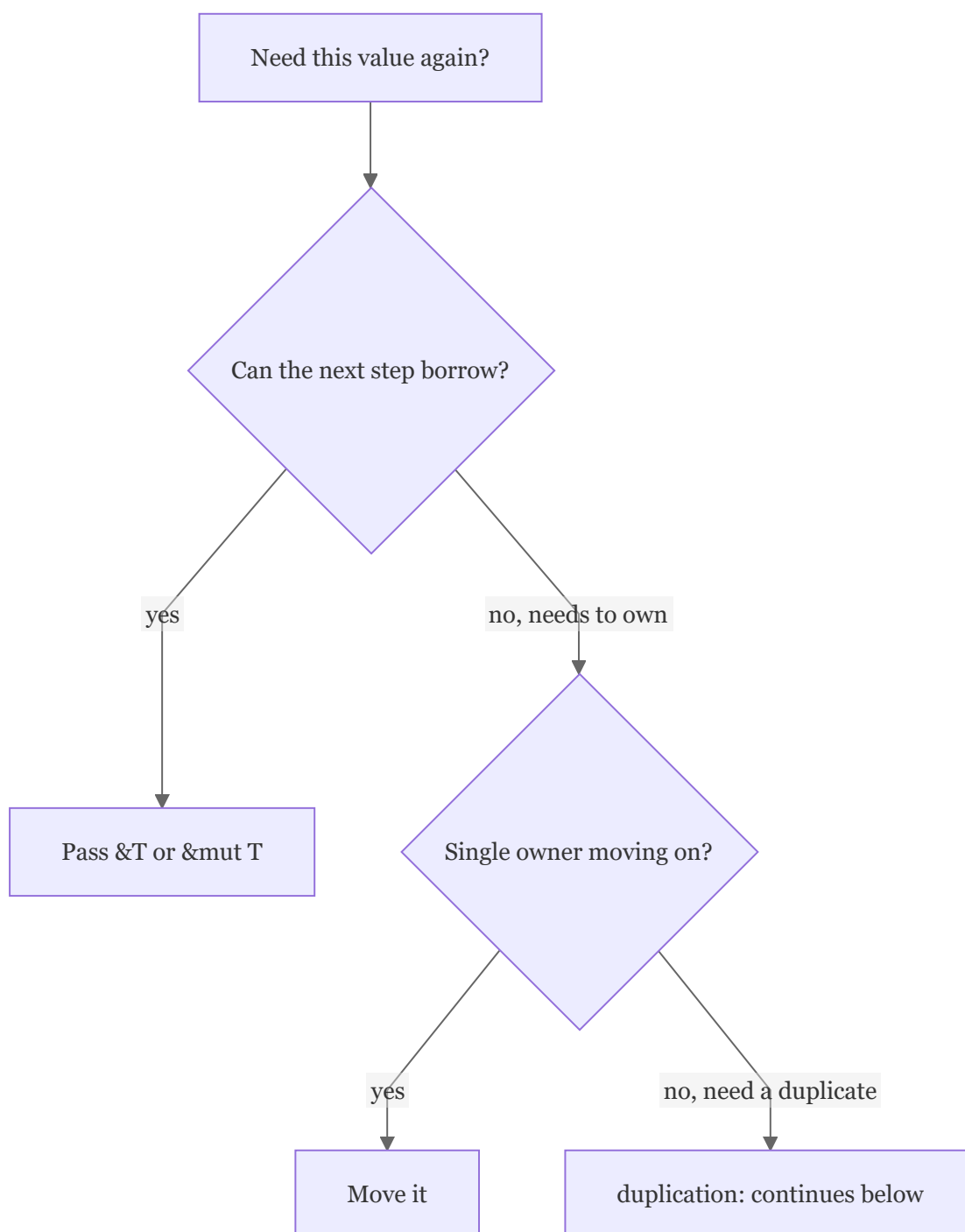
Opening scenario

A request service fans work out to several workers and keeps retry state for failed jobs. The requirement is a reviewable duplication policy: borrow read-only data on hot paths, move owned jobs at boundaries, copy tiny values implicitly when that is safe, and clone only where independent ownership is genuinely required. The goal is not to ban clones; it is to make every clone explainable in review.

A fast decision order

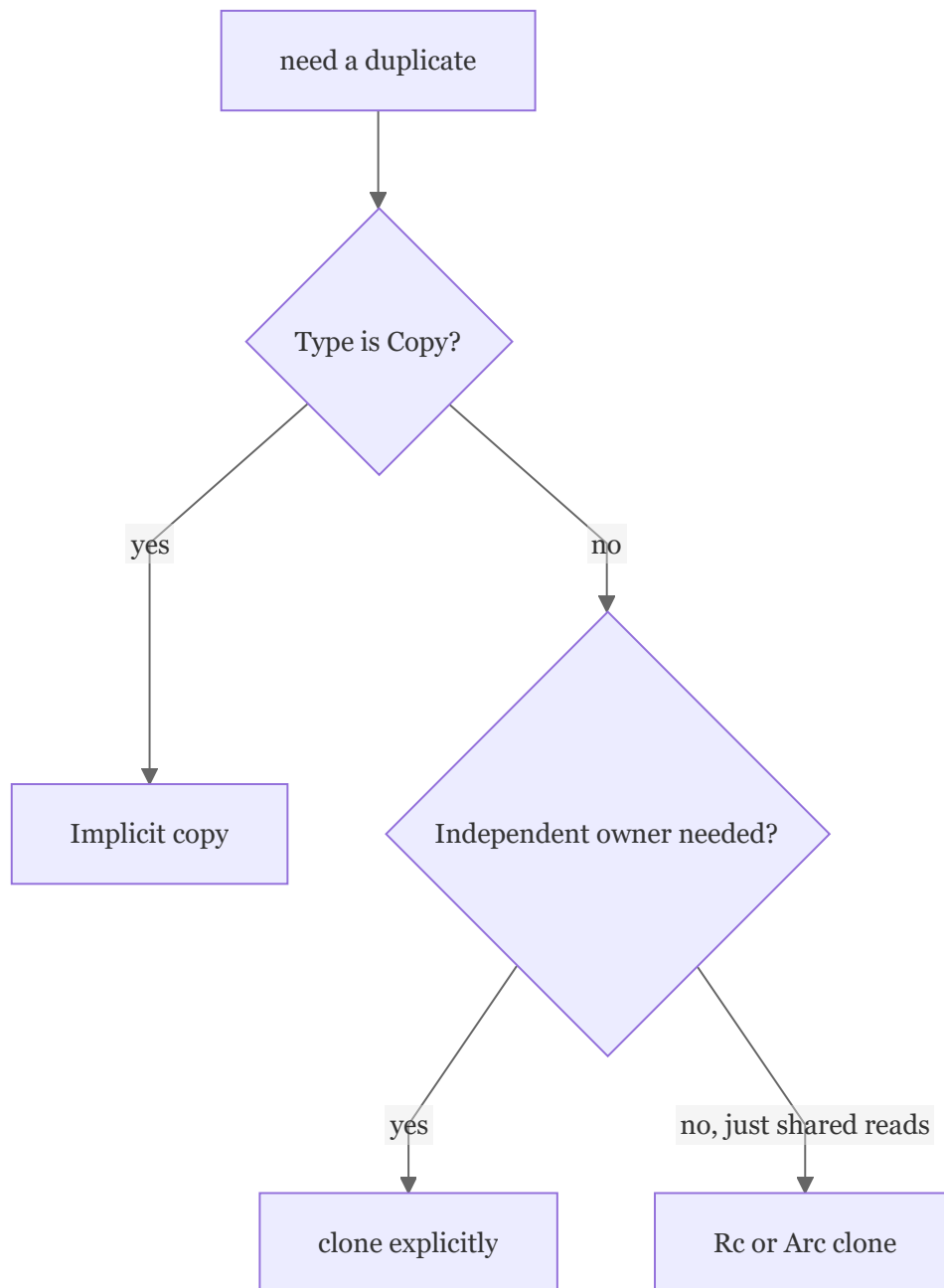
When you hit a value that needs to be used more than once, walk these questions in order. The first one that answers yes is usually the right tool, and the diagram after it shows the same decision as a single path.

1. Can the next step borrow instead of owning?
2. If ownership must transfer, is a move the correct model?
3. If duplication is needed, is the type a plain Copy value or an explicit Clone value?
4. If many readers share one value, is Rc or Arc the real model?



First half: borrow if the next step can, otherwise move when a single owner is handing the value on. If you actually need a duplicate, follow the second half.

When a duplicate is genuinely needed, the second half decides which kind:



Second half: a Copy type duplicates implicitly; otherwise clone explicitly for an independent owner, or add a shared owner with Rc or Arc when many readers share one value.

Mental model

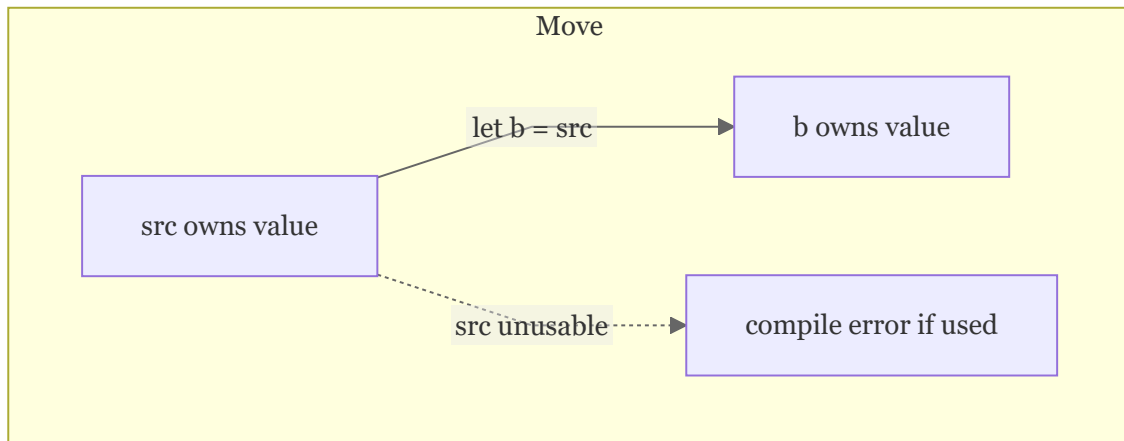
How this lands by background

Most engineers do not meet move, Copy, and Clone with a blank slate. They arrive with a duplication model from another language, and the useful question is which part of that model transfers and which part quietly misleads. The shift is almost never about API names. It is about where duplication becomes visible: Rust drags the deep copy out of the constructor and the assignment and onto a line you can read.

Core concepts

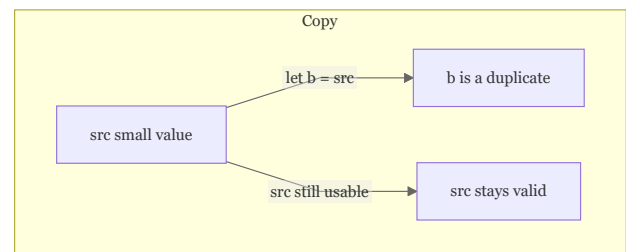
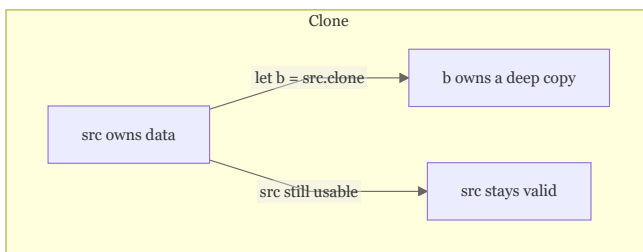
Three different events, not one blurry idea

The whole chapter rests on keeping three things apart. A move hands a single value from one owner to the next. A copy is an implicit bitwise duplicate that a type opts into for small values. A clone is an explicit, possibly expensive duplication that you write by hand. The diagram shows what happens to the source binding in each case, which is the detail that trips people up.



Move: assignment hands the single value to the new binding and the source can no longer be used.

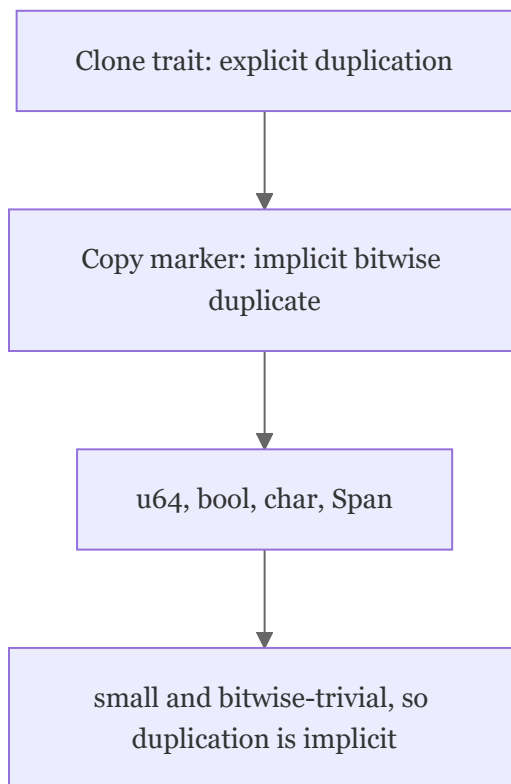
Copy and Clone behave differently: in both cases the source stays usable. The split is who pays and whether the duplication is implicit or written down.



Copy and Clone both leave the source valid. Copy is an implicit bitwise duplicate of a small value; Clone is an explicit, possibly expensive duplication you write by hand.

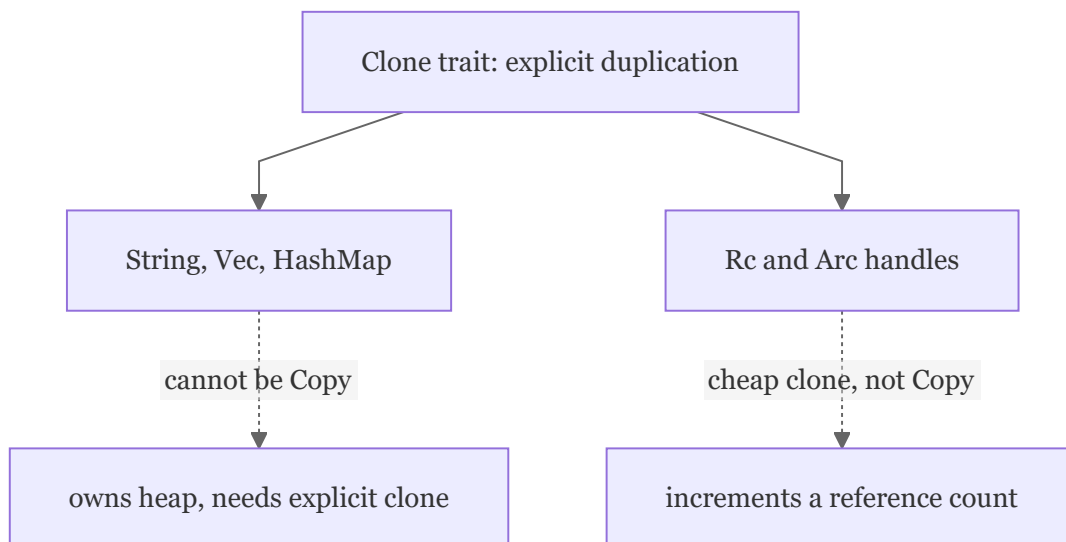
What Copy and Clone share, and where they differ

These two traits are related, not independent. Clone is the general duplication trait. Copy is a marker that sits on top of Clone and adds a promise: the duplicate is a trivial bitwise copy, so the language may make it implicit and leave the original usable. That relationship is why every Copy type must also be Clone, and why a type that owns heap data can be Clone but never Copy.



Copy is a special case of Clone: a marker for small, bitwise-trivial values where implicit duplication is safe and unsurprising.

The other families stay Clone-only on purpose. Heap-owning and reference-counted types can duplicate, but never implicitly:

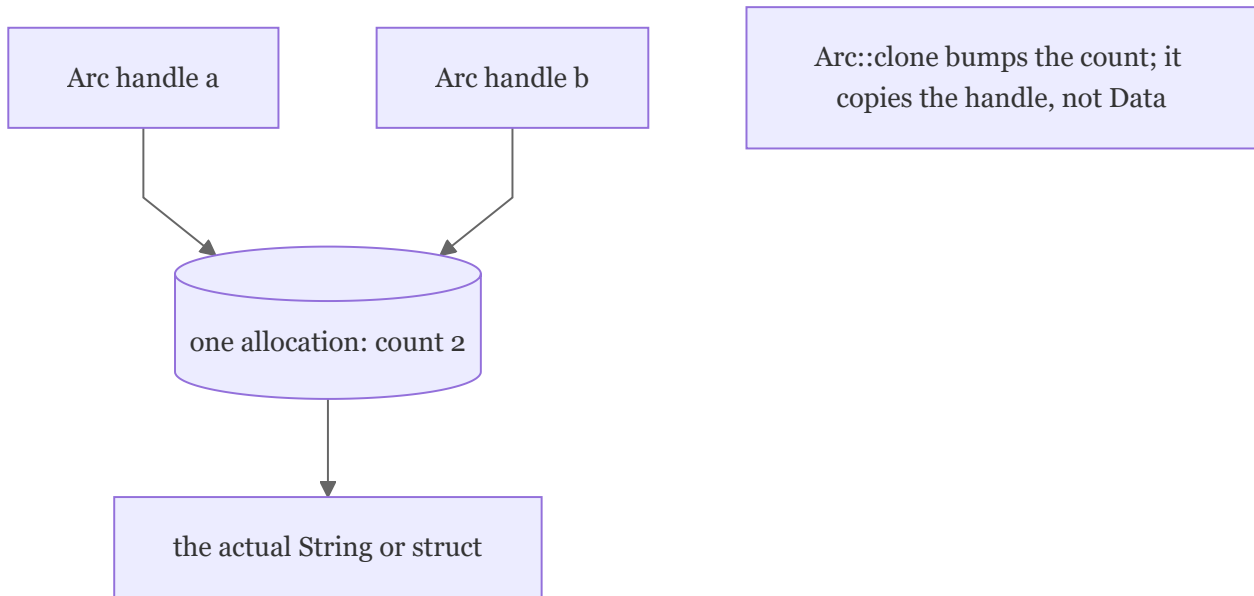


Heap-owning types must clone explicitly; reference-counted handles clone cheaply but only bump a count, so neither is ever Copy.

Topics this chapter treats as source of truth

Reference-counted cloning is sharing, not copying

The most common confusion in this whole topic is treating an Arc clone as if it deep-copied the inner value. It does not. Both handles point at the same allocation; the clone only bumps a counter so the allocation lives until the last handle is dropped. That is what makes it cheap, and also what makes it different from an independent clone: a mutation through one handle is visible to the other unless you go through `make_mut`.



An Arc clone adds a second handle to one shared allocation and increments the reference count. The underlying data is never duplicated.

When is implementing Copy safe?

A useful default is conservative: if the type owns heap memory, file descriptors, sockets, locks, or any state where two implicit duplicates would be misleading, it should not be Copy. The compiler enforces the mechanical half of this (no Drop, all fields Copy); the semantic half (would two of these be surprising?) is yours to judge.

Production patterns

Pitfalls and tradeoffs

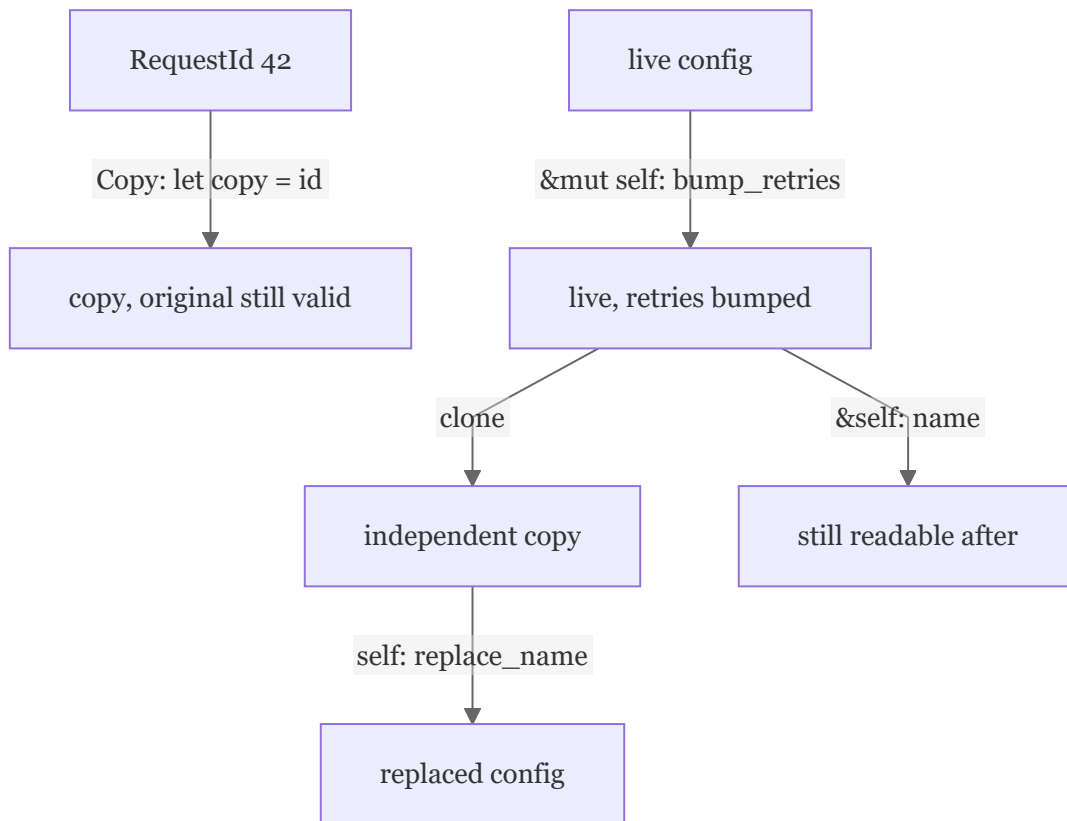
“Just clone it” is not always wrong. It becomes wrong when the reviewer can no longer explain why independent ownership is needed, or when the hidden allocation cost shows up in the hot path later.

Examples

Example 1: move, Copy, Clone, and receiver choices in one pass

One tiny ID copies, one config clones explicitly, one method reads by `&self`, one mutates by `&mut self`, and one consumes by `self`.

What to look at: follow the live config. It is mutated through `&mut self`, then explicitly cloned so the consuming `replace_name(self, ...)` can take the clone by value while the original keeps living. The diagram traces that single value; the code is the same path written out.



RequestId copies and the original survives; live is borrowed mutably, cloned once, and the clone is consumed by a by-value method while live stays usable.

Example 7-1. `move_copy_clone_receivers.rs` — Copy clone

```

#[derive(Debug, Copy, Clone)]
struct RequestId(u64);

#[derive(Debug, Clone)]
struct ServiceConfig {
    name: String,
    retries: usize,
}

impl ServiceConfig {
    fn name(&self) -> &str {
        &self.name
    }

    fn bump_retries(&mut self) {
        self.retries += 1;
    }

    fn replace_name(self, next: &str) -> Self {
        Self {
            name: next.to_string(),
        }
    }
}
  
```

```

        retries: self.retries,
    }
}

fn main() {
    let request = RequestId(42);
    let request_copy = request;

    let mut live = ServiceConfig {
        name: String::from("ingest-v1"),
        retries: 1,
    };
    live.bump_retries();

    let replaced = live.clone().replace_name("ingest-v2");

    println!("request id copy = {}", request_copy.0);
    println!("live retries = {}", live.retries);
    println!("builder consumed = {}", replaced.name());
    println!("current name = {}", live.name());
}

```

OUTPUT

```

request id copy = 42
live retries = 2
builder consumed = ingest-v2
current name = ingest-v1

```

Quick check: run the baseline, then try removing the `.clone()` before `replace_name`. The compiler refuses, because `replace_name(self)` would move `live` away and the last line still reads it. That is the move-versus-clone decision made concrete.

`RequestId` is a tiny value. Implicit duplication is predictable and the original stays valid.

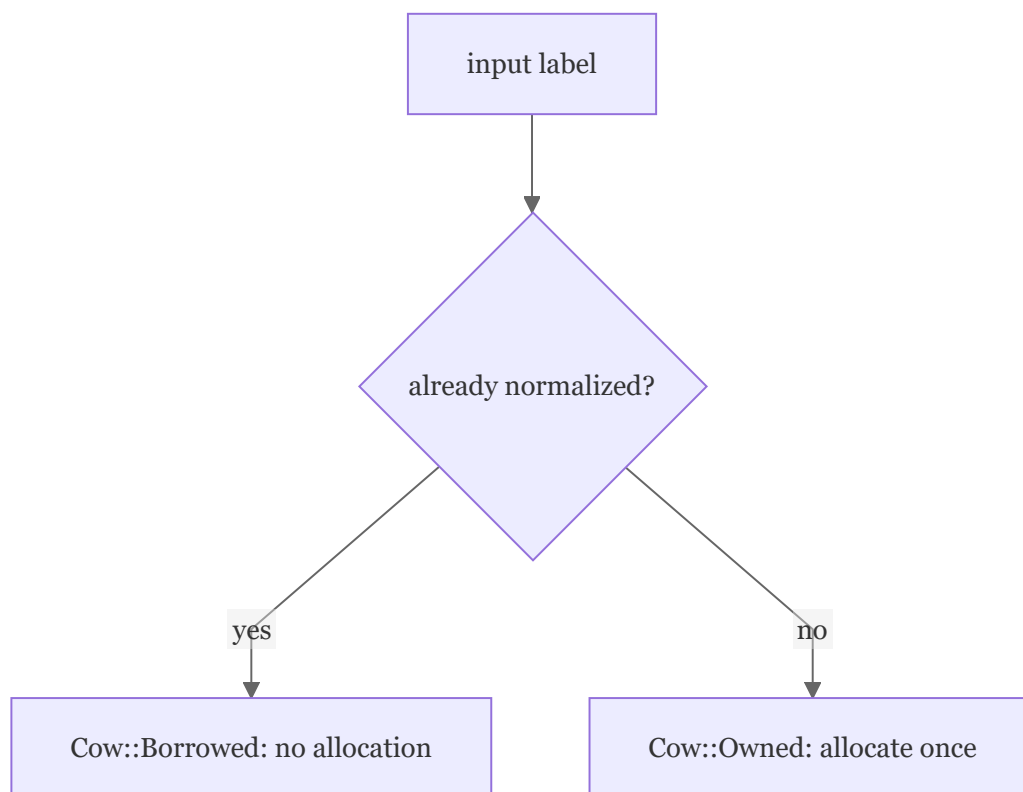
`live.clone()` creates a second independent config before the consuming builder-style call.

`&self` views, `&mut self` mutates, and `self` finalizes or transforms ownership.

Example 2: clone-on-write normalization and cheap shared ownership

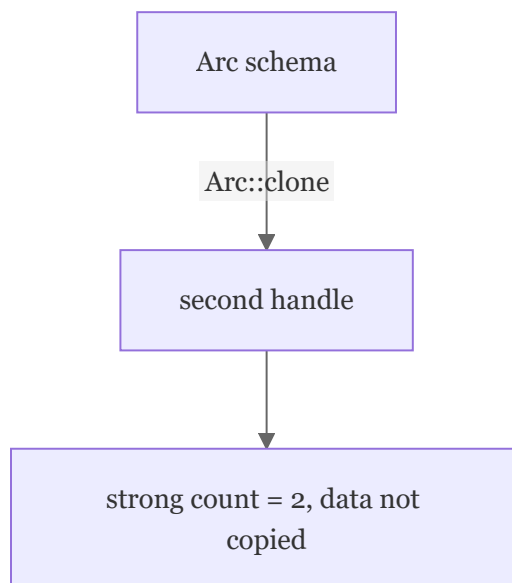
Borrow the common case with `Cow`, allocate only on transformation, and use `Arc::clone` when shared ownership is the real model.

What to look at: `normalize_label` branches on whether the input already fits the normalized form. An already-clean label returns `Cow::Borrowed` with no allocation; a dirty one returns `Cow::Owned` and pays once. The `Arc` lines are a second, separate idea: cloning a handle adds a shared owner rather than copying the string. The diagram splits those two paths so you can read each in the code.



Cow borrows the clean path and only allocates when it must transform the label.

The Arc lines are a separate idea: cloning a handle adds a shared owner rather than copying the string.



Arc::clone adds a shared owner and leaves the schema string untouched: the count goes to 2, the data is not duplicated.

Example 7-2. cow_and_arc_clone_costs.rs — Cow arc

```

use std::borrow::Cow;
use std::sync::Arc;

fn normalize_label(input: &str) -> Cow<'_, str> {

```

```

    if input.bytes().all(|b| matches!(b, b'a'..=b'z' | b'0'..=b'9' | b'-' )) {
        Cow::Borrowed(input)
    } else {
        Cow::Owned(input.trim().to_ascii_lowercase().replace(' ', "-"))
    }
}

fn main() {
    let borrowed = normalize_label("ready");
    let owned = normalize_label("Mixed Case");
    let schema = Arc::new(String::from("event.v1"));
    let worker_schema = Arc::clone(&schema);

    println!("borrowed = {}", borrowed);
    println!("owned = {}", owned);
    println!("strong count = {}", Arc::strong_count(&worker_schema));
}

```

OUTPUT

```

borrowed = ready
owned = mixed-case
strong count = 2

```

Quick check: change the labels and rerun. Inputs that already fit the normalized form stay borrowed; transformed inputs become owned. The Arc side shows the other distinction: cheap shared ownership is still not the same as deep cloning.

Cow lets the API return a borrowed value when no rewrite is needed.

The owned branch pays for normalization once, at the boundary that actually needs it.

Arc::clone increments the count. It does not copy the underlying schema string.

The repository also includes standalone Rust files under `examples/cho7_copying_data_vs_cloning_data/` so the chapter can be reviewed outside the web editor as ordinary Rust source.

Summary

- Moves, Copy, and Clone are different ownership events with different review meanings: a move invalidates the source, while Copy and Clone leave it usable.
- Copy is a marker on top of Clone for small bitwise-trivial values; Clone is the general, explicit duplication trait.
- Manual Clone should clone owned fields and copy plain scalar fields directly, and is a signal that duplication does something nontrivial.
- Rc and Arc cloning add shared owners; they increment a reference count and do not deep-copy the inner value.
- Borrow by default, allocate at real boundaries, and choose self, &self, and &mut self to match the ownership the method actually needs.

CHAPTER 8 · UNDEFINED BEHAVIOR AND UNSAFE RUST

Undefined Behavior and Unsafe Rust

What Rust considers UB, the role of unsafe, raw pointers, aliasing, data races, uninitialized memory, FFI, and safe abstractions over unsafe internals

Unsafe Rust is justified only when a small boundary must enforce invariants the compiler cannot verify. This chapter defines the audit obligations for raw pointers, initialization, aliasing, FFI, and safe wrappers.

Builds on Chapters 04 through 07

Ownership, borrowing, struct layout, vector invalidation, and move versus clone decisions all lead here. Unsafe Rust is not a separate language. It is the place where those earlier rules stop being compiler guarantees and start becoming explicit human obligations.

Opening scenario

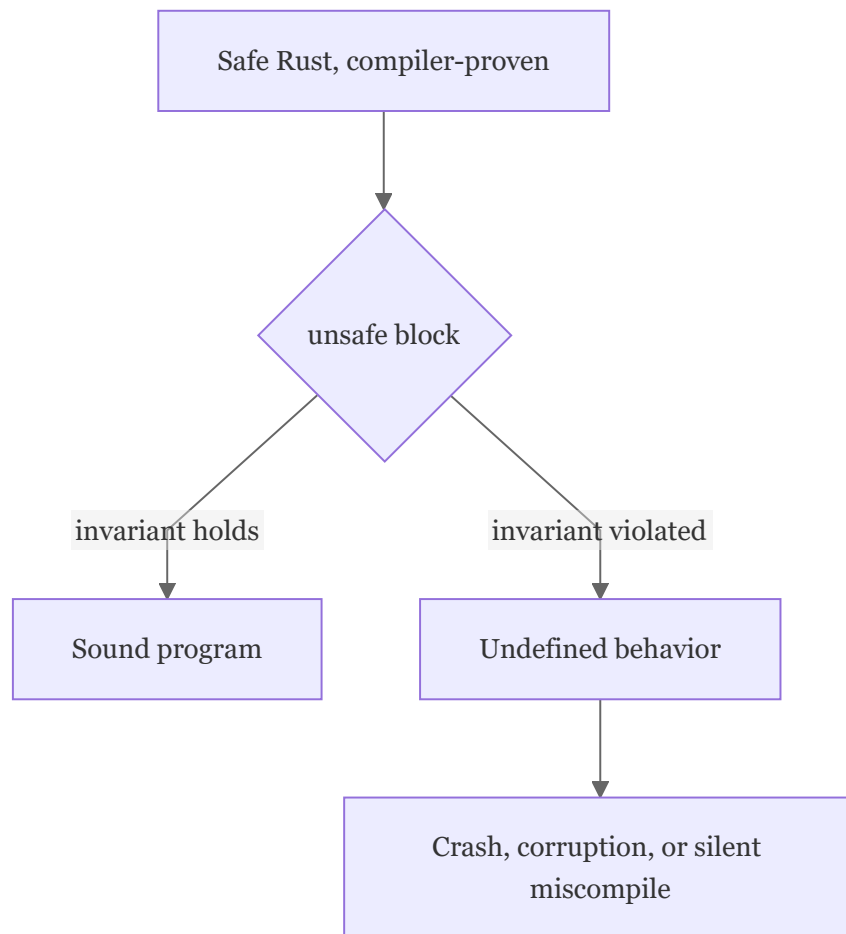
A packet-processing service validates fixed headers, calls a native library, and reuses buffers on a measured hot path. The business requirement is a narrow unsafe boundary with written invariants for pointer validity, initialization, aliasing, thread safety, and safe wrapper behavior.

A useful rule before writing unsafe code

1. Write the safe version first if the shape is still unclear.
2. State the exact reason unsafe is needed: FFI, layout, partial initialization, or a measured hot path.
3. Write down the invariant before the block, not after the bug report.

Mental model

The picture to hold in your head is a containment boundary. Almost everything you write is ordinary safe Rust, where the compiler proves memory safety. A few operations cannot be proven that way, so they live inside an explicit region with a written invariant. Cross that line correctly and the program is sound; cross it on a false assumption and you are in undefined behavior, where the optimizer is free to do anything.



Unsafe is a boundary, not a mode. The compiler proves the safe region; you prove the unsafe region. A broken invariant does not fail loudly, it leaks into undefined behavior.

Core concepts

What Rust considers undefined behavior

Treat the list below as the operational map, not as a claim of complete exhaustiveness. The key idea is that UB is exactly where the optimizer is allowed to assume the impossible states cannot occur, so reasoning based on what the hardware happens to do no longer holds.

Safe Rust vs unsafe Rust

Safe Rust rules out use-after-free, double-free, and data races in ordinary code. References, ownership transfer, `Send`, `Sync`, and borrowing rules do most of the structural work long before a test suite runs.

Unsafe Rust keeps the same language around you, but it permits operations the compiler cannot fully verify. The rest of the code still obeys ordinary Rust rules. The unsafe block is the proof boundary, not a local switch to “anything goes.”

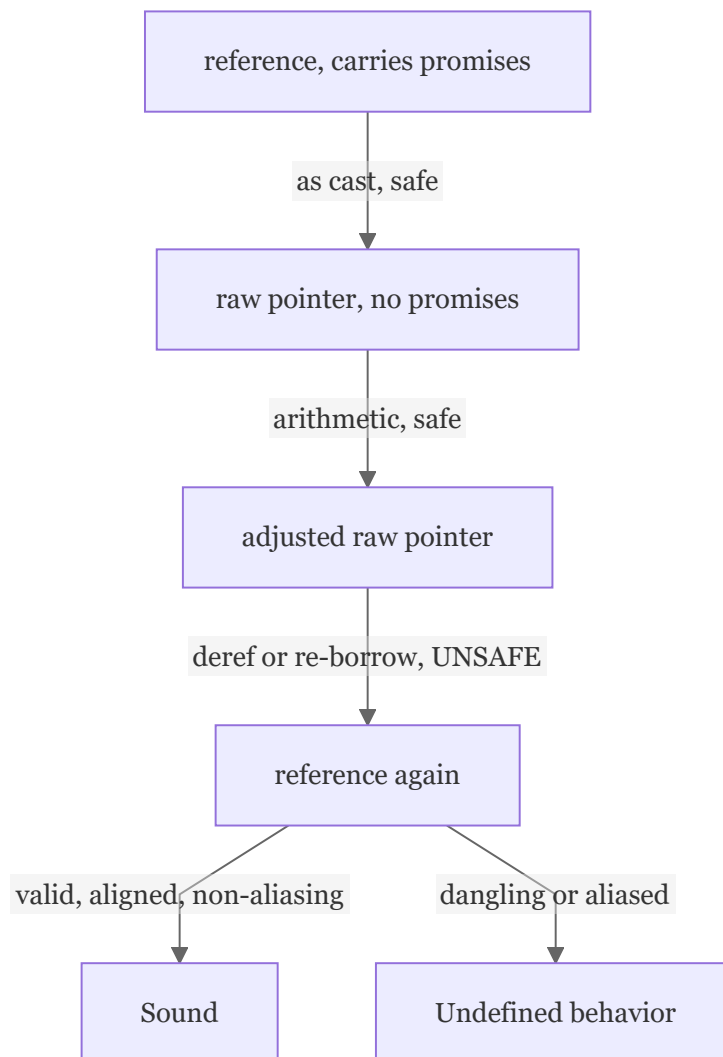
The role of unsafe blocks

`unsafe` grants specific capabilities. It does not disable ownership, type checking, or syntax rules, and it does not make UB acceptable. It simply says: the compiler cannot prove this, so a human must.

The practical consequence is simple: a tiny unsafe block with a precise `// SAFETY:` comment is usually a better design than a large region whose real invariants are only understood after several meetings.

Raw pointers and aliasing rules

The thing to watch is the asymmetry between a raw pointer and a reference. A raw pointer is just an address with no promises attached, so making one and doing arithmetic on it is always safe. A reference is an address plus a contract the optimizer relies on: `&T` promises the data stays valid and unmutated for the borrow, and `&mut T` promises exclusive access. The dangerous step is not the dereference itself, it is turning a raw pointer back into a reference, because at that moment you are re-asserting the contract. If the address is dangling or aliased, you have lied to the optimizer and the resulting reference is invalid even though the machine instruction ran without complaint.



Dropping a reference down to a raw pointer is free; the address loses its promises. Climbing back up to a reference is the unsafe step, because it re-asserts validity and exclusivity that may no longer be true.

`*const T` and `*mut T` may be null, dangling, misaligned, or aliased. That is why creating them is safe but dereferencing them is not. Raw pointers are representation tools, not evidence that a memory access is valid.

References are stronger than raw pointers because they promise something to the optimizer. Turning a raw pointer back into `&T` or `&mut T` means you are reintroducing those promises. If exclusive mutation is not

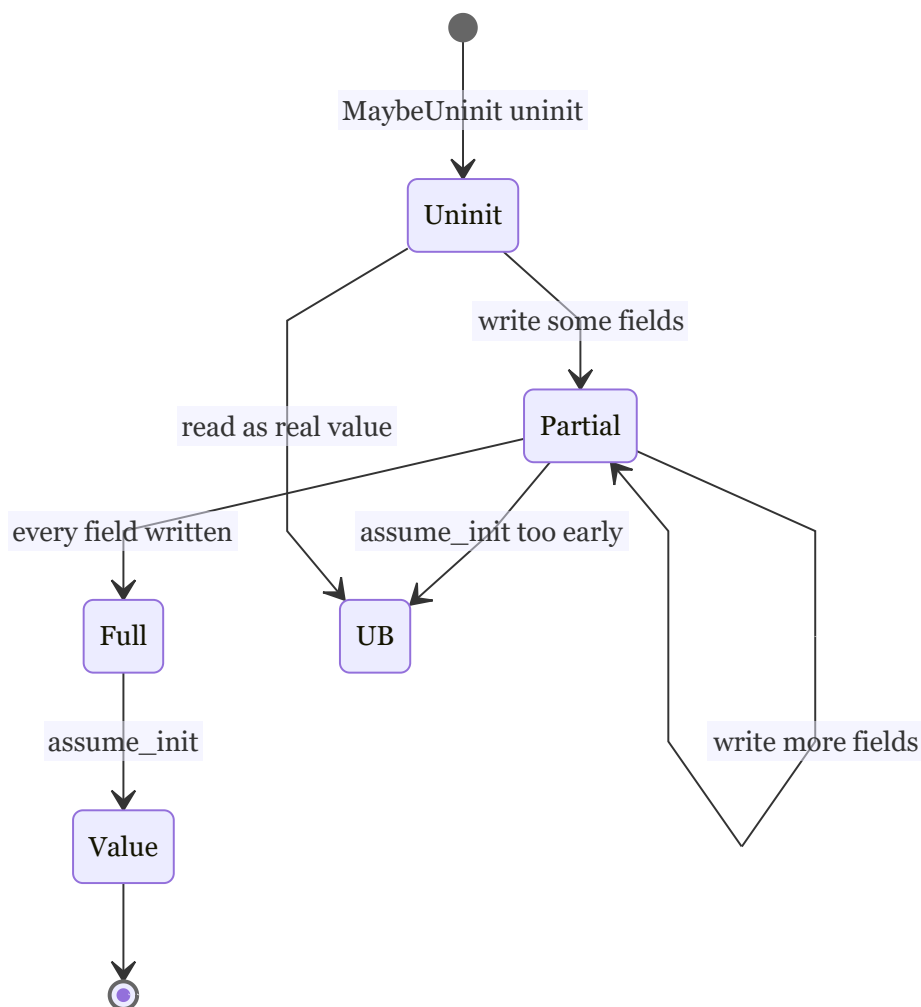
actually exclusive, the resulting reference is invalid even if the machine instruction succeeds.

Data races and UB

In Rust, a data race is not merely a bug; it is undefined behavior. Safe Rust prevents it by construction through ownership transfer, borrowing rules, and `Send / Sync` requirements at thread boundaries. Unsafe code can still create conflicting unsynchronized accesses. That is why shared mutability, raw pointers, and concurrency belong in the same audit conversation, not in separate review queues.

Uninitialized memory and `MaybeUninit<T>`

The shape to keep in mind is a one-way ratchet. Storage starts in a not-yet-valid state, you write every required byte or field exactly once, and only at the end do you assert that it now holds a real value. The error to avoid is calling `assume_init` while the storage is still partially written, which produces a value that the rest of the program will treat as valid even though it is not.



assume_init is the single legal exit, and only from the fully written state. Reading the storage as a real value before that, or asserting it early, is undefined behavior.

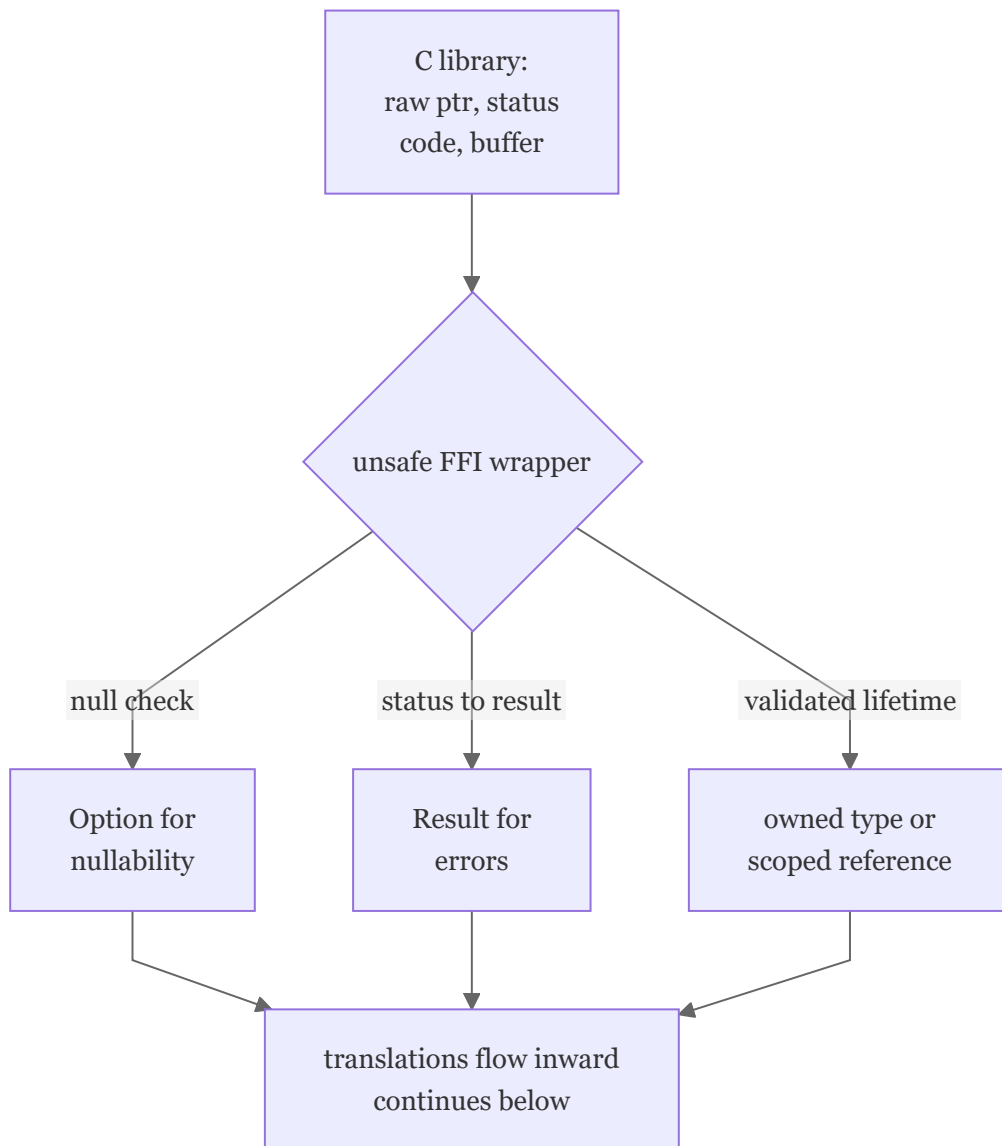
Uninitialized memory is not a valid value of any type. Reading it as a real `T` is UB. The same is true for constructing invalid bit patterns for types that have invariants, such as references, many enums, and `bool`.

`MaybeUninit<T>` is the sanctioned way to represent storage that does not yet contain a valid `T`. Write every required field or element first. Only then call `assume_init`. The type exists so the invalid state is modeled

explicitly rather than hidden behind fake values.

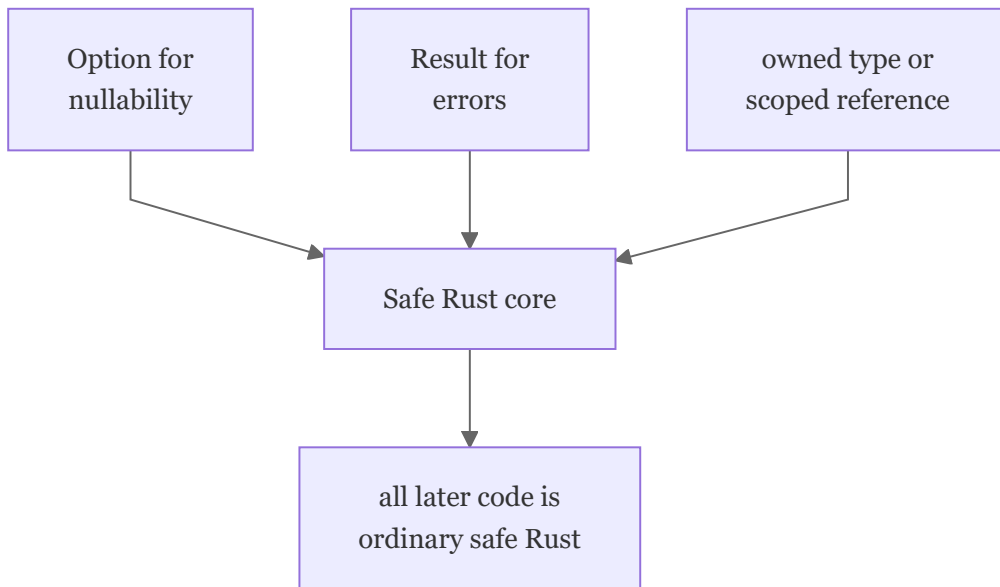
FFI-related UB

The useful framing for a C boundary is a translation layer with a single direction of travel. Raw C values come in as pointers, status codes, and lifetime-free buffers; you translate each into a Rust type that carries the guarantee, and from there inward everything is ordinary safe Rust. The UB lives in the thin translation step, so that is the only place that needs the unsafe audit. Letting raw C pointers travel deeper into your code is how an FFI contract mismatch turns into a memory-safety bug far from the boundary.



First half: the unsafe wrapper translates each C convention into a Rust type. Nullability becomes Option, status codes become Result, raw buffers become owned values or scoped references.

Inward of the wrapper, those three translated values converge into ordinary safe Rust:



Second half: the translated values feed the safe Rust core. Once past the wrapper, no raw C pointer travels deeper, so the rest of the code stays ordinary safe Rust.

- Match the ABI exactly.
- Document who owns the pointer and who frees it.
- Convert nullability into `Option` or explicit error handling at the boundary.
- Validate string assumptions: encoding, lifetime, and termination.
- Do not invent references from raw FFI pointers unless the lifetime story is real.
- Be explicit about layout expectations and `repr(C)` where required.
- Do not let unwinding expectations stay fuzzy across the boundary.
- Keep the unsafe part in one wrapper layer so the rest of the code can stay ordinary Rust.

Auditing unsafe code

Writing safe abstractions over unsafe internals

If the callee can check bounds, maintain ownership, and prevent aliasing mistakes, keep the public API safe and isolate the raw operation internally. This is the common pattern for slices, small buffer wrappers, parsers, and data-structure internals.

If correctness depends on facts the callee cannot validate, such as a pointer being valid for a chosen lifetime or a caller-controlled aliasing guarantee, make that obligation explicit with `unsafe fn` and document the preconditions precisely.

How this lands by background

Almost nobody meets unsafe Rust without a prior model of how memory safety works in another language. The useful thing to know is which part of that model transfers and which part will quietly mislead you. The shift is rarely about syntax. It is about where the proof of safety lives and who is responsible for it once the compiler steps back.

Production patterns

Pitfalls and tradeoffs

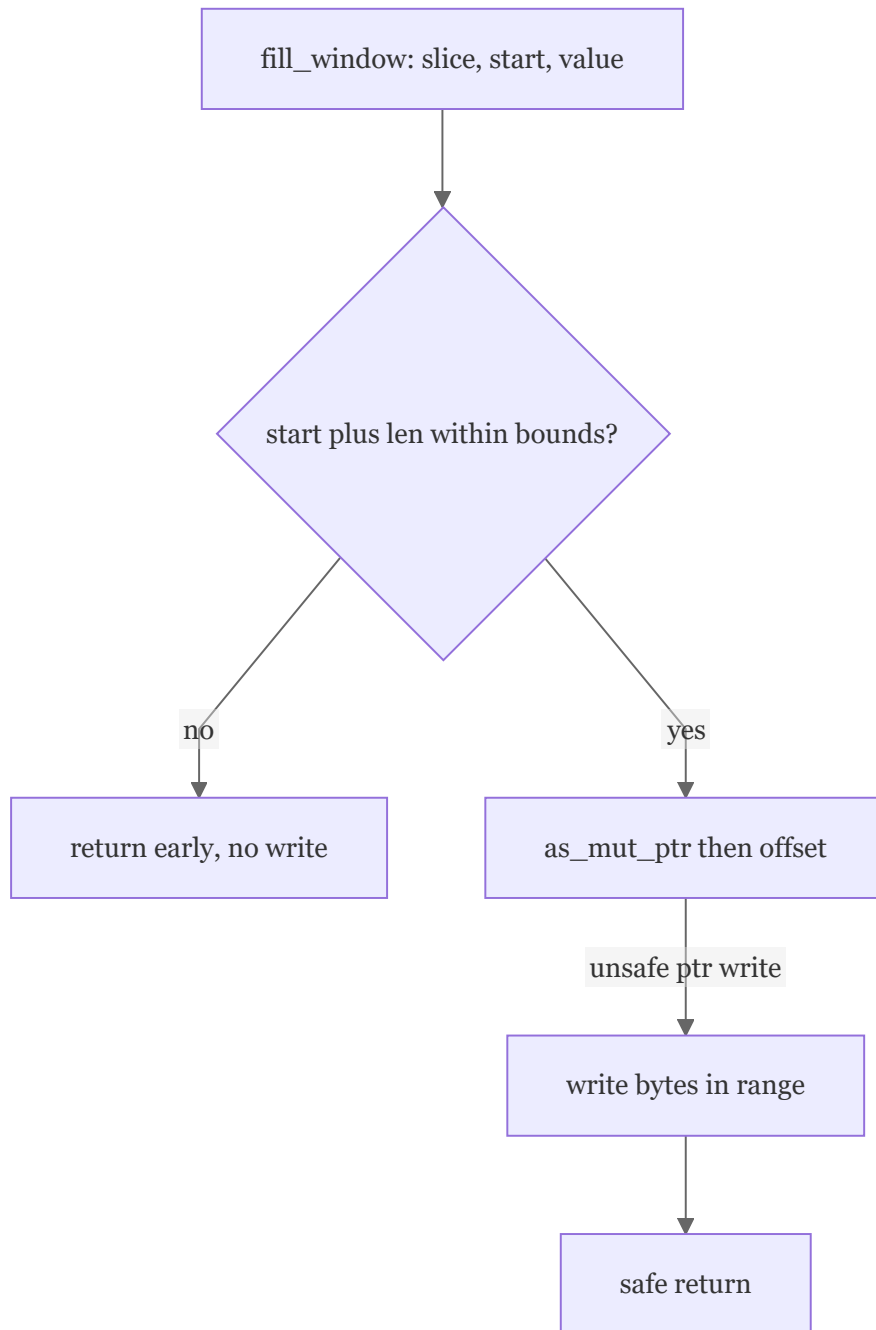
Unsafe Rust is a cost, not a shortcut. Use it where the representation win is real, document the invariant where reviewers can see it, and keep the surface area small enough that future maintainers can still re-derive the proof.

Examples

Example 1: a safe slice API over raw-pointer writes

The caller gets a safe function. Bounds and exclusivity are enforced before the raw-pointer write loop begins.

What to look at: every check that makes the unsafe write legal happens before the loop. The bounds test and the exclusive `&mut [u8]` borrow turn the caller-facing function into a safe one, so the raw-pointer write at the center inherits an already-proven invariant rather than asserting a new one.



The bounds check guards the only unsafe step. Because the slice arrives as an exclusive `&mut [u8]`, exclusivity is already established, so the write cannot alias another live reference.

Example 8-1. `fill_window_with_raw_pointers.rs` — Fill window

```
fn fill_window(buf: &mut [u8], start: usize, len: usize, value: u8) {
    assert!(start <= buf.len());
    assert!(start + len <= buf.len());

    let ptr = buf.as_mut_ptr();

    for offset in 0..len {
        unsafe {
            // SAFETY:
            // - `buf` is exclusively borrowed for the duration of the call.

```

```

        // - bounds were checked above, so `start + offset` is in-bounds.
        // - overwriting initialized `u8` values is fine because `u8` has no drop glue.
        ptr.add(start + offset).write(value);
    }
}

fn main() {
    let mut packet = String::from("header:0000").into_bytes();
    fill_window(&mut packet, 7, 4, 57);

    println!("{}", std::str::from_utf8(&packet).unwrap());
}

```

OUTPUT

```
header:9999
```

The public function is safe because it can check bounds and it borrows the slice as `&mut [u8]`, which already gives exclusive access.

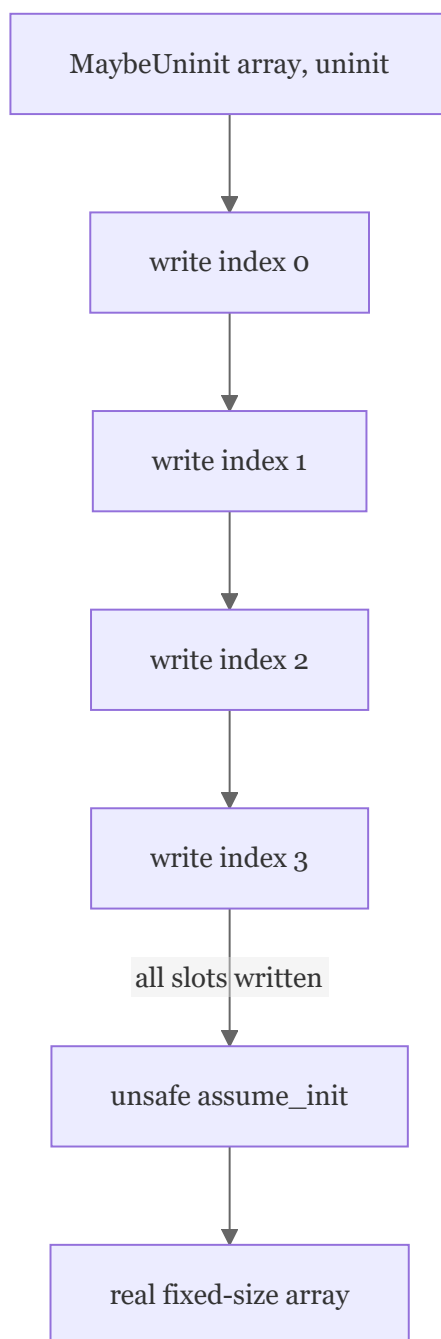
The only unsafe step is pointer arithmetic plus write. Everything around it exists to make that one step justified.

Overwriting with the raw-pointer write method (`.write()`) is fine here because the element type is `u8`. For a drop-carrying type such as `String`, skipping the old destructor would matter.

Example 2: build initialized bytes with `MaybeUninit`

The storage starts explicitly uninitialized, each byte is written once, and `assume_init` appears only after the invariant is fully true.

What to look at: the order of operations. The array is declared as uninitialized storage, each slot is written exactly once, and only the final `assume_init` turns the storage into a real value. Trace the diagram and confirm that no read happens on any branch before the last write.



Initialization is a straight line with one exit. Every slot is written before `assume_init` runs, so the value the rest of the program sees is always fully valid.

Example 8-2. `maybe_uninit_header.rs` — *Maybe uninit*

```

use std::mem::MaybeUninit;

fn build_header(tag: u8, size: u8) -> [u8; 4] {
    let mut bytes = MaybeUninit::<[u8; 4]>::uninit();
    let ptr = bytes.as_mut_ptr() as *mut u8;

    unsafe {
        // SAFETY:
        // - each element is written exactly once before `assume_init`.
        // - no reads occur before initialization completes.
        ptr.add(0).write(tag);
    }
}

```

```
        ptr.add(1).write(size);
        ptr.add(2).write(tag ^ size);
        ptr.add(3).write(255);
        bytes.assume_init()
    }
}

fn main() {
    let header = build_header(7, 10);
    println!("{:?}", header);
}
```

OUTPUT

```
[7, 10, 13, 255]
```

All four elements must be written before the array becomes a real `[u8; 4]`.

It models the not-yet-valid state explicitly, which is far better than treating arbitrary bytes as if they already represent a valid value.

The critical review question is simple: can any path read the value before every element is initialized? If yes, the abstraction is wrong.

Summary

- Unsafe Rust allows operations the compiler cannot prove safe; it does not weaken the actual safety requirements.
- Undefined behavior includes invalid pointer dereferences, reference aliasing violations, data races, invalid values, and many bad FFI contracts.
- Raw pointers are representation tools. References are promises. Turning one into the other is where invariants matter most.
- `MaybeUninit<T>` is the right model for genuinely uninitialized storage. `assume_init` belongs at the end of a proven initialization path.
- The best production pattern is usually a safe API with a tiny, documented, auditable unsafe core.

CHAPTER 9 · SMART POINTERS AND PINNING

Smart Pointers and Pinning

Box, Rc, Arc, Cell, RefCell, Mutex, Weak, Pin, and the ownership tradeoffs behind them

Smart pointers encode ownership count, mutation authority, thread sharing, and address stability. This chapter uses Box, Rc, Arc, Weak, interior mutability, and Pin as production design choices.

Builds on Chapters 05, 07, and 08

Chapter 05 introduced ownership inside structs. Chapter 07 separated move, copy, and clone. Chapter 08 explained why some low-level invariants matter enough to be audited explicitly. This chapter brings those threads together around the standard smart pointers and pinning model.

Opening scenario

A service combines recursive parser state, shared planning data, cross-thread schemas, mutable retry coordination, and pinned async work. The business requirement is to encode each ownership and movement rule in the pointer type: single owner, shared owner, synchronized mutation, non-owning back edge, or immovable state.

A good review order

1. How many owners are semantically real: one or many?
2. Is mutation local, runtime-checked, or synchronized across threads?
3. Is a back-edge observing or owning?
4. Does the value merely live on the heap, or must it actually stop moving?

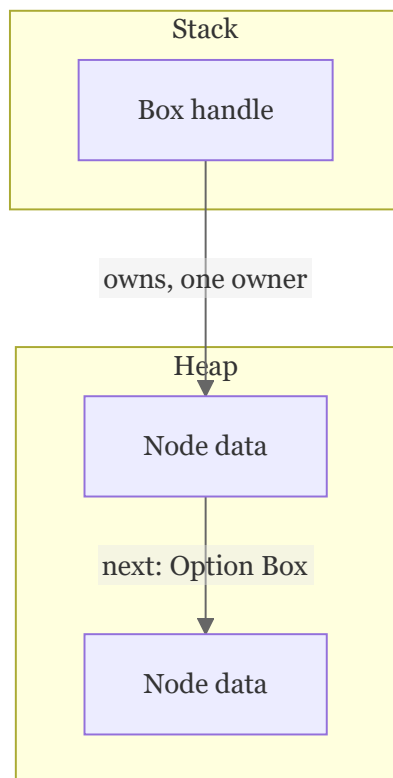
Mental model

Core concepts

Box<T>: heap ownership

Box<T> means one owner plus heap indirection. It does not imply sharing, reference counting, or interior mutability. It is the right answer when the pointee should still have one owner but the outer type needs recursive shape, a smaller stack footprint, or a stable-sized field.

What to look at: the box is a thin owning handle that lives wherever the field lives, while the pointee it owns sits on the heap. That indirection is what lets a recursive type have a finite size, because the field is one pointer wide no matter how deep the chain goes.



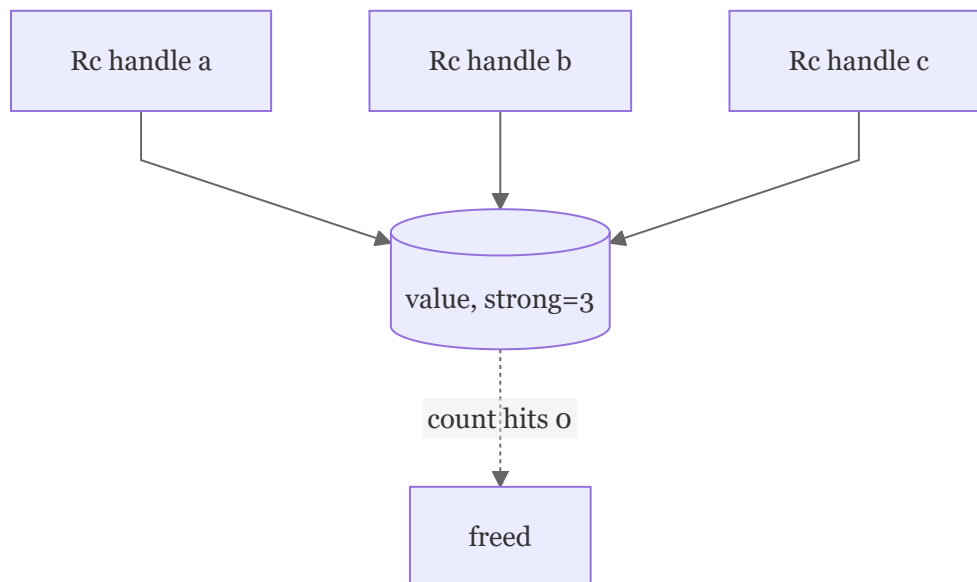
A Box is a single owning pointer on the stack; the value it owns lives on the heap, which is what makes a recursive node a fixed-size field.

Think of it as owned heap storage, not as “shared pointer lite.”

Rc<T>: single-threaded shared ownership

Rc<T> is shared ownership inside one thread. Cloning an Rc adds another owner by incrementing a non-atomic count. Because the count is not atomic, Rc<T> is not for thread boundaries.

What to look at: several handles point at one allocation, and the allocation carries a count. Each clone bumps the count; each drop lowers it; the value is freed only when the count reaches zero. The data itself is shared and read-only through these handles.



Cloning an Rc adds an owner and raises the strong count; the allocation lives until the last owner drops and the count reaches zero.

If you later need shared mutation as well, that is a second decision. Many single-thread designs use `Rc<RefCell<T>>`, but you should read that type honestly: multiple owners plus runtime borrow checking.

Arc<T>: thread-safe shared ownership

`Arc<T>` is the cross-thread sibling of `Rc<T>`. The reference count is atomic, so cloning is safe across threads. The atomic work is usually modest, but it is not free.

The key correction is this: `Arc<T>` solves ownership count, not arbitrary mutation safety. If multiple threads need mutation, add a second mechanism such as a lock or redesign toward message passing. Also remember that thread use still depends on the inner `T` satisfying the relevant `Send` and `Sync` bounds.

Interior mutability: moving the borrow check somewhere else

What to look at: all three of the types below let you mutate through a shared handle, which ordinary borrowing forbids. The difference is purely where and how the aliasing rule is enforced. A `Cell<T>` sidesteps borrowing by only swapping whole values, a `RefCell<T>` keeps the same rule but checks it at runtime and panics on violation, and a `Mutex<T>` enforces it across threads by blocking on a lock.

RefCell<T>: runtime borrow checking

`RefCell<T>` lets shared owners borrow mutably at runtime. The same borrowing law still exists, but violations become runtime panics instead of compile-time errors. That is useful in single-thread graphs, UI trees, and other genuinely dynamic borrow patterns. It is also a tradeoff you should feel.

Cell<T> and interior mutability

`Cell<T>` is the smaller, calmer interior-mutability tool when you only need to copy out, replace, or swap values. It works well for counters, flags, and small cached scalars in one thread. Unlike `RefCell<T>`, it does not hand out interior references.

Mutex<T>: ownership under synchronization

Mutex<T> provides interior mutability by handing out exclusive access only after lock acquisition. It is the shared-state sibling of ordinary exclusive borrowing. Lock scope, contention, poisoning policy, and deadlock risk are now operational concerns.

Weak<T> and cyclic references

Reference counting alone cannot collect cycles. If parent owns child and child also owns parent through a strong Rc or Arc, neither side reaches a strong count of zero. The usual repair is simple: owning edges stay strong, observational back-edges become weak.

A Weak<T> does not keep the allocation alive. Calling upgrade() returns Option<Rc<T>> or Option<Arc<T>>, which forces you to handle the “owner already gone” case explicitly.

Pin<T> and immovable values

Pinning matters when moving a value would break an address-sensitive invariant. The classic example is future polling: Future::poll takes Pin<&mut Self>. The compiler can generate state machines whose internal references assume the future stays put while polled.

Pinning does not stop all mutation. It stops moves that would invalidate the address-sensitive part of the invariant. For most ordinary types, this distinction does not matter because they are effectively movable and implement Unpin.

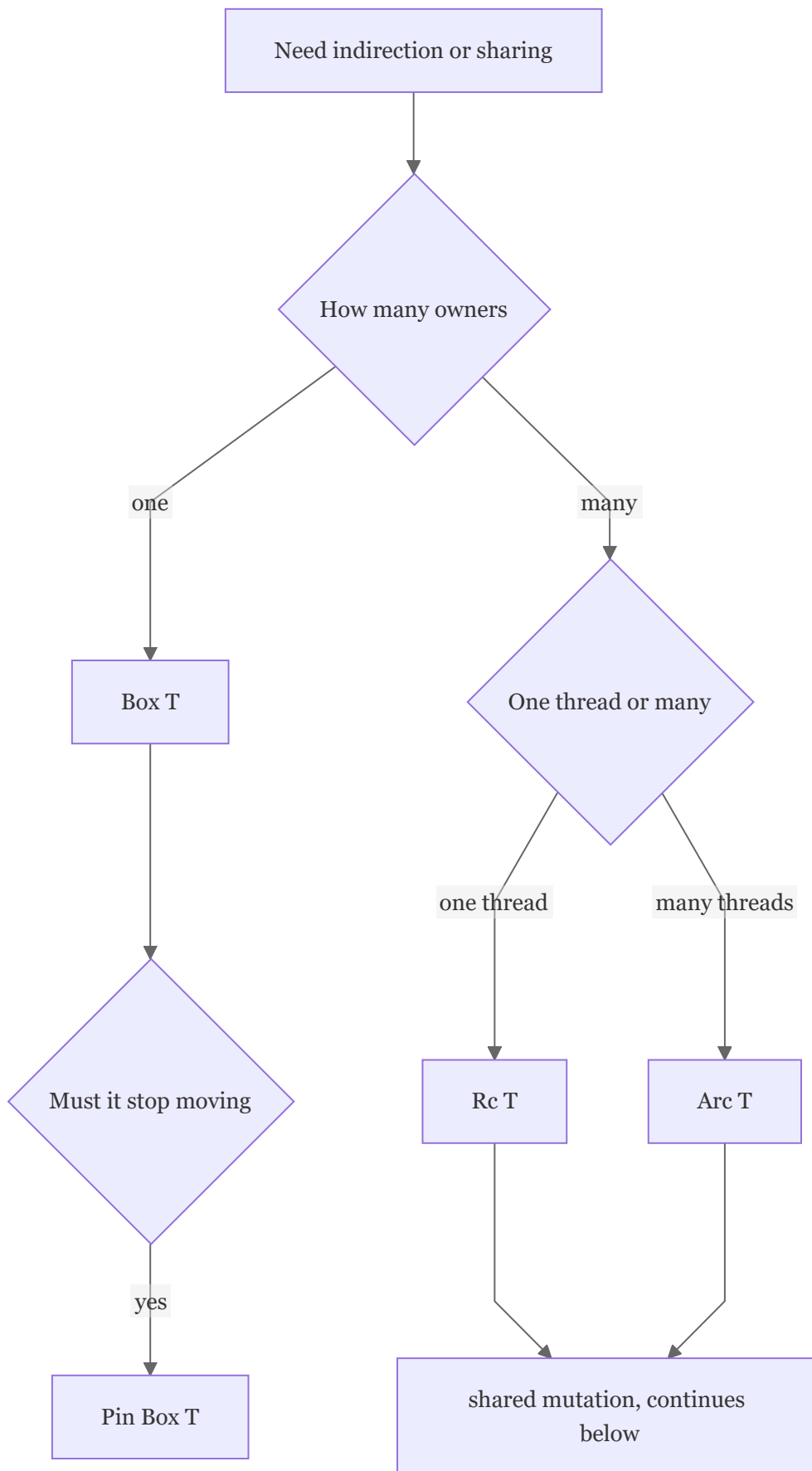
Pin<Box<T>> and Pin<&mut T>

Pin<Box<T>> means you own the value and pin the pointee in its heap allocation. This is common when storing futures you want to poll later. Pin<&mut T> is the temporary pinned view used when actually operating on the pinned value.

The sharp edge is projection. For !Unpin types, you cannot casually recover a plain &mut T because that would re-enable movement. Later async and unsafe chapters revisit this with deeper machinery.

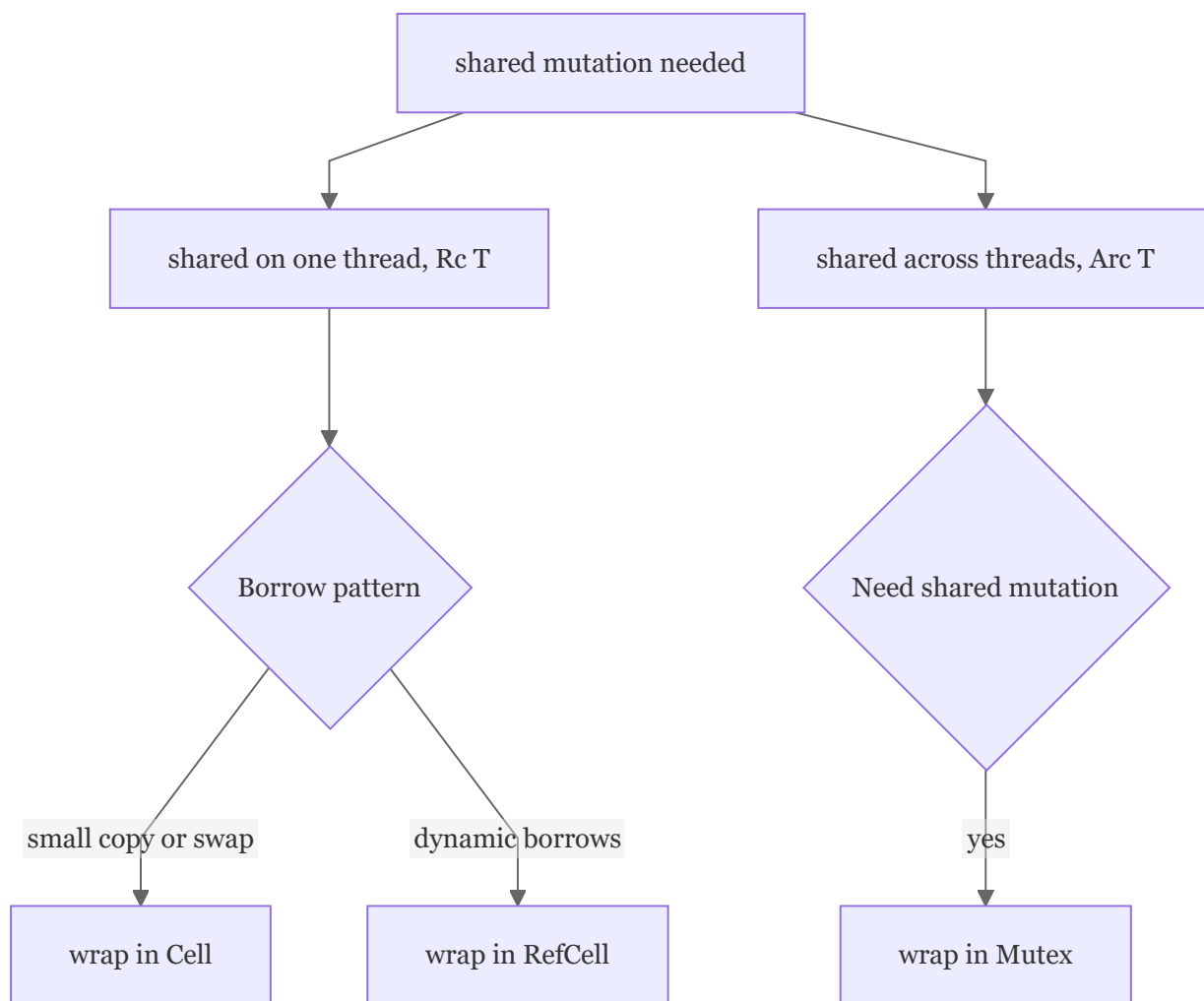
Choosing the right pointer type

What to look at: the choice is a short series of independent questions, not a single menu pick. Answer them in order, and the type usually falls out. How many owners are real? If many, do they live on one thread or cross thread boundaries? Does anything need to mutate the shared value, and if so, is the borrow pattern static enough for the compiler or dynamic enough to need a runtime check or a lock? Only at the end do you ask whether the value must also stop moving.



First the ownership questions: how many owners, and if many, do they cross thread boundaries. A single owner picks Box, and only Box faces the separate question of whether the value must stop moving.

Once owner count and thread boundary are settled, the remaining question is whether the shared value also needs mutation, and under which enforcement model:



The second half resolves shared mutation: a single-thread Rc reaches for Cell or RefCell depending on the borrow pattern, while a cross-thread Arc reaches for a Mutex.

A useful correction for senior engineers is this: pointer choice is often two or three independent questions layered together. `Arc<Mutex<T>>` means shared ownership and synchronized mutation. `Rc<RefCell<T>>` means shared ownership and runtime borrow checking on one thread. Read the composition literally.

How this lands by background

Most senior engineers do not meet smart pointers as a blank slate; they arrive with a pointer model from another language. The useful thing to know is which part of that model carries over and which part will quietly mislead you. The shift is almost never about which type maps to which library class. It is about the fact that ownership count, mutation authority, and address stability now live in the type and are checked, rather than being left to convention or a runtime.

Production patterns

Pitfalls and tradeoffs

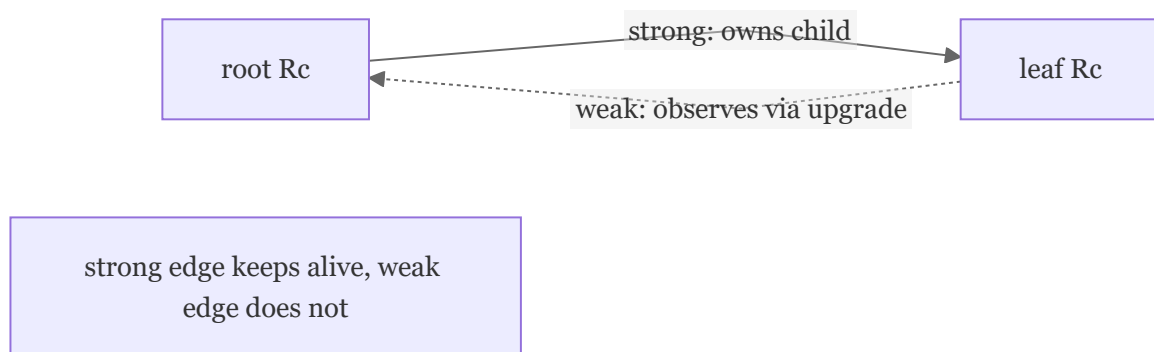
Smart pointers are not escape hatches from ownership design. They are ownership design written into the type. If the type looks complicated, the runtime story is probably complicated too.

Examples

Example 1: shared tree edges with weak back-references

The children are strongly owned. The parent edge is weak, so the graph stays acyclic.

What to look at: the two edges between root and leaf are deliberately different kinds. The root owns the leaf through a strong Rc in its children list, while the leaf only observes the root through a Weak parent pointer. Trace the two arrows in the diagram, then notice that the leaf reads its parent through `upgrade()`, which can fail if the owner is already gone. That asymmetry is what keeps the graph acyclic and lets the count actually reach zero.



Owning edges stay strong and point down the tree; the back-edge to the parent is weak, so no cycle pins the allocation alive.

Example 9-1. rc_weak_tree.rs — Pointers tree

```

use std::cell::RefCell;
use std::rc::{Rc, Weak};

#[derive(Debug)]
struct Node {
    name: String,
    parent: RefCell<Weak<Node>>,
    children: RefCell<Vec<Rc<Node>>>,
}

fn main() {
    let root = Rc::new(Node {
        name: String::from("root"),
        parent: RefCell::new(Weak::new()),
        children: RefCell::new(Vec::new()),
    });
  
```

```

let leaf = Rc::new(Node {
    name: String::from("leaf"),
    parent: RefCell::new(Weak::new()),
    children: RefCell::new(Vec::new()),
});

root.children.borrow_mut().push(Rc::clone(&leaf));
*leaf.parent.borrow_mut() = Rc::downgrade(&root);

let parent_name = leaf
    .parent
    .borrow()
    .upgrade()
    .map(|node| node.name.clone())
    .unwrap_or_else(|| String::from("none"));

println!("root children = {}", root.children.borrow().len());
println!("leaf parent = {}", parent_name);
println!("root strong = {}", Rc::strong_count(&root));
}

```

OUTPUT

```

root children = 1
leaf parent = root
root strong = 1

```

Quick check: change the node names or add another child. The useful signal is not the printout itself. It is that the ownership graph reads clearly from the types: strong edges own, weak edges observe.

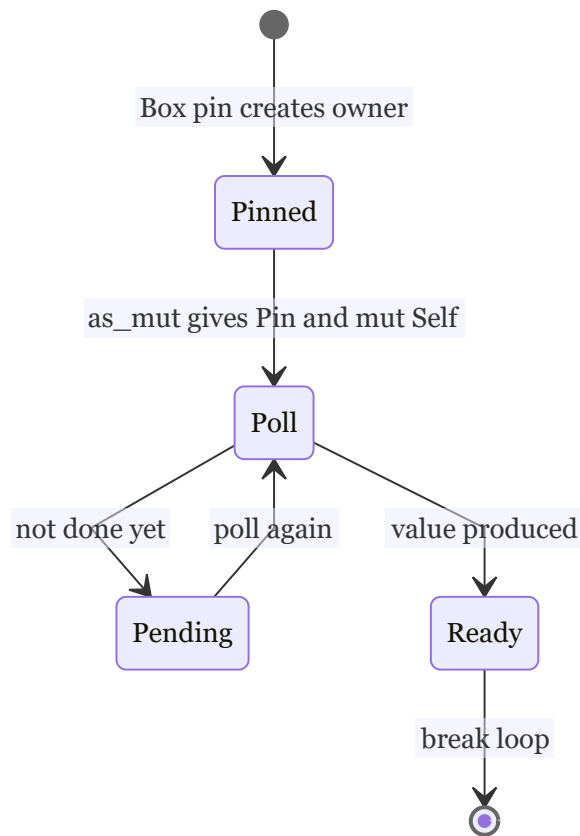
A child can discover its parent only by upgrading a weak pointer. The parent is not kept alive by the child.

Trees, observer lists, dependency graphs, and cache indexes often need one-way ownership plus non-owning lookup edges.

Example 2: polling a pinned future with `Box::pin`

The example is deliberately small so you can see both forms: owned pinned storage and a pinned mutable view during polling.

What to look at: `Box::pin` creates the owned, pinned storage once, and then the loop polls it repeatedly through `as_mut()`, which hands poll the `Pin<&mut Self>` it requires. Each poll either reports it is still pending and yields, or returns a ready value and ends the loop. Follow that one-future, two-exits cycle in the diagram, then read the same shape in the match arms.



One pinned future is polled in a loop; each poll yields Pending and repeats, or yields Ready and exits. The pin stays fixed across every poll.

Example 9-2. `pin_box_and_poll.rs` — Pin poll

```

use std::future::Future;
use std::pin::Pin;
use std::sync::Arc;
use std::task::{Context, Poll, Wake, Waker};

struct Countdown {
    remaining: u8,
}

impl Future for Countdown {
    type Output = &'static str;

    fn poll(self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<Self::Output> {
        let this = self.get_mut();

        if this.remaining == 0 {
            Poll::Ready("done")
        } else {
            this.remaining -= 1;
            Poll::Pending
        }
    }
}

struct NoopWake;

```

```
impl Wake for NoopWake {
    fn wake(self: Arc<Self>) {}
}

fn main() {
    let waker = Waker::from(Arc::new(NoopWake));
    let mut cx = Context::from_waker(&waker);
    let mut task = Box::pin(Countdown { remaining: 2 });

    loop {
        match Future::poll(task.as_mut(), &mut cx) {
            Poll::Ready(value) => {
                println!("ready = {}", value);
                break;
            }
            Poll::Pending => {
                println!("pending = {}", task.as_ref().get_ref().remaining);
            }
        }
    }
}
```

OUTPUT

```
pending = 1
pending = 0
ready = done
```

Quick check: increase the starting count and rerun. The visible behavior changes, but the structural point stays the same: `Box::pin` gives you a pinned owner, and `as_mut()` produces the pinned mutable view required by polling.

The pointee inside the box, not the box handle itself, is the thing treated as immovable for the poll protocol.

The poll API must be safe for compiler-generated state machines whose internal layout may rely on a stable address while the future is alive.

The repository also includes standalone Rust source under `examples/cho9_smart_pointers_and_pinning/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

- Choose smart pointers by ownership truth: one owner, many owners, one thread, many threads, movable, or pinned.
- `Box<T>`, `Rc<T>`, and `Arc<T>` answer ownership-count and thread questions, not arbitrary mutation questions.
- `Cell<T>`, `RefCell<T>`, and `Mutex<T>` are different interior-mutability contracts with different enforcement costs.
- `Weak<T>` is the standard repair for non-owning back-edges in reference-counted graphs.
- Pin matters only when movement itself would break correctness. Heap allocation alone is not the same thing.

CHAPTER 10 · ARRAYS, SLICES, AND VECTORS

Arrays, Slices, and Vectors

Fixed-size arrays, slice-first APIs, `Vec<T>` internals, capacity planning, mutation patterns, and contiguous-storage tradeoffs

Contiguous storage choices affect allocation, cache locality, and API flexibility. This chapter defines arrays for fixed shape, slices for borrowed access, and vectors for owned growth.

Builds on Chapters 04, 06, and 07

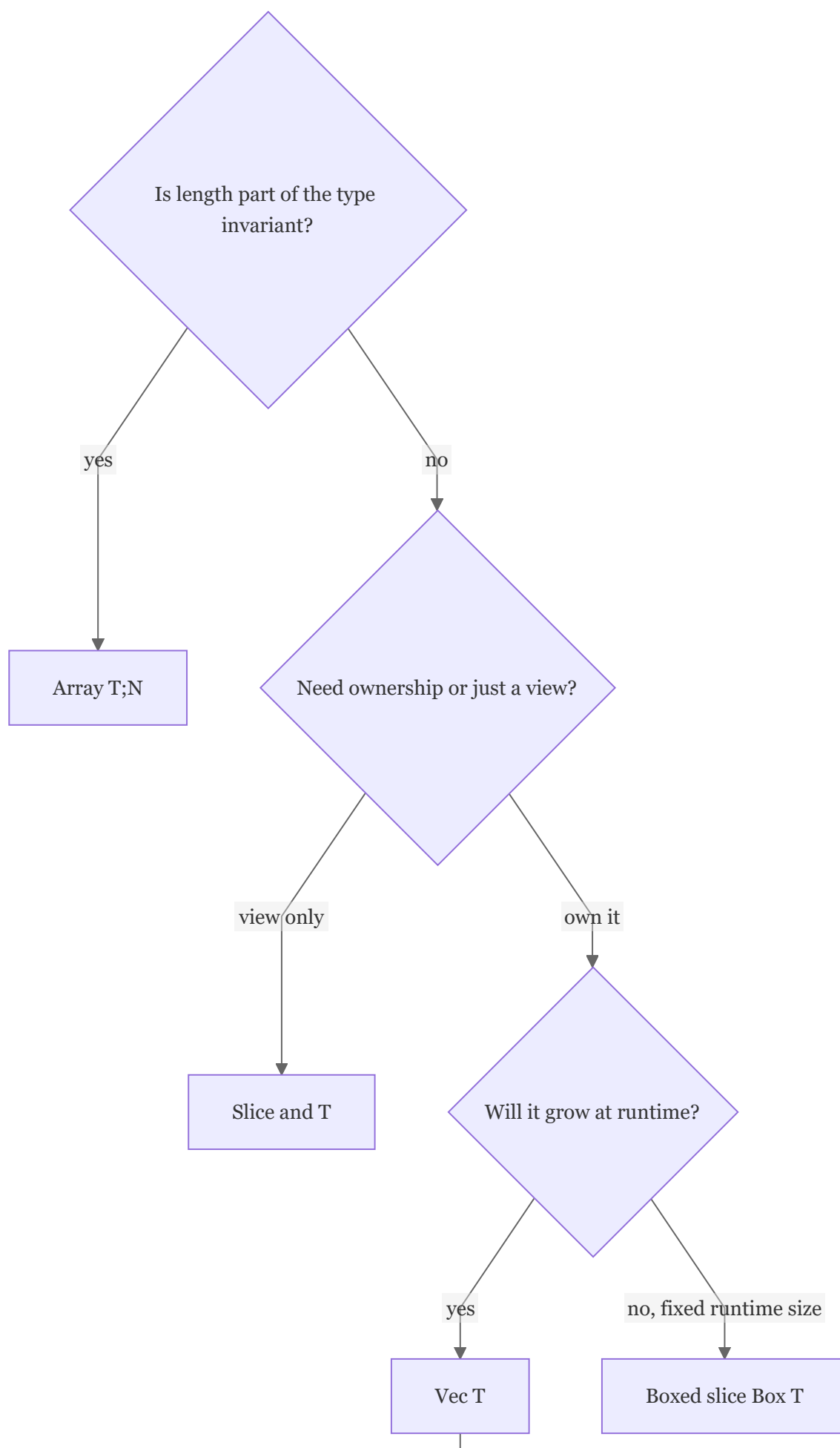
Borrowing rules from Chapter 04, vector ownership hazards from Chapter 06, and allocation-aware API design from Chapter 07 all meet here. Arrays, slices, and vectors are not beginner-only material. They are where many production Rust performance and API decisions become concrete.

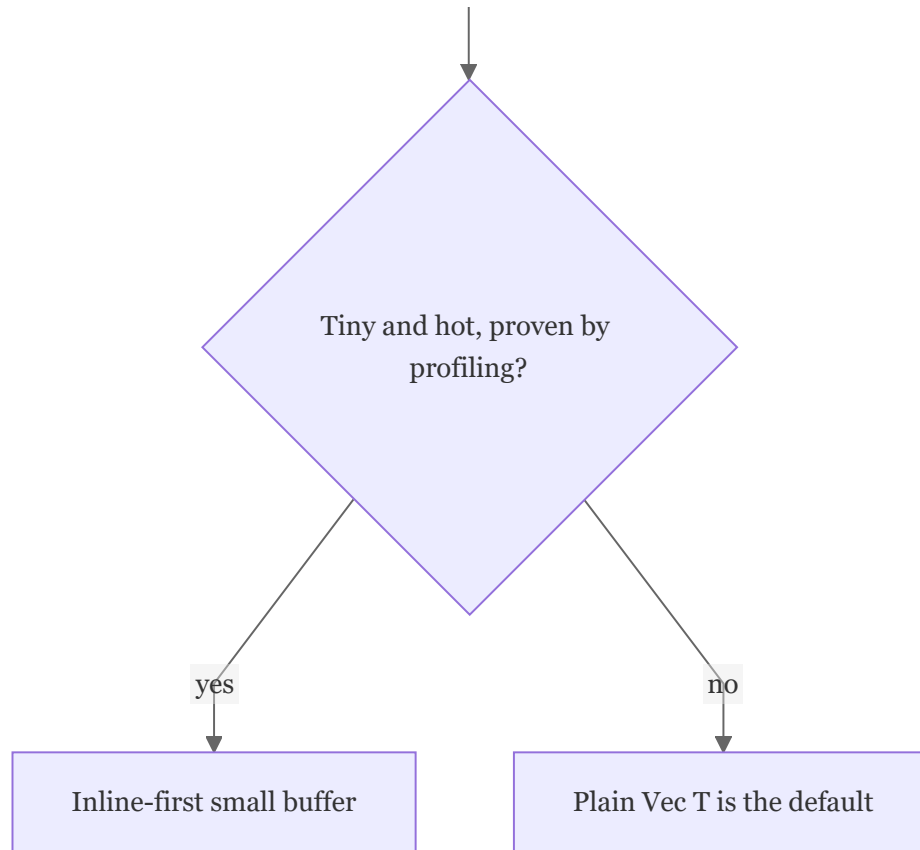
Opening scenario

An ingestion service parses fixed headers, scans metric windows, and builds dynamic retry batches. The business requirement is to choose the narrowest storage shape for each path: fixed arrays for protocol invariants, slices for borrowed contiguous access, and vectors for owned runtime growth.

A practical decision order

Read the diagram top to bottom: each question narrows the choice to one storage shape before you write a single line. The same four questions drive every example in this chapter.





Length in the type means an array; a borrowed view means a slice; runtime ownership means a vector, with inline-first reserved for profiled hot paths.

Mental model

Core concepts

Which tool when?

Fixed-size arrays

An array such as `[u8; 32]` is one value whose size is known at compile time. That makes arrays useful for bounded protocol fields, cryptographic material, coordinates, and other cases where the exact count is semantically real.

The length is in the type. `[u8; 4]` and `[u8; 8]` are different types, not the same type with a stored runtime length.

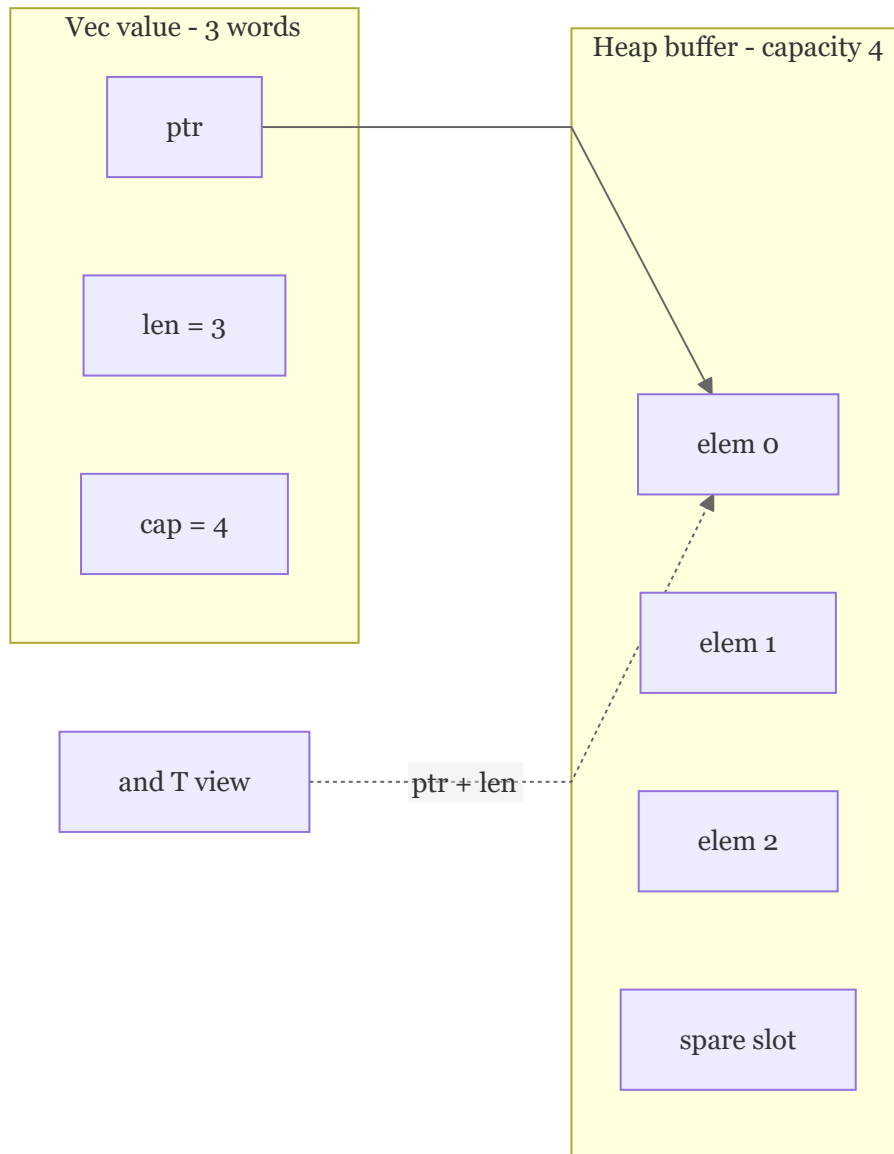
Slices as views

A slice is a borrowed view into contiguous elements. It is not an owner, and it does not decide allocation strategy. That is why slice parameters are usually the right API shape for parsing, scanning, hashing, serialization, and most read-only transforms.

The same function can accept an array, a vector borrow, or a sub-slice. That keeps callers flexible and keeps the algorithm honest about what it actually needs.

How a `Vec<T>` is laid out in memory

The thing to notice is that the vector value itself is tiny and lives wherever you put it (stack, struct, another vector). It is three words: a pointer, a length, and a capacity. The elements live somewhere else, in one contiguous heap allocation that the vector owns.



The owner is three words; the elements sit in one heap allocation. `len` marks how many are live, `cap` marks how many fit, and a slice is just a borrowed pointer-and-length into the same buffer.

Because the owner is so small, moving the vector value moves only those three words. Ownership of the buffer transfers, but the elements are not copied or re-touched at the move. This is why returning a vector from a function is cheap even when it holds millions of elements.

- `length` = how many elements are currently initialized and logically present.
- `capacity` = how many elements can fit before another allocation may be needed.
- `buffer` = contiguous owned storage, usually on the heap.

Capacity, allocation, and growth

Preallocation is one of the simplest meaningful vector optimizations. If you know an upper bound, use `Vec::with_capacity` or reserve space before the hot push loop. That reduces allocation churn and often makes profiles cleaner.

Do not depend on an exact growth factor. Rust does not promise one in the language contract. Treat growth behavior as an implementation detail and validate the workload with measurements, capacity logs, or allocation tracing.

Capacity planning is an optimization, not a license to hard-code folklore. Use it when a bound is real. Skip it when the bound is fake or when the complexity costs more than the saved allocations.

Iterators over arrays, slices, and vectors

Mutation patterns

The recurring rule is familiar by now: if the algorithm only mutates existing elements, slices and `iter_mut()` are excellent. If the algorithm also changes vector shape, end any active borrows first or split the work into phases.

Small-vector optimizations

Some workloads create many tiny vectors whose most common size is just a few elements: tags, short path segments, tiny batch keys, or parser-side scratch buffers. In those cases, an inline-first representation can avoid heap allocation for the common path and spill to the heap only when the collection grows past an inline threshold.

This is an ecosystem pattern, not the default answer. Inline-first containers usually make the parent struct larger, increase move cost, and can inflate stack pressure. Use them only after profiling shows that ordinary vectors are a real problem.

Cache locality and contiguous storage

What changes coming from another language

The three shapes have close analogues in most languages, so the syntax is rarely the hard part. The mental shift is that ownership and resize rules are now part of the type system, which changes how you read an API signature and which mistakes the compiler catches for you.

Production patterns

Pitfalls and tradeoffs

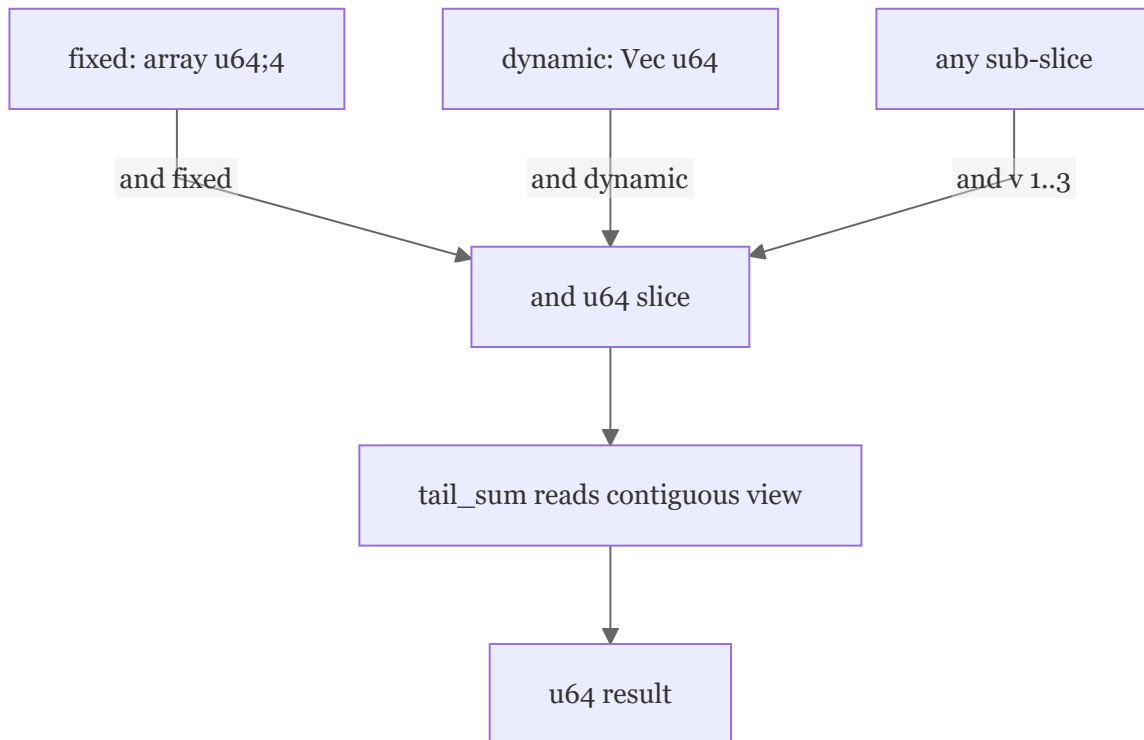
The cheapest-looking signature is not always the cheapest design. A slice-first API often removes needless ownership constraints. A vector-only API often leaks an allocation assumption that the algorithm never needed in the first place.

Examples

Example 1: one slice-first function works for arrays and vectors

The algorithm wants a contiguous read-only view, so the function accepts a slice and callers keep their own storage choices.

What to look at: three different owners all reach the same function through one `&[u64]` parameter. The array, the vector borrow, and a sub-slice converge to the same borrowed view, so no caller is forced to allocate or give up ownership.



One slice parameter accepts an array, a vector borrow, or a sub-slice; the function never learns or cares which owner it came from.

Example 10-1. `slice_first_api.rs` — Slice api

```

fn tail_sum(values: &[u64], take: usize) -> u64 {
    let start = values.len().saturating_sub(take);
    values[start..].iter().copied().sum()
}

fn main() {
    let fixed = [3_u64, 5, 8, 13];
    let dynamic = vec![1_u64, 2, 3, 4, 5, 6];

    println!("fixed tail = {}", tail_sum(&fixed, 2));
    println!("dynamic tail = {}", tail_sum(&dynamic, 3));
}

```

OUTPUT

```
fixed tail = 21
```

```
dynamic tail = 15
```

Quick check: run the baseline, then change the array contents or the take counts. The useful lesson is not the arithmetic. It is that the API shape stays calm because the function borrows a slice rather than forcing a vector.

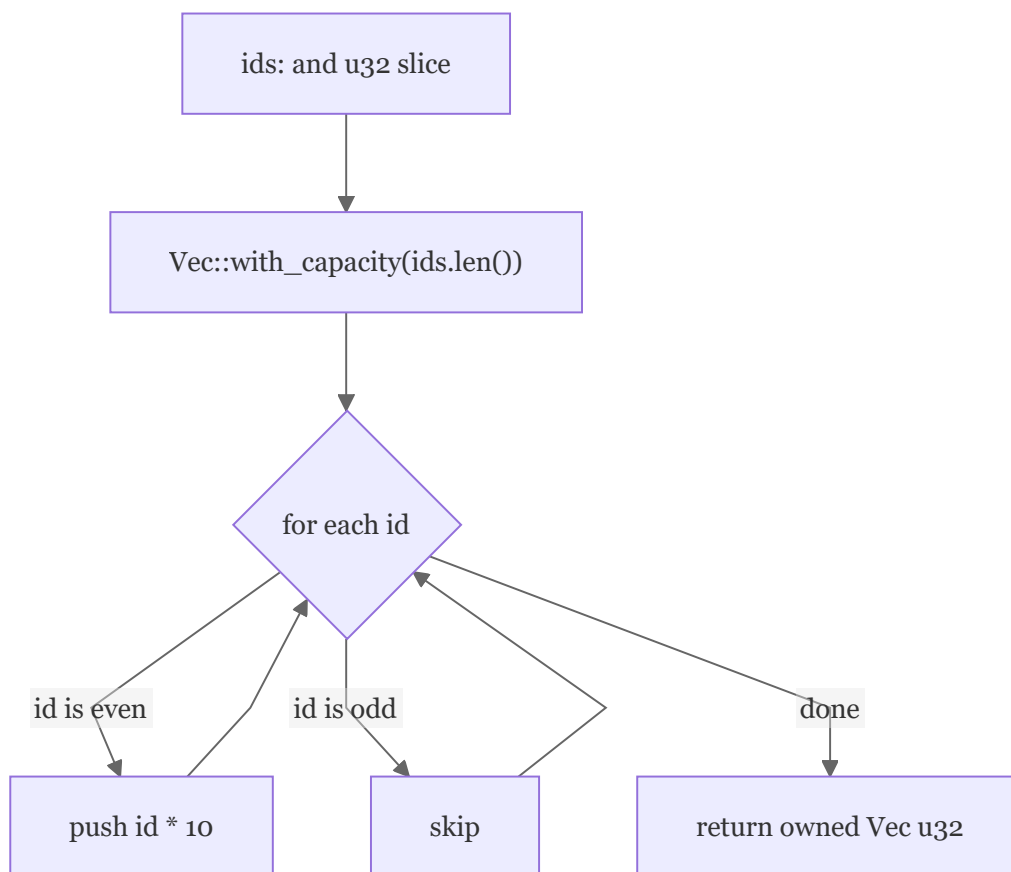
A slice says “contiguous borrowed data.” That is exactly the contract this function needs.

Arrays, vectors, and sub-slices all call the same function without extra allocation or a narrower public signature.

Example 2: preallocate when a trustworthy upper bound exists

The batch size is bounded by the input slice length, so capacity planning is simple and justified.

What to look at: the capacity is reserved once from a real bound (the result can never be longer than the input), and then the loop only filters and pushes. The diagram is the pipeline; the listing fills in the arithmetic.



Reserve once from a bound that cannot be exceeded, then filter-and-push without further reallocation; the helper hands back an owned vector the caller keeps.

Example 10-2. `vec_capacity_and_growth.rs` — Vectors capacity

```
fn collect_even_scaled(ids: &[u32]) -> Vec<u32> {
    let mut out = Vec::with_capacity(ids.len());

    for &id in ids {
        if id % 2 == 0 {
```

```

        out.push(id * 10);
    }
}

out
}

fn main() {
    let ids = [10_u32, 11, 12, 13, 14];
    let mut out = collect_even_scaled(&ids);

    println!("len = {}", out.len());
    println!("can fit two more = {}", out.len() + 2 <= out.capacity());

    out.extend([200, 220]);
    println!("last = {}", out.last().copied().unwrap());
}

```

OUTPUT

```

len = 3
can fit two more = true
last = 220

```

Quick check: keep the upper bound tied to the input length, then change the IDs or appended values and rerun. The point is that preallocation follows a real bound, not a guessed folklore capacity.

The function returns an owned vector because the batch must survive after the helper returns.

Capacity is chosen from a real bound: the result cannot exceed the number of input elements.

You pay a small planning step up front to reduce later allocation churn in the push loop.

The repository also includes standalone Rust source under `examples/ch10_arrays_slices_and_vectors/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

- Use arrays when size is part of the invariant, slices when the algorithm needs a view, and vectors when the code needs owned growth.
- A `Vec<T>` is an owner of contiguous storage with separate logical length and allocation capacity.
- Slice-first APIs are usually the most flexible and honest public contract for contiguous read or write algorithms.
- Preallocation helps when the bound is real. Exact growth strategy is not something production code should assume.
- Contiguous storage buys locality for scans and batch work, but middle insertions, stable-address needs, and inline-first tricks all carry tradeoffs.

CHAPTER 11 · HASH MAPS AND SETS

Hash Maps and Sets

HashMap and HashSet fundamentals, borrowed lookups, Entry API, BTreeMap tradeoffs, stable ordering, and hasher choices

Hash maps and sets sit on critical lookup paths. This chapter covers key ownership, borrowed lookup, entry-based updates, deterministic output, and hashing policy for production data access.

Builds on Chapters 06, 07, and 10

Chapter 06 explained durable handles versus temporary borrows in collections. Chapter 07 separated moves, copies, and clones. Chapter 10 covered contiguous storage and allocation-aware API shape. This chapter applies those same design instincts to hashed and ordered associative collections.

Opening scenario

An ingress service maintains route counters, active feature flags, in-memory indexes, and deterministic operator-facing configuration dumps. The business requirement is explicit key ownership and lookup policy: own normalized keys in collections, borrow for read paths, update with one table access, and choose ordered structures when output order is contractual.

A reliable review order

1. Does this call path own keys and values, or only borrow them temporarily?
2. Is the collection unordered membership, unordered key-value lookup, or ordered key traversal?
3. Is the hot update path doing one lookup or several?
4. Does the boundary need denial-of-service resilience more than raw hash speed?

Mental model

Core concepts

HashMap fundamentals

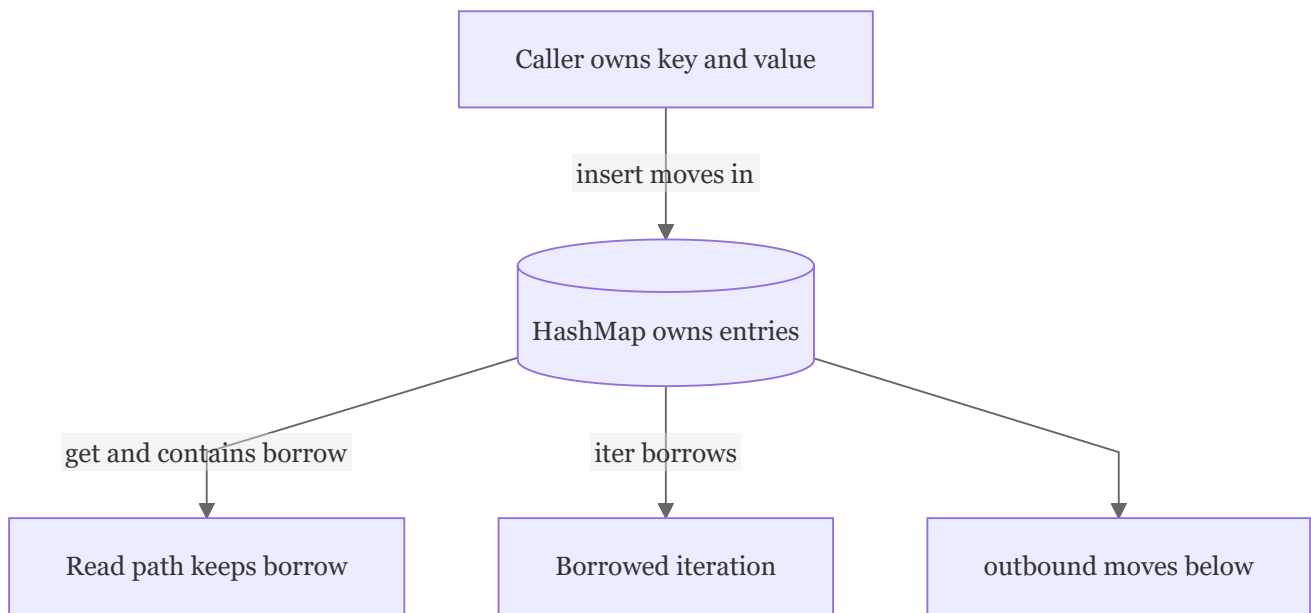
Use HashMap when you want expected fast key lookup and update, and key order is not part of the external contract. Use HashSet when the value is really membership and deduplication rather than key-value association.

Sets are not a secondary topic. In production code they often replace improvised boolean maps, duplicate filtering passes, or repeated linear searches over vectors. If the question is “have I seen this key already?”, a set is usually the right structure.

Ownership of keys and values

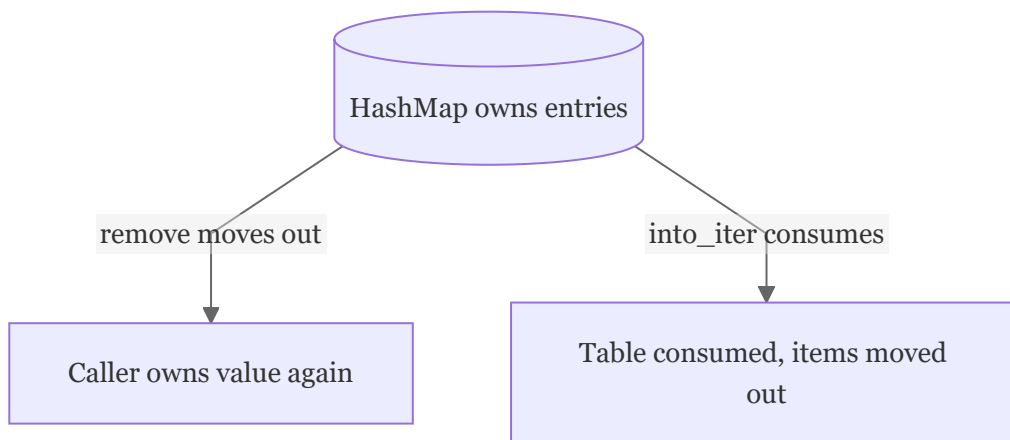
The whole API divides cleanly along one line: which calls move data across the table boundary and which only borrow across it. Insert is the one inbound move; remove and into_iter are the outbound moves;

everything on the read path borrows and leaves the table owning what it owned before. Hold that picture and the rest of the chapter is mostly choosing the right edge for each call site.



Inbound: insert moves owned data in; read paths borrow and leave ownership where it was.

The same table also has two paths that move data back out:



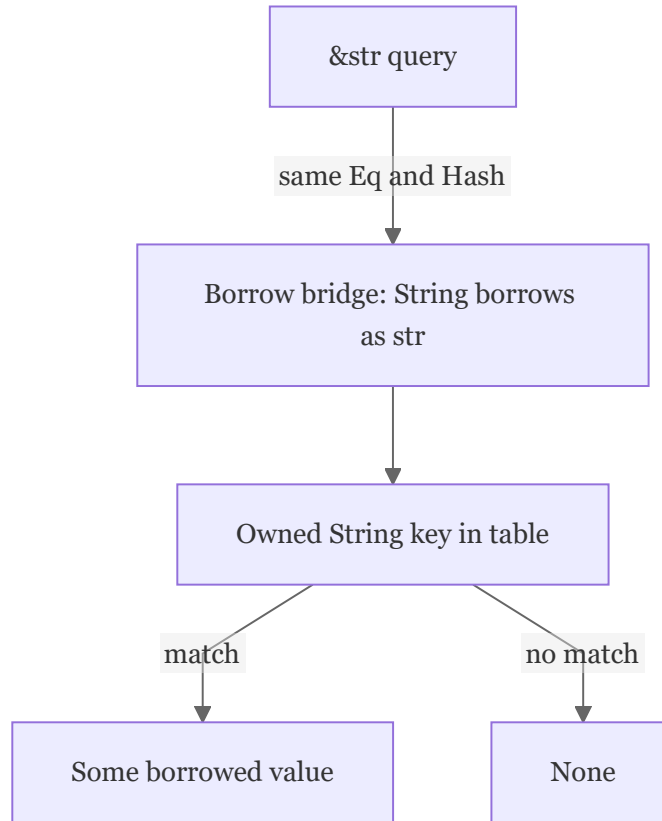
Outbound: remove moves one value back out; into_iter consumes the table and moves every item out.

- insert moves owned keys and values into the collection.
- get and contains usually borrow.
- remove moves the removed value back out.
- iter borrows; into_iter consumes the collection and moves items out.

That ownership story matters operationally. If your parser already produced an owned route key, the map can take it directly. If your request path only has a borrowed `&str`, you can often look up by borrow without allocating, but storing a new entry still requires some owned key to live in the table.

Borrowed lookups

The reason a table of owned String keys accepts a borrowed `&str` query is a small trait relationship rather than magic: the lookup methods are generic over a borrowed form of the key, and String can be borrowed as `str` with matching equality and hashing. The diagram shows that bridge; the code below is what it looks like at the call site.



The query type and the stored key share equality and hashing through a borrow relationship, so no owned key is built to look one up.

One of Rust's best collection features is borrowed lookup. A `HashMap` can be queried by `&str`. A `HashSet` can test membership with `&str`.

This is usually what you want on the read path. Store owned strings once. Query them by borrowed slices. That avoids allocating just to answer a membership or lookup question. For standard string keys, the equality and hashing behavior line up the way you expect.

Entry API

The Entry API is the right tool when one key path does read-modify-write work: counters, last-seen timestamps, aggregation, or “initialize if absent, then mutate.” It removes duplicate table lookups and makes the intent obvious.

There is one nuance worth keeping explicit: with ordinary stable std Entry APIs, the entry key itself is owned. If your key path is a borrowed `&str`, `entry(route.to_owned())` still allocates the owned key. That can still be the right design, but do not pretend the allocation disappeared. What disappeared is the second hash-table lookup.

The practical rule is simple: Entry is perfect when the caller already owns the key. Borrowed lookup is perfect when you only need to read. Treat those as two different fast paths.

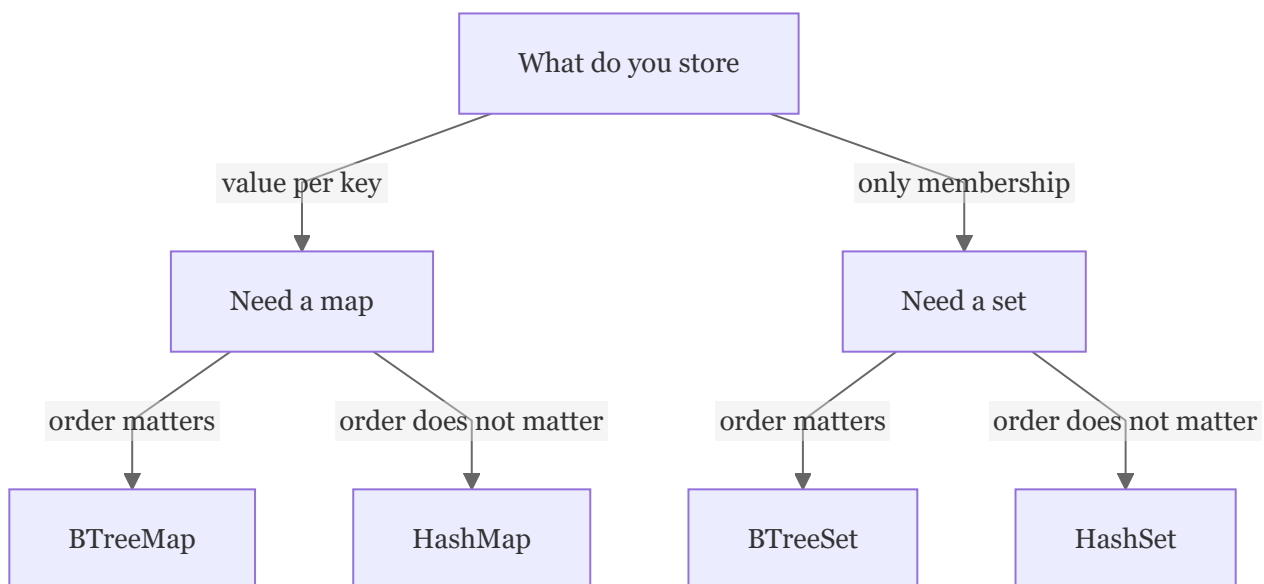
Custom hashers

A standard library hash map has a third type parameter for the build hasher. In ordinary code you rarely need to name it. The default choice uses randomized state and is a strong default for public or untrusted inputs because collision behavior is part of your operational risk surface.

Faster non-randomized or domain-specific hashers can be a good fit for trusted internal workloads with measured hot paths, especially when keys are small and frequent. But that is an engineering decision, not a reflex. Benchmark first. Document the trust boundary. Keep security posture in view.

BTreeMap vs HashMap

The choice between the four collections is two yes-or-no questions, not a performance debate. First: do you need a value per key, or only membership? That picks map versus set. Second: does anything downstream depend on iteration order? That picks the hashed variant versus the ordered B-tree variant. Answer those two and the type falls out.



Two questions decide the collection: value-per-key versus membership, then ordered iteration versus not.

- Expected fast lookup, insert, and membership checks.
- No stable iteration order guarantee.
- Good default for caches, counters, indexes, and dedup sets.
- Ordered keys and deterministic iteration.
- Natural fit for sorted output and range-oriented logic.
- Often simpler than “use HashMap, then sort later” when ordering is always required.

Stable ordering considerations

If operators, tests, logs, snapshots, or downstream systems care about output order, plain hash iteration is the wrong boundary contract. Choose an ordered structure or sort at the render edge.

A common production pattern is this: keep the hot mutable index in a `HashMap`, then project into a `BTreeMap` or a sorted vector only when you emit deterministic output. If ordering is needed everywhere, start ordered.

Hashing performance and denial-of-service tradeoffs

- Hash cost is real work. Key size, equality cost, and cardinality all matter in hot paths.
- Collision behavior is also real work. Public endpoints should treat adversarial keys as part of the design.
- Ordered trees may win operationally when they remove a later sort pass or simplify deterministic behavior.
- Capacity planning still matters here too. Use `with_capacity` when the bound is real, not guessed.
- Benchmark workload shapes, not only microbenchmarks on one synthetic key distribution.

How this lands by background

Every one of these languages gives you a dictionary and a set, so the data structure is not the new part. What changes in Rust is who owns the key after you insert it, and the fact that reading does not have to build a key at all. The cards below name the one mental-model shift each background tends to trip over, not the API spelling, which you can read off the standard library in a minute.

Production patterns

Pitfalls and tradeoffs

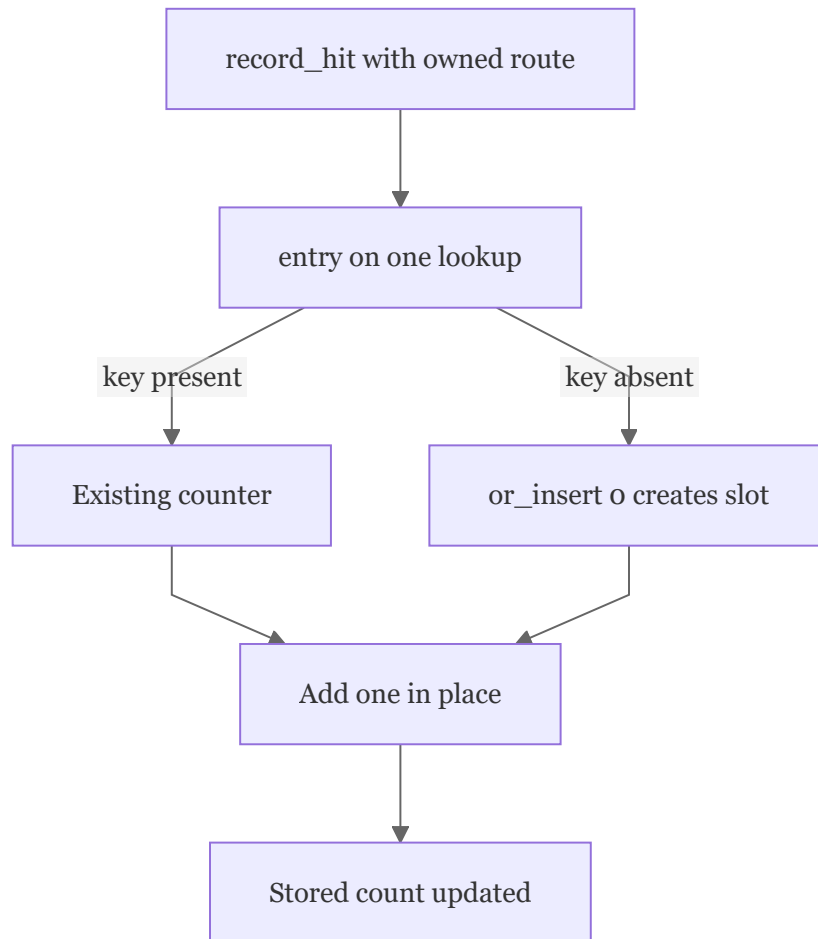
The collection choice is not only about asymptotic lookup speed. In real systems it also shapes ownership, determinism, test stability, memory pressure, and how resilient the service is to hostile key patterns.

Examples

Example 1: owned key handoff plus Entry API

When the caller already owns the key, `Entry` is the clean update path: one table lookup, direct mutation, and no extra duplicate logic.

What to look at: `entry(route)` is a single trip into the table. If the slot is missing it is created with `or_insert(o)`; either way you get a mutable reference and add one in place. There is no separate `contains_key` branch and no second lookup. Follow the two paths in the diagram, then read the same fork in `record_hit`.



Both the present and absent cases converge on one mutable reference, so the update is a single table access.

Example 11-1. entry_api_owned_keys.rs — Entry api

```

use std::collections::HashMap;

fn record_hit(counts: &mut HashMap<String, usize>, route: String) {
    let slot = counts.entry(route).or_insert(0);
    *slot += 1;
}

fn main() {
    let mut counts = HashMap::with_capacity(4);

    record_hit(&mut counts, String::from("api"));
    record_hit(&mut counts, String::from("api"));
    record_hit(&mut counts, String::from("billing"));

    println!("api = {}", counts.get("api").copied().unwrap_or(0));
    println!("billing = {}", counts.get("billing").copied().unwrap_or(0));
    println!("routes = {}", counts.len());
}
  
```

OUTPUT

```

api = 2
billing = 1
  
```

```
routes = 2
```

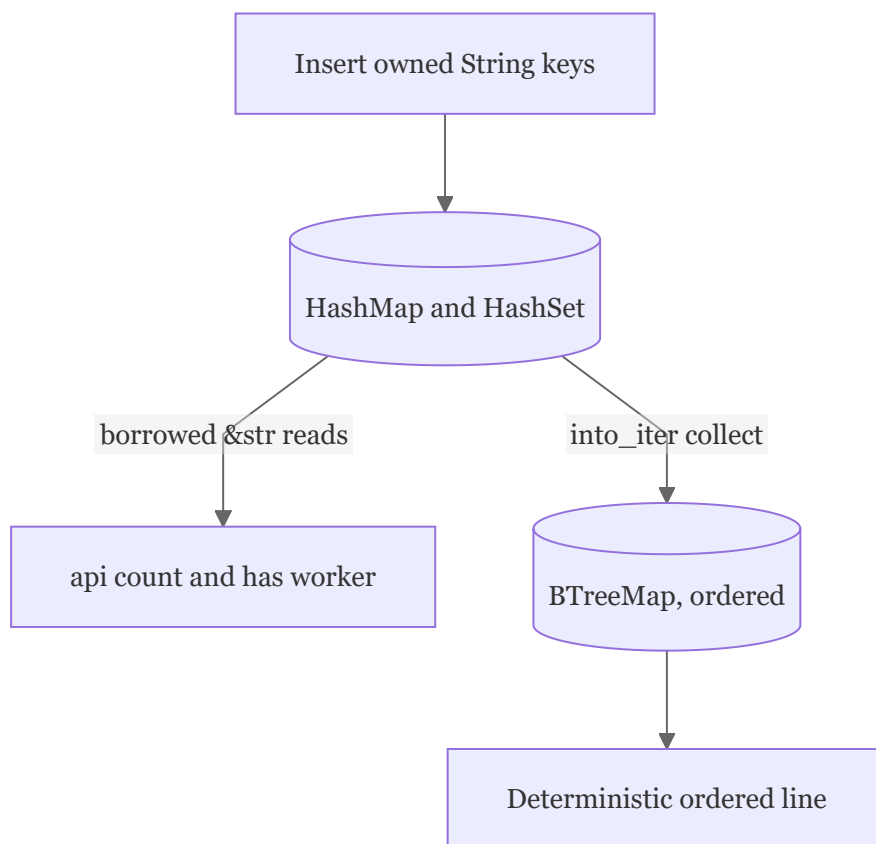
The route key is moved into the map. The returned entry reference mutates the stored counter in place.

No duplicated `contains_key` branch, no second hash-table lookup, and no ambiguity about who owns the key.

Example 2: borrowed lookup for reads, ordered projection for output

Store owned strings once. Query them by borrow. Project into an ordered structure only when stable render order matters.

What to look at: this is a pipeline with three distinct stages. The reads in the middle, `get("api")` and `contains("worker")`, borrow the owned-string keys without allocating. Only at the render edge does `into_iter().collect()` move the entries into a `BTreeMap` so output is deterministic. The diagram is that pipeline; the three printed lines are its three outputs.



Build with owned keys, read by borrow, and only convert to an ordered structure at the moment output order has to be stable.

Example 11-2. `borrowed_lookup_and_ordered_output.rs` — *Lookup ordered*

```
use std::collections::{BTreeMap, HashMap, HashSet};

fn main() {
    let mut active = HashSet::new();
    active.insert(String::from("api"));
    active.insert(String::from("worker"));

    let mut counts = HashMap::new();
```

```

counts.insert(String::from("worker"), 3);
counts.insert(String::from("api"), 2);
counts.insert(String::from("billing"), 1);

let api_count = counts.get("api").copied().unwrap_or(0);
let has_worker = active.contains("worker");

let ordered: BTreeMap<_, _> = counts.into_iter().collect();
let ordered_text = ordered
    .iter()
    .map(|(name, count)| format!("{}", name, count))
    .collect::<Vec<_>>()
    .join(",");

println!("api count = {}", api_count);
println!("has worker = {}", has_worker);
println!("ordered = {}", ordered_text);
}

```

OUTPUT

```

api count = 2
has worker = true
ordered = api=2,billing=1,worker=3

```

`get("api")` and `contains("worker")` do not need fresh owned strings.

The set answers membership directly. There is no need for a parallel boolean map when membership is the only question.

Converting to a `BTreeMap` gives deterministic output without pretending hash iteration order was stable all along.

The repository also includes standalone Rust source under `examples/ch11_hash_maps_and_sets/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Next chapter

Hashed and ordered associative collections cover keyed access. Chapter 12 turns to matrices and multidimensional data, where the question shifts from who owns a key to how a logical grid maps onto one contiguous block of memory. The same ownership and borrowing instincts carry over; the new concern is layout, indexing, and the cost of the access pattern.

Summary

- Maps and sets are ownership boundaries. Insert moves owned data in; lookups usually borrow.
- Borrowed lookup is one of the strongest Rust-native patterns for string-keyed indexes and membership sets.
- The Entry API removes duplicate table lookups on update paths, but ordinary stable entry still needs an owned key.
- `HashMap` and `HashSet` are about expected fast lookup; `BTreeMap` and `BTreeSet` are about ordered traversal and deterministic behavior.

- Hasher choice is a production tradeoff involving speed, key shape, and denial-of-service posture, not just a benchmark number.

CHAPTER 12 · MATRICES AND MULTIDIMENSIONAL DATA

Matrices and Multidimensional Data

Dense flat storage, row-major vs column-major layout, views, const generics, sparse formats, and HPC interop boundaries

Matrix and multidimensional workloads need explicit layout, shape, ownership, and traversal rules. This chapter covers dense and sparse representations used for numeric pipelines and accelerator handoff.

Builds on Chapters 08, 10, and 11

Chapter 08 explained unsafe Rust and FFI proof obligations. Chapter 10 covered contiguous storage and slice-first APIs. Chapter 11 covered workload-aware data-structure choice. This chapter applies those same instincts to multidimensional numeric and domain data, where layout becomes part of the performance, correctness, and interop contract.

Opening scenario

Picture a data-processing service that handles three shapes of numeric data at once: dense image grids it convolves with a kernel, a recommendation score matrix it slices into per-user windows, and a large mostly-zero feature matrix it hands to an accelerator. None of these is well served by the same representation, and the expensive failures here are not arithmetic bugs. They are a column-oriented BLAS call fed a row-major buffer, a view that copied a megabyte it should have borrowed, or a sparse matrix stored in the format that was cheap to build and ruinous to multiply. The job of this chapter is to make layout, ownership, and traversal explicit decisions you take on purpose rather than defaults you discover under a profiler.

A reliable design order

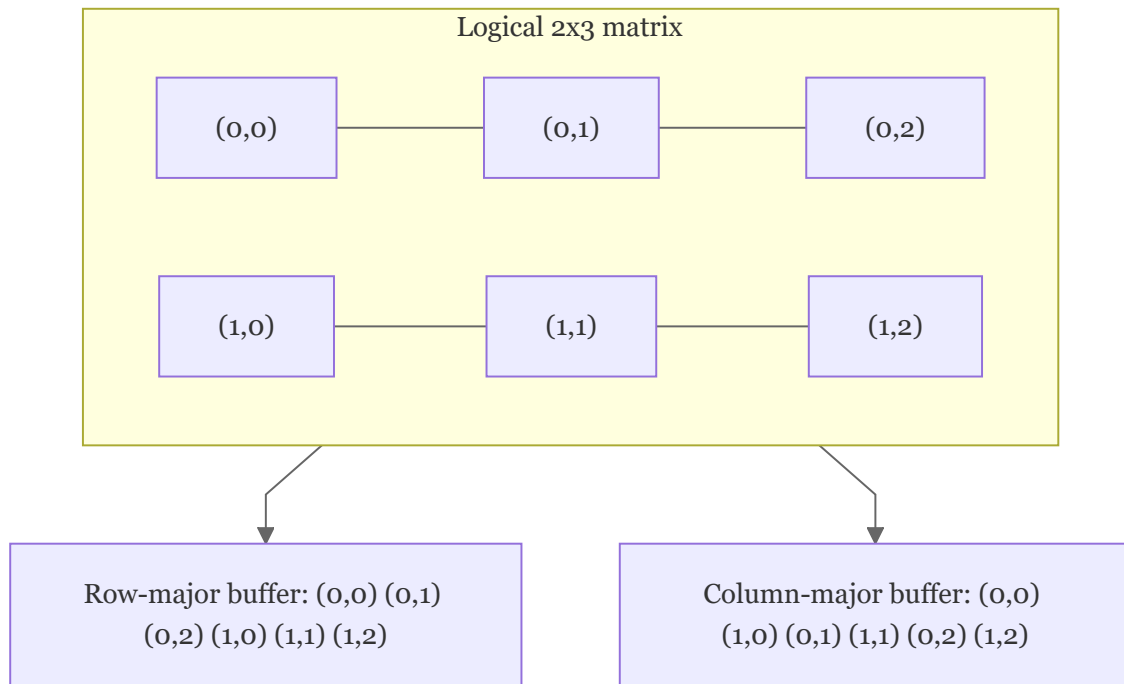
1. Decide whether the data is dense, jagged, or sparse.
2. Choose the physical layout before you optimize the arithmetic.
3. Choose owned storage for long-lived buffers and borrowed views for windows and tiles.
4. Make foreign-library layout assumptions explicit at the boundary, not in a comment after the bug.

Mental model

Core concepts

Row-major and column-major representations

What to look at: both layouts store the same six logical cells in the same six bytes of memory; the only difference is the order. Row-major lays the first row down first, so walking a row is unit-stride and walking a column jumps by the row width. Column-major flips that. The diagram below shows the same 2x3 matrix under both rules, so you can see why one loop order is contiguous and the other is not.



One logical matrix, two linearizations. Row-major keeps each row contiguous; column-major keeps each column contiguous.

The important operational consequence is not the formula itself. It is which loop order is unit-stride, which access pattern is cache-friendly, and which foreign library can consume the buffer without conversion.

Nested vectors and their costs

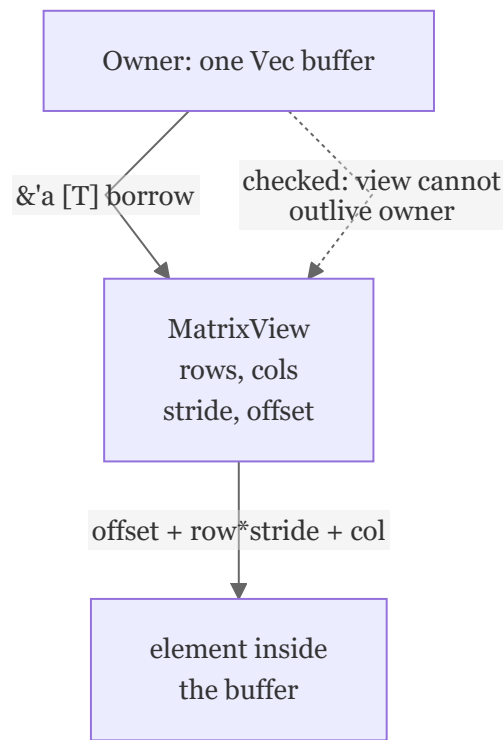
The difference between a flat Vec and a Vec of Vecs is not syntax, it is the heap shape. A flat buffer is one allocation the CPU can stream through. A nested buffer is an outer array of pointers, each one leading to a separately allocated row that can sit anywhere in memory, so every row access pays for an extra hop and the locality is gone before any arithmetic begins.

- Each row is a separate allocation and a separate owner.
- Rows need not be adjacent in memory.
- Jagged shapes are easy, but dense numeric kernels pay with pointer chasing.
- FFI handoff becomes awkward because there is no single buffer to pass.

That does not make nested vectors wrong. They are a good fit when row lengths truly differ or when the domain is naturally ragged. They are just not the calm default for dense matrices.

Matrix views and slices

What to look at: a view does not allocate. It points back into the owner's buffer and carries the metadata needed to skip past the rows it does not care about. The lifetime parameter is the part that earns its keep: the borrow checker refuses to let the view be used after the buffer is dropped, so a dangling submatrix is a compile error rather than a runtime fault.



A view is a borrow plus an index rule. It reads the owner's bytes through stride and offset; the lifetime keeps it from outliving the buffer.

A view usually borrows a slice and carries metadata: rows, cols, stride, and maybe an offset. That lets you describe submatrices, tiles, and windows without copying data into a second owner.

The view does not own the buffer. It interprets borrowed storage under an explicit layout contract.

SIMD-friendly layouts

Whether you use compiler auto-vectorization, explicit intrinsics, or an ecosystem SIMD abstraction, the layout question arrives first. Syntax cannot fix a bad stride pattern.

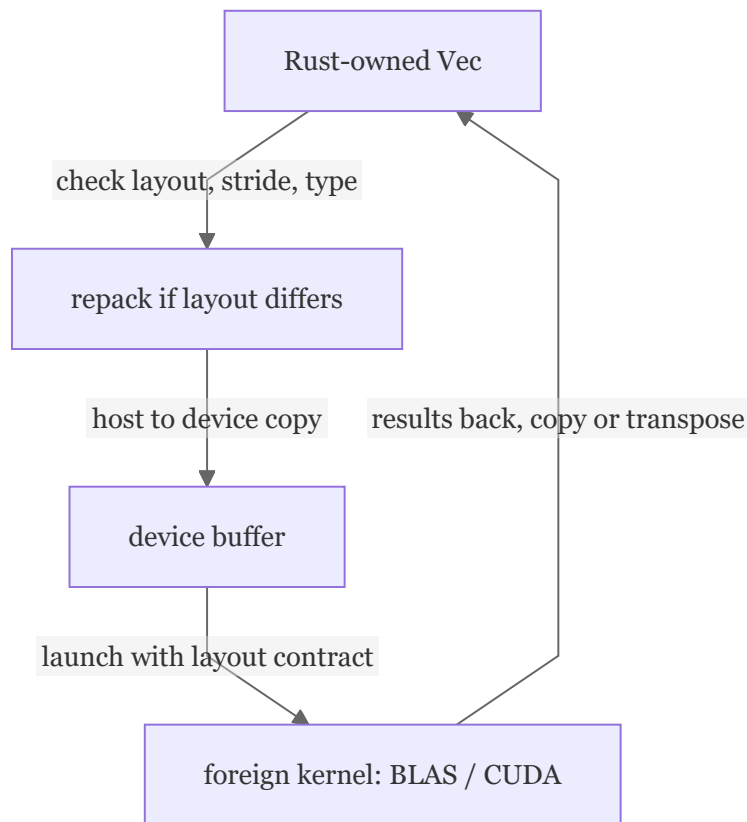
Sparse matrices

When most cells are zero, you stop storing the grid and start storing only the nonzeros plus the structure needed to find them. COO is the easy ingestion format, a bag of (row, col, value) triples; CSR and CSC are the compressed formats that traverse fast in one direction. The usual lifecycle is to build in COO at the edges, then compress into CSR or CSC for the traversal your kernels actually perform.

A useful rule is to separate build format from execution format. The easiest format to ingest is not always the cheapest format to multiply or traverse.

Interoperating with BLAS, LAPACK, CUDA, and HPC libraries

What to look at: the cost at a numeric boundary is rarely the math. It is the handoff. Your Rust buffer may be the wrong layout, the wrong leading dimension, or in host memory when the kernel wants device memory, and every mismatch turns into a repack or a transfer before any arithmetic runs. The diagram lays out the stages so you can see where the hidden copies hide and measure them on purpose.



Each arrow is a potential copy. The arithmetic is one box; the repacks and transfers around it often dominate the runtime.

Be careful with claims of zero-copy interop. A foreign kernel may still require a different layout, leading dimension, alignment rule, memory domain, or device residency than your Rust-side buffer currently has.

Interop boundary checklist

This checklist is deliberately boring. Boring is good here. Most expensive matrix interop bugs come from unstated layout or ownership assumptions, not from exotic arithmetic.

How this lands by background

Almost nobody meets Rust matrices as a blank slate. You arrive with a mental model from another ecosystem, and the useful question is which part of that model carries over and which part will quietly mislead you. With matrices the shift is rarely about API names; it is about who owns the buffer, where the layout contract lives, and how much of the indexing and lifetime work the language does for you versus asks you to spell out.

Production patterns

Pitfalls and tradeoffs

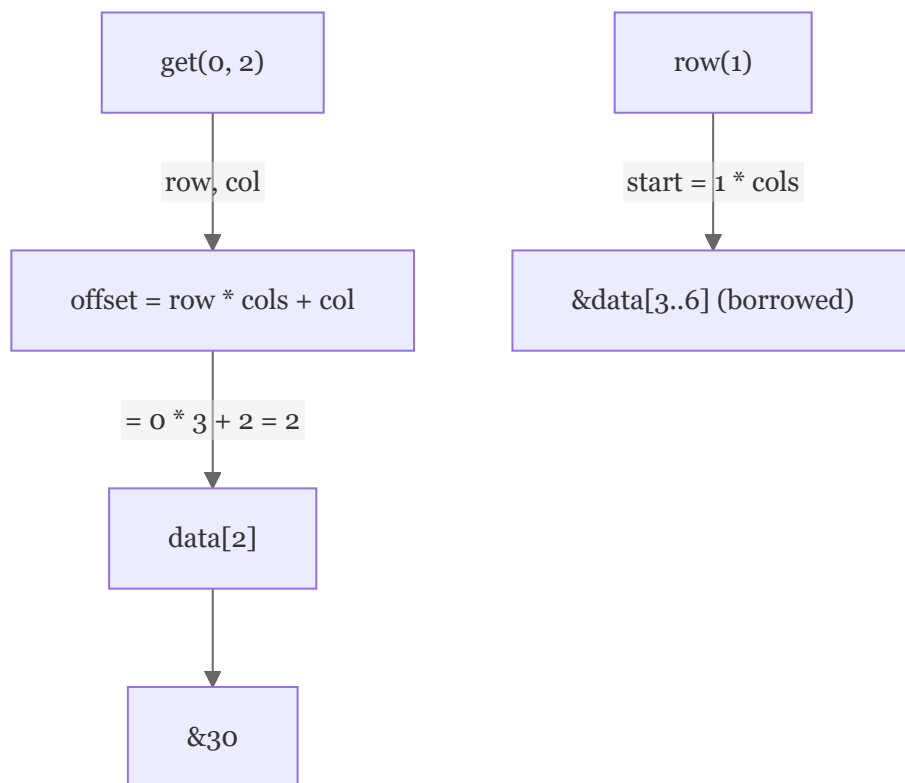
Dense numeric code often looks like “just math,” but the hard bugs and hard performance losses are usually layout bugs. Treat layout as a first-class API decision.

Examples

Example 1: row-major Matrix<T> over Vec<T>

One contiguous owner, explicit offset calculation, and row views derived from the flat buffer.

What to look at: there is one buffer and one place where a (row, col) becomes a linear index, the offset method computing $\text{row} * \text{cols} + \text{col}$. Every accessor (get, set, row) routes through it, and row hands back a borrowed slice rather than a copy. Trace one cell through the diagram, then read the same path in the code.



One offset rule feeds every accessor. get returns a borrowed element; row returns a borrowed slice of the same buffer.

Example 12-1. row_major_matrix.rs — Major dense

```

#[derive(Debug)]
struct Matrix<T> {
    rows: usize,
    cols: usize,
    data: Vec<T>,
}

impl<T: Clone> Matrix<T> {
    fn from_elem(rows: usize, cols: usize, value: T) -> Self {
        Self {
            rows,
            cols,
            data: vec![value; rows * cols],
        }
    }
}

```

```

impl<T> Matrix<T> {
    fn offset(&self, row: usize, col: usize) -> usize {
        row * self.cols + col
    }

    fn get(&self, row: usize, col: usize) -> Option<T> {
        if row < self.rows && col < self.cols {
            Some(&self.data[self.offset(row, col)])
        } else {
            None
        }
    }

    fn set(&mut self, row: usize, col: usize, value: T) {
        let index = self.offset(row, col);
        self.data[index] = value;
    }

    fn row(&self, row: usize) -> Option<&[T]> {
        if row < self.rows {
            let start = row * self.cols;
            Some(&self.data[start..start + self.cols])
        } else {
            None
        }
    }
}

fn main() {
    let mut m = Matrix::from_elem(2, 3, 0_i32);
    m.set(0, 0, 10);
    m.set(0, 1, 20);
    m.set(0, 2, 30);
    m.set(1, 0, 4);
    m.set(1, 1, 5);
    m.set(1, 2, 6);

    let row1_sum: i32 = m.row(1).unwrap().iter().copied().sum();

    println!("last in row0 = {}", m.get(0, 2).copied().unwrap());
    println!("row1 sum = {}", row1_sum);
}

```

OUTPUT

```

last in row0 = 30
row1 sum = 15

```

Quick check: change the inserted values or the matrix dimensions and keep the offset logic row-major. The exercise is not the printing. It is making the storage contract obvious in code.

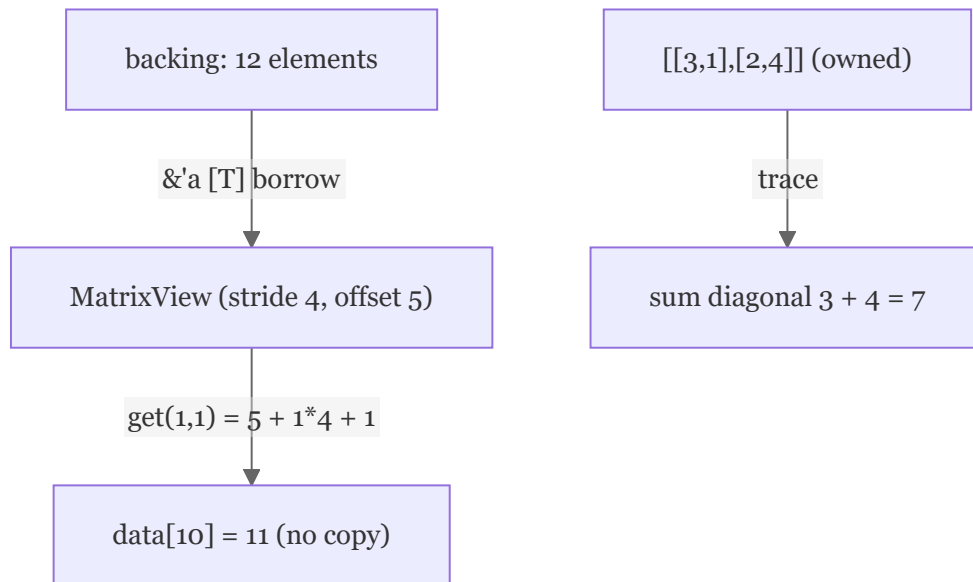
One allocation keeps the matrix contiguous. Row slicing is just range arithmetic over one buffer.

Dense grids, image kernels, tensor-like feature blocks, and FFI handoff all benefit from one explicit owner plus a documented layout policy.

Example 2: borrowed matrix view plus const-generic fixed kernel

A view borrows without copying, while a fixed-size compile-time kernel keeps shape in the type.

What to look at: two different ways shape is carried. The `MatrixView` borrows a 12-element backing buffer and uses runtime stride and offset to address a 2x2 window without copying, so `get(1, 1)` resolves to index $5 + 1 * 4 + 1 = 10$, the value 11. The `trace` function instead takes the size as a const-generic `N` so the shape is fixed at compile time. Follow both paths in the diagram before reading the code.



Left path: a borrowed view addresses a window by runtime stride and offset. Right path: a const-generic kernel carries `N` in the type.

Example 12-2. `matrix_views_and_const_generics.rs` — Const generics

```

struct MatrixView<'a, T> {
    data: &'a [T],
    rows: usize,
    cols: usize,
    stride: usize,
    offset: usize,
}

impl<'a, T> MatrixView<'a, T> {
    fn get(&self, row: usize, col: usize) -> Option<&T> {
        if row < self.rows && col < self.cols {
            let index = self.offset + row * self.stride + col;
            self.data.get(index)
        } else {
            None
        }
    }
}

fn trace<const N: usize>(matrix: [[i32; N]; N]) -> i32 {
    let mut sum = 0;
    let mut i = 0;

    while i < N {

```

```

        sum += matrix[i][i];
        i += 1;
    }

    sum
}

fn main() {
    let backing = vec![
        1_i32, 2, 3, 4,
        5, 6, 7, 8,
        9, 10, 11, 12,
    ];

    let view = MatrixView {
        data: &backing,
        rows: 2,
        cols: 2,
        stride: 4,
        offset: 5,
    };

    let fixed = [[3_i32, 1], [2, 4]];

    println!("view corner = {}", view.get(1, 1).copied().unwrap());
    println!("fixed trace = {}", trace(fixed));
}

```

OUTPUT

```

view corner = 11
fixed trace = 7

```

Quick check: change the backing data or the fixed matrix values, then rerun. The useful distinction is that the view borrows storage while the fixed kernel carries shape information at compile time.

The submatrix window is metadata plus a borrowed slice, not a copied dense owner.

The trace helper knows the square size at compile time and works for any fixed `N`.

View metadata such as stride and offset is exactly the kind of information foreign numeric APIs often need.

The repository also includes standalone Rust source under `examples/ch12_matrices_and_multidimensional_data/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

- A matrix in Rust is usually explicit storage plus explicit indexing policy, not a built-in abstraction.
- Row-major and column-major layouts change address calculation, traversal cost, and interop correctness.
- Dense matrices usually want flat `Vec<T>` storage; nested vectors are better for ragged data than for dense numeric kernels.

- Const generics help when shape is part of the invariant. Borrowed views help when you need windows without copying.
- SIMD, sparse formats, and foreign-library interop all begin with layout discipline rather than clever syntax.

CHAPTER 13 · ARENA ALLOCATION AND REGION-BASED MEMORY

Arena Allocation and Region-Based Memory

Bump allocators, arena-backed ASTs, region lifetimes, generational handles, slabs, and locality tradeoffs

Arena allocation supports workloads that create many related values under one lifecycle. This chapter uses region ownership and stable handles to reduce allocation overhead and simplify teardown.

Builds on Chapters 04, 06, 09, and 10

Borrowing rules from Chapter 04, handle design from Chapter 06, smart-pointer tradeoffs from Chapter 09, and contiguous-storage reasoning from Chapter 10 all converge here. Arenas are easiest to understand once you see them as lifetime grouping plus locality, not as a mysterious allocator trick.

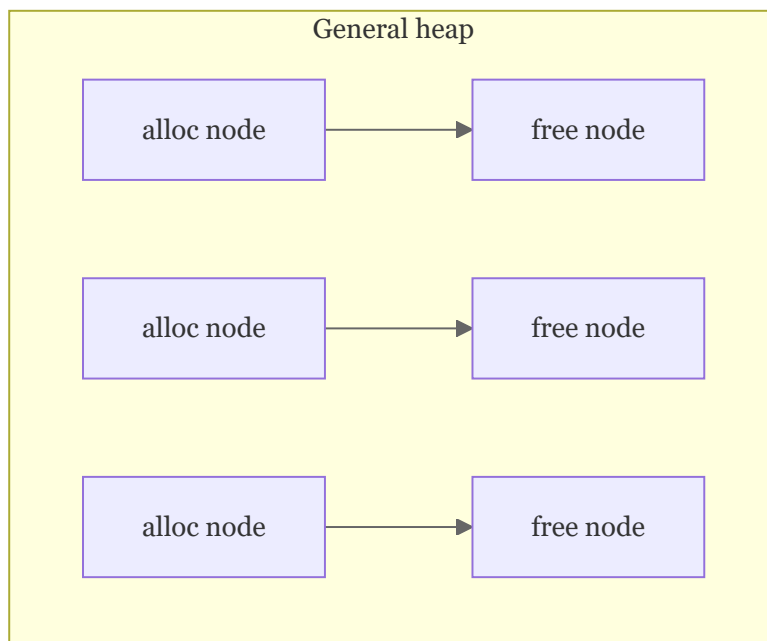
Opening scenario

A query service parses requests, builds ASTs, resolves names, creates plans, and discards the working graph when each request finishes. The business requirement is region ownership: allocate phase-local data together, traverse it through arena references or stable handles, and release the whole region predictably.

Why arena allocation matters

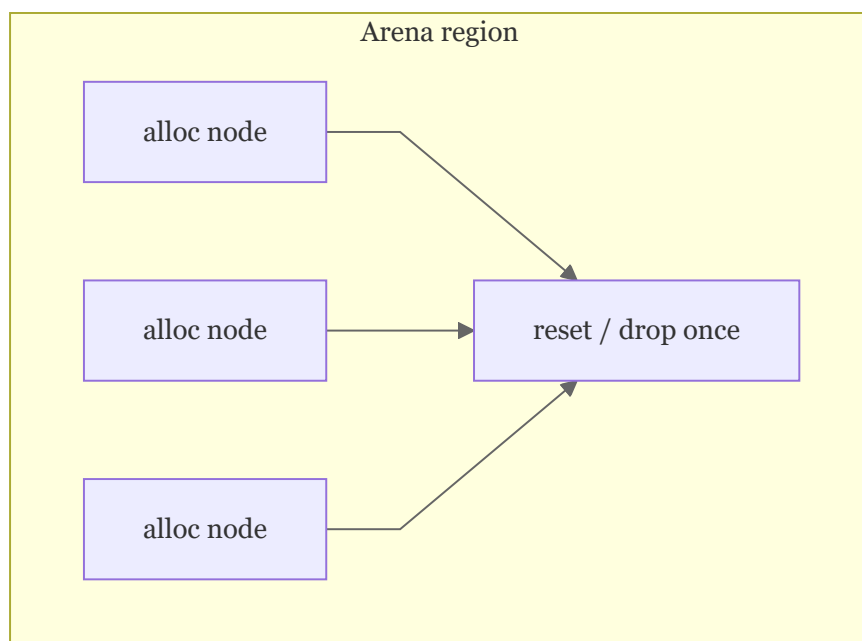
Mental model

The shape to hold in your head is two columns. On the left, the general heap tracks each object on its own: every allocation and every free is an independent event the allocator has to bookkeep. On the right, an arena tracks one region: every object is born into it, and the whole region is released in a single step. The arena does not make any one allocation magically faster so much as it collapses many free operations into one.



General heap: every object is freed on its own, so each allocation has a matching independent free the allocator must track.

Side by side, the arena collapses all of those individual frees into one region teardown:



Arena region: many objects share one lifetime, so a single reset or drop reclaims all of them at once.

How this maps to other languages

Most engineers arrive at arenas with a habit from another language, and the habit is usually close enough to be misleading. The cards below name the one mental-model shift that matters for each background, rather than the API differences, because the API is the easy part once the model is right.

Core concepts

Bump allocators

A bump allocator is the simplest arena shape. It maintains a cursor into a region. Allocation moves the cursor forward. There is usually no per-object free. The cheap operation is whole-region reset or drop.

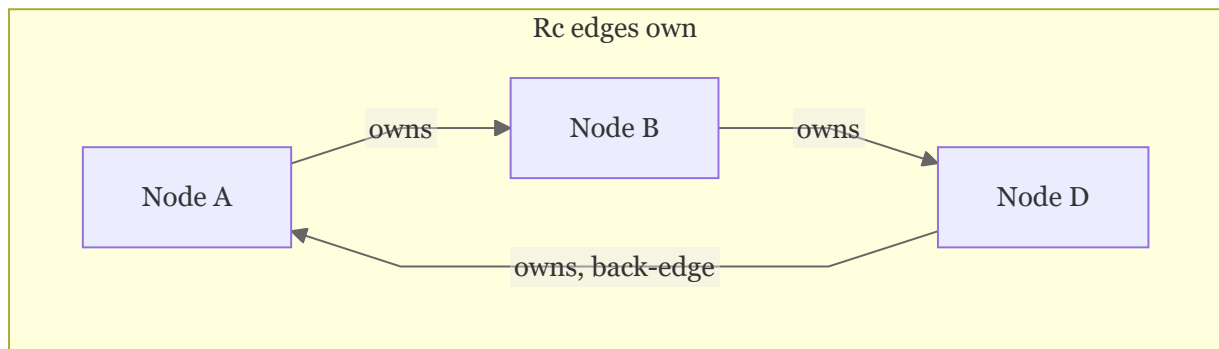
That model is strong when most temporary objects share one scope. It is weak when individual objects must be freed independently.

Object graphs without reference cycles

A common Rust repair for graph-shaped data is simple: let one arena own all nodes, then let edges store `NodeId` handles instead of `Rc<Node>` pointers. The graph becomes one owner plus cheap handles, not a web of ref-counted owners.

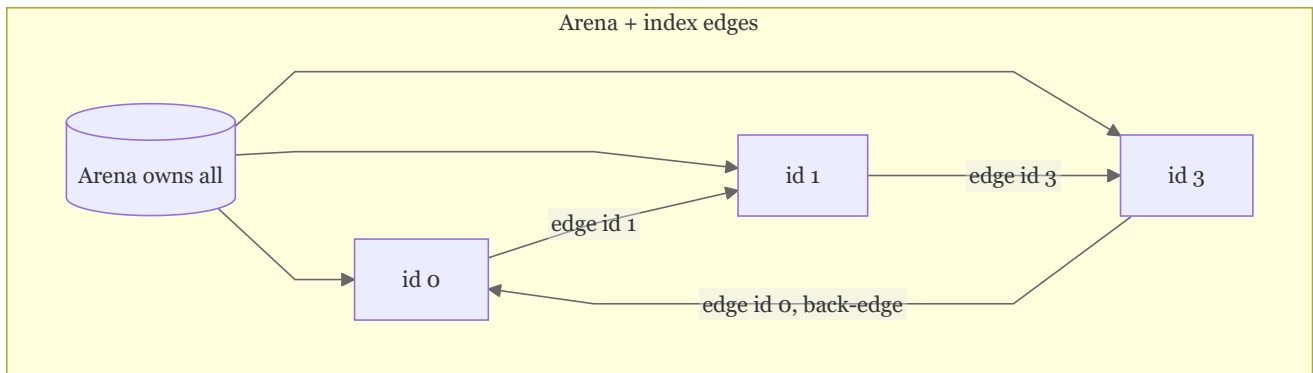
This is often calmer than `Rc<RefCell<T>>` for ASTs, IR, routing graphs, and domain DAGs. You remove cycles by construction because edges are not owners. You also regain explicit control over mutation and traversal from the arena root.

The diagram below is the same graph drawn two ways. Look at where ownership lives. On the left every edge is an owner, so a back-edge from D to A forms a reference cycle that never drops to zero and leaks. On the right the arena is the only owner and every edge is just an index, so the identical back-edge is harmless data and one drop frees the whole region.



Owning Rc edges: the back-edge from D to A closes a reference cycle that never drops to zero, so the whole loop leaks.

Side by side, the same shape redrawn so the arena is the only owner and every edge is just an index:



Index edges into a single-owner arena: the identical back-edge is now a harmless integer, and one drop frees the whole region.

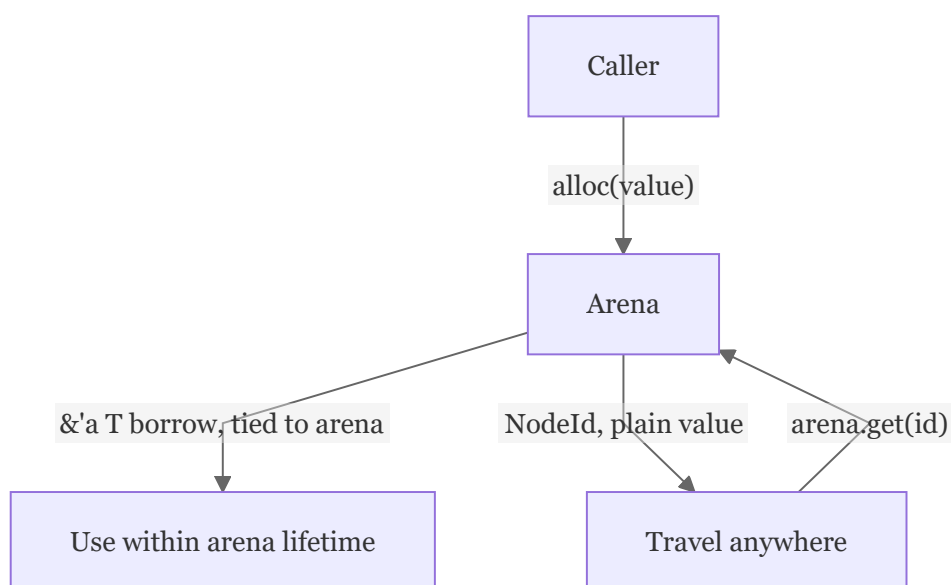
Arena-backed ASTs

ASTs are a near-perfect arena workload. You parse once, build many small nodes, connect them densely, run analysis passes, and then discard the whole tree. A region allocator or ID arena usually matches the operational truth better than individually boxed nodes.

- Node creation is append-only during parsing.
- Traversal is read-heavy after construction.
- Edges are usually logical identity, not shared ownership.
- The whole tree often dies as one batch.

Lifetimes with arenas

The difference is what crosses the boundary back to the caller. A reference arena hands back a borrow that is chained to the arena's lifetime, so the borrow checker will not let that reference outlive the arena. An index arena hands back a small plain value that owns nothing, so it can travel anywhere; the cost is that you must go back to the arena to turn the handle into data.



Two return shapes from the same alloc call: a borrow welded to the arena's lifetime, or a handle that owns nothing and must be resolved back through the arena.

A useful rule: if you want data to cross async tasks, queues, caches, or other long-lived subsystem boundaries, prefer owned values or stable handles. Arena-borrowed references are strongest when the whole phase stays local and synchronous.

Generational arenas and slab allocation

The whole point of a generation counter is what happens to a slot after it is freed and reused. A plain usize index would still point at the slot and silently read whatever now lives there. A generational handle carries the generation it was issued for: when a slot is removed its generation is bumped, so a handle minted earlier no longer matches and the lookup fails cleanly instead of aliasing the new occupant. That clean failure is exactly the bug class a raw index cannot protect against.

Tradeoffs: deallocation, fragmentation, locality

Practical crates and implementation patterns

The choice is less about crate popularity than about model fit. Ask three questions first: do I want arena-tied references or handles, do I need deletion, and does stale-handle protection matter?

Production patterns

Pitfalls and tradeoffs

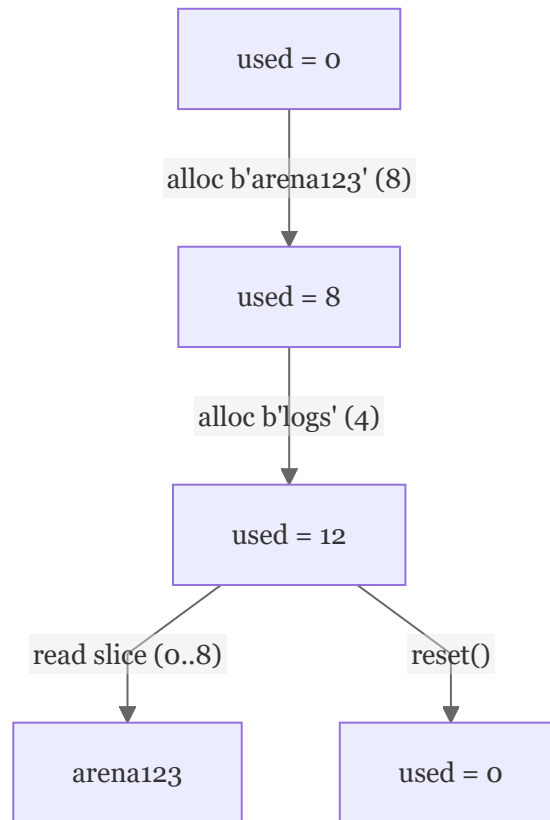
The best arena optimization is often not “allocation became faster.” It is “the lifetime model became honest.” If the workload does not actually share one region lifetime, the arena will eventually expose that mismatch.

Examples

Example 1: a tiny bump allocator for request-scoped scratch

Allocation only moves the cursor forward. Reset drops the whole region's logical contents in one step.

What to look at: there is exactly one piece of mutable state, the used cursor. Every allocation only moves it forward, and the single cheap teardown sets it back to zero. There is no per-allocation free path anywhere in the type. The diagram traces what the cursor does for the exact byte strings in main.



The cursor only ever moves forward on alloc; reset snaps it back to zero in one step, logically discarding everything at once.

Example 13-1. fixed_bump_buffer.rs — *Bump scratch*

```

struct Bump<const N: usize> {
    buf: [u8; N],
    used: usize,
}

impl<const N: usize> Bump<N> {
    fn new() -> Self {
        Self {
            buf: [0; N],
            used: 0,
        }
    }

    fn alloc_bytes(&mut self, bytes: &[u8]) -> Option<(usize, usize)> {
        let end = self.used.checked_add(bytes.len())?;
        if end > N {
            return None;
        }

        let start = self.used;
        self.buf[start..end].copy_from_slice(bytes);
        self.used = end;
        Some((start, end))
    }

    fn slice(&self, range: (usize, usize)) -> &[u8] {

```

```

        &self.buf[range.0..range.1]
    }

    fn used(&self) -> usize {
        self.used
    }

    fn reset(&mut self) {
        self.used = 0;
    }
}

fn main() {
    let mut arena = Bump::<32>::new();
    let first = arena.alloc_bytes(b"arena123").unwrap();
    let _second = arena.alloc_bytes(b"logs").unwrap();

    println!("used = {}", arena.used());
    println!("first = {}", std::str::from_utf8(arena.slice(first)).unwrap());

    arena.reset();
    println!("after reset = {}", arena.used());
}

```

OUTPUT

```

used = 12
first = arena123
after reset = 0

```

Quick check: change the byte strings or arena size and rerun. The important part is the lifetime model: append-only allocation now, coarse reset later.

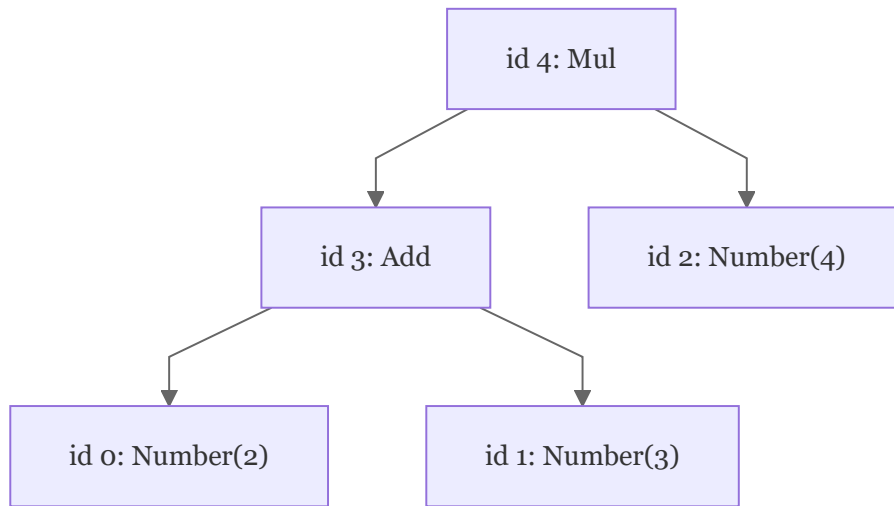
The example returns ranges, not long-lived borrowed slices, so later allocations stay simple and safe.

This is the shape to use for request-scoped scratch buffers, parser workspaces, and temporary packing regions.

Example 2: arena-backed AST with stable IDs instead of Rc edges

One owner stores all nodes. Expressions point to each other by `ExprId`, and evaluation reborrows from the arena root.

What to look at: the enum variants hold `ExprId` values, not `Box<Expr>` or `Rc<Expr>`. A node never owns another node; the arena owns them all in one vector, and an edge is just an index into that vector. The diagram is the tree this program builds, with each node labelled by the slot it lands in.



The expression $(2 + 3) * 4$ as five arena slots. *eval* walks the tree by resolving each *ExprId* back through the arena, so $2 + 3 = 5$ then $5 * 4 = 20$, across 5 nodes.

Example 13-2. arena_indexed_ast.rs — Index ast

```

#[derive(Clone, Copy, Debug, PartialEq, Eq)]
struct ExprId(usize);

#[derive(Debug)]
enum Expr {
    Number(i64),
    Add(ExprId, ExprId),
    Mul(ExprId, ExprId),
}

#[derive(Default)]
struct ExprArena {
    nodes: Vec<Expr>,
}

impl ExprArena {
    fn alloc(&mut self, expr: Expr) -> ExprId {
        let id = ExprId(self.nodes.len());
        self.nodes.push(expr);
        id
    }

    fn get(&self, id: ExprId) -> &Expr {
        &self.nodes[id.0]
    }

    fn eval(&self, id: ExprId) -> i64 {
        match self.get(id) {
            Expr::Number(value) => *value,
            Expr::Add(left, right) => self.eval(*left) + self.eval(*right),
            Expr::Mul(left, right) => self.eval(*left) * self.eval(*right),
        }
    }

    fn len(&self) -> usize {

```

```
        self.nodes.len()
    }
}

fn main() {
    let mut arena = ExprArena::default();

    let two = arena.alloc(Expr::Number(2));
    let three = arena.alloc(Expr::Number(3));
    let four = arena.alloc(Expr::Number(4));
    let sum = arena.alloc(Expr::Add(two, three));
    let root = arena.alloc(Expr::Mul(sum, four));

    println!("value = {}", arena.eval(root));
    println!("nodes = {}", arena.len());
}
```

OUTPUT

```
value = 20
nodes = 5
```

Quick check: change the numbers or operator structure and rerun. The real lesson is that the graph has one owner and no reference cycle problem to solve.

The arena owns every node in one contiguous table.

ExprId is logical identity, not shared ownership.

You give up direct references in exchange for simpler mutation, simpler graphs, and clearer ownership.

Summary

- Arena allocation is batch lifetime management, not only fast allocation.
- Bump allocators are strongest when many objects share one phase and can be reset together.
- Arena-backed graphs often replace pointer ownership with handles, which removes many cycle and mutation problems.
- Arena-tied references are powerful for local read-heavy phases; handle-based arenas are often calmer for mutation and subsystem boundaries.
- Generational arenas and slabs exist for workloads that need deletion, reuse, and stale-handle protection.

PART III

Designing Types and Abstractions

Express interfaces, domain models, generics, and serialization the way Rust's type system wants.

-
- 14** Interfaces in Rust: Traits
 - 15** OOP Models in Rust
 - 16** Domain-Driven Design in Rust
 - 17** Refactoring Toward Idiomatic Rust
 - 18** Generics Instead of Templates
 - 19** Serialization and Data Contracts
 - 20** Metaprogramming
 - 21** Reflection and Type Introspection
-

CHAPTER 14 · INTERFACES IN RUST: TRAITS

Interfaces in Rust: Traits

Traits as Rust's interface system, trait bounds, associated types, dyn Trait, dispatch choices, and object safety

Trait interfaces define capability contracts across library and service boundaries. This chapter treats bounds, associated types, generic dispatch, trait objects, and object safety as reviewable API choices rather than syntax, so you can decide deliberately what a type must prove and what each call site pays for at runtime.

Builds on Chapter 01

Chapter 01 noted that traits describe behavior and that generics usually compile via monomorphization rather than dynamic dispatch. This chapter makes that operational: how to write the contract, how bounds check it, and when to pay for a vtable instead.

Opening scenario

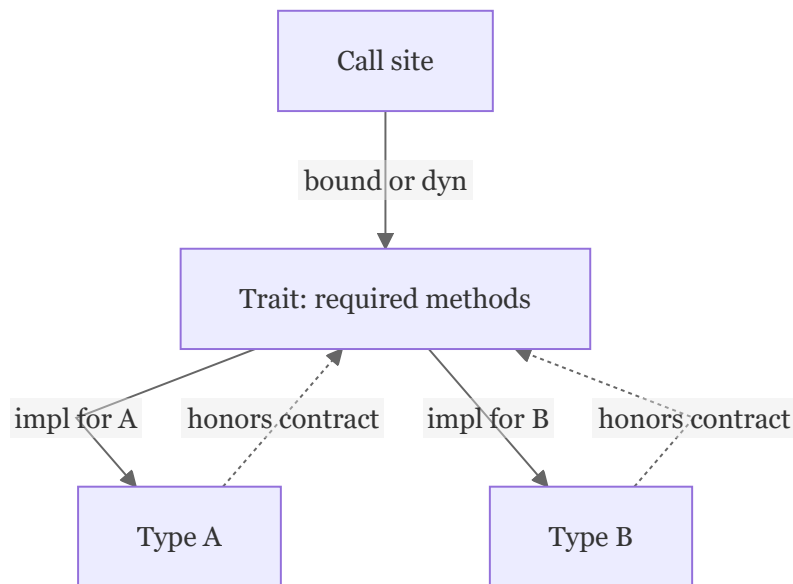
A service needs a pluggable processing stage: today it normalizes text, tomorrow it will validate, redact, or enrich, and operators want to enable stages by configuration. At the same time, a retry layer needs to ask a policy whether another attempt is allowed, and that policy must be checkable at compile time so a misuse fails the build, not production. Both are interface problems, and both are solved with traits, but they pull in opposite directions on dispatch.

A practical decision order

1. Name the capability as a small trait: the exact methods a type must provide.
2. Decide what each implementer outputs once: associated type, or a generic parameter if it varies.
3. Pick dispatch per call site: generic bound for speed, dyn for a heterogeneous collection.
4. If dyn is needed, keep the trait object-safe and move generic helpers out of the core.

Mental model

The shape worth memorizing is a triangle. At one corner sits the trait, a named set of methods. At another sit the concrete types, each with its own impl block. At the third sit the call sites, which name the trait as a bound or as a dyn object. A type and a trait are linked only by an impl, and a call site reaches a type only through the trait. Nothing is connected by inheritance; everything is connected by an explicit implementation.



A trait links to types only through impl blocks; call sites reach types only through the trait. There is no shared base.

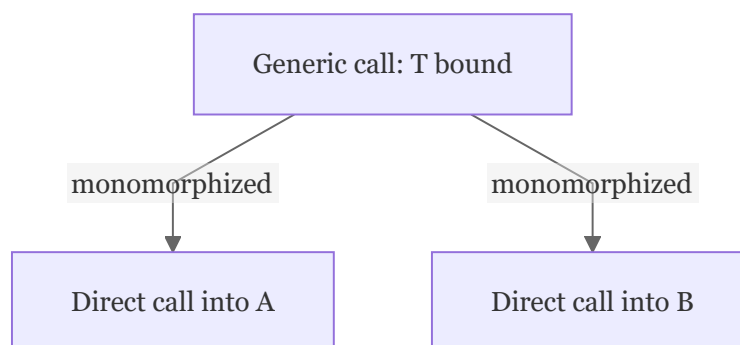
Core concepts

Traits, impls, and default methods

Because an impl is separate from the type definition, the same struct can implement Display, Iterator, and a domain trait of your own, none of which know about the others. That is the practical meaning of composition over inheritance here: capability is added in independent layers rather than fixed in a chain of parents.

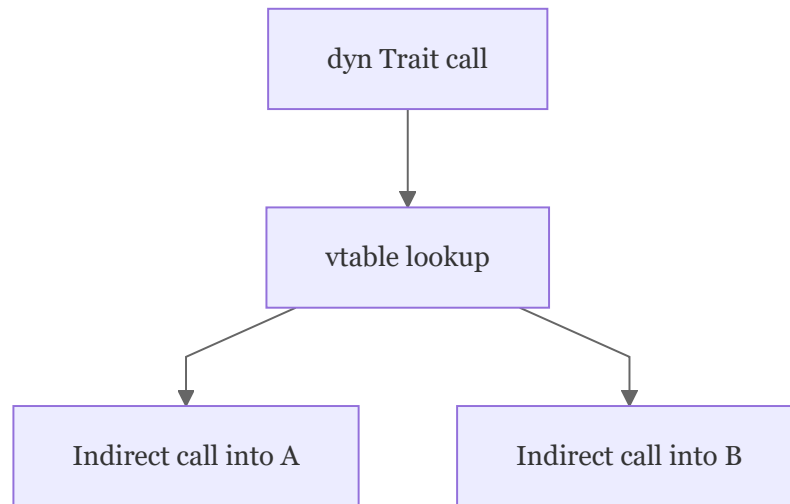
Static dispatch and dynamic dispatch

The single most consequential trait decision is how a call reaches the implementation. A generic bound is resolved at compile time into a direct call; a trait object is resolved at runtime through a vtable. The diagram below contrasts the two paths so the cost of each is visible before you commit a call site to one.



Static dispatch: a generic call is specialized per concrete type into a direct, inlinable call.

The dynamic path instead funnels every concrete type through one shared indirection:



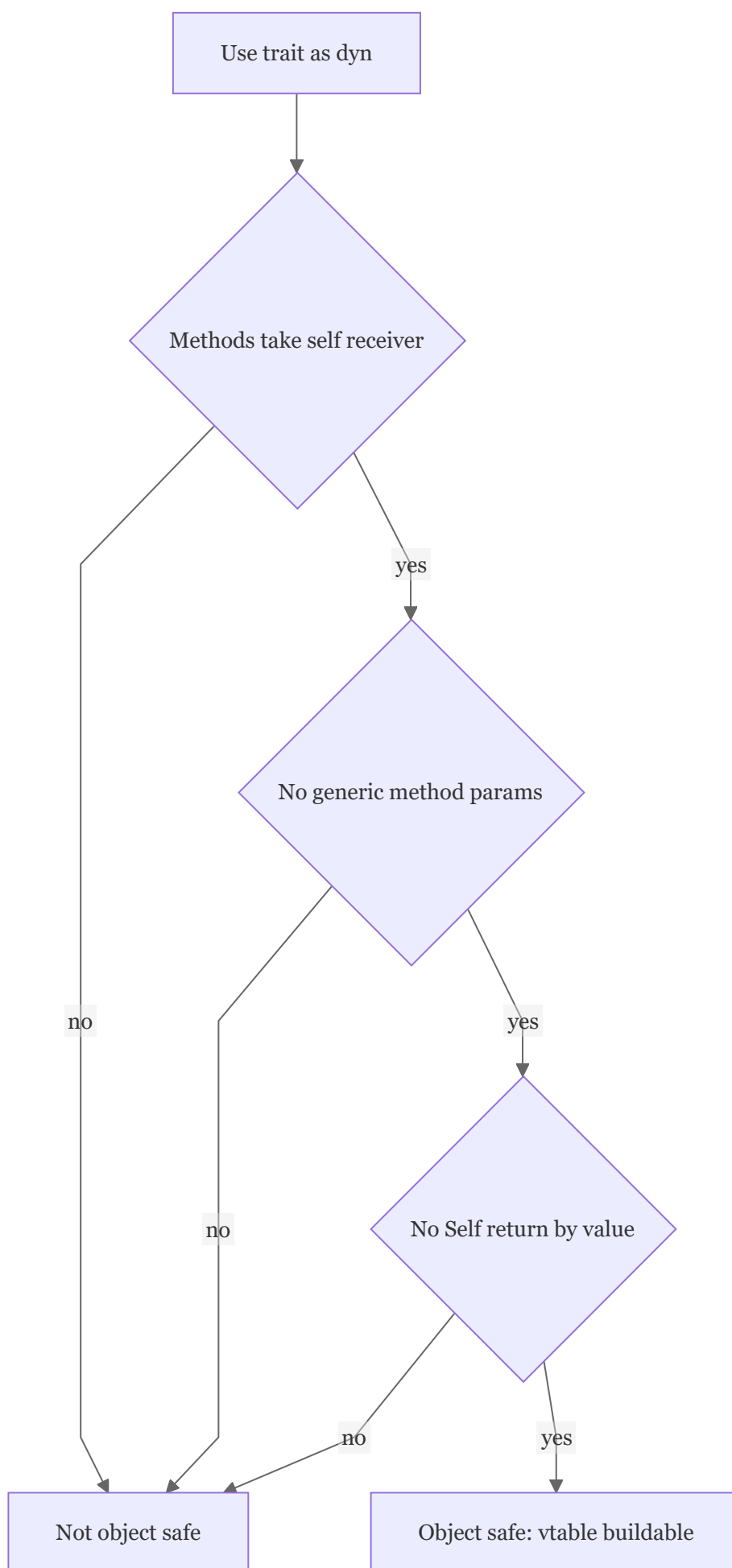
Dynamic dispatch: a dyn call funnels every type through one vtable indirection before reaching the implementation.

Associated types versus generic parameters

Both let a trait talk about types it does not yet know, but they answer different questions. An associated type asks the implementer to commit to one output. A generic parameter lets the same type implement the trait several ways. Choosing the wrong one shows up later as turbofish noise on every caller or as an implementation you cannot write.

Object safety

You can only build a Box when the trait is object-safe, because the compiler has to lay out a vtable with one fixed slot per method. The rules are not arbitrary; each one exists so that a single indirect call works without knowing the concrete type. The flow below is the check the compiler runs when you reach for a trait object.



Object safety is a checklist: a self receiver, no generic methods, and no by-value Self return. Fail any one and dyn is rejected.

Supertraits, blanket impls, and markers

Traits compose. A supertrait says one capability presupposes another, a blanket impl grants a behavior to every type that already meets a bound, and a marker trait carries no methods but still participates in the type system. The snippet shows a supertrait: any Audit type must also be Identify, so the default method can call across the requirement.

Traits as API design

At a boundary, the trait is the contract you publish and the concrete type is an implementation detail you keep. The discipline is the same one you would apply to any interface, with one Rust-specific lever: the bound on a parameter is the precise list of capabilities the caller must supply, so it is worth writing exactly, no wider and no narrower than the body needs.

How this lands by background

Most engineers already have an interface mechanism in muscle memory. The useful question is which part of that mechanism transfers to traits and which part quietly misleads. The shift is rarely about syntax; it is about where implementation is attached, when dispatch is resolved, and whether the compiler verifies the contract before the program runs.

Production patterns

Pitfalls and tradeoffs

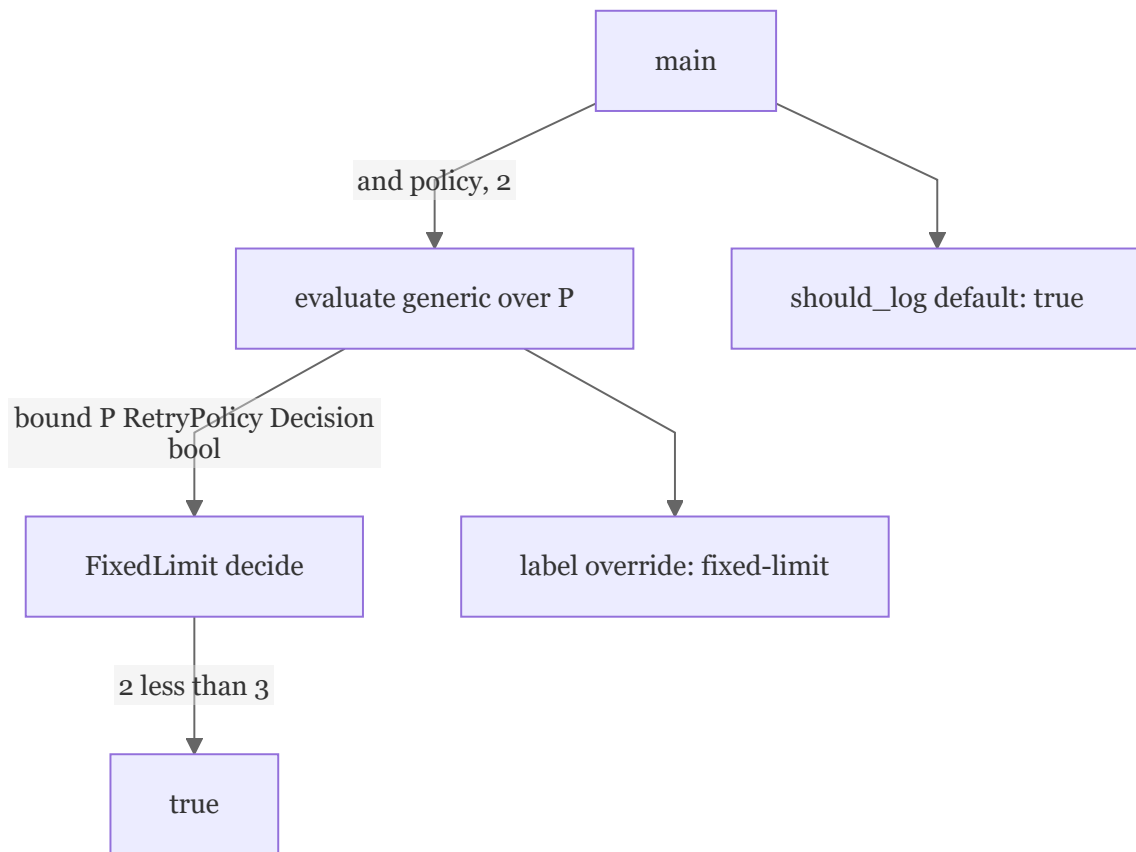
A trait is a contract you will live behind for a long time. Adding a generic method or a by-value Self return is easy today and quietly forecloses dynamic dispatch tomorrow. Decide early whether the trait must work behind dyn, because retrofitting object safety is a breaking change.

Examples

Trait bounds and associated types

A retry policy trait with an associated output type, a default method, and a generic function bounded so the output is exactly the bool the caller can use.

What to look at: the trait declares type Decision, so each implementer commits to one output type. The function evaluate is generic over the policy but bounds the associated type with Decision = bool, which is static dispatch: the compiler specializes it for FixedLimit. The should_log call uses the trait default, untouched by the impl. Trace the call through the diagram, then read it in code.



evaluate is specialized for FixedLimit; the associated Decision is pinned to bool, label is overridden, and should_log falls through to the default.

Example 14-1. `trait_bounds_associated_types.rs` — Associated types

```

trait RetryPolicy {
    type Decision;

    fn decide(&self, attempts: u32) -> Self::Decision;

    fn label(&self) -> &'static str {
        "policy"
    }

    fn should_log(&self) -> bool {
        true
    }
}

struct FixedLimit {
    max: u32,
}

impl RetryPolicy for FixedLimit {
    type Decision = bool;

    fn decide(&self, attempts: u32) -> Self::Decision {
        attempts < self.max
    }
}
  
```

```

    fn label(&self) -> &'static str {
        "fixed-limit"
    }
}

fn evaluate<P>(policy: &P, attempts: u32) -> String
where
    P: RetryPolicy<Decision = bool>,
{
    format!("{}", policy.label(), policy.decide(attempts))
}

fn main() {
    let policy = FixedLimit { max: 3 };

    println!("{}", evaluate(&policy, 2));
    println!("log = {}", policy.should_log());
}

```

OUTPUT

```

fixed-limit => true
log = true

```

Each implementer commits to one Decision output.

The where-clause pins the output so the caller can trust it is a bool.

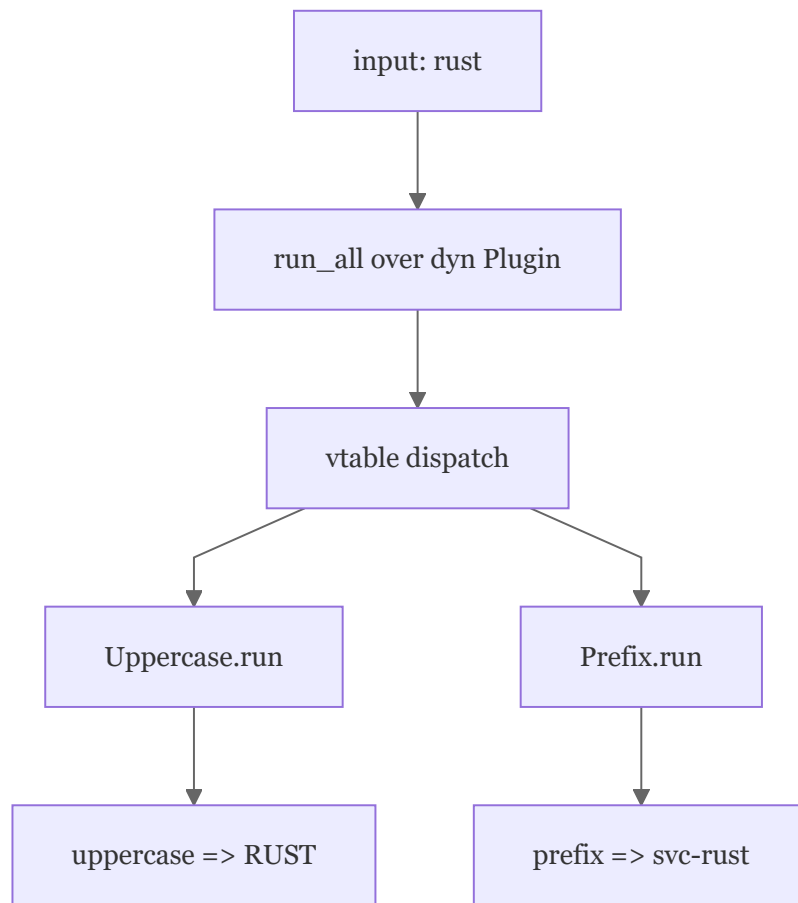
should_log is inherited without an override.

Quick check: run it to confirm the baseline, then change max or the attempts passed to evaluate and watch the decision flip while the contract stays the same.

dyn Trait plugin pipeline

A heterogeneous list of plugins stored as trait objects, each running the same input through one shared function that dispatches dynamically.

What to look at: two unrelated structs, Uppercase and Prefix, implement the same object-safe Plugin trait. They live together in a Vec<, which is only possible because the concrete types are erased. The function run_all iterates that list and calls through the vtable, so one loop drives every plugin. The diagram is that fan-out.



One run_all loop calls each boxed plugin through its vtable; different concrete types share a single code path because the type is erased.

Example 14-2. dyn_plugin_pipeline.rs — Plugin pipeline

```

trait Plugin {
    fn name(&self) -> &'static str;
    fn run(&self, input: &str) -> String;
}

struct Uppercase;

struct Prefix {
    value: &'static str,
}

impl Plugin for Uppercase {
    fn name(&self) -> &'static str {
        "uppercase"
    }

    fn run(&self, input: &str) -> String {
        input.to_uppercase()
    }
}

impl Plugin for Prefix {
    fn name(&self) -> &'static str {
        "prefix"
    }
}
  
```



```

    }

    fn run(&self, input: &str) -> String {
        format!("{}", self.value, input)
    }
}

fn run_all(plugings: &[Box<dyn Plugin>], input: &str) -> Vec<String> {
    plugings
        .iter()
        .map(|plugin| format!("{}", plugin.name(), plugin.run(input)))
        .collect()
}

fn main() {
    let plugings: Vec<Box<dyn Plugin>> = vec![
        Box::new(Uppercase),
        Box::new(Prefix { value: "svc-" }),
    ];

    for line in run_all(&plugings, "rust") {
        println!("{}", line);
    }
}

```

OUTPUT

```

uppercase => RUST
prefix => svc-rust

```

Both methods take `&self` and avoid generics, so the trait can be boxed.

`Vec<>` holds different types behind one element type.

Each run call goes through the vtable at runtime.

Quick check: run it, then add a third plugin or change the prefix value and confirm the pipeline keeps working without touching `run_all`, which is the whole point of the trait-object boundary.

Next chapter

Traits are the interface mechanism. Chapter 15 uses them to express the patterns engineers reach for out of object-oriented habit: polymorphism, encapsulation, and shared behavior modeled with composition, enums, and trait objects instead of inheritance. The dispatch and object-safety choices from this chapter are exactly the tools that work makes use of.

Summary

CHAPTER 15 · OOP MODELS IN RUST

OOP Models in Rust

Composition over inheritance, traits and dyn Trait, enums, state patterns, visitors, and Rust-native encapsulation

Rust object modeling uses composition, traits, enums, and explicit state transitions to represent domain behavior. This chapter maps those choices to maintainable service and library APIs.

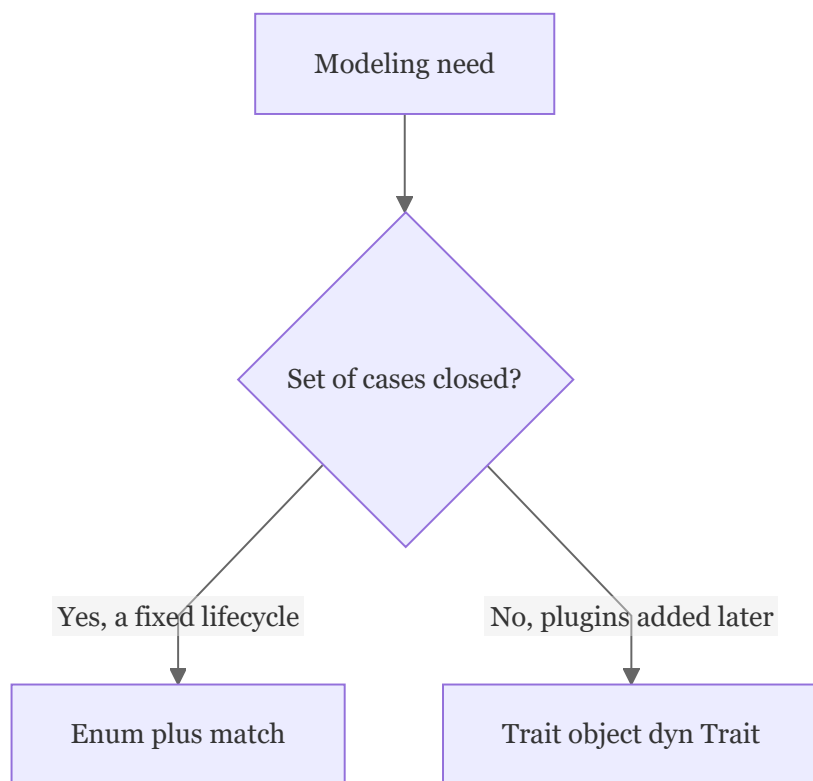
Builds on Chapters 05, 13, and 14

Chapter 05 explained how structs carry ownership and layout. Chapter 13 showed that storage and identity are separate design choices. Chapter 14 covered traits, dispatch, and object safety. This chapter puts those pieces together into Rust's practical answer to OOP modeling.

A worked scenario: the document-workflow platform

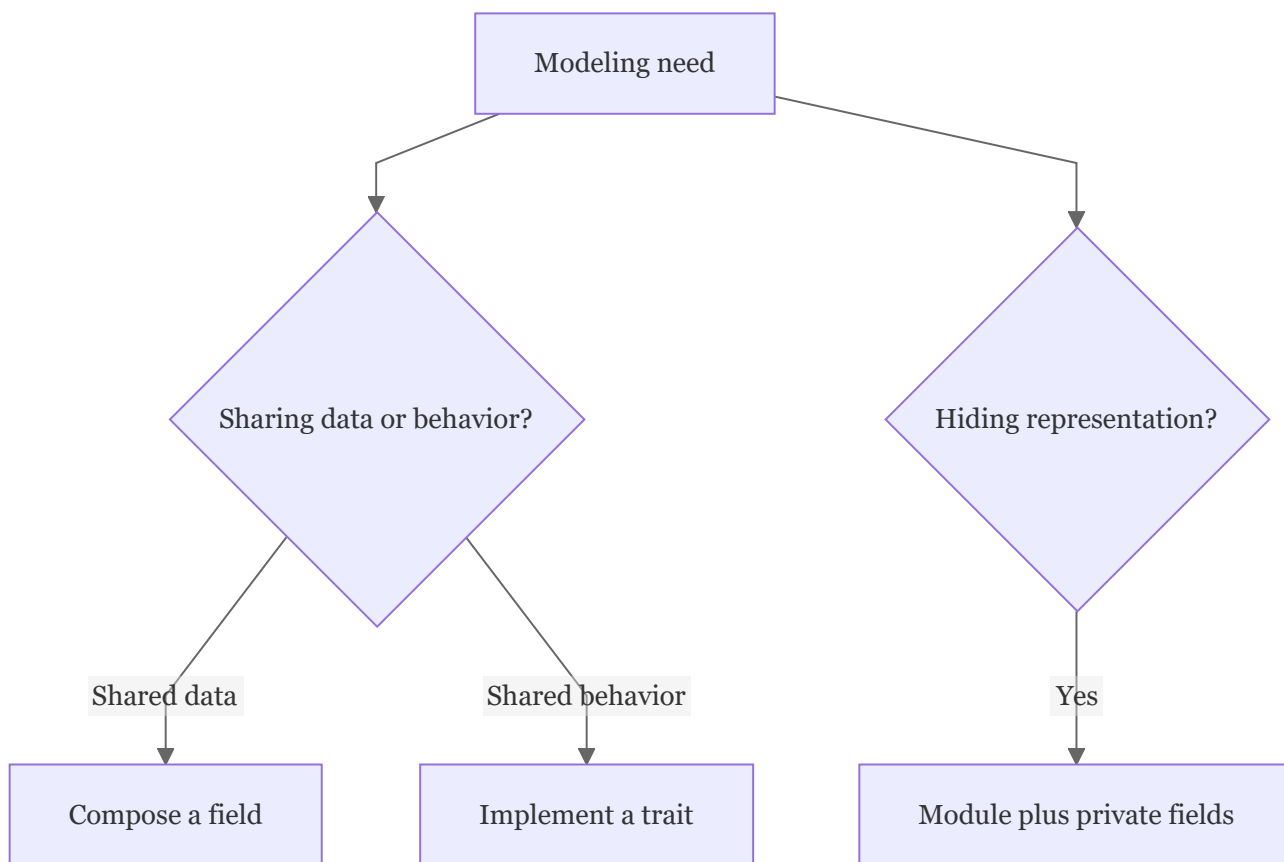
Throughout the chapter we will keep one concrete system in view. A document-workflow platform moves documents through a fixed approval lifecycle, lets services share retry and notification logic, hides invoice representation behind a clean API, and accepts third-party transport plugins for delivering output. Each of those four needs maps to a different Rust tool, and the whole point of the chapter is to see why. In a class-first language all four would likely become subclasses of some Workflow base type; in Rust they deliberately do not.

The deciding question for each need is the same: is the set of cases closed and known at compile time, or open and extended later? That single question routes the design to an enum, a struct, a module boundary, or a trait object, and reading the diagrams below in that order is most of the chapter in one picture.



First question: is the set of variants closed (an enum) or open (a trait object)?

The same modeling need also routes the other two questions, sharing and hiding, to their own tools:



Sharing routes to a composed field or a trait; hiding routes to a module with private fields. Rust does not fold all four into one inheritance tree.

A practical decision order

1. Start with concrete structs and enums before you invent a trait hierarchy.
2. Decide whether the variant set is open or closed.
3. Use traits for shared behavior, not for pretending Rust has base-class fields.
4. Use runtime trait objects only when runtime heterogeneity is semantically real.

Mental model

Core concepts

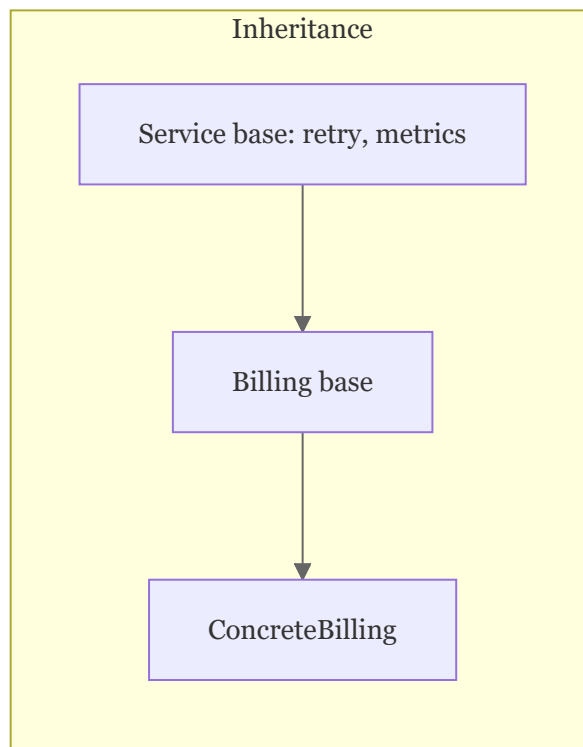
Rust without classical inheritance

Rust has no class inheritance with shared fields. A type does not extend another type to inherit state. That is a sharp correction for C++ and C# engineers: traits describe behavior, but data reuse comes from ordinary fields, helper types, and composition.

This sounds like a loss until you notice what disappears with it: fragile base classes, partially initialized parent state, hidden inherited invariants, and broad object graphs whose aliasing story is unclear. Rust makes each of those choices explicit instead.

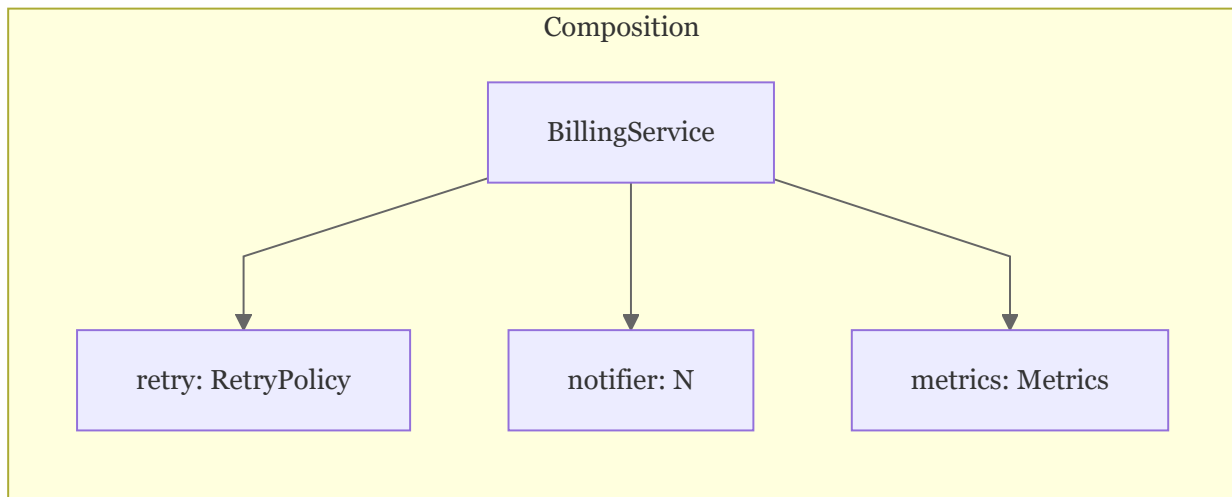
Composition over inheritance

The shape to notice is that inheritance grows data downward through a chain of parents, while composition grows it sideways as named fields you can name, swap, and test one at a time. The two pictures hold the same information, but only the composed one keeps each dependency visible and independently replaceable.



Inheritance buries shared state in a parent chain: each level inherits the one above.

Side by side, composition holds the same parts but lays them out as fields the struct owns directly:



Composition lists the same parts as named fields, each one visible, swappable, and testable on its own.

The default Rust reuse pattern is simple: a struct owns the components it needs. Shared logic lives in those components or in traits implemented by them. The design reads directly from the fields instead of from a base-class tree, so there is no hidden parent state to reason about when you read the type.

This is especially useful in services and pipelines. A `BillingService` can compose a retry policy, a notifier, a cache, and a metrics recorder without pretending those pieces are one inheritance family. Each dependency remains visible, testable, and replaceable, and a test can supply a fake notifier by setting one field rather than subclassing the whole service.

Encapsulation through modules and visibility

In Rust, encapsulation usually lives in modules, private fields, narrow constructors, and focused methods. You do not need inheritance to hide representation. You need a good module boundary and the discipline to expose only what callers actually need.

The translation is this: the public surface is the contract. The internal layout is still allowed to change. Rust's module system often gives tighter encapsulation than a deep protected-member hierarchy because the exported API is narrower and more deliberate.

Polymorphism with traits

Traits are Rust's main behavior abstraction. They let generic code ask for capabilities without naming one concrete type too early. This is Rust's primary answer to interface-like OOP design.

The important distinction from inheritance is that the trait does not bring data fields with it. The implementor chooses its own storage and ownership shape. That is why the same trait can fit a stack value, a boxed object, or an arena-backed node equally well.

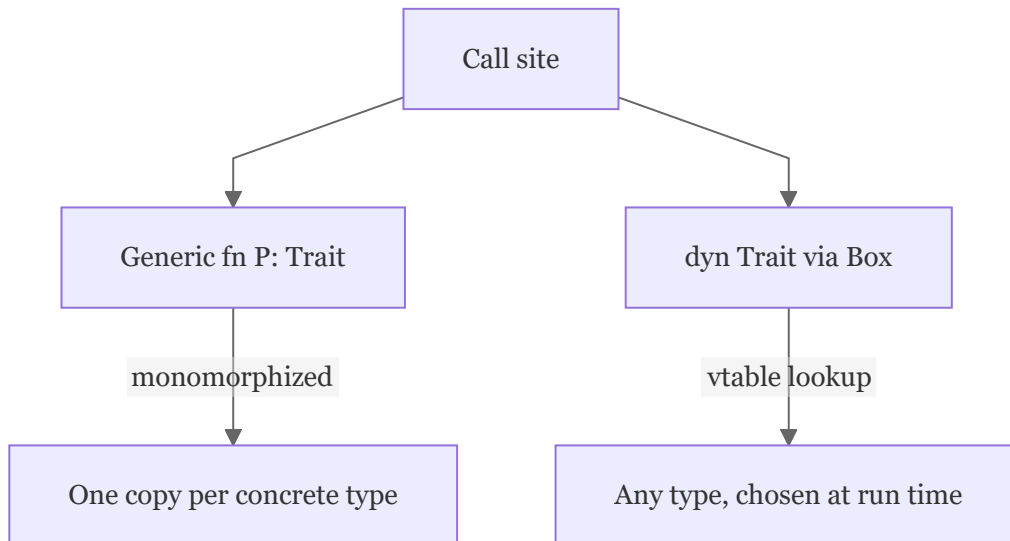
Trait objects as runtime polymorphism

When the system truly needs runtime heterogeneity, Rust uses trait objects such as `Box<dyn Processor>`. This is the closest thing Rust has to virtual-dispatch OOP, but it is opt-in and object-safety rules still apply.

Use this for plugins, strategy registries, sinks, handlers selected from configuration, or any boundary where one collection must hold several concrete implementations at once. Do not use it just because an old base class existed. Chapter 14 covered the dispatch and object-safety details underneath this choice.

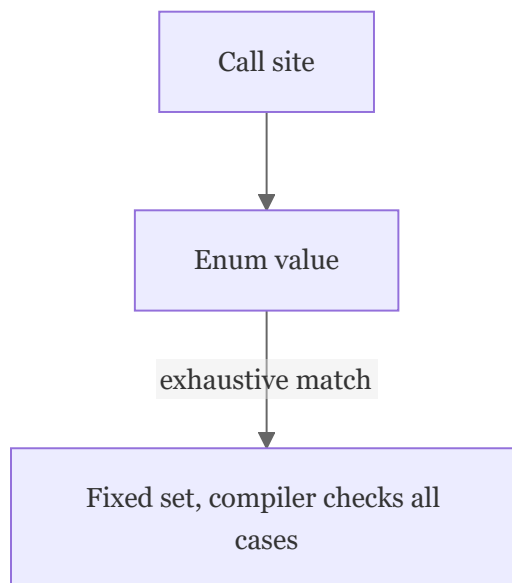
Enums as closed polymorphism

The contrast worth holding in your head is how the three options dispatch. A generic trait bound is resolved at compile time into one specialized copy per type; a trait object defers the choice to a vtable at run time; an enum keeps every case in one type and lets the compiler force every match to cover all of them. The diagrams below line them up so the tradeoff is visible before you read any code.



Static path: a generic bound is monomorphized at compile time; a trait object defers the choice to a vtable at run time.

The third option keeps every case in one type rather than dispatching across types:



Closed path: an enum holds a fixed set of cases and the compiler forces every match to cover all of them.

Enums are Rust's strongest answer when the variant set is closed. Instead of a base class plus derived types, you declare the full set of variants and let pattern matching drive behavior. The data for each case lives inline, so there is no boxing and no pointer chase to reach it.

This is often better than a traditional OOP hierarchy for commands, events, workflow state, AST nodes, transport types, and many domain models. The compiler checks exhaustiveness, which means adding a new variant turns into a compile-time repair across every match rather than a quiet runtime default that ships to production unhandled.

State pattern in Rust

A lifecycle is the clearest case of a closed set: there is a finite list of states and a finite list of legal transitions between them. The classical state pattern models each state as its own object behind an interface; in Rust the same shape is usually an enum whose match arms are the transitions. The three cards below split the choice by where you want an illegal transition to fail: at run time, at compile time, or only when the set is genuinely open. The document example later in the chapter draws the actual lifecycle graph.

The practical default is usually enum state or `typestate`, not boxed state objects. A state object model can still be correct, but Rust makes the alternatives so strong that many classical implementations become needlessly dynamic.

Visitor pattern and alternatives

Rust often improves on classical visitor-heavy designs because enums and exhaustiveness make closed trees easier to traverse. But when operations or node families stay open across crates, visitor-like trait seams can still be the right model.

Translating instincts from other languages

The hardest part of OOP in Rust is rarely a missing feature; it is an instinct that no longer pays off. Each card below names the one mental-model shift that trips up engineers from that language, not a table of which crate replaces which library.

Production patterns

Pitfalls and tradeoffs

The most common translation mistake is trying to preserve the old class shape instead of preserving the actual domain rules. Rust rewards re-modeling. It often punishes mechanical porting.

When Rust improves on traditional OOP

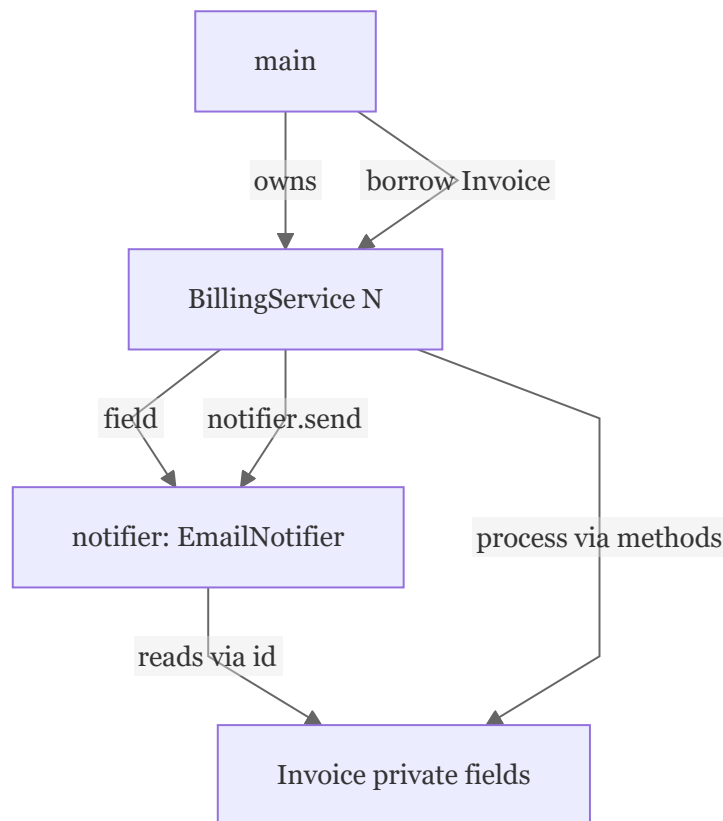
When Rust makes OOP harder

Examples

Example 1: composition, visibility, and trait-based behavior

The service composes a notifier instead of inheriting a base class, and the invoice keeps its fields private behind methods.

What to look at: trace where the behavior comes from. The service holds the notifier as a field (notifier: N), reaches the invoice only through public methods because its fields are private to the billing module, and delegates notification to whatever Notifier it was built with. Nothing is inherited; every capability is a field or a trait bound.



Reuse flows through a field and a trait bound, not a parent class. The invoice is reached only through its public methods.

Example 15-1. composition_and_traits.rs — Composition traits

```

mod billing {
    pub struct Invoice {
        id: u64,
        cents: u64,
    }
}
  
```



```

impl Invoice {
    pub fn new(id: u64, cents: u64) -> Self {
        Self { id, cents }
    }

    pub fn id(&self) -> u64 {
        self.id
    }

    pub fn cents(&self) -> u64 {
        self.cents
    }
}

trait Notifier {
    fn send(&self, invoice: &billing::Invoice) -> String;
}

struct EmailNotifier {
    prefix: &'static str,
}

impl Notifier for EmailNotifier {
    fn send(&self, invoice: &billing::Invoice) -> String {
        format!("{}", self.prefix, invoice.id())
    }
}

struct BillingService<N> {
    notifier: N,
    sent: usize,
}

impl<N: Notifier> BillingService<N> {
    fn process(&mut self, invoice: &billing::Invoice) -> String {
        self.sent += 1;
        format!("invoice = {} cents = {}", invoice.id(), invoice.cents())
    }

    fn notify(&self, invoice: &billing::Invoice) -> String {
        self.notifier.send(invoice)
    }
}

fn main() {
    let mut service = BillingService {
        notifier: EmailNotifier {
            prefix: "queued email for ",
        },
        sent: 0,
    };

    let invoice = billing::Invoice::new(41, 1250);
    let notification = service.notify(&invoice);

```

```
println!("{}", service.process(invoice));
println!("notification = {}", notification);
println!("sent = {}", service.sent);
}
```

OUTPUT

```
invoice = 41 cents = 1250
notification = queued email for 41
sent = 1
```

Quick check: change the invoice ID, cents, or notifier prefix and rerun. The point is not the string formatting. It is that reuse comes from fields plus trait bounds, not from a parent service class.

BillingService owns a notifier component directly.

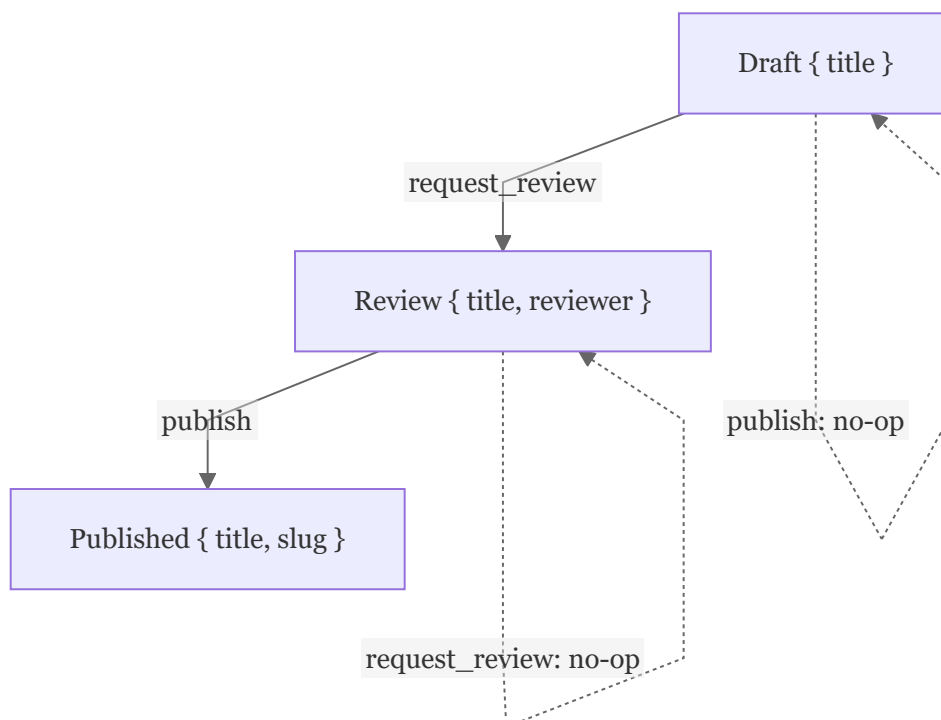
The invoice fields stay private, so callers cannot violate representation casually.

Trait-based behavior stays open without dragging storage inheritance along with it.

Example 2: enum-based state pattern

The state set is closed, so an enum plus explicit transitions is calmer than a family of dynamic state classes.

What to look at: each transition takes self by value and returns a new Document, so the old state is consumed and cannot be used again. The chain in main walks the lifecycle one move at a time, and any transition that does not apply to the current state falls through to the catch-all arm that returns the state unchanged.



The builder chain consumes self at each step: Draft becomes Review becomes Published. Dashed edges are transitions that do not apply and return the value untouched.

Example 15-2. `enum_state_machine.rs` — *State machine*

```

#[derive(Debug)]
enum Document {
    Draft { title: String },
    Review { title: String, reviewer: String },
    Published { title: String, slug: String },
}

impl Document {
    fn request_review(self, reviewer: &str) -> Self {
        match self {
            Document::Draft { title } => Document::Review {
                title,
                reviewer: reviewer.to_string(),
            },
            state => state,
        }
    }

    fn publish(self) -> Self {
        match self {
            Document::Review { title, .. } => {
                let slug = title.to_lowercase().replace(' ', "-");
                Document::Published { title, slug }
            }
            state => state,
        }
    }

    fn state(&self) -> &'static str {
        match self {
            Document::Draft { .. } => "draft",
            Document::Review { .. } => "review",
            Document::Published { .. } => "published",
        }
    }

    fn slug(&self) -> &str {
        match self {
            Document::Published { slug, .. } => slug,
            _ => "none",
        }
    }
}

fn main() {
    let doc = Document::Draft {
        title: String::from("Rust OOP"),
    }
    .request_review("mina")
    .publish();

    println!("state = {}", doc.state());
    println!("slug = {}", doc.slug());
}

```

OUTPUT

```
state = published
slug = rust-oop
```

Quick check: change the title and rerun. The variant set stays closed and the transition logic stays in one visible place, which is usually easier to review than a dynamic state-object lattice.

The compiler knows every valid document state ahead of time.

Illegal transitions can be made explicit instead of disappearing behind shared mutable flags.

If you want compile-time-only transitions, the `typestate` exercise page pushes this idea one step further.

The repository also includes standalone Rust source under `examples/ch15_oop_models_in_rust/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

- Rust does not offer classical inheritance with shared fields. It splits OOP concerns into smaller, clearer tools.
- Composition is the default reuse mechanism. Traits express behavior. Modules and visibility express encapsulation.
- Traits and trait objects solve open polymorphism. Enums solve closed polymorphism. Confusing the two is a common design mistake.
- State machines are usually better modeled with enums or `typestate` than with boxed state objects ported mechanically from older patterns.
- Rust often improves traditional OOP by making ownership, state validity, and variant closure explicit, but it also makes some inheritance-shaped designs harder on purpose.

CHAPTER 16 · DOMAIN-DRIVEN DESIGN IN RUST

Domain-Driven Design in Rust

Entities, value objects, aggregates, repositories, invariants, event sourcing, and DDD boundaries

Domain-driven Rust encodes business rules in types, constructors, aggregate boundaries, and repository interfaces. This chapter keeps invalid states and persistence concerns out of core domain code.

Builds on Chapters 05, 14, and 15

Chapter 05 established ownership inside structs. Chapter 14 covered traits as interface boundaries. Chapter 15 explained why Rust prefers composition, enums, and explicit state models over classical inheritance. DDD in Rust sits on top of all three.

Opening scenario

An order platform coordinates pricing, inventory reservation, payment authorization, and fulfillment. The business requirement is to encode business language and invariants directly in Rust types: domain identifiers, value objects, aggregate methods, repository seams, and transport-independent error contracts.

A practical decision order

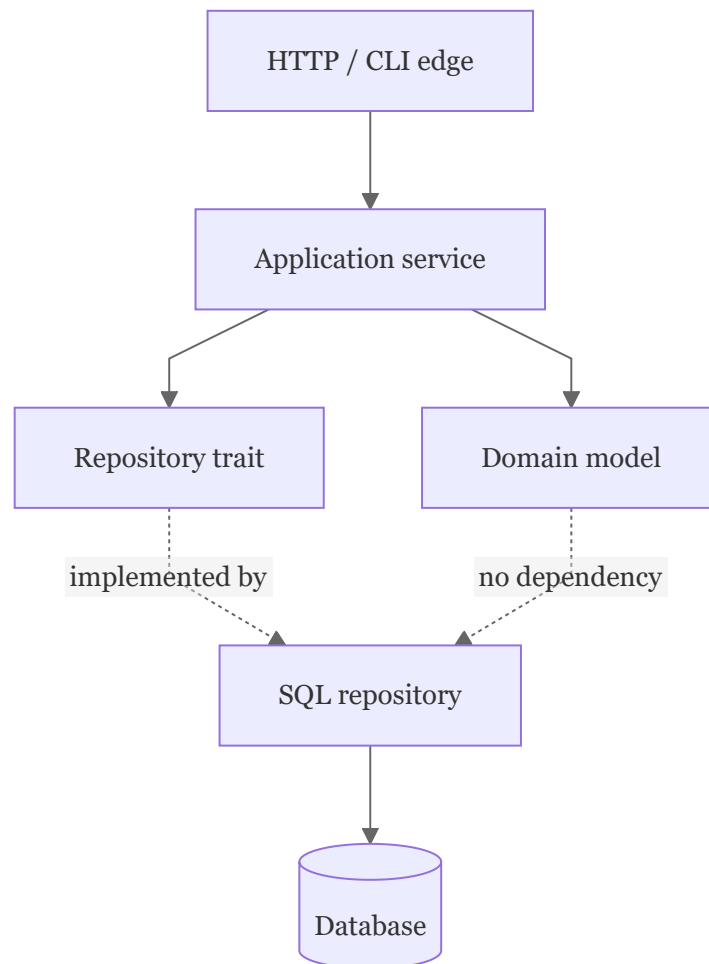
1. Identify the domain concepts that deserve names in types.
2. Decide which invariants belong in value-object constructors and which belong at aggregate methods.
3. Keep application services responsible for orchestration, not for becoming the only place rules exist.
4. Make repository and integration boundaries speak in domain language or explicit translations.

Mental model

What changes by background

Domain-driven design is older than Rust, so most senior engineers arrive with habits from a language whose type system made different promises. The vocabulary carries over; the mechanism does not. The shift below is the one worth internalizing before reading any code in this chapter.

The common thread across all four: in Rust the domain model is not enforced by a framework at runtime, by a base class, or by convention. It is enforced by types that refuse to hold an invalid value. The diagram below shows where those types sit relative to the infrastructure they protect.

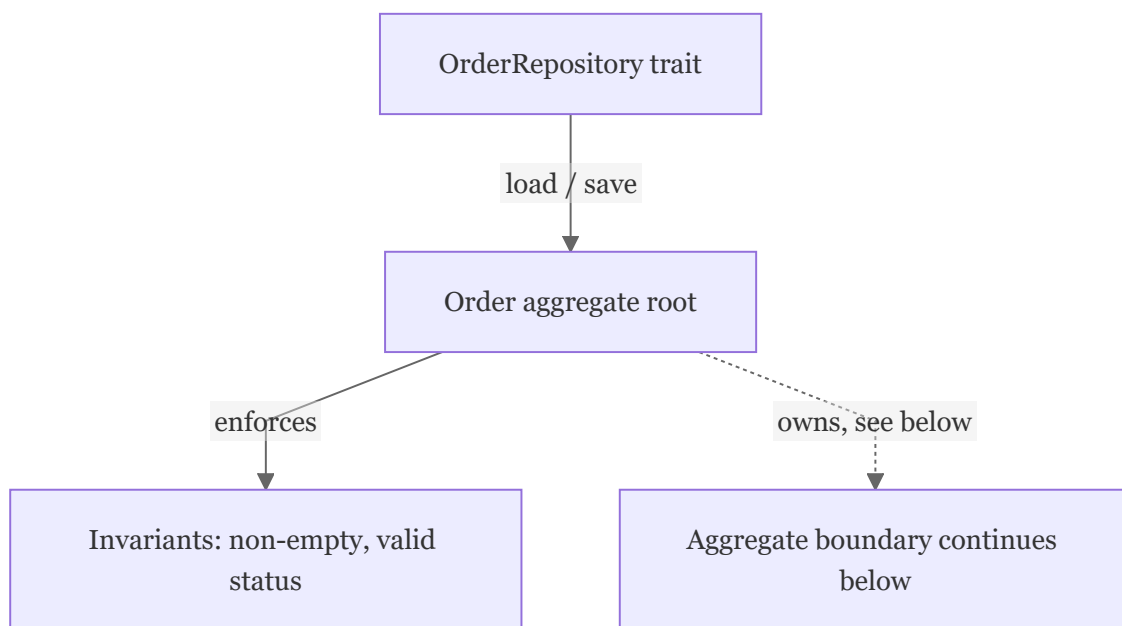


Dependencies point inward toward the domain. The application service depends on a repository trait; the SQL implementation depends on the trait, not the other way round, so the domain never imports infrastructure.

Core concepts

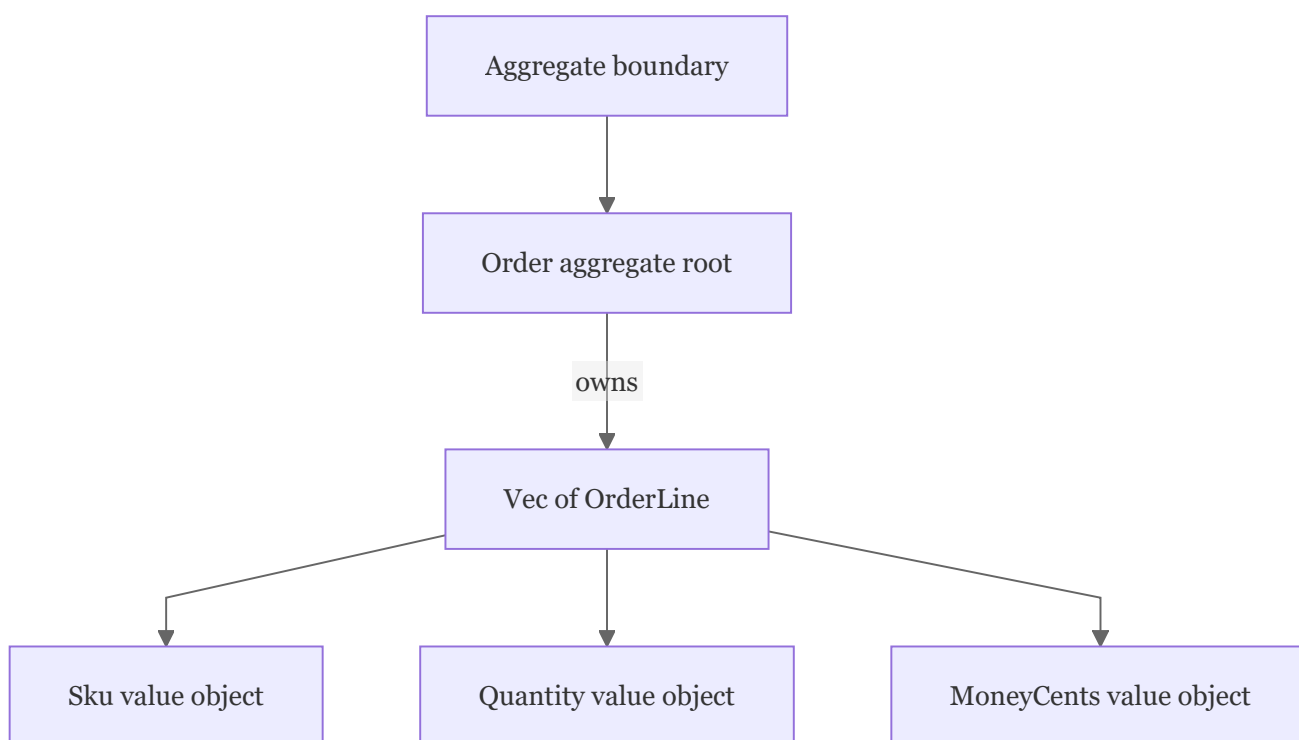
Entities, value objects, aggregates, repositories

These four pieces nest into one shape worth memorizing. Value objects are owned by the aggregate root, which is the entity that guards the invariants; the repository is the only door through which the whole aggregate is loaded and saved. The root owns everything inside the boundary, so there is no way to mutate a line item without going through a method that can re-check the rule.



Outside view: the repository moves the whole aggregate in and out of storage, and invariants are checked on the root rather than scattered across callers.

Looking inside that boundary, the root owns the order lines and the value objects they carry:



Inside view: the root owns its value objects through the order lines, so there is no way to mutate a line without going through a method that can re-check the rule.

Encoding invariants in types

The important rule is simple: if the invalid state is local and stable, encode it in the type. If the rule depends on aggregate-wide state, encode it in the aggregate method. Rust gives you both tools.

Newtypes for domain safety

Newtypes are one of the most practical DDD tools in Rust. They are cheap, explicit, and easy to compose with traits. They prevent primitive obsession without pretending a runtime framework will keep your identifiers straight for you.

That small amount of ceremony buys strong review benefits: fewer swapped parameters, clearer logs, narrower traits, and sharper boundaries between internal and external identifiers.

Rich domain models without inheritance

A strong correction for C++ and C# readers is this: a rich model in Rust means behavior near state and invariants, not a base class with protected fields. Rust wants `impl` blocks, value objects, traits, and modules to each do their own job.

Error modeling in domain layers

Application services and domain services

Repository interfaces with traits

Async repositories and lifetime issues

A reliable production rule is this: if the repository method performs IO and may suspend, return owned data or an owned projection. Do not fight the lifetime model just to preserve a borrowed shape that no longer matches the boundary.

Event sourcing in Rust

DDD boundaries in monoliths and distributed systems

Production patterns

Pitfalls and tradeoffs

DDD is not an excuse for heavy ceremony. In Rust, the best version is often the lightest one that gives the domain real names, protects invariants locally, and keeps infrastructure outside the core language of the model.

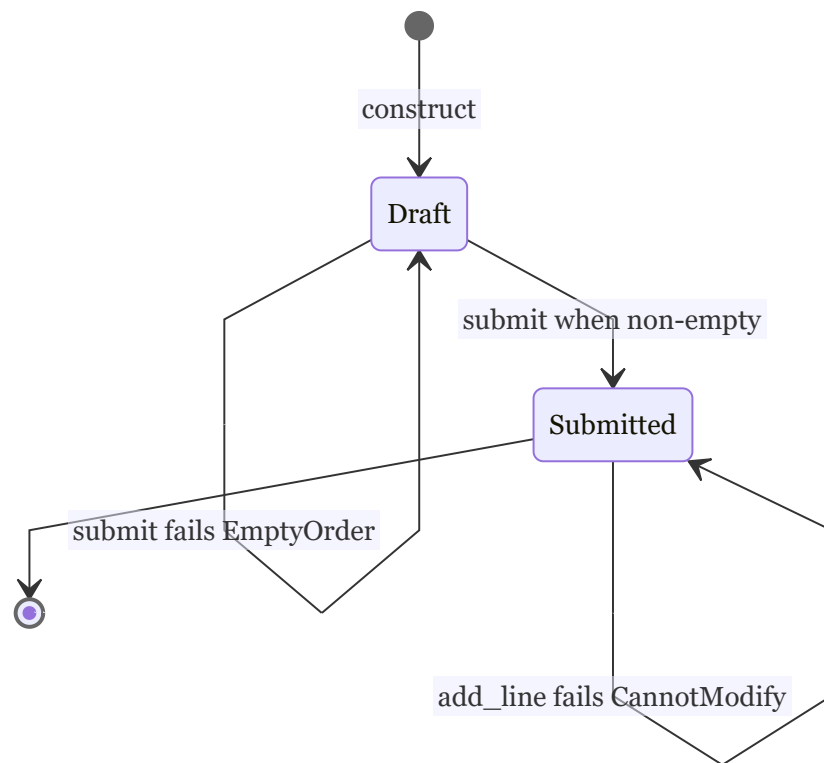
Examples

Example 1: aggregate invariants with newtypes and rich methods

The aggregate root owns the mutable consistency boundary. Value objects reject invalid local state, and the aggregate decides when the order may be submitted.

What to look at: the order is a tiny state machine with two states. While it is Draft you may add lines; submit refuses an empty order and otherwise moves it to Submitted, after which `add_line` is rejected. The value-object constructors reject the local errors first, so by the time a line reaches the aggregate every quantity is

non-zero and every SKU is non-empty. Trace the transitions in the diagram, then read the same guards in the code.



Two states, with the guards drawn as edges: add_line only works in Draft, and submit only advances when at least one line exists.

Example 16-1. order_aggregate.rs — Order aggregate

```

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
struct OrderId(u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
struct CustomerId(u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
struct Quantity(u32);

impl Quantity {
    fn new(value: u32) -> Result<Self, DomainError> {
        if value == 0 {
            return Err(DomainError::InvalidQuantity);
        }
        Ok(Self(value))
    }

    fn get(self) -> u32 {
        self.0
    }
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
struct MoneyCents(u64);

```

```

impl MoneyCents {
    fn new(value: u64) -> Self {
        Self(value)
    }

    fn get(self) -> u64 {
        self.0
    }
}

#[derive(Debug, Clone, PartialEq, Eq)]
struct Sku(String);

impl Sku {
    fn new(value: &str) -> Result<Self, DomainError> {
        if value.trim().is_empty() {
            return Err(DomainError::EmptySku);
        }
        Ok(Self(value.to_string()))
    }
}

#[derive(Debug)]
struct OrderLine {
    sku: Sku,
    qty: Quantity,
    unit_price: MoneyCents,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
enum OrderStatus {
    Draft,
    Submitted,
}

#[derive(Debug)]
enum DomainError {
    InvalidQuantity,
    EmptySku,
    EmptyOrder,
    CannotModifySubmittedOrder,
}

#[derive(Debug)]
struct Order {
    id: OrderId,
    customer_id: CustomerId,
    status: OrderStatus,
    lines: Vec<OrderLine>,
}

impl Order {
    fn new(id: OrderId, customer_id: CustomerId) -> Self {
        Self {
            id,

```

```

        customer_id,
        status: OrderStatus::Draft,
        lines: Vec::new(),
    }
}

fn add_line(&mut self, sku: Sku, qty: Quantity, unit_price: MoneyCents) -> Result<(), DomainError> {
    if self.status == OrderStatus::Submitted {
        return Err(DomainError::CannotModifySubmittedOrder);
    }

    self.lines.push(OrderLine {
        sku,
        qty,
        unit_price,
    });

    Ok(())
}

fn submit(&mut self) -> Result<(), DomainError> {
    if self.lines.is_empty() {
        return Err(DomainError::EmptyOrder);
    }

    self.status = OrderStatus::Submitted;
    Ok(())
}

fn total_cents(&self) -> u64 {
    self.lines
        .iter()
        .map(|line| line.qty.get() as u64 * line.unit_price.get())
        .sum()
}

fn status(&self) -> &'static str {
    match self.status {
        OrderStatus::Draft => "draft",
        OrderStatus::Submitted => "submitted",
    }
}
}

fn main() {
    let mut order = Order::new(OrderId(1001), CustomerId(7));

    order
        .add_line(
            Sku::new("BOOK-1").unwrap(),
            Quantity::new(2).unwrap(),
            MoneyCents::new(1500),
        )
        .unwrap();

    order

```

```
        .add_line(  
            Sku::new("PEN-9").unwrap(),  
            Quantity::new(3).unwrap(),  
            MoneyCents::new(400),  
        )  
        .unwrap();  
  
order.submit().unwrap();  
  
println!("lines = {}", order.lines.len());  
println!("total cents = {}", order.total_cents());  
println!("state = {}", order.status());  
}
```

OUTPUT

```
lines = 2  
total cents = 4200  
state = submitted
```

`Order` has identity and behavior. It is not only a record bag.

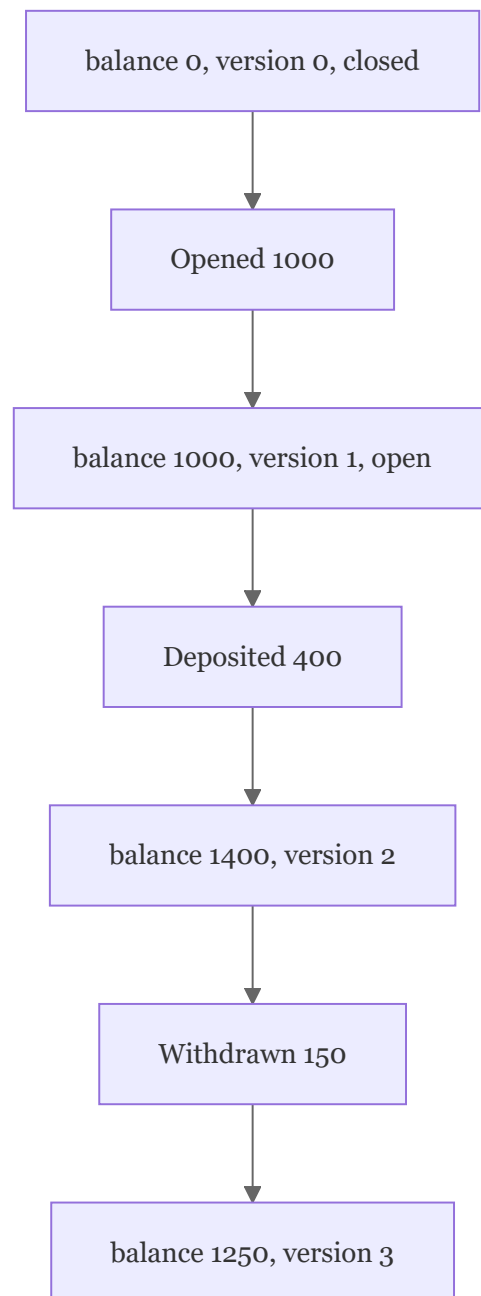
`Quantity`, `MoneyCents`, and `Sku` protect local rules at construction time.

`submit` refuses an empty order. The rule stays on the model, not in a handler comment.

Example 2: event-sourced aggregate rehydration

The aggregate is rebuilt from domain events. Rehydration is a fold, and versioning follows the applied stream length.

What to look at: there is no stored balance to read. Current state is computed by folding the event stream into a fresh accumulator, one event at a time. Each call to `apply` validates against the state built so far — a deposit before the account is open is rejected, a withdrawal larger than the running balance is rejected — and the version counter is simply how many events were applied. The diagram is that fold; the three output numbers are the accumulator after the last event.



Rehydration is a left fold: each event is applied to the running state, version increments per applied event, and the final accumulator is the current account.

Example 16-2. `event_sourced_account.rs` — Sourced account

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
struct AccountId(u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
enum AccountEvent {
    Opened { opening_balance_cents: i64 },
    Deposited { cents: i64 },
    Withdrawn { cents: i64 },
}

#[derive(Debug)]
enum DomainError {
```

```

    AlreadyOpened,
    NotOpen,
    InsufficientFunds,
}

#[derive(Debug)]
struct Account {
    id: AccountId,
    balance_cents: i64,
    version: usize,
    is_open: bool,
}

impl Account {
    fn rehydrate(id: AccountId, history: &[AccountEvent]) -> Result<Self, DomainError> {
        let mut account = Self {
            id,
            balance_cents: 0,
            version: 0,
            is_open: false,
        };

        for event in history {
            account.apply(*event)?;
            account.version += 1;
        }

        Ok(account)
    }

    fn apply(&mut self, event: AccountEvent) -> Result<(), DomainError> {
        match event {
            AccountEvent::Opened {
                opening_balance_cents,
            } => {
                if self.is_open {
                    return Err(DomainError::AlreadyOpened);
                }

                self.balance_cents = opening_balance_cents;
                self.is_open = true;
                Ok(())
            }
            AccountEvent::Deposited { cents } => {
                if !self.is_open {
                    return Err(DomainError::NotOpen);
                }

                self.balance_cents += cents;
                Ok(())
            }
            AccountEvent::Withdrawn { cents } => {
                if !self.is_open {
                    return Err(DomainError::NotOpen);
                }
            }
        }
    }
}

```

```

        if self.balance_cents < cents {
            return Err(DomainError::InsufficientFunds);
        }

        self.balance_cents -= cents;
        Ok(())
    }
}

fn main() {
    let history = vec![
        AccountEvent::Opened {
            opening_balance_cents: 1000,
        },
        AccountEvent::Deposited { cents: 400 },
        AccountEvent::Withdrawn { cents: 150 },
    ];

    let account = Account::rehydrate(AccountId(7), &history).unwrap();

    println!("events = {}", history.len());
    println!("balance cents = {}", account.balance_cents);
    println!("version = {}", account.version);
}

```

OUTPUT

```

events = 3
balance cents = 1250
version = 3

```

The stream records domain facts, not persistence internals.

`rehydrate` applies each event in order and rebuilds current state explicitly.

You gain temporal history, but you also accept projection and migration work.

The repository also includes standalone Rust source under `examples/ch16_domain_driven_design_in_rust/` so the chapter examples can be reviewed as ordinary Rust files outside the in-browser editor.

Summary

- Entities, value objects, aggregates, and repositories map cleanly onto Rust structs, enums, traits, and ownership boundaries.
- Newtypes and constructors are practical tools for encoding invariants and defending domain language from primitive obsession.
- Rich domain models in Rust do not rely on inheritance. They rely on `impl` blocks, composition, enums, and clear visibility.
- Application services orchestrate use cases. Domain services are narrower and should exist only when a rule does not belong to one aggregate or value object.

- Async repository seams usually want owned returns, and event sourcing is a domain choice with real operational tradeoffs, not a free architectural upgrade.

CHAPTER 17 · REFACTORING TOWARD IDIOMATIC RUST

Refactoring Toward Idiomatic Rust

Refactor C++-style, C#-style, and Go-style Rust into clearer ownership, errors, traits, enums, async seams, and testable APIs

Idiomatic Rust refactoring reduces operational risk by clarifying ownership, error flow, data shape, and execution boundaries. This chapter turns review findings into small, verifiable code changes.

Builds on Chapters 04, 07, 08, 14, 15, and 16

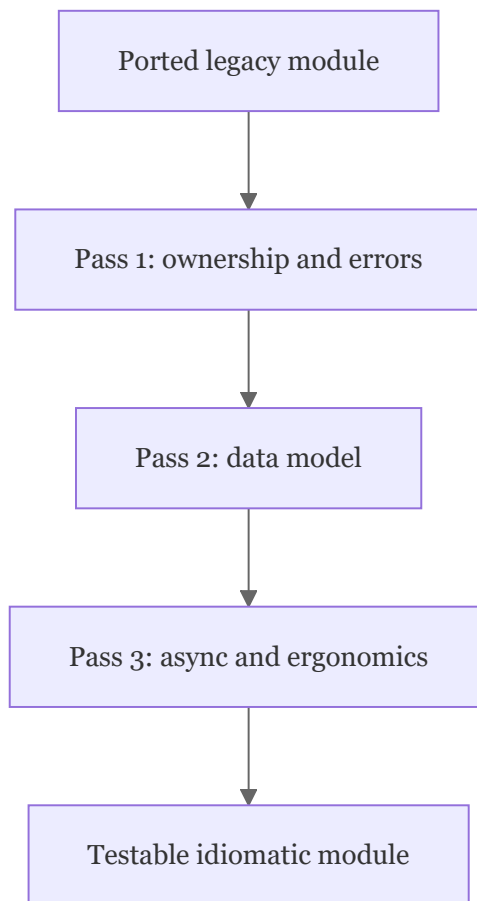
Ownership from Chapter 04, clone economics from Chapter 07, safe abstraction boundaries from Chapter 08, traits from Chapter 14, enum-versus-trait modeling from Chapter 15, and aggregate boundaries from Chapter 16 all feed directly into good Rust refactoring work.

Opening scenario

A mature service has accumulated pointer-heavy objects, panic-based parsing, sentinel failures, public mutable records, and broad runtime dispatch. None of this is wrong in isolation; it is the residue of porting an existing C++, C#, or Go design and then bolting on features under deadline. The business requirement is not a rewrite. It is a staged refactor that makes ownership, fallibility, state shape, and test seams explicit before anyone touches syntax cosmetics or async structure.

The ordering matters more than any single change. When you correct ownership and error flow first, a surprising amount of the lifetime noise and clone pressure simply evaporates, because those problems were symptoms of the wrong boundary rather than independent defects. The three passes below run in that order on purpose: each one removes the pressure that was making the next one hard.

Read the pipeline left to right. Each pass takes the output of the previous one and is only worth attempting once the earlier boundary is clean.



Refactor in passes. Ownership and error flow first, data model second, async and ergonomics last; each pass clears the way for the next.

Mental model

What changes by background

Most of the friction in an idiomatic refactor is not about Rust syntax. It is about the instinct you brought from your previous language and where that instinct now points you wrong. The cards below name the specific mental-model shift for each background, not a library mapping. Find the one that fits you and keep it in mind while you read the core concepts.

Core concepts

Each concept below is a recurring move you make during the three passes. They are grouped roughly by the language habit they correct, but they share one theme: every one of them turns an implicit decision the old code made by convention into an explicit decision the type system can check. Where a concept has a shape worth seeing, a short pointer and a diagram come before the code so you read the structure first and the syntax second.

What to look at:

Clone audit checklist

Async refactor checklist

Testability patterns

Production patterns

Pitfalls and tradeoffs

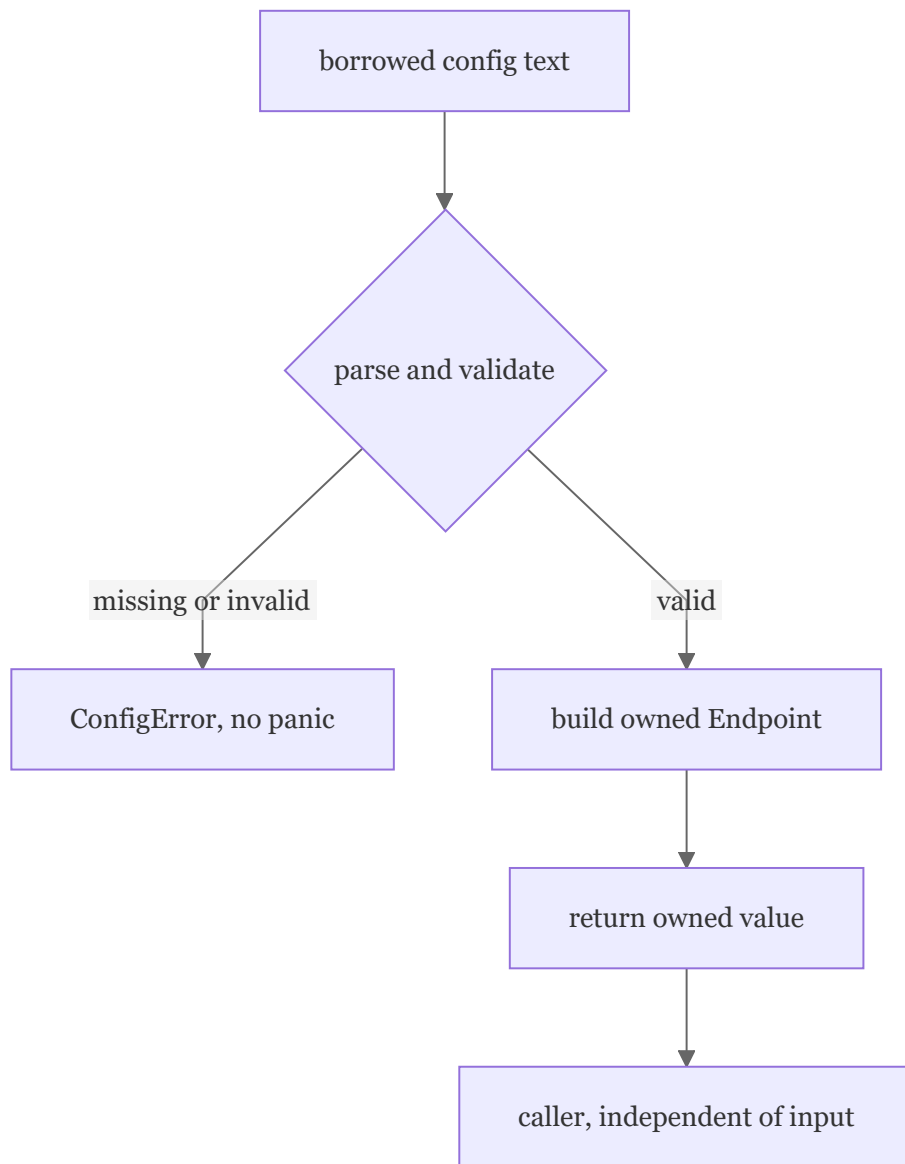
An idiomatic Rust refactor is not a contest to remove every `clone`, every trait object, or every async call. It is a contest to make the remaining ones obviously correct, operationally justified, and easy to review.

Examples

Example 1: replace panic-based loading with an owned Result boundary

The refactor keeps borrowed input local, moves normalized output into an owned struct, and turns routine failure into a small error enum.

What to look at: follow the data left to right. Borrowed config text comes in, parsing either fails with a named error or produces an owned endpoint, and that owned value is what leaves the function so the caller never depends on the input buffer.



Routine failure becomes a Result error and the normalized result is owned, so the signature alone tells the caller what they get and how it can fail.

Example 17-1. result_owned_boundary.rs — Owned api

```

#[derive(Debug)]
enum ConfigError {
    MissingPort,
    InvalidPort,
}

#[derive(Debug)]
struct Endpoint {
    service: String,
    bind: String,
}

impl Endpoint {
    fn requires_tls(&self) -> bool {
        self.bind.ends_with(":443")
    }
}
  
```

```

}

fn parse_port(raw: Option<&str>) -> Result<u16, ConfigError> {
    let text = raw.ok_or(ConfigError::MissingPort)?;
    text.parse::<u16>().map_err(|_| ConfigError::InvalidPort)
}

fn load_endpoint(service: &str, host: &str, port_raw: Option<&str>) -> Result<Endpoint, ConfigError> {
    let port = parse_port(port_raw)?;

    Ok(Endpoint {
        service: service.to_string(),
        bind: format!("{}", host, port),
    })
}

fn main() {
    let endpoint = load_endpoint("billing", "127.0.0.1", Some("443")).unwrap();

    println!("endpoint = {}@{}", endpoint.service, endpoint.bind);
    println!("requires tls = {}", endpoint.requires_tls());
}

```

OUTPUT

```

endpoint = billing@127.0.0.1:443
requires tls = true

```

Missing or invalid ports become ordinary Result errors, not panics or sentinel values.

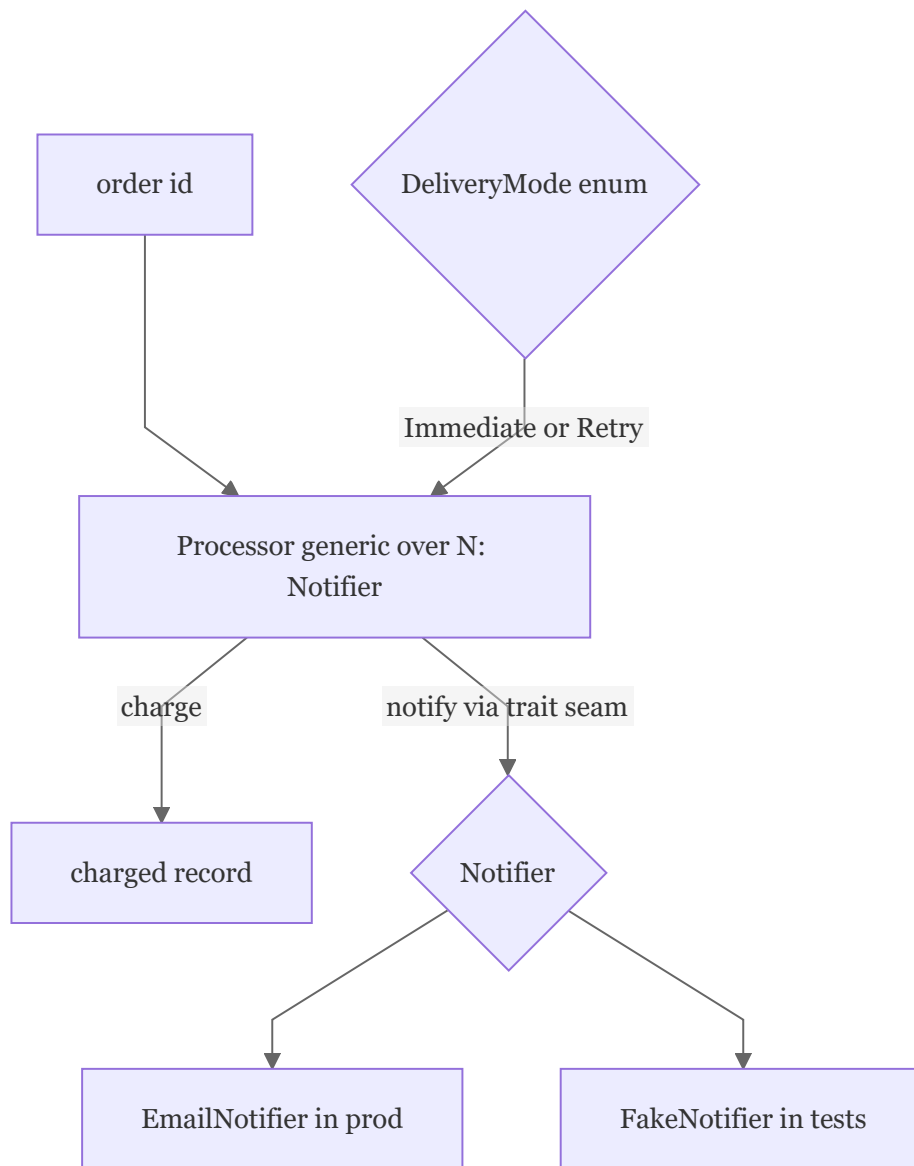
The returned endpoint owns its normalized bind address, so the caller is not coupled to input buffer lifetime.

The signature now says exactly what the caller gets and exactly how failure is represented.

Example 2: replace hierarchy pressure with enum state and a narrow test seam

A closed mode becomes an enum, a notifier becomes a small trait seam, and the processor becomes easy to test with a fake implementation.

What to look at: the old hierarchy splits into three parts. The closed delivery mode is an enum the processor matches on, the notifier is a small trait the processor is generic over, and a fake notifier can stand in for the real one in tests without touching anything else.



Closed state is an enum, behavior is a trait the processor is generic over, and the trait seam is where a test fake swaps in. Dispatch stays static because the concrete notifier is known at compile time.

Example 17-2. traits_enums_testable.rs — *Enums testable*

```

#[derive(Debug, Clone, Copy)]
enum DeliveryMode {
    Immediate,
    Retry,
}

trait Notifier {
    fn notify(&self, order_id: u64) -> String;
}

struct EmailNotifier;

impl Notifier for EmailNotifier {
    fn notify(&self, order_id: u64) -> String {
        format!("email:order-{}", order_id)
    }
}

```

```

    }
}

struct OrderProcessor<N> {
    notifier: N,
    processed: usize,
}

impl<N: Notifier> OrderProcessor<N> {
    fn new(notifier: N) -> Self {
        Self {
            notifier,
            processed: 0,
        }
    }

    fn process(&mut self, order_id: u64, mode: DeliveryMode) -> String {
        self.processed += 1;

        match mode {
            DeliveryMode::Immediate => format!("charged = order-{}", order_id),
            DeliveryMode::Retry => format!("queued = order-{}", order_id),
        }
    }

    fn notify(&self, order_id: u64) -> String {
        self.notifier.notify(order_id)
    }
}

fn main() {
    let mut processor = OrderProcessor::new(EmailNotifier);
    let order_id = 7_u64;

    println!("{}", processor.process(order_id, DeliveryMode::Immediate));
    println!("notified = {}", processor.notify(order_id));
    println!("processed = {}", processor.processed);
}

```

OUTPUT

```

charged = order-7
notified = email:order-7
processed = 1

```

The closed delivery mode is an enum, not a polymorphic class family with runtime type checks.

The notifier boundary is small enough to swap with a fake in tests without dragging in broad framework state.

The generic processor keeps dispatch static until a real runtime plugin boundary actually appears.

The repository also includes standalone Rust source under `examples/ch17_refactoring_toward_idiomatic_rust/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

CHAPTER 18 · GENERICS INSTEAD OF TEMPLATES

Generics Instead of Templates

Rust generics vs templates, monomorphization, trait bounds, where clauses, associated types, const generics, and performance tradeoffs

Generics let one Rust API serve many types without giving up concrete contracts. This chapter covers trait bounds, where clauses, associated types, const generics, and the monomorphization costs that matter when you ship a production library: what the compiler checks, when it checks it, and what it stamps into the binary.

Builds on Chapter 17

Chapter 17 was about refactoring concrete code toward idiomatic Rust. Generics are the next step: once a pattern recurs across more than one concrete type, you lift it into a single parameterized API. This chapter is deliberately about doing that lifting late and on purpose, not as a reflex.

Opening scenario

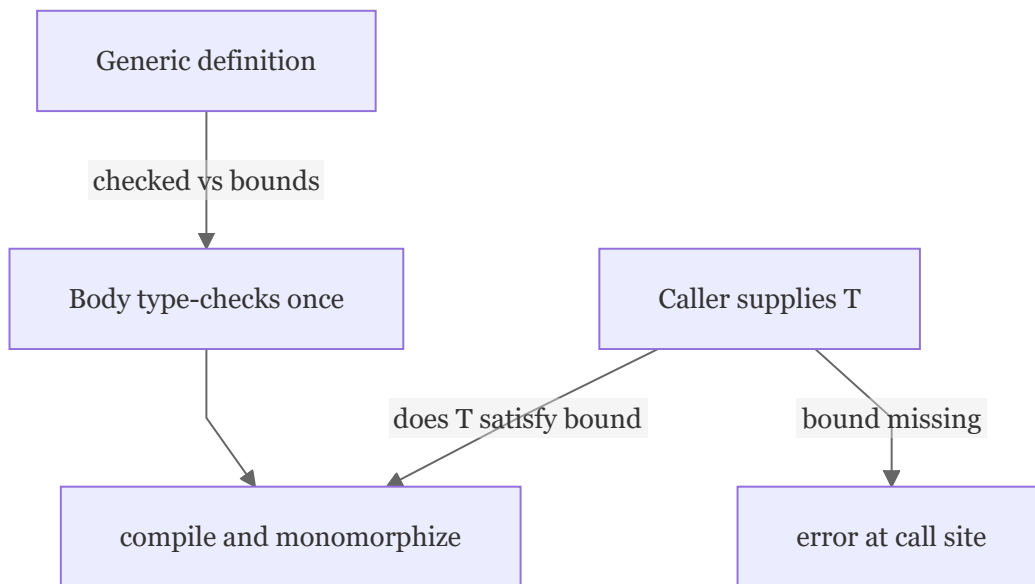
A team is building a job-processing library. They start with one concrete batch of Job values, then a second caller needs the same batching logic over a different item type, then a third needs a fixed-size encoding buffer whose length is known at compile time. The temptation is to copy the code three times or to reach immediately for a fully generic abstraction. The business requirement is a single API that compiles to the same machine code as the hand-written versions, rejects misuse at the call site with a clear message, and does not quietly bloat the binary.

A practical decision order

1. Write the concrete version first and ship it if there is only one caller.
2. When a second real type appears, lift to a generic and name the trait bound the body actually uses.
3. Pick associated types over extra parameters when the trait has one natural related type.
4. Use const generics only when the size is genuinely fixed at compile time.
5. Drop to dyn Trait when runtime polymorphism or binary size, not raw speed, is the constraint.

Mental model

The shape worth memorizing is two checks at two different times. A C++ template defers all type checking until a caller instantiates it, so a missing operation surfaces deep inside the body. A Rust generic is checked twice: once at the definition against its declared bounds, and once again, trivially, when a caller supplies a type and the compiler confirms that type satisfies the bound. The body never gets re-typechecked per caller, which is why the diagnostics point at the call site instead of the template internals.



The body is checked once against its bounds at the definition; each caller is only checked for whether its type satisfies the bound.

Core concepts

Trait bounds, where clauses, associated types, const generics

These four features form a ladder. A bare type parameter such as `Vec<T>` says nothing about `T`. Adding a bound `(T: Keyed)` lets the body call methods. A `where` clause keeps the bound legible as it grows. An associated type fixes a trait's one natural related type so callers do not repeat it, and a `const` parameter like `const N: usize` lets the type carry a compile-time size such as `[T; N]`.

Generics versus templates: where the difference lives

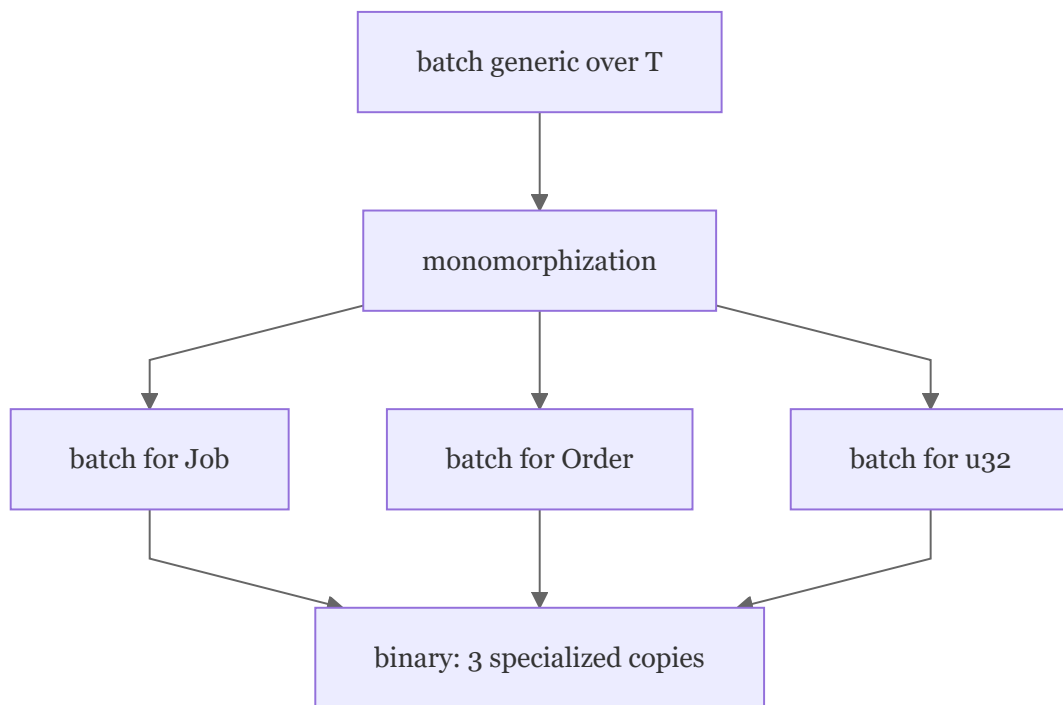
It is worth being precise about what is the same and what is different, because the runtime story is identical and only the front end changes. Both languages generate one specialized copy of the code per concrete type. The contrast is entirely in when type errors are reported and whether a contract has to be declared. The table below is the whole comparison in one view.

- No declared contract; the body just tries operations.
- Type-checked at instantiation, per caller.
- Errors surface inside the template body.
- Monomorphized to specialized machine code.
- Trait bounds declare the contract explicitly.
- Type-checked once at the definition against the bound.
- Errors surface at the call site as a missing trait.
- Monomorphized to specialized machine code.

Monomorphization and its costs

Monomorphization is the compiler stamping out a separate, fully specialized copy of a generic for each concrete type that uses it. That is what makes generics fast: the copy for `u32` knows it is operating on four-

byte integers and inlines accordingly, with no boxing and no vtable. The price is paid in the binary, not at runtime: one generic over many types becomes many functions in the output.



One generic becomes one specialized copy per concrete type. Runtime cost stays low; binary size and compile time scale with the number of instantiations.

Generics are free at runtime, not free at build time. A widely instantiated generic in a core crate can dominate compile time and binary size even when every call is as fast as hand-written code.

How this lands by background

Most engineers meet Rust generics with a model from another language, and the useful question is which part of that model transfers. The runtime behavior is rarely the surprise; the surprise is where and when the type contract is enforced, and whether you are allowed to leave it implicit.

Production patterns

Pitfalls and tradeoffs

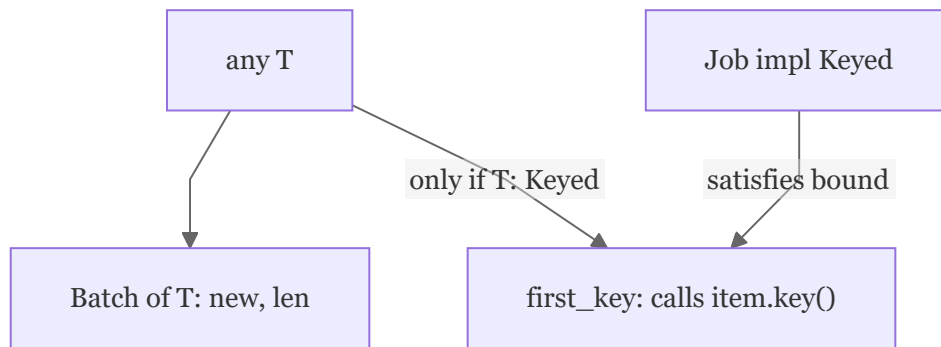
A generic is not automatically better than a concrete type. It is a contract you maintain and a set of instantiations you pay for. Generalize when a second real case proves the abstraction, not before.

Examples

Example 1: Generic batch with trait bounds

One `Batch<T>` holds any item type, but the helper that reads a key requires the items to implement a trait.

What to look at: notice which operations are gated by which bound. `Batch<T>` itself has no bound, so `new` and `len` work for every type. Only `first_key` adds where `T: Keyed`, which is what lets its body call `item.key()`. Trace the bound in the diagram, then read the same split in the code.



The container is unbounded; only `first_key` requires `T: Keyed`, so only it may call `key()`. `Job` satisfies the bound, so the call compiles.

Example 18-1. `generic_batch_bounds.rs` — Batch bounds

```

trait Keyed {
    fn key(&self) -> u64;
}

#[derive(Debug)]
struct Job {
    id: u64,
    name: String,
}

impl Keyed for Job {
    fn key(&self) -> u64 {
        self.id
    }
}

#[derive(Debug)]
struct Batch<T> {
    items: Vec<T>,
}

impl<T> Batch<T> {
    fn new(items: Vec<T>) -> Self {
        Self { items }
    }

    fn len(&self) -> usize {
        self.items.len()
    }
}

fn first_key<T>(batch: &Batch<T>) -> Option<u64>
where
    T: Keyed,
{

```

```

    batch.items.first().map(|item| item.key())
}

fn main() {
    let batch = Batch::new(vec![
        Job {
            id: 10,
            name: String::from("parse"),
        },
        Job {
            id: 22,
            name: String::from("persist"),
        },
    ]);

    println!("len = {}", batch.len());
    println!("first key = {}", first_key(&batch).unwrap());
}

```

OUTPUT

```

len = 2
first key = 10

```

`Batch<T>` stores any type; storage needs no trait.

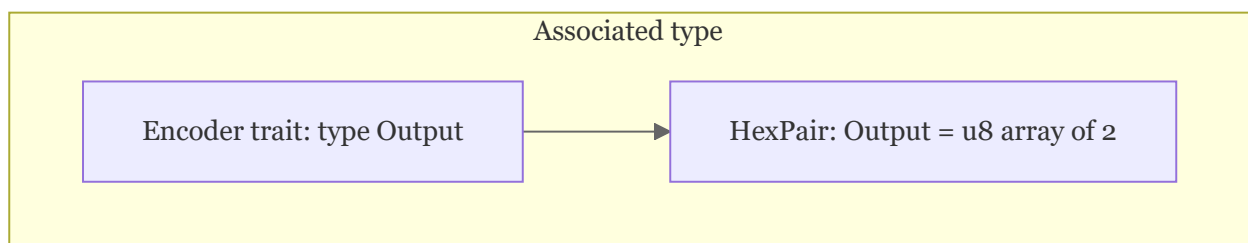
`first_key` requires `Keyed` because it calls a trait method.

Pass a type without `Keyed` and the error names the missing bound at the call.

Example 2: Associated types and const generics

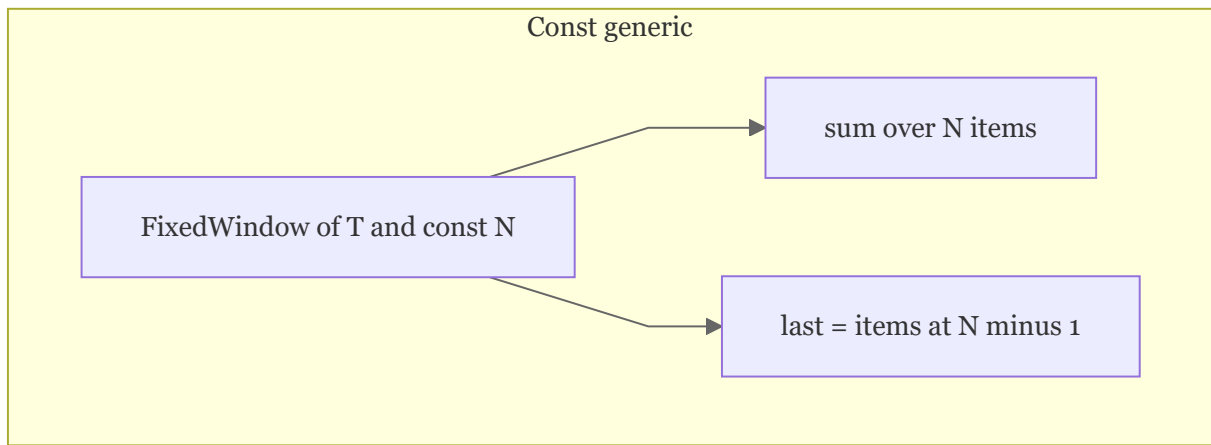
An `Encoder` fixes its output type as an associated type, and a fixed window carries its length as a `const` parameter.

What to look at: two different parameterization choices in one file. The `Encoder` trait owns its result type via type `Output`, so `HexPair` pins it to `[u8; 2]` once and callers never spell it out. Separately, `FixedWindow<T, N>` is generic over a compile-time length `N`, so `last()` can index `items[N - 1]` with the size known at compile time. The diagram separates the by-type axis from the by-value axis.



By type: an associated type lets the `Encoder` trait own its result type, so `HexPair` pins `Output` to a fixed array once and callers never spell it out.

The other axis parameterizes by a compile-time value rather than a type:



By value: a const generic makes the length N part of the type, so a `FixedWindow` sums over N items and indexes the last as `items[N - 1]` with the size known at compile time.

Example 18-2. `associated_types_const_generics.rs` — *Types const*

```

fn hex_digit(nibble: u8) -> u8 {
    match nibble {
        0..=9 => b'0' + nibble,
        10..=15 => b'A' + (nibble - 10),
        _ => b'?',
    }
}

trait Encoder {
    type Output;

    fn encode(&self, input: &[u8]) -> Self::Output;
}

struct HexPair;

impl Encoder for HexPair {
    type Output = [u8; 2];

    fn encode(&self, input: &[u8]) -> Self::Output {
        let byte = input[0];
        [hex_digit(byte >> 4), hex_digit(byte & 0x0F)]
    }
}

#[derive(Debug)]
struct FixedWindow<T, const N: usize> {
    items: [T; N],
}

impl<const N: usize> FixedWindow<u32, N> {
    fn sum(&self) -> u32 {
        self.items.iter().copied().sum()
    }

    fn last(&self) -> u32 {

```

```

        self.items[N - 1]
    }
}

fn main() {
    let encoder = HexPair;
    let encoded = encoder.encode(&[31_u8]);
    let window = FixedWindow {
        items: [3_u32, 5, 8, 13],
    };

    println!("hex = {}", std::str::from_utf8(&encoded).unwrap());
    println!("sum = {}", window.sum());
    println!("last = {}", window.last());
}

```

OUTPUT

```

hex = 1F
sum = 29
last = 13

```

type Output fixes one result type per implementor, so callers do not repeat it.

const N: usize makes the length part of the type, so [T; N] stays on the stack.

A length mismatch becomes a compile error, not a runtime bounds check.

Try changing the inputs to feel the two cost models. Edit the byte passed to encode or the numbers in items and the output follows directly; change the number of items and the const length N moves with it because the length is part of the type.

Next chapter

Generics let one type or function serve many types in memory. Chapter 19 takes the same data across process and version boundaries: serialization and data contracts, where the question shifts from "what can this type do" to "how does this value survive being written, stored, and read back by possibly older code." The bounds-and-contracts discipline from this chapter carries directly into designing stable wire formats.

Summary

CHAPTER 19 · SERIALIZATION AND DATA CONTRACTS

Serialization and Data Contracts

Serde fundamentals, wire formats, schema evolution, custom serializers, zero-copy deserialization, and boundary contracts for distributed systems, WASM, and FFI

Serialization defines the contract between Rust services, files, queues, and external clients. This chapter covers versioned payloads, schema ownership, compatibility, and explicit transport boundaries.

Builds on Chapters 08, 14, 16, and 18

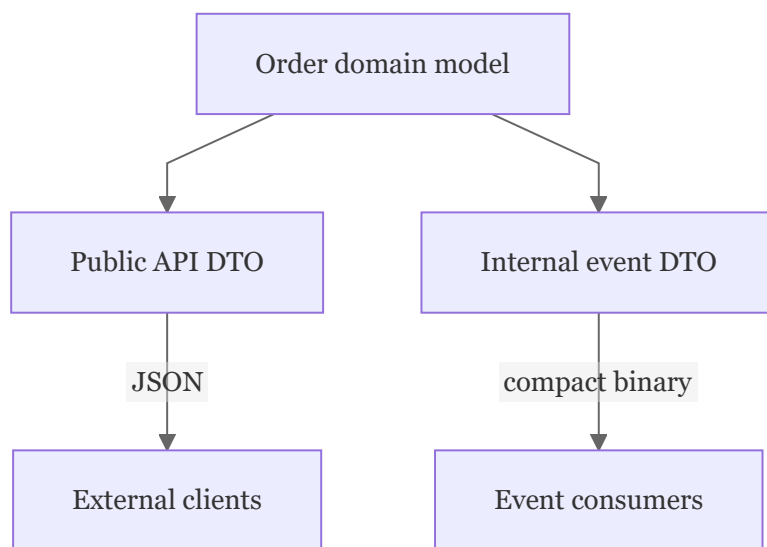
Chapter 08 established FFI and unsafe boundary discipline. Chapter 14 covered traits, which matter because Serde is trait-driven. Chapter 16 gave us domain events and aggregates worth serializing. Chapter 18 covered generics and associated types, which help explain why transport wrappers and wire DTOs stay composable.

Opening scenario

Serialization is where a Rust program stops being a self-contained world of owned values and starts being one participant in a larger system. Inside the process you have rich types, lifetimes, and a compiler that proves things hold together. The instant a value is written to a socket, a file, or a queue, all of that disappears, and what remains is bytes plus an implicit agreement about how to read them. That agreement is the data contract, and most serialization pain in production is really a contract that drifted without anyone deciding it should.

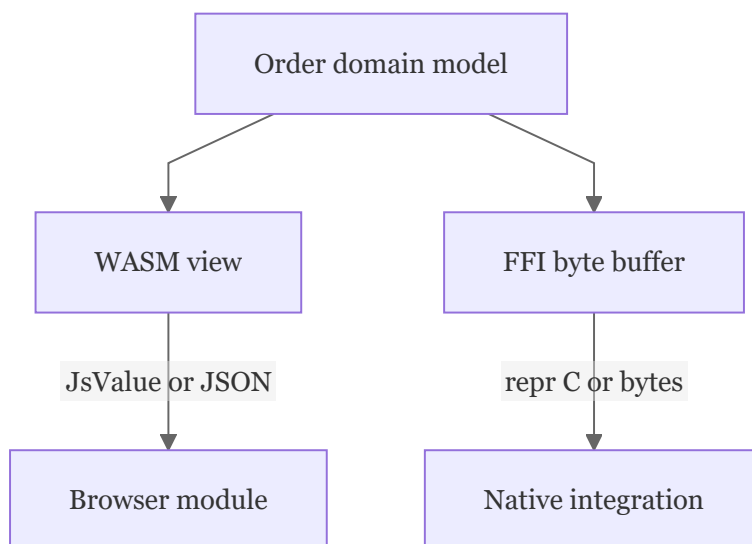
The running example for this chapter is an order system that emits the same domain events to four very different consumers. A public HTTP API needs human-readable, debuggable JSON. An internal event stream wants something compact and fast. A browser module across a WASM boundary needs a shape JavaScript can consume cheaply. A native integration across an FFI boundary needs an explicit byte contract rather than accidental Rust memory layout. One domain model, four boundaries, four trade-offs.

Look at where each contract lives in the diagram below: the domain model stays in the center, and each edge translates it into a format chosen for that consumer. The point is that no single edge dictates the shape of the core.



The first two boundaries: a human-readable JSON API and a compact internal event stream, both derived from the same core.

The same domain model also feeds two foreign-runtime boundaries:



The WASM and FFI boundaries, again derived from the same core. Four edges, four contracts, and none of them dictates the shape of the center.

Repository note

The in-browser editor simulates output so you can focus on contract shape. In a real Cargo project, these examples assume Serde plus the chosen format crate such as `serde_json`, a YAML/TOML adapter, or a binary format adapter.

Mental model

How this maps from other languages

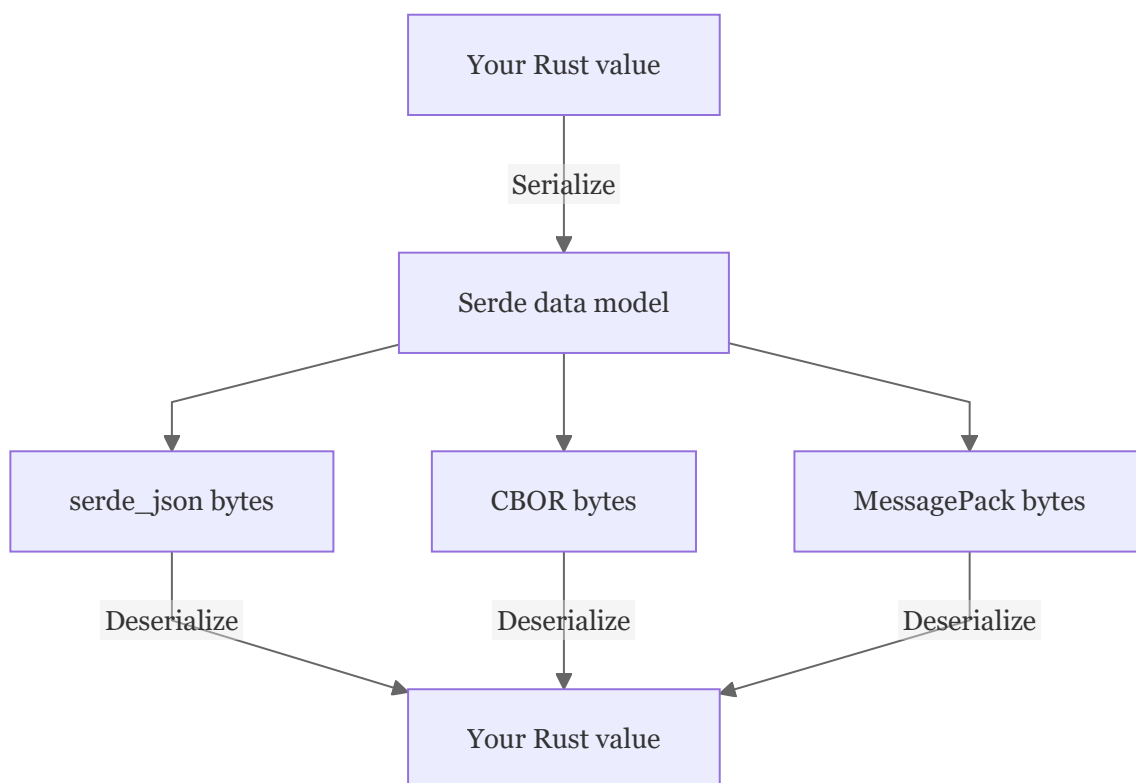
Every senior engineer arrives with serialization instincts that are mostly correct and quietly misleading in one specific spot. The list below names the mental-model shift, not a crate-to-library translation table. The recurring theme is the same one that runs through the rest of Rust: ownership and lifetime do not stop mattering at the boundary, and the wire shape is a contract you design rather than a side effect of your in-memory layout.

Core concepts

Serde: two traits and a format layer

Serde is built on two traits, `Serialize` and `Deserialize`, plus a clean split between your data model and the concrete format. Your type knows how to describe itself as a sequence of fields and values; a format crate such as `serde_json` or a binary adapter knows how to turn that description into bytes and back. This is why one derive can target JSON, YAML, MessagePack, or CBOR without rewriting the type: the type talks to an abstract data model, and the format sits underneath it.

The trace from a Rust value to bytes runs through this abstract model. Notice that the format is a swappable layer at the bottom, not something baked into your struct.



One Serialize/Deserialize implementation talks to an abstract data model; the concrete format is a layer you swap underneath it.

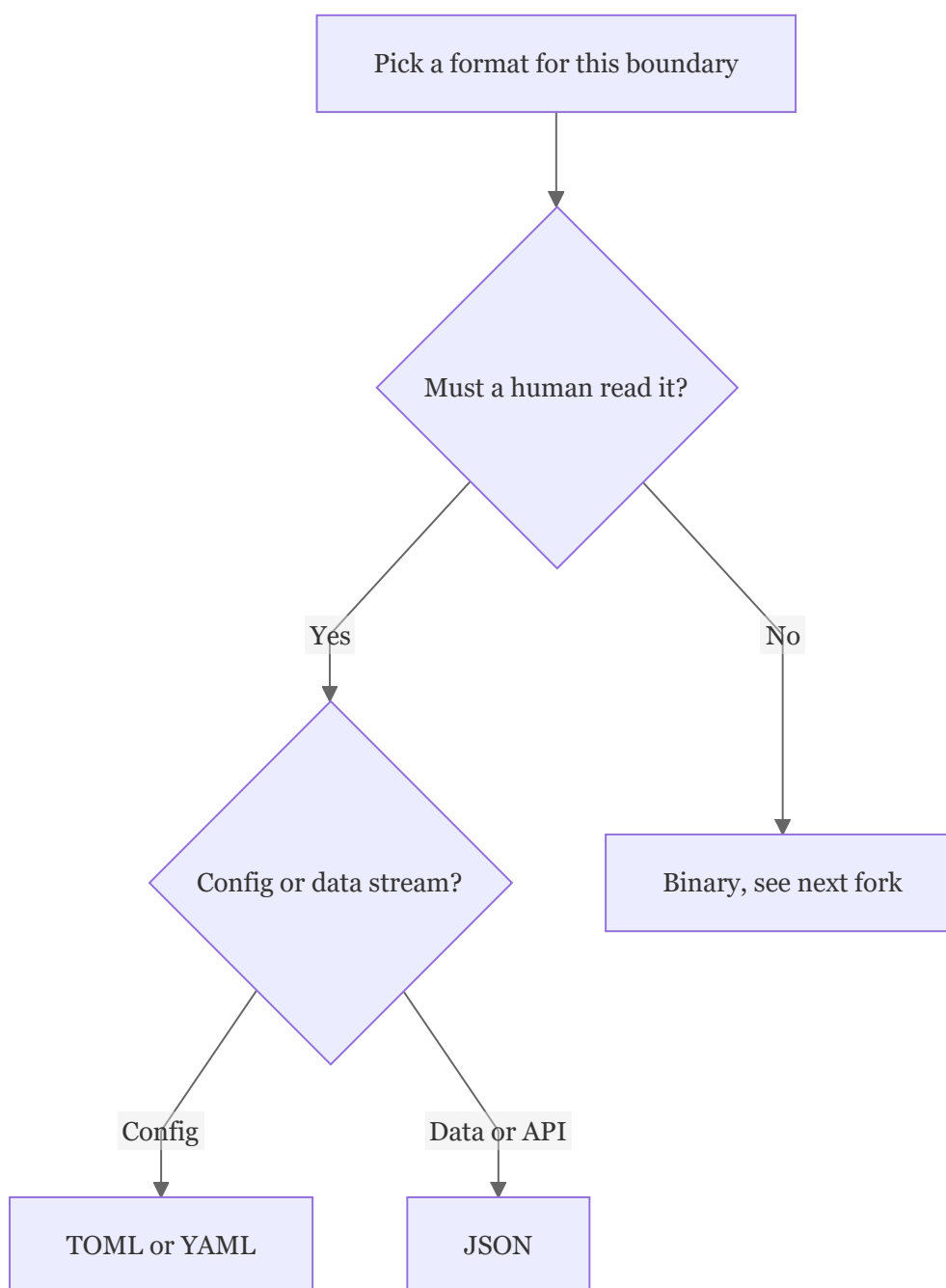
Derive macros generate the trait implementations, but the design still lives in the attributes: field names, enum tagging strategy, defaults, borrowing, and which model is even allowed to cross the boundary. A generic envelope is a common starting shape because it lets a stable wrapper carry many payload types without re-deriving the wrapper for each one.

A strong default is to separate internal domain types from wire-facing DTOs when evolution pressure differs. Your aggregate may want one shape. Your public event or API contract may want another. Serde makes the conversion pleasant, which means you do not have to force one type to do both jobs badly. The cost of a second type and a `From` impl is usually far lower than the cost of a domain refactor that silently changes a public payload.

JSON, YAML, TOML, MessagePack, and CBOR

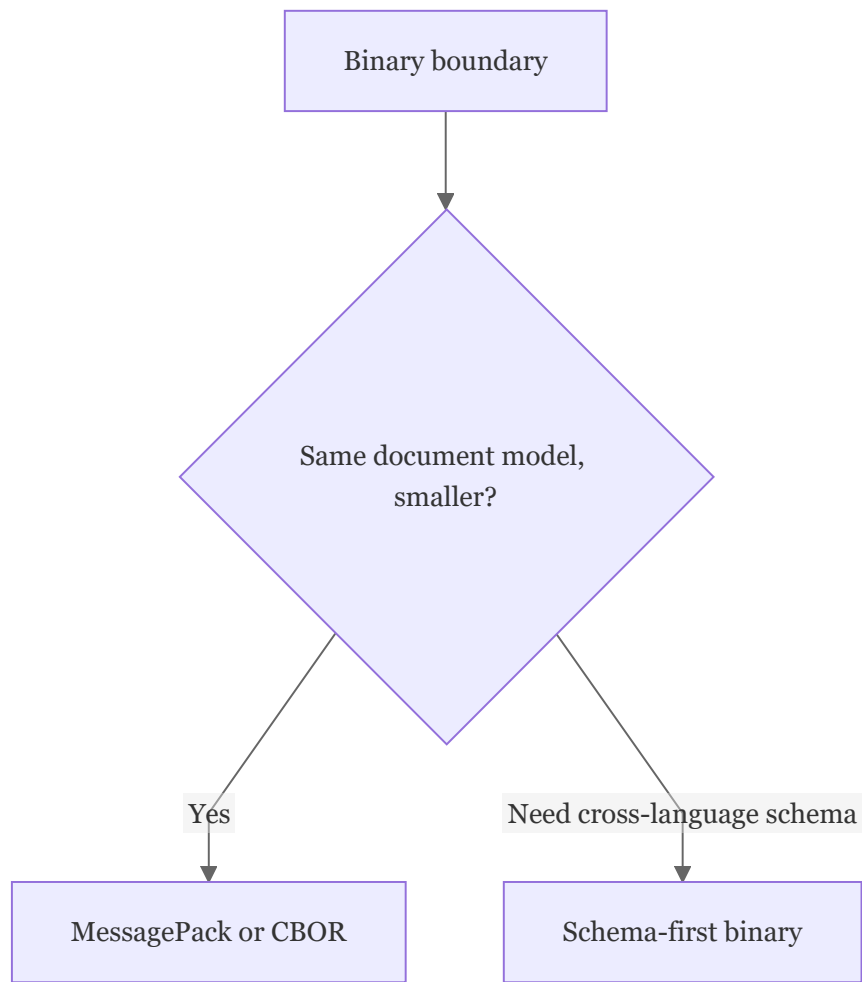
These five formats all map cleanly onto Serde's data model, so the choice between them is rarely about what Rust can express. It is about who reads the bytes and under what pressure. JSON, YAML, and TOML are text: easy to diff, log, and hand-edit, which matters enormously during an incident. MessagePack and CBOR are binary cousins of the same document model: smaller and faster to parse, at the cost of needing a tool to inspect. The decision usually collapses to two axes, human readability and payload size, with compatibility and tooling as tie-breakers.

Before the cards, look at the decision as a fork rather than a ranking. There is no single best format; there is a best format for a given boundary.



First fork: if a human must read the bytes, the answer is a text format keyed on config versus data.

When the boundary does not need to be human-readable, the binary branch forks again:



Second fork: a binary boundary chooses between a compact document format and a schema-first contract. The format question is keyed on who reads the bytes and how much size matters, not a single winner.

The useful decision is not “which format is best?” It is “which format fits this boundary’s human-readability, payload size, compatibility, and debugging needs?” A public API and an internal control-plane message do not have to make the same trade, and there is no rule that one service must speak only one format.

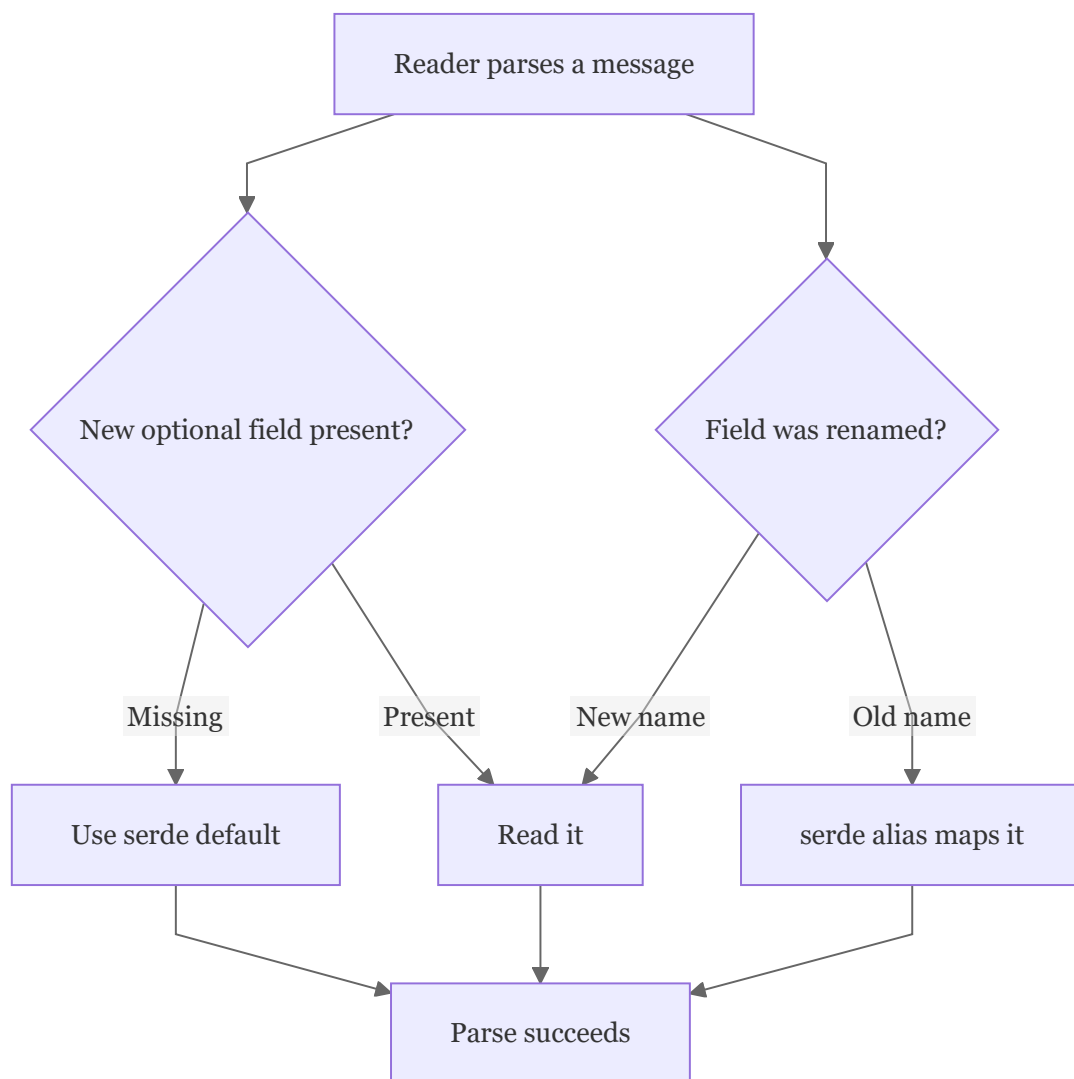
Binary serialization formats

Once a boundary no longer needs to be human-readable, a different set of formats opens up, and they sort roughly by how much they commit to in exchange for speed and size. Rust-centric compact encodings are the fastest to adopt and the least portable. Schema-first formats cost you an interface-definition step but buy you a contract that other languages can compile against. Archived, zero-copy representations go furthest: they let you read a value straight out of a mapped buffer without parsing, in return for the strictest layout and evolution discipline. The cards move left to right along that same axis of convenience versus cross-language and durability guarantees.

Schema evolution

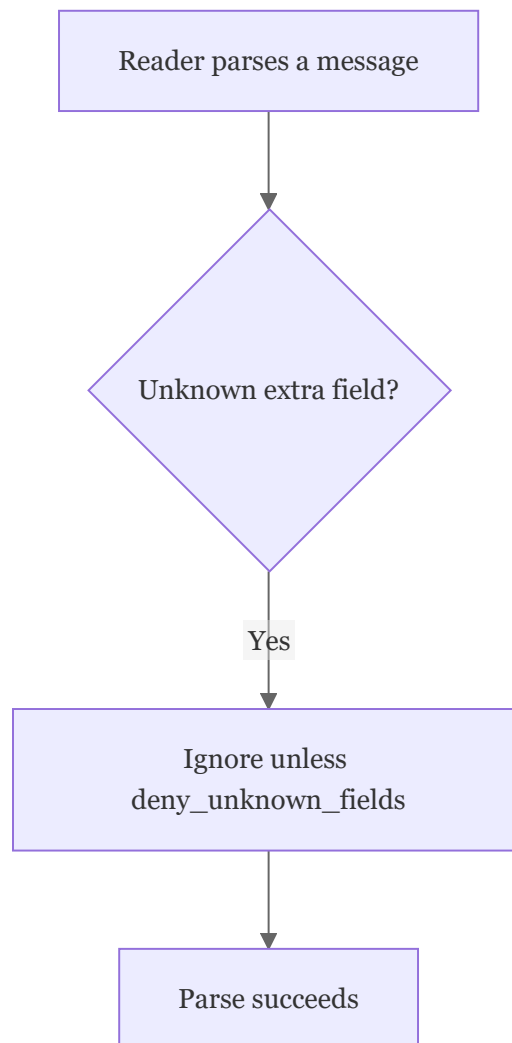
Contracts change, and the only question is whether old and new code can coexist during the change. The safe moves are the ones an old reader can survive without being redeployed first. Adding an optional field with a default is invisible to existing readers. Renaming a field is safe only if you keep the old name as an alias until every producer has moved. Changing the meaning of an existing field is the dangerous one, because it

compiles, deploys, and corrupts data silently. The diagram below traces what a tolerant reader does when it meets each kind of change.



Added and renamed fields are survivable: a missing optional field falls back to a default, and a renamed field is mapped by an alias.

The same tolerant reader also handles fields it does not recognize at all:



An unknown extra field is ignored unless `deny_unknown_fields` is set. The change that breaks a tolerant reader is repurposing an existing field, which has no safe edge in either half.

A compatibility break is not only a code change. It is a deployment coordination problem. Make the version boundary visible in data, tests, and logs before a mixed producer-consumer rollout forces emergency repair.

Version-tolerant models

Version tolerance is usually about reader discipline. Readers should survive additive fields, tolerate missing optional fields with defaults, and keep deprecated aliases around long enough to drain old producers safely.

A practical pattern is this: internal domain type on one side, wire DTO on the other, plus explicit conversion. That makes compatibility work visible and prevents one old wire compromise from contaminating the core model forever.

Custom serializers and deserializers

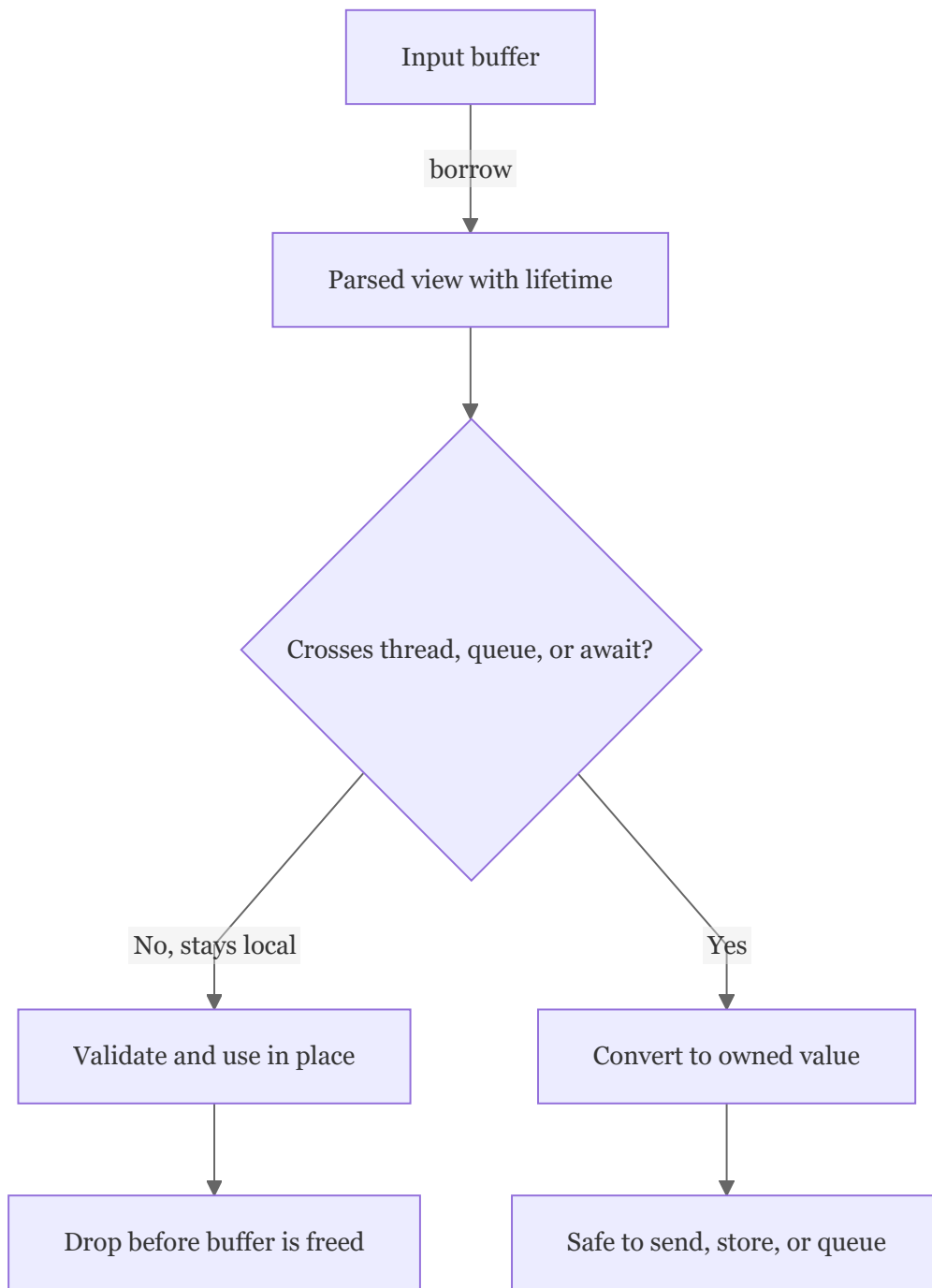
Custom serializers are for boundary-specific representation: money as decimal text, compact IDs, legacy booleans, or normalized timestamps. The goal is to keep the internal model honest while still meeting an external contract.

Custom logic belongs at the edge. Do not store a decimal money string in the domain only because one API contract needed it. Keep the internal unit cheap and precise, then translate at serialization time.

Be careful with naive split-on-dot money parsing: it is a classic correctness trap. The fractional part must be validated as exactly two digits, or scaled by its length, before being added to the whole part. A reader that accepts "12.5" as 1205 cents has silently lost money.

Zero-copy deserialization

Some formats let a deserialized value borrow directly from the input buffer instead of copying each string out. A field typed as `&str` or `Cow<'de, str>` can point into the bytes you parsed, which avoids allocation and copying on a hot path. The catch is pure Rust: the borrowed view cannot outlive the buffer it points into. That is fine for parse-time validation, and wrong the moment the value needs to live on a queue, in a cache, or across an await point. The decision is therefore about lifetime, and the diagram makes the fork explicit.



Borrow while the buffer is alive and the work stays local; convert to owned the moment the value must outlive the buffer or cross a boundary.

The operational rule is simple. Borrow while parsing if that reduces copying locally. Own the data once the message crosses a wider subsystem boundary.

Serialization for distributed systems

Across a network, the contract is no longer a convenience; it is the interface. Producers and consumers deploy on different schedules, so at any instant an old producer may be sending messages to a new consumer or the reverse. The patterns below all serve one goal: make the contract and its version explicit in the data, in the tests, and in the logs, so that a version skew is something you detect rather than something that detects you in production.

Serialization for WASM and FFI boundaries

Not every boundary is a network, but every boundary still has a contract. At the WASM edge you are handing data to JavaScript, and the choice is whether to translate into a structured JS value or pass a text or binary payload that JavaScript parses. At the FFI edge you are handing data to C, and the cardinal rule is to never let Rust's in-memory layout become the contract by accident: a struct's field order and padding are implementation details until you pin them with `repr(C)` or replace them with an explicit byte buffer and length. The honest move at either edge is to decide what the foreign side actually wants and give it exactly that.

Production patterns

Pitfalls and tradeoffs

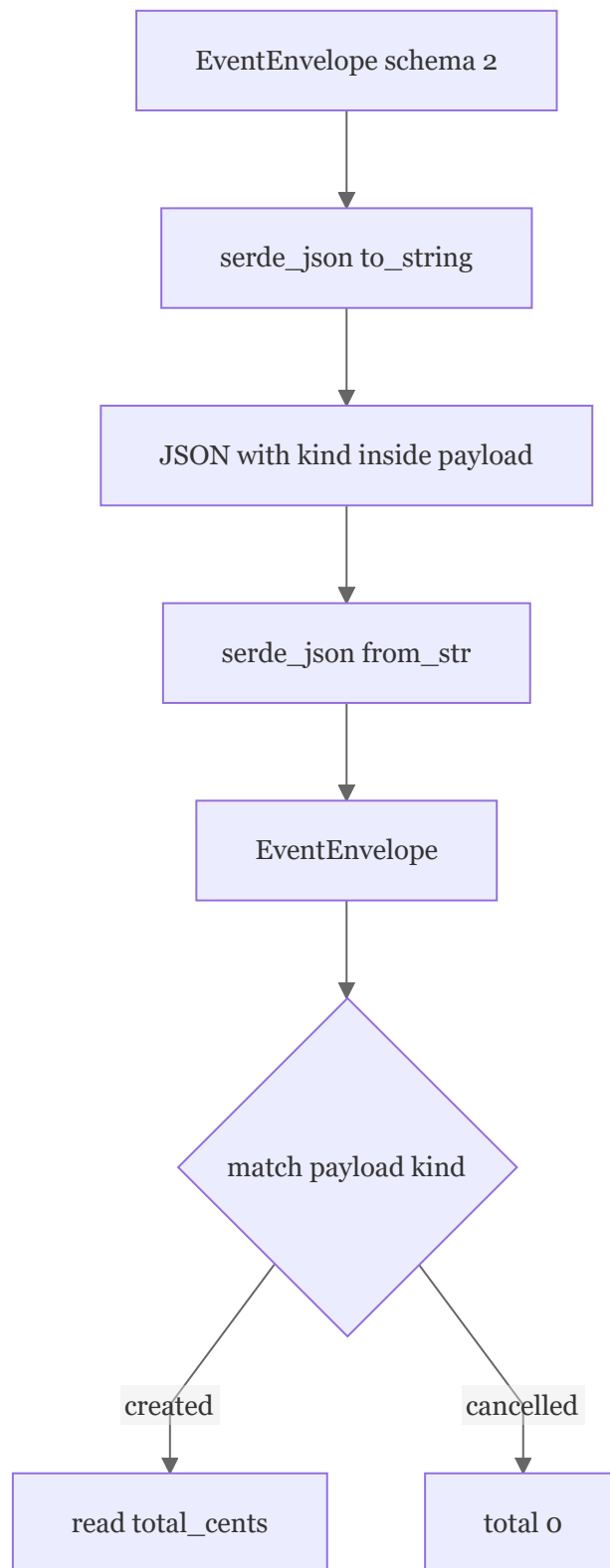
The most expensive serialization bug is often not a parser crash. It is a silent contract drift between producer and consumer that still compiles, still deploys, and only breaks when mixed versions meet.

Examples

Example 1: a versioned domain event envelope

The envelope carries explicit versioning, and the event payload stays tagged so readers can tell what happened without guessing from field shape.

What to look at: the envelope wraps the payload and carries the version, while the inner enum is internally tagged with `#[serde(tag = "kind")]` so the kind travels inside the JSON object rather than being inferred from which fields happen to be present. Trace the round trip below before reading the code: serialize to JSON, parse back, then match on the tag.



The version rides on the envelope and the kind rides inside the tagged payload, so a reader recovers both from the bytes without guessing.

Example 19-1. versioned_domain_event.rs — Versioned event

```

use serde::{Deserialize, Serialize};

#[derive(Debug, Serialize, Deserialize, PartialEq)]
#[serde(tag = "kind", rename_all = "snake_case")]

```

```

enum OrderEvent {
    Created {
        order_id: String,
        customer_id: String,
        total_cents: u64,
    },
    Cancelled {
        order_id: String,
        reason: Option<String>,
    },
}

#[derive(Debug, Serialize, Deserialize, PartialEq)]
struct EventEnvelope {
    schema_version: u16,
    #[serde(default)]
    trace_id: Option<String>,
    event_id: String,
    payload: OrderEvent,
}

fn main() {
    let envelope = EventEnvelope {
        schema_version: 2,
        trace_id: None,
        event_id: String::from("evt-100"),
        payload: OrderEvent::Created {
            order_id: String::from("ord-7"),
            customer_id: String::from("cust-9"),
            total_cents: 4200,
        },
    };

    let json = serde_json::to_string(&envelope).unwrap();
    let decoded: EventEnvelope = serde_json::from_str(&json).unwrap();

    let kind = match &decoded.payload {
        OrderEvent::Created { .. } => "created",
        OrderEvent::Cancelled { .. } => "cancelled",
    };

    let total_cents = match decoded.payload {
        OrderEvent::Created { total_cents, .. } => total_cents,
        OrderEvent::Cancelled { .. } => 0,
    };

    println!("schema = {}", decoded.schema_version);
    println!("kind = {}", kind);
    println!("total cents = {}", total_cents);
}

```

OUTPUT

```

schema = 2
kind = created
total cents = 4200

```

schema_version and event ID make the contract visible to code and logs.

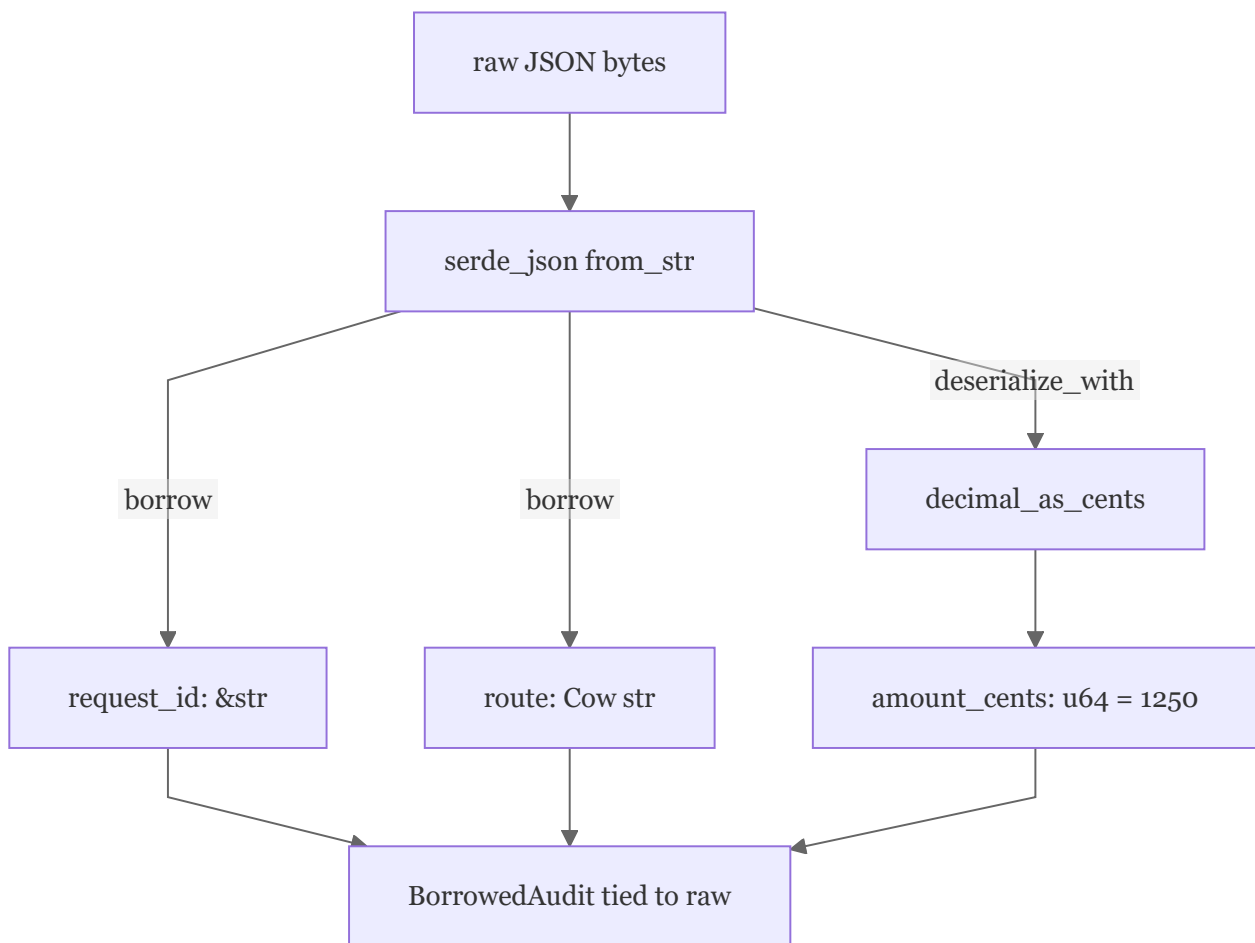
The payload declares its kind explicitly instead of forcing consumers to infer it from a field pattern.

Optional envelope fields can be added with defaults without destabilizing older readers immediately.

Example 2: custom money format plus borrowed fields

The wire format wants decimal text. The internal model wants cents. The deserialized view borrows request text until the boundary decides whether ownership is needed.

What to look at: two boundary concerns sit side by side on one struct. The amount_cents field routes through a custom serializer pair so the wire sees decimal text while the domain keeps an exact integer, and the string fields are typed to borrow from the input buffer. Follow how the raw JSON splits into a borrowed view plus a converted integer below.



The custom deserializer converts '12.50' into 1250 cents, while the string fields stay borrowed from the raw buffer the BorrowedAudit is tied to.

Example 19-2. custom_serializer_and_zero_copy.rs — Zero copy

```

use std::borrow::Cow;

use serde::{Deserialize, Deserializer, Serialize, Serializer};

fn cents_as_decimal<S>(value: &u64, serializer: S) -> Result<S::Ok, S::Error>
where
    S: Serializer,

```

```

{
    let text = format!("{}", value / 100, value % 100);
    serializer.serialize_str(&text)
}

fn decimal_as_cents<'de, D>(deserializer: D) -> Result<u64, D::Error>
where
    D: Deserializer<'de>,
{
    let text = Cow::<str>::deserialize(deserializer)?;
    let (units, cents) = text
        .split_once('.')
        .ok_or_else(|| serde::de::Error::custom("expected decimal amount"))?;

    if cents.len() != 2 {
        return Err(serde::de::Error::custom(
            "expected exactly two fractional digits",
        ));
    }

    let whole = units.parse::<u64>().map_err(serde::de::Error::custom)?;
    let frac = cents.parse::<u64>().map_err(serde::de::Error::custom)?;

    Ok(whole * 100 + frac)
}

#[derive(Debug, Serialize, Deserialize)]
struct BorrowedAudit<'a> {
    #[serde(borrow)]
    request_id: &'a str,
    #[serde(borrow)]
    route: Cow<'a, str>,
    #[serde(
        serialize_with = "cents_as_decimal",
        deserialize_with = "decimal_as_cents"
    )]
    amount_cents: u64,
}

fn main() {
    let raw = "{\"request_id\":\"req-7\",\"route\":\"/checkout\",\"amount_cents\":\"12.50\"}";
    let audit: BorrowedAudit<'_> = serde_json::from_str(raw).unwrap();

    println!("request = {}", audit.request_id);
    println!("route = {}", audit.route);
    println!("amount cents = {}", audit.amount_cents);
}

```

OUTPUT

```

request = req-7
route = /checkout
amount cents = 1250

```

The boundary gets decimal text without forcing the domain to store money as strings.

The request ID and route can borrow from the input buffer while parsing stays local.

The next subsystem can still choose to own the fields before queueing or storing the event.

The repository also includes standalone Rust source under `examples/ch19_serialization_and_data_contracts/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

- Serde removes boilerplate, but the real work is still contract design.
- Whatever language you came from, the shift is the same: the wire shape is a contract you design, and ownership and lifetime still matter at the boundary.
- JSON, YAML, TOML, MessagePack, CBOR, and binary formats make different tradeoffs across readability, size, and interoperability.
- Schema evolution is easiest when you prefer additive changes, defaults, aliases, and explicit envelopes.
- Custom serializers protect the internal model from transport-specific compromises.
- Zero-copy deserialization is useful at local parse boundaries, but owned handoff is usually better at wider system boundaries.
- WASM and FFI edges need explicit contracts just as much as distributed systems do, even when the transport is not a network protocol.

CHAPTER 20 · METAPROGRAMMING

Metaprogramming

Declarative and procedural macros, token streams, compile-time DSLs, code generation strategies, and when not to use macros

Metaprogramming in Rust is a build-time tool for removing repetitive syntax while preserving clear APIs. This chapter treats macros and generated code as maintainable compile-time infrastructure: when each kind earns its place, how expansion fits into the compiler pipeline, and where a plain function, generic, or offline generator is the calmer answer.

Builds on Chapters 03, 14, 18, and 19

Chapter 03 matters because proc macros and larger code generation strategies live at crate and build-system boundaries. Chapter 14 matters because many derive macros emit trait impls. Chapter 18 matters because macros and generics solve different problems. Chapter 19 matters because derive and attribute macros often sit directly on serialization and contract surfaces.

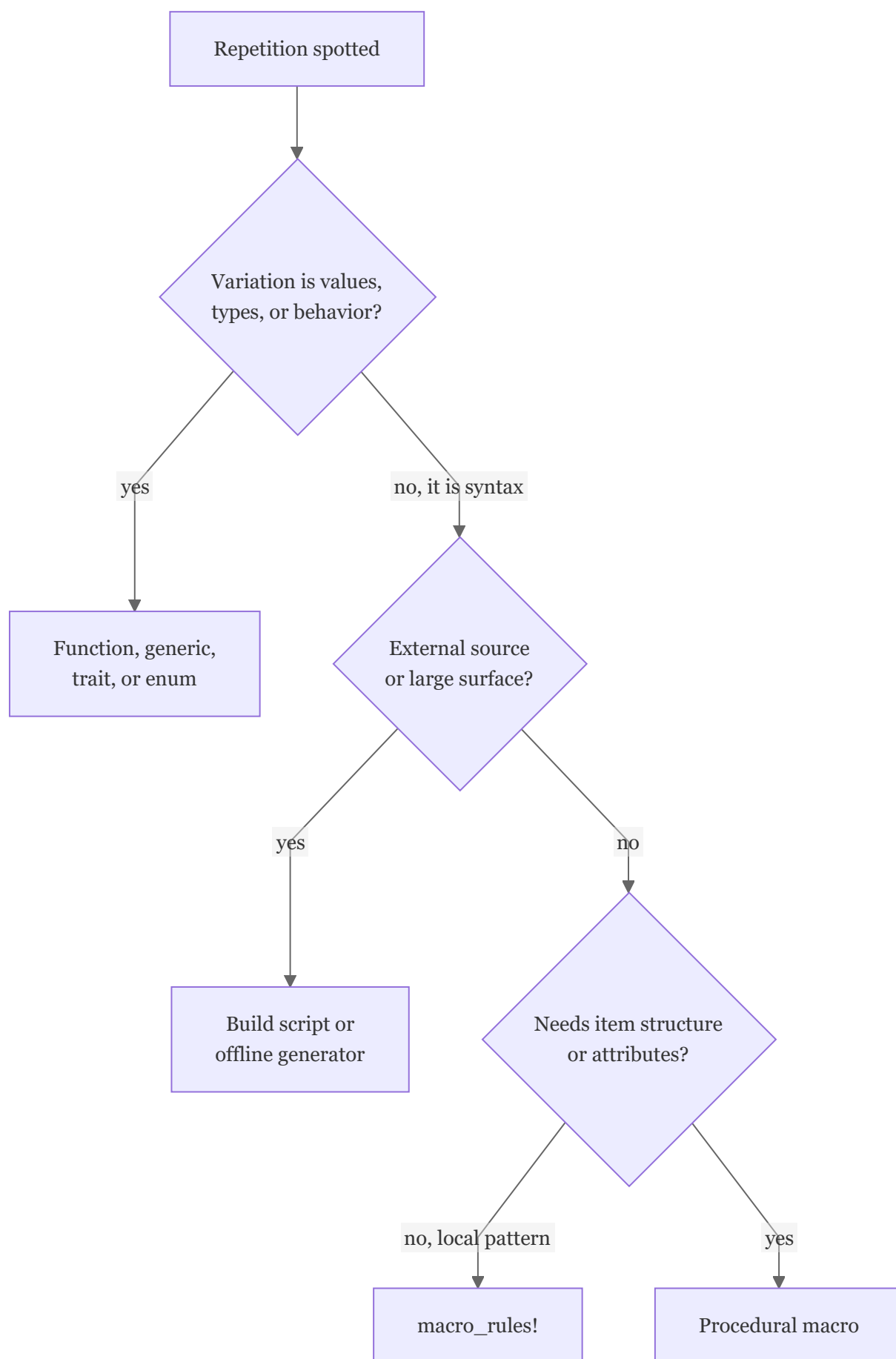
Opening scenario

Picture a service platform that has grown a familiar set of repetitions: route declarations that all follow the same method-path-auth shape, DTO metadata duplicated next to every struct, validation boilerplate copied across handlers, and adapter glue generated by hand from an external schema. Each of these looks like a candidate for metaprogramming, but they are not the same kind of problem, and the same tool does not fit all four. The goal of this chapter is to remove the structural repetition without hiding runtime behavior behind decorative syntax.

The four mechanisms map onto four distinct shapes of duplication. Plain functions, generics, and traits handle anything where the variation is in values, types, or behavior. Declarative macros handle small, local syntax patterns. Procedural macros handle item-level generation that has to read the structure of a type or an attribute. Build-time codegen handles surfaces whose source of truth lives outside Rust entirely. The skill is matching the shape, not reaching for the most powerful tool by reflex.

A practical decision order

Read the decision as a ladder you climb only when the rung below genuinely cannot hold the weight. Each step up buys expressive power and pays for it in compile time, diagnostics, and the size of the API surface you commit to keeping stable.



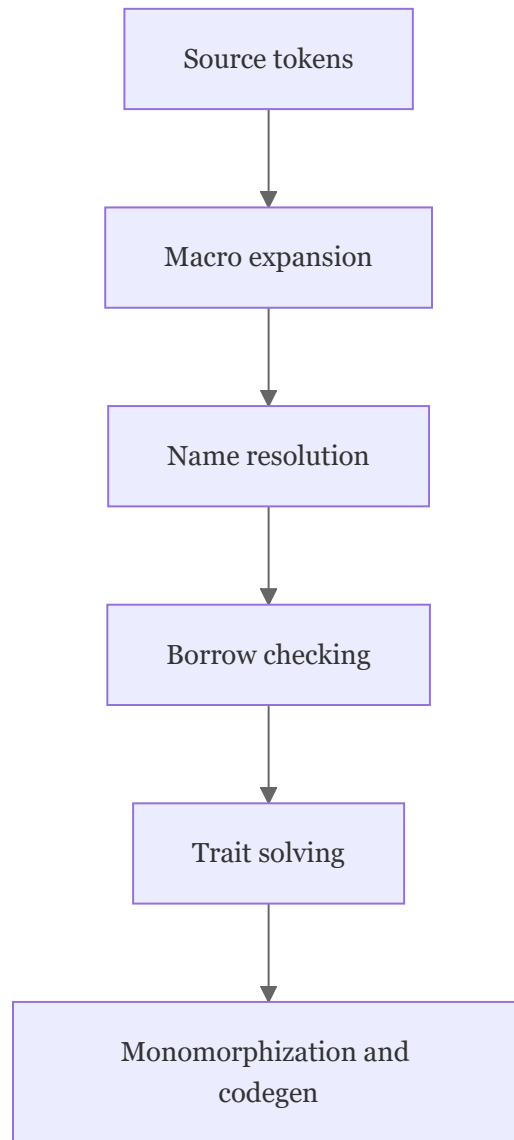
The decision ladder: most repetition is not syntactic at all. Only after ruling out functions, generics, and offline generation do the two macro forms come into play.

1. Start with a normal function, generic, trait, or enum and confirm why it is insufficient.
2. If the duplication is syntax-shaped inside Rust, try `macro_rules!` first.
3. If the API must attach to items, inspect attributes, or emit impls from type definitions, consider a procedural macro.
4. If the source of truth is external or the generated surface is large, prefer a build script or offline generator.

Mental model

Where expansion sits in the pipeline

The single fact that explains most macro behavior is ordering. Expansion runs near the front of compilation, and every safety check you rely on runs on the output. A macro can decide what code exists; it cannot decide what that code is allowed to do.



Macros transform tokens at the first stage. Borrow checking and trait solving run afterward on the expanded code, exactly as if you had written it by hand.

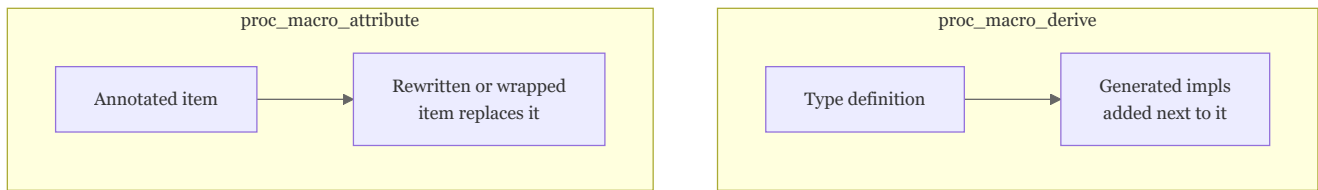
Core concepts

Declarative macros with `macro_rules!`

One correction to keep in mind: `macro_rules!` is not a type system escape hatch. It is a token-pattern expansion tool. Borrow checking, trait solving, and monomorphization still happen after expansion.

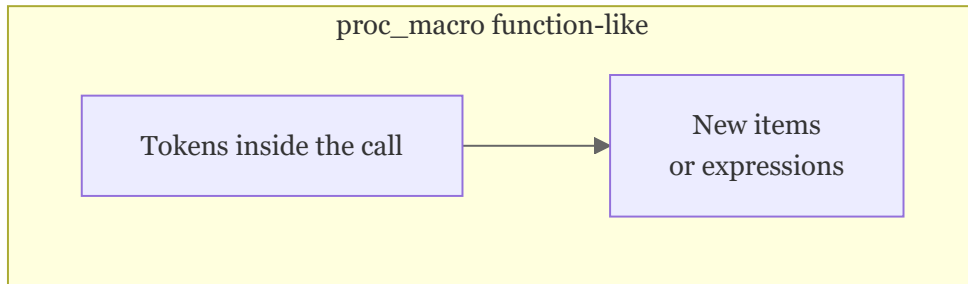
Procedural macros

Procedural macros are Rust functions that run at compile time, take a token stream in, and return a token stream out. They come in three forms, distinguished by what they are attached to and what they receive. The shape below is worth fixing in your head before reading the cards: a `derive` sees a copy of the type and adds alongside it, an `attribute` receives the item and replaces it, and a `function-like macro` receives whatever tokens sit inside its delimiters.



The two item-attached forms: a derive leaves the type intact and adds impls beside it, while an attribute consumes the item it sits on and replaces it with a rewritten version.

The third form is not attached to an item at all. It receives whatever tokens sit inside its call delimiters:

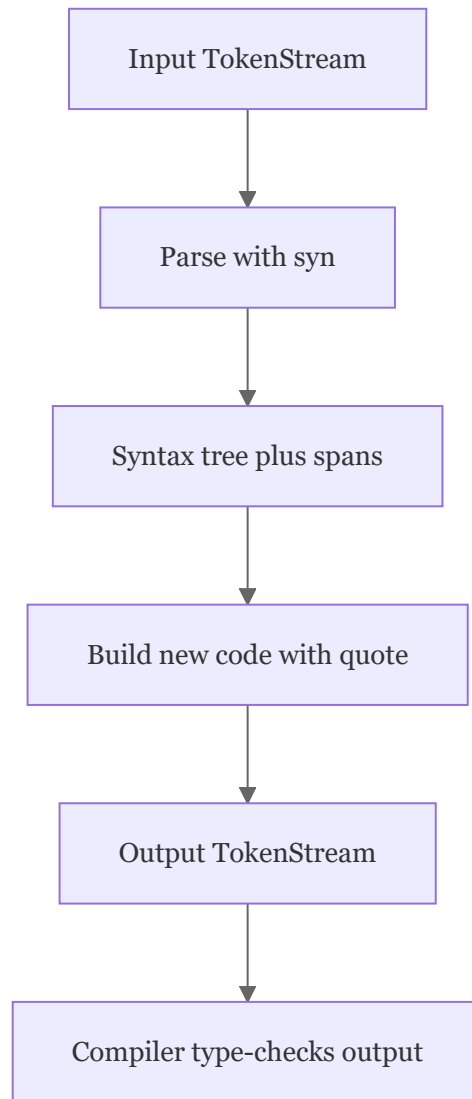


The function-like form: it expands an arbitrary token blob into new items or expressions, with no surrounding item to inspect.

Procedural macros are powerful because they can inspect and rewrite item-level syntax. They are also heavy because they add a crate boundary, compile-time work, and a bigger diagnostic surface. Reach for them only when that power is actually needed.

Token streams

A procedural macro never sees your program as values or as a resolved type graph. It sees a `TokenStream`: a flat sequence of identifiers, literals, punctuation, and grouped sub-streams, each carrying a span that points back at the original source. Most real proc macros parse that stream into a syntax tree with the `syn` crate, build new code with `quote`, and convert back to a token stream for the compiler. The diagram shows that round trip.



The typical proc-macro round trip: tokens in, parse to a syntax tree, generate new tokens, hand them back to the compiler for the usual checks.

One practical consequence follows immediately. Because the macro only ever holds syntax, it cannot consult inferred types or resolved trait impls to decide what to generate: that information simply does not exist yet at expansion time. If a design depends on knowing a field's concrete type or whether some trait is implemented, the macro is fighting the pipeline, and a different approach will be calmer.

Code generation strategies

Macro hygiene

Compile-time DSLs

A route table macro, a compact compile-time query wrapper, or a repetitive test-case generator can be excellent. A DSL that hides ownership, control flow, or expensive runtime work behind decorative syntax is usually not.

When not to use macros

If you are coming from another language

Every mainstream language has some form of metaprogramming, and that prior experience is the most common source of wrong instincts here. The cards below name the one mental-model shift each background tends to need. The through-line is that Rust does this work at compile time, on syntax, and feeds the result back through the same type checker that judges hand-written code.

The shared correction across all four: do not pick a macro because the borrow checker, the trait system, or type design felt inconvenient. A macro can change the syntax you write, but it cannot exempt the resulting code from any of those rules. When the friction is about ownership or types, the fix lives in ownership or types, not in a layer that runs before they are even checked.

Production patterns

Pitfalls and tradeoffs

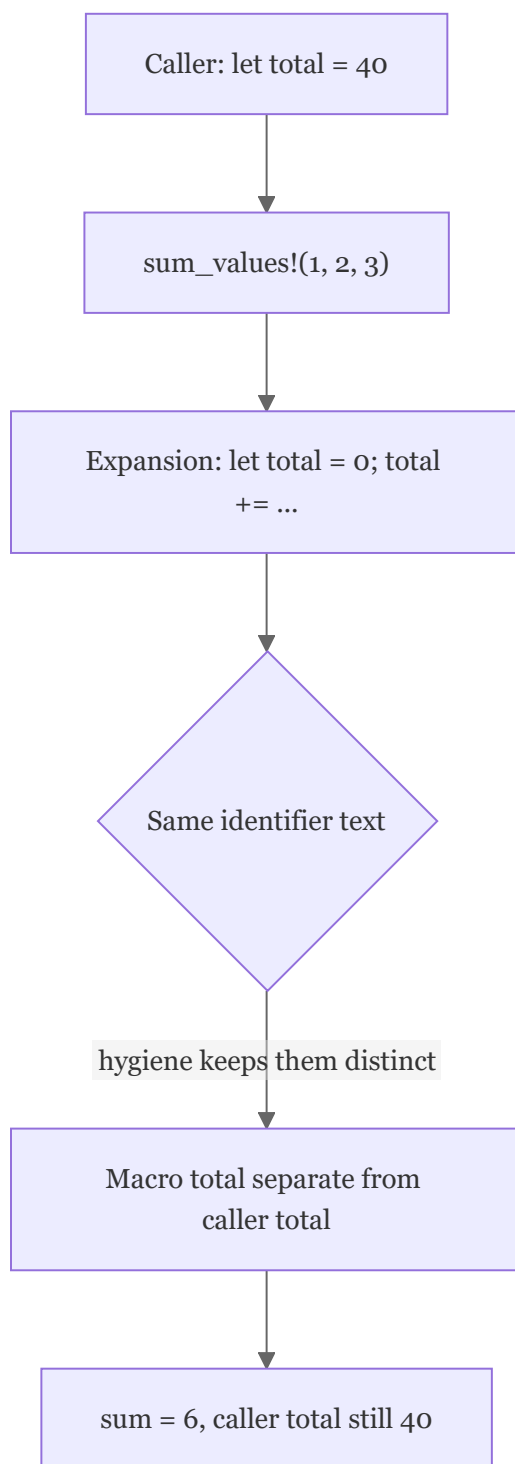
Macros compress source code, but they can expand maintenance cost. If the generated surface, diagnostics, and review burden are worse than the original repetition, the metaprogramming gave you nothing.

Examples

Example 1: `macro_rules!` repetition plus hygiene

One macro introduces a local binding, another expands a repeated list. The caller keeps its own total untouched.

What to look at: both the caller and the `sum_values!` macro define a binding called `total`, yet the final outer `total = 40` proves they never collide. Hygiene gives the macro's internal name a different identity from the caller's, so expansion cannot accidentally overwrite a variable that happens to share a name.



*The macro's internal total and the caller's total share spelling but not identity.
Hygiene keeps the expansion from clobbering the caller's binding.*

Example 20-1. macro_rules_hygiene_and_repetition.rs — Rules hygiene

```
macro_rules! add_one {  
    ($value:expr) => {{  
        let temp = $value;  
        temp + 1  
    }};  
}
```

```
macro_rules! sum_values {
    ($($value:expr),* $(,)? ) => {{
        let mut total = 0;
        $(
            total += $value;
        )*
        total
    }};
}

fn main() {
    let total = 40;
    println!("next = {}", add_one!(total));
    println!("sum = {}", sum_values!(1, 2, 3));
    println!("outer total = {}", total);
}
```

OUTPUT

```
next = 41
sum = 6
outer total = 40
```

`$( $value:expr ),*` removes the repeated accumulator code at the call site.

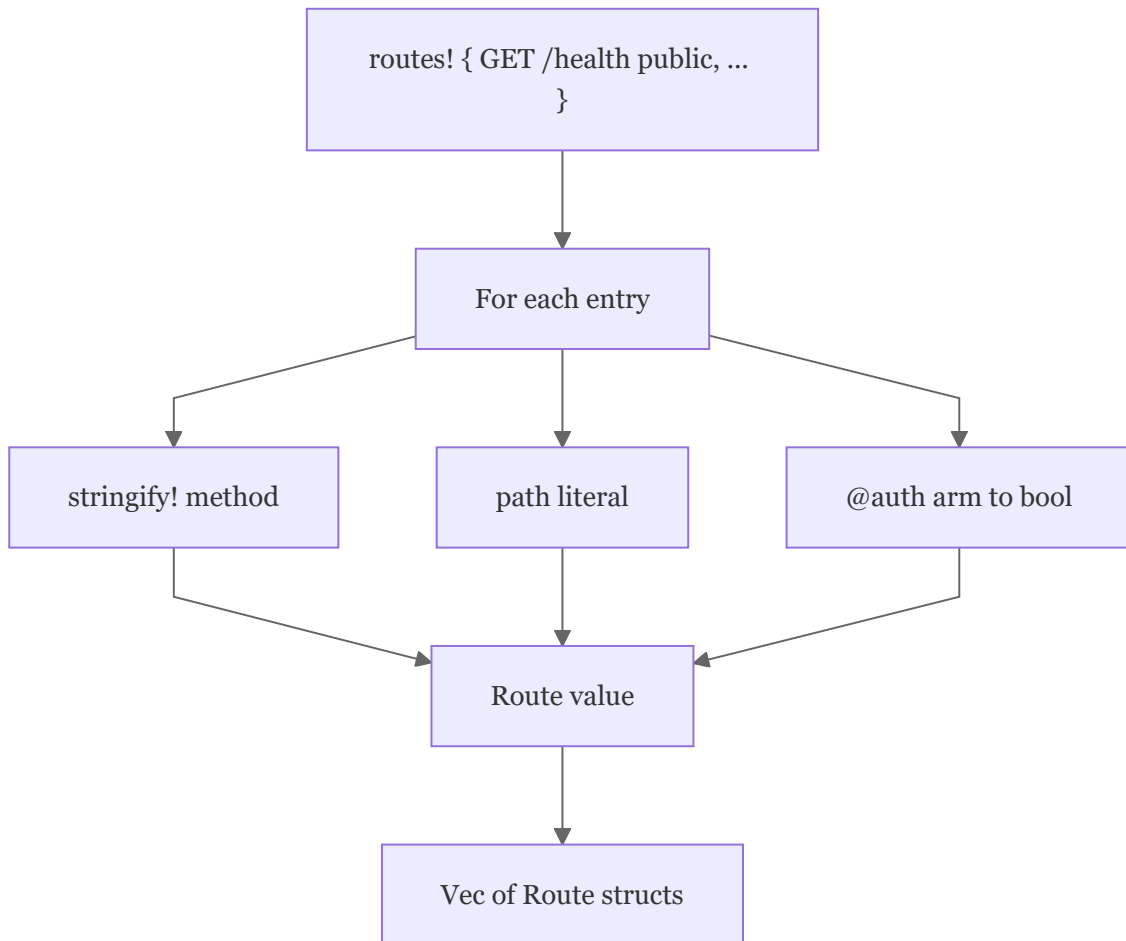
The macro's internal bindings do not steal the caller's `total` name.

This is the right scale for `macro_rules!`: small syntax compression, ordinary emitted Rust.

Example 2: a small compile-time DSL for route tables

The macro call site reads like a compact route declaration, but it still expands into ordinary Rust data with explicit fields.

What to look at: each line in the macro call is one route, and the recursive `@auth` arms turn the public and private markers into ordinary booleans. The whole DSL expands to nothing more exotic than a `Vec<Route>` of plain structs, which is exactly why it stays reviewable.



Each DSL entry expands into one Route struct, and the call as a whole becomes a plain vector of explicit values with no hidden IO or allocation policy.

Example 20-2. `compile_time_dsl_routes.rs` — Time dsl

```

#[derive(Debug)]
struct Route {
    method: &'static str,
    path: &'static str,
    auth: bool,
}

macro_rules! routes {
    ($($method:ident $path:literal => $auth:ident ),* $(,)? ) => {{
        vec![
            $(
                Route {
                    method: stringify!($method),
                    path: $path,
                    auth: routes!(@auth $auth),
                }
            ),*
        ]
    }};
    (@auth public) => { false };
    (@auth private) => { true };
}

```

```
fn main() {
    let table = routes!(
        GET "/health" => public,
        POST "/orders" => private,
        DELETE "/orders/:id" => private,
    );

    let private_count = table.iter().filter(|route| route.auth).count();

    println!("routes = {}", table.len());
    println!("first = {} {}", table[0].method, table[0].path);
    println!("private = {}", private_count);
}
```

OUTPUT

```
routes = 3
first = GET /health
private = 2
```

The grammar is intentionally tiny: method, path, and one auth marker. Small DSLs survive reviews better.

The expansion is only a vector of `Route` values. No hidden IO, no hidden allocation policy surprise.

If this grammar grew much larger, a proc macro or an offline generator might become the better tool.

The repository also includes standalone Rust source under `examples/ch20_metaprogramming/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor. The runnable editors focus on `macro_rules!` because real procedural macros require a separate proc-macro crate.

Summary

- `macro_rules!` is the lightest metaprogramming tool when token-pattern expansion is enough.
- Procedural macros operate on token streams in a separate proc-macro crate and are best for derive, attribute, and token-level DSL use cases.
- Macro hygiene helps, but public macros still create a public syntax contract that must be maintained carefully.
- Compile-time DSLs should stay small and expand into understandable Rust data or items.
- When the abstraction is about values, types, or behavior instead of syntax, a function, generic, trait, enum, or build script is usually the better tool.

CHAPTER 21 · REFLECTION AND TYPE INTROSPECTION

Reflection and Type Introspection

Why Rust limits runtime reflection, Any and TypeId, downcasting, macro-based introspection, schema generation, and explicit metadata systems

Runtime introspection is limited in Rust, so production systems need explicit metadata and generated descriptors. This chapter covers trait-based inspection, registries, and schema surfaces.

Builds on Chapters 14, 19, and 20

Chapter 14 introduced traits and trait objects. Chapter 19 covered schema and wire contracts. Chapter 20 covered compile-time metaprogramming. Reflection and introspection in Rust sit across those boundaries.

Opening scenario

A service platform needs typed request extensions, plugin capability metadata, and admin-facing schema descriptors. The business requirement is narrow introspection: use `TypeId` and `Any` only for type-keyed registries, expose plugin metadata as ordinary trait methods, and generate schema artifacts where field structure must be documented.

A practical decision order

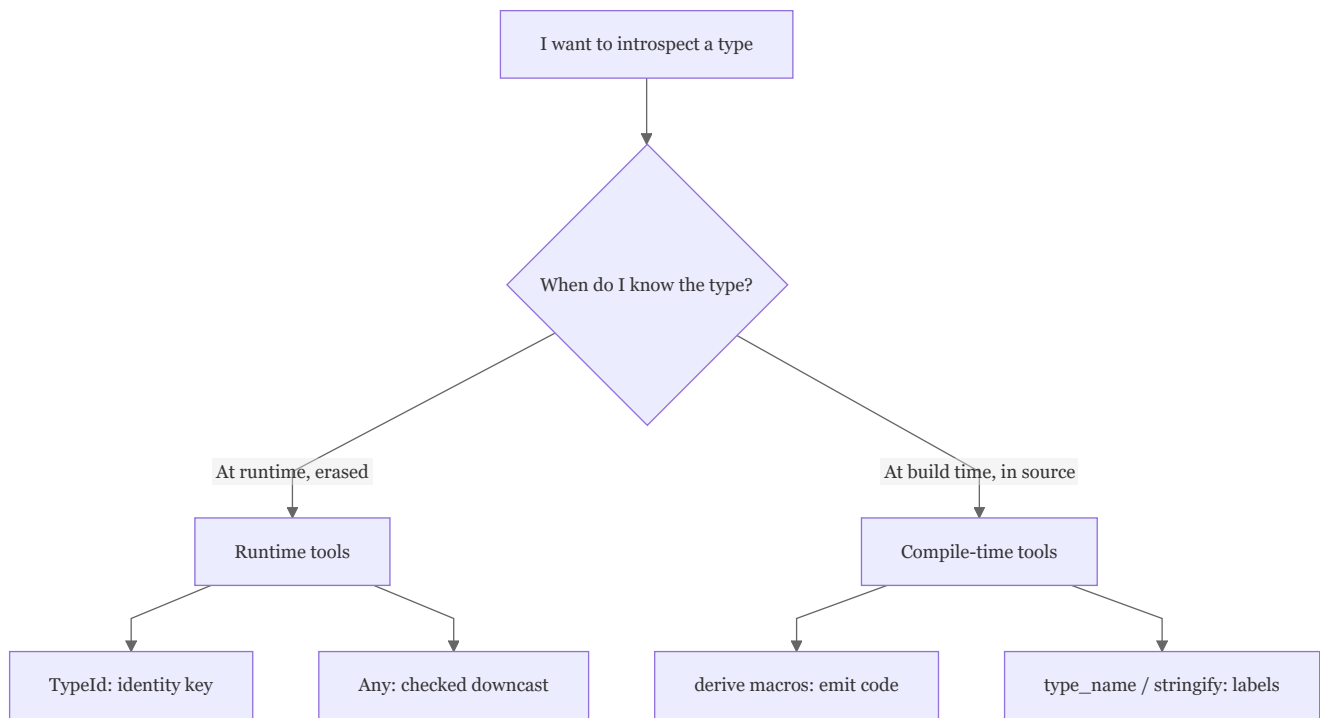
1. Start with ordinary structs, enums, and trait methods.
2. If one container truly must store heterogeneous values, consider `Any` plus `TypeId`.
3. If the source code already knows the shape, prefer macros or schema generation at compile time.
4. If operators or plugins need metadata, make that metadata an explicit API.

Mental model

Core concepts

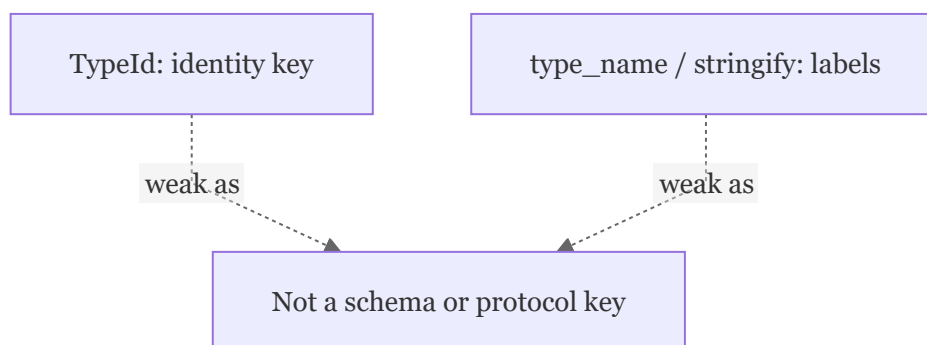
The four tools, and what each one is for

It helps to see the whole landscape before the details, because “reflection” in Rust is not one feature but four narrow tools that solve four different problems. Two of them work at runtime and two of them work at compile time. The runtime pair, `TypeId` and `Any`, gives you type identity and checked recovery of a value you erased on purpose. The compile-time pair, `derive` macros and compiler-built labels such as `type_name`, lets the source code that already knows a type’s shape emit new code or descriptors before the program runs. Almost every real need maps onto one of these four; the trap is reaching for a fifth that does not exist.



The first question is when you know the type: erased at runtime, or visible in source at build time. Each answer leads to its own pair of tools.

Two of those four tools are routinely misused as stable identifiers. The second half marks which ones:



TypeId and the compiler-built labels are process-local or diagnostic only; neither is a stable schema or protocol key, so the dotted edges warn against persisting them.

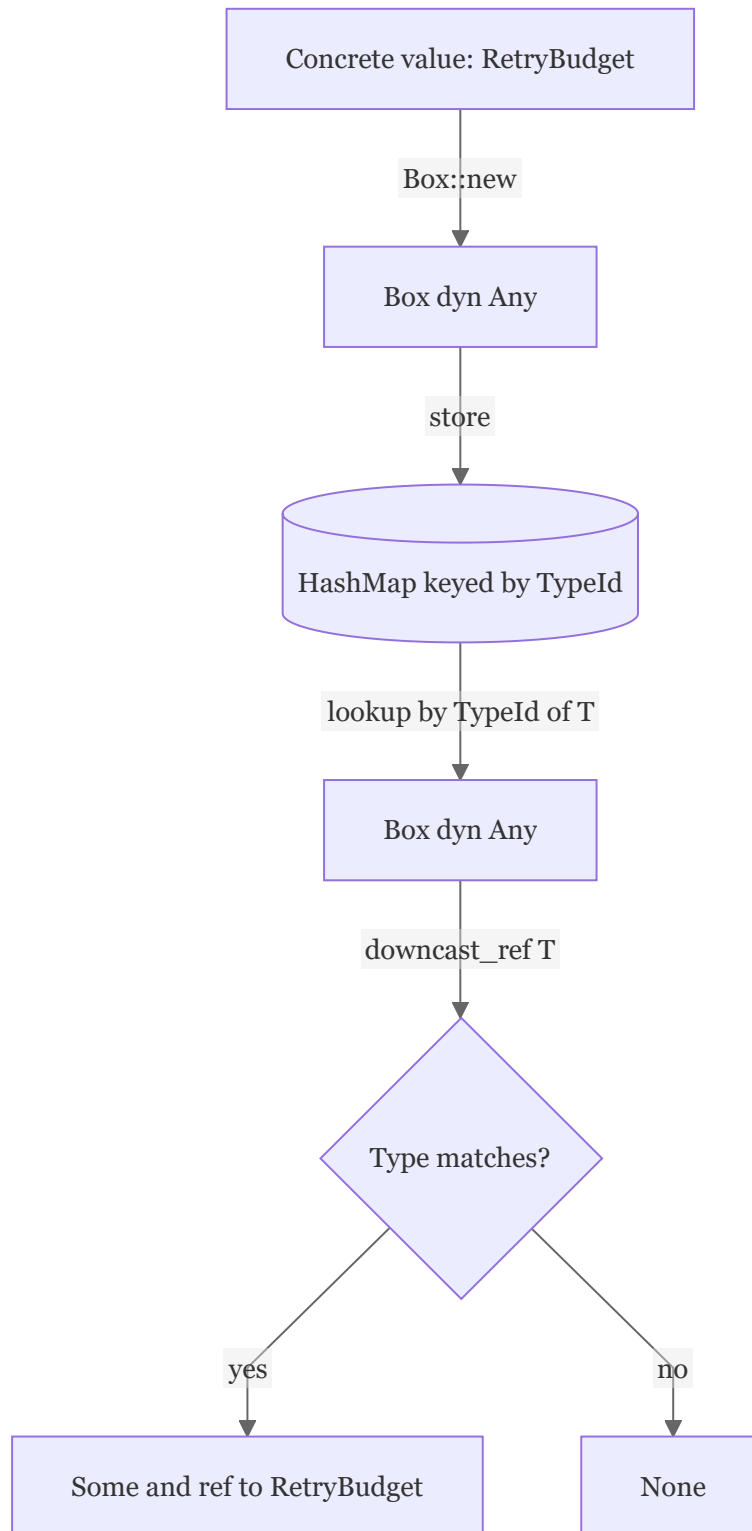
Why Rust has limited runtime reflection

The absence of broad reflection is a design choice, not a missing feature. Rust specializes generic code by monomorphization and resolves most calls statically, which means the per-type metadata a reflective runtime would need to keep alive has usually been compiled away by the time the program runs. Carrying it anyway would add weight to every binary and pull assumptions that belong in source into runtime tables. The practical consequence is the three cards below: there is no field walk, layout and codegen stay explicit, and the needs people actually have turn out to be narrower than “reflect over anything.”

Any and TypeId: erase a type, then recover it safely

TypeId is a small, copyable token that uniquely identifies one concrete type, valid within a single process and a single build of your program. Any is the trait that lets you store a value behind an erased pointer such as

`Box<dyn Any>` and later ask, at runtime, “is the thing in here actually a `RetryBudget`”? The two work together: you erase a value to put it in a heterogeneous container, you key or check it by `TypeId`, and you recover it as exactly one named type. The recovery is always checked, so a wrong guess returns `None` rather than reinterpreting bytes. One constraint shapes all of this: `Any` is for ‘static types, because a runtime identity cannot encode a borrow that only lives for part of the program.



A value is erased into `Box<dyn Any>`, stored under its `TypeId`, and only ever comes back as the one concrete type you ask for; a mismatch is `None`, never a reinterpretation.

A key correction for reflection-heavy backgrounds: `TypeId` is runtime type identity, not a schema, not a field list, and not a stable public protocol key.

Downcasting: recovering one named type by borrow, by mut, or by value

Downcasting is the act of recovery, and it comes in three forms that mirror Rust's ownership choices. The form you pick follows how the erased container holds the value and what the caller needs to do with it: read it, mutate it in place, or take ownership back. None of these is a cast in the C-language sense; each one checks the type first and only succeeds if it matches.

Downcasting is precise and explicit. You only recover a concrete type you already know how to name. That is why it is useful as an edge tool and weak as a general application architecture.

Trait-object-based reflection patterns

Compile-time introspection through macros

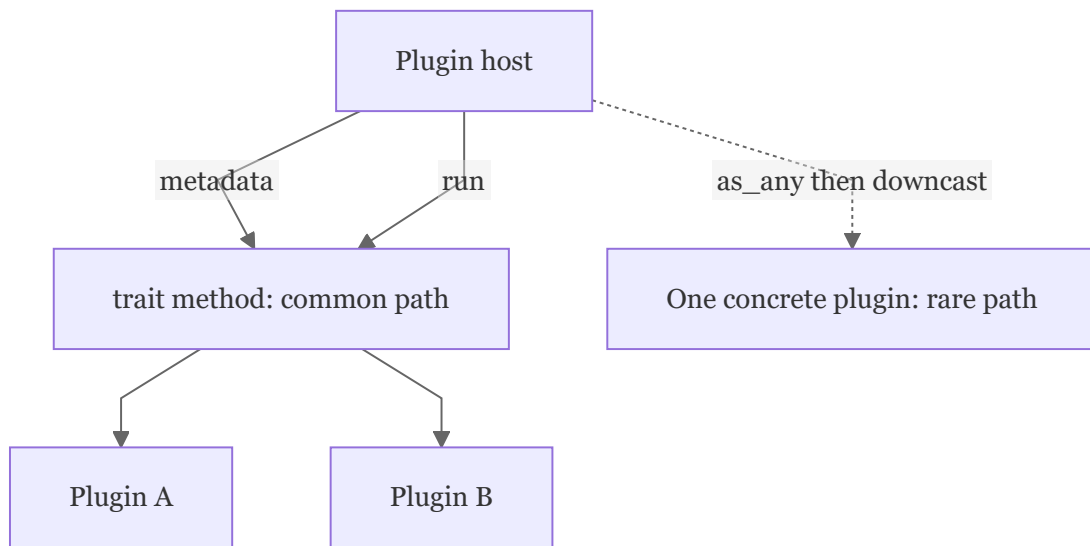
This is where most of what other languages call reflection actually happens in Rust, just earlier in the timeline. A `derive` macro reads the syntax of your struct or enum and writes more Rust before type checking, so a `Serialize` implementation that “knows” every field is generated rather than discovered. The result is the same capability a reflective serializer offers, with the cost paid at build time and the field walk visible to the compiler. The compiler-built labels in the third card are a separate, smaller tool: good for logs and panics, but never a contract.

Generating schema instead of inspecting values

When you need a machine-readable description of a type for documentation, client generation, or validation, generate it from the source rather than inspecting live values. A crate such as `schemars` uses the same `derive` machinery to turn a transport DTO into a JSON Schema at build time. That keeps the schema close to the type it describes and frees the domain model to evolve behind the wire DTO, instead of tying a public contract to whatever fields a value happens to expose at runtime.

Plugin systems that declare rather than infer

A plugin host is the classic place where engineers from reflective languages reach for runtime inspection: load a plugin, scan it for its name and capabilities, dispatch accordingly. The calmer Rust design inverts that. Each plugin returns a descriptor through an ordinary trait method, so the host reads metadata the same way it reads any other return value, and downcasting is held back as a rare escape hatch for the one specialized path that genuinely needs a concrete type. The diagram makes the split concrete: the wide path is trait dispatch, and the narrow dotted path is the occasional downcast.



The common path is a trait method every plugin implements; downcasting via `as_any` is the narrow dotted exception, not the main API.

Designing explicit metadata systems

What changes by background

Reflection is one of the topics where prior experience cuts both ways: the vocabulary transfers, but the reflexes mislead. The shift to keep in mind is that Rust moves structural introspection from a runtime capability you call to a compile-time capability you generate, and keeps only a narrow runtime identity check for the cases that truly need it. Read the card for your own background before the production patterns.

Production patterns

Pitfalls and tradeoffs

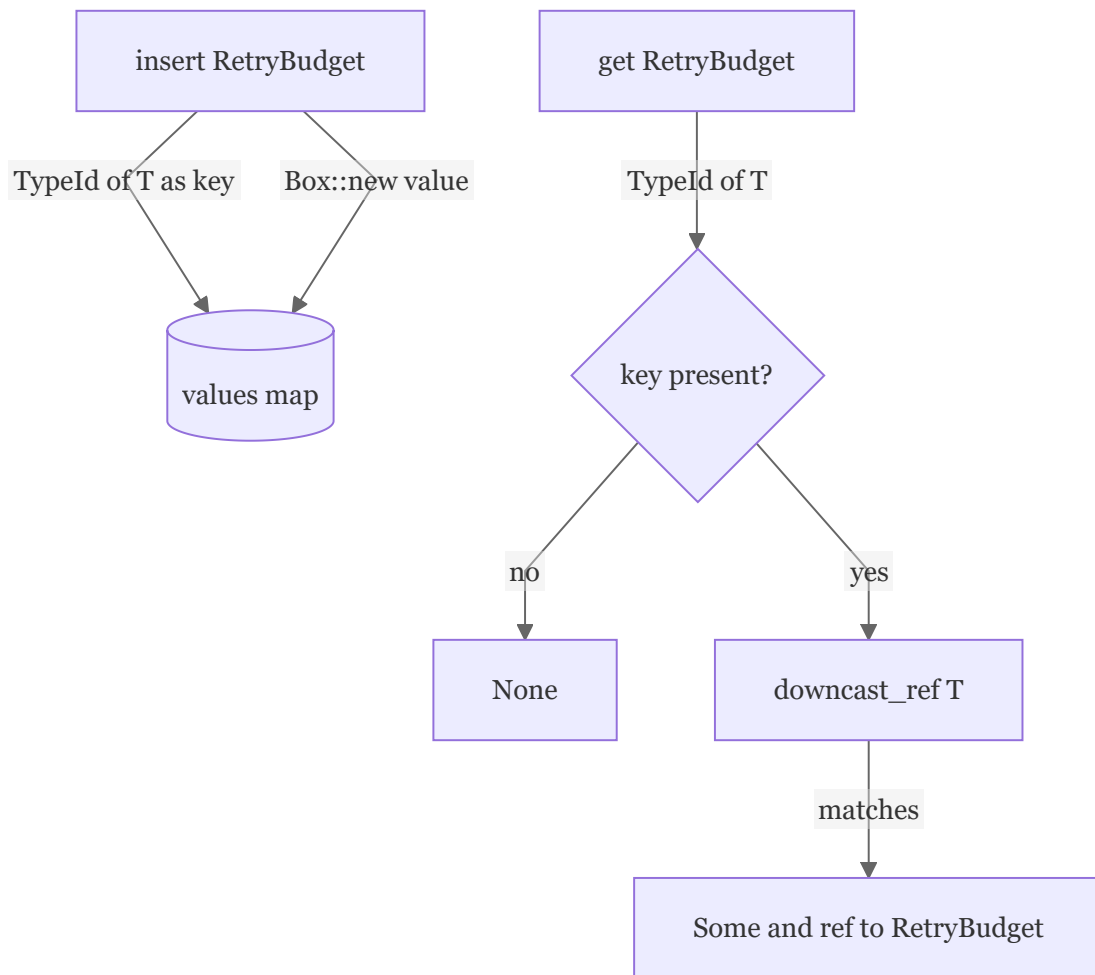
If the code only works after many downcasts, the system is probably missing an explicit contract. Rust reflection tools are strongest when they stay narrow and explicit.

Examples

Example 1: a small `TypeId` + `Any` registry

The container is erased, but recovery is still type-checked and explicit.

What to look at: every method on `TypeMap` uses the generic parameter `T` twice over, once to compute `TypeId::of::<T>()` as the map key and once to drive the `downcast_ref::<T>()` on the way out. The store itself is untyped (`HashMap<TypeId, Box<dyn Any>>`), yet the public API only ever returns the concrete type the caller named. Trace one insert and one get through the diagram, then read the same two paths in the code.



insert keys the box by `TypeId::of::<T>()`; get recomputes the same `TypeId`, then `downcast_ref::<T>()` turns the erased box back into a typed reference.

Example 21-1. `type_id_tmap.rs` — *Typeid registry*

```

use std::any::{Any, TypeId};
use std::collections::HashMap;

#[derive(Debug)]
struct RequestContext {
    trace_id: &'static str,
}

#[derive(Debug)]
struct RetryBudget(u32);

#[derive(Default)]
struct TypeMap {
    values: HashMap<TypeId, Box<dyn Any>>,
}

impl TypeMap {
    fn insert<T: 'static>(&mut self, value: T) {
        self.values.insert(TypeId::of::<T>(), Box::new(value));
    }
}

```



```

fn get<T: 'static>(&self) -> Option<&T> {
    self.values.get(&TypeId::of::())?.downcast_ref::()
}

fn contains<T: 'static>(&self) -> bool {
    self.values.contains_key(&TypeId::of::())
}
}

fn main() {
    let mut request_scoped = TypeMap::default();
    request_scoped.insert(RequestContext { trace_id: "req-7" });
    request_scoped.insert(RetryBudget(3));

    println!("has context = {}", request_scoped.contains::());
    println!("retries = {}", request_scoped.get::().unwrap().0);
    println!("trace = {}", request_scoped.get::().unwrap().trace_id);
}

```

OUTPUT

```

has context = true
retries = 3
trace = req-7

```

`TypeId` keys the erased store by concrete type, not by string name.

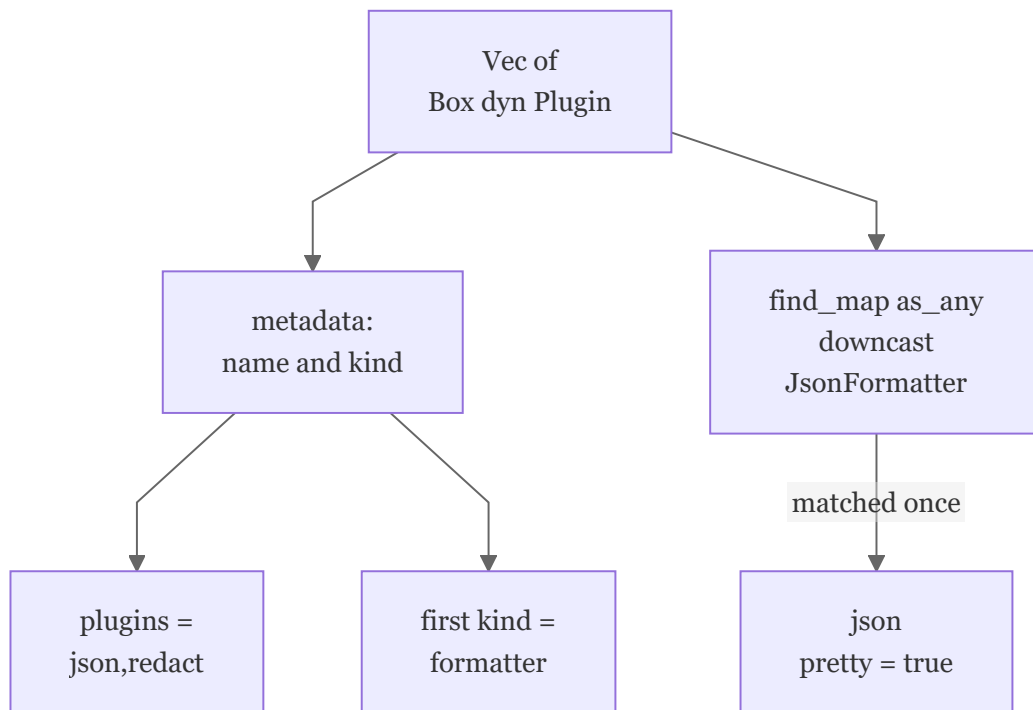
`downcast_ref::()` recovers one known type and returns `Option`.

Request extensions and typed registries are simpler than trying to reflect arbitrary structs.

Example 2: explicit plugin metadata plus optional downcast

The common path uses trait methods. Downcasting exists only for the rare specialized path.

What to look at: two of the three results come straight off the trait. Listing the names and reading `metadata().kind` never touches `Any` at all, because that data lives on the trait every plugin implements. Only the third result, the pretty flag, is specific to one concrete plugin, so it is the single place that calls `as_any()` and `downcast_ref::()`. The diagram shows that asymmetry: the wide trait path and the one narrow downcast.



Names and kinds come from the trait method on every plugin; only the pretty flag requires a downcast to the one concrete `JsonFormatter`.

Example 21-2. `plugin_metadata_and_downcasting.rs` — *Metadata downcast*

```

use std::any::Any;

#[derive(Debug, Clone, Copy)]
struct PluginMetadata {
    name: &'static str,
    kind: &'static str,
    config_format: &'static str,
}

trait Plugin {
    fn metadata(&self) -> PluginMetadata;
    fn run(&self, input: &str) -> String;
    fn as_any(&self) -> &dyn Any;
}

struct JsonFormatter {
    pretty: bool,
}

struct RedactSecrets;

impl Plugin for JsonFormatter {
    fn metadata(&self) -> PluginMetadata {
        PluginMetadata {
            name: "json",
            kind: "formatter",
            config_format: "json",
        }
    }
}

```

```

    fn run(&self, input: &str) -> String {
        if self.pretty {
            format!("pretty({})", input)
        } else {
            format!("compact({})", input)
        }
    }

    fn as_any(&self) -> &dyn Any {
        self
    }
}

impl Plugin for RedactSecrets {
    fn metadata(&self) -> PluginMetadata {
        PluginMetadata {
            name: "redact",
            kind: "filter",
            config_format: "none",
        }
    }

    fn run(&self, input: &str) -> String {
        input.replace("token=", "token=***")
    }

    fn as_any(&self) -> &dyn Any {
        self
    }
}

fn main() {
    let plugins: Vec<Box<dyn Plugin>> = vec![
        Box::new(JsonFormatter { pretty: true }),
        Box::new(RedactSecrets),
    ];

    let listed = plugins
        .iter()
        .map(|plugin| plugin.metadata().name)
        .collect:::<Vec<_>>()
        .join(",");

    let pretty = plugins
        .iter()
        .find_map(|plugin| plugin.as_any().downcast_ref:::<JsonFormatter>())
        .map(|json| json.pretty)
        .unwrap_or(false);

    println!("plugins = {}", listed);
    println!("json pretty = {}", pretty);
    println!("first kind = {}", plugins[0].metadata().kind);
}

```

OUTPUT

```
plugins = json,redact
json pretty = true
first kind = formatter
```

Names and kinds are ordinary trait data, not values inferred at runtime.

`as_any()` is there for one narrow specialized query, not for every call.

Plugin systems are usually easiest to operate when capabilities and config formats are explicit.

The repository also includes standalone Rust source under `examples/ch21_reflection_and_type_introspection/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

- Rust has limited runtime reflection on purpose. It offers narrow tools, not broad object inspection.
- `Any` and `TypeId` are useful for erased storage and checked recovery inside one process.
- Downcasting is an edge tool. Common behavior should stay on traits, enums, and explicit descriptors.
- Compile-time macros and schema generation cover many reflection-like needs more cleanly than runtime inspection.
- Plugin systems, admin surfaces, and public protocols are usually best served by explicit metadata systems.

PART IV

Concurrency, Async, and Systems IO

Threads, synchronization, futures, Tokio, and low-level IO — what makes Rust a systems language, not just a safe one.

-
- 22** Multithreading in Rust
 - 23** Synchronization Primitives
 - 24** Coroutines, Futures, and Async Rust
 - 25** Tokio
 - 26** Task Libraries and Parallel Execution
 - 27** IO Tricks and Systems Programming Patterns
-

CHAPTER 22 · MULTITHREADING IN RUST

Multithreading in Rust

Rust's thread safety model, Send and Sync, spawning and scoped threads, shared state, channels, work stealing, and cross-language tradeoffs

Multithreaded Rust systems need clear ownership transfer, shared-state policy, and shutdown behavior. This chapter covers threads, scoped work, channels, and synchronization as production boundaries.

Builds on Chapters 04, 07, 09, and 14

Chapter 04 established ownership and borrowing. Chapter 07 separated moves, copies, and clones. Chapter 09 introduced Arc, Mutex, and related pointer choices. Chapter 14 explained trait-based boundaries. This chapter applies those ideas to OS threads directly.

The problem this chapter solves

Picture a request-processing service with three kinds of work happening at once. It does CPU-heavy preprocessing on incoming payloads, it keeps a shared metrics index up to date, and it hands off slower background jobs to a pool of worker threads. In another language you might reach for one shared object guarded by a lock and call it a day. In Rust the more durable design names each boundary by its ownership story: jobs are moved into the workers that run them, immutable configuration is shared read-only through an Arc, genuinely shared mutation goes behind a lock and nothing else does, and a one-shot parallel pass over local data uses a scoped split so the threads can borrow instead of clone.

That is the spine of the chapter. Each of those four moves, move, share, synchronize, and borrow-in-scope, is a distinct primitive with a distinct cost, and the skill is matching the primitive to the workload rather than defaulting to whichever one compiled first.

One distinction to hold onto: this is about OS threads

Everything here concerns OS threads, the kind the operating system schedules and that cost real memory and scheduler attention. It is deliberately not about async tasks, futures, or executors, even though those also do concurrent work. The two models look similar from a distance and solve different problems, so keep them in separate boxes: threads are scheduled by the OS, async tasks are scheduled by an executor inside your process, and distributed workers are other processes or machines entirely. We cover the async model in its own chapter.

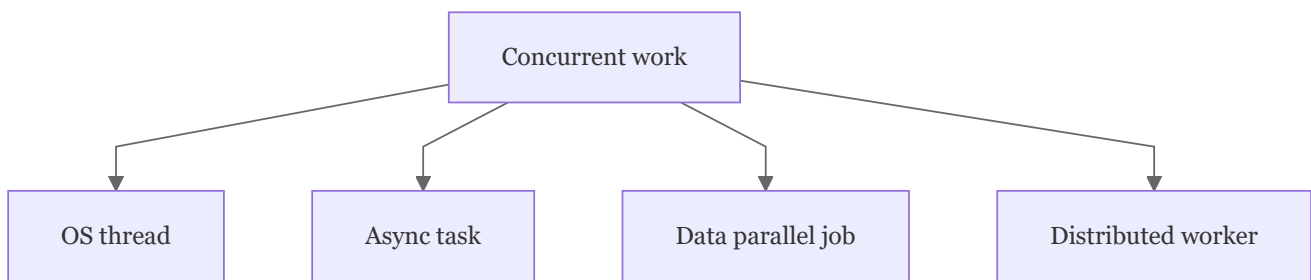
Mental model

Core concepts

Naming the execution models before the APIs

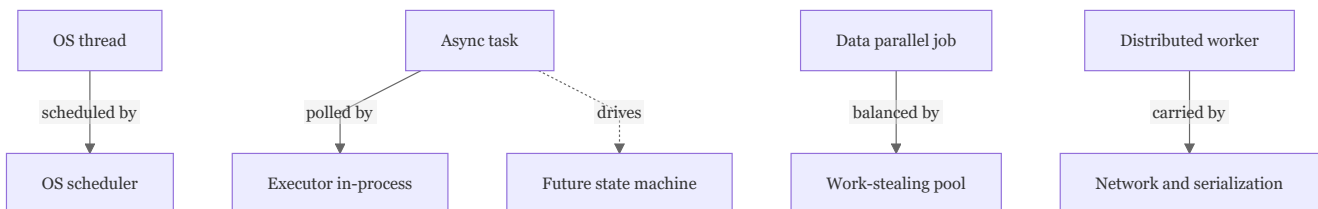
Most confusion in this area is vocabulary, not code. "Concurrency" gets stretched across several unrelated mechanisms with different schedulers, different costs, and different ownership rules. Before any API, fix the

terms. The diagram shows who schedules what: the OS schedules threads, an executor schedules tasks inside one process, and distributed work runs in other processes entirely.



One label, four execution models. Each branch is a different kind of concurrent work with a different cost.

Each of those four kinds is scheduled by something different. The same four branches map to four schedulers:



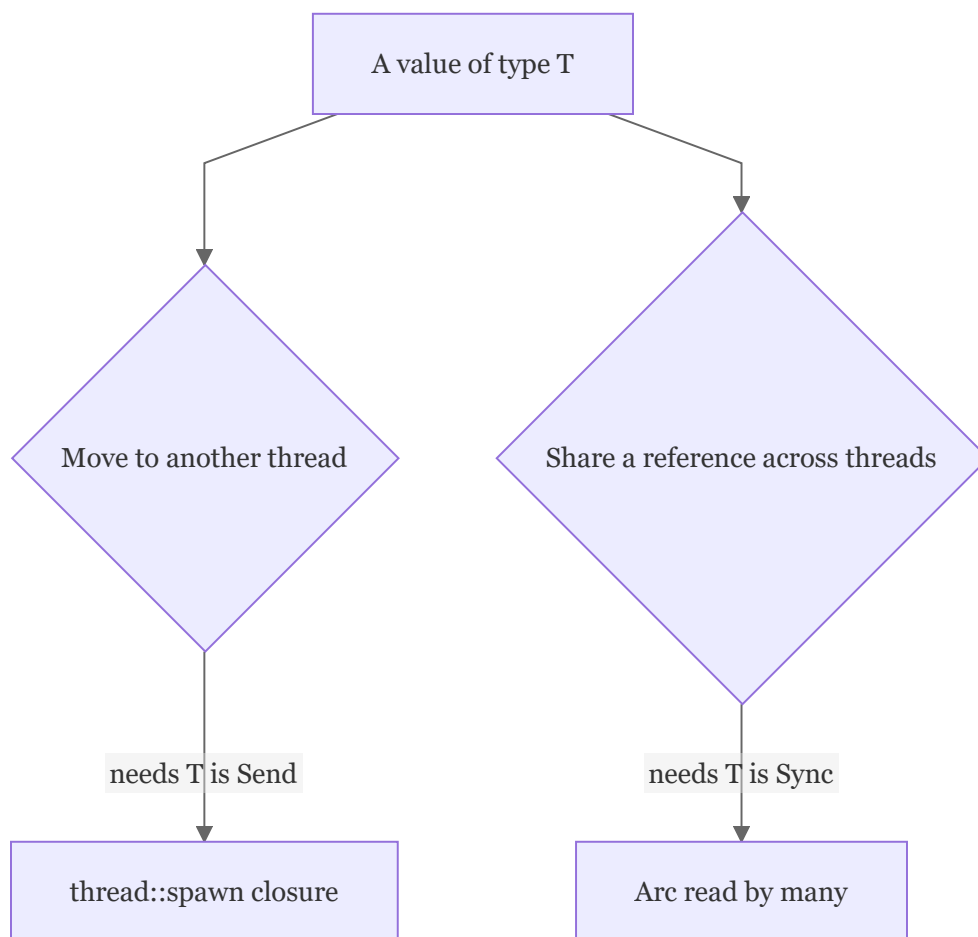
Four schedulers for the four models. This chapter is the OS-thread branch; the others have their own contracts and costs.

Rust's thread safety model

The important part is not the number. It is that the closure owns what it uses, and the caller decides when failure from the child thread becomes visible by joining.

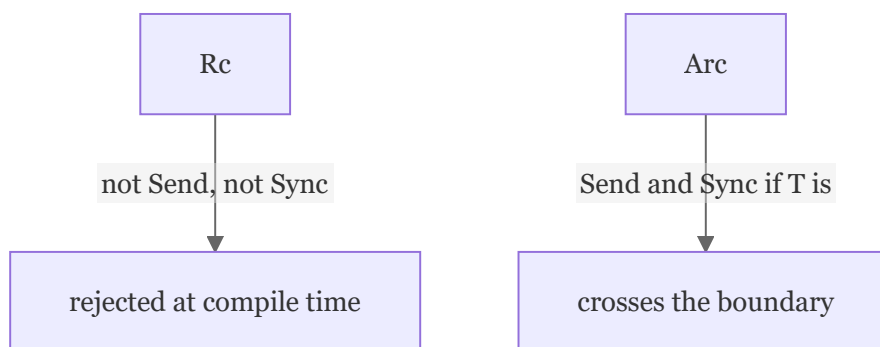
Send and Sync: the two traits that gate every thread boundary

These two auto traits are the entire mechanism behind "data races do not compile." They are marker traits with no methods; the compiler derives them automatically for a type when all of its fields qualify. When you call `thread::spawn`, its signature quietly requires the closure (and everything it captures) to be `Send`. A type that is not `Send` simply will not fit through that boundary, and you get a compile error pointing at the capture rather than a race at 3 a.m. The diagram shows the two questions the compiler is really asking.



Two questions the compiler asks: moving a value across a thread needs `Send`; sharing a reference needs `Sync`.

Those two questions are exactly what separates the two reference-counted pointers at a thread boundary:



`Rc` fails both checks because its `refcount` is non-atomic; `Arc` passes when its contents do, so only `Arc` crosses the boundary.

Manual `unsafe impl Send` or `unsafe impl Sync` is only correct if you can prove the type's hidden aliasing, mutation, and destruction behavior is thread-safe. Most application code should never need to write those impls.

Spawning threads

Use `std::thread::spawn` when you need a new OS thread. The closure usually needs `move` because the child may outlive the current stack frame.

Production code often uses `std::thread::Builder` when thread names, stack size, or clearer diagnostics matter. Thread names are cheap observability.

Moving data into threads

A `move` closure transfers ownership of captured non-Copy values into the child thread. That is the calm default for jobs, owned buffers, and request-local domain data that must survive independently in the worker.

Clone only when two threads truly need independent ownership. Share through `Arc` when the data is genuinely shared. Borrow only inside a scope that guarantees the borrow cannot outlive the parent.

Shared state concurrency

A good default question is this: if only one thread should really own the mutable state, why not send owned commands to that thread instead of putting the state behind a global lock?

Message-passing concurrency

Unbounded and bounded channels tell different operational stories. Use the one that matches memory, latency, and producer-pressure expectations.

Scoped threads

Work stealing

A useful production rule: if the work is CPU-bound collection processing, a Rayon-style work-stealing pool is often calmer than hand-rolling many `thread::spawn` calls. If the work is IO-bound waiting, an async executor is usually the more honest model. If the work crosses the network, you are in distributed systems.

Coming from another language

If you already write concurrent code somewhere else, the syntax is the easy part. What trips people up is the mental-model shift, so read whichever card matches your background before the examples.

Production patterns

Pitfalls and tradeoffs

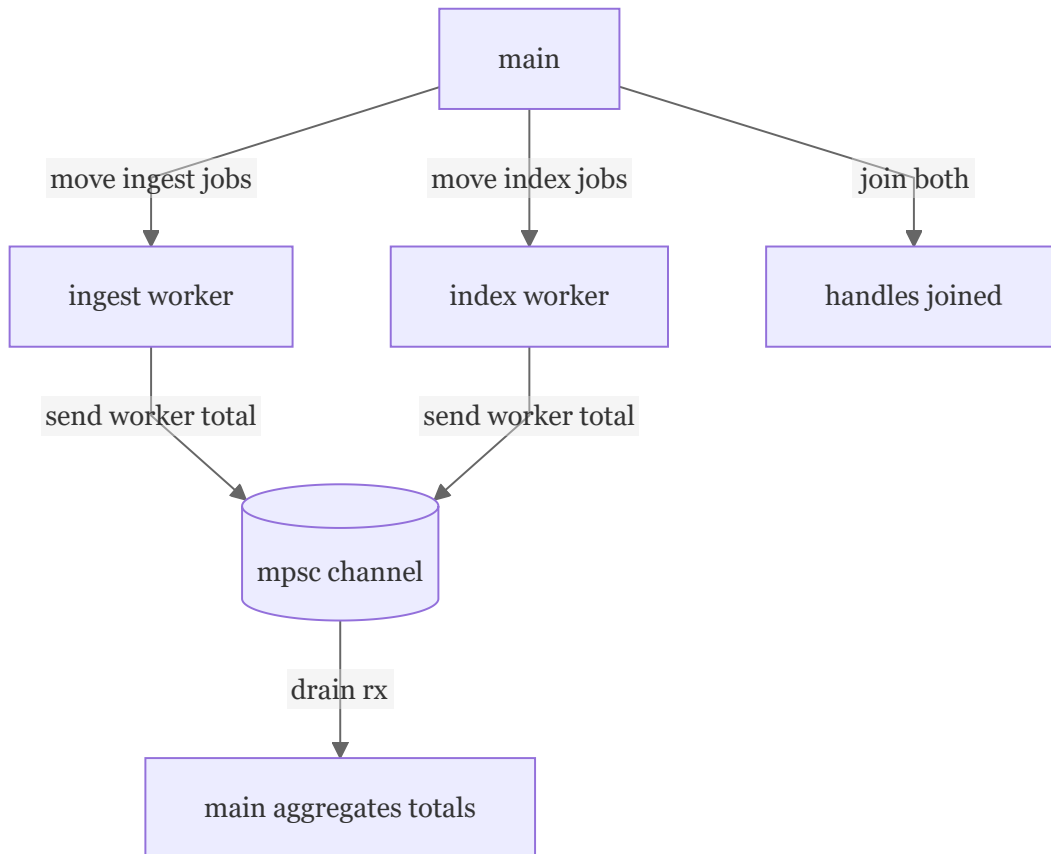
The right concurrency primitive is the one that makes ownership calm under load. A channel, a scoped thread, and a lock can all be correct. The mistake is using one of them because it resembles another language rather than because the workload actually wants it.

Examples

Example 1: spawn workers, move owned jobs, collect by channel

This is an OS-thread design with owned job batches and message passing for results. No worker mutates a shared central map directly.

What to look at: ownership flows in one direction. Each job Vec is moved into its worker, so the worker is the sole owner while it runs; results travel back as small owned tuples over the channel, and only the main thread builds the final map. No worker ever touches that map, so there is nothing to lock.



Jobs move out to the workers; totals come back over one channel; aggregation happens in a single owner. Move out, message back.

Example 22-1. owned_jobs_over_channel.rs — Jobs channel

```

use std::collections::HashMap;
use std::sync::mpsc;
use std::thread;

#[derive(Debug)]
struct Job {
    name: &'static str,
    cost: u32,
}

fn spawn_worker(
    tx: mpsc::Sender<(&'static str, u32)>,
    worker: &'static str,

```

```

    jobs: Vec<Job>,
) -> thread::JoinHandle<()> {
    thread::spawn(move || {
        let total: u32 = jobs.iter().map(|job| job.cost).sum();
        tx.send((worker, total)).unwrap();
    })
}

fn main() {
    let (tx, rx) = mpsc::channel();

    let ingest_jobs = vec![
        Job { name: "parse", cost: 3 },
        Job { name: "validate", cost: 2 },
    ];
    let index_jobs = vec![
        Job { name: "index", cost: 4 },
        Job { name: "flush", cost: 1 },
    ];

    let ingest = spawn_worker(tx.clone(), "ingest", ingest_jobs);
    let index = spawn_worker(tx, "index", index_jobs);

    ingest.join().unwrap();
    index.join().unwrap();

    let mut totals = HashMap::new();
    for (worker, total) in rx {
        totals.insert(worker, total);
    }

    let grand: u32 = totals.values().copied().sum();

    println!("ingest total = {}", totals.get("ingest").copied().unwrap_or(0));
    println!("index total = {}", totals.get("index").copied().unwrap_or(0));
    println!("grand total = {}", grand);
}

```

OUTPUT

```

ingest total = 5
index total = 5
grand total = 10

```

Each worker is a real OS thread with a join handle.

The job vectors cross the boundary by ownership through a move closure.

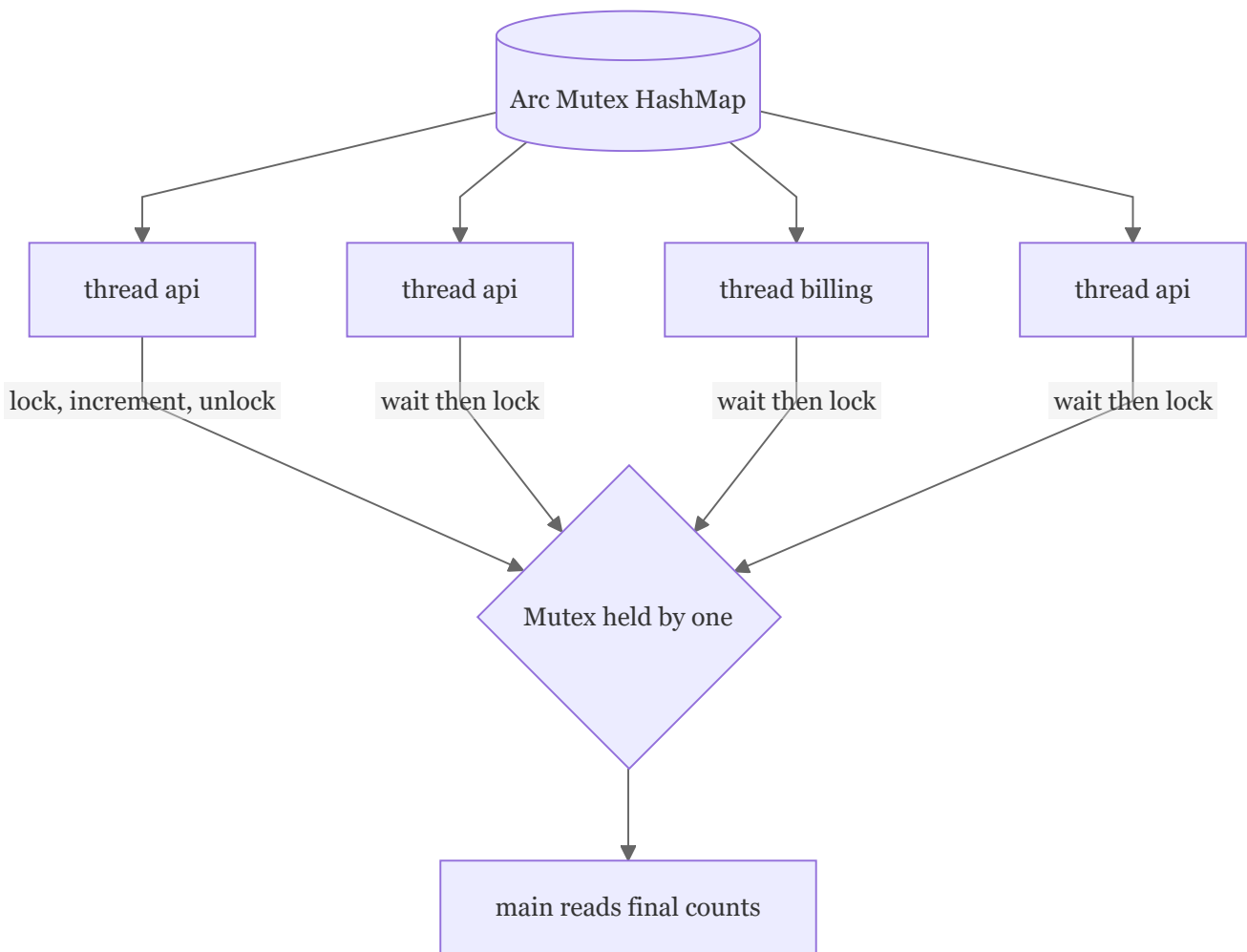
Workers return results by channel, so the aggregation boundary stays explicit and deterministic.

Example 2: shared state with Arc<Mutex<...>>

Shared state is sometimes the honest model. The important engineering work is then lock scope, contention, and who really needs to mutate what.

What to look at: the two wrappers do two separate jobs. Arc is what lets every thread reach the same map (each thread gets its own clone of the handle, all pointing at one allocation), and Mutex is what serializes the

writes so two increments cannot collide. Read the diagram as: everyone shares the pointer, but only the lock holder may mutate.



Shared ownership via Arc, exclusive mutation via Mutex. The lock is the bottleneck, so keeping the critical section tiny is the whole optimization.

Example 22-2. `shared_state_route_counts.rs` — State metrics

```

use std::collections::HashMap;
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counts = Arc::new(Mutex::new(HashMap::new()));
    let mut handles = Vec::new();

    for route in ["api", "api", "billing", "api"] {
        let counts = Arc::clone(&counts);
        handles.push(thread::spawn(move || {
            let mut map = counts.lock().unwrap();
            *map.entry(route).or_insert(0) += 1;
        }));
    }

    for handle in handles {
        handle.join().unwrap();
    }
}

```

```
}

let map = counts.lock().unwrap();
println!("api = {}", map.get("api").copied().unwrap_or(0));
println!("billing = {}", map.get("billing").copied().unwrap_or(0));
println!("routes = {}", map.len());
}
```

OUTPUT

```
api = 3
billing = 1
routes = 2
```

Arc makes the counter map reachable from several threads.

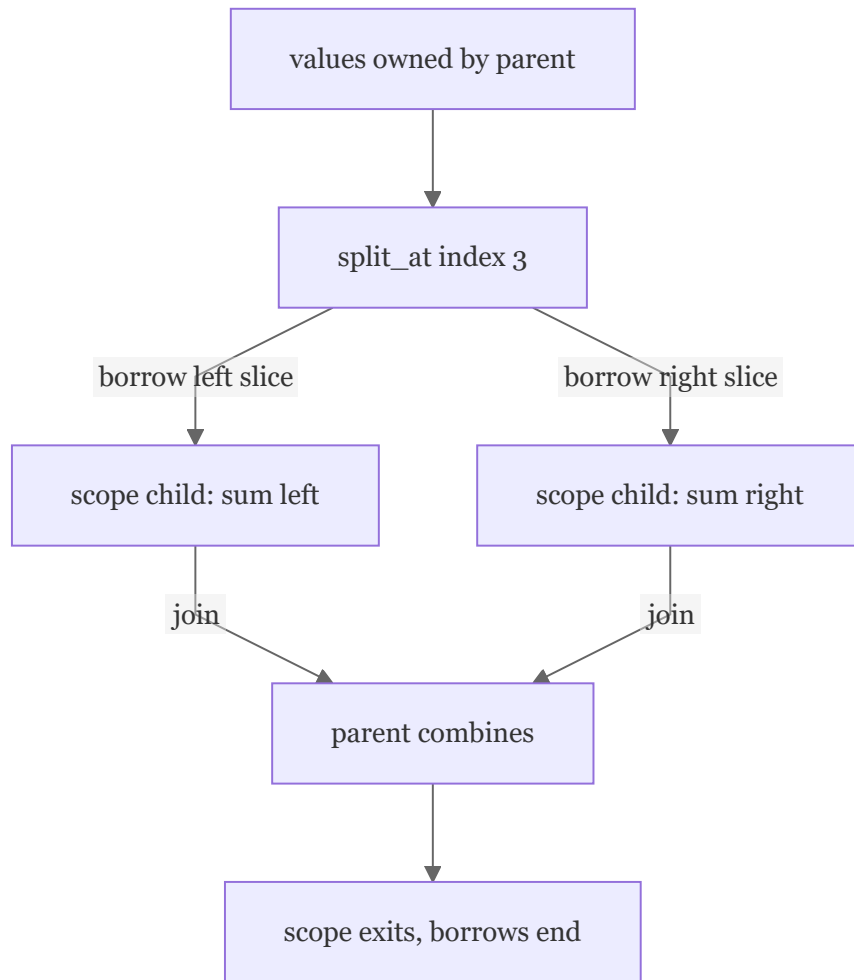
Mutex narrows mutation to one thread at a time.

The model is honest, but lock contention is now a real runtime cost to observe.

Example 3: scoped threads over borrowed slices

This is still multithreading, but the parent keeps ownership and children borrow from it safely inside the scope.

What to look at: nothing is cloned and nothing is wrapped in Arc. The parent owns the array the whole time; the children borrow non-overlapping slices of it. This compiles only because `thread::scope` guarantees every child joins before the scope returns, so the borrows provably cannot outlive the data. That join-before-exit guarantee is the entire reason scoped threads can do what a plain spawn cannot.



The parent keeps ownership; children borrow disjoint slices and are joined before the scope can return. No Arc, no clone, no 'static requirement.

Example 22-3. `scoped_threads_slice_sum.rs` — *Threads sum*

```

use std::thread;

fn main() {
    let values = [2_u32, 4, 6, 8, 10, 12];
    let split_at = 3;

    thread::scope(|scope| {
        let (left, right) = values.split_at(split_at);

        let left_handle = scope.spawn(move || left.iter().copied().sum::<u32>());
        let right_handle = scope.spawn(move || right.iter().copied().sum::<u32>());

        let left_total = left_handle.join().unwrap();
        let right_total = right_handle.join().unwrap();

        println!("left = {}", left_total);
        println!("right = {}", right_total);
        println!("total = {}", left_total + right_total);
    });
}

```

OUTPUT

```
left = 12
right = 30
total = 42
```

The slices are borrowed from the parent array instead of cloned into separate owners.

The scope guarantees both child threads join before the borrowed data can disappear.

This pattern is excellent for local CPU-bound slice processing with no detached worker lifetime.

The repository also includes standalone Rust source under `examples/ch22_multithreading_in_rust/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

- Rust multithreading is ordinary ownership plus two auto traits, `Send` and `Sync`, that turn most data races into compile errors.
- OS threads, async tasks, futures, executors, data parallelism, and distributed workers are different execution models with different schedulers and different costs.
- `thread::spawn` usually wants owned captured data; `thread::scope` is the tool when borrowing local stack data is the honest model.
- Shared state with `Arc` and a lock can be correct, but a channel is often calmer when only one thread should own the mutation.
- Work stealing is a scheduler strategy used by data-parallel libraries, not a synonym for spawning threads by hand.
- Good production thread design is observable, join-aware, and explicit about queue growth, lock contention, and panic handling.

CHAPTER 23 · SYNCHRONIZATION PRIMITIVES

Synchronization Primitives

Mutex, RwLock, Condvar, atomics, memory ordering, barriers, channels, lock-free patterns, deadlock prevention, and choosing the right primitive

Synchronization primitives protect shared state and coordinate progress under contention. This chapter covers mutexes, read-write locks, atomics, barriers, and channels as explicit workload choices.

Builds on Chapters 04, 09, and 22

Chapter 04 gave us ownership and borrowing. Chapter 09 explained Arc, Mutex, and related pointer choices. Chapter 22 established OS-thread ownership boundaries and message passing. This chapter zooms into the synchronization contracts themselves.

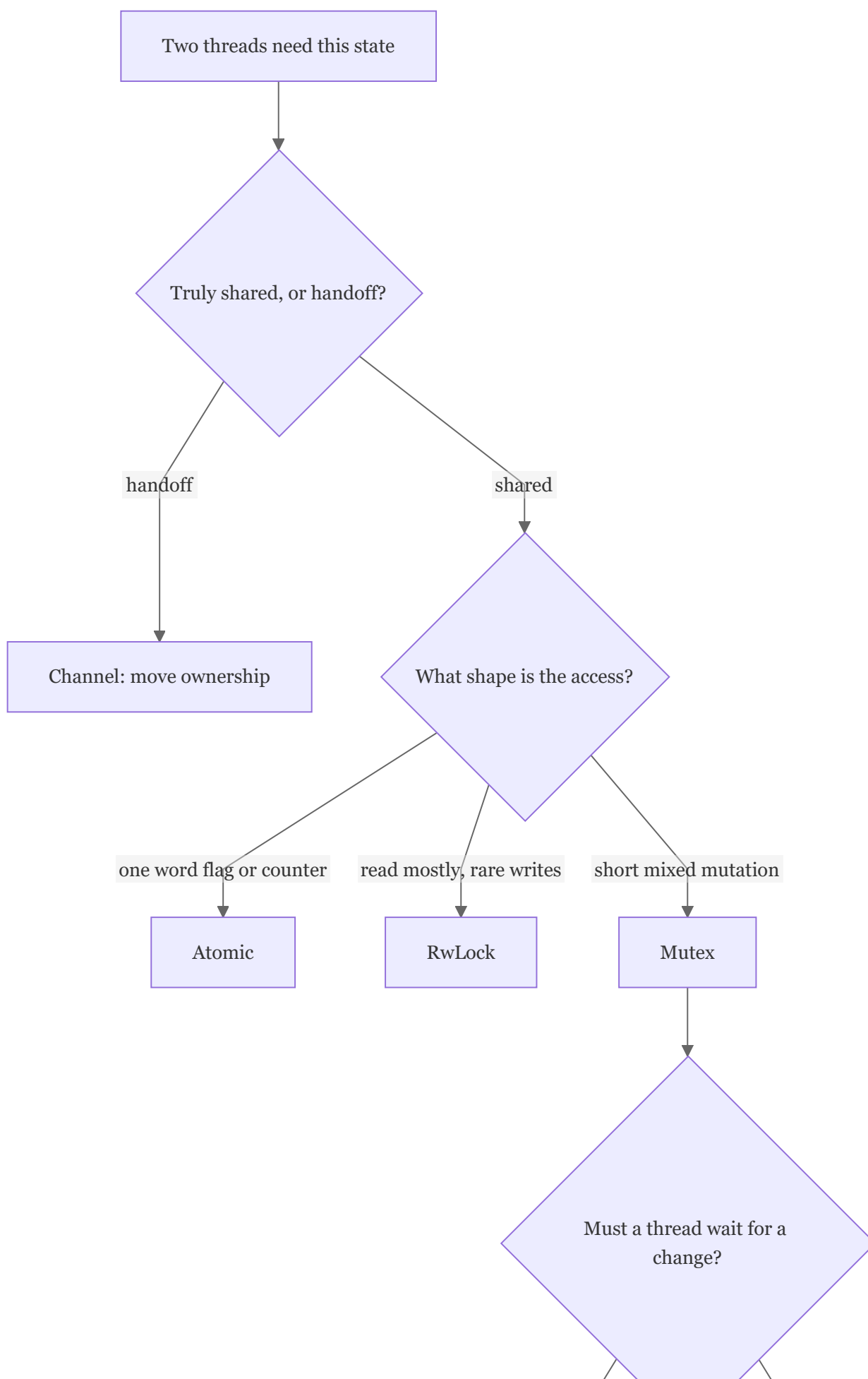
Opening scenario

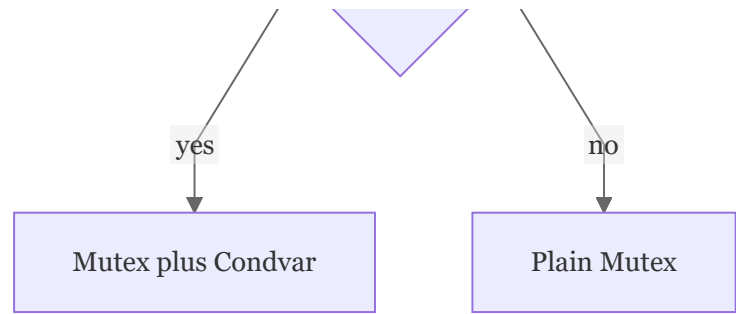
Picture a small retry service running several worker threads. It holds three pieces of state with three very different access patterns: a table of in-flight requests that workers mutate constantly, a configuration snapshot that is read on every request and rewritten only when an operator pushes a change, and a queue of jobs that really wants a single consumer draining it. Each piece is a different concurrency problem, and the mistake that causes the most pain is reaching for the same primitive for all three.

The discipline this chapter teaches is to derive the primitive from the access pattern rather than from habit. A short, contended mutation wants a mutex. A read-mostly snapshot wants a read-write lock. A single-word flag or counter wants an atomic. A thread that must wait for state to change wants a condvar tied to that state. And a job stream with one owner wants a channel, because the real model is ownership transfer, not sharing. Naming the access pattern first turns primitive selection from a guessing game into a lookup.

A useful decision order

Read the diagram top to bottom: the first fork is the most important one, because deciding whether state is shared at all often eliminates locks entirely in favor of message passing.





The decision order: rule out handoff first, then pick by access shape, and only add a condvar when a thread genuinely needs to wait on a predicate.

1. Decide whether the state is actually shared or whether ownership transfer is the cleaner model.
2. If the state is shared, decide whether it is write-heavy, read-mostly, or just one-word atomic state.
3. If threads wait, name the predicate they wait on before choosing condvars, barriers, or channels.
4. Only consider lock-free patterns after a simple design has been measured and found wanting.

Mental model

What changes by background

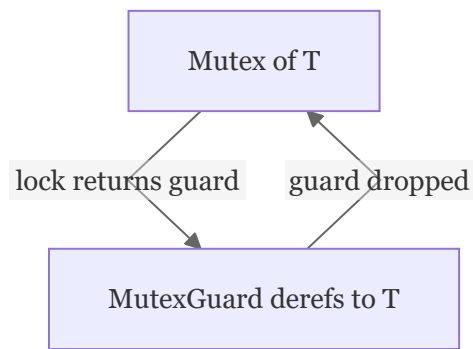
Every reader brings a synchronization model from another language. Most of the friction in Rust comes not from the primitives, which look familiar, but from where Rust moves the safety boundary. The shift to internalize is the same in each case: things that were team discipline or runtime conveniences elsewhere become facts the compiler checks here.

Core concepts

Mutex and RwLock

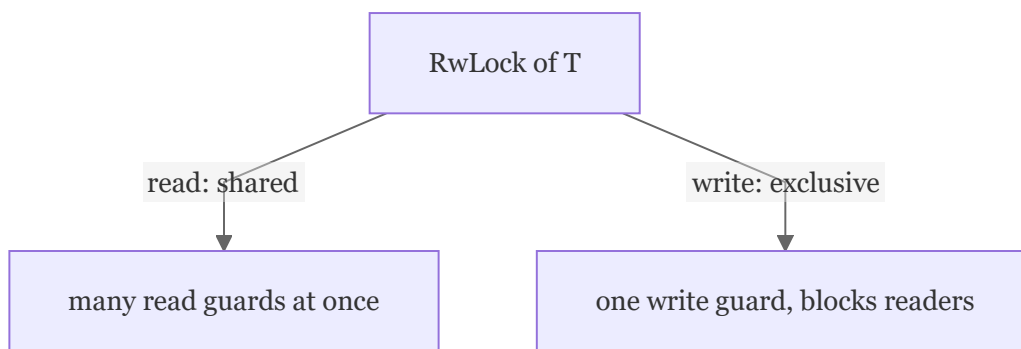
The structural idea that distinguishes Rust here is that the lock owns the data. A Mutex wraps the value of type `T`, and the only way to reach the value is to call `lock`, which hands back a guard. While you hold the guard you have exclusive access; when the guard is dropped, the lock is released. There is no path to the data that skips the lock, which is why a forgotten lock is not a category of bug that exists in safe Rust.

Look at the relationship below before the listings: the guard is a borrow with a lifetime, and releasing the lock is simply that borrow ending. The diagram shows the single path data takes through a mutex versus the two paths a read-write lock allows.



A mutex offers a single exclusive path to the value. The guard's lifetime is the critical section; dropping it releases the lock.

An `RwLock` splits that single path into two guard kinds:



An `RwLock` offers many shared read guards or one exclusive write guard, so reads can overlap while a write still excludes everyone.

A common misconception is worth correcting: an `RwLock` is not “a faster mutex.” It is a different fairness and contention trade. Multiple readers can hold the lock at once, but a writer must wait for all of them to leave and then excludes everyone, so under frequent writes or tiny read sections the bookkeeping can cost more than a plain mutex would. Reach for it only when you can show the workload is genuinely read-mostly.

Condvar

A condition variable solves a specific problem: a thread needs to wait until some shared state changes, and busy-spinning on the mutex would waste a core. The condvar lets the waiter atomically release the mutex and sleep, then re-acquire the mutex when it is woken. The crucial detail is that a wakeup is only a hint to re-check; it is not a promise that the predicate is now true, because of spurious wakeups and because another thread may have consumed the state first. That is why the wait always sits inside a while loop that tests the predicate.

The condvar is not the source of truth. The mutex-protected predicate is. The wait loop exists to keep that distinction explicit.

Atomics and memory ordering

An atomic operation is indivisible: no other thread can observe it half-done. That alone covers counters and flags. The harder concept is memory ordering, which controls whether the other writes a thread made before an atomic store become visible to a thread that observes that store. Modern CPUs and compilers reorder ordinary memory accesses freely, so without an ordering constraint a reader can see the flag flip to true

while the data it was supposed to publish is still stale. The ordering you choose is a contract about that visibility, not about the atomic value itself.

A practical rule is simple. If one atomic flag publishes other writes, the writer usually uses `Release` on the flag store and the reader uses `Acquire` on the flag load. If you cannot explain the publication story in one paragraph, a mutex or channel may be the better tool.

Barriers and channels

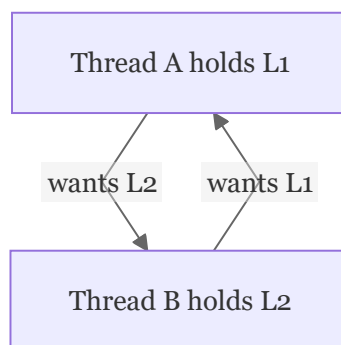
Channels deserve to stay in this chapter because they are often the simplest synchronization primitive for a one-owner mutable subsystem. The standard library gives `mpsc::channel` and `sync_channel` as solid baselines, and the ecosystem offers richer channel semantics when the workload needs them.

Lock-free patterns

In real systems, lock-free structures usually imply unsafe code somewhere in the implementation, plus a memory reclamation story. That is why “avoid all locks” is not a mature design principle by itself.

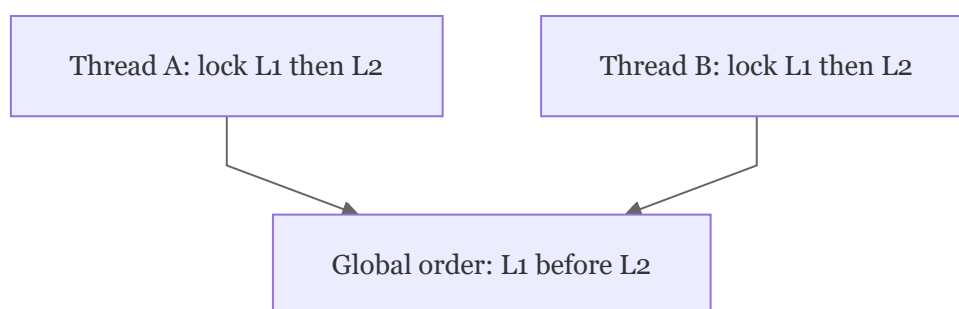
Deadlock prevention

Almost every classic deadlock is a cycle: thread A holds lock 1 and wants lock 2 while thread B holds lock 2 and wants lock 1, and neither will ever yield. The diagram contrasts that cycle with the fix, which is to impose one global order on locks so every thread always acquires them in the same sequence. With a total order there is no cycle to form. Most of deadlock prevention is this kind of design discipline rather than any special primitive.



The deadlock: a wait cycle. A holds L1 and wants L2 while B holds L2 and wants L1, so neither can proceed.

The fix is a single global lock ordering that both threads obey:



With one global order, every thread takes L1 before L2, so no cycle can form and the deadlock disappears.

Choosing the right primitive

Production patterns

Pitfalls and tradeoffs

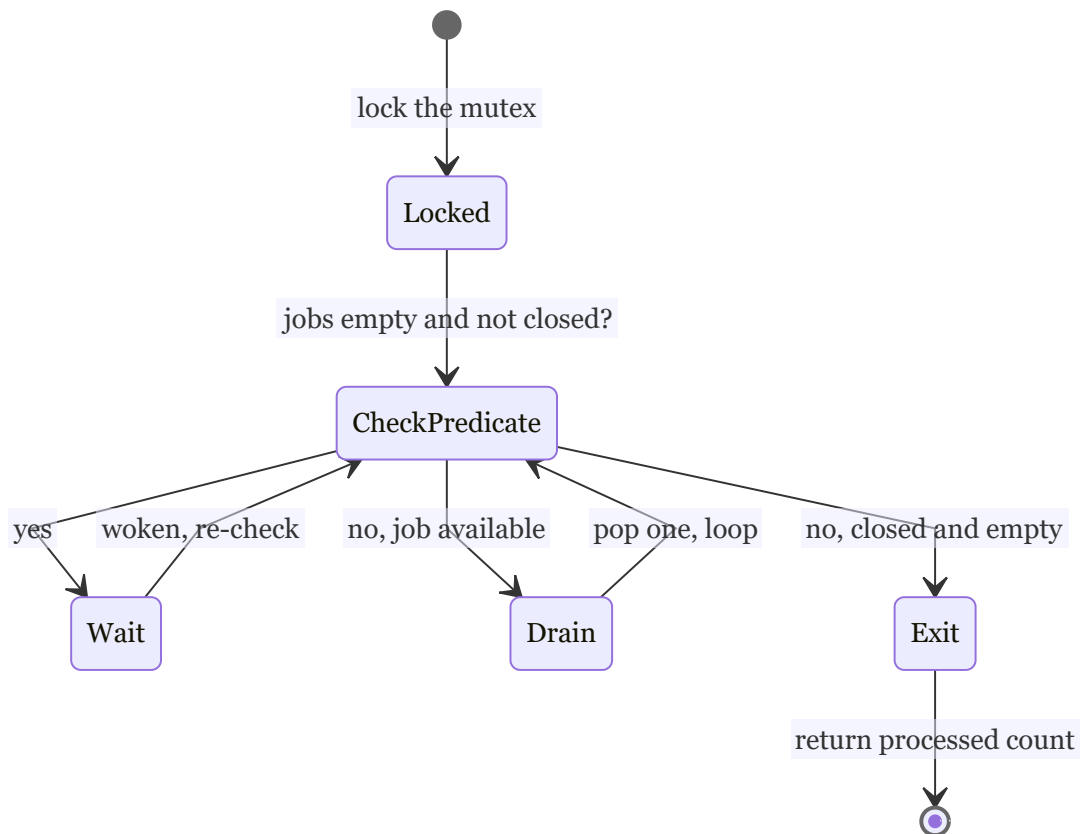
The hardest synchronization bug is often not a missing lock. It is choosing shared mutable state when the workload really wanted one owner and message passing.

Examples

Example 1: Mutex plus Condvar for a tiny shared queue

The condvar only wakes the worker. The queue state under the mutex remains the real predicate.

What to look at: the worker holds the lock, then loops on `jobs.is_empty() && !closed`, calling `wait` inside the loop. Follow that loop in the diagram before reading the code.



The worker re-checks the mutex-protected predicate every wakeup. The condvar only says re-check now; the queue state is the source of truth.

Example 23-1. `mutex_condvar_batch_queue.rs` — Condvar queue

```

use std::collections::VecDeque;
use std::sync::{Arc, Condvar, Mutex};
use std::thread;

```

```

#[derive(Debug, Default)]
struct QueueState {
    jobs: VecDeque<&'static str>,
    closed: bool,
}

#[derive(Debug)]
struct SharedQueue {
    state: Mutex<QueueState>,
    ready: Condvar,
}

fn main() {
    let shared = Arc::new(SharedQueue {
        state: Mutex::new(QueueState::default()),
        ready: Condvar::new(),
    });

    let worker = {
        let shared = Arc::clone(&shared);
        thread::spawn(move || {
            let mut processed = 0;

            loop {
                let mut state = shared.state.lock().unwrap();

                while state.jobs.is_empty() && !state.closed {
                    state = shared.ready.wait(state).unwrap();
                }

                if state.jobs.is_empty() && state.closed {
                    break;
                }

                let _job = state.jobs.pop_front().unwrap();
                processed += 1;
            }

            processed
        })
    };

    {
        let mut state = shared.state.lock().unwrap();
        state.jobs.push_back("parse");
        state.jobs.push_back("flush");
        state.closed = true;
        shared.ready.notify_all();
    }

    let processed = worker.join().unwrap();
    let remaining = shared.state.lock().unwrap().jobs.len();

    println!("processed = {}", processed);
    println!("remaining = {}", remaining);
}

```

OUTPUT

```
processed = 2
remaining = 0
```

The queue state has one synchronized owner at a time.

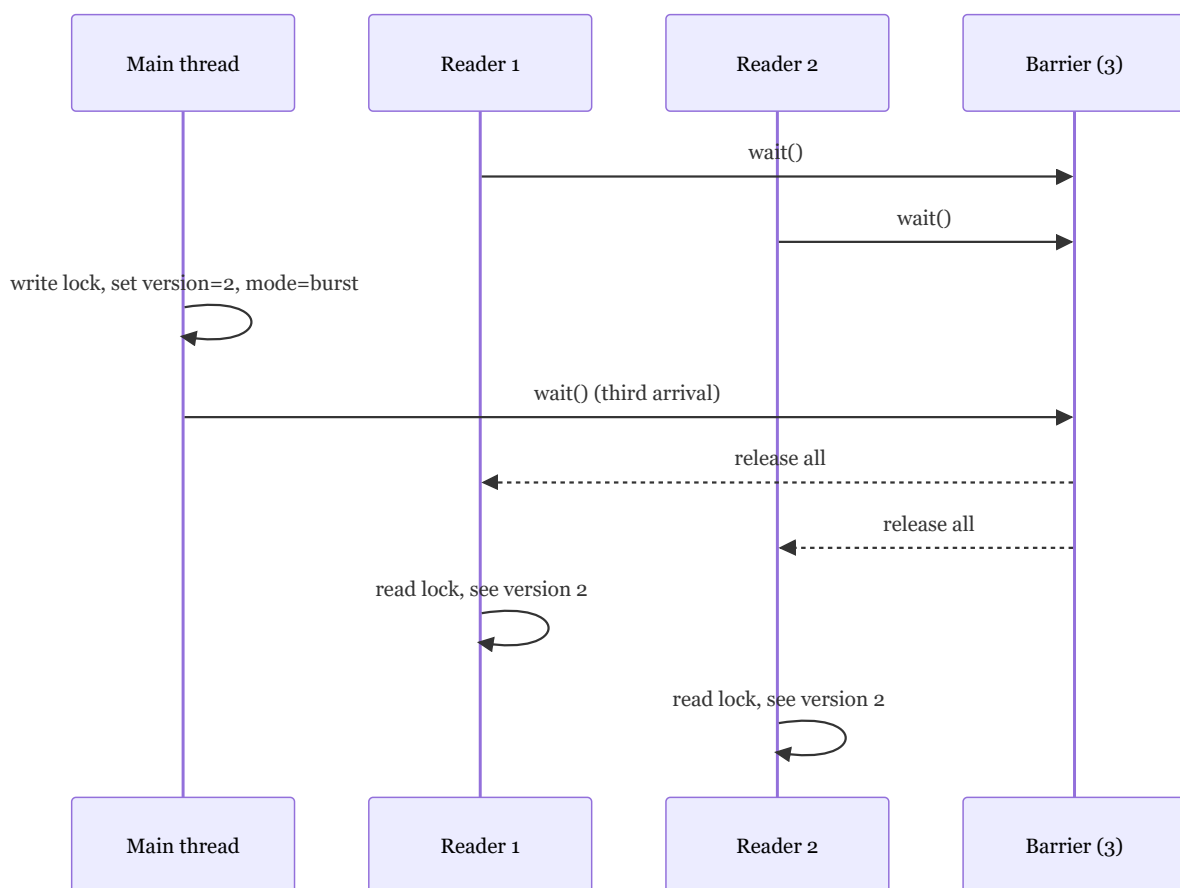
The worker sleeps until the predicate might have changed, then re-checks in a loop.

This is correct shared-state synchronization, but a channel would be simpler if one thread should own the queue.

Example 2: RwLock plus Barrier for read-mostly configuration

The write happens once before the phase boundary. Readers then observe the same snapshot together.

What to look at: the write to the config happens before the main thread reaches `start.wait()`, so when the barrier releases all three threads, both readers see the same updated snapshot. The timeline below shows that gate.



Readers park at the barrier; the writer updates the snapshot and then arrives third. The barrier releasing is the phase gate that makes the write visible to both readers.

Example 23-2. `rwlock_barrier_startup.rs` — *Rwlock barrier*

```
use std::sync::{Arc, Barrier, RwLock};
use std::thread;

#[derive(Debug)]
struct Config {
```

```

    version: usize,
    mode: &'static str,
}

fn main() {
    let config = Arc::new(RwLock::new(Config {
        version: 1,
        mode: "steady",
    }));
    let start = Arc::new(Barrier::new(3));
    let mut handles = Vec::new();

    for _ in 0..2 {
        let config = Arc::clone(&config);
        let start = Arc::clone(&start);
        handles.push(thread::spawn(move || {
            start.wait();
            let snapshot = config.read().unwrap();
            snapshot.version
        }));
    }

    {
        let mut guard = config.write().unwrap();
        guard.version = 2;
        guard.mode = "burst";
    }

    start.wait();

    let reader_version_sum: usize = handles
        .into_iter()
        .map(|handle| handle.join().unwrap())
        .sum();
    let mode = config.read().unwrap().mode;

    println!("reader version sum = {}", reader_version_sum);
    println!("mode = {}", mode);
}

```

OUTPUT

```

reader version sum = 4
mode = burst

```

The configuration is shared and read-mostly, so parallel reads make sense.

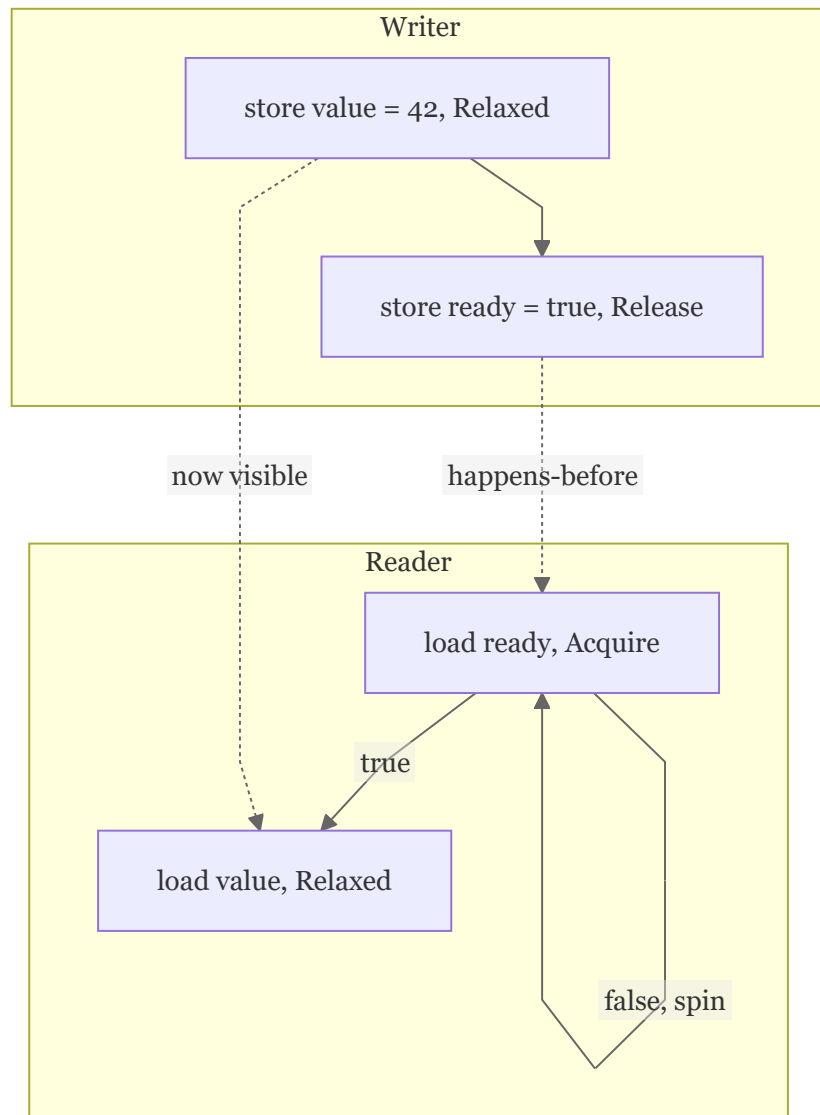
The barrier is a phase gate, not a data container.

If writes become common, this design should be re-measured against a simpler mutex or one-owner model.

Example 3: Atomic publication with Acquire and Release

The data word stays relaxed. The publication flag carries the ordering relationship.

What to look at: the value is stored with Relaxed, but the flag is stored with Release and loaded with Acquire. That one pairing is the synchronization edge. Trace it in the diagram first.



The Release store paired with the Acquire load forms a happens-before edge. Crossing it makes the earlier Relaxed write to value visible, so the reader is guaranteed to see 42.

Example 23-3. `atomics_acquire_release_publish.rs` — *Atoms ordering*

```

use std::sync::atomic::{AtomicBool, AtomicUsize, Ordering};
use std::thread;

fn main() {
    let value = AtomicUsize::new(0);
    let ready = AtomicBool::new(false);

    thread::scope(|scope| {
        scope.spawn(|| {
            value.store(42, Ordering::Relaxed);
            ready.store(true, Ordering::Release);
        });

        scope.spawn(|| {
            while !ready.load(Ordering::Acquire) {
                thread::yield_now();
            }
        });
    });
}

```

```
    }

    println!("ready = {}", true);
    println!("value = {}", value.load(Ordering::Relaxed));
  });
});
}
```

OUTPUT

```
ready = true
value = 42
```

The flag is the synchronization edge. The reader only trusts the payload after the Acquire load succeeds.

Release on the writer and Acquire on the reader express the visibility contract directly.

This pattern is for small publication state, not for a whole shared object graph.

The repository also includes standalone Rust source under `examples/ch23_synchronization_primitives/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

- Choose a synchronization primitive from the ownership and state model, not from fashion.
- Mutex, RwLock, Condvar, atomics, barriers, and channels each answer a different concurrency question.
- Acquire and Release are about visibility between threads, not about preserving source-code order by wishful thinking.
- Deadlock prevention is mostly design discipline: lock ordering, short critical sections, and avoiding nested shared-state ownership where channels would be clearer.
- Lock-free structures are specialist tools. Measure before you replace a simple lock, and prefer proven implementations when the complexity is justified.

CHAPTER 24 · COROUTINES, FUTURES, AND ASYNC RUST

Coroutines, Futures, and Async Rust

Coroutines vs futures, async fn and await, compiler-generated state machines, Pin, executors, cancellation, structured concurrency, async traits, and lifetime pitfalls

Async Rust services depend on futures, executors, cancellation, and ownership across await points. This chapter defines the runtime contract behind coroutines and asynchronous state machines.

Builds on Chapters 09, 14, 22, and 23

Chapter 09 introduced pinning. Chapter 14 covered traits and object safety. Chapter 22 separated OS threads from other execution models. Chapter 23 covered synchronization primitives that still matter in async flows. This chapter puts those pieces together into Rust's async model.

Repository note

The in-browser editor uses tiny std-only examples so the poll model stays visible without pulling in a full runtime. In production, you will usually pair these ideas with an ecosystem executor and reactor such as Tokio or another async runtime.

The problem async is solving

You are refactoring a service that performs request parsing, several outbound RPCs, cache lookups, and a final database write. Most of that wall-clock time is spent waiting on sockets, not burning CPU. The threads chapter already taught you not to dedicate one OS thread to every waiting connection: threads have stack and scheduler cost, and ten thousand mostly-idle threads is a poor use of the machine. Async is the alternative. It lets a single thread hold thousands of in-flight operations and only do work for the ones that are actually ready to make progress.

That efficiency is not free, and the price is paid in questions the runtime makes you answer explicitly. Which values stay borrowed only locally, and which must become owned before a task is spawned? What happens to the other outbound requests if one of them times out and the fan-out should stop? Who is responsible for cleanup when work is cancelled halfway through? In a thread-per-request model many of these answers are implicit in the stack; in async they become part of your types. Rust's async story holds up in production precisely because it refuses to hide them.

A practical decision order

1. Decide which boundaries are truly IO-bound and therefore worth making async.
2. Decide which data must become owned before a spawned or retried future can outlive the current scope.
3. Decide how cancellation propagates when the parent request no longer cares.
4. Only then choose runtime-specific spawn, join, timeout, or task-group APIs.

Three ideas to anchor the rest of the chapter

If you keep only three things in mind, keep these. Everything later in the chapter is a consequence of them.

Coming from another language

Async exists in C++, C#, Go, and Python too, so you already have an instinct for it. Each of those instincts is right about something and wrong about something else when applied to Rust. The corrections below are the mental-model shifts that matter most before you write any code.

The thread that runs through all four: in Rust, calling an async function produces a value and starts nothing. Execution, thread placement, and cancellation are decisions made by the executor and visible in your types, not conveniences supplied by an ambient runtime.

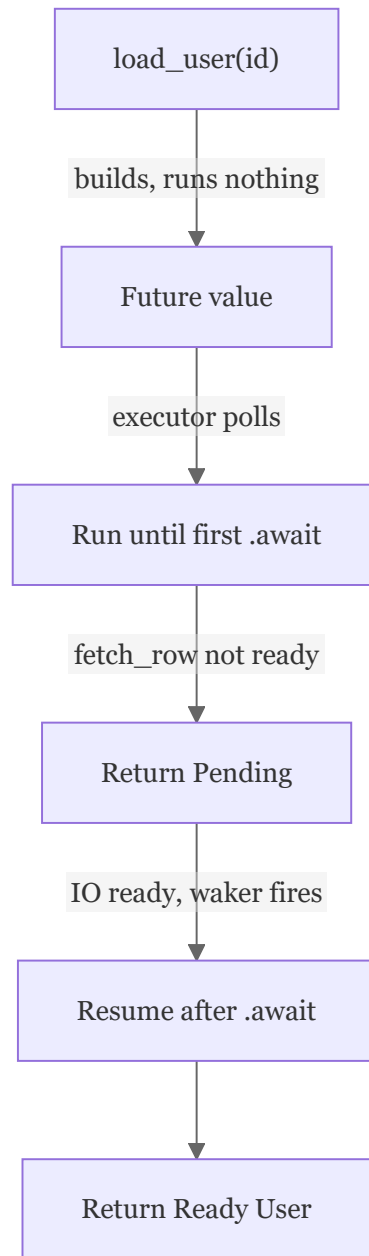
Core concepts

Coroutines are the intuition, futures are the mechanism

“Coroutine” is a fine word for the shape of the code: a function that can pause in the middle and pick up later. But coroutine is not what Rust actually exposes. What you get is a plain value that implements the `Future` trait, and the only way it ever runs is for an executor to call `poll` on it. Keeping that distinction sharp explains almost every surprise newcomers hit.

Rust’s async model

An `async fn` is sugar for a function that returns an opaque future. Inside that future, each `.await` is a place where the current future may hand control back to the executor instead of blocking the thread. Read the listing below as three phases: the call builds a future, the executor drives it, and the first `.await` is the seam where waiting turns into a return of `Pending` rather than a blocked thread.

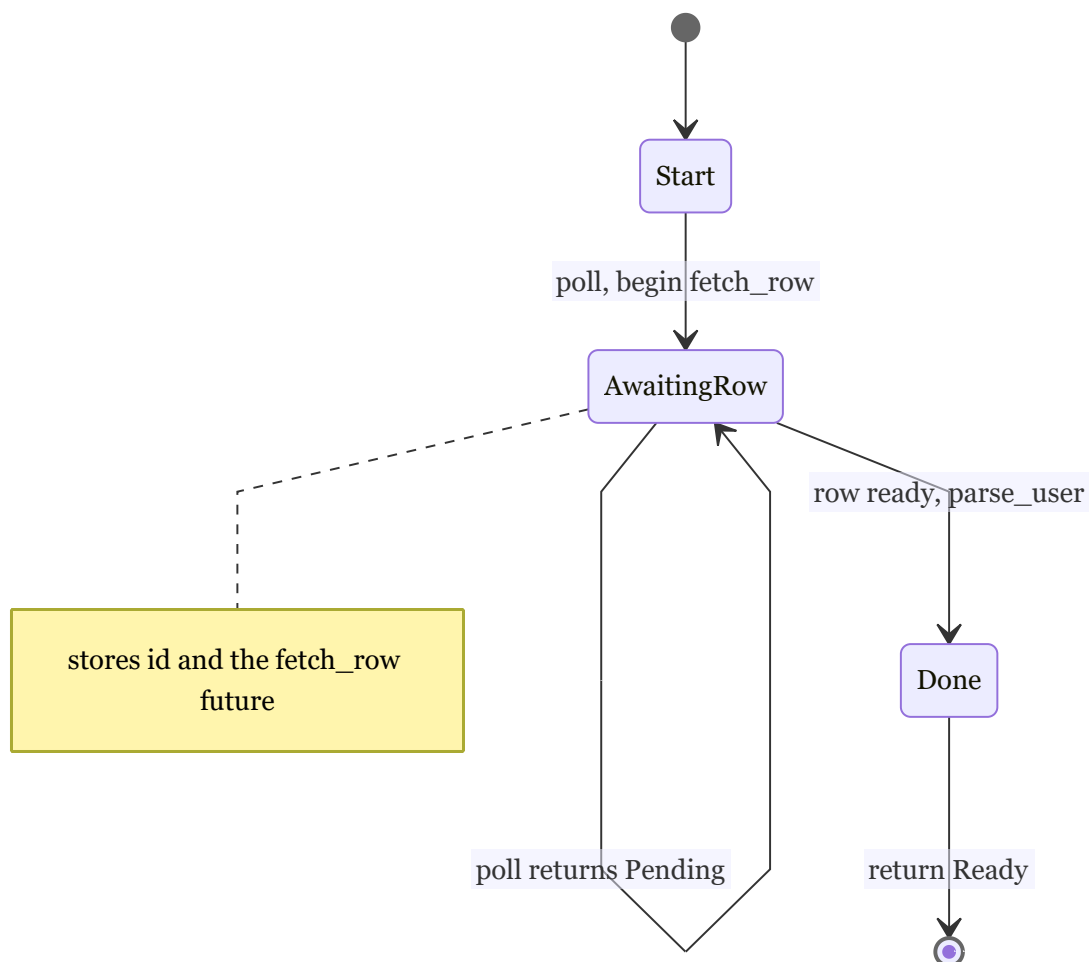


Calling the async fn only builds the future. The executor polls it, the body runs up to `fetch_row().await`, yields `Pending`, and resumes from there once the row is ready.

The operational point is simple: no thread is blocked by the syntax itself. At `.await` the current future yields `Pending`, the executor is free to run other tasks, and it polls this one again only after a waker signals that `fetch_row` can make progress.

What the compiler builds from your async function

The single most useful model of async Rust is this: the compiler rewrites your function into one enum-shaped state machine. Each `.await` becomes a variant, the variant holds exactly the locals that must survive that suspension, and `poll` is a match that advances from one variant to the next. The diagram shows the shape for `load_user` above.



Each `.await` is a state. The `AwaitingRow` variant carries the locals that must outlive suspension, and `poll` moves `Start` to `AwaitingRow` to `Done`.

A useful senior-level translation is this: an async function is ordinary Rust control flow lowered into one enum-like state machine plus the locals it must keep alive between polls. Once you see it that way, the lifetime and Send rules later in this chapter stop being surprises and start being consequences.

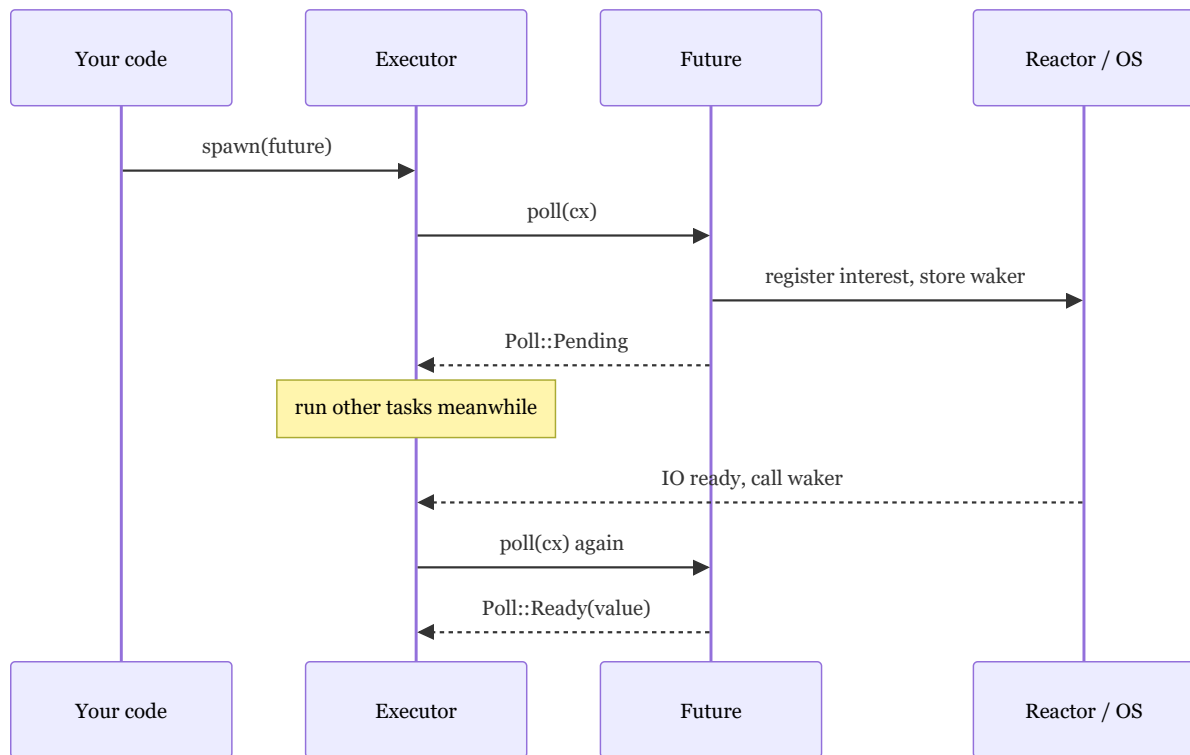
Why `poll` takes a pinned reference

Here is the one subtle part. The state machine the compiler generates can hold a reference into itself, for example a borrow of one local that lives across an `.await`. Such a self-referential value would be corrupted if it were moved in memory, because the internal pointer would still point at the old address. `Pin` is the type-system promise that, once polling has begun, the future will not move. That is why `poll` takes `Pin<&mut Self>` and not a plain mutable reference.

Many everyday futures are effectively movable and implement `Unpin`. The runtime boundary still has to respect the general pinned case, which is why the `poll` signature stays as strong as it does.

Who actually runs the future: executors and reactors

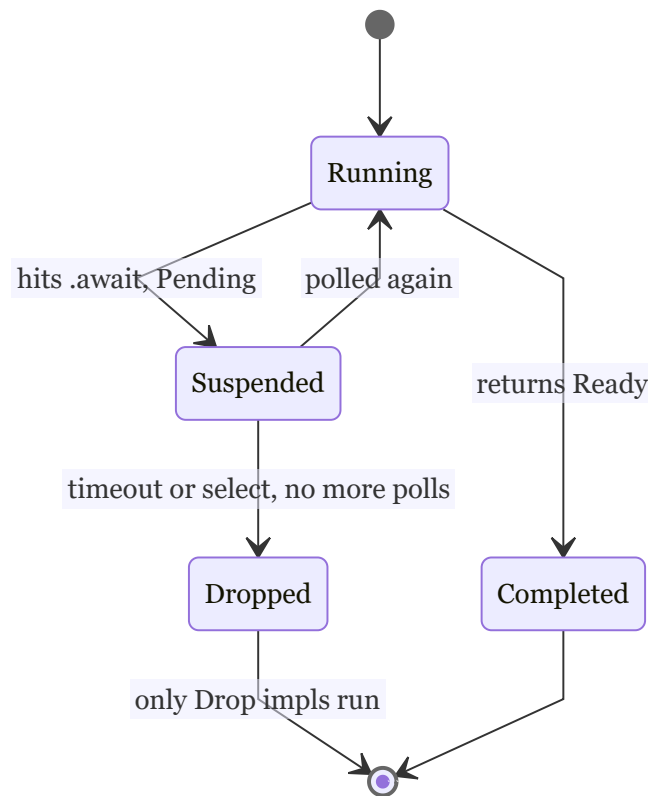
A future is inert, so something has to drive it. In production that something is a runtime, and a runtime is really two cooperating parts. The executor is the scheduler that decides which ready task to poll next; the reactor (or IO driver) is the part that watches the operating system for readiness and fires the waker that makes a sleeping task ready again. The two cards name the pieces; the sequence below shows how one waiting operation travels between them.



The executor polls, the future parks itself with the reactor and returns Pending, and a readiness event wakes the task so the executor polls it to completion.

Cancellation means dropping the future

In a thread, cancellation is awkward because you cannot safely stop a thread at an arbitrary instruction. In async Rust it is the opposite: a future only advances when polled, so to cancel it you simply stop polling it and drop it. A timeout, a `select!` that picks the other branch, or a parent task that finishes will all drop the loser. The catch is that the future stops at whatever `.await` it was parked on, and only its destructors run. Anything that needed a second `.await` to finish is left undone.

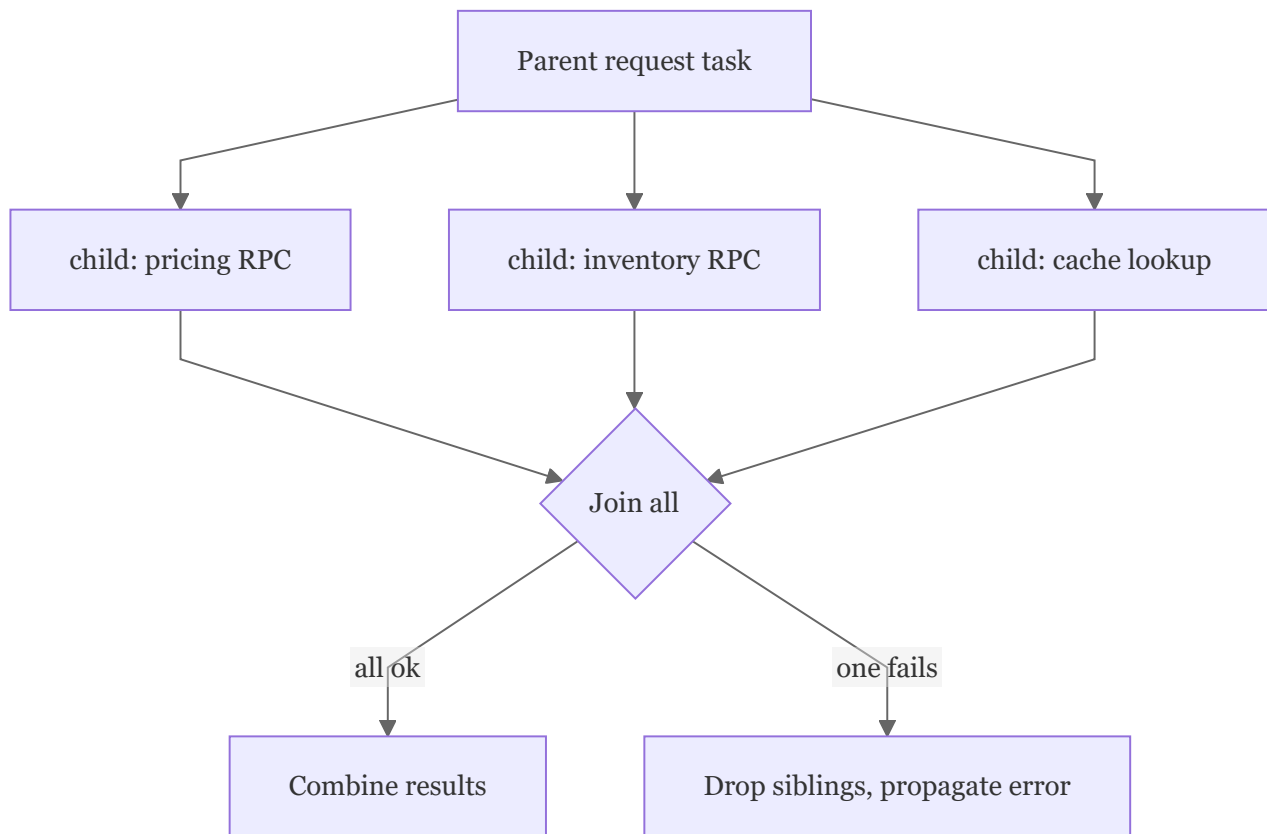


A suspended future is cancelled by being dropped instead of polled again. Cleanup is whatever runs in Drop; logic past the await point never executes.

The important correction is this: “cancel” usually means “drop the future.” If the future must clean up partially acquired state, that cleanup must be explicit in the future’s own design.

Keeping child tasks tied to a parent scope

Back to the opening service: one request fans out to several outbound calls and then joins their results. Structured concurrency is the discipline of making the parent own that whole fan-out, so that the children cannot outlive it. The parent waits for all of them, surfaces the first failure, and drops the rest, which (as the cancellation section just showed) is how you cancel them. The alternative, detaching tasks and forgetting them, is easy to type and hard to operate.



The parent owns every child. Join collects all results, or the first failure drops the remaining siblings and the error flows back up one scope.

In runtime-specific code, this often translates to join-set style task groups, request-scoped task scopes, or parent-owned cancellation tokens. The runtime API may differ. The ownership rule should not.

Async methods in traits, and where they get sharp

You will want async methods on traits the moment you have a repository or service seam with more than one implementation. Recent stable Rust supports `async fn` in traits directly, which covers most generic, statically dispatched code. The edge appears at runtime polymorphism: a `dyn Trait` with an async method has to erase the returned future type, which is where boxing, object safety, and `Send` bounds come back into the design.

Where lifetimes bite in async code

Almost every async error message that mentions lifetimes or `Send` is the state-machine model from earlier coming back to collect. A borrow held across an `.await` becomes a field of the future and therefore lives as long as the future does; a future handed to `spawn` may be moved across threads and must be `Send + 'static`. The recurring fix is the same one from Chapter 1: own the data before it crosses the boundary.

Production patterns

Pitfalls and tradeoffs

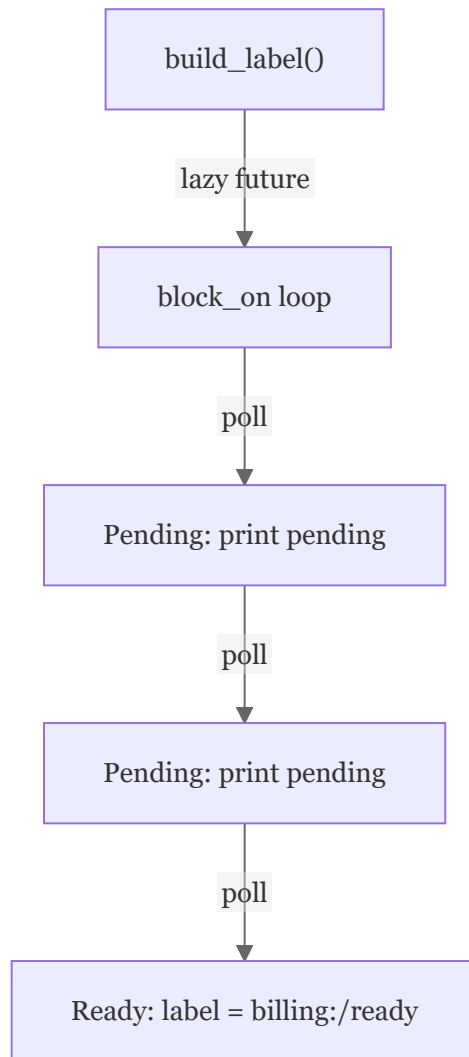
Async Rust is not hard because the syntax is exotic. It is hard because suspension makes ownership, task lifetime, and cancellation more visible. That is exactly why it holds up well in production.

Examples

Example 1: ``async fn``, ``await``, and a tiny ``block_on``

The future is lazy. The tiny executor polls it. Each awaited subfuture yields once before becoming ready.

Watch the loop, not the syntax: each printed poll = pending is one trip around `block_on`, and the future only returns its label after it has been polled enough times to advance through its states.



build_label returns a lazy future; the block_on loop polls it repeatedly, printing pending until the state machine reaches Ready.

Example 24-1. `async_fn_and_manual_block_on.rs` — *Block on*

```

use std::future::Future;
use std::pin::Pin;
use std::sync::Arc;
use std::task::{Context, Poll, Wake, Waker};

struct YieldOnce {
    value: &'static str,
    yielded: bool,
}

```

```

impl YieldOnce {
    fn new(value: &'static str) -> Self {
        Self {
            value,
            yielded: false,
        }
    }
}

impl Future for YieldOnce {
    type Output = &'static str;

    fn poll(mut self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<Self::Output> {
        if self.yielded {
            Poll::Ready(self.value)
        } else {
            self.yielded = true;
            Poll::Pending
        }
    }
}

async fn build_label(service: &'static str, route: &'static str) -> String {
    let left = YieldOnce::new(service).await;
    let right = YieldOnce::new(route).await;
    format!("{}", left, right)
}

struct NoopWake;

impl Wake for NoopWake {
    fn wake(self: Arc<Self>) {}
}

fn block_on<F: Future>(future: F) -> F::Output {
    let waker = Waker::from(Arc::new(NoopWake));
    let mut cx = Context::from_waker(&waker);
    let mut future = Box::pin(future);

    loop {
        match future.as_mut().poll(&mut cx) {
            Poll::Ready(value) => return value,
            Poll::Pending => println!("poll = pending"),
        }
    }
}

fn main() {
    let label = block_on(build_label("billing", "/ready"));
    println!("label = {}", label);
}

```

OUTPUT

```

poll = pending
poll = pending

```

```
label = billing:/ready
```

Calling `build_label` only constructs a future value.

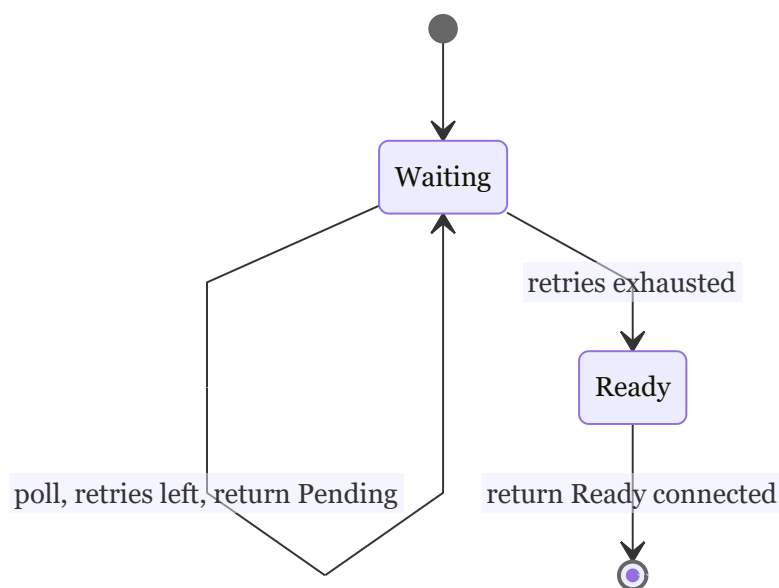
Each awaited subfuture yields one `Pending` before the state machine advances.

The tiny `block_on` loop is the component turning description into progress.

Example 2: a manual `Future` that looks like a state machine

This is the low-level shape the compiler normally generates for you: explicit states, explicit pending, explicit ready.

Follow the `Stage` field and the retry counter: the future stays in `Waiting` and returns `Pending` while retries remain, then transitions to `Ready`. This is exactly the machine the compiler wrote for you in Example 1, only spelled out by hand.



The hand-written poll matches on `Stage`. `Waiting` decrements retries and returns `Pending`; once they hit zero it moves to `Ready` and returns the value.

Example 24-2. `manual_future_state_machine.rs` — State machine

```
use std::future::Future;
use std::pin::Pin;
use std::sync::Arc;
use std::task::{Context, Poll, Wake, Waker};

#[derive(Debug, Clone, Copy)]
enum Stage {
    Start,
    Waiting,
    Done,
}

struct Handshake {
    stage: Stage,
    remaining_polls: u8,
}
```

```

impl Future for Handshake {
    type Output = &'static str;

    fn poll(mut self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<Self::Output> {
        match self.stage {
            Stage::Start => {
                self.stage = Stage::Waiting;
                Poll::Pending
            }
            Stage::Waiting if self.remaining_polls > 0 => {
                self.remaining_polls -= 1;
                Poll::Pending
            }
            Stage::Waiting => {
                self.stage = Stage::Done;
                Poll::Ready("connected")
            }
            Stage::Done => Poll::Ready("connected"),
        }
    }
}

struct NoopWake;

impl Wake for NoopWake {
    fn wake(self: Arc<Self>) {}
}

fn main() {
    let waker = Waker::from(Arc::new(NoopWake));
    let mut cx = Context::from_waker(&waker);
    let mut future = Box::pin(Handshake {
        stage: Stage::Start,
        remaining_polls: 1,
    });

    loop {
        match future.as_mut().poll(&mut cx) {
            Poll::Pending => {
                println!("pending stage = {:?}", future.as_ref().get_ref().stage);
                println!("pending retries = {}", future.as_ref().get_ref().remaining_polls);
            }
            Poll::Ready(value) => {
                println!("ready = {}", value);
                break;
            }
        }
    }
}

```

OUTPUT

```

pending stage = Waiting
pending retries = 1
pending stage = Waiting

```

```
pending retries = 0  
ready = connected
```

The enum-like `Stage` values are the shape the compiler usually hides.

Even a tiny manual future must accept `Pin<&mut Self>` in its `poll` method.

Compiler-generated async futures are easier to write, but they obey the same poll protocol.

The repository also includes standalone Rust files under `examples/ch24_coroutines_futures_and_async_rust/` including extra files for a Send-safe spawn boundary and drop-based cancellation cleanup beyond the in-browser editors.

Summary

CHAPTER 25 · TOKIO

Tokio

Runtime architecture, tasks and scheduling, spawn vs spawn_blocking, timers, async TCP and UDP, async fs, channels, graceful shutdown, backpressure, and production Tokio patterns

Tokio provides the runtime layer for network services, timers, tasks, and blocking work isolation. This chapter covers backpressure, task ownership, shutdown, and runtime configuration.

Builds on Chapters 22, 23, and 24

Chapter 22 established OS-thread boundaries and message passing. Chapter 23 covered synchronization and visibility. Chapter 24 explained futures, `async fn`, and `Pin`. Tokio turns those concepts into an operational runtime for production services.

Opening scenario

An ingress service accepts TCP traffic, sends owned jobs through internal queues, reads filesystem snapshots, handles UDP health messages, and must shut down without losing in-flight work. The business requirement is to use Tokio where work is waiting, isolate blocking or CPU-heavy stages, bound internal queues, and make graceful shutdown part of the runtime contract.

Repository note

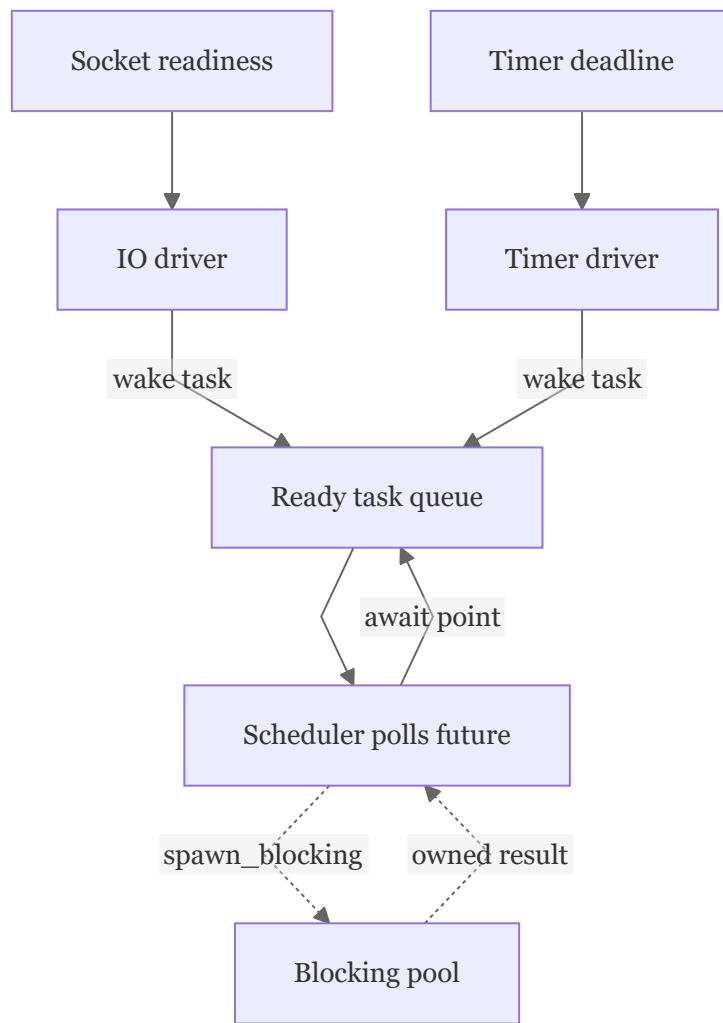
The in-browser editor simulates output so you can focus on runtime shape. In a real Cargo project, the Tokio dependency normally enables the runtime, macros, time, net, fs, sync, and signal features used by this chapter.

Mental model

Core concepts

What the runtime is made of

The runtime macro on `main` hides four moving parts. The thing to notice in the diagram below is that the scheduler never blocks waiting for IO or time: the IO driver and the timer driver turn external events into wakeups and feed ready tasks back to the scheduler, while genuinely blocking work is shunted onto a separate pool so it cannot freeze the workers.



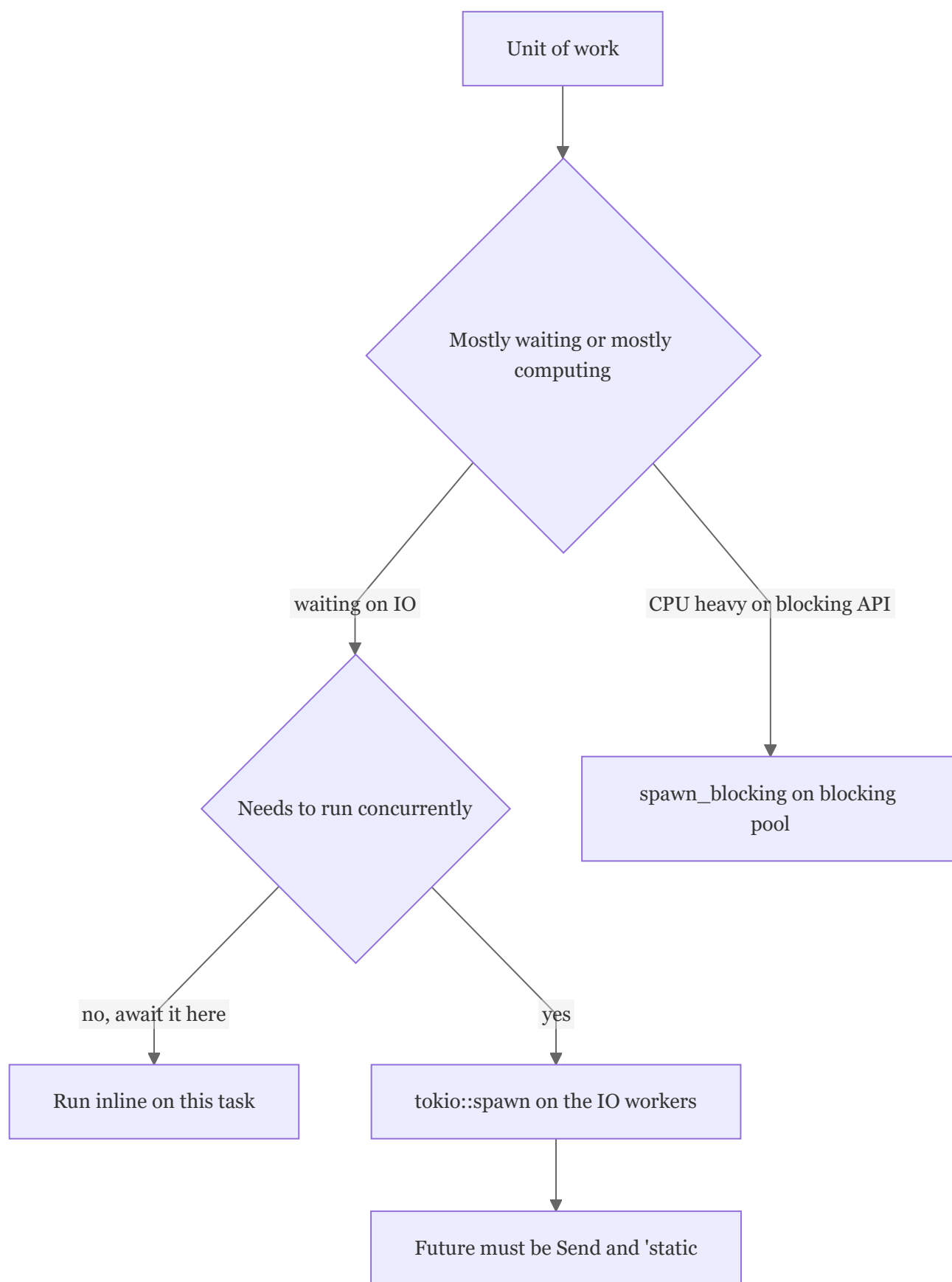
IO and timer events become wakeups, the scheduler polls ready futures on its workers, and blocking closures run off to the side so they never stall the workers.

The usual default for a service is the multithread runtime, because it can schedule `Send` tasks across a pool of worker threads and steal work to keep them busy. A current-thread runtime is still the right choice when the application is small, embedded, or deliberately built around non-`Send` local tasks.

Tasks and scheduling

A task is one future scheduled by the runtime. It is cheap enough to use widely, but not free enough to spray without a policy. A task that never reaches an `await` point monopolizes its worker thread, and a service that spawns without bounds builds its own backlog and memory pressure even when every individual task is small. The first design question for any unit of work is therefore not how to spawn it, but where it belongs.

The decision tree is short. Work that mostly waits on IO stays inline or goes to `tokio::spawn`; work that mostly computes goes to `spawn_blocking`. Putting CPU work on the IO workers is the single most common way to make a Tokio service mysteriously slow.



Decide by the nature of the work: inline for local IO, tokio::spawn for concurrent IO that must satisfy Send, spawn_blocking for anything CPU-bound or blocking.

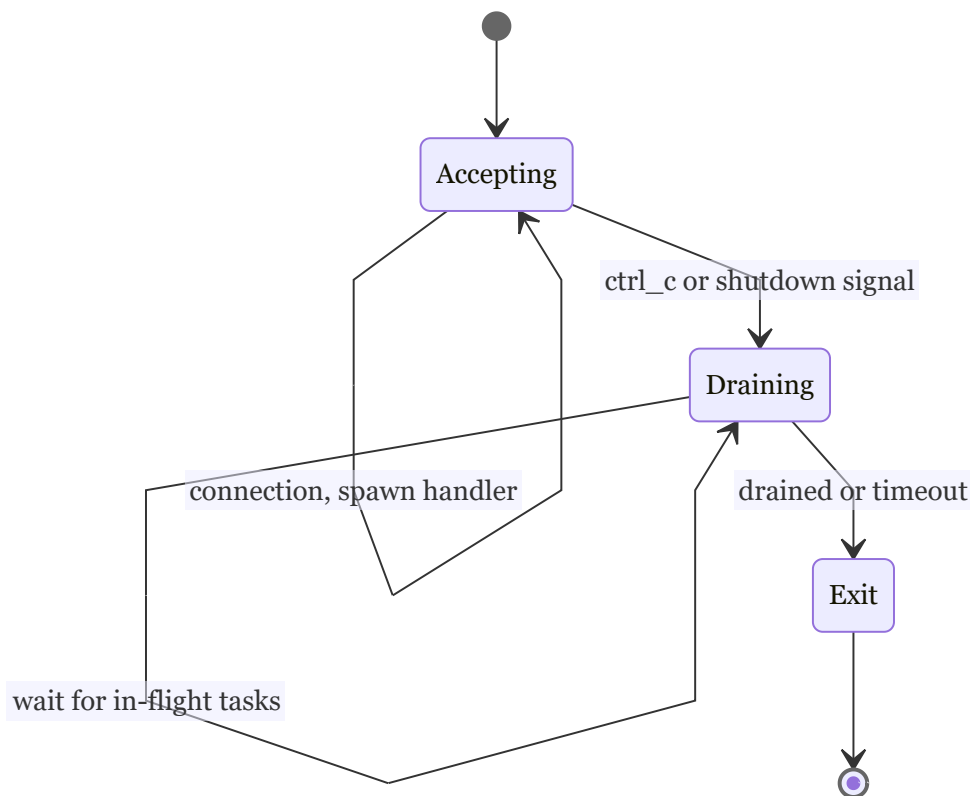
A common production mistake is to reach for `tokio::spawn` when the real problem was concurrency control. Spawning faster than downstream work can finish is not throughput. It is queue growth with nicer syntax.

Tokio channels

A bounded `mpsc` is one of the simplest backpressure tools in Tokio. If a producer should slow down when the consumer falls behind, make the capacity explicit so a full queue makes `send` await, instead of discovering the policy later from a memory graph.

Graceful shutdown

The shape to keep in mind is a small state machine: the accept loop races a stop signal against the next connection, and once the signal wins, the service moves to draining before it exits.



Stop admitting first, drain in-flight work, then exit. The select! below is the Accepting state choosing between a new connection and the stop signal.

Backpressure

How this lands by background

Almost no one meets Tokio with an empty mental model; they arrive with async habits from another ecosystem. The useful thing to know is which of those habits transfer and which one will quietly mislead you. The shift is rarely about API names. It is about where the runtime stops being ambient and starts being something you configure, and where the compiler now insists on facts your previous language let you assume.

Production Tokio patterns

Pitfalls and tradeoffs

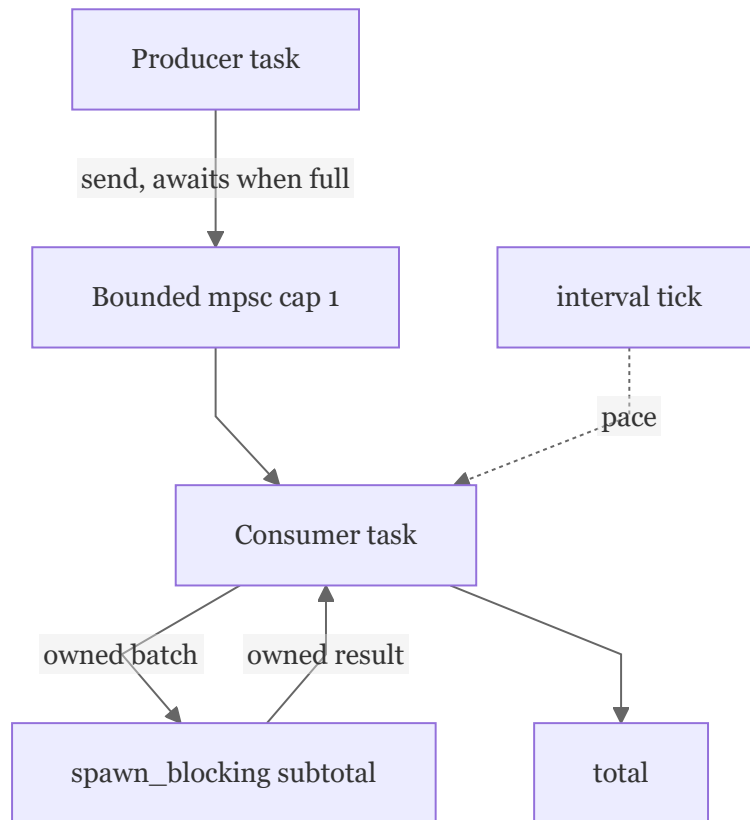
Tokio is excellent at waiting and multiplexing. It is not a free pass to ignore CPU budgets, queue budgets, cancellation, or ownership. The runtime gets calmer when those policies are explicit in the design.

Examples

Example 1: bounded channel, interval pacing, and spawn_blocking

The queue is bounded, so producer speed has a budget. The async side waits on the channel and the timer, and the CPU step moves to the blocking pool explicitly.

Read the pipeline before the code: items flow producer to bounded channel to consumer, the consumer paces itself on a timer, and the only CPU-heavy step is pushed across the blocking boundary so it never holds an async worker. Watch how a channel capacity of one is what couples producer and consumer speed.



A capacity-1 channel makes the producer wait for the consumer, the interval paces the loop, and the CPU subtotal runs on the blocking pool.

Example 25-1. task_boundaries_and_backpressure.rs — Spawn blocking

```

use tokio::sync::mpsc;
use tokio::time::{self, Duration};

#[tokio::main]
async fn main() {

```

```

let (tx, mut rx) = mpsc::channel:::<Vec<u32>>(1);

let producer = tokio::spawn(async move {
    tx.send(vec![1_u32, 2, 3]).await.unwrap();
    tx.send(vec![4_u32, 5]).await.unwrap();
});

let consumer = tokio::spawn(async move {
    let mut interval = time::interval(Duration::from_millis(10));
    let mut batches = 0_u32;
    let mut total = 0_u32;

    while let Some(batch) = rx.recv().await {
        interval.tick().await;

        let subtotal = tokio::task::spawn_blocking(move || batch.into_iter().sum:::<u32>())
            .await
            .unwrap();

        total += subtotal;
        batches += 1;
    }

    (batches, total)
});

producer.await.unwrap();
let (batches, total) = consumer.await.unwrap();

println!("buffer = 1");
println!("batches = {}", batches);
println!("total = {}", total);
}

```

OUTPUT

```

buffer = 1
batches = 2
total = 15

```

A capacity of one forces producer and consumer speed to meet, instead of buffering without limit.

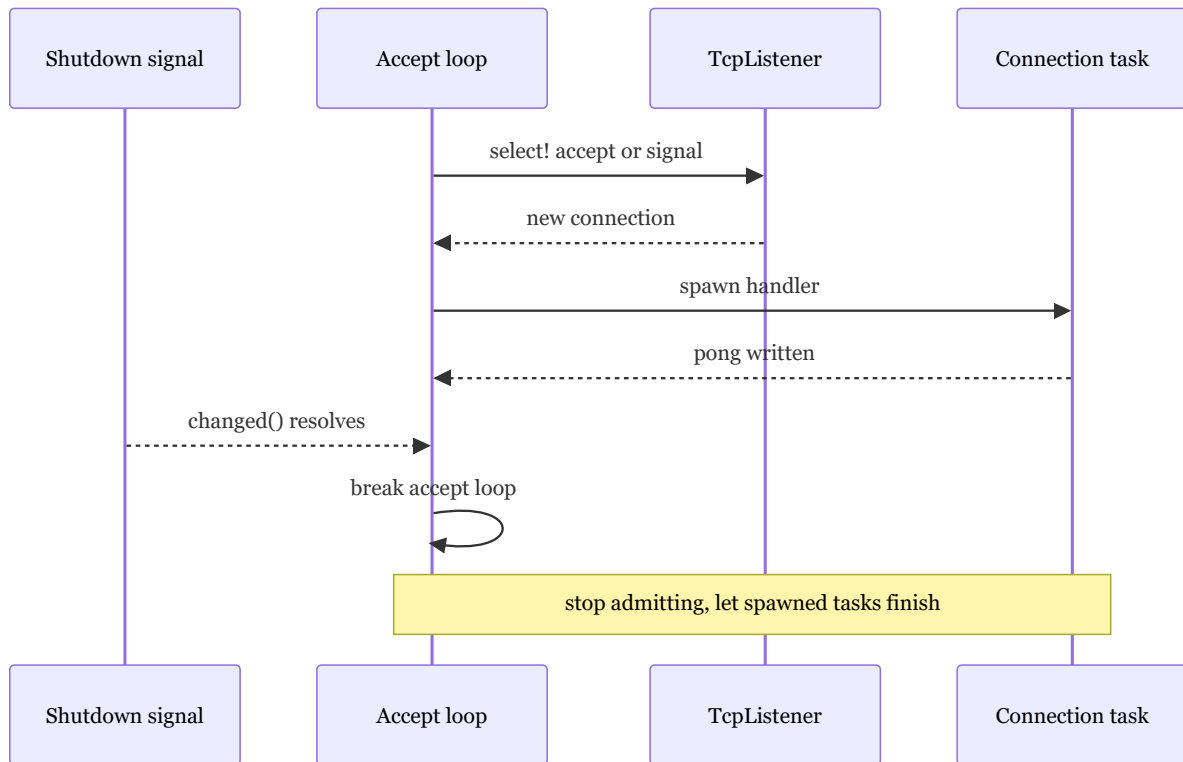
Async waiting stays on the workers; the CPU subtotal step moves behind `spawn_blocking`.

The interval is a runtime wakeup, not a background thread and not a busy loop.

Example 2: TCP accept loop with graceful shutdown

The listener owns admission, per-connection work is spawned deliberately, and a watch signal stops the accept loop cleanly before process exit.

The key moment is the race inside the accept loop: each turn waits on both the next connection and the stop signal at once, and whichever resolves first decides what happens. Follow the messages below to see how a normal accept and a shutdown signal flow through the same loop.



Every loop turn races accept against the stop signal; a connection spawns a handler, while the signal breaks the loop so no new work is admitted.

Example 25-2. tcp_graceful_shutdown.rs — Graceful shutdown

```

use tokio::io::{AsyncReadExt, AsyncWriteExt};
use tokio::sync::watch;

async fn handle(mut stream: tokio::net::TcpStream) -> std::io::Result<> {
    stream.write_all(b"pong\n").await?;
    Ok(())
}

#[tokio::main]
async fn main() -> std::io::Result<> {
    let listener = tokio::net::TcpListener::bind("127.0.0.1:0").await?;
    let addr = listener.local_addr()?;
    let (shutdown_tx, mut shutdown_rx) = watch::channel(false);

    let server = tokio::spawn(async move {
        let mut accepted = 0_usize;

        loop {
            tokio::select! {
                changed = shutdown_rx.changed() => {
                    if changed.is_err() || *shutdown_rx.borrow() {
                        break accepted;
                    }
                }
                result = listener.accept() => {
                    match result {
                        Ok((stream, _peer)) => {

```

```

        accepted += 1;
        tokio::spawn(handle(stream));
    }
    Err(_e) => {
        break accepted;
    }
}
}
}
});

let client_a = tokio::spawn(async move {
    let mut stream = tokio::net::TcpStream::connect(addr).await.unwrap();
    let mut buf = [0_u8; 5];
    stream.read_exact(&mut buf).await.unwrap();
    String::from_utf8_lossy(&buf).trim().to_string()
});

let client_b = tokio::spawn(async move {
    let mut stream = tokio::net::TcpStream::connect(addr).await.unwrap();
    let mut buf = [0_u8; 5];
    stream.read_exact(&mut buf).await.unwrap();
    String::from_utf8_lossy(&buf).trim().to_string()
});

let a = client_a.await.unwrap();
let b = client_b.await.unwrap();

shutdown_tx.send(true).unwrap();
let accepted = server.await.unwrap();

println!("accepted = {}", accepted);
println!("client_a = {}", a);
println!("client_b = {}", b);
Ok(())
}

```

OUTPUT

```

accepted = 2
client_a = pong
client_b = pong

```

Bind plus accept is the core server loop. Admission is an explicit task, not implicit magic.

A watch channel carries the stop signal into the loop, and `select!` makes that branch visible in code. `changed()` also resolves with an error when every sender is dropped, so the branch treats that as a stop; the `borrow()` re-check exists because a watch can carry values other than the stop sentinel.

Connection tasks are separate from listener ownership, which keeps admission, draining, and metrics easier to reason about.

The repository also includes standalone Rust source under `examples/ch25_tokio/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

- Tokio is a runtime made of a scheduler plus IO and timer drivers, not only a macro on the main function.
- `tokio::spawn` runs async work concurrently; `spawn_blocking` is the honest boundary for blocking or CPU-heavy work.
- Timers, TCP, UDP, filesystem APIs, and channels all become calmer once task ownership and shutdown behavior are explicit.
- Bounded queues, semaphores, and admission control are backpressure tools, not optional polish.
- Graceful shutdown is part of the design surface in production Tokio services, not a final cleanup task.

CHAPTER 26 · TASK LIBRARIES AND PARALLEL EXECUTION

Task Libraries and Parallel Execution

Tokio tasks, Rayon, Crossbeam, futures orchestration, CPU vs IO workloads, thread pools, backpressure, cancellation, retry, and task API design

Parallel execution requires matching the library to the workload: async IO, CPU-bound data parallelism, scoped threads, or coordination primitives. This chapter compares those choices through ownership and scheduling.

Builds on Chapters 22 through 25

Chapter 22 covered OS threads and ownership across them. Chapter 23 covered synchronization and visibility. Chapter 24 explained futures and async state machines. Chapter 25 explained Tokio itself. This chapter puts those models side by side with Rayon, Crossbeam, and the futures crate.

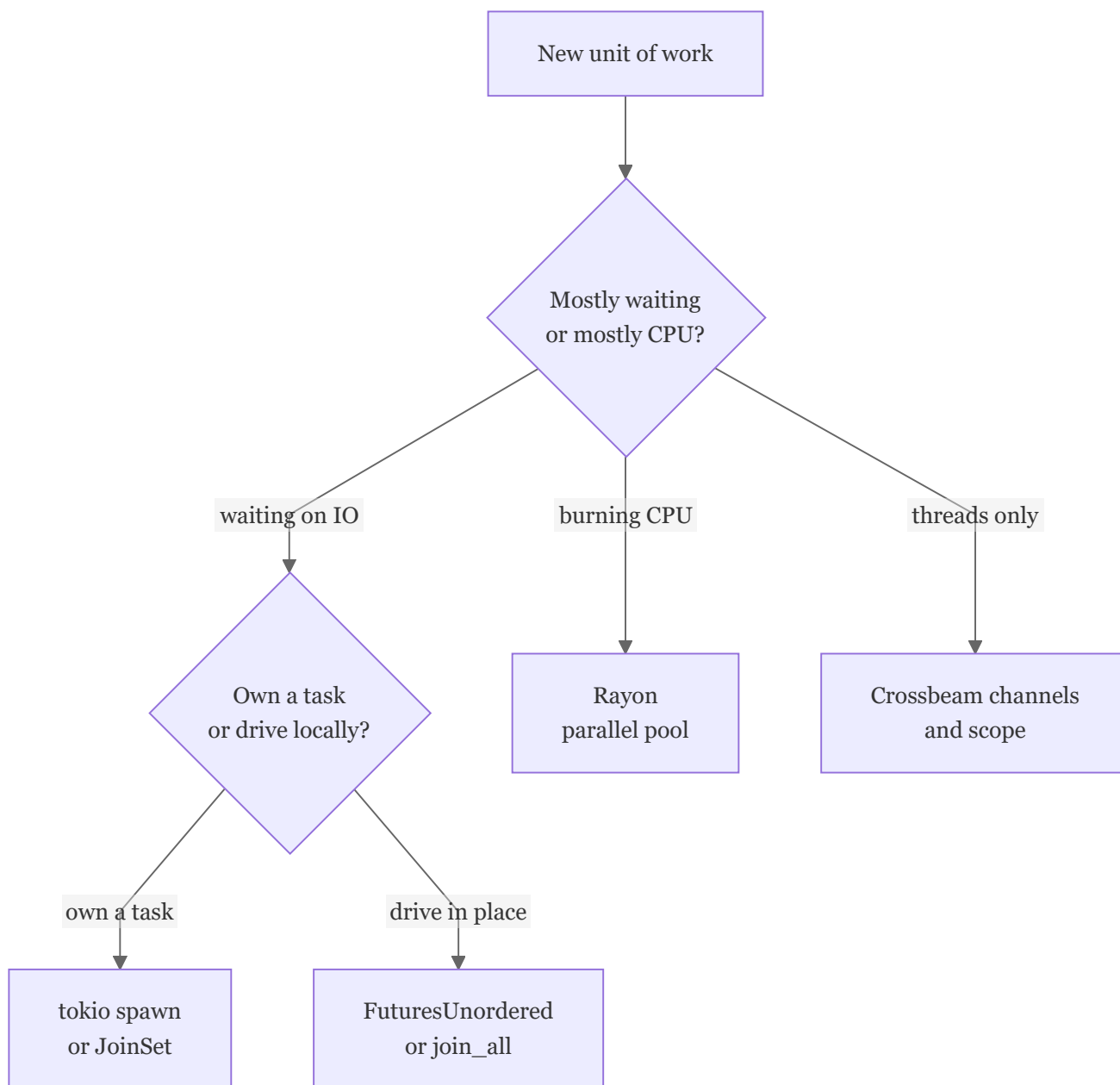
Opening scenario

A service has a socket-heavy front door, CPU-heavy enrichment, bounded control-plane queues, and retry handling under overload. The business requirement is to match each execution lane to its workload: Tokio for waiting, Rayon for CPU-parallel batches, Crossbeam for thread coordination, and futures combinators for local async orchestration.

A practical decision order

1. Ask whether the work is mostly waiting or mostly burning CPU.
2. Ask whether the unit should be a spawned task, a local future, a pool job, or a channel message.
3. Ask where backpressure and cancellation should be visible.
4. Then choose Tokio, Rayon, Crossbeam, futures utilities, or a combination.

The same decision drawn as a flow: one question about the nature of the work splits almost every case, and a second question about whether you want a runtime-owned task or a locally driven future splits the rest.



The workload nature decides the executor family; the ownership question then picks the exact tool within the async branch.

Mental model

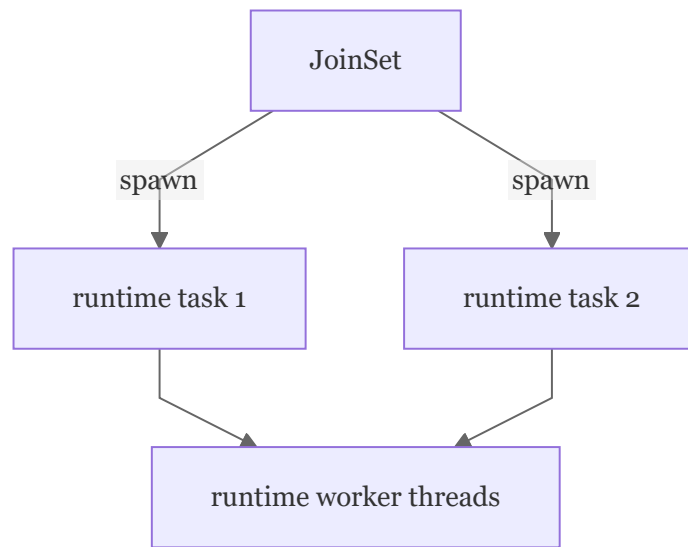
Core concepts

The tool families at a glance

Async task orchestration

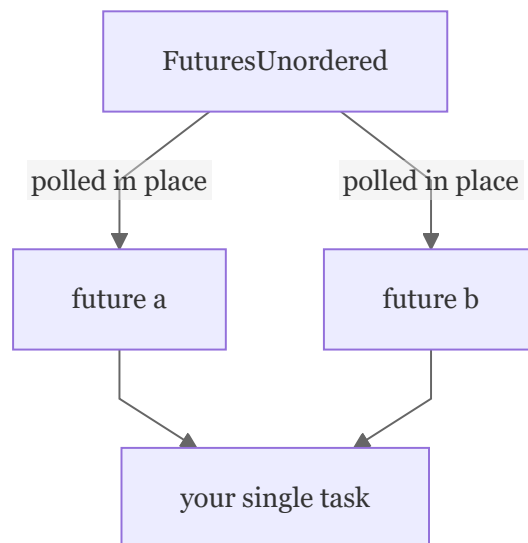
A useful distinction is this: use `JoinSet` when runtime-spawned tasks are the unit you own. Use `FuturesUnordered` when you already have futures and only need to drive them together.

The two differ in where the work actually runs. A `JoinSet` hands each future to the runtime as an independent task, so the runtime's worker threads make progress on them even while you are not awaiting the set. A `FuturesUnordered` keeps the futures inside your own task and only advances them while you poll the collection, so it never crosses a task boundary and never requires the futures to be `Send`.



JoinSet path: each future becomes a separate runtime task that the worker threads advance on their own.

The local path keeps every future inside one task instead, so nothing crosses a task boundary:



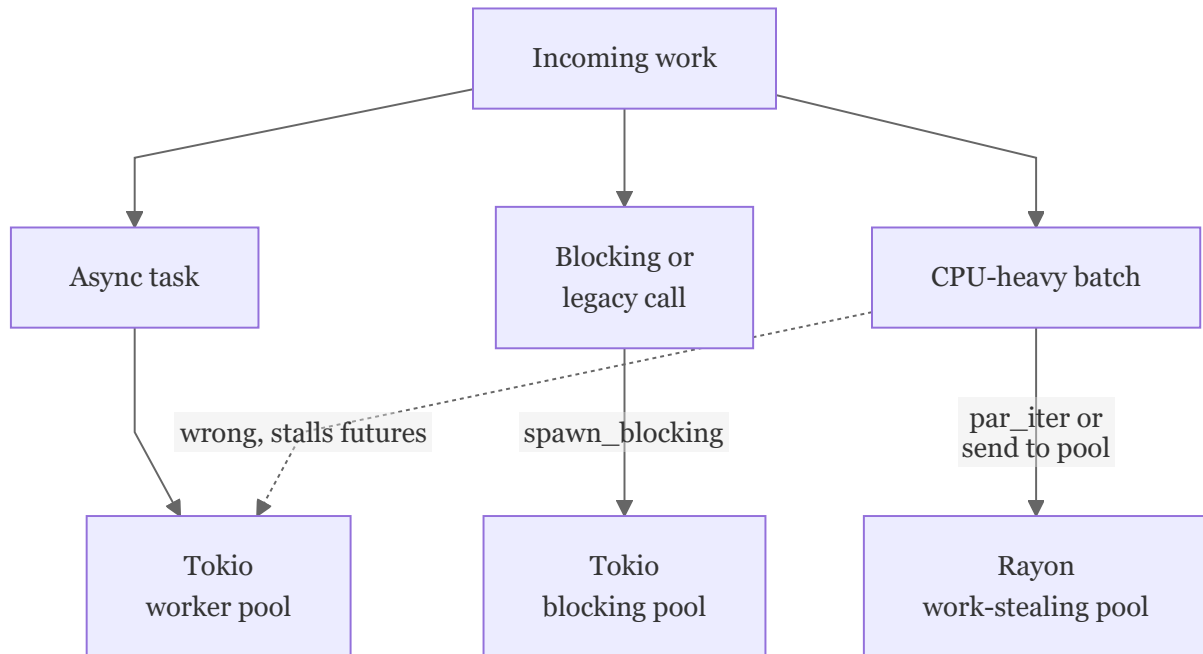
FuturesUnordered path: futures advance inside your own task only while you poll the collection, so they never need to be `Send`.

CPU-bound vs IO-bound workloads

The usual production bug is not using the wrong syntax. It is running CPU-heavy work on the async worker pool or using an async runtime where a straight CPU pool would have been clearer and faster.

Thread pools

A production service rarely has just one pool. The shape that keeps latency predictable is to route each kind of work to the pool built for it: async tasks on the Tokio worker pool, blocking bridges on the Tokio blocking pool, dense compute on a Rayon pool. The diagram below shows the routing, and the one edge to avoid is a CPU loop landing directly on a Tokio worker, where it stalls every unrelated future on that thread.



Async work stays on Tokio workers, blocking calls move to the blocking pool, and CPU batches go to Rayon. The dotted edge is the bug to avoid.

Backpressure and bounded queues

Cancellation and retry patterns

Designing task APIs

What changes by background

Engineers arriving from other languages tend to carry one habit that misfires here: a single concurrency tool that quietly covered both waiting and computing. Rust splits that into separate executors on purpose, so the useful mental adjustment is less about syntax and more about which assumption from your previous runtime no longer holds.

Production patterns

Pitfalls and tradeoffs

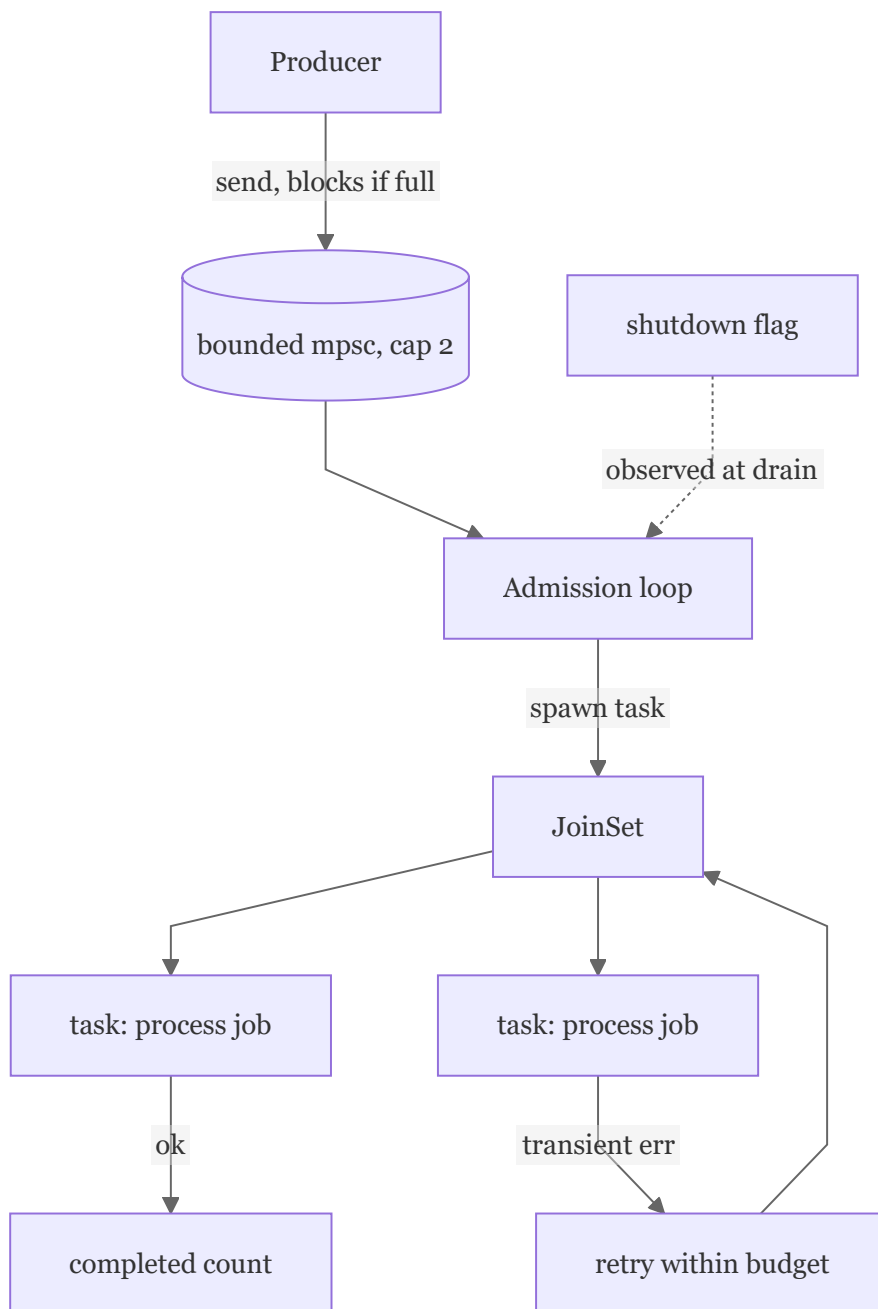
The most common mistake is solving all concurrency with the library you learned first. Tokio, Rayon, Crossbeam, and futures combinators are complementary, not interchangeable.

Examples

Example 1: Tokio tasks, JoinSet, bounded admission, and a visible shutdown signal

The queue is bounded, tasks are explicit runtime units, and retry accounting stays in one orchestration shell instead of being hidden inside many handlers.

What to look at: every job flows through one bounded channel before it ever becomes a task, the tasks live in a JoinSet that drains results as they finish, and the retry and shutdown bookkeeping stays in the orchestration shell rather than scattered across handlers. Trace that path in the diagram before reading the code.



Producers push into a bounded channel; the admission loop spawns each job into the JoinSet, accounts retries centrally, and watches one shutdown flag.

Example 26-1. tokio_joinset_backpressure.rs — Tokio orchestration

```

use tokio::sync::{mpsc, watch};
use tokio::task::JoinSet;

#[derive(Debug, Clone, Copy)]
struct Job {
    id: u32,
    needs_retry: bool,
}

#[tokio::main]
async fn main() {
    let (tx, mut rx) = mpsc::channel::<Job>(2);
    let (shutdown_tx, mut shutdown_rx) = watch::channel(false);

    let producer = tokio::spawn(async move {
        tx.send(Job { id: 1, needs_retry: false }).await.unwrap();
        tx.send(Job { id: 2, needs_retry: true }).await.unwrap();
        tx.send(Job { id: 3, needs_retry: false }).await.unwrap();
    });

    let worker = tokio::spawn(async move {
        let mut set = JoinSet::new();
        let mut completed = 0_u32;
        let mut retries = 0_u32;

        while let Some(job) = rx.recv().await {
            set.spawn(async move {
                if job.needs_retry {
                    Err(job.id)
                } else {
                    Ok(job.id)
                }
            });
        }

        while let Some(result) = set.join_next().await {
            match result.unwrap() {
                Ok(_id) => completed += 1,
                Err(id) => {
                    retries += 1;
                    set.spawn(async move { Ok::<u32, u32>(id) });
                }
            }
        }

        shutdown_rx.changed().await.unwrap();
        (completed, retries, *shutdown_rx.borrow())
    });

    producer.await.unwrap();
    shutdown_tx.send(true).unwrap();
    let (completed, retries, cancelled) = worker.await.unwrap();

    println!("buffer = 2");
}

```

```
println!("completed = {}", completed);
println!("retries = {}", retries);
println!("cancelled = {}", cancelled);
}
```

OUTPUT

```
buffer = 2
completed = 3
retries = 1
cancelled = true
```

`tokio::spawn` creates owned runtime tasks. `JoinSet` drains them as they finish.

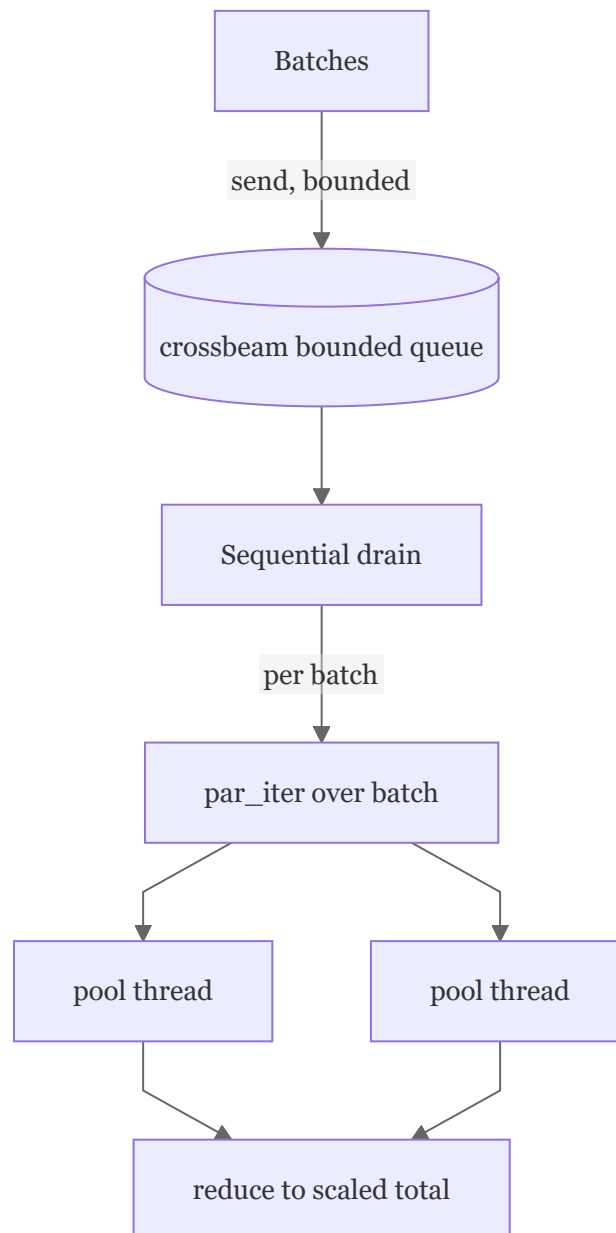
The `mpsc` capacity is visible at the boundary, so producer speed already has a budget.

The worker drains its queue, then observes an explicit shutdown flag at exit. Wiring the signal in once here makes adding real mid-flight cancellation, with `select!` and `abort`, a small step rather than a redesign.

Example 2: Rayon CPU pool plus Crossbeam bounded queue

One pool owns the CPU-parallel loop. One bounded queue controls intake. This is a clearer model than forcing a waiting-oriented runtime to do dense CPU scheduling.

What to look at: the bounded Crossbeam channel is the only intake, each batch that comes off it is handed to Rayon's parallel iterator so the work fans across the pool and rejoins into one sum, and the two stages stay separate (admission is sequential, compute is parallel). The diagram shows that split before the code.



Admission is one sequential bounded queue; the per-batch compute fans out across the Rayon pool and reduces back to a single total.

Example 26-2. rayon_crossbeam_bounded_pipeline.rs — Rayon crossbeam

```

use rayon::prelude::*;

fn main() {
    let pool = rayon::ThreadPoolBuilder::new()
        .num_threads(2)
        .build()
        .unwrap();

    let (tx, rx) = crossbeam::channel::bounded::<Vec<u64>>(2);

    let (batches, total) = std::thread::scope(|scope| {
        scope.spawn(move || {
            tx.send(vec![1_u64, 2, 3, 4]).unwrap();
            tx.send(vec![5_u64, 6, 7, 8]).unwrap();
        });
    });
}

```

```

        tx.send(vec![9_u64, 10]).unwrap();
    });

    pool.install(|| {
        let mut batches = 0_u64;
        let mut total = 0_u64;

        while let Ok(batch) = rx.recv() {
            let subtotal: u64 = batch
                .par_iter()
                .copied()
                .map(|value| value * 2)
                .sum();

            total += subtotal;
            batches += 1;
        }

        (batches, total)
    })
});

println!("batches = {}", batches);
println!("scaled total = {}", total);
println!("pool threads = {}", 2);
}

```

OUTPUT

```

batches = 3
scaled total = 110
pool threads = 2

```

`par_iter` and a small CPU pool make batch arithmetic parallel without manual thread choreography.

The bounded channel makes admission policy visible and keeps the thread-based pipeline simple.

This is CPU work, so a work-stealing data-parallel pool is the right boundary.

The [repository](#) also includes standalone Rust source under `examples/ch26_task_libraries_and_parallel_execution/` including an extra `FuturesUnordered` fan-in example for the futures crate path.

Summary

- Tokio is the right default for waiting-heavy service orchestration, not for every CPU-heavy inner loop.
- Rayon is the right default for CPU-bound data parallelism, especially when work stealing helps uneven batches.
- Crossbeam remains a strong tool for thread-based coordination, bounded queues, and scoped concurrency outside async runtimes.
- The futures crate helps compose futures as data, while Tokio's task tools help schedule runtime-owned async work.

- Backpressure, cancellation, retry budgets, and queue capacity should be visible in task APIs and production metrics.

IO Tricks and Systems Programming Patterns

Buffered IO, zero-copy concepts, memory-mapped files, scatter/gather IO, blocking vs async IO, descriptor ownership, socket tuning, backpressure, and IO profiling

Systems IO performance comes down to a few measurable quantities: how many syscalls you make, how many times bytes are copied, who owns each file descriptor, and whether overload is bounded or hidden. This chapter applies those constraints to ordinary Rust IO and process-boundary code.

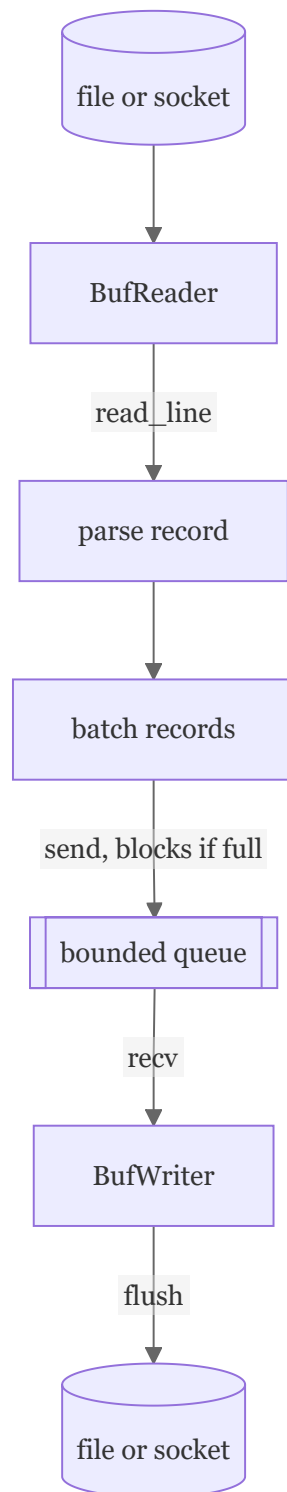
Builds on Chapters 10, 22, 23, 24, 25, and 26

Chapter 10 gave us contiguous buffers and slice-first APIs. Chapter 22 gave us thread ownership. Chapter 23 covered synchronization and visibility. Chapters 24 through 26 separated futures, Tokio, and CPU pools. This chapter ties those ideas to practical IO pipelines and systems edges.

Opening scenario

A log and event pipeline tails files, parses line-oriented records, proxies small TCP responses, and moves bursts through an internal queue. The business requirement is to reduce syscall churn, copies, and unbounded buffering by assigning clear ownership to bytes and handles, then choosing buffering, batching, vectored IO, or async orchestration from measured cost rather than from instinct.

The shape worth holding in your head is a pipeline of stages connected by a bounded queue. Bytes arrive at a reader, get parsed into records, get grouped into batches, cross the queue under backpressure, and leave through a buffered writer. Every box in that pipeline is a place where a copy can be saved or wasted, and the queue in the middle is the only thing that keeps a slow writer from forcing the reader to buffer the whole file in memory.



The pipeline shape matters more than any single wrapper: a bounded queue between reader and writer is what makes backpressure visible instead of letting memory grow without limit.

A practical decision order

1. Count copies, syscalls, and queue hops before rewriting the whole subsystem.
2. Decide where one clear owner should hold the bytes or the handle.
3. Decide whether the hot path wants buffering, batching, or both.
4. Make overload behavior explicit with bounded queues or admission limits.

Mental model

Core concepts

Buffered IO, batching, and backpressure

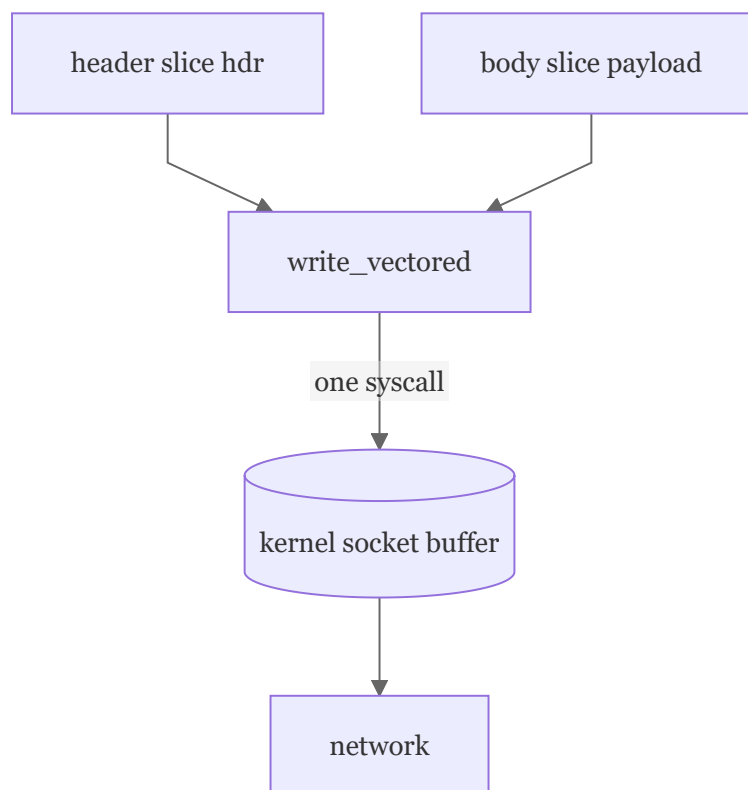
These three levers are often discussed together and confused for one another, but they act on different parts of the pipeline. Buffering changes how often bytes cross into the kernel. Batching changes how many logical records a single flush or wakeup covers. Backpressure changes what happens when the downstream stage cannot keep up. You usually want all three, and you want to know which one you are reaching for.

That pipeline shape is often more important than the specific wrapper type. If the queue is unbounded or the writer flushes every record, the best parser in the system still loses to syscall churn and memory growth.

Zero-copy concepts, scatter/gather, and memory mapping

"Zero-copy" is a precise claim, not a mood. The useful version names exactly which copy you removed and admits which copies remain. Scatter/gather and memory mapping are two concrete techniques that remove specific copies, each with its own cost: vectored IO removes the stitching of small buffers, and mapping removes the read syscall in exchange for page faults and a sharper lifetime story.

The vectored-write idea is the easiest one to picture. Instead of allocating a new buffer to hold the header followed by the body, you hand the kernel a small array of slices and let it gather them in one syscall. The header and body keep their own owners; nothing is concatenated in user space.

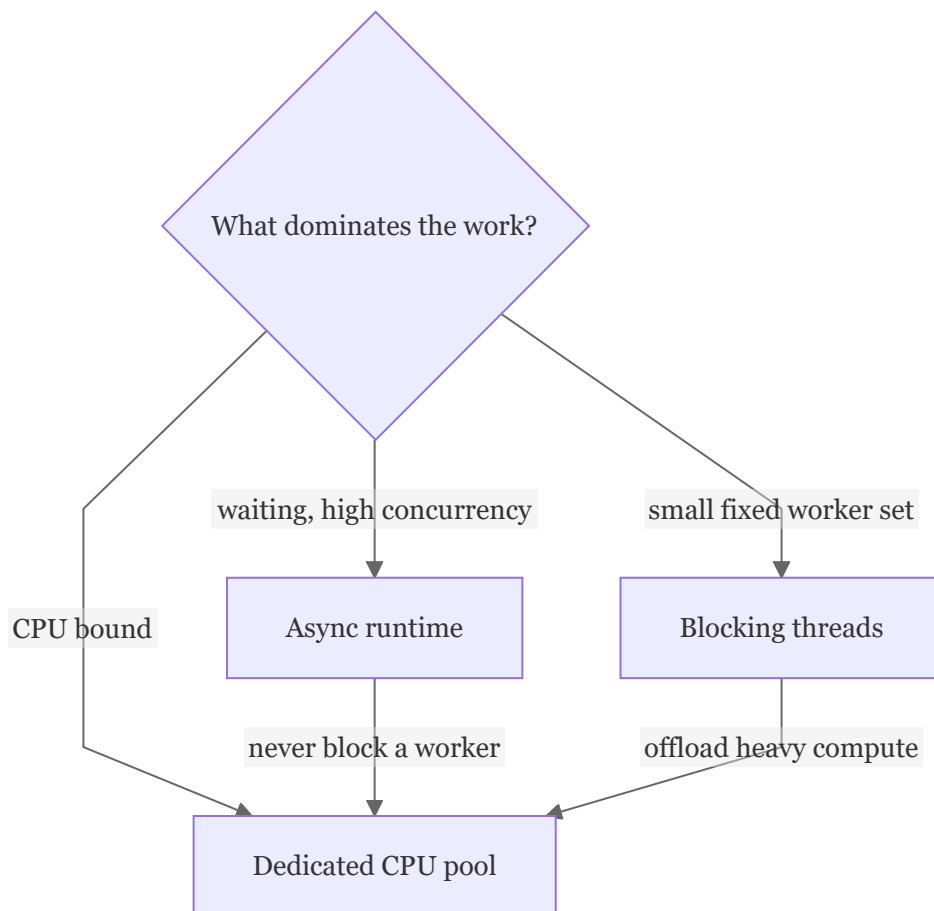


Scatter/gather hands several borrowed slices to one syscall, so a framed response is sent without first building one contiguous owned buffer.

Rust's standard library does not expose a cross-platform memory-mapping API directly. In practice, this is an ecosystem-level choice plus an OS contract. That is exactly why the workload and lifetime story should be clear before you adopt it.

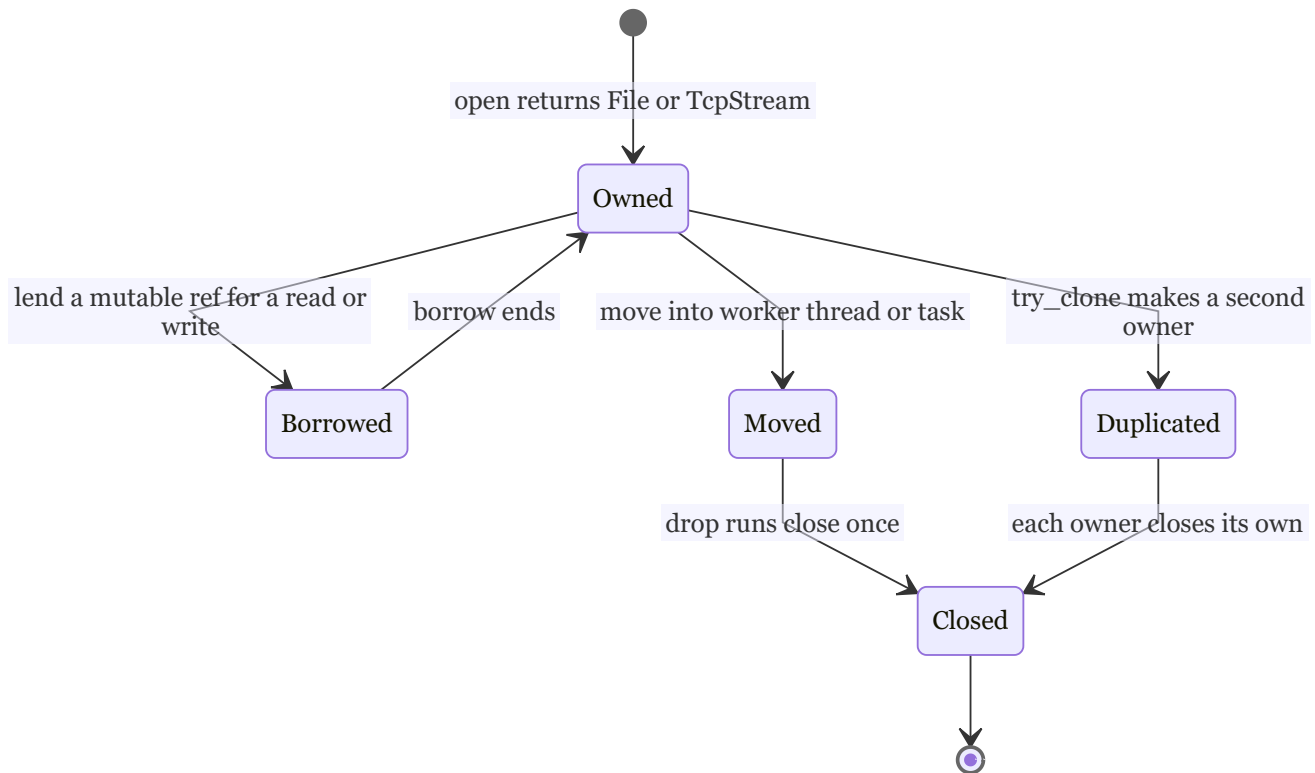
Blocking IO, async IO, and handle ownership

The choice between blocking and async IO is a queueing and scheduling decision, not a matter of taste. A blocking read parks one thread until data arrives, which is simple and cheap when you have a small, fixed set of workers. An async read parks a task and frees the thread to serve thousands of other waiting tasks, which is the right trade only when waiting genuinely dominates and the concurrency is high.



Pick the IO model from where the time actually goes: waiting-heavy and highly concurrent leans async, a small worker set leans blocking, and CPU work needs its own budget either way.

Whichever IO model you pick, the handle underneath is a resource with a close protocol, and ownership is what keeps that protocol correct. Moving a handle into a worker is the calm default; duplicating it with `try_clone` is a real semantic choice; and dropping to a raw descriptor integer is where double-close and use-after-close bugs live. Socket tuning is a smaller, related set of decisions you make per workload.



A handle has one owner at a time. Borrowing is temporary, moving transfers responsibility, try_clone makes a real second owner, and drop runs the single close path.

Blocking versus async is not a style preference. It is a queueing and scheduling choice. Low-concurrency dedicated workers often want blocking simplicity. Large waiting-heavy front doors often want async orchestration. CPU work wants its own budget either way.

IO profiling

An IO subsystem that is slow for an unknown reason cannot be fixed with folklore. Before reaching for a fancy technique, measure where the time and the syscalls go. The checklist below is the minimum set of signals that tells you whether the bottleneck is copying, flushing, queueing, waiting, or parsing, so the optimization you choose addresses the cost that actually exists.

How this lands by background

Most engineers reach this chapter already knowing the IO vocabulary from another ecosystem. The buffering and vectored-IO ideas transfer almost unchanged; the part that trips people up is ownership. In Rust a file handle or a buffer is a value with a single owner and a deterministic close, so the mental shift is about where that ownership lives as bytes cross thread, task, and process boundaries, not about new API names.

Production patterns

Pitfalls and tradeoffs

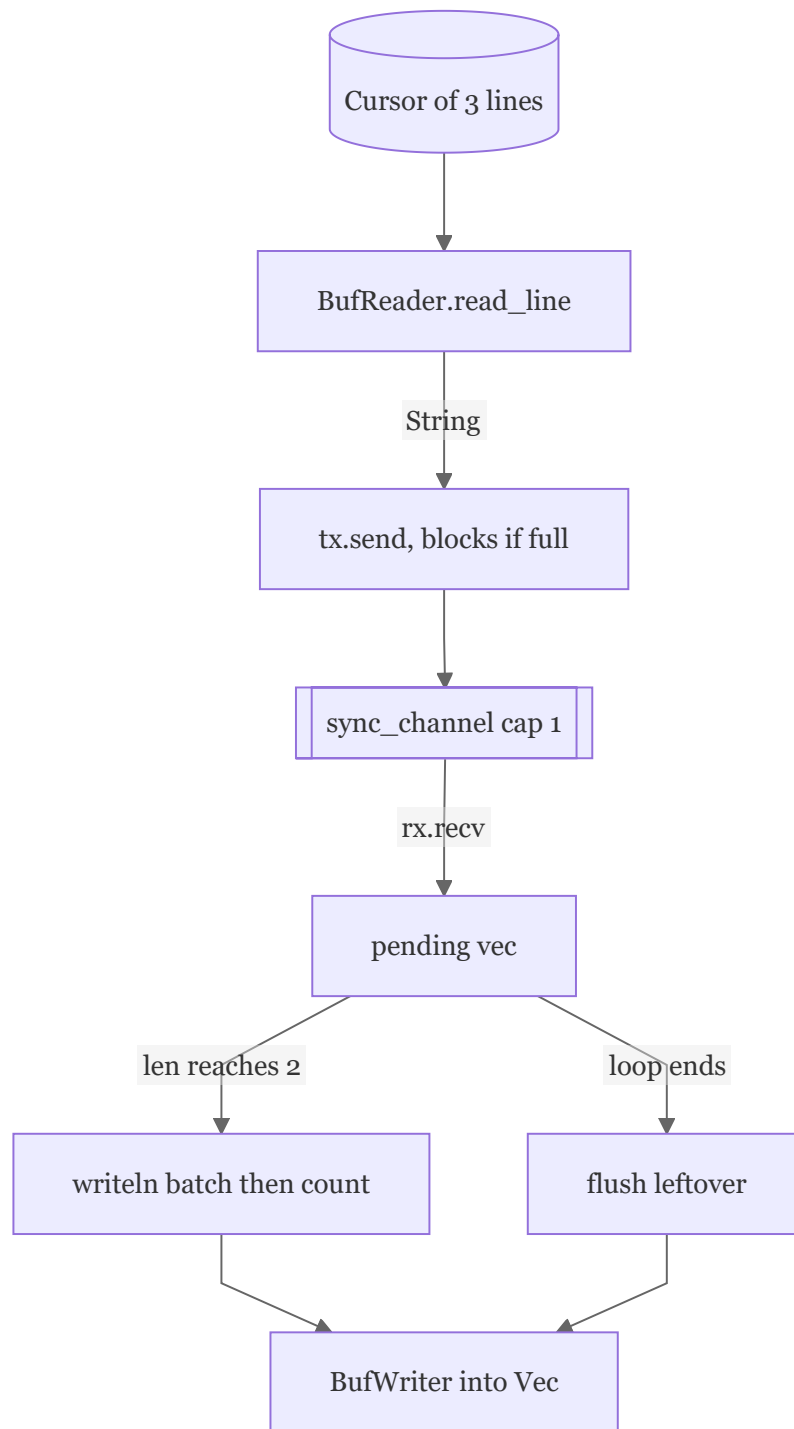
The fastest-looking IO optimization is often the wrong one. If you do not know whether the hot cost is copying, flushing, queueing, waiting, or parsing, you are still choosing by folklore.

Examples

Example 1: buffered reads, buffered writes, and bounded handoff

The producer reads line by line, the queue is bounded, and the consumer batches two records per buffered flush. This is a practical shape for line-oriented log or event pipelines.

What to look at: the producer thread and the consumer run on two clocks joined only by a `sync_channel(1)`. Because the channel holds at most one in-flight item, a fast reader cannot race ahead of a slow writer; the send simply blocks until there is room. On the consumer side, records accumulate in a small pending vector and are flushed two at a time, with a final partial flush so nothing is left behind. Follow one line through the diagram, then read the same path in code.



Three lines flow reader to bounded channel to consumer; the consumer flushes in batches of two and then drains the final leftover, giving two batches for three records.

Example 27-1. buffered_bounded_pipeline.rs — Buffered backpressure

```

use std::io::{self, BufRead, BufReader, BufWriter, Cursor, Write};
use std::sync::mpsc;
use std::thread;

fn main() -> io::Result<()> {
    let input = Cursor::new("alpha\nbeta\ngamma\n".as_bytes());
    let mut reader = BufReader::new(input);

```



```

let (tx, rx) = mpsc::sync_channel:::<String>(1);

let producer = thread::spawn(move || {
    let mut line = String::new();
    let mut sent = 0usize;

    loop {
        line.clear();
        let read = reader.read_line(&mut line).unwrap();
        if read == 0 {
            break;
        }

        tx.send(line.trim_end().to_string()).unwrap();
        sent += 1;
    }

    sent
});

let mut out = Vec::new();
let mut received = 0usize;
let mut batches = 0usize;
let mut pending = Vec::with_capacity(2);

{
    let mut writer = BufWriter::new(&mut out);

    while let Ok(line) = rx.recv() {
        pending.push(line);

        if pending.len() == 2 {
            for item in pending.drain(..) {
                writeln!(writer, "{}", item)?;
            }
            batches += 1;
        }

        received += 1;
    }

    if !pending.is_empty() {
        for item in pending.drain(..) {
            writeln!(writer, "{}", item)?;
        }
        batches += 1;
    }

    writer.flush()?;
}

println!("sent = {}", producer.join().unwrap());
println!("received = {}", received);
println!("batches = {}", batches);

```

```
println!("bytes = {}", out.len());  
Ok::<(), ()>()  
}
```

OUTPUT

```
sent = 3  
received = 3  
batches = 2  
bytes = 17
```

BufReader and BufWriter smooth many small reads and writes into fewer kernel interactions.

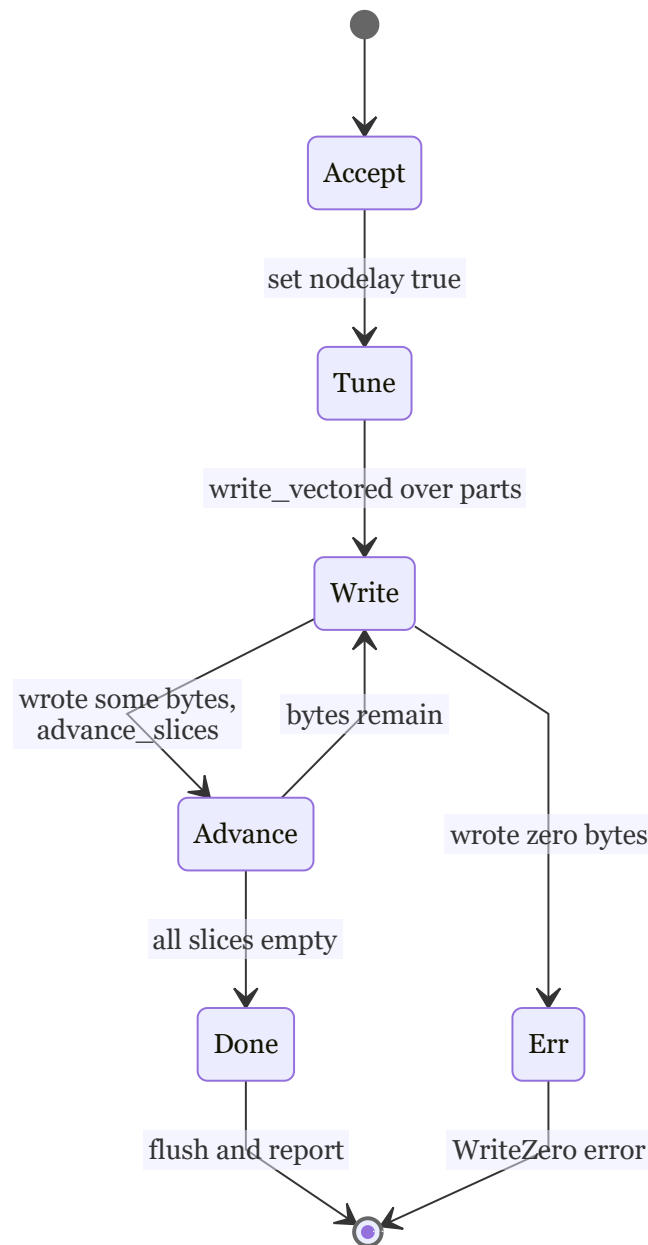
sync_channel(1) means the producer eventually waits when the consumer falls behind.

Two logical records per flush is a small but explicit batching policy.

Example 2: scatter/gather response with socket tuning and stream handoff

The listener moves the accepted stream into one owner, sets a latency-sensitive socket option, and writes a small header plus body with vectored IO.

What to look at: the accepted stream is moved into the server thread so exactly one owner drives the response, and the response is two borrowed slices, a header and a body, that are never concatenated. The write_all_vectored helper is the load-bearing detail: a single write_vectored can write only part of the buffers, so the loop calls IoSlice::advance_slices to skip the bytes already sent and keeps going until both slices are fully drained. The diagram is that retry loop.



Partial writes are part of the contract: advance past the bytes already sent and loop until both slices are empty, treating a zero-byte write as a closed socket.

Example 27-2. `scatter_gather_socket_handoff.rs` — Gather socket

```

use std::io::{self, IoSlice, Read, Write};
use std::net::{TcpListener, TcpStream};
use std::thread;

fn write_all_vectored(stream: &mut TcpStream, mut parts: &mut [IoSlice<'_>]) -> io::Result<> {
    while !parts.is_empty() {
        let wrote = stream.write_vectored(parts)?;
        if wrote == 0 {
            return Err(io::Error::new(
                io::ErrorKind::WriteZero,
                "socket closed before full response",
            ));
        }
        IoSlice::advance_slices(&mut parts, wrote);
    }
}

```

```

    }

    Ok(())
}

fn main() -> io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:0")?;
    let addr = listener.local_addr()?;

    let server = thread::spawn(move || -> io::Result<(bool, usize)> {
        let (mut stream, _) = listener.accept()?;
        stream.set_nodelay(true)?;

        let mut parts = [IoSlice::new(b"hdr:"), IoSlice::new(b"payload")];
        write_all_vectored(&mut stream, &mut parts)?;
        stream.flush()?;
        Ok((stream.nodelay()?, parts.len()))
    });

    let mut client = TcpStream::connect(addr)?;
    let mut buf = [0_u8; 11];
    client.read_exact(&mut buf)?;

    let (nodelay, parts) = server.join().unwrap()?;
    println!("nodelay = {}", nodelay);
    println!("vectored parts = {}", parts);
    println!("client = {}", String::from_utf8_lossy(&buf));
    Ok(())
}

```

OUTPUT

```

nodelay = true
vectored parts = 2
client = hdr:payload

```

This example already uses a small `write_all_vectored` helper so the logical response is fully sent even if one vectored syscall only writes part of the buffers. Keep that pattern near the socket boundary in real services instead of assuming small responses always finish in one call.

`IoSlice` avoids stitching a tiny response together just to make one small write call.

The accepted stream moves into one thread and one closure. That keeps ownership simple.

`set_nodelay(true)` is a workload decision. Small request-response protocols often care more about latency than about coalescing tiny writes.

The repository also includes standalone Rust source under `examples/ch27_io_tricks_and_systems_programming_patterns/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Next chapter

This chapter kept everything inside Rust, where ownership decided who held each buffer and handle. Chapter 28 pushes those same ownership and lifetime questions across a language boundary into C and

C++, where the compiler can no longer check the contract for you and the handoff has to be encoded by hand.

Summary

- Buffered IO reduces syscall churn, but it does not remove parse cost or ownership decisions.
- Zero-copy is only meaningful when you say which copy disappeared and which copies still remain.
- Memory mapping, vectored IO, and raw handle escape hatches are strongest when the workload and invariants are explicit.
- Blocking and async IO solve different scheduling problems. CPU work still needs its own budget either way.
- Backpressure and profiling belong in the design from the start: bounded queues, flush policy, syscall counts, and p99 latency all matter.

PART V

Integration, Distributed Systems, and HPC

When your problem leaves a single process: FFI, WASM, brokers, distributed and MPI workloads, GPUs, and the profiling they demand.

-
- 28** C++ Integration
 - 29** JavaScript and C++ Integration for WASM
 - 30** AMQP and Message Brokers
 - 31** Distributed Task Execution
 - 32** MPI and High-Performance Computing
 - 33** Performance-Oriented Rust
 - 34** Memory Profiling
 - 35** Performance Profiling
 - 36** Distributed Tasks Profiling
 - 37** CUDA and GPU Acceleration
 - 38** Merkle Tree Games and Challenges
 - 39** Graph Search Games
 - 40** Matrix Optimization Games
-

CHAPTER 28 · C++ INTEGRATION

C++ Integration

Rust FFI fundamentals, C ABI seams, ownership, status-code error handling, unwind policy, and interop testing

Linking Rust to a C or C++ library is not a syntax detail; it is a binary contract the compiler cannot check for you. Calling convention, memory layout, who frees what, how long a pointer stays valid, and what happens on failure all become things you state by hand. This chapter treats that boundary as a small, deliberate seam to design rather than a library to call.

Builds on Chapters 08, 09, and 17

Chapter 08 established unsafe Rust and explicit invariants. Chapter 09 covered raw-pointer-adjacent ownership tools such as Box, Arc, and pinning. Chapter 17 showed how to refactor toward honest boundaries. C++ integration is where all three become operational.

Opening scenario

A product is replacing one hot native subsystem with Rust while keeping an existing host process, plugin loader, and native test harness. The business requirement is a narrow ABI contract with explicit layout, pointer validity, memory ownership, status-code errors, and no cross-language unwinding.

A reliable decision order

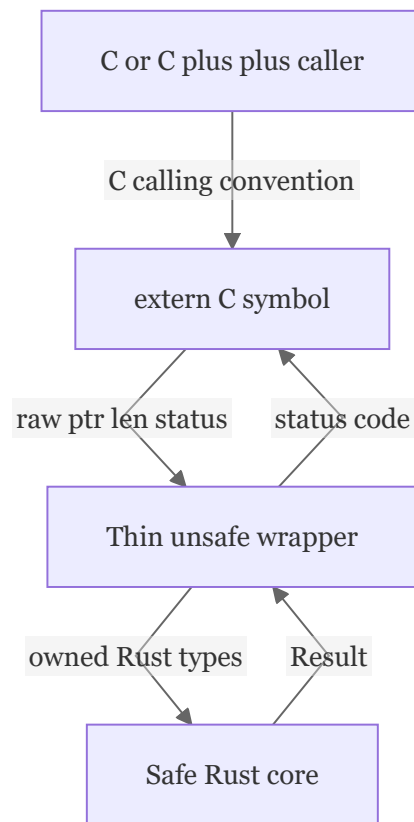
1. Flatten the seam to a C ABI contract unless you have a strong reason for a richer direct C++ bridge.
2. State ownership in raw terms: pointer plus length, caller-allocated buffer, or opaque handle.
3. Translate errors to status codes or explicit error payloads.
4. Ban cross-language unwinding and test the wrapper as a first-class subsystem.

Mental model

Core concepts

The four FFI fundamentals

Almost every Rust interop layer is built from the same four primitives. They are small, but together they are what turns an internal Rust function into a symbol another language can call without guessing about representation. The diagram shows how a single call splits into a thin unsafe edge over a larger safe core; the cards below name each piece.



*One narrow unsafe edge sits between the foreign caller and a large safe Rust core.
Raw types live only on the edge; rich types live behind it.*

What may and may not cross a C ABI boundary

The single most useful filter when designing an interop layer is to sort every type into one of two buckets: representations with a stable, agreed-upon shape that may cross the seam, and Rust-specific types whose layout is an implementation detail that must stay behind it. The rules below are that filter spelled out.

A useful correction for C++ teams is this: “C++ integration” often still means “a deliberate C ABI seam around C++ code.” That seam buys you a smaller contract surface and fewer compiler- and standard-library assumptions.

Calling C from Rust

When Rust is the caller, you describe the foreign function in an extern "C" declaration block, which tells the compiler the symbol exists elsewhere and must be called with the C calling convention. The call itself is unsafe, because Rust cannot verify that the foreign code honors the signature, so the practice is to wrap that one call in a small safe function whose body documents why the call is sound. In a real Cargo build the native library is linked through build-system configuration, typically a build.rs that points the linker at the artifact. In this chapter’s runnable demo the symbol is defined in Rust with the C ABI so the example stays self-contained while exercising the exact same declaration-and-wrap pattern.

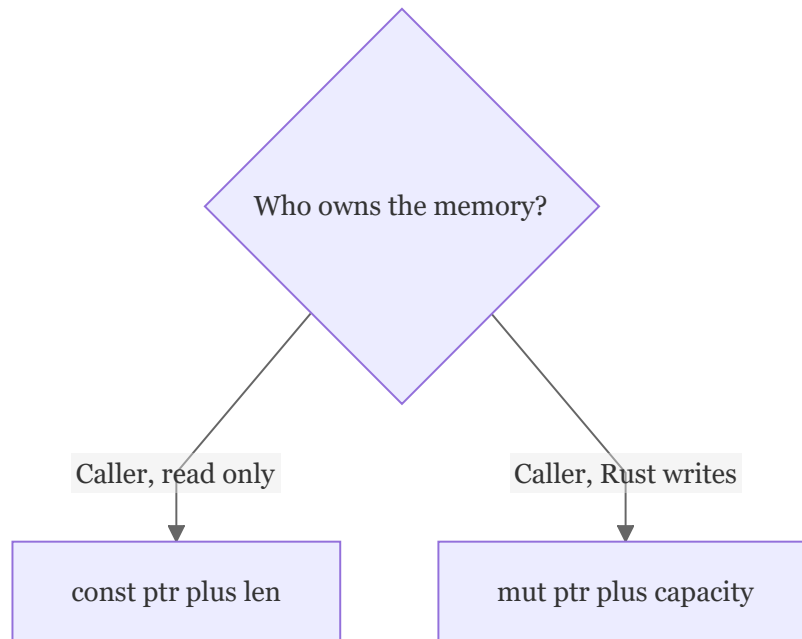
Calling Rust from C and C++

When Rust is the callee, the goal is to expose a deliberately tiny surface that another language can rely on: extern "C" functions marked so their names are not mangled, C-friendly layout for any shared struct, raw buffers or opaque handles instead of Rust ownership types, and an explicit destroy function for anything Rust allocates and foreign code later frees. The crate is built as a C-facing static or dynamic library rather

than as a Rust ABI artifact, and the public header that describes these symbols becomes the real contract. Keeping that header small is the whole discipline: every symbol on it is something you have promised to keep stable.

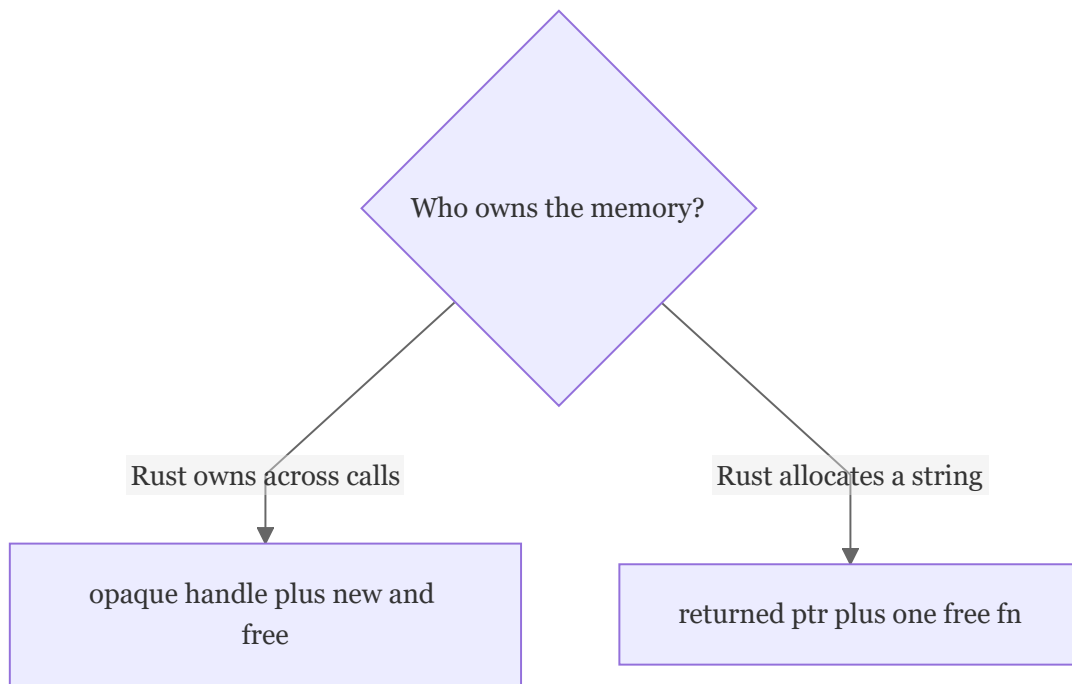
Modelling ownership across the seam

Ownership is the part of the contract that has no syntax in C, so you express it by choosing which of a few fixed shapes each parameter takes. The decision is always the same question asked precisely: who allocated this memory, who is allowed to read or write it, and who is responsible for freeing it. The diagram sorts the four common answers; the cards below give the signature pattern for each.



Caller-owned memory: a const pointer plus length for read-only views, or a mutable pointer plus capacity for buffers Rust fills.

When Rust owns the memory instead of the caller, the same question routes to the other two shapes:



Rust-owned memory: an opaque handle with explicit new and free functions, or a returned pointer paired with exactly one free function.

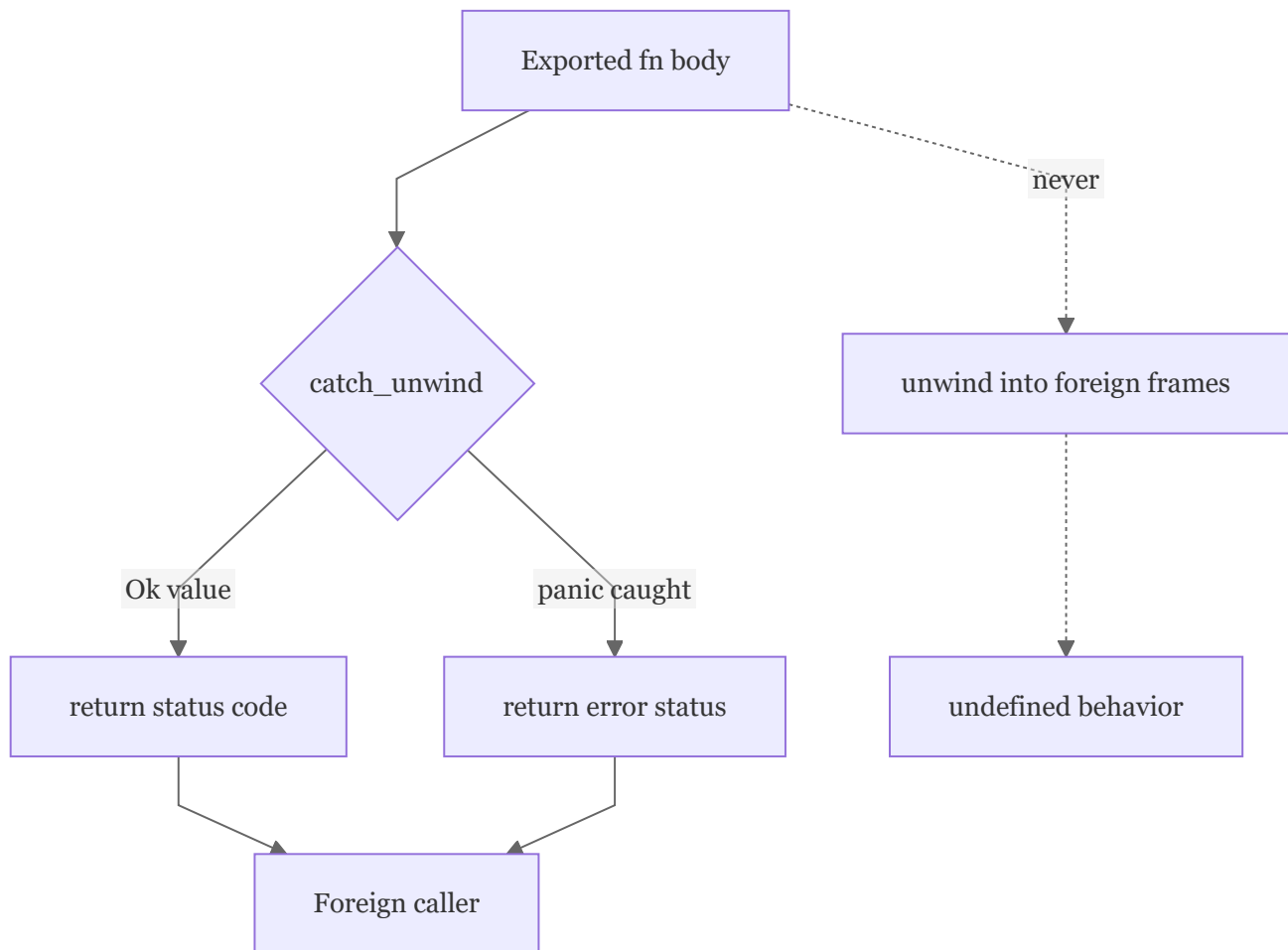
A Rust reference encodes aliasing and lifetime promises the C or C++ side cannot honor, which is why a mature FFI surface flattens everything into these four shapes. The payoff is that ownership becomes readable from the function signature instead of relying on a comment everyone has to remember.

Returning errors across the seam

A Rust `Result<T, E>` has no C representation, so failure has to be re-expressed in terms the foreign side can read. The conventional answer is an integer status as the return value, with any successful payload delivered through caller-owned out parameters. The richness of your error type stays inside Rust; only its outcome crosses the line.

Keeping unwinding inside its own language

The most dangerous thing that can cross an FFI boundary is not a bad value but an in-flight unwind. A Rust panic or a C++ exception unwinding through frames the other language does not know how to clean up is undefined behavior, and it tends to crash far from the cause. The operable policy is to stop every unwind at the boundary: catch it, convert it to a status code, and let only ordinary return values cross. The diagram shows that gate; the rules below state it precisely.

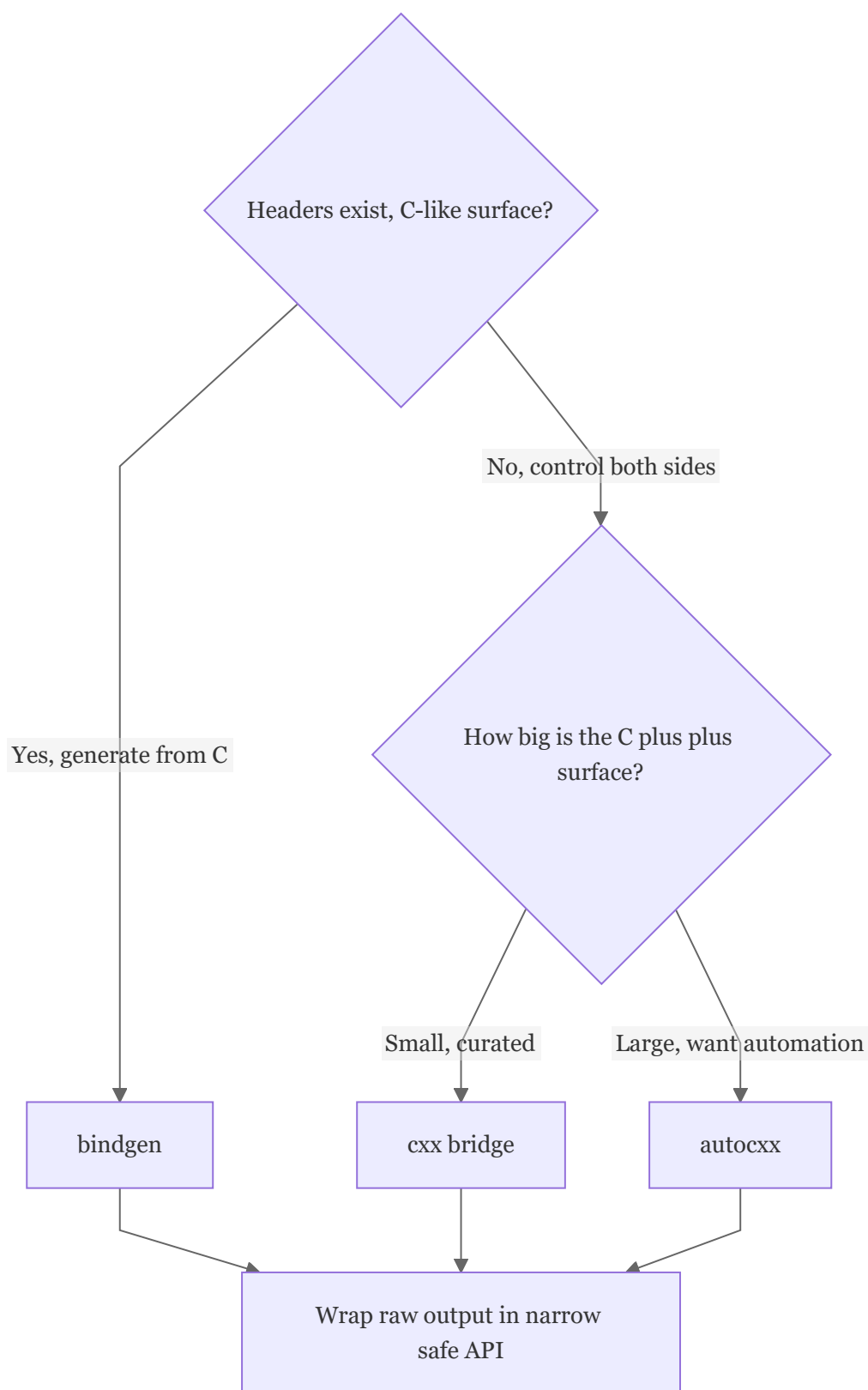


An exported function catches its own panic and returns a status. The dashed path, an unwind leaking into foreign frames, is the one you must make impossible.

The conservative rule is also the most operable one: no cross-language unwinding. Translate failure to an integer status or explicit error object before control crosses the boundary.

Choosing a binding tool: `bindgen`, `cxx`, or `autocxx`

These tools do not change the contract you owe; they change how much of the boilerplate is written for you. The choice mostly follows two questions: do you control both sides of the boundary, and how large is the C++ surface you need to reach. The decision tree below routes those questions to a default; the cards explain what each tool is actually for.



The tool depends on whether headers already exist and how much C++ you must reach. Whatever the tool, the generated surface still gets wrapped in a narrow safe API.

ABI stability

Testing and sanitizing FFI layers

How this lands by background

Most engineers reaching for FFI already carry a model from another ecosystem, and the useful question is which part of that model transfers and which part will quietly mislead you. The shift is almost never about which functions to call. It is about who is now responsible for ownership, lifetime, and failure once a boundary the compiler cannot see sits between your code and the other language.

Production patterns

Pitfalls and tradeoffs

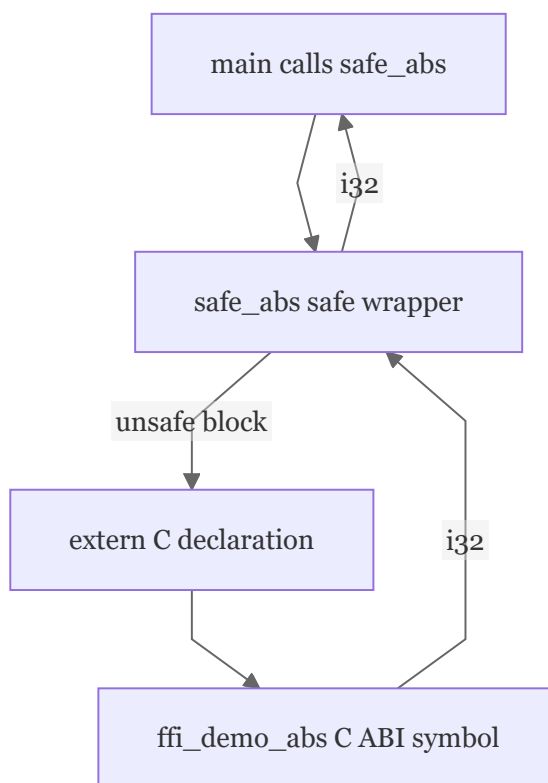
The most expensive FFI bug is rarely a missing semicolon in the header. It is a boundary that looked simple, but never actually said who owns the memory, who catches the panic, or which compiler contract is supposed to stay stable over time.

Examples

Example 1: call a C ABI symbol from Rust through a safe wrapper

The demo keeps the symbol self-contained by defining it in Rust with the C ABI, then calling it through an extern "C" declaration as if it came from native code. Even a trivial exported function needs a panic policy at the boundary: the body uses `wrapping_abs` so an `i32::MIN` input cannot panic across the C ABI seam.

What to look at: there are three layers, and only the middle one is unsafe. The foreign symbol on the left is reached through an extern "C" declaration; the one-line unsafe wrapper `safe_abs` performs the actual call and documents why it is sound; and main only ever sees a normal safe function. Trace that one call left to right in the diagram, then read the same three layers in code.



safe_abs is the only place the unsafe call lives; main and the rest of the program see a plain safe function.

Example 28-1. calling_c_from_rust.rs — C abi

```

mod c_shim {
    #[unsafe(no_mangle)]
    pub extern "C" fn ffi_demo_abs(input: i32) -> i32 {
        input.wrapping_abs()
    }
}

unsafe extern "C" {
    fn ffi_demo_abs(input: i32) -> i32;
}

fn safe_abs(input: i32) -> i32 {
    unsafe {
        // SAFETY:
        // - ffi_demo_abs uses the C ABI.
        // - the function takes a plain i32 and returns a plain i32.
        // - there is no cross-language ownership transfer in this call.
        ffi_demo_abs(input)
    }
}

fn main() {
    let left = -7_i32;
    let right = 11_i32;
}

```

```
println!("abs({}) = {}", left, safe_abs(left));
println!("abs({}) = {}", right, safe_abs(right));
}
```

OUTPUT

```
abs(-7) = 7
abs(11) = 11
```

The raw call stays in one small wrapper instead of leaking through the rest of the codebase.

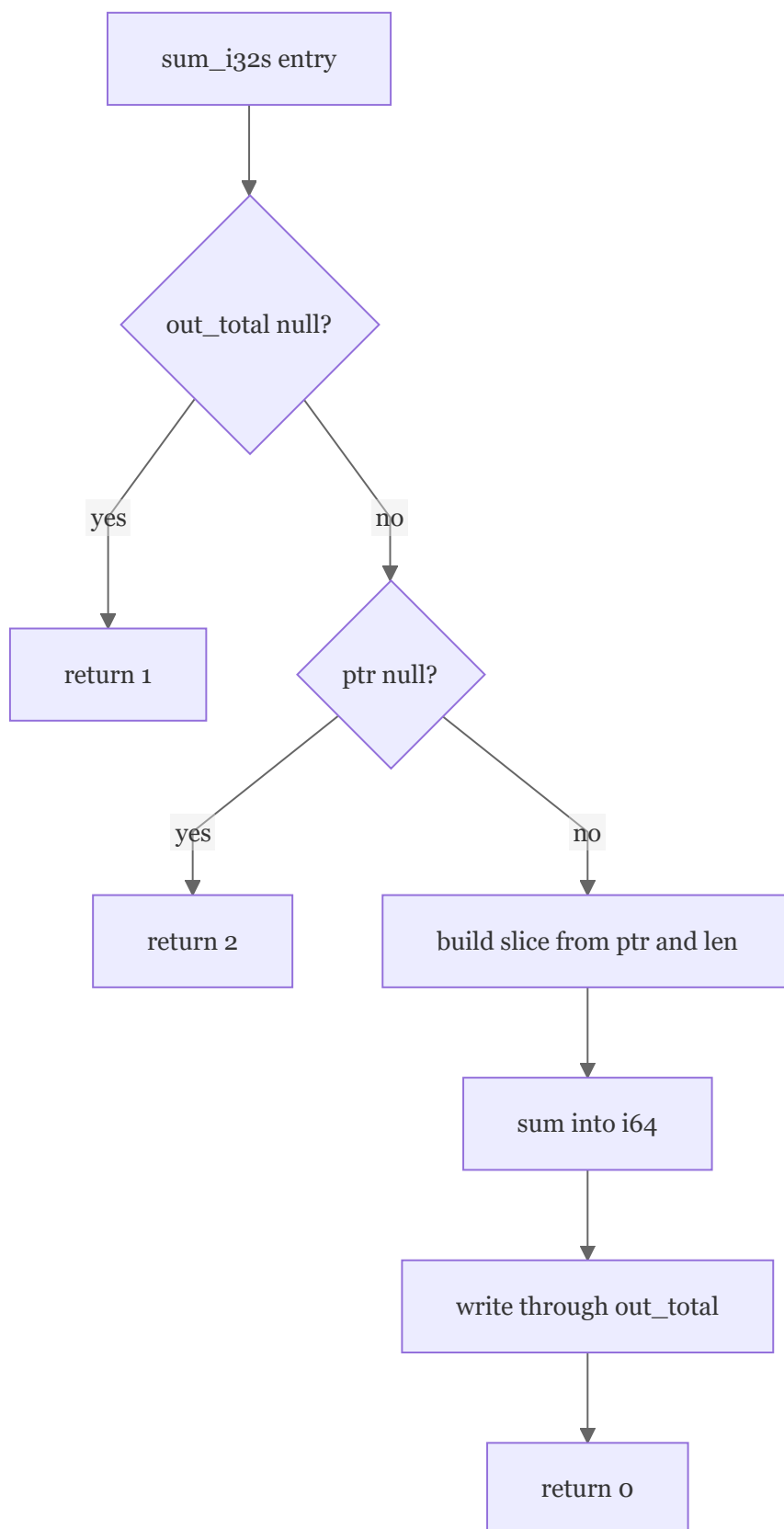
The seam is plain `i32 -> i32`, so no ownership transfer is required.

In production, the same wrapper pattern works when the symbol really comes from a C or C++ library.

Example 2: export a Rust function with a C ABI-safe wrapper

The exported entry point uses raw pointers, length, an out parameter, integer status codes, and explicit null checks. The richer Rust model can stay behind this seam.

What to look at: the function is a guard sequence before any real work happens. It validates the out pointer, then the input pointer, returning a distinct nonzero status for each failure; only once both are known non-null does it build a slice from ptr and len, compute the sum, write it through the out parameter, and return 0. The caller owns both buffers; Rust only borrows them for the call. Follow the branches in the diagram, then read them top to bottom in code.



Validate the out pointer, then the input pointer, each with its own status code, before any memory is read or written. Success returns 0.

Example 28-2. exporting_rust_c_abi.rs — C abi

```

#[unsafe(no_mangle)]
pub extern "C" fn sum_i32s(ptr: *const i32, len: usize, out_total: *mut i64) -> i32 {
    if out_total.is_null() {
        return 1;
    }

    if ptr.is_null() {
        return 2;
    }

    let slice = unsafe {
        // SAFETY:
        // - ptr is non-null and the caller promises it is valid for len i32 values.
        // - from_raw_parts requires a non-null, aligned pointer even when len == 0.
        std::slice::from_raw_parts(ptr, len)
    };

    let total = slice.iter().map(|&value| value as i64).sum::<i64>();

    unsafe {
        // SAFETY:
        // - out_total was checked for null above.
        // - the caller promises it points to writable i64 storage.
        *out_total = total;
    }

    0
}

fn main() {
    let values = [3_i32, 4, 5];
    let mut total = -1_i64;

    let status = sum_i32s(values.as_ptr(), values.len(), &mut total);

    println!("status = {}", status);
    println!("total = {}", total);
}

```

OUTPUT

```

status = 0
total = 12

```

The foreign edge speaks in raw addresses and sizes, not Rust references.

The caller owns the input array and the output storage. Rust only borrows both for the duration of the call.

Status integers make the contract portable without leaking Rust's Result shape.

The same pattern extends naturally to opaque handles, caller buffers, and explicit free functions.

The repository also includes standalone Rust source under `examples/ch28_cpp_integration/`. That directory includes a small opaque-handle example alongside the two editor labs so the ownership patterns can be reviewed outside the browser.

Summary

- FFI is a binary contract the compiler cannot check for you. ABI, layout, ownership, and unwind policy must all be stated explicitly.
- A narrow C ABI seam is usually the most stable integration layer even when the far side is C++, because C is the one calling convention every toolchain agrees on.
- Rust references, String, Vec<T>, Result, and panics normally stop at the Rust side of the wrapper and are rebuilt the moment control returns to safe Rust.
- bindgen, cxx, and autocxx reduce boilerplate but do not remove the obligation to wrap their output in a narrow, stable interop layer.
- Testing the interop layer means more than “it linked once.” Include null, length, ownership, sanitizer, and foreign-caller harness coverage.

JavaScript and C++ Integration for WASM

Rust to WebAssembly, wasm-bindgen, JS interop, boundary data choices, C++ shims, WASI, performance limits, serialization, and debugging

WebAssembly integration needs explicit host calls, memory ownership, data batching, and ABI rules. The module and its JavaScript host keep separate heaps, so every value richer than a number is encoded and copied across a boundary you design. This chapter defines the boundary contracts for browser-facing modules and the native shims they call.

Builds on Chapters 19 and 28

Chapter 19 covered wire data contracts and versioned encodings; the WebAssembly boundary is one more wire, with the same evolution and encoding concerns. Chapter 28 covered the C ABI and Rust FFI; the C and C++ shim in this chapter is that same seam, now compiled into a browser-shaped module instead of a native binary.

Opening scenario

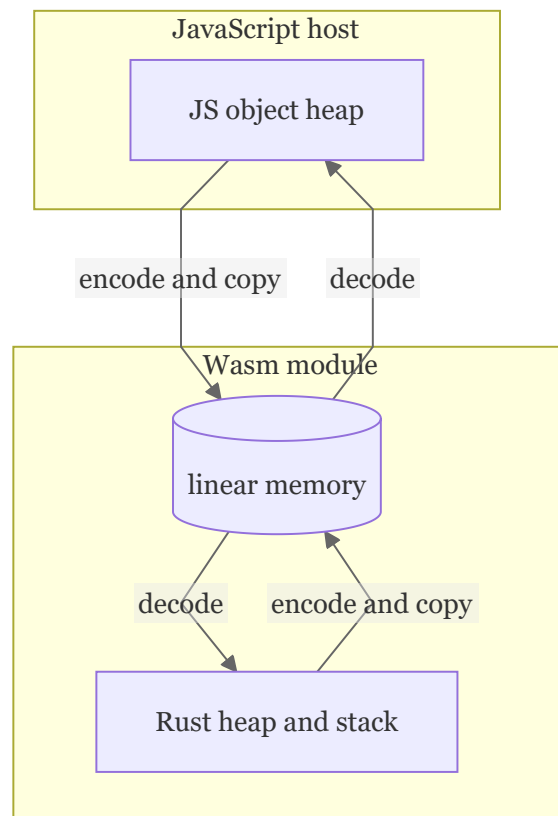
A web product is moving its hot client-side work, image processing, parsing, and a routing-score calculation, from JavaScript into a Rust WebAssembly module, and part of the routing logic already exists as a small C++ function the team wants to reuse rather than rewrite. The requirement is concrete: expose a clean JavaScript-callable surface, keep the per-call cost honest by choosing the right boundary representation, and wire the existing C++ code through a thin, auditable shim, all without leaking copy and lifetime concerns into the rest of the application.

A practical decision order

1. Decide what actually has to cross the boundary, and how often.
2. Pick the representation per value: a number, a copied slice, a serialized blob, or a resident handle.
3. Batch hot loops into one call that runs inside the module.
4. Confine any C or C++ seam to a thin unsafe shim with stated invariants.

Mental model

The shape worth memorizing is two separate heaps with one shared byte array between them. JavaScript holds its own objects; the module holds its own stack and heap inside a single linear memory. The only thing both sides can touch is that linear memory, so every string, array, or struct that crosses is written into it as bytes by one side and read out by the other. Trace one string across the diagram below: it leaves the JavaScript heap, lands as bytes in linear memory, and is reconstructed as a Rust value, with a copy at the crossing.



Two heaps, one shared byte array. Every non-numeric value is copied into linear memory at the crossing and reconstructed on the far side.

Core concepts

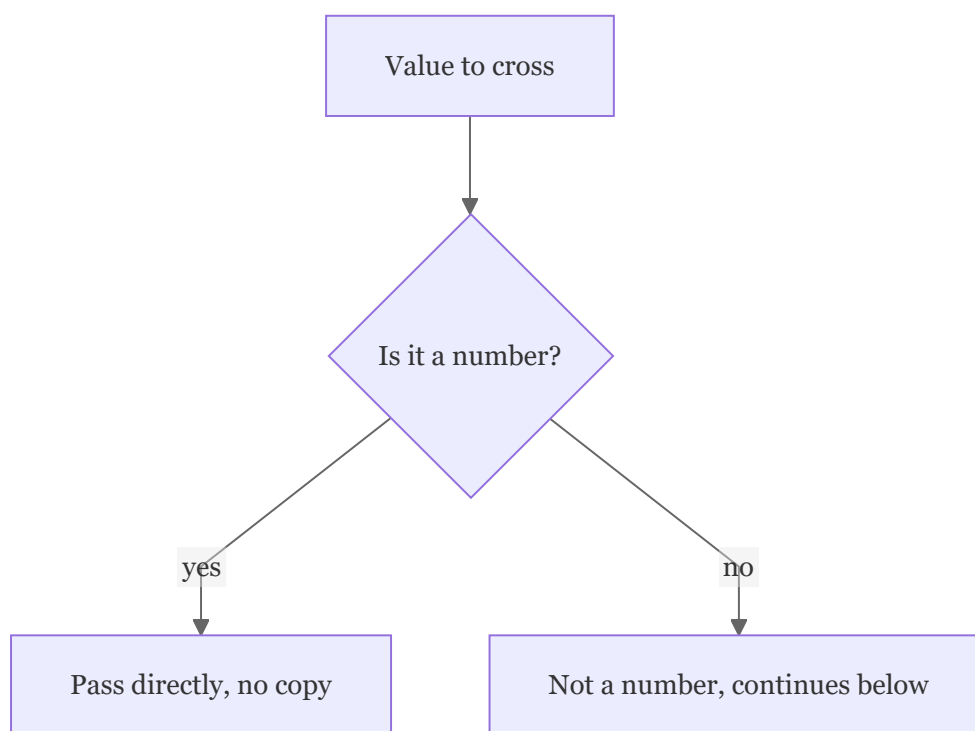
From Rust to WebAssembly

A browser-facing crate is built as a `cdylib` so the linker produces a single `.wasm` artifact, then `wasm-bindgen` post-processes it to emit the JavaScript glue that knows how to lay out and read back each annotated value.

JavaScript interop and boundary data choices

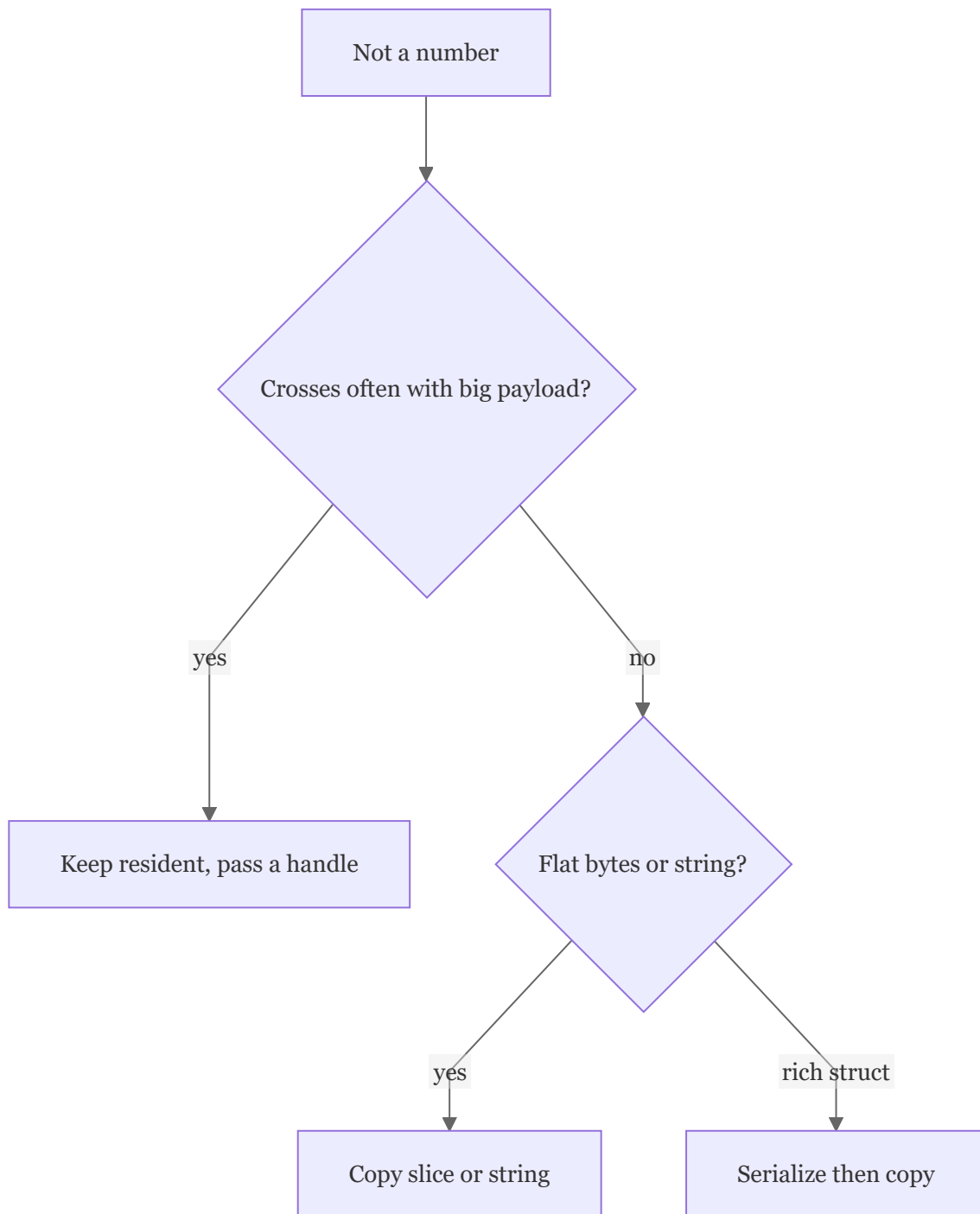
The interop surface is where the design lives. `wasm-bindgen` will happily generate glue for a string, an array, or a serialized struct, but the four representations below have very different costs. Choosing among them deliberately, rather than reaching for the most convenient type, is what keeps a boundary fast and auditable.

The decision is essentially a cost ladder. A number rides the native interface for free. A string or byte array costs one copy per crossing. A struct costs an encode plus a copy plus a decode. A resident handle costs almost nothing per call but adds a lifetime you must manage. Reading these as a ladder makes the right representation obvious for any given call frequency and payload size.



Step one of the ladder: a number rides the native interface for free. Anything else falls through to the next question.

For everything that is not a number, the remaining questions decide how it crosses:



Steps two and three: a frequently crossed big payload stays resident behind a handle, flat bytes cost one copy, and a rich struct costs an encode plus a copy.

Calling C and C++ from a WebAssembly module

Reusing existing C or C++ code inside the module means crossing a second, lower boundary: the C ABI. This is the same FFI seam covered in Chapter 28, and it behaves the same way under WebAssembly. The shim exports plain functions, Rust calls them through an unsafe extern block, and the safety of the whole thing rests on invariants you state rather than the compiler checks.

The two boundaries stack but stay distinct. The outer edge, Rust to JavaScript, is managed by generated glue. The inner edge, Rust to the C shim, is raw pointers and lengths with manual invariants. Keep them conceptually separate so a fault at one seam is not misdiagnosed at the other.

WASI and host capabilities

A module has no ambient operating system. In the browser it can compute freely but can only reach the outside world through functions the host imports into it. WASI is the standardized way a non-browser host grants files, clocks, and randomness, but the underlying rule is the same in both worlds: capabilities are imports, not assumptions.

Performance limits and serialization at the boundary

WebAssembly computes at near-native speed inside the module, so the performance conversation is almost never about the arithmetic. It is about the boundary. A design that crosses the seam per element, or copies a large buffer on every call, will be slower than the JavaScript it replaced no matter how fast the inner loop is. The fixes are structural: batch bulk work into a single call, and keep large state resident so it crosses once instead of repeatedly.

Serialization is the other half of this. A rich struct that crosses as JSON pays an encode and a parse on top of the copy; the same struct crossing as a compact binary layout the host reads as a typed array can be dramatically cheaper. As in Chapter 19, the encoding is a contract: pick one that both sides can read, evolve it additively, and measure it rather than guessing.

Debugging WebAssembly

A fault inside a module is harder to read than a native crash because, by default, you see a trap and an offset into linear memory rather than a Rust line. The good news is that a small amount of setup recovers most of the familiar debugging experience: a panic hook surfaces messages to the console, source maps map traps back to source, and logging at the seam exposes the pointer and length values where the hard bugs actually live.

How this lands by background

Most engineers reach WebAssembly with a marshalling model from somewhere else, and the useful question is which part of that model still holds. The shift is rarely about API names. It is about accepting that the host and the module keep separate heaps, that every non-numeric value is copied across the seam, and that you now name the representation and ownership of each value instead of trusting a bridge to do it invisibly.

Production patterns

Pitfalls and tradeoffs

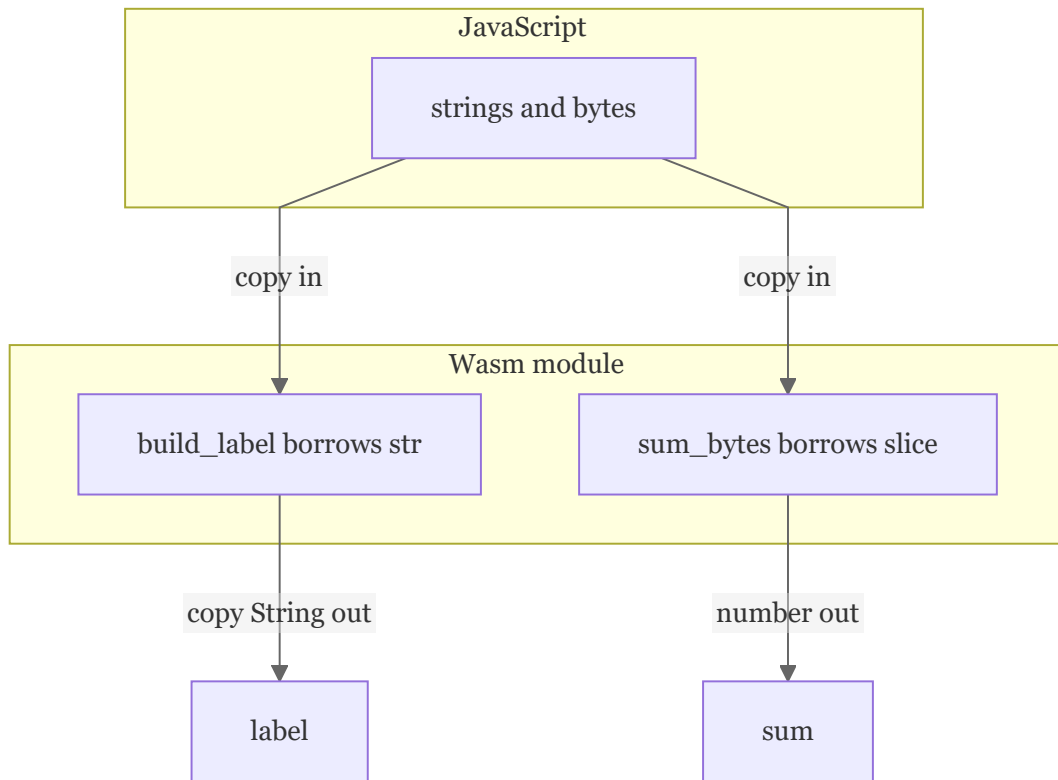
WebAssembly does not make the boundary disappear; it makes it explicit. The cost you used to pay invisibly in a marshaller is now a copy you can see, measure, and design around.

Examples

Example 1: wasm-bindgen string and byte boundary

Two exported functions show the two common non-numeric crossings: a string in and out, and a byte slice in with a number out.

What to look at: the `#[wasm_bindgen]` attribute is what makes each function callable from JavaScript, and the parameter types decide how the data crosses. `build_label` takes two borrowed `&str` values, each copied into linear memory, and returns an owned `String` copied back out; `sum_bytes` receives a `&[u8]` slice and returns a plain number, so only the bytes in are copied and nothing comes back but an integer. Follow each value across the diagram, then read the same crossings in code.



Strings and the byte slice are copied into the module; the label String is copied back out, while the sum returns as a plain number with no copy.

Example 29-1. `wasm_bindgen_boundary.rs` — Array boundary

```

use wasm_bindgen::prelude::*;

#[wasm_bindgen]
pub fn build_label(service: &str, route: &str) -> String {
    format!("{}", service, route)
}

#[wasm_bindgen]
pub fn sum_bytes(bytes: &[u8]) -> u32 {
    bytes.iter().map(|&value| value as u32).sum()
}

fn main() {
    let label = build_label("billing", "/ready");
    let total = sum_bytes(&[1_u8, 2, 3, 4]);

    println!("label = {}", label);
    println!("sum = {}", total);
}
  
```


OUTPUT

```
label = billing::ready
sum = 10
```

A `&str` or `&[u8]` is copied into linear memory for the call and not owned by the module afterwards.

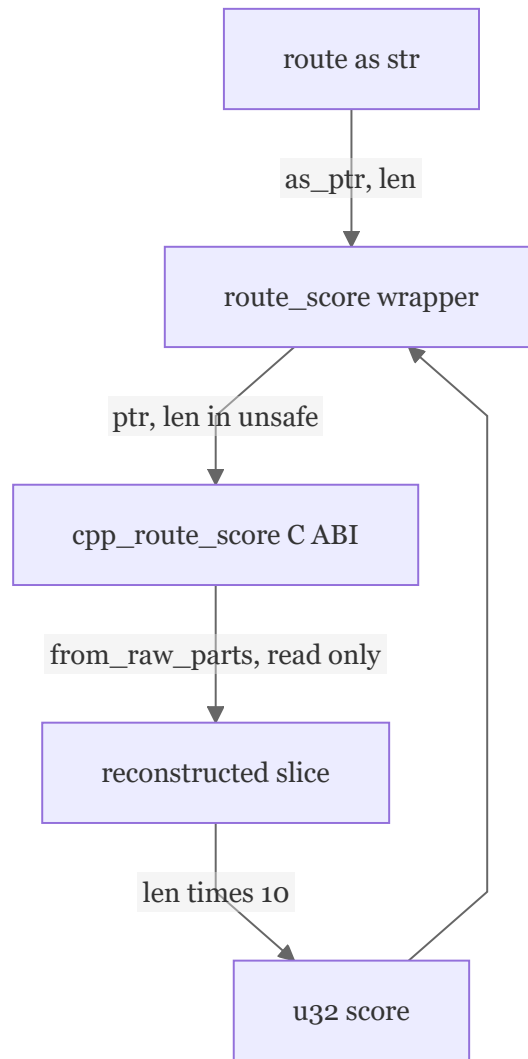
The returned `String` is copied back into the host, which then owns its own separate value.

The summed `u32` rides the native interface with no allocation and no copy.

Example 2: Wasm plus C ABI boundary

A safe Rust function hands a string's bytes to a C-ABI shim as a pointer and a length, mirroring how a reused C or C++ routine is called inside the module.

What to look at: the safe wrapper `route_score` is the only place that touches raw pointers, and it does so inside a narrow `unsafe` block with a written `SAFETY` comment naming each invariant. It passes `route.as_ptr()` and `route.len()` to the shim, which reconstructs a slice for the duration of the call and computes a score from its length. The contract is pointer plus length plus a promise the bytes stay valid and are only read while the call runs. Trace that single crossing in the diagram, then read it in code.



One crossing: the wrapper passes pointer and length into the C-ABI shim, which rebuilds a read-only slice for the call and returns a number. Validity of the bytes is a stated invariant, not a compiler check.

Example 29-2. `wasm_cpp_ffi_boundary.rs` — *Ffi boundary*

```

mod cpp_shim {
    #[unsafe(no_mangle)]
    pub extern "C" fn cpp_route_score(ptr: *const u8, len: usize) -> u32 {
        if ptr.is_null() && len != 0 {
            return 0;
        }

        let bytes = unsafe {
            // SAFETY:
            // - ptr is either null with len == 0 or valid for len bytes.
            // - the shim only reads the bytes for the duration of the call.
            std::slice::from_raw_parts(ptr, len)
        };

        (bytes.len() as u32) * 10
    }
}

```

```
unsafe extern "C" {
    fn cpp_route_score(ptr: *const u8, len: usize) -> u32;
}

pub fn route_score(route: &str) -> u32 {
    unsafe {
        // SAFETY:
        // - route.as_ptr() is valid for route.len() bytes for the duration of the call.
        // - the shim borrows the bytes only for the duration of the call.
        cpp_route_score(route.as_ptr(), route.len())
    }
}

fn main() {
    let route = "/orders";
    println!("route = {}", route);
    println!("score = {}", route_score(route));
}
```

OUTPUT

```
route = /orders
score = 70
```

The C ABI has no slice type, so the borrowed bytes cross as a raw pointer and an explicit length.

The SAFETY comment names what must hold: valid pointer, correct length, read-only for the call's duration.

Only the wrapper and the shim are unsafe; the rest of the program stays in safe Rust.

Both examples run here as ordinary Rust so the boundary logic is testable without a browser or a WebAssembly runtime. In a real build, the first compiles to a cdylib processed by wasm-bindgen, and the second links the C or C++ shim into the same module, but the crossing contracts, copy in, copy out, and pointer plus length, are exactly what you see above.

Next chapter

This chapter handed values across a boundary inside one machine. Chapter 30 widens that boundary to a message broker between services, where the wire contract, copy semantics, and explicit ownership habits from this chapter reappear as durable messages, acknowledgments, and idempotent consumers.

Summary

CHAPTER 30 · AMQP AND MESSAGE BROKERS

AMQP and Message Brokers

Broker fundamentals, AMQP exchanges and queues, RabbitMQ with Rust, acknowledgments, retries, dead-letter queues, idempotency, backpressure, serialization, and observability

A message broker lets services hand work to each other without being online at the same time. That convenience comes with a bill: routing, redelivery, idempotent consumers, dead-letter policy, and backpressure all become your responsibility. This chapter treats AMQP not as a library to call but as an operational contract to design.

Builds on Chapters 19, 22, 23, 25, and 26

Chapter 19 covered wire DTOs and versioned envelopes. Chapters 22 and 23 covered ownership across threads, message passing, and synchronization. Chapters 25 and 26 covered Tokio, bounded queues, retries, and task orchestration. AMQP systems sit across all of that.

Opening scenario

An order pipeline is being split into checkout, billing, search indexing, and notification services. The business requirement is reliable asynchronous handoff: route messages by contract, preserve idempotency, ack only after durable effects, isolate poison messages, and expose queue depth and retry pressure.

A practical decision order

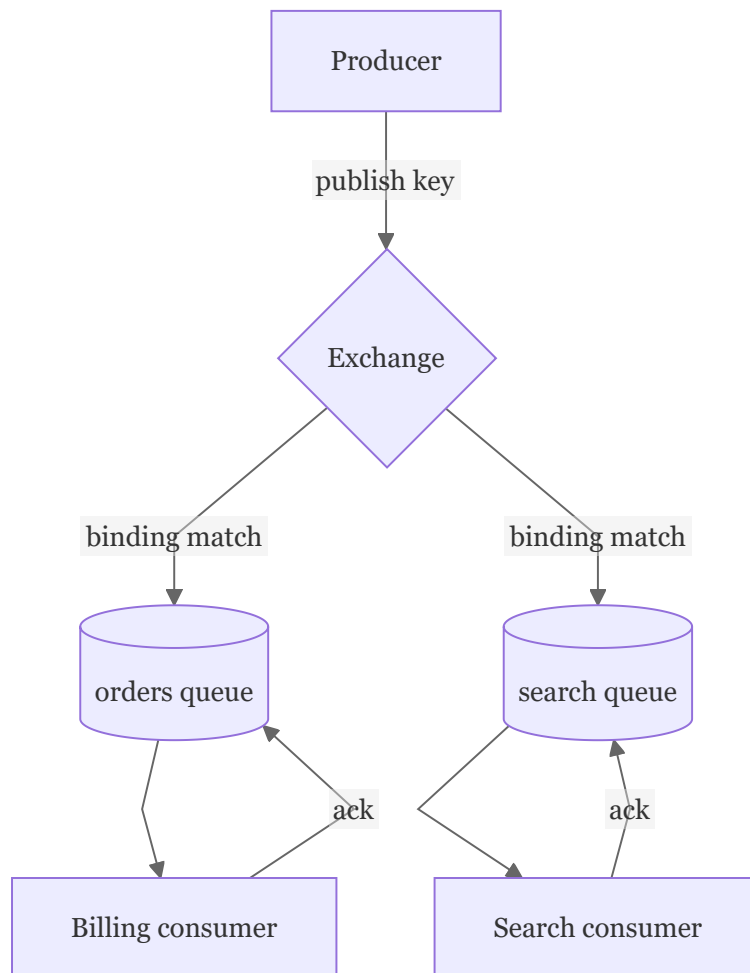
1. Pick the broker boundary only when producer and consumer should truly decouple in time or failure.
2. Design topology second: exchange, routing key, queue boundaries, dead-letter flow.
3. Ack only after the durable effect or durable checkpoint.
4. Cap consumer admission both in the broker and in the local worker pool.

Mental model

Core concepts

Message broker fundamentals

The shape worth memorizing is a one-way pipeline with the broker sitting in the middle. A producer never names a queue; it publishes to an exchange with a routing key. The exchange consults its bindings, copies the message into every matching queue, and each queue feeds its own consumers at their own pace. Everything between the publish call and the ack is the broker's responsibility, which is exactly why it can buffer a burst or survive a consumer restart.



The producer talks only to the exchange; bindings fan one message out to the queues that subscribed, and each consumer acks its own queue.

AMQP concepts: exchanges, queues, bindings, and routing keys

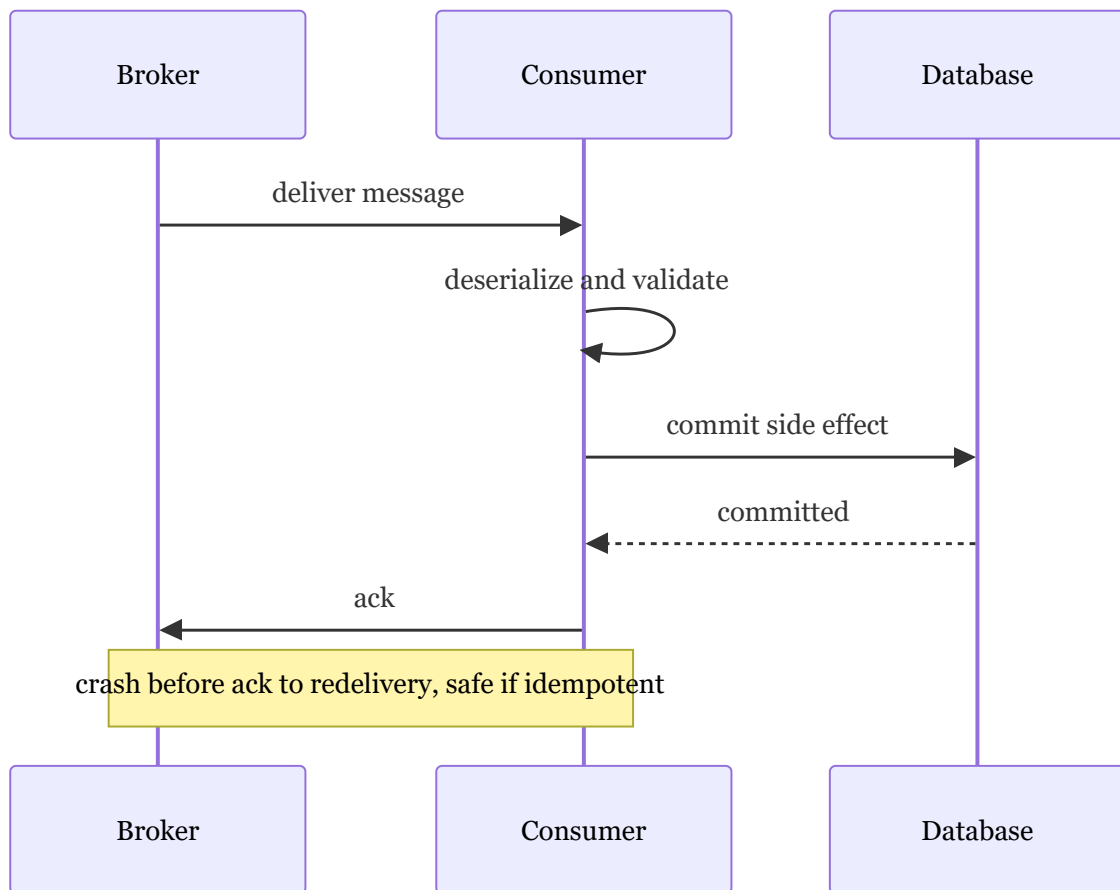
The critical operational point is this: the producer chooses a routing key, but the exchange owns routing policy. That lets teams change routing topology without turning every publisher into a queue-aware special case.

RabbitMQ with Rust

In a real Cargo service, the transport adapter typically holds the AMQP client connection and channel setup, then hands owned envelopes or commands into ordinary Rust handlers. Keep the transport edge narrow so the retry, idempotency, and serialization logic remains testable without a live broker.

Producers and consumers

The single most important sequence in this chapter is the order in which a consumer commits its side effect and sends its ack. The broker holds the message as unacknowledged until the consumer says it is done. If the consumer commits the durable effect first and then acks, a crash in between costs you only a redelivery, which an idempotent handler absorbs. If it acks first, the broker forgets the message and a crash loses the work for good.



Commit the durable effect, then ack. A crash before the ack only causes a safe redelivery; acking first would lose the work.

Acknowledgments and retries

An ack is a promise that the work is safely done and the broker may forget the message. A retry is a decision that the work is not done yet and deserves another attempt. Both are choices the consumer makes explicitly, and getting them right is most of what separates a robust consumer from one that quietly drops or endlessly reprocesses work.

Dead-letter queues

Not every failure should be retried. A malformed payload or an unsupported command will fail the same way on every attempt, and requeuing it forever just blocks the queue behind a message that can never succeed. A dead-letter queue is the escape hatch: the broker moves a message there after it has exhausted its retries or been explicitly rejected, so the main queue keeps flowing and a human can inspect the casualties later.

Idempotency

Because delivery is at-least-once, the only way to make duplicates harmless is to make the handler idempotent: applying the same message twice must leave the system in the same state as applying it once. Sometimes that is free, when the operation is naturally a set-it-to-this upsert. More often you need a durable record of which message IDs have already been applied, committed in the same step as the effect, so a second delivery sees the record and does nothing.

A duplicate delivery is not a surprising edge case. In broker-based systems, it is the design center.

Backpressure

A queue is a shock absorber, but a shock absorber with no limit just hides the fact that you are overloaded. Backpressure is how the system says no gracefully instead of falling over. It has two layers that are easy to confuse: the broker side, which caps how many unacked messages a consumer may hold at once, and the local side, which caps how much in-process work those messages can spawn. Set one without the other and you have only moved the overload from the broker into your own heap.

Serialization for messages

A message often outlives the code that produced it: it can sit in a queue across a deploy, be replayed weeks later, or be consumed by a service on an older version. That is why broker payloads earn their keep when they travel inside an explicit envelope rather than as a bare struct dump. The envelope carries a schema version so consumers can branch on format, a message ID so they can deduplicate, and optional trace metadata for debugging, while the tagged payload keeps the wire shape stable. Evolve it additively with optional fields rather than redefining what existing fields mean, and old and new consumers can keep reading the same stream.

Observability in broker-based systems

A broker is a place work can pile up invisibly, so you cannot run one on faith. The signals below are the minimum set that tells you whether the system is healthy or merely quiet because nothing is being processed. A growing queue with rising message age, a climbing retry rate, or a dead-letter queue that keeps filling are early warnings; treat them as first-class production metrics rather than transport trivia you only look at during an incident.

How this lands by background

Most engineers do not arrive at message brokers as a blank slate; they arrive with a mental model from another ecosystem. The useful thing to know is which part of that model transfers and which part will quietly mislead you. The shift is almost never about API names. It is about where reliability and consistency work has to happen now that a durable, distributed boundary sits between your services.

Production patterns

Pitfalls and tradeoffs

A broker can decouple services, but it will not decouple you from consistency. It merely changes where the consistency work must be done.

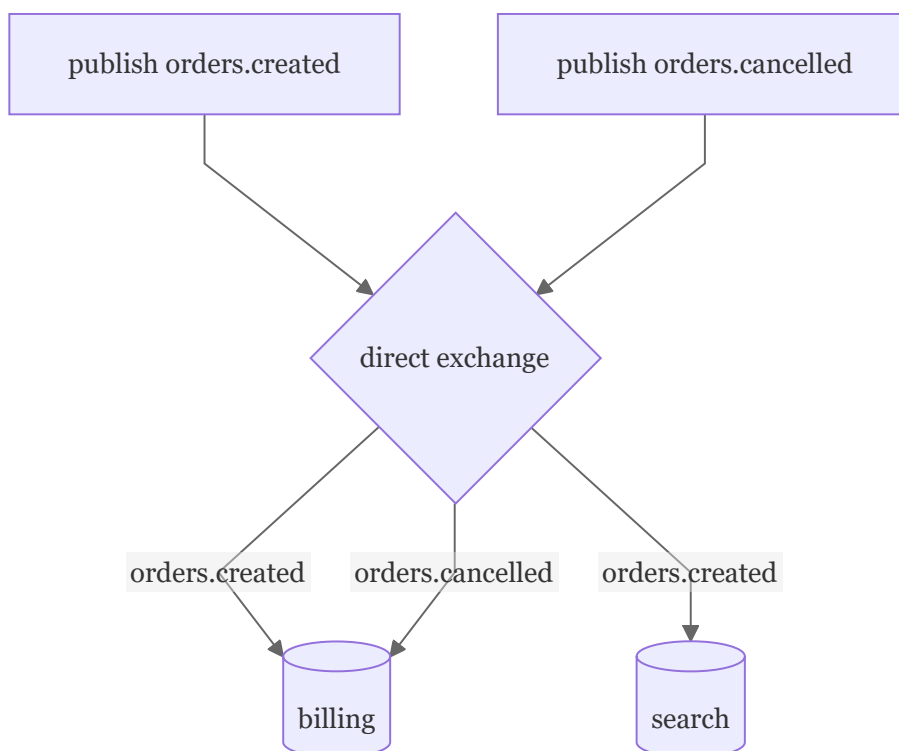
Examples

Example 1: direct exchange routing

The example keeps the routing model tiny: producers publish one routing key, the exchange owns binding policy, and queues receive only what their bindings match.

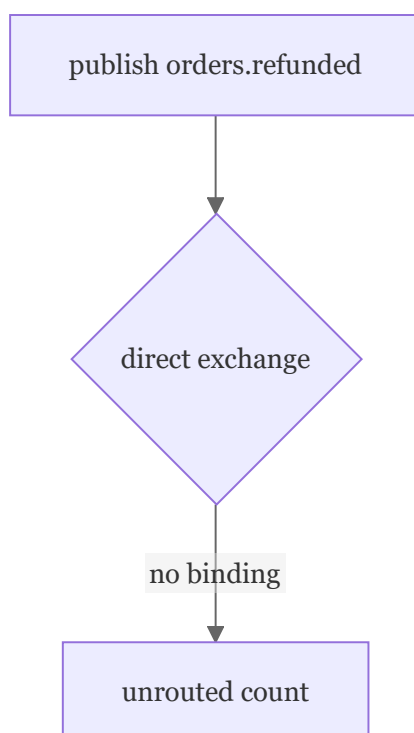
What to look at: the exchange holds a map from routing key to the set of bound queues, and publish is just a lookup that copies the message into each match. The key `orders.created` is bound to two queues, so one

publish produces two deliveries; an unbound key produces zero and is counted as unrouted instead of silently vanishing. Trace one message through the diagram below, then read the same path in code.



Bound keys: created fans out to billing and search, cancelled goes to billing only.

An unbound key takes the other exit instead of silently vanishing:



The refunded key matches no binding, so the exchange charges it to the unrouted counter.

Example 30-1. `direct_exchange_routing.rs` — *Exchange routing*

```

use std::collections::HashMap;

#[derive(Debug, Default)]
struct DirectExchange {
    bindings: HashMap<String, Vec<String>>,
}

impl DirectExchange {
    fn bind(&mut self, routing_key: &str, queue: &str) {
        self.bindings
            .entry(routing_key.to_string())
            .or_default()
            .push(queue.to_string());
    }

    fn route(&self, routing_key: &str) -> Vec<String> {
        self.bindings
            .get(routing_key)
            .cloned()
            .unwrap_or_default()
    }
}

fn main() {
    let mut exchange = DirectExchange::default();

    exchange.bind("orders.created", "billing");
    exchange.bind("orders.created", "search");
    exchange.bind("orders.cancelled", "billing");

    let created = exchange.route("orders.created");
    let cancelled = exchange.route("orders.cancelled");

    println!("created = {}", created.join(", "));
    println!("cancelled = {}", cancelled.join(", "));
    println!("unrouted = {}", exchange.route("orders.refunded").len());
}

```

OUTPUT

```

created = billing,search
cancelled = billing
unrouted = 0

```

The exchange owns routing policy and binding state.

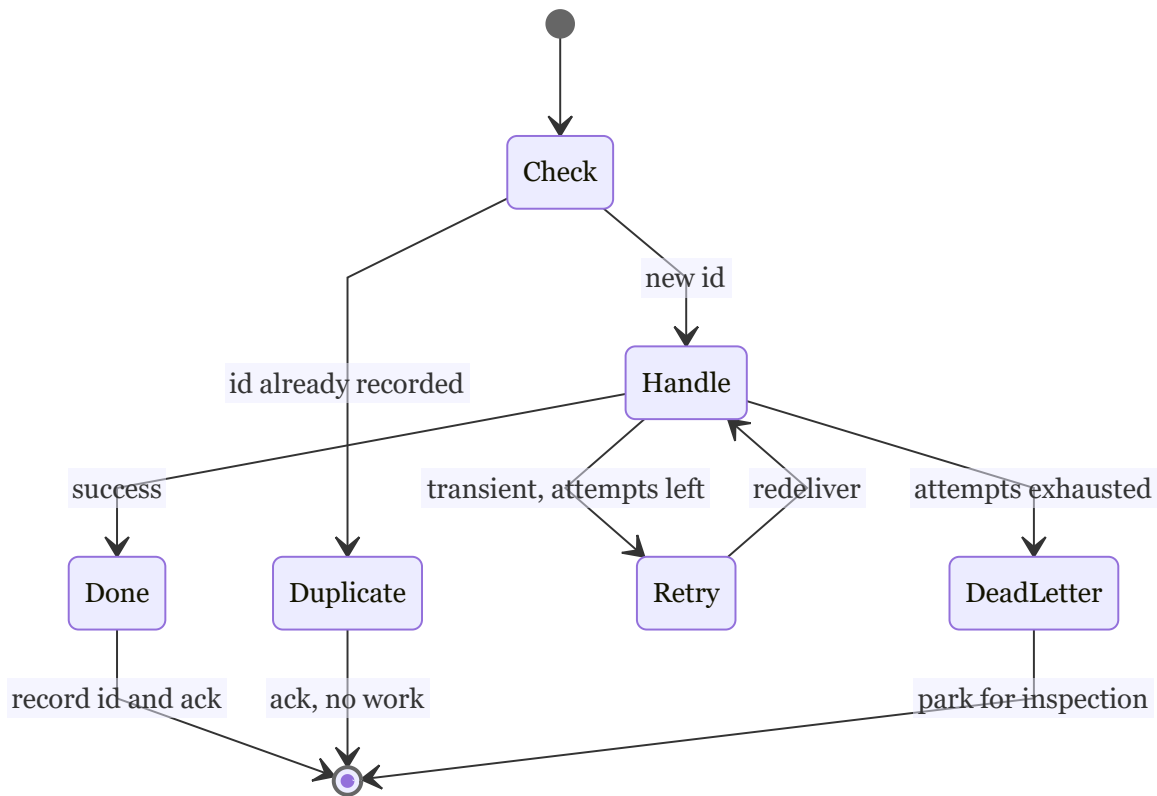
Producers publish one semantic key such as `orders.created`.

Queue selection stays declarative instead of being hard-coded into every publisher.

Example 2: idempotent consumer with retries and a DLQ

The handler only acks successful unique work. Duplicates are counted and ignored, transient failure retries are capped, and terminal failures move to a DLQ.

What to look at: every delivery flows through one small decision machine. First the handler checks whether the message ID is already in the seen set; if so it is a duplicate and is acked without doing the work again. Otherwise it runs the handler, and the outcome routes it: success records the ID and acks, a transient failure under the attempt cap goes back for a counted retry, and a failure that exhausts the cap is sent to the dead-letter queue rather than requeued forever. The diagram is that state machine; the four output counters are just tallies of which path each message took.



One delivery, four exits: duplicate, done, retry, or dead-letter. The counters in the output map directly to these edges.

Example 30-2. `idempotent_consumer_retry_dlq.rs` — *Idempotent consumer*

```

use std::collections::{HashSet, VecDeque};

#[derive(Debug, Clone)]
struct Delivery {
    message_id: &'static str,
    order_id: &'static str,
    attempt: u8,
}

#[derive(Default)]
struct Consumer {
    processed_ids: HashSet<String>,
    retry_queue: VecDeque<Delivery>,
    dlq: Vec<Delivery>,
    processed: usize,
    duplicates: usize,
    retried: usize,
}

```

```

impl Consumer {
    fn on_delivery(&mut self, delivery: Delivery) {
        if self.processed_ids.contains(delivery.message_id) {
            self.duplicates += 1;
            return;
        }

        if delivery.order_id == "ord-fail" {
            if delivery.attempt < 3 {
                self.retried += 1;
                self.retry_queue.push_back(Delivery {
                    attempt: delivery.attempt + 1,
                    ..delivery
                });
            } else {
                self.dlq.push(delivery);
            }
            return;
        }

        self.processed += 1;
        self.processed_ids.insert(delivery.message_id.to_string());
    }
}

fn main() {
    let mut consumer = Consumer::default();

    consumer.on_delivery(Delivery {
        message_id: "msg-1",
        order_id: "ord-1",
        attempt: 1,
    });
    consumer.on_delivery(Delivery {
        message_id: "msg-1",
        order_id: "ord-1",
        attempt: 1,
    });
    consumer.on_delivery(Delivery {
        message_id: "msg-2",
        order_id: "ord-fail",
        attempt: 1,
    });

    while let Some(delivery) = consumer.retry_queue.pop_front() {
        consumer.on_delivery(delivery);
    }

    consumer.on_delivery(Delivery {
        message_id: "msg-3",
        order_id: "ord-3",
        attempt: 1,
    });

    println!("processed = {}", consumer.processed);
    println!("duplicates = {}", consumer.duplicates);
}

```

```
println!("retried = {}", consumer.retried);
println!("dlq = {}", consumer.dlq.len());
}
```

OUTPUT

```
processed = 2
duplicates = 1
retried = 2
dlq = 1
```

A completed message ID is recorded so replay becomes harmless.

Retry remains explicit, counted, and bounded by attempt number.

After the last allowed attempt, the delivery stops cycling and moves to a terminal bucket.

Processed, duplicate, retry, and DLQ counts become ordinary metrics, not incident archaeology.

The repository also includes standalone Rust source under `examples/ch30_amqp_and_message_brokers/` including a small versioned message-envelope example alongside the two runnable worked examples.

Next chapter

A broker hands single messages between services. Chapter 31 scales that idea up to distributed task execution: dispatching units of work across many workers, tracking their state, and recovering when a worker dies mid-task. The acknowledgment, retry, and idempotency habits from this chapter become the foundation for that work.

Summary

CHAPTER 31 · DISTRIBUTED TASK EXECUTION

Distributed Task Execution

Task queues, work distribution models, delivery semantics, leasing, retries, failure isolation, result aggregation, task graphs, tracing, and profiling for distributed workloads

Once work leaves your process for another machine, the borrow checker can no longer help. What protects you instead is durable handoff, leases, retries, idempotency, and traceable recovery. This chapter shows how those requirements show up as concrete shapes in Rust worker and queue boundaries.

Builds on Chapters 22, 25, 26, and 30

Chapter 22 covered thread ownership and channels inside one process. Chapter 25 covered Tokio task orchestration and graceful shutdown. Chapter 26 separated waiting-heavy runtimes from CPU pools and bounded queues. Chapter 30 covered AMQP brokers and at-least-once delivery. This chapter generalizes those ideas to distributed task systems as a whole.

Opening scenario

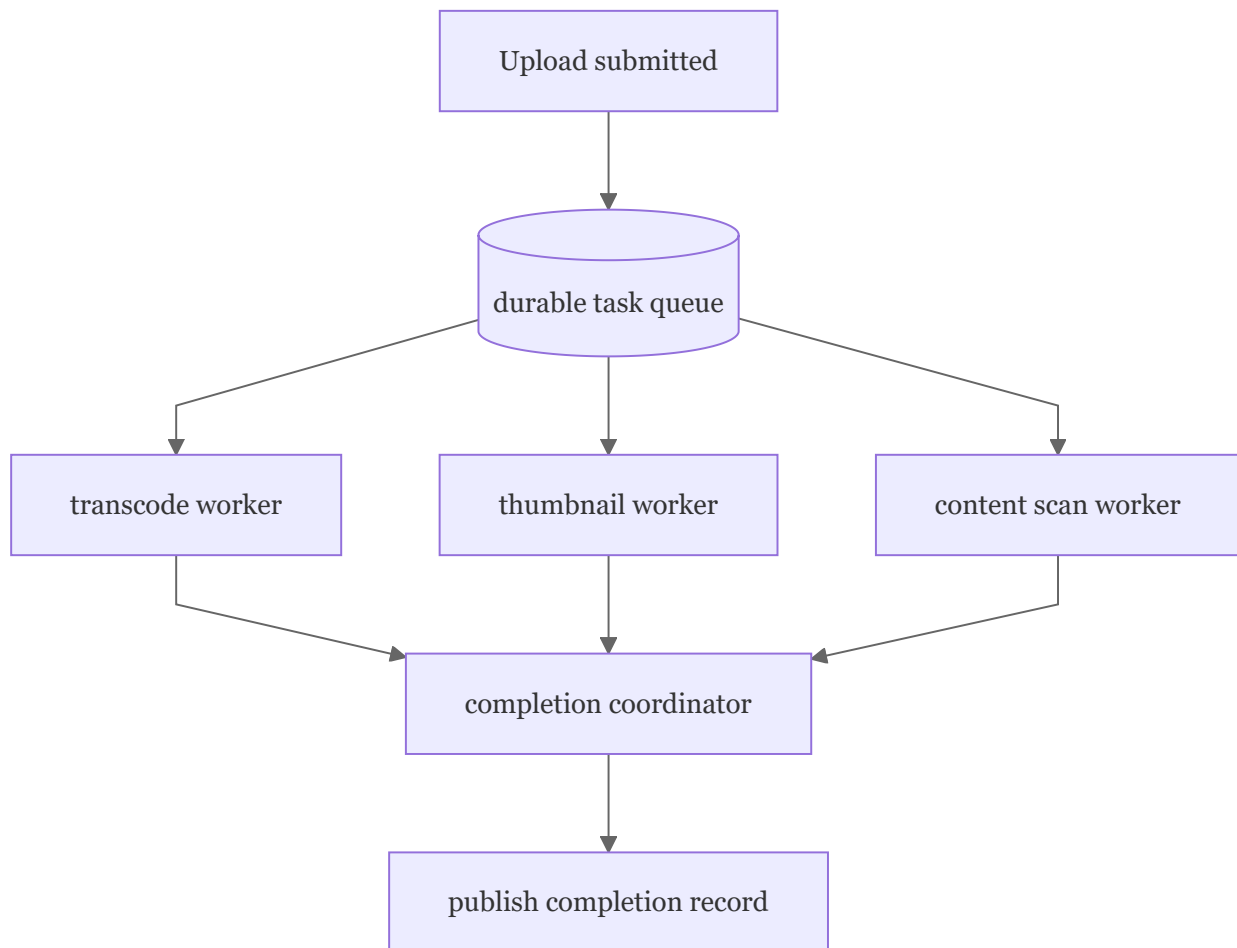
A media platform ingests an upload, transcodes several quality variants, generates thumbnails, scans the content for policy violations, and finally publishes one completion record that the rest of the product reads. None of those steps run in the request that triggered them. They run on a fleet of workers that can be deployed, scaled, and killed at any moment. The business requirement is durable distributed work: leases so a dead worker's task is recovered, retry budgets so a flaky dependency does not turn into an infinite loop, idempotent completion so a redelivered task does not double-publish, graph-aware aggregation so the final record waits for every variant, and tracing so an operator can reconstruct what happened after the fact.

The trap for an engineer arriving from a single-process background is to reach for the queue brand first. The queue is the easy part. The hard part is the contract you wrap around it: what an in-flight task means, what a duplicate means, and what recovery looks like when a machine disappears mid-job. The decision order below puts that contract before the throughput tuning.

A practical decision order

1. Define the owned task envelope before choosing a queue backend.
2. Choose delivery semantics assumptions next, usually at-least-once.
3. Pick lease, retry, and failure-isolation policy before tuning throughput.
4. Make tracing and profiling part of the scheduler contract, not an afterthought.

The shape of the whole scenario is a fan-out followed by a fan-in. One submission turns into several independent worker tasks, each pulling from the same durable queue, and a single coordinator waits for all of them before it writes the completion record. Hold this picture in mind for the rest of the chapter: the queue is the shared boundary, the workers are interchangeable and disposable, and the coordinator is the one place that knows the job is truly done.



One upload fans out to disposable workers through a shared queue, then fans back in to a single coordinator that publishes the final record.

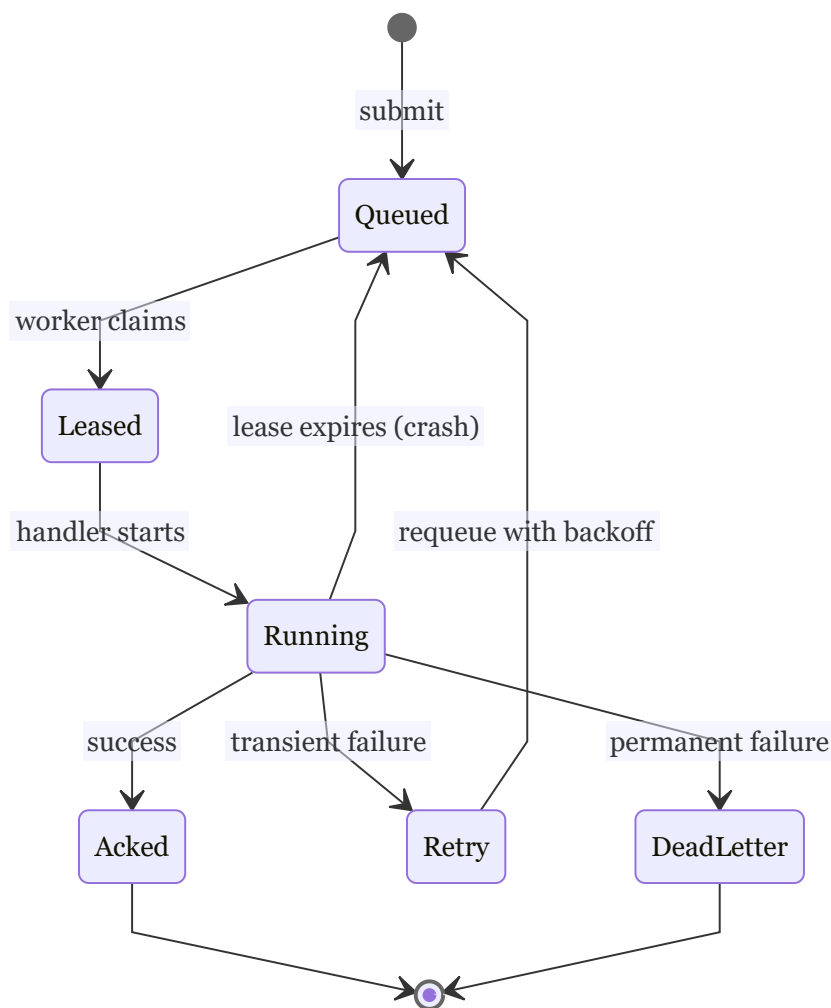
Mental model

Core concepts

Task queues and work distribution models

A task queue exists to separate the moment a request is made from the moment the work is done. The submitter writes a durable record and returns immediately; a worker picks the record up later, when it has capacity. That decoupling is what buys you elasticity and failure isolation, but it also means three things you used to ignore become part of your service budget: how deep the queue is, how old the oldest task is, and how a task that fails halfway gets replayed. The cards below contrast the common ways work is handed from queue to worker; the diagram after them is the lifecycle every one of those models has to implement.

Every task moves through the same small state machine no matter which distribution model you pick. The two branches worth staring at are the loop back to Queued on retry and the one-way exit to DeadLetter. A system without that terminal exit will requeue a permanently broken task forever, which is the most common way a queue quietly turns into an infinite loop.



The task lifecycle. Retry loops back to Queued; a lease expiry also returns the task to the pool; only DeadLetter and Acked are terminal.

Exactly-once vs at-least-once processing

Delivery semantics describe what the system promises about how many times your handler runs for one logical task. There are three honest answers, and only one of them is the production default. At-least-once is what durable queues actually give you, because the safe move after a crash is to redeliver. At-most-once trades that safety for the risk of silently dropping work. True exactly-once across the whole path is not a setting you enable; it is something you assemble from at-least-once delivery plus deduplication you own.

When another engineer says “exactly once,” ask whether they mean transport delivery, handler execution, or durable side effects. Those are different claims.

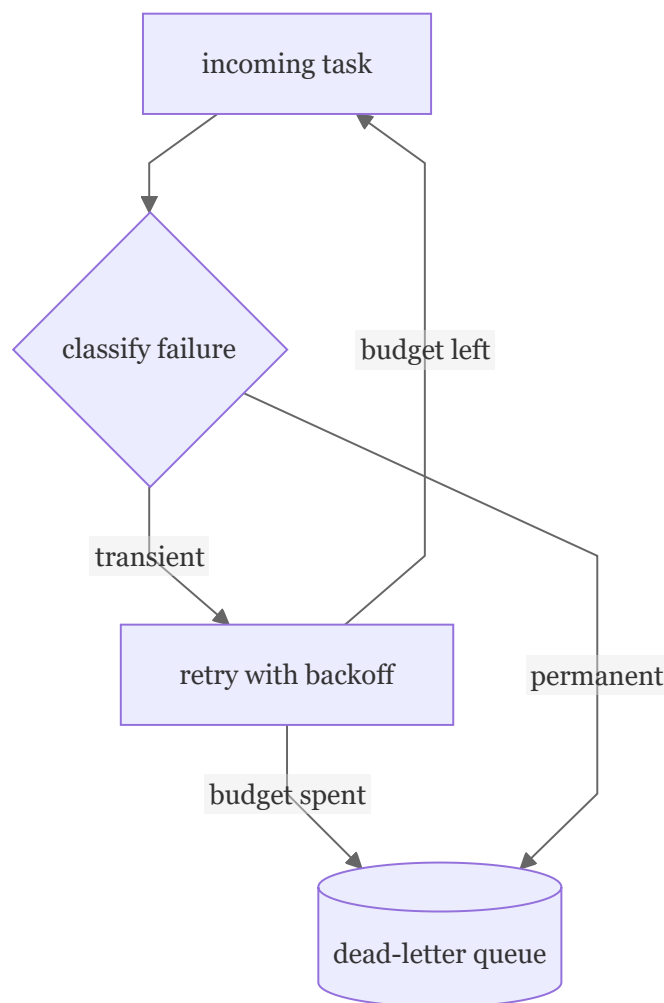
Leasing and visibility timeouts

A lease is how the queue answers the question every distributed system eventually faces: what happens to a task whose worker stopped responding. When a worker claims a task, the queue hands it a lease with an expiry and hides the task from everyone else for that window, which is the visibility timeout. As long as the worker is healthy it finishes the work and acknowledges the task before the lease runs out. If it crashes, the lease simply expires and the task becomes claimable again by a peer. Nothing detects the crash directly; the absence of an acknowledgement is the signal.

Lease design is really failure-recovery design. A distributed worker should be assumed killable at any point. The queue recovers by lease expiry. The application recovers by idempotent completion. Those are two separate guarantees, and you need both: lease expiry alone gets the task run again, but only idempotent completion keeps the second run from doing damage.

Distributed retries and failure isolation

Retry is the most over-applied tool in this whole space, because the transport usually makes blind requeue a one-line call. The discipline is to classify failures before you retry them. A timeout to a throttled dependency is worth another attempt after a backoff; a malformed payload or a rule violation will fail identically every time, so retrying it just burns capacity and delays the moment someone notices. The diagram below shows the second half of the discipline, isolation: keeping a noisy task class on its own queue and worker pool so its retry storm cannot starve the lanes that are still healthy.



First half: retry is a budgeted loop. A transient failure retries with backoff while budget remains; a spent budget or a permanent failure goes to the dead-letter queue.

The second half is the isolation discipline. Classification decides whether a task retries; isolation decides where its retries run, so one noisy class cannot starve healthy lanes:

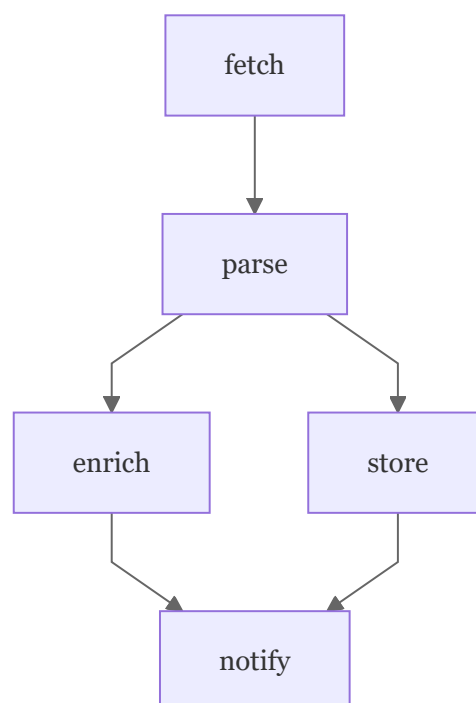


Second half: routing tasks of different classes onto separate worker pools keeps one retry storm contained to its own pool instead of starving the rest.

Result aggregation and scheduling large task graphs

Most real work is not one task; it is a small graph of tasks with dependencies. A scheduler for that graph does not run nodes in source order. It maintains a ready frontier, the set of nodes whose dependencies are all complete, and it only schedules from that frontier. When a node finishes, its outputs are folded into a running aggregate and any successors that just became unblocked join the frontier. The genuinely hard part is rarely the topological order; it is the bookkeeping around it, dependency tracking, fairness between graphs, resource quotas, and the fact that fan-in concentrates load on one reducer.

Read the diagram as a sequence of frontiers, not a static picture. The graph starts with fetch alone in the frontier. Once it completes, parse unblocks. Parsing then unblocks enrich and store together, which is where parallelism appears, and notify is the join point that must wait for both before it can run. That final join is exactly the fan-in pacing boundary the cards warn about.



A four-frontier task graph. fetch then parse run alone; enrich and store run in parallel; notify is the join point that waits for both.

Distributed tracing

In one process a stack trace tells you the causal chain. Across a queue there is no shared stack, so the causal chain has to be carried inside the message itself. That is why a production task envelope holds more than the business payload: it carries the identity and lineage that let an operator stitch a span in one worker to its parent in another. The struct below is the minimum useful envelope. Note that `attempt` and `dedupe_key` are not tracing fields by accident; the same metadata that makes a task traceable also makes it safe to replay.

Distributed tasks profiling

The reflex carried over from single-process work is to profile the handler, find the hot function, and optimize it. In a distributed task system that reflex usually points at the wrong thing. A task can be cheap to run and expensive to wait for, so the first number worth separating is queue wait time versus handler run time. After that, the costs that hide are the ones that only exist because work crosses a process boundary: serialization, redelivery, and the reducer that everything fans into.

A distributed task system often fails because it is profiled like an in-process function call. Queue wait, replay, serialization, and aggregation are the real cost centers more often than the task body alone.

How this lands for each background

Most engineers arrive at distributed tasks with a mental model from another language's concurrency story, and the model is usually close enough to be dangerous. The shift is the same in every case: the primitives you trusted assumed one process with one lifetime, and a task system assumes many processes that fail independently. Each card below names the specific instinct to drop and the one to put in its place.

Production patterns

These are the habits that separate a task system that survives its first incident from one that creates it. None of them are exotic; the value is in doing them before you need to, because each one is much cheaper to build in than to retrofit under load.

Pitfalls and tradeoffs

Nearly every one of these starts as a reasonable assumption carried over from single-process work. They are listed not because they are obscure but because they are easy to ship and slow to notice, surfacing only when the system is already under the load that exposes them.

The hardest distributed-task bug is often not a worker crash. It is a silently wrong recovery model: replay without idempotency, retry without a budget, or tracing without queue-age visibility.

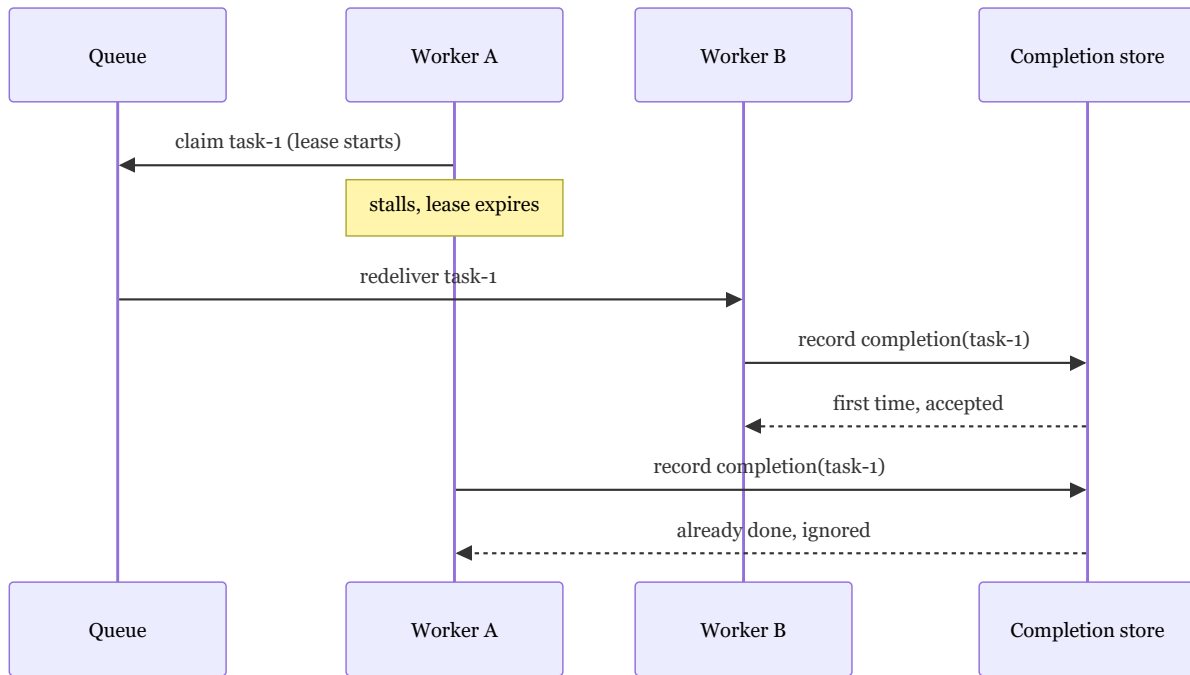
Examples

Both examples are deliberately small enough to run in the editor, and each one models a flow we have already drawn. Read the short note and the diagram above each listing before the code, so the variables in the program map onto a shape you already recognize. Press Run first to anchor the baseline output, then change something and re-run.

Example 1: lease queue with visibility timeout and idempotent completion

One worker claims a task, misses its lease, and another worker reclaims it. The application-side completion path still makes the duplicate harmless.

What to look at: this is the lease lifecycle from the core-concepts diagram, played out between two workers. Worker A claims the task and then goes quiet, so its lease expires and the queue hands the same task to worker B. The line that matters most is the completion check: when worker B finishes, it records the completion against the task ID, and a second completion for the same ID is simply ignored. That single guard is what turns an at-least-once delivery into an exactly-once side effect. Trace the sequence below, then find each step in the code.



Worker A's lease expires and task-1 is redelivered to B. Both workers try to complete, but the completion store dedupes by task ID, so the effect happens once.

Example 31-1. lease_queue_idempotent_worker.rs — Lease idempotent

```

use std::collections::{HashMap, HashSet, VecDeque};

#[derive(Debug, Clone)]
struct Task {
    id: &'static str,
    payload: &'static str,
}

#[derive(Debug)]
struct Lease {
    task: Task,
    deadline: u64,
}

struct LeaseQueue {
    visible: VecDeque<Task>,
    leased: HashMap<&'static str, Lease>,

```

```

    completed: HashSet<&'static str>,
    visibility_timeout: u64,
    now: u64,
}

impl LeaseQueue {
    fn new(visibility_timeout: u64) -> Self {
        Self {
            visible: VecDeque::new(),
            leased: HashMap::new(),
            completed: HashSet::new(),
            visibility_timeout,
            now: 0,
        }
    }

    fn push(&mut self, id: &'static str, payload: &'static str) {
        self.visible.push_back(Task { id, payload });
    }

    fn claim(&mut self, worker: &'static str) -> Option<(&'static str, &'static str, &'static str)> {
        let task = self.visible.pop_front()?;
        let deadline = self.now + self.visibility_timeout;
        let task_id = task.id;
        let payload = task.payload;
        self.leased.insert(task_id, Lease { task, deadline });
        Some((worker, task_id, payload))
    }

    fn advance_time(&mut self, delta: u64) {
        self.now += delta;
    }

    fn requeue_expired(&mut self) {
        let expired: Vec<&'static str> = self
            .leased
            .iter()
            .filter(|(&_, lease)| lease.deadline <= self.now)
            .map(|(&task_id, _)| task_id)
            .collect();

        for task_id in expired {
            let lease = self.leased.remove(task_id).unwrap();
            self.visible.push_back(lease.task);
        }
    }

    fn finish_once(&mut self, task_id: &'static str) -> bool {
        if !self.completed.insert(task_id) {
            return false;
        }

        self.leased.remove(task_id);
        true
    }
}

```

```
fn main() {
    let mut queue = LeaseQueue::new(5);
    queue.push("task-1", "rebuild-index");

    let first = queue.claim("worker-a").unwrap();
    queue.advance_time(6);
    queue.requeue_expired();

    let redelivery = queue.claim("worker-b").unwrap();
    let _applied = queue.finish_once(redelivery.1);
    let duplicate = !queue.finish_once(redelivery.1);

    println!("claimed = {}", first.1);
    println!("redelivered = {}", redelivery.1);
    println!("completed = {}", queue.completed.len());
    println!("duplicates ignored = {}", duplicate);
}
```

OUTPUT

```
claimed = task-1
redelivered = task-1
completed = 1
duplicates ignored = true
```

The queue records one temporary ownership claim plus a deadline.

The task disappears locally until the lease expires.

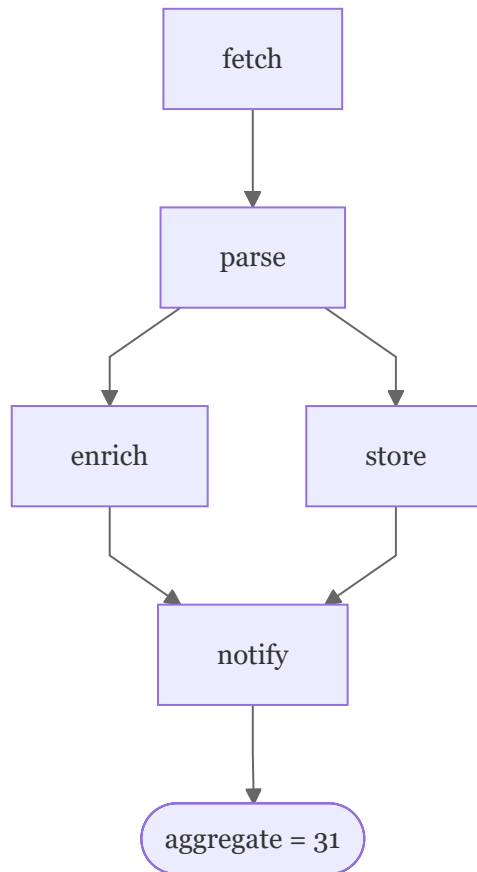
A redelivery is expected behavior after lease expiry, not a surprise incident.

Completion recording makes the second finish attempt harmless.

Example 2: task-graph scheduling, result aggregation, and trace propagation

A tiny DAG scheduler runs tasks whose dependencies are complete, accumulates outputs, and keeps one trace lineage for the whole graph.

What to look at: the scheduler never iterates the nodes in declaration order. On each pass it builds the ready set, the nodes whose dependencies have all completed, runs them, folds their outputs into one aggregate, and repeats until nothing is left. Watch how the same trace ID rides along through every node so the whole run shares one lineage. The five nodes complete across the frontiers shown below; notify is the join that runs last because it depends on the parallel pair before it.



Example 2's graph. The scheduler advances frontier by frontier; outputs fold into one aggregate, and a single trace ID spans every node.

Example 31-2. `task_graph_scheduler_and_trace.rs` — Graph trace

```

use std::collections::BTreeSet;

type TaskId = usize;

#[derive(Debug)]
struct TaskSpec {
    name: &'static str,
    deps: Vec<TaskId>,
    output: u32,
}

#[derive(Default)]
struct TaskGraph {
    nodes: Vec<TaskSpec>,
}

impl TaskGraph {
    fn add(&mut self, name: &'static str, deps: Vec<TaskId>, output: u32) -> TaskId {
        let id = self.nodes.len();
        self.nodes.push(TaskSpec { name, deps, output });
        id
    }

    fn ready(&self, done: &BTreeSet<TaskId>) -> Vec<TaskId> {

```

```

        self.nodes
            .iter()
            .enumerate()
            .filter(|(id, node)| !done.contains(id) && node.deps.iter().all(|dep| done.contains(dep)))
            .map(|(id, _)| id)
            .collect()
    }
}

fn main() {
    let mut graph = TaskGraph::default();
    let fetch = graph.add("fetch", vec![], 5);
    let parse = graph.add("parse", vec![fetch], 8);
    let enrich = graph.add("enrich", vec![parse], 13);
    let store = graph.add("store", vec![parse], 3);
    let _notify = graph.add("notify", vec![enrich, store], 2);

    let trace_id = "trace-7";
    let mut done = BTreeSet::new();
    let mut order = Vec::new();
    let mut aggregate = 0_u32;

    while done.len() < graph.nodes.len() {
        let ready = graph.ready(&done);

        for id in ready {
            let node = &graph.nodes[id];
            aggregate += node.output;
            order.push(node.name);
            done.insert(id);
        }
    }

    println!("completed = {}", order.len());
    println!("aggregate = {}", aggregate);
    println!("final = {}", order.last().copied().unwrap_or("none"));
    println!("trace = {}", trace_id);
}

```

OUTPUT

```

completed = 5
aggregate = 31
final = notify
trace = trace-7

```

Only dependency-satisfied nodes enter the runnable set.

Partial outputs roll into one final aggregate under one owner.

One trace ID ties the graph together across several task boundaries.

Large graphs need fairness, quotas, and persistence beyond this tiny demo.

The repository also includes standalone Rust source under `examples/ch31_distributed_task_execution/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

- Distributed task execution begins with an owned task envelope and a recovery model, not with a queue brand.
- At-least-once processing is the durable default assumption; exactly-once side effects require application-level design.
- Leases and visibility timeouts are crash-recovery tools whose duration is a tradeoff, not a magic number.
- Retries, DLQs, and failure isolation belong in one explicit control plane instead of being copied into each handler.
- Result aggregation, task-graph scheduling, distributed tracing, and profiling all need their own visible budgets and metrics.

CHAPTER 32 · MPI AND HIGH-PERFORMANCE COMPUTING

MPI and High-Performance Computing

MPI process model, collectives, data layout, Rust MPI crates, hybrid MPI plus threads, profiling, and HPC production environments

A cluster job is not one program running faster. It is dozens or thousands of separate processes, on separate machines, agreeing on when to exchange data. This chapter is about writing the Rust half of that agreement: partitioning work into flat buffers, choosing collective operations that match the exchange you actually need, and keeping the cost of communication visible instead of buried.

Builds on Chapters 12, 22, 26, and 31

Chapter 12 established dense layout and matrix views. Chapter 22 separated processes, threads, and thread-safety inside one address space. Chapter 26 compared waiting-heavy orchestration with CPU pools. Chapter 31 showed that once work crosses process boundaries, recovery and replay matter. MPI sits across all four.

Opening scenario

A scientific simulation that already runs on a cluster is being ported to Rust. The team wants to keep everything that works about the existing setup: the MPI launch model, the scheduler that places one rank per socket, and the per-rank threading that fills each socket with local CPU work. What they want from Rust is the part that has been fragile in the old code base, namely confidence about which buffer is owned by which rank and which thread, so that a layout mistake fails at compile time rather than as a silent wrong answer at scale.

That goal turns into four concrete commitments, and the rest of the chapter is really an expansion of them. Partition the data into flat, contiguous buffers so a rank's share is a slice, not a graph. Use collective operations when the whole group participates in one structured exchange, and reserve point-to-point messages for the irregular cases. Keep general-purpose serialization off the hot numeric paths, where a flat typed buffer is both faster and easier to reason about. And measure communication separately from computation, because at cluster scale the time a job spends waiting in collectives is usually the number that decides whether it scales.

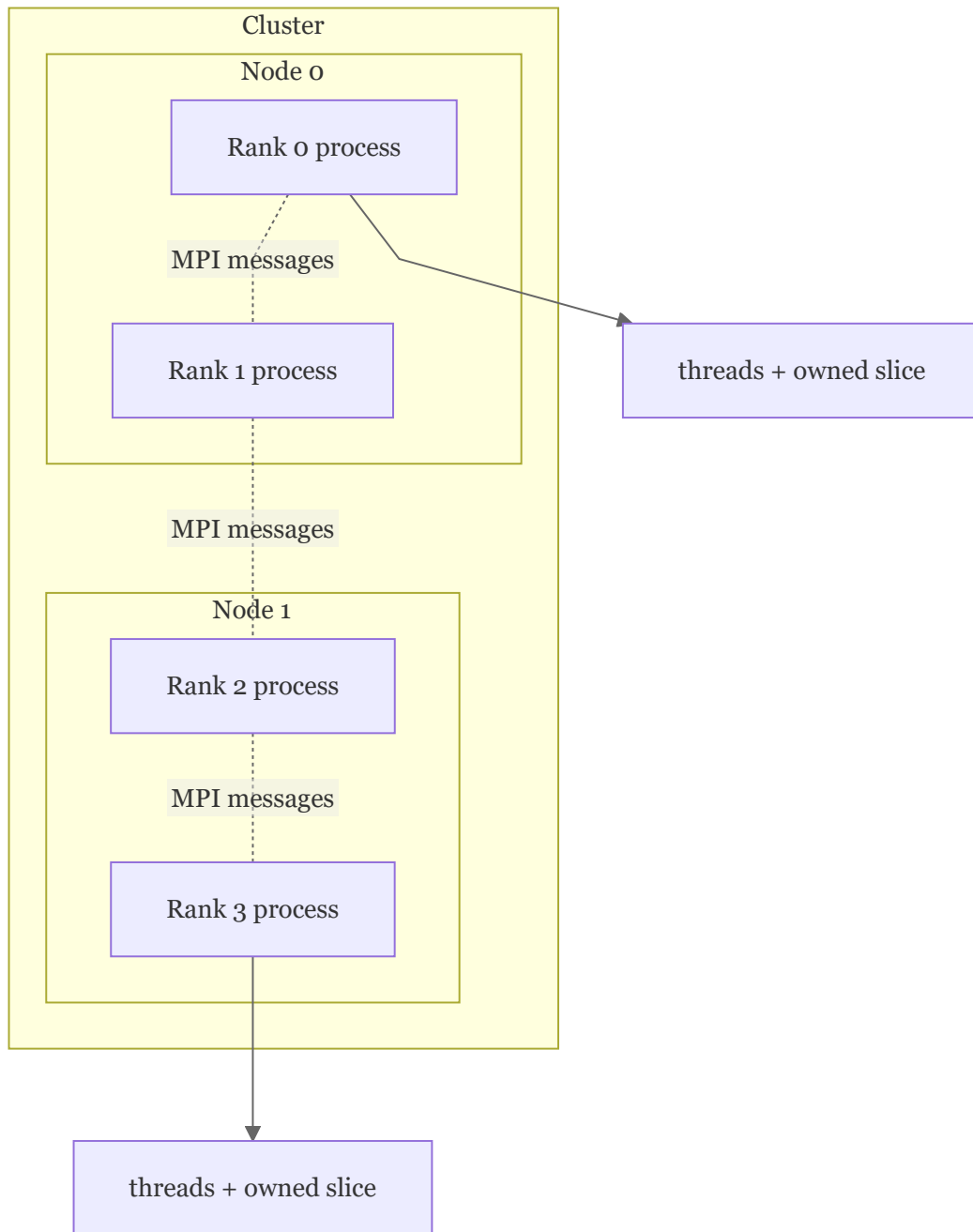
A practical decision order

1. Choose the data layout before choosing the abstraction vocabulary.
2. Choose the partition function before tuning the collective pattern.
3. Choose process count and per-rank thread count together, not independently.
4. Profile communication, packing, and imbalance before blaming the arithmetic kernel.

Mental model

The single most useful idea to hold onto is that an MPI program has two completely different boundaries stacked on top of each other, and they obey different rules. The outer boundary is between ranks. A rank is a full operating-system process with its own address space, so nothing inside it is reachable from another rank

except through data you explicitly send. The inner boundary is between threads inside one rank. There the familiar Rust concurrency model applies unchanged: shared references, Send, Sync, channels, and locks all mean what they mean everywhere else. The diagram below is the shape worth memorizing before reading any further.



Two boundaries: explicit messages between ranks (processes), ordinary Rust ownership between threads inside a rank.

Core concepts

The three nouns every MPI program starts from

MPI has a large API surface, but almost everything is built from three ideas. A communicator is the group of processes that are allowed to talk to each other. A rank is one process's integer index inside that group. The

size is how many ranks the group has. Every partition decision in this chapter is just arithmetic on a rank and a size: rank 1 of 3 takes this slice, rank 2 of 3 takes the next one. The cards below name the building blocks, and the snippet after them is the opening of essentially every MPI program written with the Rust mpi crate.

What to read for: `initialize()` hands back a universe whose lifetime is the MPI session, and rank and size are the two numbers the rest of your code branches on.

Why ranks and threads are not the same tool

This is the distinction that most often trips up engineers arriving from a shared-memory background. Both ranks and threads give you parallelism, so it is tempting to treat them interchangeably, but they solve different problems and fail in different ways. Ranks give you more memory and more machines at the price of having to move every shared byte explicitly. Threads give you cheap sharing at the price of having to reason about exclusive mutation. A real cluster job almost always uses both, and the skill is keeping clear about which one you are reaching for in any given line.

Message passing at HPC scale

At small scale, many communication patterns look cheap enough to ignore. At cluster scale, collective count, message size, synchronization frequency, and packing cost often dominate. A barrier inside a hot loop is not a style choice. It is a global pacing decision across every rank.

This is one reason Rust fits HPC work well. Once the buffers and ownership are explicit, the remaining questions are the same ones HPC engineers already ask: what is contiguous, what is copied, and which ranks are waiting for which other ranks?

Choosing a Rust MPI crate

Rust does not ship its own MPI implementation, and it should not. MPI is a standard with several mature implementations (Open MPI, MPICH, and vendor variants) that are tuned for specific interconnects and already installed on the cluster. A Rust crate is a binding to whichever one is present, not a replacement for it. That framing matters because it tells you where your real dependency lives: not in `Cargo.toml`, but in the system library and launcher your job will actually link and run against.

Keep crate choice subordinate to system choice. The real dependencies are the installed MPI runtime, the launcher, the scheduler, and the fabric assumptions on the cluster.

Why flat buffers win in MPI

Data layout is the decision that quietly determines how hard everything else will be. MPI sends operate on contiguous regions described by a starting pointer and a count, and so do the caches and the FFI boundary into the system MPI library. A flat row-major buffer satisfies all three at once: a rank's rows are a single slice, the send is one range, and the CPU walks memory in order. The moment you reach for a nested structure such as a vector of vectors, you trade that single predictable buffer for many scattered allocations, and every collective and every FFI call now has to reassemble what should have stayed contiguous. The rules below are how to stay on the easy path.

The arithmetic that makes this work is one line: a cell's position in the flat buffer is $\text{row} * \text{cols} + \text{col}$, so a rank's row range maps to a cell range by multiplying both ends by cols.

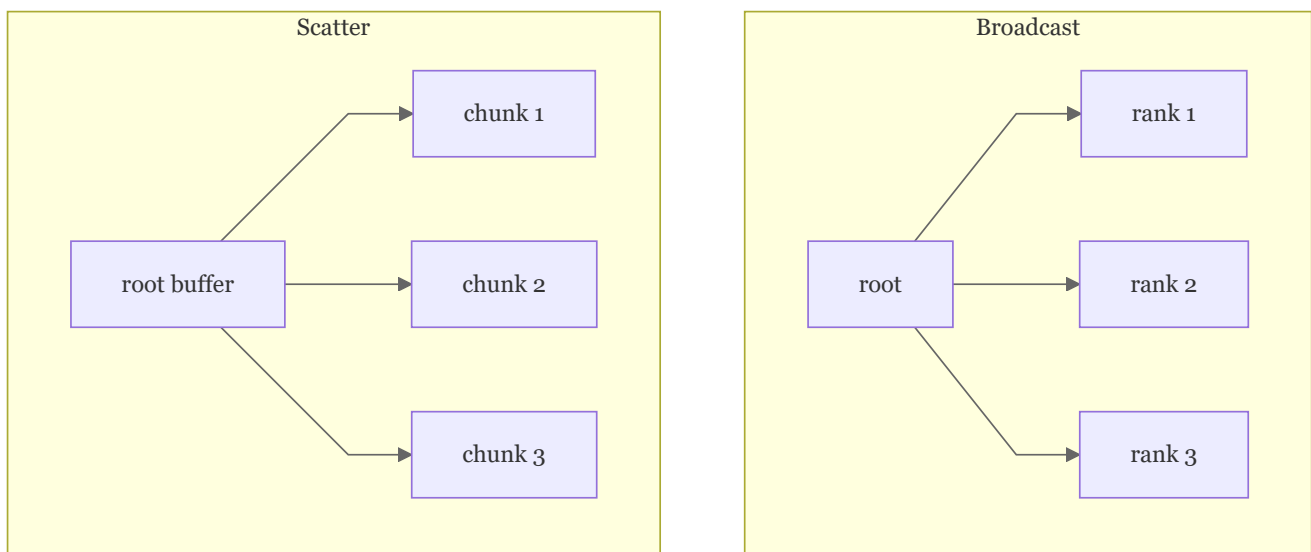
When to serialize and when not to

Rust's serialization story is excellent, and that is exactly why it is worth saying clearly where it does not belong. A dense numeric exchange (an allreduce over a million doubles, repeated every iteration) should move as raw f64 bytes, because the data already has a fixed shape and any envelope is pure overhead on the hottest path in the program. A control message that says "reconfigure to this topology" runs once and benefits from a real schema. The guideline below is just that split made explicit: match the ceremony of the payload to how often it crosses the wire.

A useful rule is simple: the hotter the collective, the less general the payload should be. Dense numeric exchange wants flat typed buffers. Rich envelopes belong on colder control paths.

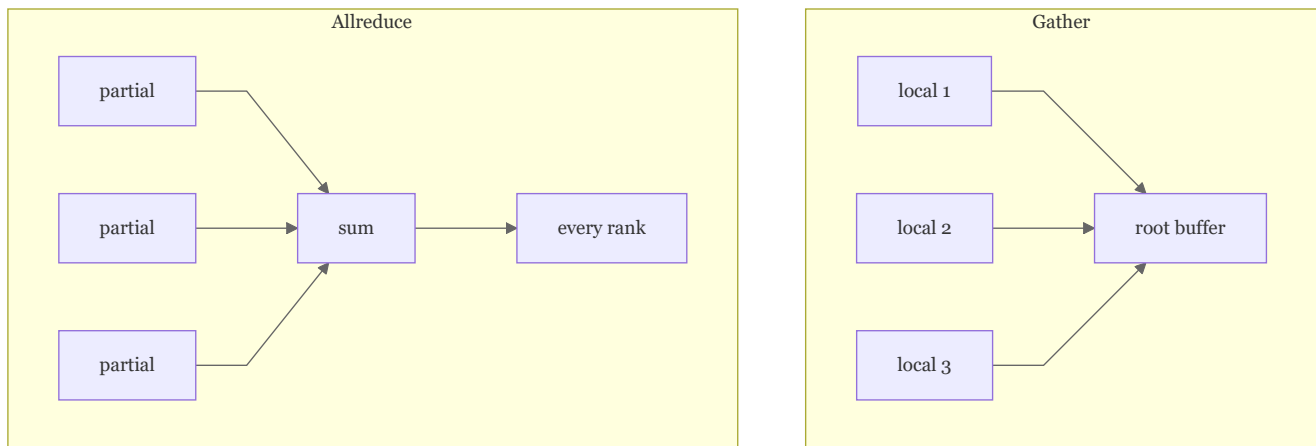
Collective operations and the shapes they make

A collective is a single operation that every rank in a communicator calls together. The payoff over hand-rolled sends and receives is twofold: far less boilerplate, and a synchronization contract the MPI implementation can optimize as a whole (often using tree or ring algorithms you would not want to write by hand). The trick to choosing among them is to picture the direction the data flows. Broadcast pushes one buffer out to everyone; scatter splits one buffer into pieces; gather collects pieces back; reduce folds them into one value at the root; allreduce folds them and hands the result back to everyone. The diagram makes those four shapes concrete.



Fan-out collectives: broadcast pushes one buffer to everyone, scatter splits one buffer into per-rank pieces.

The other two shapes run in the opposite direction, collecting data back toward the root:



Fan-in collectives: gather collects pieces back to the root, allreduce folds them then redistributes the result to every rank.

Combining MPI ranks with per-rank threads

The dominant production layout is not "one rank per core." It is one rank per socket or per NUMA domain, with each rank running a thread pool (Rayon, or plain threads) across the cores it owns. The reasoning is economic: crossing the network is expensive, so you want as few ranks as the memory budget allows, and you fill each rank's cores with cheap shared-memory parallelism. This gives you two ownership layers to keep straight. Between ranks, ownership moves by message. Inside a rank, ownership is the ordinary Rust story of owned chunks and scoped borrowed slices handed to the pool. The one configuration mistake that wrecks this is oversubscription: if every rank spawns a full machine's worth of threads, the node thrashes on context switches instead of computing.

Where the time actually goes

The instinct from single-process work is to profile the arithmetic kernel, and in HPC that instinct is frequently wrong. A collective is a barrier: it does not complete until the slowest rank arrives. So when a profiler shows ranks blocked in an allreduce, that time is usually not the cost of the reduction itself, it is the cost of one rank having more work to do than the others. The first question to ask of a slow cluster job is therefore not "which line is hot" but "are all the ranks finishing their local work at the same time." Measure compute, packing, and time-blocked-in-collectives as three separate numbers, and look at load imbalance before you touch the math.

What the cluster environment demands

A surprising share of failed cluster jobs have nothing to do with the algorithm and everything to do with the environment the job runs in. The binary must link against the same MPI family it launches against, the launcher and scheduler have to agree on how many ranks go where, and logging has to survive the fact that there are now many processes producing output at once. None of this is exotic, but it is easy to leave implicit until the first run at scale fails in a way that single-machine testing never reproduced. The cards below name the assumptions worth writing down before launch day.

How to think about this coming from another language

MPI predates Rust by decades and exists in C, C++, Fortran, and Python, so most readers arrive with some prior model of either MPI or of parallelism in their home language. The useful question is not "what is the

Rust API for X" but "what mental habit do I need to drop." Each card below is one such shift, not a library cheat sheet.

Production patterns

Pitfalls and tradeoffs

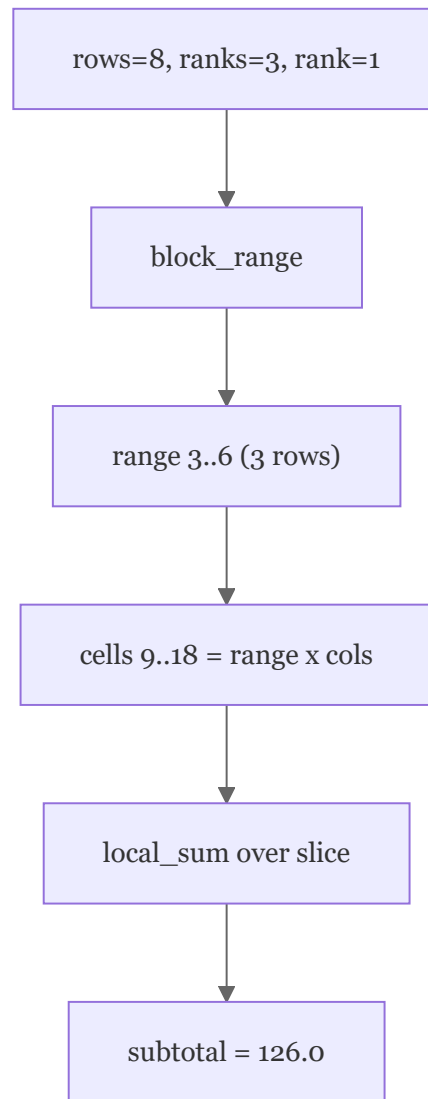
The hardest MPI performance bug is often not one slow instruction. It is one wrong boundary: too many collectives, the wrong layout, one rank doing more work, or one node running too many local threads.

Examples

Example 1: block partitioning for dense row-major work

This is the partition math every MPI rank runs before it touches any data. In a real program rank and size come from the communicator; here they are constants so the example runs standalone. The function is identical either way.

What to look at: `block_range` turns the pair (rank, size) into a half-open row range, handing one extra row to each of the first few ranks so the eight rows split as 3, 3, 2 instead of leaving a remainder stranded. Then `local_sum` converts that row range into a cell range by multiplying by cols and sums only that slice. Follow the data through the three boxes below before reading the code.



Rank and size in, a contiguous row range out, then a slice sum on only this rank's rows.

Example 32-1. `mpi_partition_dense_rows.rs` — Dense rows

```

#[derive(Debug, Clone, Copy)]
struct Partition {
    start_row: usize,
    end_row: usize,
}

fn block_range(rows: usize, ranks: usize, rank: usize) -> Partition {
    let base = rows / ranks;
    let remainder = rows % ranks;
    let start = rank * base + rank.min(remainder);
    let len = base + usize::from(rank < remainder);

    Partition {
        start_row: start,
        end_row: start + len,
    }
}
  
```

```
fn local_sum(matrix: &[f64], cols: usize, part: Partition) -> f64 {
    let start = part.start_row * cols;
    let end = part.end_row * cols;
    matrix[start..end].iter().copied().sum()
}

fn main() {
    let rows = 8_usize;
    let cols = 3_usize;
    let ranks = 3_usize;
    let rank = 1_usize;

    let matrix: Vec<f64> = (1..=(rows * cols)).map(|value| value as f64).collect();
    let part = block_range(rows, ranks, rank);
    let subtotal = local_sum(&matrix, cols, part);

    println!("rank = {}", rank);
    println!("rows = {}..{}", part.start_row, part.end_row);
    println!("local rows = {}", part.end_row - part.start_row);
    println!("subtotal = {:.1}", subtotal);
}
```

OUTPUT

```
rank = 1
rows = 3..6
local rows = 3
subtotal = 126.0
```

Extra rows are distributed to the first ranks so the load stays balanced when the row count is not divisible evenly.

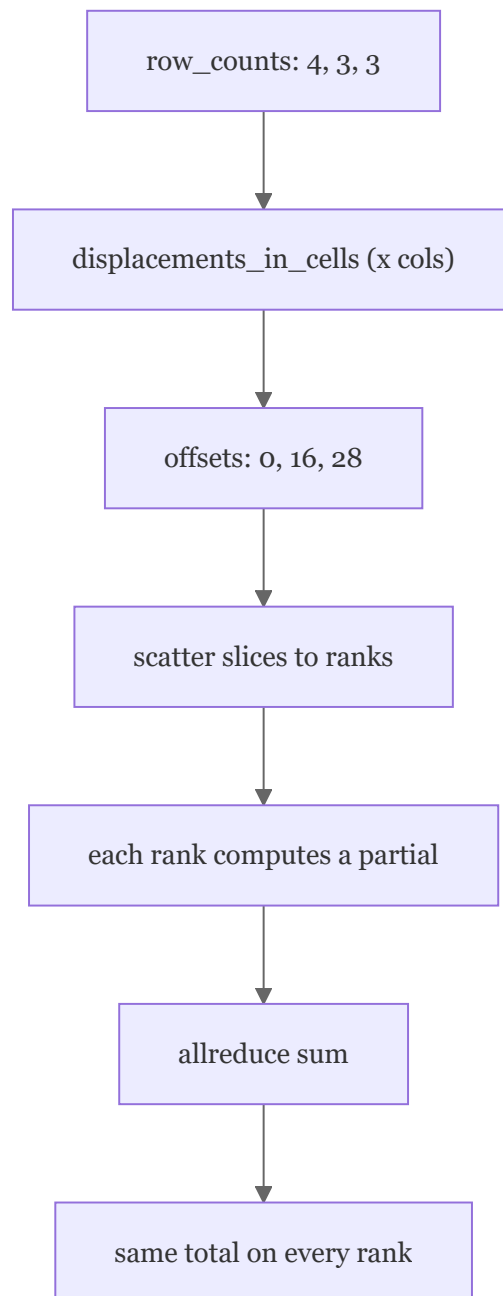
One flat buffer plus one row-major offset keeps both cache behavior and MPI send ranges simple.

The repository example file shows the same idea with real `mpi` crate calls and an allreduce.

Example 2: counts, displacements, and collective-friendly layout

A scatter or gather needs two parallel tables: how many elements go to each rank, and at what offset each rank's slice begins. Most scatter and gather bugs are simply one of these tables computed in the wrong unit. The discipline is to keep counts in the unit the algorithm thinks in (rows) and convert to the unit the collective wants (cells) at exactly one place.

What to look at: `row_counts` answers the load question in rows (10 rows over 3 ranks becomes 4, 3, 3). Then `displacements_in_cells` walks those counts once, multiplying by `cols` to produce the per-rank starting offset a real scatter would consume. The final allreduce is a separate fan-in that hands every rank the same total. The diagram traces those two tables and the reduction.



Counts in rows, displacements in cells, then a scatter feeds local work that an allreduce folds back together.

Example 32-2. `mpi_collective_counts_and_allreduce.rs` — And allreduce

```

fn row_counts(rows: usize, ranks: usize) -> Vec<usize> {
    let base = rows / ranks;
    let remainder = rows % ranks;

    (0..ranks)
        .map(|rank| base + usize::from(rank < remainder))
        .collect()
}

fn displacements_in_cells(counts: &[usize], cols: usize) -> Vec<usize> {
    let mut out = Vec::with_capacity(counts.len());
    let mut offset = 0_usize;

```

```

    for &rows_for_rank in counts {
        out.push(offset);
        offset += rows_for_rank * cols;
    }

    out
}

fn main() {
    let rows = 10_usize;
    let cols = 4_usize;
    let ranks = 3_usize;
    let rank = 1_usize;

    let counts = row_counts(rows, ranks);
    let displs = displacements_in_cells(&counts, cols);
    let local_norms = [7_u64, 9, 5];
    let allreduce_sum: u64 = local_norms.iter().copied().sum();

    println!("counts = {:?}", counts);
    println!("displs = {:?}", displs);
    println!("send cells = {}", counts[rank] * cols);
    println!("allreduce = {}", allreduce_sum);
}

```

OUTPUT

```

counts = [4, 3, 3]
displs = [0, 16, 28]
send cells = 12
allreduce = 21

```

Row counts answer the load question first.

Collective APIs often want offsets in typed elements, not in rows.

Every rank can receive the same final scalar when the next phase needs it locally.

Keep the hot numeric path flat and typed; send richer metadata elsewhere.

The repository also includes standalone Rust source under `examples/ch32_mpi_and_high_performance_computing/` including real `mpi` crate examples for row partitioning, collective planning, and hybrid MPI plus Rayon execution.

Summary

- MPI is process-based message passing, so cross-rank ownership is always a buffer or protocol boundary.
- Processes and threads solve different problems. Hybrid MPI plus threads is common, but the two levels should stay conceptually separate.
- Rust MPI work is usually easiest when dense payloads stay flat, contiguous, and explicit about layout.
- Serialization for hot numeric collectives should stay minimal; richer envelopes belong on colder orchestration paths.

- Collective choice, queueing inside ranks, thread budgets, and cluster build assumptions are all part of the production design.
- Profile communication, packing, and imbalance before blaming the arithmetic kernel or the language.

CHAPTER 33 · PERFORMANCE-ORIENTED RUST

Performance-Oriented Rust

Rust's performance model, allocation awareness, cache locality, branch prediction, static dispatch, iterator tradeoffs, benchmarking methodology, and release profile choices

Performance work starts with allocation, data movement, cache behavior, and measurement. This chapter turns Rust optimization into a repeatable review process tied to workload evidence.

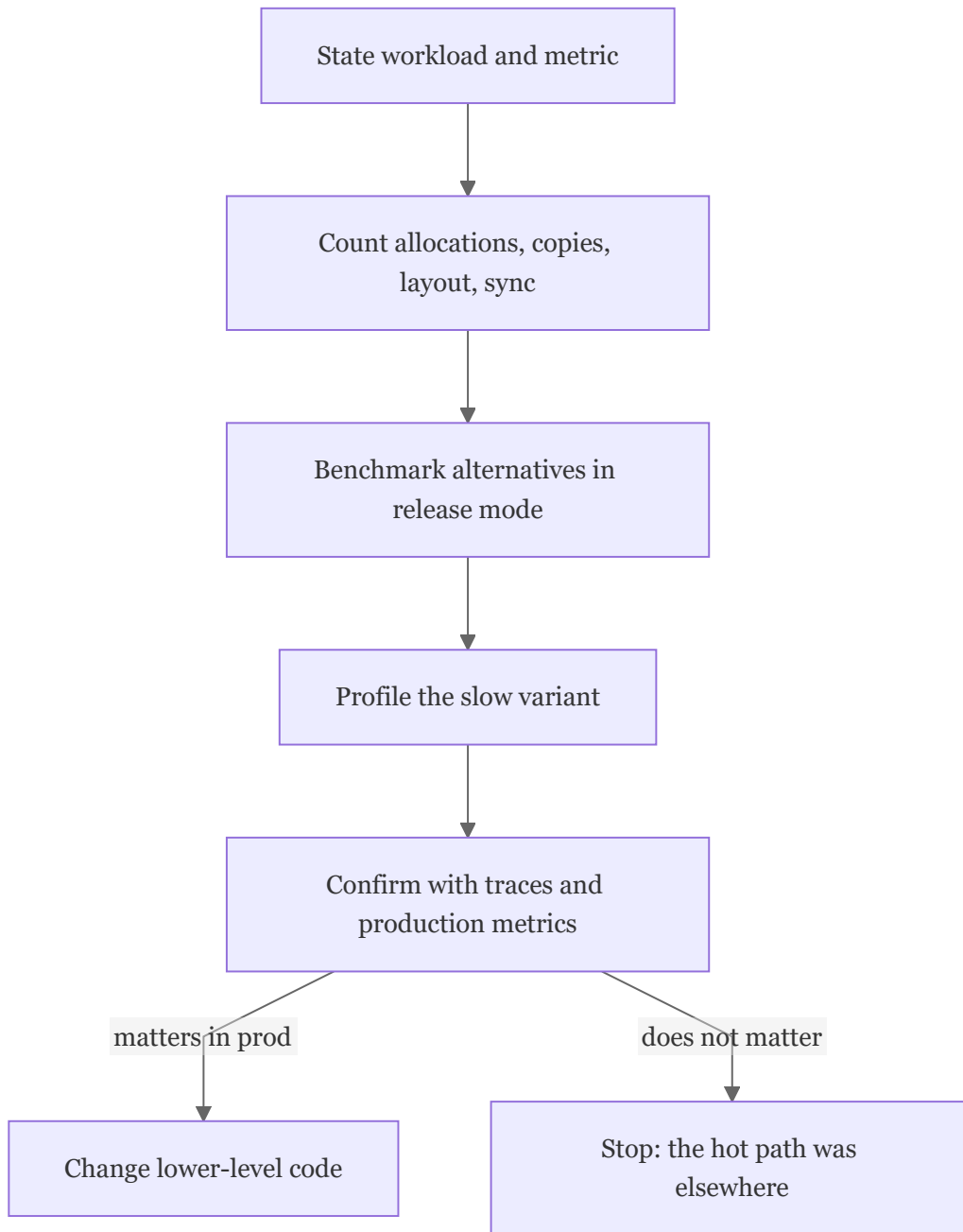
Builds on Chapters 10, 18, 22, 27, and 32

Chapter 10 covered arrays, slices, and vectors as layout choices. Chapter 18 covered generics and monomorphization. Chapter 22 covered multithreading costs and ownership across workers. Chapter 27 covered IO and batching. Chapter 32 covered flat buffers and communication-heavy HPC thinking. This chapter turns those threads into a practical Rust performance model.

Opening scenario

A request-processing service regressed after a refactor that was meant to improve readability. p99 latency climbed, heap traffic rose, and queue wait that had been invisible became a line on the dashboard. Nobody changed an algorithm; the team changed where data was owned and copied, and the cost showed up downstream. The job now is not to guess. It is to rebuild the cost model deliberately, measuring allocation, data movement, layout, dispatch, synchronization, and live production latency before touching any lower-level code.

That last clause is the whole chapter. The fastest way to waste a week is to start by rewriting the inner loop. The reliable path is to ask the questions in order, let each one rule out a class of explanation, and only descend to instruction-level work once the evidence points there. The diagram below is the order; the sections after it explain each box.



The performance review runs top to bottom. Each step can end the investigation before you reach the code.

The review order, in words

1. State the workload and the metric first, so every later number has a question it answers.
2. Count allocations, copies, cache-unfriendly layout, and synchronization boundaries by reading the code.
3. Benchmark the alternatives in release mode, never in debug.
4. Profile the slow one to learn where the time actually goes, then confirm the live system with traces and production metrics.

Mental model

Before any technique, fix the frame. Rust does not make programs fast; it removes whole categories of accidental cost and then hands you a clear view of the cost that remains. The phrase “zero-cost abstraction” is often misread as “zero work.” It means the opposite of free: it means an abstraction does not add overhead beyond the hand-written equivalent, so the real work is still entirely present and still entirely yours to manage. The three ideas below are the lens for the rest of the chapter.

Core concepts

The four costs worth tracking

Almost every Rust performance problem reduces to one of four costs. Keeping the list short is the point: when latency regresses, you walk these four in order and ask which one the change touched. Most regressions announce themselves here long before you need a profiler.

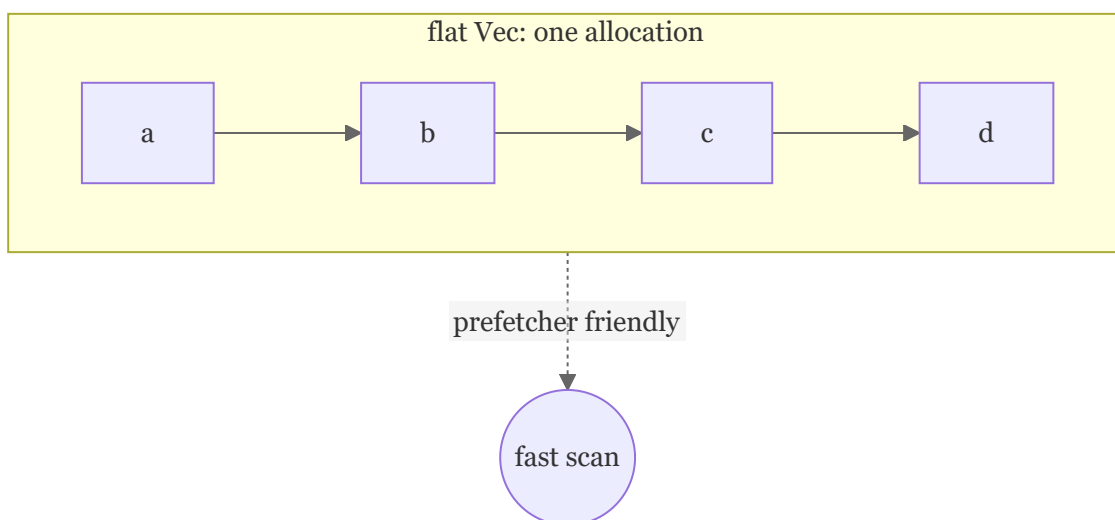
Allocation awareness

Allocation is the cost most engineers underestimate because no single call looks expensive. The damage is cumulative: a per-request String here, a clone() there, a Vec that grows by reallocating instead of being sized once, and suddenly the allocator is on your hot path and your tail latency depends on it. The practical moves are unglamorous and they pay reliably.

A useful correction for senior engineers is this: ownership clarity and performance clarity often improve together. The cleanest API shape frequently allocates less because the borrow or owned boundary is now explicit instead of accidental.

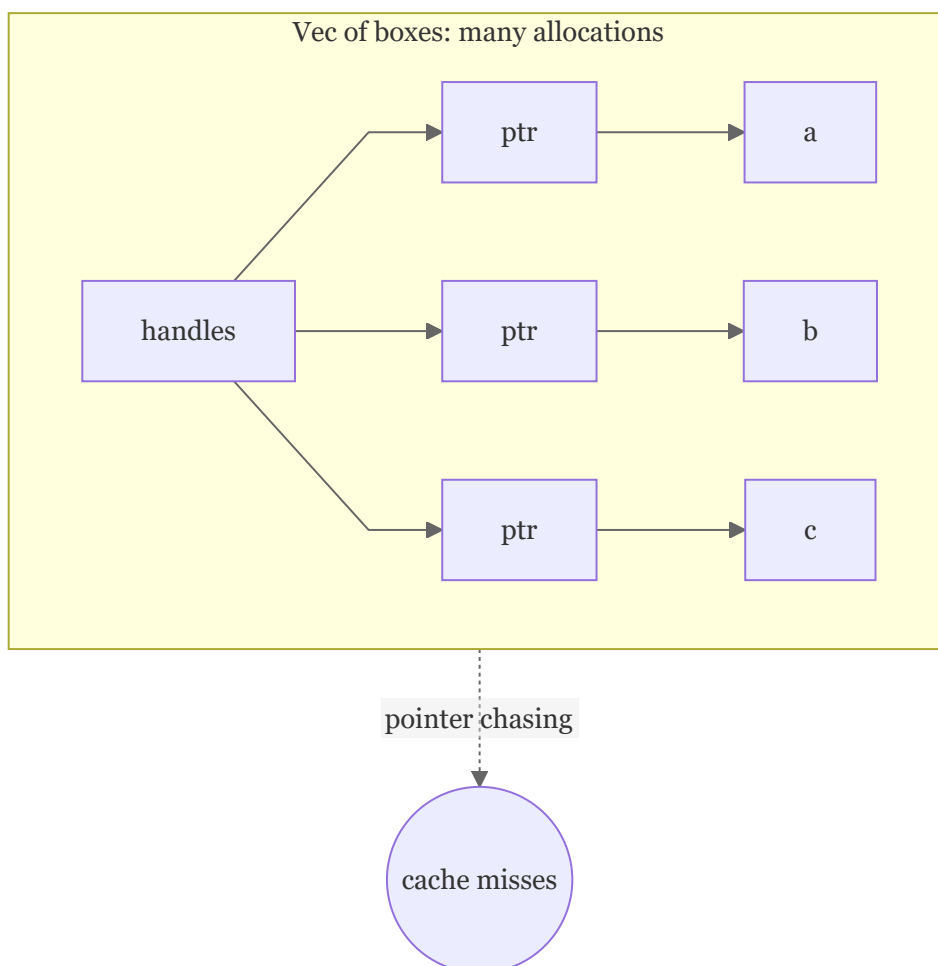
Cache locality and branch prediction

Two data shapes can hold identical values and run at very different speeds. The difference is what the hardware sees while it walks them. A single flat allocation lets the prefetcher pull the next cache line before you ask for it; a structure made of many small allocations forces the CPU to chase a pointer, stall on the load, then chase the next one. The diagram below is the same data in both shapes — this is the picture to keep in mind whenever you reach for a nested collection.



Flat layout: one allocation, values adjacent, the prefetcher pulls the next line ahead of you.

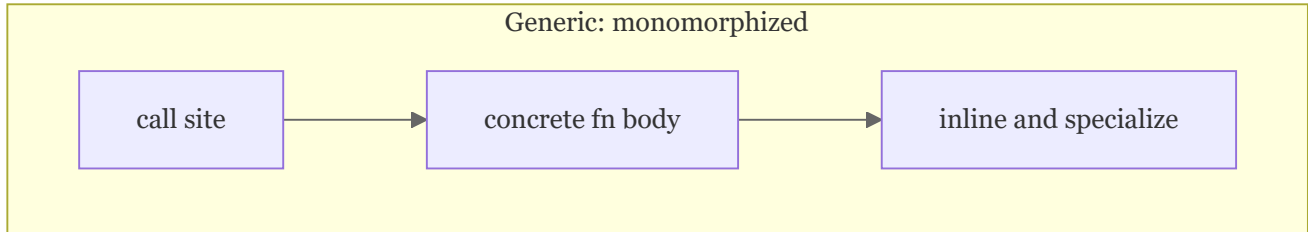
The same values stored as a graph of small allocations look very different to the hardware:



Boxed layout: each element is a separate allocation, so the CPU stalls chasing one pointer after another.

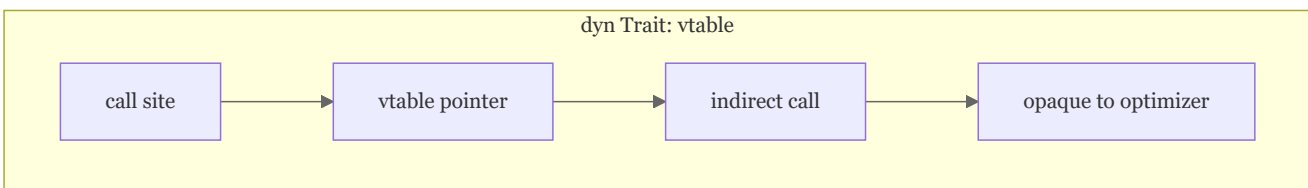
Static dispatch and inlining

Generics and trait objects are the two ways Rust calls behavior you do not know concretely at the call site, and they sit at opposite ends of the cost spectrum. A generic function is monomorphized: the compiler stamps out a separate copy per concrete type, so each copy sees an exact call target it can inline and optimize through. A dyn Trait value carries a pointer to a vtable, and the call goes through that pointer, which the optimizer usually cannot see past. The diagram shows where the indirection lives.



Static dispatch: the call site sees the concrete body, so the optimizer can inline and specialize through it.

Dynamic dispatch puts a vtable on the same path, and the indirection is where the optimizer loses sight of the target:



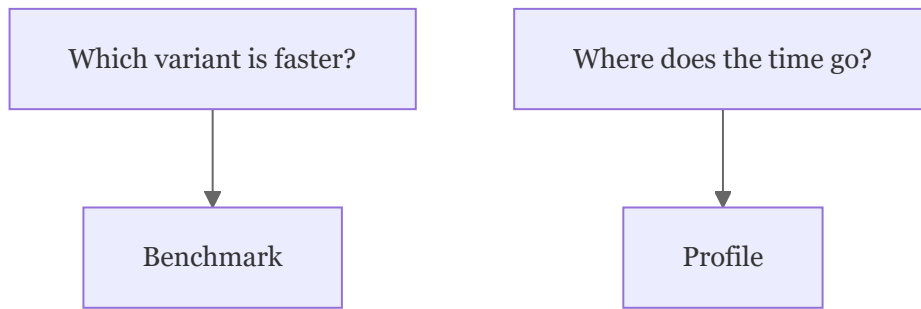
Dynamic dispatch: the call routes through a vtable pointer the optimizer usually cannot follow past.

Iterator performance, bounds checks, and unnecessary clones

There is a persistent folklore that iterator chains are slow and hand-written index loops are fast. The truth is closer to the reverse, and the reason is bounds checks. When you index a slice manually with a data-dependent index, the compiler must insert a check that the index is in range, because it cannot prove otherwise. When you iterate — over a slice, over chunks, over windows — the range is encoded in the iterator itself, so the check is provably unnecessary and the compiler removes it. The honest answer is always to read the generated code, but the default instinct should be: express the loop in terms of slices and let the compiler discharge the proof for you.

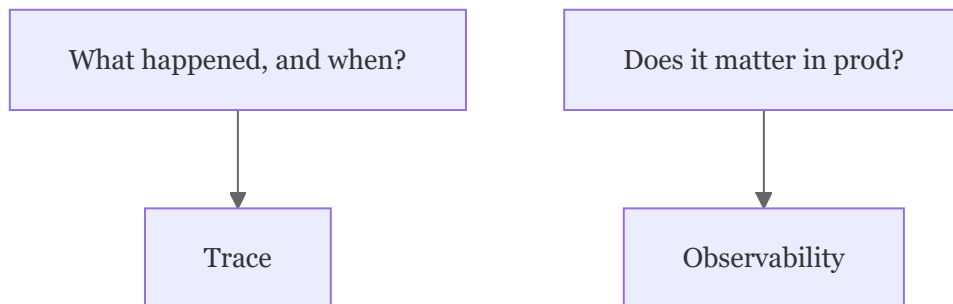
Picking the right measuring tool

The four tools below are not a hierarchy where one replaces the others; they answer four different questions, and a confident answer from the wrong tool is how teams optimize code that was never on the critical path. Match the tool to the question first. The diagram is that mapping; the cards explain each one.



Comparing alternatives is a benchmark question; finding where the time goes is a profiling question.

The other two questions are about timeline and real load, and they map to two more tools:



Reconstructing a timeline is a tracing question; whether it matters under real load is an observability question.

Do not claim a speedup without workload context. A cleaner benchmark is not a substitute for profiling, and a flatter profile is not a substitute for production latency or throughput data.

Release builds and compiler flags

Measure performance in release mode. Debug builds are for fast iteration and diagnostics, not for trustworthy throughput or latency numbers. The first repair for “Rust is slow” is often simply `--release`.

Release profile settings are workload tradeoffs. More optimization or LTO may help hot binaries, but it also affects compile time, binary size, and sometimes panic behavior.

How performance thinking shifts by background

Senior engineers do not arrive at Rust as blank slates. The useful question is not which crate replaces which library; it is which of your instincts still hold and which one quietly leads you wrong. Each card names the mental-model shift for one background.

Production patterns

These are the habits that keep a service fast without a heroics phase. None of them is a micro-optimization; each one shapes the data and the API so the expensive cost is paid once, at a boundary you chose, instead of silently on every request.

Pitfalls and tradeoffs

Each of these is a true story that started with a reasonable-sounding sentence. The common thread is declaring victory from one number while a different number quietly got worse, or reaching for a lower-level change before the evidence pointed there.

The most common performance mistake in senior Rust code is not “using iterators.” It is skipping the step where the team states what is actually expensive and how that expense will be measured.

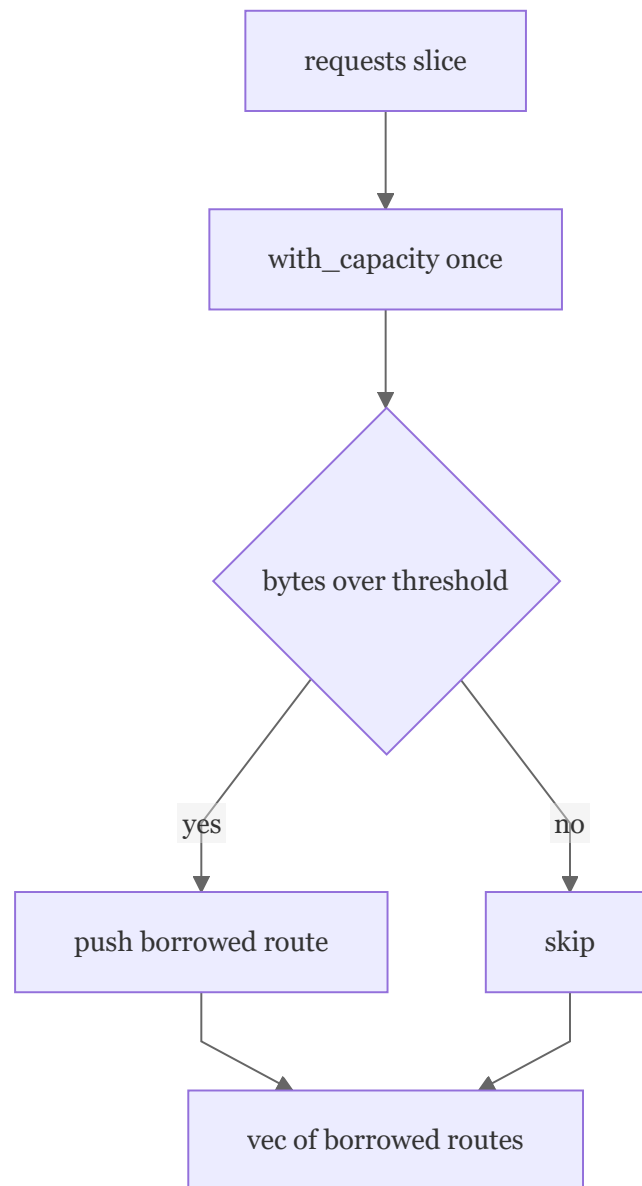
Examples

Both examples are small on purpose. Each one is a single design decision from the sections above, made concrete and runnable. Read the flow diagram first, then the code, then press Run and confirm the output before changing anything — the point is to see the cost decision in the signature, not just to watch it print.

Example 1: allocation-aware filtering without route clones

The function preallocates once, borrows route labels from the input slice, and keeps the hot path allocation-light.

What to look at: the return type is `Vec<&str>`, not `Vec<String>`. The function never copies a route; it borrows each label out of the input and hands back views that live exactly as long as the input slice. The one allocation in the whole function is the result vector, and `Vec::with_capacity` sizes it once up front so the push loop never reallocates. The flow is: size the buffer, then for each request keep or skip on a single threshold test.



One allocation for the result, borrowed labels throughout, one branch per request.

Example 33-1. allocation_borrowed_filter.rs — Borrowed filter

```

#[derive(Debug)]
struct Request<'a> {
    route: &'a str,
    bytes: usize,
}

fn hot_routes<'a>(requests: &'a [Request<'a>], min_bytes: usize) -> Vec<&'a str> {
    let mut out = Vec::with_capacity(requests.len());

    for request in requests {
        if request.bytes >= min_bytes {
            out.push(request.route);
        }
    }

    out
}

```

```

}

fn main() {
    let requests = [
        Request {
            route: "/health",
            bytes: 128,
        },
        Request {
            route: "/search",
            bytes: 900,
        },
        Request {
            route: "/checkout",
            bytes: 512,
        },
        Request {
            route: "/metrics",
            bytes: 64,
        },
    ];

    let hot = hot_routes(&requests, 512);

    println!("hot = {}", hot.len());
    println!("first = {}", hot.first().copied().unwrap_or("none"));
    println!("capacity ok = {}", hot.capacity() >= requests.len());
}

```

OUTPUT

```

hot = 2
first = /search
capacity ok = true

```

`Vec::with_capacity` makes the result budget visible instead of relying on repeated growth.

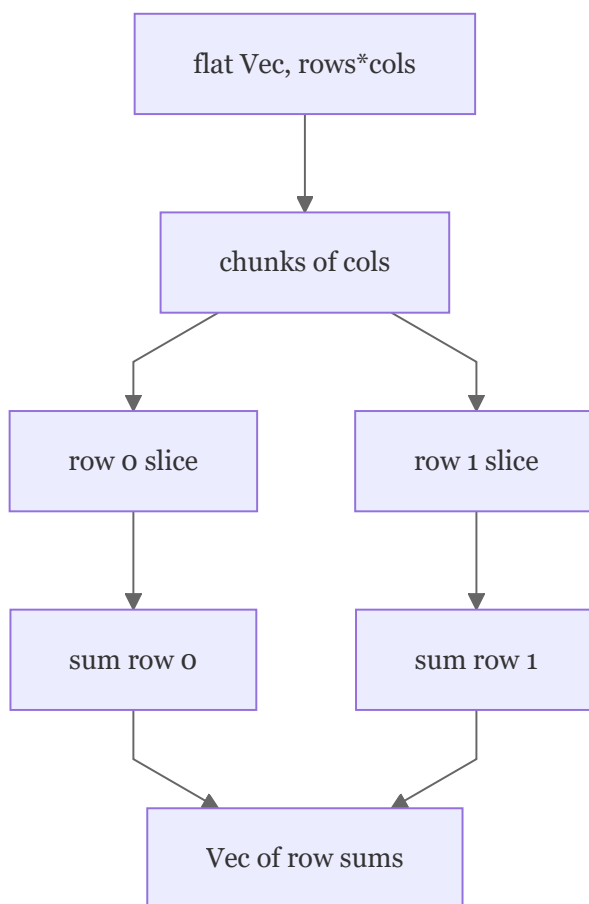
The output stores `&str` views because the result only needs to live as long as the input slice.

The loop does one clear threshold test and pushes only on the hot predicate.

Example 2: row-major scanning for predictable locality

The grid stays flat and contiguous. Row sums are derived with chunked iteration instead of nested owners or scattered indexing.

What to look at: the grid is one `Vec<u32>`, not a `Vec<Vec<u32>>`. The rows are not separate allocations; they are slices carved out of the same flat buffer by chunks(`self.cols`). That single choice is what keeps the scan contiguous and lets the compiler drop bounds checks, because the chunk width proves the row boundary instead of a hand-written index. The flow is: take the flat buffer, split it into row-width chunks, sum each chunk.



One contiguous buffer, sliced into rows by width. No nested allocation, no manual index proof.

Example 33-2. row_major_scan.rs — Major scan

```

#[derive(Debug)]
struct Grid {
    rows: usize,
    cols: usize,
    data: Vec<u32>,
}

impl Grid {
    fn row_sums(&self) -> Vec<u32> {
        debug_assert_eq!(self.data.len(), self.rows * self.cols);
        self.data
            .chunks(self.cols)
            .map(|row| row.iter().copied().sum())
            .collect()
    }

    fn total(&self) -> u32 {
        self.data.iter().copied().sum()
    }
}

fn main() {
    let grid = Grid {
        rows: 2,

```

```
    cols: 4,  
    data: vec![1_u32, 2, 3, 4, 5, 6, 7, 8],  
};  
  
let sums = grid.row_sums();  
  
println!("row0 = {}", sums[0]);  
println!("row1 = {}", sums[1]);  
println!("total = {}", grid.total());  
}
```

OUTPUT

```
row0 = 10  
row1 = 26  
total = 36
```

One flat buffer plus `chunks(self.cols)` keeps row traversal predictable for the CPU.

The iterator is working with exact slices, which is often easier to optimize than ad hoc indexing.

Chunking expresses the row boundary structurally instead of relying on a manual index proof.

The repository also includes standalone Rust source under `examples/ch33_performance_oriented_rust/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

CHAPTER 34 · MEMORY PROFILING

Memory Profiling

Measuring allocations, heap profiling, clone pressure, cycle leaks, fragmentation, arena retention, async memory usage, allocator behavior, and footprint reduction

Memory profiling identifies where allocations, retention, and queue growth affect cost and reliability. This chapter connects Rust ownership decisions to measurable memory behavior.

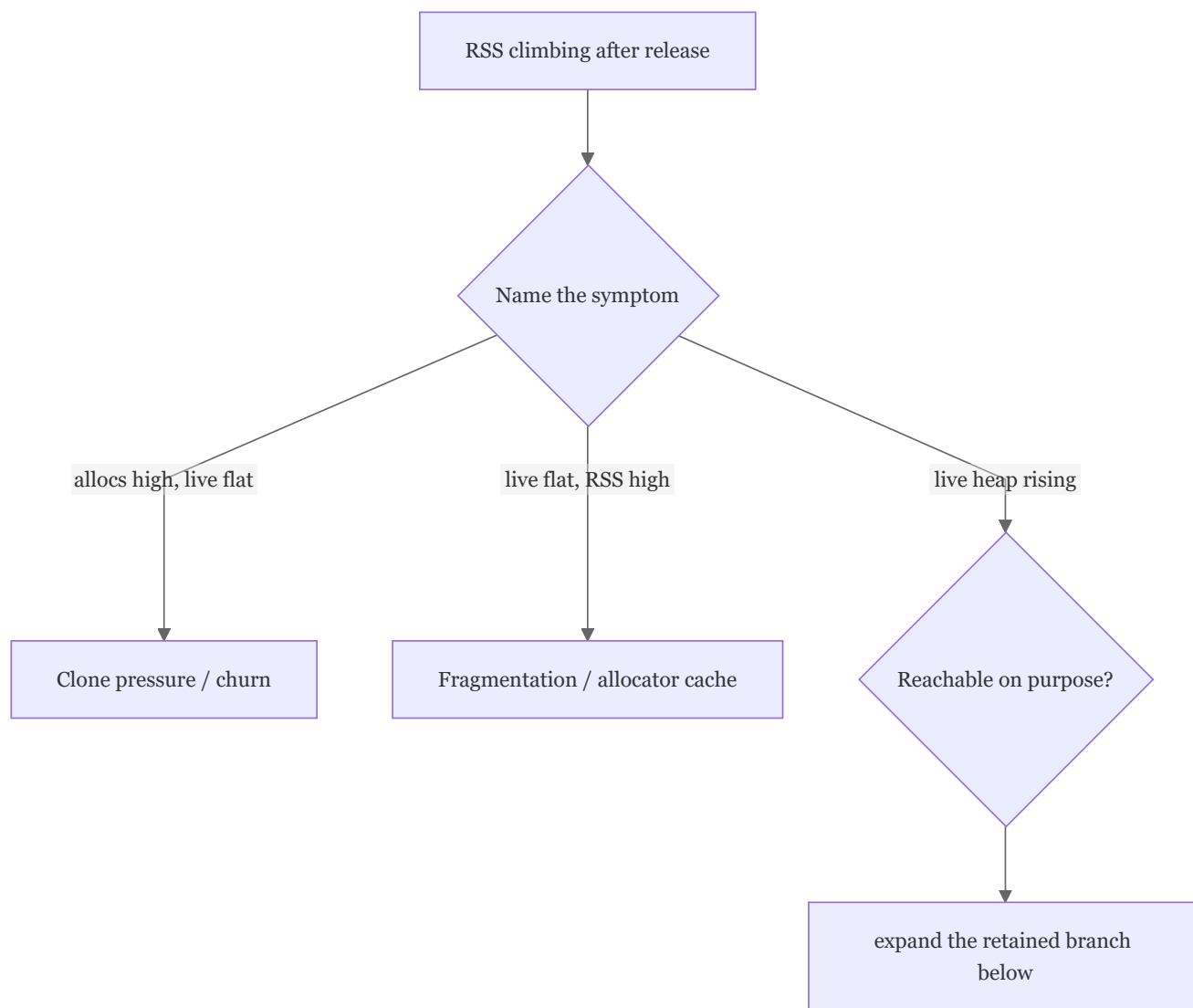
Builds on Chapters 07, 09, 13, 25, and 33

Chapter 07 separated deep clone cost from ordinary moves and `Arc::clone`. Chapter 09 covered `Rc`, `Arc`, `Weak`, and cycle risk. Chapter 13 introduced arena lifetimes and region retention. Chapter 25 explained task and queue backpressure in Tokio. Chapter 33 established the broader performance review model that this chapter narrows to memory.

Opening scenario

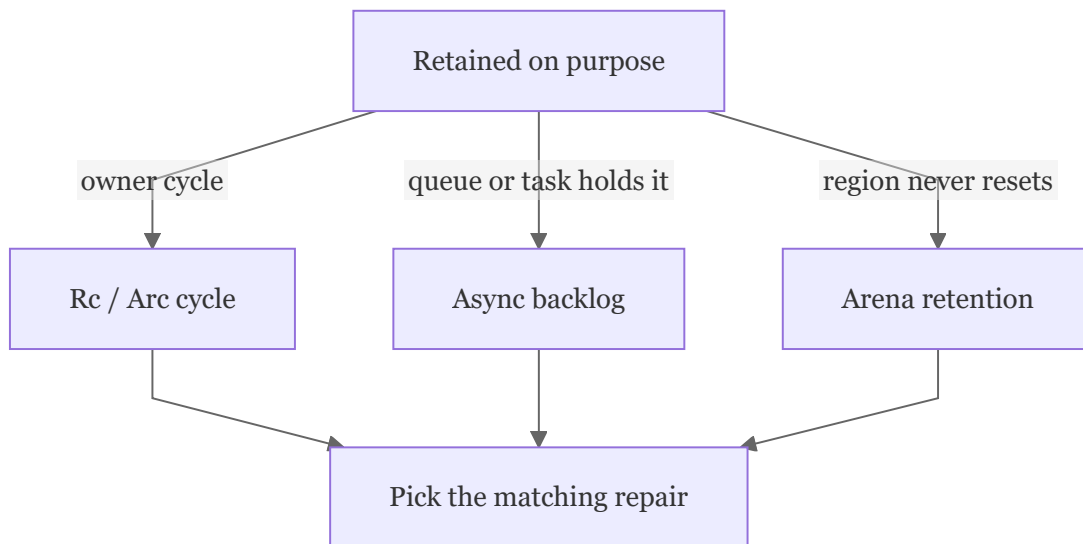
A burst load test shows the resident set climbing after a routine service release, and it does not fall back down when the burst ends. The graph alone cannot tell you what is wrong: the same upward curve is produced by allocation churn, by genuinely retained heap, by an allocator caching freed pages, by a reference-count cycle, by an arena that never resets, and by an async backlog of in-flight work. The job before touching capacity or rewriting code is to decide which of those it actually is. Guessing is expensive here, because each cause has a different repair and several of the repairs make the others worse.

The fastest way through is a fixed triage order rather than a favorite tool. Name the symptom precisely first, then measure allocation and retention together, then correlate the memory curve with the workload curve, and only then commit to a diagnosis. The flow below is the path this chapter follows; each branch maps to one of the core-concept sections that come after it.



First half: name the symptom, then split churn and fragmentation off from genuinely retained heap.

When the heap is genuinely retained, the second half decides why it is still reachable:



Second half: each retained leaf is one of the diagnoses the core-concept sections cover.

A practical triage order

1. Identify the symptom precisely: churn, growth, leak, or backlog.
2. Count allocations and retained owners at the same time.
3. Correlate memory with queue depth, task count, and work phase.
4. Only then decide whether the culprit is clone pressure, cycles, fragmentation, arena retention, or async admission policy.

Mental model

Core concepts

Measuring allocations

The cheapest signal in the whole toolbox is simply how often the program asks the allocator for memory and how many bytes it asks for. Those are two separate numbers and they describe two different problems. One hundred tiny allocations on every request is a count problem, usually solved by reusing a buffer or borrowing instead of cloning; one enormous allocation per request is a bytes problem, usually solved by streaming or trimming the payload. Measure them apart, and measure steady state apart from bursts, because a request spike often exposes churn that an average comfortably hides. Where a system has obvious batch lifetimes, per-phase or per-request counters localize the cost far better than a single process-wide total. Treat the absolute numbers as comparative: the question worth answering is not “is 4,000 allocations a lot,” it is “which boundary started allocating twice as much after this change.”

A global allocator that wraps the system allocator and bumps an atomic counter on every call is the smallest tool that answers the count question. It adds one relaxed atomic increment per allocation, so it is cheap enough to leave behind a feature flag and read whenever a regression appears.

Keep this kind of tool local and comparative. The exact count is less important than the fact that one refactor introduced twice as many allocations on the request path.

Heap profiling: who allocated, and how much is still live

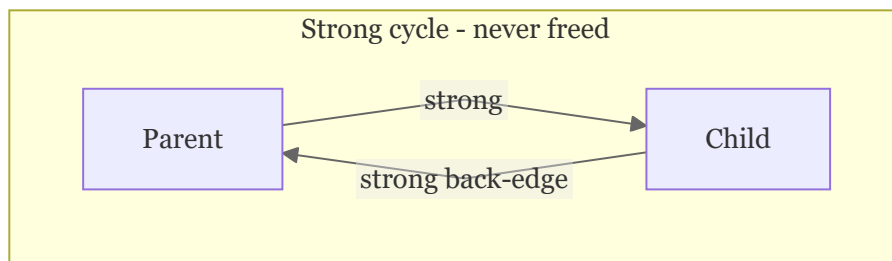
A counter tells you that allocations rose; a heap profiler tells you where they came from and whether they are still alive. The two questions a good heap profile answers are which callsite allocated and how much of what it allocated is retained right now. That second question is the one that separates a function which allocates aggressively but releases immediately from a function which allocates modestly but holds on forever. The first is fine; the second is your leak. Heap profiles are also far more useful as deltas than as snapshots: one picture of a large object graph rarely says whether growth is a one-time warmup or steady accumulation, but two pictures taken at known points in the workload almost always do.

Tracking clone pressure

In safe Rust the most common cause of allocation churn is the word `clone`, and the trap is that one verb covers two completely different operations. A deep clone of a `String` or `Vec<T>` copies the entire backing buffer and allocates fresh heap; an `Arc::clone` or `Rc::clone` copies nothing but a pointer and bumps a counter. Both show up as “a clone” in review, but only one allocates. Most clone regressions are born not in the hot loop but in API shape: a helper that takes `String` by value when it only reads forces every caller to hand over an owned copy. Instrument deep clones separately from owner clones, start with a tiny local counter on the suspect path before you reach for a full heap profiler, and review the ownership in your signatures before you optimize the body of the function.

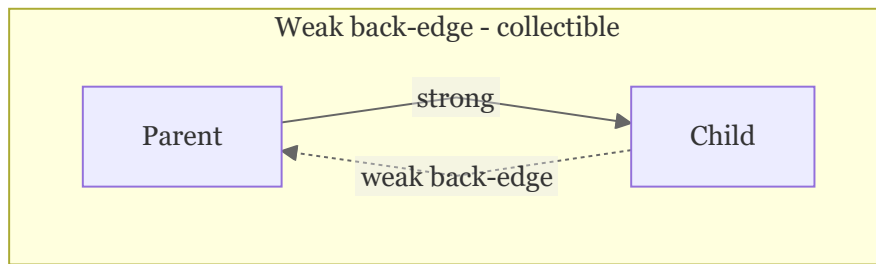
Detecting leaks with `Rc` and `Arc` cycles

Safe Rust guarantees you will never read freed memory, but it does not guarantee that memory you no longer need will be freed. The destructor for a reference-counted value runs only when its strong count reaches zero, and a cycle keeps that count permanently above zero. The classic shape is a parent owning its children with `Rc<T>` while each child also owns a strong handle back to the parent: neither end can ever drop, so the whole subgraph leaks even though nothing in your code can still reach it. The fix is to make exactly one direction of the relationship non-owning. A back-edge that exists only to observe — a parent pointer, a listener registry, a cache index — should be a `Weak<T>`, which keeps the link without keeping the value alive. The diagram contrasts the two shapes; note that even without a cycle, a registry that never removes completed entries leaks just as effectively.



A strong back-edge pins both ends forever, so neither can ever drop.

Making one direction non-owning turns the same shape into a collectible graph:



A Weak back-edge keeps the link without keeping the value alive, so the strong count can fall to zero.

Fragmentation and allocator retention

Fragmentation is the gap between what your program logically holds and what the process resident set actually shows. When you free memory, the allocator does not necessarily hand the pages back to the operating system; it usually keeps them to satisfy the next request quickly. On top of that, a heap that mixes long-lived large buffers among short-lived small ones can end up unable to reuse the holes it creates, so resident memory stays high even after the live set has shrunk. This is why the single most clarifying measurement here is live bytes plotted against RSS. If live bytes are flat while RSS keeps climbing, you are almost certainly looking at fragmentation or allocator caching, not a leak — and the repair is an allocator or layout change, not a hunt for unreachable objects. Confirm it before you call it a leak; the two diagnoses lead to opposite fixes.

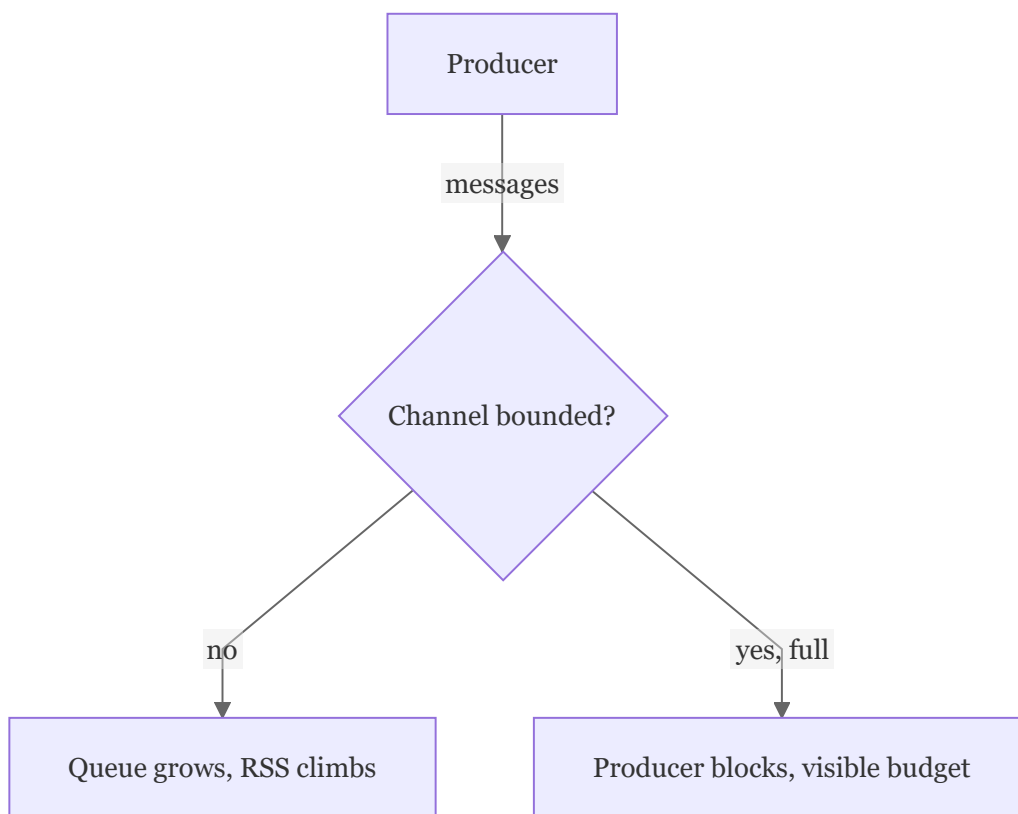
Arena allocation profiling

A bump arena trades per-object freeing for one bulk reset: allocations are nearly free and the memory is reclaimed all at once when the region resets. That is excellent for locality and churn, but it changes what you have to measure. Inside an arena, the relevant number is no longer how much is live right now but the peak region size between resets, because nothing inside is reclaimed until the whole region goes. An arena that is reset too rarely simply holds dead data until it does. The subtler failure is a lifetime mismatch: if a long-lived cache ends up holding data that the arena owns, the arena can no longer reset cleanly, and the symptom looks like an allocator problem when the real bug is that two lifetimes were wired together that should not have been. Profile peak size and reset cadence first, and treat anything that escapes the region as a design question rather than an allocator-tuning question.

Profiling async memory usage

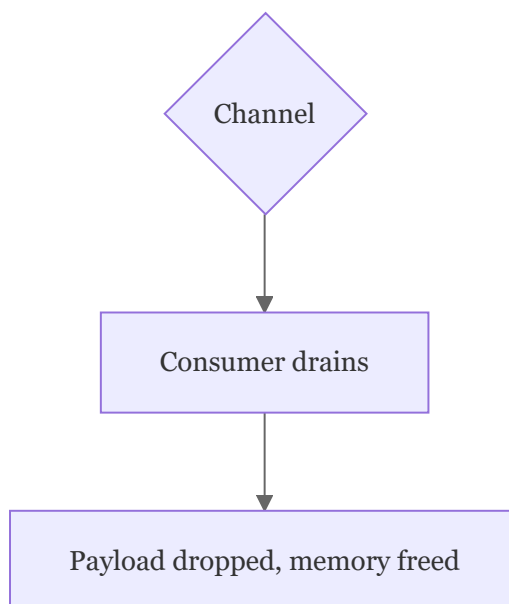
Async memory growth usually traces back to a boundedness failure rather than a leak. When producers run faster than consumers and the channel between them is unbounded, the queue absorbs the difference, and every queued message keeps its payload alive. The same thing happens when tasks are spawned without a concurrency cap: each in-flight task owns its working set, and a thousand of them own a thousand working sets at once. Four numbers explain most of these spikes — queue depth, message age, in-flight task count, and bytes per message — and the important thing is that they are read together. The diagram shows why a bounded channel turns this from an accidental backlog reservoir into a visible budget: once the queue is full, the producer blocks instead of growing memory, which surfaces the imbalance as backpressure you can see rather than as RSS you discover later.

What to look at in the sketch below: the four fields are tracked as one struct on purpose. A leak would move only one of them; an admission problem moves several at once, which is what tells the two apart.



First half: boundedness decides whether the imbalance becomes memory growth or visible backpressure.

Either way, the consumer side eventually drains and frees the payload:



Second half: draining the queue releases each message's payload, whichever policy the channel used.

If these numbers rise together, you probably have an admission problem before you have a leak problem.

Tools for Linux, macOS, and Windows

The platform tools differ in name but cluster into three jobs that map onto the signals above: growth by callsite, resident-versus-logical disagreement, and timeline correlation with the workload. Reach for the callsite tool when you need to know who allocated, the resident-versus-logical tool when live bytes and RSS disagree, and the timeline tool when a memory spike needs to be lined up against queue waits and task counts. Pick the column for your platform, but choose the row by the question you are actually asking.

Interpreting allocator behavior

Once you have the curves, reading them correctly is most of the work. The same shape means different things depending on which signals move together, and the table below is the lookup. The recurring mistake is treating any rising line as a problem: an allocator releasing memory back to its own free lists is not the operating system reclaiming RSS, and a one-time jump after startup is often just caches and lookup tables becoming resident. Interpret the combination before you change a single line of code.

Reducing memory footprint

Reducing a footprint is usually a design repair before it is an allocator trick. The largest wins come from owning less data, and owning it in fewer places: borrow on read paths instead of cloning, shrink a payload before it crosses a queue or task boundary rather than carrying the same large blob everywhere, and flatten pointer-heavy intermediate structures where locality matters. Admission control belongs in the same toolbox — channel capacity, semaphore permits, and batch size are memory controls every bit as much as throughput controls, because they cap how much work can be alive at once. And the graph repairs from earlier apply directly: a `Weak<T>` breaks a non-owning edge, and promptly removing completed entries keeps a registry from quietly becoming a cache that never evicts.

How to think about this coming from another language

Every one of these languages has a memory profiler you already trust, and the tools mostly transfer. What does not transfer is the mental model of where memory goes and what reclaims it. The shift that matters in Rust is that cleanup is tied to ownership and runs deterministically, which removes one whole class of bug and renames the rest. Read these as the assumption each background should drop at the door.

Production patterns

Pitfalls and tradeoffs

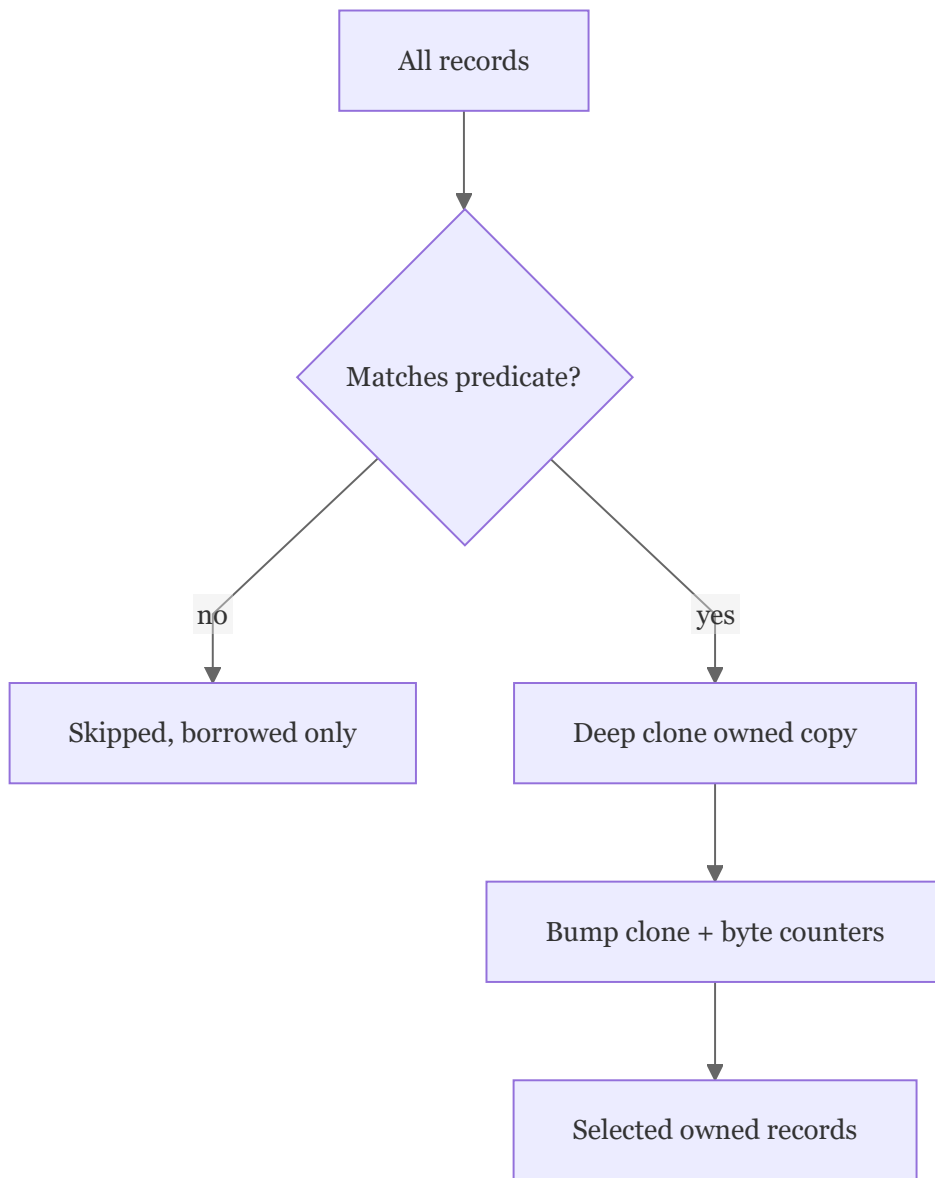
The most expensive memory optimization is often the one you make before the symptom is understood. A clone counter, a queue-depth graph, or one heap snapshot delta is often enough to avoid rewriting the wrong subsystem.

Examples

Example 1: measure clone pressure before you profile the whole heap

This local counter is intentionally simple. It helps you prove that one hot route filter is creating owned strings, how many times it does that, and how many bytes those clones account for.

What to look at: the flow is filter, then clone, then count. The filter chooses which records survive, and only the surviving records get cloned, so the clone count and cloned-byte total are a direct measure of how much owned data this path chose to duplicate. Watch the two counters move with the filter, not with the input size. The diagram is the shape the code implements.



Only records that pass the filter are cloned, so the counters measure exactly the deep-copy work this path chose.

Example 34-1. `clone_pressure_counter.rs` — Pressure counter

```

#[derive(Debug, Default)]
struct CloneStats {
    clones: usize,
    cloned_bytes: usize,
}

fn select_api_routes(routes: &[&str], stats: &mut CloneStats) -> Vec<String> {
    let mut out = Vec::with_capacity(routes.len());

    for &route in routes {

```

```

        if route.starts_with("/api/") {
            stats.clones += 1;
            stats.cloned_bytes += route.len();
            out.push(route.to_string());
        }
    }

    out
}

fn main() {
    let routes = ["/api/orders", "/health", "/api/users"];
    let mut stats = CloneStats::default();
    let selected = select_api_routes(&routes, &mut stats);

    println!("selected = {}", selected.len());
    println!("clones = {}", stats.clones);
    println!("cloned bytes = {}", stats.cloned_bytes);
}

```

OUTPUT

```

selected = 2
clones = 2
cloned bytes = 21

```

The counter answers one narrow question: how much deep-copy work did this path choose?

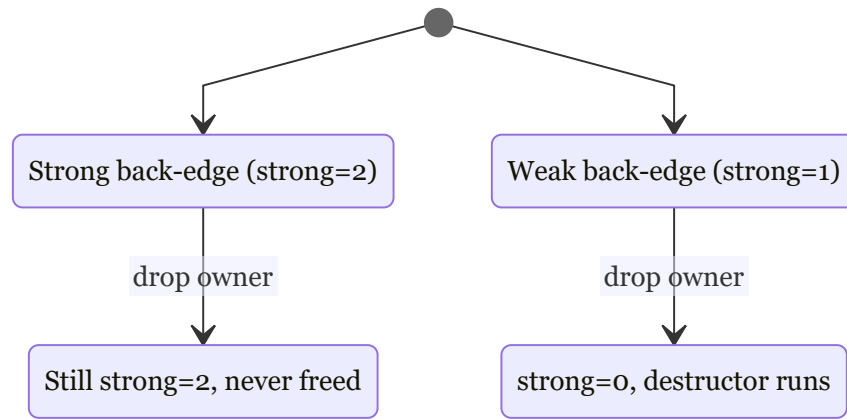
This is not a whole-process allocator trace. It is a local instrument to guide the next profiling pass.

If the result surprises you, the next review question is usually whether the output could stay borrowed longer.

Example 2: find an Rc cycle before you call it a leak

The first half creates a strong cycle. The second half repairs the parent edge with `Weak`, which turns the retained graph into an observational link instead of another owner.

What to look at: the program runs the same parent/child relationship twice and prints the parent strong count each time. In the first build the child holds a strong edge back to the parent, so the count stays at two and the subgraph can never drop — that is the leak. In the second build the back-edge is a `Weak<T>`, so the count returns to one and the parent is still reachable for reads. The number to track is the strong count, not the output text. The diagram traces that transition.



The strong back-edge pins the count at two; the Weak back-edge lets it fall to zero so cleanup can run.

Example 34-2. `rc_cycle_leak_detection.rs` — Cycle leak

```

use std::cell::RefCell;
use std::rc::{Rc, Weak};

#[derive(Debug)]
struct BadNode {
    name: &'static str,
    parent: RefCell<Option<Rc<BadNode>>>,
    children: RefCell<Vec<Rc<BadNode>>>,
}

#[derive(Debug)]
struct GoodNode {
    name: &'static str,
    parent: RefCell<Weak<GoodNode>>,
    children: RefCell<Vec<Rc<GoodNode>>>,
}

fn main() {
    let bad_root = Rc::new(BadNode {
        name: "root",
        parent: RefCell::new(None),
        children: RefCell::new(Vec::new()),
    });
    let bad_leaf = Rc::new(BadNode {
        name: "leaf",
        parent: RefCell::new(None),
        children: RefCell::new(Vec::new()),
    });

    bad_root.children.borrow_mut().push(Rc::clone(&bad_leaf));
    *bad_leaf.parent.borrow_mut() = Some(Rc::clone(&bad_root));

    let good_root = Rc::new(GoodNode {
        name: "root",
        parent: RefCell::new(Weak::new()),
        children: RefCell::new(Vec::new()),
    });
    let good_leaf = Rc::new(GoodNode {

```



```

        name: "leaf",
        parent: RefCell::new(Weak::new()),
        children: RefCell::new(Vec::new()),
    });

    good_root.children.borrow_mut().push(Rc::clone(&good_leaf));
    *good_leaf.parent.borrow_mut() = Rc::downgrade(&good_root);

    println!("bad strong = {}", Rc::strong_count(&bad_root));
    println!("good strong = {}", Rc::strong_count(&good_root));
    println!(
        "good parent = {}",
        good_leaf
            .parent
            .borrow()
            .upgrade()
            .map(|node| node.name)
            .unwrap_or("none")
    );
}

```

OUTPUT

```

bad strong = 2
good strong = 1
good parent = root

```

A child holding a strong parent edge means the root can never reach strong count zero.

`Weak` keeps reachability explicit without claiming ownership.

Before you chase allocator details, prove whether the graph is logically collectible at all.

The repository also includes standalone Rust source under `examples/ch34_memory_profiling/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor, including a small async budgeting example for local experimentation.

Summary

CHAPTER 35 · PERFORMANCE PROFILING

Performance Profiling

CPU profiling, flame graphs, sampling vs instrumentation, Criterion benchmarks, async and lock profiling, serialization and IO analysis, and WASM and FFI profiling boundaries

Performance profiling converts slow paths into verifiable bottlenecks with owners, budgets, and measurements. This chapter covers benchmarks, flame graphs, counters, and regression checks.

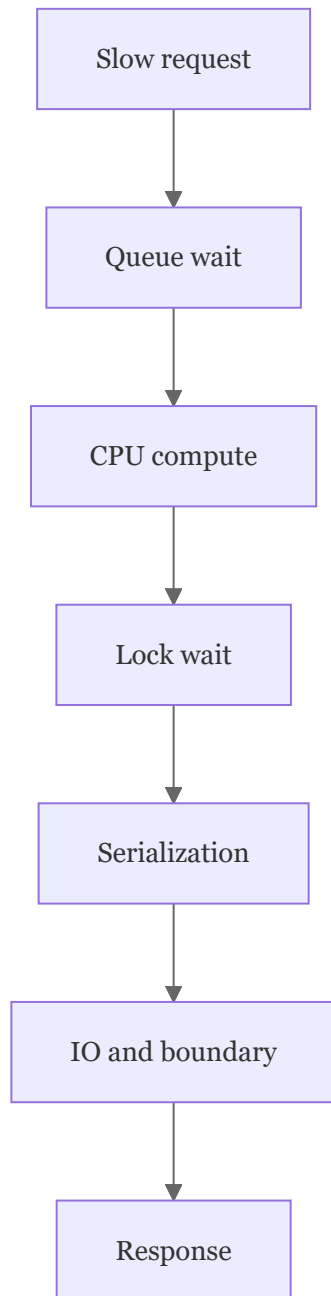
Builds on Chapters 25, 27, 28, 29, 33, and 34

Chapter 25 covered Tokio task boundaries. Chapter 27 covered IO pipelines and batching. Chapter 28 covered FFI boundaries. Chapter 29 covered WASM boundaries. Chapter 33 established Rust's broader performance model, and Chapter 34 narrowed that to memory symptoms. This chapter turns those cost surfaces into a practical profiling discipline.

Opening scenario

A release went out last night and p99 latency climbed. The on-call dashboards are not helping, because every team is reading a different signal: one engineer points at a CPU flame graph, another at growing queue depth, a third at serialization timings that look slightly worse than last week. They are all looking at fragments of the same wall-clock budget, and none of those fragments alone tells you which subsystem to change.

The job for this chapter is the discipline that resolves that argument. Before anyone rewrites a function, the wall time of a slow request has to be split into the layers it actually spends time in: CPU compute, time spent waiting in a queue, time blocked on a lock, IO, serialization, and the overhead of crossing a boundary such as FFI or WASM. Once the budget is decomposed, the owning subsystem is usually obvious, and the next code change is aimed instead of guessed.



Wall time is a chain of layers. A profile that names one number hides which link is long.

Each question below maps to a different tool. The skill is matching the question to the instrument that answers it with the least distortion.

At a glance

- Choose the profiling tool from the question: benchmark, sampler, instrumentation, trace, or live telemetry.
- Read flame graphs as stack-width evidence, not as a chronological movie.
- Separate CPU time from queue wait, lock wait, serialization, IO, and boundary overhead.
- Use each profile to narrow the next experiment instead of reaching for a rewrite immediately.

Questions to ask before you optimize

- What is the unit of work the profile is actually describing?
- Which layer owns the wall-clock budget: compute, waiting, locking, copying, or crossing a boundary?
- Which tool will answer the next question with the least distortion?
- What observable signal would prove the follow-up change mattered in production?

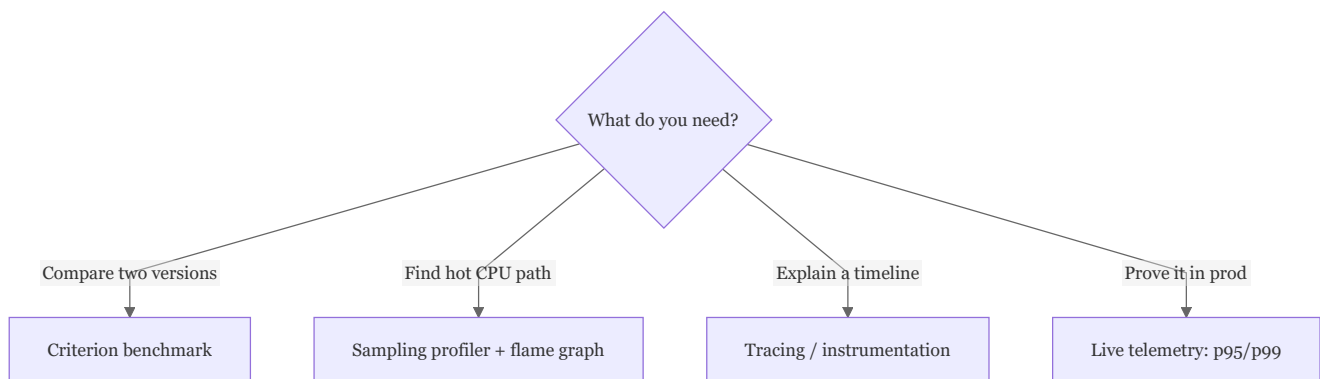
Mental model

Profiling is less about the tools than about how you frame the work. Three ideas sit underneath everything in this chapter, and they are the difference between a profile that ends an argument and one that starts a new one.

Core concepts

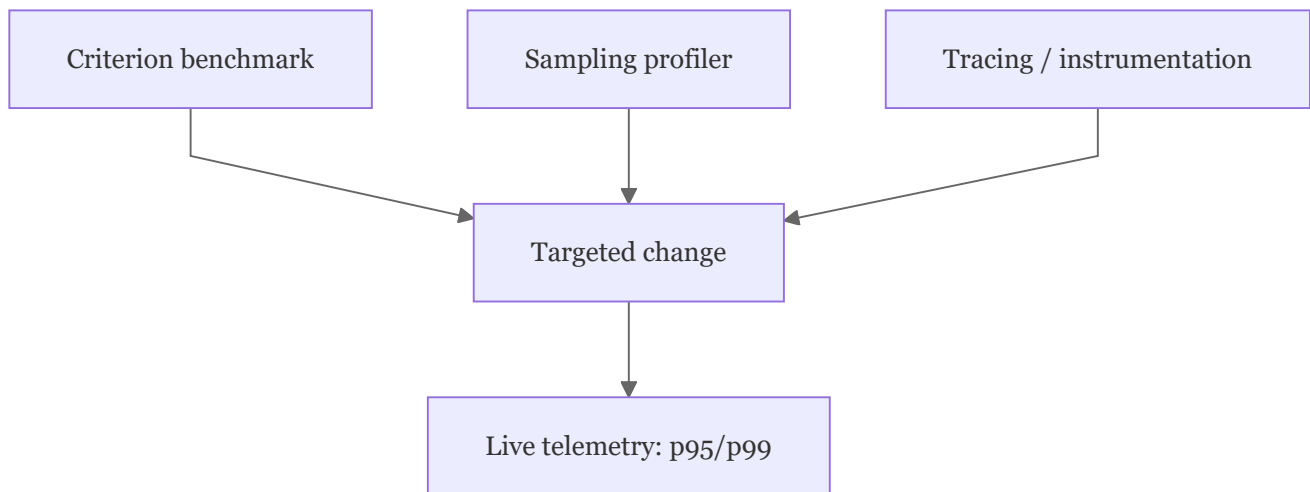
Choosing the tool from the question

The whole toolkit fans out from a single decision: what kind of answer do you need? The diagram below is the one to keep in your head. Walk it from the question, not from the tool you happen to have open. A benchmark compares two implementations under a fixed workload; a sampler finds the hot CPU path; instrumentation and tracing explain where a request spends its timeline; live telemetry confirms the fix mattered in production.



First half: the question picks the instrument. Match the answer you need to the tool before you open anything.

All four instruments feed the same loop: make a targeted change, then confirm it with live telemetry.



Second half: every tool leads to one targeted change, which is then verified in production.

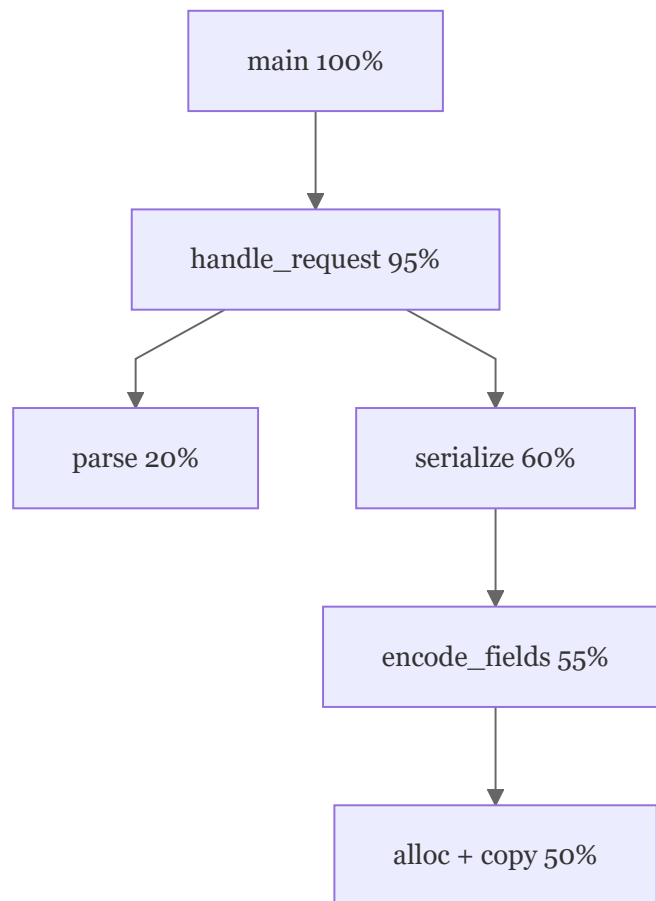
CPU profiling

A sampling CPU profiler is your first move when the symptom is "something is burning cores." It interrupts the running binary many times per second, records the call stack, and aggregates those samples into a picture of where compute time concentrates. The output is statistical, not exhaustive, which is exactly why it stays cheap enough to run on something close to a real workload. The most important habit is to separate inclusive time (a function plus everything it calls) from exclusive time (the function's own body): a caller can look enormously wide only because one child beneath it is doing all the work.

The exact tool varies by platform. The discipline does not: release build, symbols good enough for stacks, representative input, and one bounded question at a time.

Reading a flame graph

A flame graph is the standard way to display sampled CPU data, and it is misread constantly. It is not a timeline. The x-axis is not chronological; boxes are grouped by the shape of the call stack, and width is the total sampled time attributed to that stack. The stack grows upward, so a parent sits below the callees it invoked. The diagram below shows the one reading move worth memorizing: find the widest plateau, then follow it upward until the work is specific enough to change.



Width is sampled time, height is call depth. The wide leaf at the top of a wide stack is the change target.

The trap is optimizing the first named function you recognize. In the graph above, `serialize` looks like the culprit, but the real width is the allocation and copy beneath it. Tuning the serializer's control flow would barely move the number; cutting the copy would.

Sampling versus instrumentation

These are the two ways to measure a running program, and the choice is a tradeoff between overhead and precision. Sampling watches from the outside at a fixed rate, so it perturbs the program very little but only sees code that runs often enough to be caught between samples. Instrumentation places explicit timers inside the code, so it can account for a single rare event exactly, at the cost of adding work to every measured region. Reach for sampling when you do not yet know where the time goes; reach for instrumentation when you already suspect a specific boundary and want its number to the microsecond.

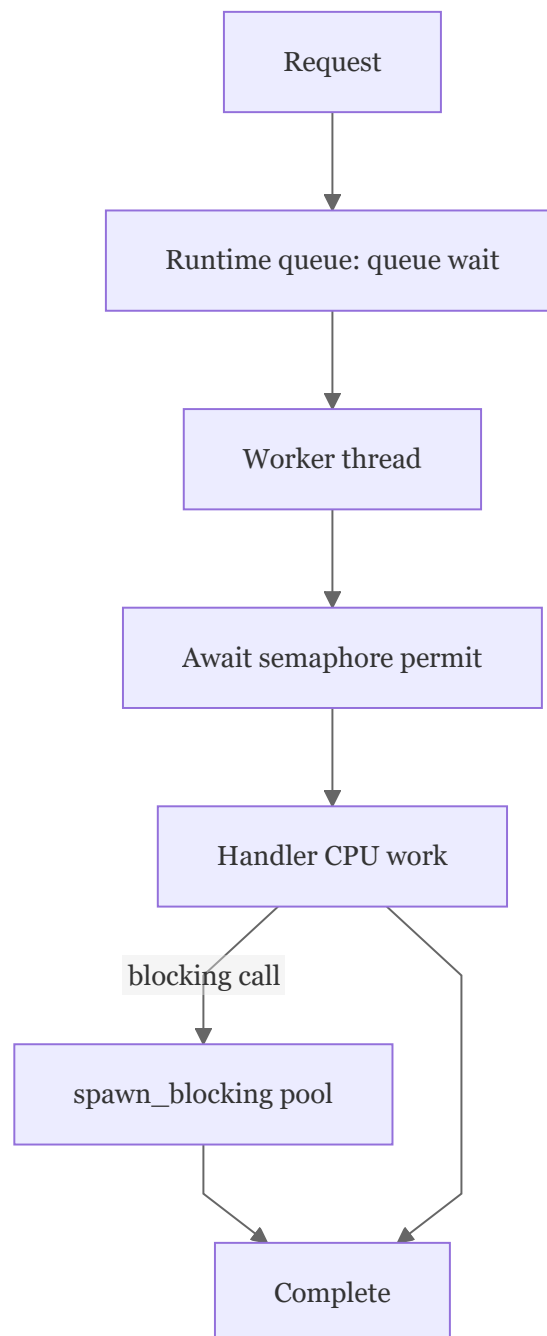
Benchmarking with Criterion

When the question is "is version B actually faster than version A?" a profiler is the wrong tool; you want a benchmark. Criterion is the ecosystem default. It runs the measured closure many times, discards warmup, and reports a distribution with confidence intervals rather than a single brittle number, which is what makes its comparisons trustworthy. The catch is that a benchmark is only as honest as its setup: it must run in release mode, on representative input, and it must defeat the optimizer's habit of deleting work whose result is never used. That last point is what `black_box` is for — it hides a value from dead-code elimination so the compiler cannot "optimize away" the very thing you are timing.

Criterion tells you which alternative won and by how much. It does not tell you why. Once you have the comparison, profile the slower variant so the result comes with an explanation instead of just a number.

Profiling async Rust

Async services break the assumption that a CPU profiler tells the whole story. A task can take 200ms of wall time while using almost no CPU, because most of that time was spent enqueued, waiting on a semaphore permit, or parked on the blocking thread pool. The diagram below traces a single request through a Tokio runtime so you can see where the clock runs without the cores running. When async "feels slow," profile each of those wait points before you profile the handler body.



Most async latency hides in the waits: queue, permit, and blocking-pool pressure, not the handler body.

An async regression usually lives in admission policy, queue wait, or CPU work left on runtime workers. The first fix is almost never "rewrite the executor." It is "measure where the task actually waits or blocks," which is exactly what tracing spans around each transition in the diagram will tell you.

Profiling lock contention

Lock cost has two halves that must be measured separately. One is the time a thread spends holding the guard and doing work; the other is the time other threads spend blocked, waiting to acquire it. They fail in opposite ways: a lock held briefly but acquired constantly is a contention problem, while a lock held for a long time by one path starves everyone regardless of frequency. Counting contended acquisitions tells you which failure you have, and that determines whether the fix is to shrink the critical section, shard the state, or move to a one-owner message-passing design.

A hot lock is not only a synchronization problem. It is often an ownership problem hiding underneath: too-broad shared state, too-long critical sections, or one subsystem missing its own owner. Reach for lower-level lock-free structures last, after the cheaper redesigns have been ruled out.

Profiling serialization

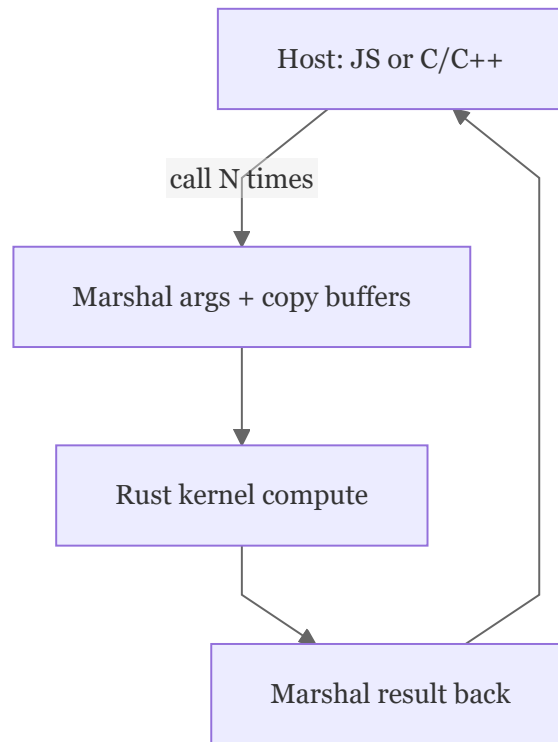
Serialization shows up on flame graphs as a wide, busy stack, and it is easy to blame the encoder when the real cost is the data itself. Measure encode and decode time alongside two numbers that explain them: allocations per message and bytes per message. A format that looks slow is often just moving too many bytes, or allocating a fresh buffer per field. It is also worth separating the serializer from the compression, hashing, or encryption frequently layered on top of it, since those can dominate while masquerading as "serialization cost." And when a queue or task boundary keeps large owned payloads alive, the retained bytes can matter as much as the encode time.

Profiling IO pipelines

IO is where flame graphs lie most convincingly. A sampler shows the CPU busy inside a parser, so the parser gets blamed, while a trace reveals the request actually spent most of its life queued before the parser ever ran. The honest measurements for an IO path are not CPU samples at all: syscalls per request, bytes per write, flush frequency, queue depth, and blocked time. Many tiny writes, partial writes, or an unbounded admission queue downstream cause more real latency than any single read loop, and none of them show up clearly in a CPU profile.

Profiling boundary crossings: WASM and FFI

WASM and FFI share one performance failure mode: the cost is rarely in the Rust kernel and almost always in the crossing. A native or WASM function can be blazing fast per call and still dominate a workload because it is called a million times, or because each call marshals a string, copies a typed array, or transfers ownership. The diagram makes the layers visible. Count boundary crossings first, then measure marshalling and copy cost, and only then look at the compute inside.



The kernel is fast. The per-call marshalling and copies, multiplied by call count, are usually the bottleneck.

The single highest-leverage experiment for both is to compare a noop call (tiny or empty payload) against the real call. The difference is your fixed per-crossing overhead, and if it is large, a batched API that crosses once will beat any amount of tuning on the callee.

How to think about this coming from another language

The tools overlap with what you already know, but the mental model shifts. The useful question is not "which Rust crate replaces my old profiler," it is "what does Rust make visible that my old runtime hid, and where does that change where I point the instrument first."

Production patterns

These are the habits that keep a profiling effort honest over time, especially across a team where the person who captured a regression is rarely the person who later has to reproduce it. The common thread is discipline about scope: one question, one build, one recorded workload.

Pitfalls and tradeoffs

Each of these is a mistake made by people who know how to use the tools. They come not from ignorance of the profiler but from trusting a single view too far, or from optimizing the thing that was easy to see rather than the thing that was actually slow.

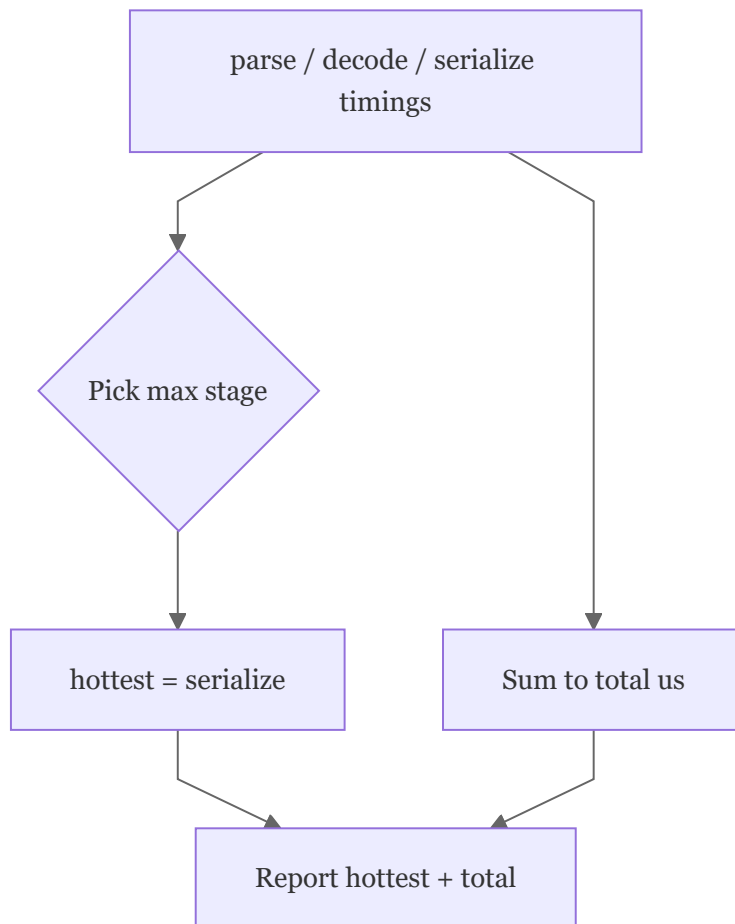
A profiling tool does not remove judgment. It only makes a narrower judgment possible. The wrong metric, the wrong build, or the wrong workload can still send a senior team straight at the wrong fix.

Examples

Both examples model the core move of this chapter in a few lines you can run: take per-stage timings, find the one that owns the most time, and report it. A real flame graph or trace is richer, but the logic of "decompose, then point at the widest layer" is identical, and seeing it as plain data makes the graphical tools easier to read.

Example 1: find the hottest stage before drawing the flame graph

What to watch: the code records a duration for each stage, then scans for the maximum. That single reduction — from a list of timings to one named winner — is exactly the question a flame graph answers visually. The diagram below shows that flow before you read the code.



Decompose into per-stage timings, reduce to the widest one, report it. That is the whole profiling loop in miniature.

Example 35-1. hot_stage_summary.rs — Stage summary

```

#[derive(Debug, Clone, Copy)]
struct StageSample {
    name: &'static str,
    micros: u64,
}

fn hottest_stage(samples: &[StageSample]) -> (&'static str, u64, u64) {
    let total: u64 = samples.iter().map(|sample| sample.micros).sum();
    let hottest = samples

```

```

        .iter()
        .max_by_key(|sample| sample.micros)
        .copied()
        .unwrap();

    (hottest.name, hottest.micros, total)
}

fn main() {
    let samples = [
        StageSample {
            name: "parse",
            micros: 180,
        },
        StageSample {
            name: "serialize",
            micros: 620,
        },
        StageSample {
            name: "lock_wait",
            micros: 40,
        },
    ];

    let (stage, hottest, total) = hottest_stage(&samples);

    println!("hottest = {}", stage);
    println!("stage us = {}", hottest);
    println!("total us = {}", total);
}

```

OUTPUT

```

hottest = serialize
stage us = 620
total us = 840

```

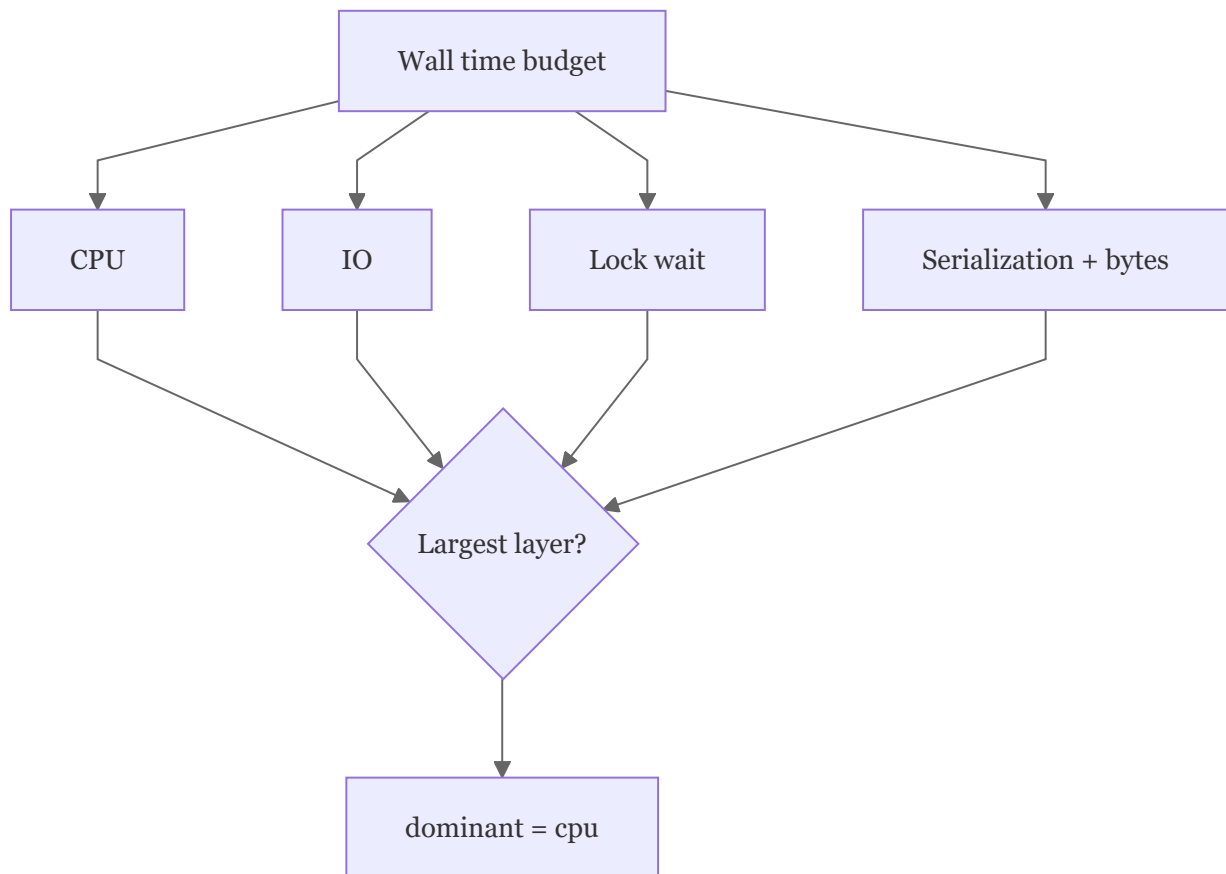
The widest stage is the first suspect because it owns the most aggregate sampled time.

If serialization is wide here, the next question is whether bytes, schema shape, or compression underneath it are the real cost.

This kind of summary often comes from instrumentation first, then a sampler confirms the call stack beneath it.

Example 2: separate CPU, IO, lock, and serialization before tuning

What to watch: the code attributes wall time to named layers, picks the dominant one, and reports the bytes that crossed the serialization boundary. The output (dominant = cpu) is the classification that decides which fix is even worth trying. The diagram shows how the layers feed that decision.



Attribute wall time to layers, then let the largest layer choose the fix. A queue tweak cannot help a CPU-bound path.

Example 35-2. pipeline_bottleneck.rs — Pipeline bottleneck

```

#[derive(Debug)]
struct PipelineStats {
    cpu_us: u64,
    io_wait_us: u64,
    lock_wait_us: u64,
    serialize_us: u64,
    serialized_bytes: usize,
}

fn dominant(stats: &PipelineStats) -> &'static str {
    [
        ("cpu", stats.cpu_us),
        ("io", stats.io_wait_us),
        ("lock", stats.lock_wait_us),
        ("serialize", stats.serialize_us),
    ]
    .into_iter()
    .max_by_key(|(_, value)| *value)
    .map(|(name, _)| name)
    .unwrap()
}

fn wall_time(stats: &PipelineStats) -> u64 {
    stats.cpu_us + stats.io_wait_us + stats.lock_wait_us + stats.serialize_us
}

```

```
}

fn main() {
    let stats = PipelineStats {
        cpu_us: 410,
        io_wait_us: 120,
        lock_wait_us: 90,
        serialize_us: 180,
        serialized_bytes: 16_384,
    };

    println!("dominant = {}", dominant(&stats));
    println!("serialized bytes = {}", stats.serialized_bytes);
    println!("wall us = {}", wall_time(&stats));
}
```

OUTPUT

```
dominant = cpu
serialized bytes = 16384
wall us = 800
```

If CPU is dominant, a queue tweak will not save the path. If lock or IO dominates, a loop micro-optimization probably will not either.

Serialization cost grows with message size, allocation shape, and follow-on compression or encryption work.

After a summary like this, benchmark the candidate fix, then profile the slower variant to explain the difference.

The repository also includes standalone Rust source under `examples/ch35_performance_profiling/` including small async and FFI boundary cost sketches alongside the two in-browser worked examples.

Summary

- CPU profiling, flame graphs, benchmarks, tracing, and production metrics answer different questions.
- Sampling finds broad hot paths. Instrumentation explains chosen boundaries such as lock wait, queue wait, serialization, and FFI or WASM calls.
- Criterion is a strong default for controlled comparisons, but it is not a replacement for profiles or live-service observability.
- Async, lock, IO, serialization, WASM, and FFI performance work all improve when wall time is decomposed before code is rewritten.
- The best profile is the one attached to one explicit workload, one explicit build, and one explicit question.

CHAPTER 36 · DISTRIBUTED TASKS PROFILING

Distributed Tasks Profiling

End-to-end latency, queue latency, worker saturation, retry storms, tail latency, distributed tracing, metrics design, task-graph profiling, and capacity planning

When work travels through brokers, leases, worker pools, and reducers, “slow” stops being a single number. This chapter is about taking the latency a user actually feels and splitting it into the layers that produced it — queue wait, worker execution, retries, fan-out, and downstream pressure — so a Rust service can point at the boundary that is really failing instead of guessing.

Builds on Chapters 25, 26, 30, 31, and 35

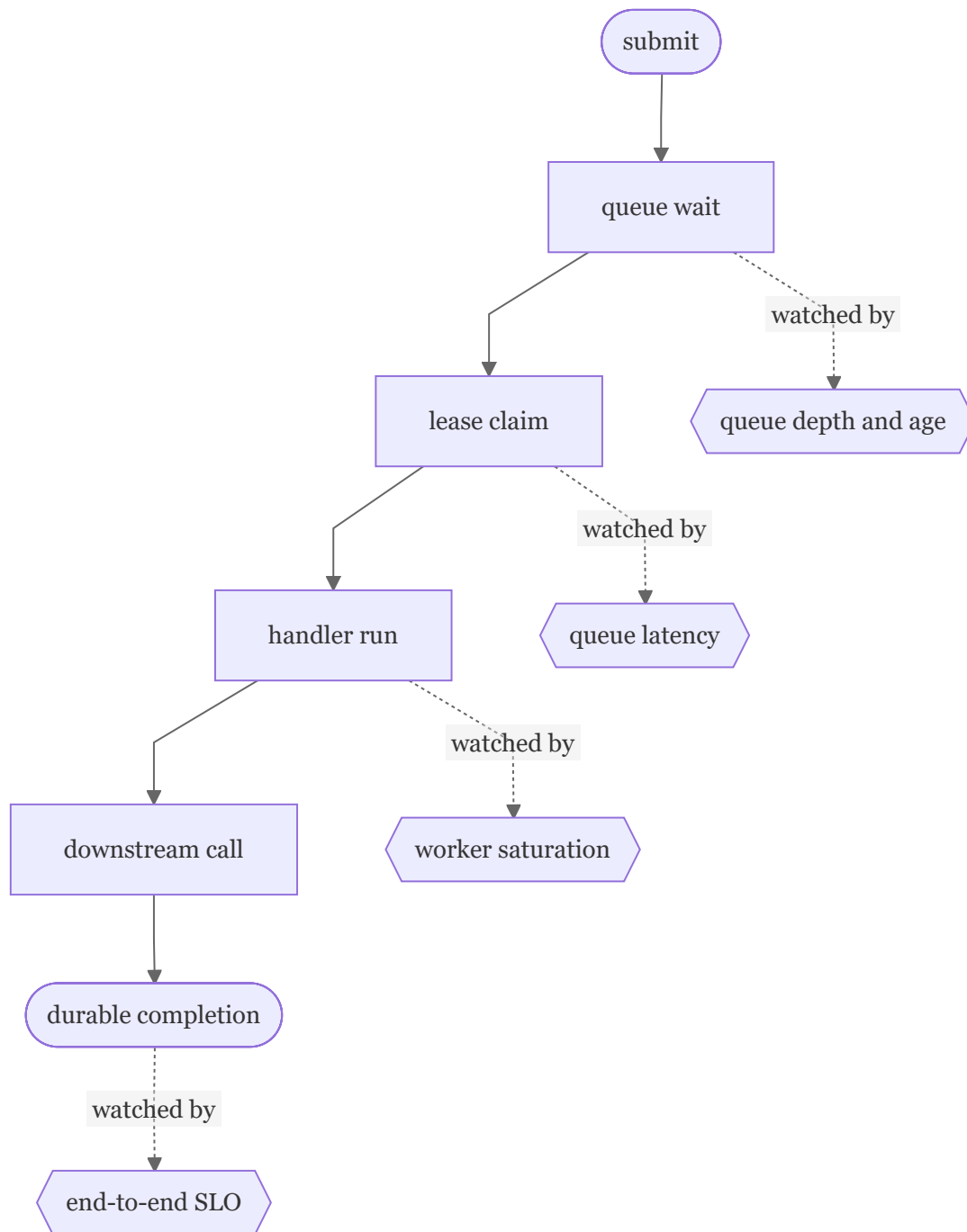
Chapter 25 covered Tokio runtime behavior, Chapter 26 covered task orchestration and backpressure, Chapter 30 covered at-least-once broker delivery, Chapter 31 covered leases and distributed task graphs, and Chapter 35 covered profiling discipline. This chapter applies those ideas to distributed worker systems end to end.

Opening scenario

A media-processing pipeline starts getting complaints that uploads take “forever” to finish, yet every dashboard the team trusts looks healthy. Handler time is flat. CPU is moderate. Error rates are normal. The handler traces show a job running in a few hundred milliseconds, exactly as designed. Despite all of that, the customer’s clock keeps running long after the handler returned, because the part the team measured is only one stretch of a much longer trip.

The business requirement is to profile that whole trip, not just the handler. A submitted job waits in a queue, gets claimed under a lease, runs in a worker, calls a downstream service, possibly retries, and only then commits a durable completion record. Each of those segments has its own latency and its own failure mode, and the user-visible delay is their sum. The job of this chapter is to make every segment measurable so the team can say which one is actually eating the budget.

Read the path below as a stopwatch with several intermediate splits. The arrows are where time accumulates; the labels underneath name the metric that watches each split. The single most common mistake is to stare at handler run while the real cost is sitting in queue wait or hiding inside a retry loop that the handler trace never shows.



The user feels the whole left-to-right path. Each metric underneath watches one segment of it, so a spike can be assigned to a boundary instead of blamed on the handler by default.

At a glance

- Profile distributed work from enqueue to durable completion, not only inside the handler body.
- Treat queue wait, retry amplification, worker saturation, and graph fan-in as first-class latency layers.
- Use critical-path thinking for DAG workloads instead of trusting stage-local averages.
- Count retries as real traffic before sizing queues, workers, or accelerators.

Operator questions to keep asking

- Is user-visible latency dominated by waiting, execution, replay, or aggregation?
- Which metric should have paged first: queue age, retry rate, or worker saturation?
- Can one trace explain the full path from submit to completion for a single task?
- Which budget is actually exhausted right now: concurrency, queue capacity, downstream dependency, or specialist hardware?

Mental model

Core concepts

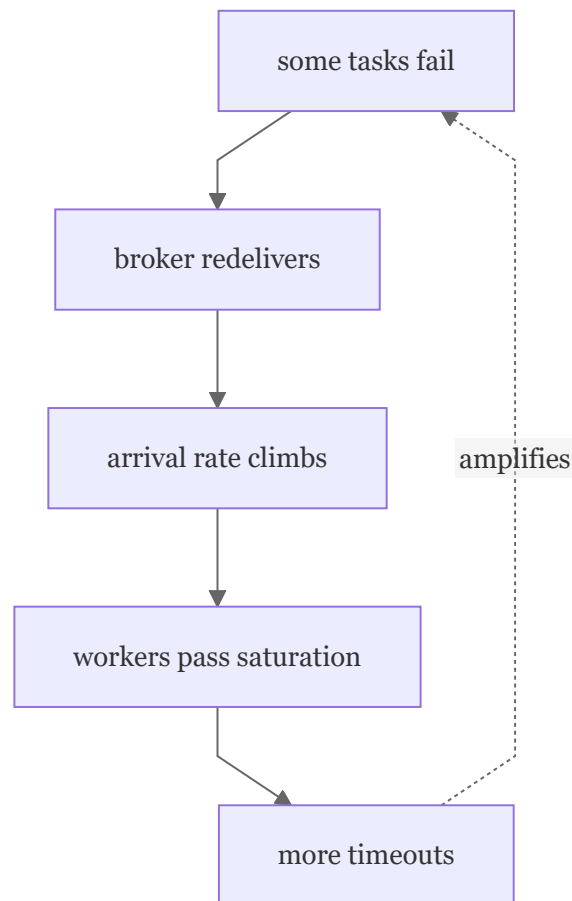
End-to-end latency, queue latency, and worker saturation

These three measurements are the backbone of everything else in the chapter, and they answer different questions. End-to-end latency tells you what the user feels. Queue latency tells you how long work waited before anyone touched it. Worker saturation tells you whether there is any headroom left to absorb the next burst. Read in isolation, each one can lie; read together, they triangulate the bottleneck.

The practical rule is short enough to keep on a sticky note: if end-to-end latency rises while handler run time stays flat, the time is being spent before the handler, not inside it. The first question is queueing or admission — backlog, lease contention, a closed pool — not inner-loop CPU. Reaching for a flame graph at that moment optimizes the one part of the system that was never slow.

Retry storms and tail latency

A retry storm is a feedback loop, and that loop is what makes it dangerous. Some tasks start failing, the broker redelivers them, the redeliveries add to arrival rate, the higher arrival rate pushes workers past saturation, saturation causes more timeouts, and the timeouts produce more retries. The diagram below traces that cycle. Notice that the dashed edge feeds back into the top: nothing breaks the loop on its own, so it compounds until a budget or a circuit breaker cuts it.



A retry storm is a self-reinforcing cycle. Each lap raises arrival rate faster than workers can drain it, so retries are load before they are diagnostics.

The good news is that a storm announces itself before it becomes catastrophic. Retry rate rises, queue age rises, worker saturation rises, and the oldest visible message drifts far above the median — usually in that order. If those four signals share a dashboard, you get minutes of warning instead of a page after the queue has already buried you.

Distributed tracing

In a single process, a tracing library can stitch spans together automatically because they share a call stack. The moment work crosses a broker, that stack is gone — the worker that picks up a message has no idea where it came from unless the message itself carries the context. So distributed tracing here is mostly a discipline about what you put in the task envelope. Every message has to carry enough identity to rejoin its trace on the other side.

The envelope below is the minimum that makes correlation possible. The two fields most teams forget are the ones that matter most under load: `attempt` lets you separate the first try from the third when a storm hits, and `parent_task_id` is what lets a fan-out graph reassemble itself into a tree instead of a pile of unrelated spans.

Metrics design

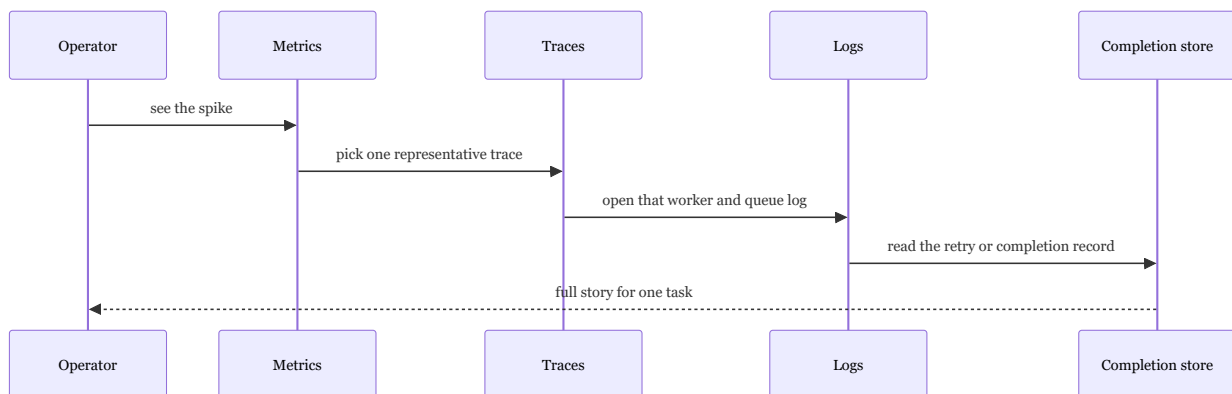
Three shapes of metric answer three different kinds of question, and choosing the wrong shape is a common way to end up with dashboards that cannot answer the question you actually have. Counters answer “how much of this happened?”; Gauges answer “what is the level right now?”;

Histograms answer “what does the distribution look like, especially the tail?” The fourth card is the one that keeps the system affordable: keep identity out of labels, because a counter with a per-task label is no longer a metric — it is a log line that your monitoring system cannot store.

Correlating logs, metrics, and traces

Metrics, traces, and logs are three views of the same incident, and they are only useful together if a single identity threads through all three. The checklist below is really one idea repeated: pick a small set of fields and a shared timestamp vocabulary, then carry them everywhere so an operator can walk from an aggregate spike down to one concrete record without re-deriving the link by hand.

That correlation buys you a calm, repeatable order of investigation under pressure. The point of the sequence below is the direction of travel: you start from the aggregate that paged you and narrow toward a single record, instead of reading four tools in parallel and trying to merge them in your head.

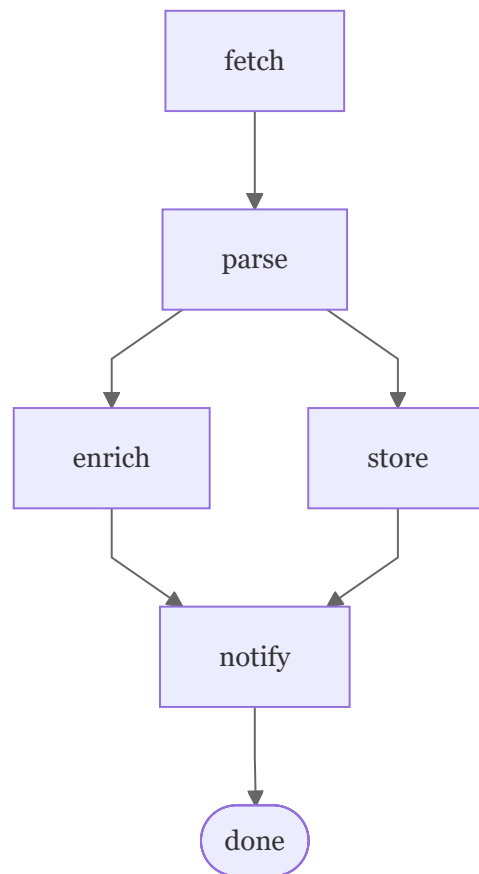


The calm incident loop: aggregate spike first, one trace second, worker and queue logs third, one durable record last. Identity fields are what make each hop possible.

Profiling task graphs

When a job is a DAG rather than a single handler, the intuition that “total time is the sum of the stages” quietly becomes wrong. Stages that run in parallel overlap, so adding their durations double-counts time the system never spent. The number that predicts completion is the critical path: the longest dependency chain from start to finish. Everything off that path has slack and can get slower without anyone noticing — until it gets slow enough to become the new critical path.

Look at the graph below. After parse, the work forks into enrich and store, which run at the same time, and notify waits for both. The completion time is governed by whichever fork is slower, not by their sum. That is exactly what the formula under the diagram computes: the max over the two competing chains, not the total of every box.



enrich and store run in parallel after parse, and notify joins them. The critical path is the slower of the two branches plus the shared head and tail, not the sum of all four stages.

Capacity planning

Capacity planning for a worker fleet rests on two old results that are worth keeping in your head. The first is Little’s Law: the average number of items in flight equals arrival rate times the time each item spends in the system. The second is the queueing cliff: as utilization approaches one, waiting time does not rise gently — it heads toward infinity. That is why the formulas below divide by a target utilization below one rather than sizing for exactly the average load. You are buying the headroom that keeps queue age finite when a burst or a retry wave arrives.

The two lines below are deliberately rough planning rules, not a simulator. The first sizes a pool from how fast work arrives and how long each task runs; the second estimates how much work is in flight at any moment. The trap to avoid is plugging in happy-path numbers — arrival rate has to include retry traffic, or the pool you size will be the one that triggers the storm it cannot survive.

How this looks coming from another language

Most senior engineers arrive with a profiling reflex that worked well on their previous stack and quietly misleads them here. The shift is the same in every case: the expensive thing has moved out of one process and into the spaces between processes — queues, leases, retries, and joins — where your old tools cannot see it. These cards name the specific reframes each background needs, not a table of crate equivalents.

Production patterns

Pitfalls and tradeoffs

The fastest way to misprofile a distributed worker system is to measure only handler CPU and call the rest “overhead.” Queue age, retries, fan-in, and replay often are the system.

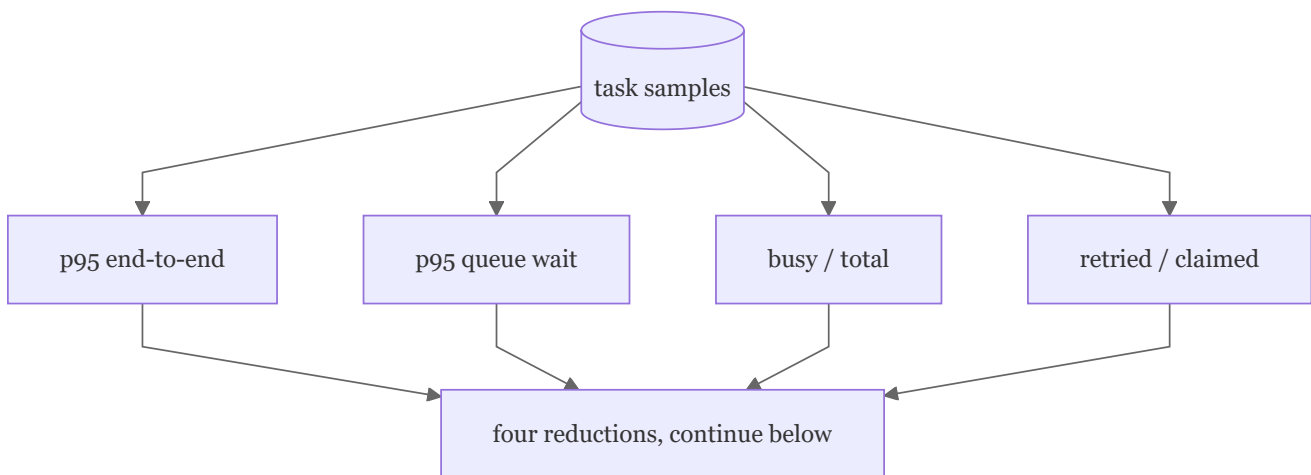
Examples

Both examples are deliberately small enough to read in one sitting and to run in the editor below them. The first builds the four-signal window an operator should see at a glance; the second computes a critical path through a DAG. Run each one to confirm the baseline output, then change an input and watch which number moves. The goal is to feel how the metric reacts, not to memorize the arithmetic.

Example 1: end-to-end latency, queue latency, saturation, and retry rate

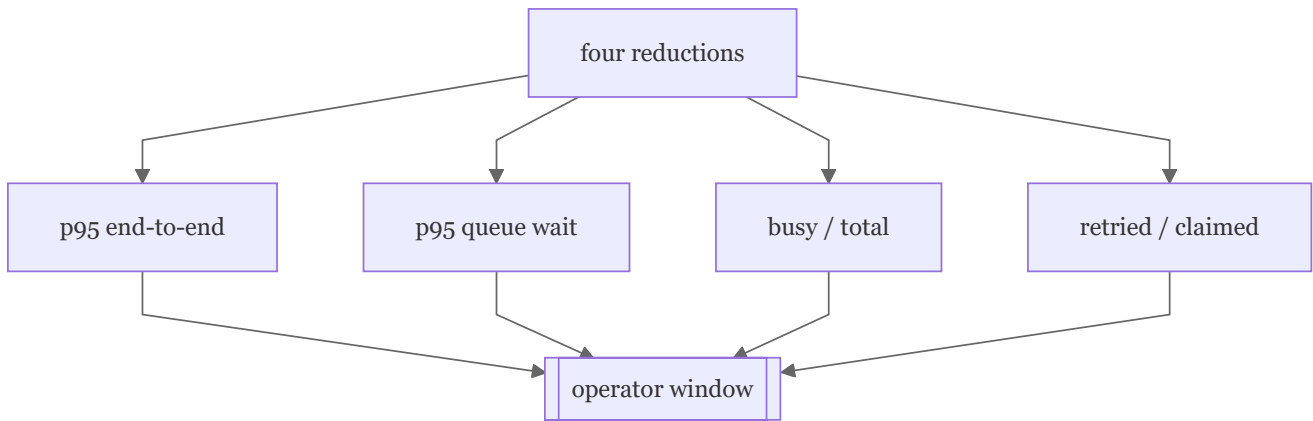
The point is not one magic metric. The point is that queue wait, run time, and retry pressure belong in one window summary before the incident page fills with guesses.

What to look at: the code takes one batch of completed-task samples and folds it down into four headline numbers. Follow the data flow in the diagram first — raw samples in on the left, four independent reductions in the middle (percentiles for the latencies, a ratio for saturation, a rate for retries), and the compact window an operator reads on the right. The key idea is that all four come out of the same sample window, which is what lets you compare them honestly.



Fan-out: one sample window splits into four independent reductions — percentiles for the latencies, a ratio for saturation, a rate for retries.

The four reductions then converge back into a single operator window:



Fan-in: the four numbers fold back into one summary. Read together, they say whether the cost is waiting, running, capacity, or replay.

Example 36-1. latency_window_and_saturation.rs — Latency window

```

#[derive(Debug, Clone, Copy)]
struct WindowStats {
    claimed: u32,
    completed: u32,
    retried: u32,
    queue_p95_ms: u64,
    run_p95_ms: u64,
    busy_workers: u32,
    total_workers: u32,
}

fn end_to_end_p95_ms(stats: &WindowStats) -> u64 {
    stats.queue_p95_ms + stats.run_p95_ms
}

fn worker_saturation(stats: &WindowStats) -> f64 {
    stats.busy_workers as f64 / stats.total_workers as f64
}

fn retry_rate(stats: &WindowStats) -> f64 {
    stats.retried as f64 / stats.claimed as f64
}

fn main() {
    let stats = WindowStats {
        claimed: 120,
        completed: 110,
        retried: 18,
        queue_p95_ms: 140,
        run_p95_ms: 320,
        busy_workers: 17,
        total_workers: 20,
    };

    println!("e2e p95 = {}", end_to_end_p95_ms(&stats));
    println!("queue p95 = {}", stats.queue_p95_ms);
  
```

```
println!("worker saturation = {:.2}", worker_saturation(&stats));  
println!("retry rate = {:.2}", retry_rate(&stats));  
}
```

OUTPUT

```
e2e p95 = 460  
queue p95 = 140  
worker saturation = 0.85  
retry rate = 0.15
```

Queue wait plus run time gives a far better first diagnostic than handler time alone.

The queue p95 is often the first overload signal visible to operators.

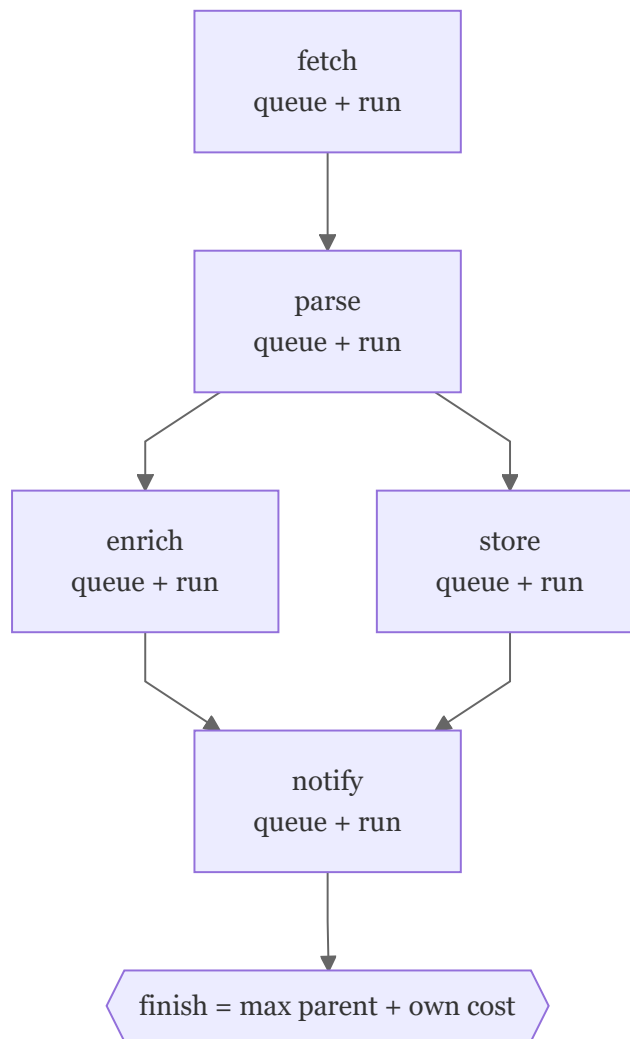
Busy workers divided by total workers is simple, but it becomes useful only when read beside queue age.

Retry rate belongs in the same window because retries are load, not only failure bookkeeping.

Example 2: profile a task graph by critical path

One graph stage can be locally fast and still sit on the longest path after queue time and dependency lag are both counted.

What to look at: this is the graph from the concepts section above, but now each stage carries two costs instead of one — the time it waited in its queue plus the time it actually ran. The algorithm walks the DAG and, for every node, keeps the longest finishing time among its parents and adds the node’s own queue and run cost. The diagram shows that per-stage accumulation. The answer is the largest finishing time at the end, and the stage that set it is the tail stage to investigate first.



Each stage contributes queue wait plus run time. Walking the DAG and keeping the max parent finish at every join yields the critical path and names the tail stage.

Example 36-2. task_graph_critical_path.rs — Task graph

```

type StageId = usize;

#[derive(Debug)]
struct Stage {
    name: &'static str,
    deps: Vec<StageId>,
    queue_ms: u64,
    run_ms: u64,
}

fn critical_path(stages: &[Stage]) -> (u64, &'static str) {
    let mut totals = vec![0_u64; stages.len()];
    let mut best = 0_u64;
    let mut tail = "none";

    for (id, stage) in stages.iter().enumerate() {
        let upstream = stage
            .deps
            .iter()
            .map(|&dep| totals[dep])
    
```

```

        .max()
        .unwrap_or(0);

    totals[id] = upstream + stage.queue_ms + stage.run_ms;

    if totals[id] >= best {
        best = totals[id];
        tail = stage.name;
    }
}

(best, tail)
}

fn main() {
    let stages = vec![
        Stage {
            name: "fetch",
            deps: vec![],
            queue_ms: 20,
            run_ms: 70,
        },
        Stage {
            name: "parse",
            deps: vec![0],
            queue_ms: 30,
            run_ms: 90,
        },
        Stage {
            name: "enrich",
            deps: vec![1],
            queue_ms: 40,
            run_ms: 120,
        },
        Stage {
            name: "store",
            deps: vec![1],
            queue_ms: 10,
            run_ms: 60,
        },
        Stage {
            name: "notify",
            deps: vec![2, 3],
            queue_ms: 15,
            run_ms: 30,
        },
    ];

    let (critical_path_ms, tail_stage) = critical_path(&stages);
    let total_queued: u64 = stages.iter().map(|stage| stage.queue_ms).sum();

    println!("critical path ms = {}", critical_path_ms);
    println!("tail stage = {}", tail_stage);
    println!("queued ms = {}", total_queued);
}

```


OUTPUT

```
critical path ms = 415  
tail stage = notify  
queued ms = 115
```

The longest dependency chain is the first scheduling story to explain, not the mean stage duration.

A join or notification stage often becomes the visible tail even when earlier stages look bigger on paper.

Summed queue cost gives one quick signal for whether the graph is compute-bound or backlog-bound.

The repository also includes standalone Rust source under `examples/ch36_distributed_tasks_profiling/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

- Profile distributed work from enqueue to durable completion, not only inside the handler body.
- Queue latency, worker saturation, retry rate, and oldest visible age usually explain incidents faster than mean handler time.
- Tracing should include queue wait and graph lineage, not only one flat request span.
- Good metrics design separates low-cardinality aggregate signals from high-cardinality identity that belongs in logs and traces.
- Capacity planning should include retry traffic, reducer bottlenecks, and drain-time goals, not only happy-path throughput.

CHAPTER 37 · CUDA AND GPU ACCELERATION

CUDA and GPU Acceleration

When GPU acceleration pays, safe launch wrappers, device-memory ownership, transfer and launch overhead, profiling, and async or distributed integration

A GPU is a throughput machine guarded by an expensive boundary. The kernel does not run until you have paid to move data across the bus, configured a launch, and synchronized the result back. This chapter is about owning that boundary from Rust honestly: when the accelerator actually pays for itself, how to wrap a kernel launch so its invariants are checked instead of assumed, and how to run a GPU as a bounded resource inside a real service.

Builds on Chapters 28, 32, 33, 35, and 36

Chapter 28 gave us FFI boundary discipline. Chapter 32 covered flat numeric layout and HPC thinking. Chapter 33 explained Rust's broader performance model. Chapter 35 covered profiling, and Chapter 36 covered queue wait, saturation, and distributed task profiling. CUDA work sits across all five.

Opening scenario

A batch-scoring service already runs on the CPU and meets its latency target most of the time. Under load it starts to fall behind, and someone proposes adding a GPU-backed execution lane beside the existing CPU path. The card is approved on the assumption that “the GPU is faster.” That assumption is the thing this chapter refuses to take on faith.

The real requirement is to prove the accelerator boundary pays for itself before it ships. That means measuring the whole round trip, not just the kernel: how long requests wait in the queue, how many bytes cross to the device and back, what each launch costs, how long the kernel actually runs, where the host blocks on a synchronize, and what happens when the device is busy or out of memory. A kernel that is individually fast can still produce a feature that is slower and less reliable than the CPU path it replaced. The Rust code in this chapter is built to make every one of those costs visible instead of hidden behind a convenient wrapper.

At a glance

- Offload only when dense, regular work can amortize transfer and launch cost.
- Count bytes, launches, and synchronization points before tuning arithmetic.
- Keep device-memory ownership explicit and the unsafe launch wrapper as small as possible.
- Treat the GPU like a bounded specialist worker inside the larger Rust system.

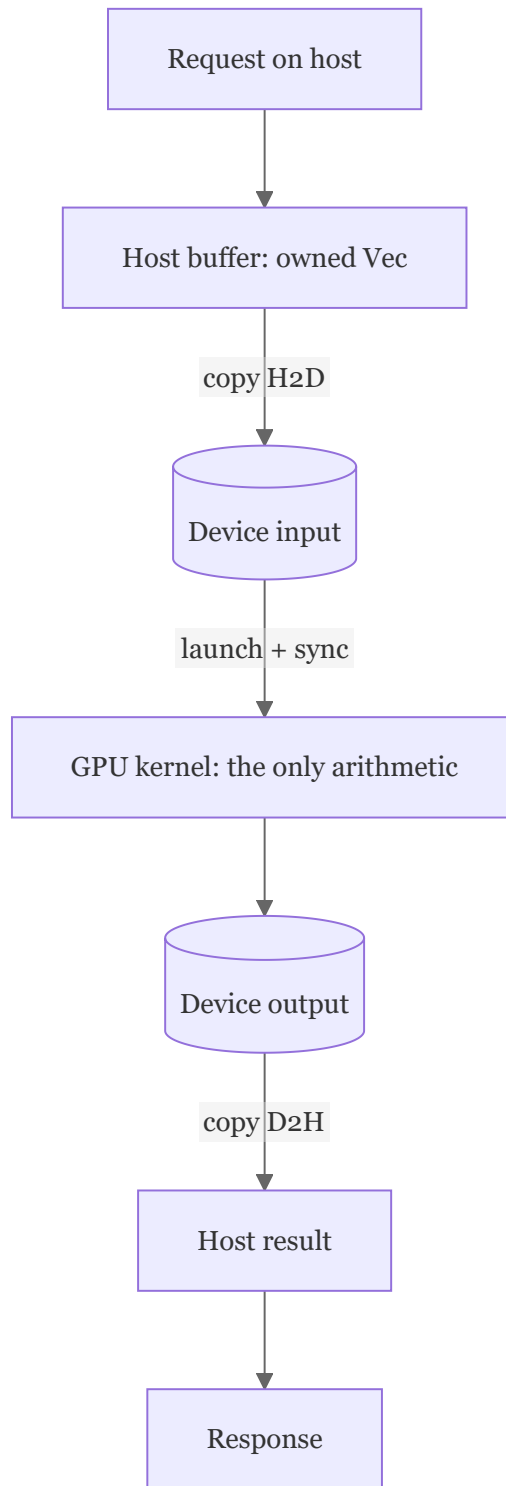
Design questions

- Would a CPU or Rayon path still win for this batch size and latency target?
- Where does the first owned boundary appear: API edge, queue lane, or device wrapper?
- What must be validated before the raw launch becomes legal?

- Which metric will tell you the path is transfer-bound, launch-bound, queue-bound, or actually kernel-bound?

The shape of a GPU offload

Before any of the details, fix the overall shape in your head. A request arrives on the host, where ordinary Rust owns the data. To use the GPU you copy that data across the bus into device memory, launch a kernel, wait for it, then copy the result back. Each arrow in the picture below is a cost the kernel itself does not pay for: the host-to-device copy, the launch, the synchronize, and the device-to-host copy. The kernel is the one box in the middle that does arithmetic; everything around it is boundary tax. When people say the GPU “won”; they usually mean the middle box shrank, while the surrounding arrows are what actually decide whether the feature is faster end to end.



The kernel is one box; the copies, the launch, and the synchronize are the boundary tax around it.

Mental model

Three ideas carry most of the weight in this chapter. The GPU is a throughput device, not a latency device, so it only repays you when the work is big and regular enough to amortize that boundary. Rust's job is rarely to write the kernel; it is to keep the host side of the boundary honest, so ownership, lengths, and lifetimes are stated before any unsafe launch runs. And the decision to accelerate is a systems decision, not a math one: a perfect kernel sitting behind bad queueing or too many tiny launches still produces a slow service.

Core concepts

When GPU acceleration makes sense

The single number that predicts whether a GPU helps is arithmetic intensity: how much math you do per byte you move across the bus. A workload that does a great deal of computation on a small amount of data can hide the transfer cost behind the kernel and win comfortably. A workload that touches each byte once and then ships it back spends most of its time on the bus, and the GPU rarely beats a tuned CPU loop there. The cards below are really four facets of the same question: is there enough regular, dense, batched work to make the boundary worth crossing?

CUDA is one option on a spectrum, not the default destination for anything slow. Before reaching for the device, place the work on this ladder of escalating mechanisms. Most service code never needs to climb past the first two rungs, and the right answer is frequently “a CPU path that does the same logical work, measured honestly”; rather than any accelerator at all.

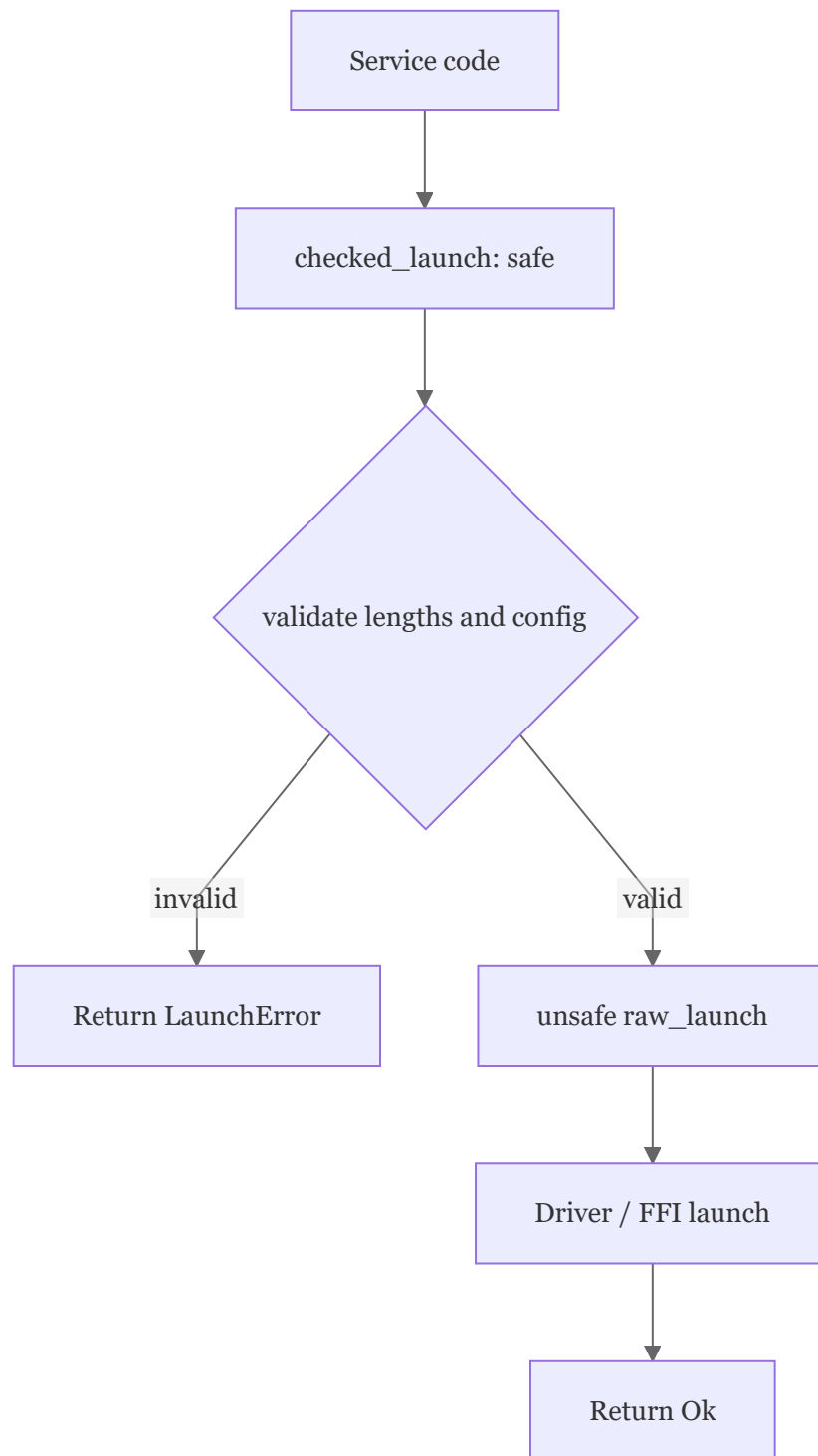
How Rust reaches the GPU

Rust does not have a single blessed CUDA story, and that is fine, because the choice is mostly about where your team wants the boundary to sit rather than which crate is fashionable. There are three broad shapes, and they differ in how much of the device you are choosing to own directly.

The key decision is less about one specific crate and more about boundary shape. If your team already owns CUDA kernels in native code, FFI may be the calmest first move. If Rust should also own the host-side driver boundary, a wrapper crate can reduce boilerplate. If the product already lives behind a tensor or inference runtime, the right abstraction may be even higher.

Calling CUDA kernels from Rust

Whatever FFI or wrapper layer you pick, the safe call shape is the same and it is worth memorizing. There is one public, safe function the rest of the service calls, and one tiny private unsafe function that does the actual launch. The safe function’s only job is to check the invariants the unsafe launch depends on — buffer lengths, shapes, launch configuration — and only then enter the unsafe region. Look at how `checked_launch` calls `validate_lengths` first and treats a validation failure as a normal `Result` error: the unsafe block is never reached unless the preconditions already hold. That gate is the whole pattern.

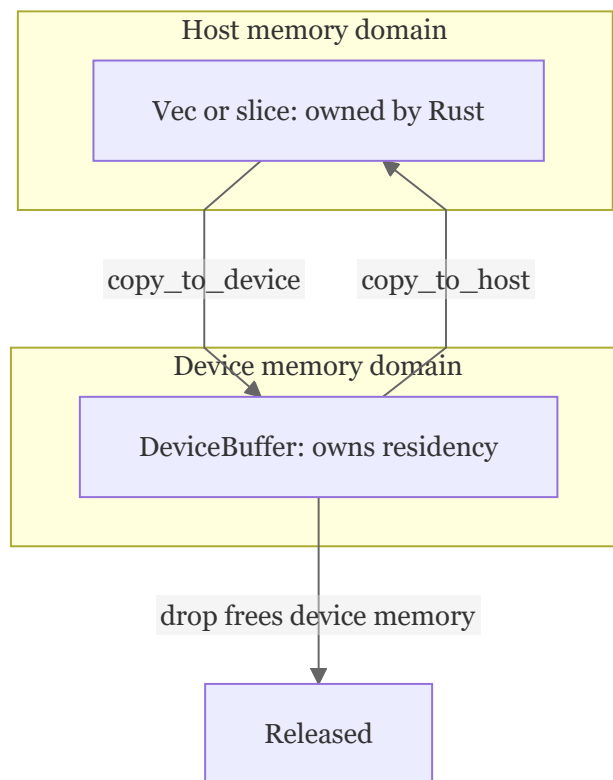


One safe gate validates every precondition; the unsafe launch is only reachable once the checks pass.

This is one of the cleanest places for Rust to earn its keep. In C++ host code the buffer shape and launch invariants usually live in convention and review comments, so a mismatched length is a silent corruption waiting to happen. The Rust version makes those checks routine and impossible to skip, without ever pretending the device call itself is safe by default — the `unsafe` keyword still marks exactly where the responsibility shifts from the compiler to you.

GPU memory ownership

The most common mental-model mistake is to treat device memory as if it were just host memory with a faster processor attached. It is not. Host memory and device memory are two separate domains with separate allocators, and a pointer into one is meaningless in the other. A `Vec<T>` on the host describes a host layout and nothing more; getting that data to the device is an explicit copy that allocates a separate device buffer with its own lifetime. Model that device buffer as its own owner handle — something that allocates on construction and frees on drop — so the residency on the card is tied to a Rust value the host controls. The diagram shows the two domains and the only legal way to move between them.



Two separate domains. A host owner and a device owner; copies cross between them and drop frees the device side.

Host-device transfer costs and kernel launch overhead

Two fixed costs sit between you and the kernel, and both are easy to forget because neither shows up in a kernel-only profile. The first is the copy across the bus, paid in both directions: every input you move to the device and every result you move back is bytes on the wire. For small vector-shaped operations this copy alone can dwarf the kernel. The second is the launch itself, which carries real per-call overhead in the driver. A service that launches one tiny kernel per request often loses to a plain CPU loop even when each individual launch is efficient, simply because it pays the launch tax hundreds of times. The way out is usually fewer, bigger launches and keeping intermediate data resident on the device instead of round-tripping it.

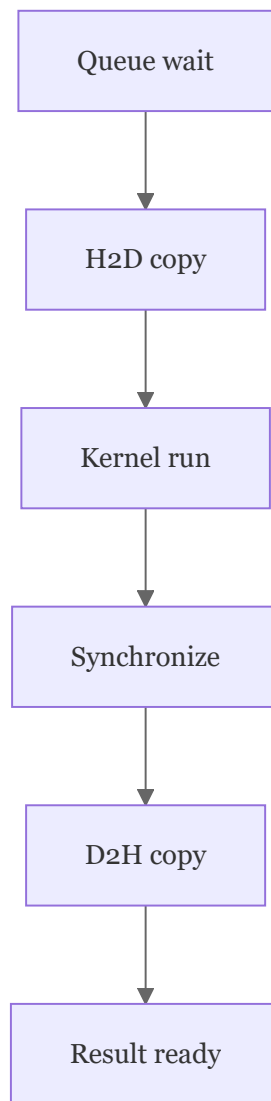
The fastest wrong architecture is often a service that moves small tensors to the GPU, launches one tiny kernel, synchronizes immediately, and moves the result back. Measure the whole round trip before you call the kernel a win.

Matrix and vector workloads

The classic GPU demo is a vector add, and it is a misleading one. Vector add reads two arrays, adds them, and writes a third: one arithmetic operation per several bytes moved, which makes it memory-bandwidth-limited and acutely transfer-sensitive. It is a fine way to learn the boundary, but a poor argument that offloading pays. Dense matrix multiply is the opposite case. Multiplying two N-by-N matrices moves on the order of N-squared bytes but does on the order of N-cubed arithmetic, so the math grows faster than the data and the transfer cost gets buried under real work. That is why GEMM, batched GEMM, convolutions, and large reductions are where the GPU earns its reputation, while tiny elementwise transforms often do not.

Profiling CUDA workloads

The point of profiling here is to refuse to call the whole feature “GPU work” and instead break the wall-clock time into named stages. A request spends time waiting in the queue, copying host-to-device, launching and running the kernel, synchronizing, and copying device-to-host. Each stage points at a different fix: a queue-bound path needs admission and capacity changes, a transfer-bound path needs bigger batches or pinned memory, a launch-bound path needs fewer launches, and only a kernel-bound path is improved by tuning occupancy or registers. The timeline below is the question every profile should answer.



Attribute wall-clock time to each stage. The widest stage, not the kernel, names your real bottleneck.

A practical first question is simple: are you transfer-bound, launch-bound, queue-bound, or actually kernel-bound? A kernel-only profile cannot answer that by itself.

Rust abstractions over GPU code

Good GPU code tends to layer into three levels, and the most common design mistake is exposing the wrong one to the rest of the system. At the bottom is a narrow safe wrapper around a single launch. In the middle are the handles that make reuse and admission explicit — a device buffer, a stream, a job queue. At the top is a domain API. The rest of your service should almost never see `launch_kernel`; it should see `score_batch` or `run_inference`. Kernel names are an implementation detail, and leaking them upward couples your business logic to the device.

Integrating CUDA with async and distributed systems

Once a GPU lives inside an async service, treat it the way you would treat any scarce shared resource: as one worker behind a bounded queue, not as an open destination for fan-out. Async tasks can generate work far faster than a single device can absorb it, so the queue is where you decide who waits, when work spills to a CPU fallback, and when overload becomes visible to operators. Two further rules keep the integration

honest. Own your input data before you hand it to a task, because borrowed request-local buffers cannot safely back a job that outlives the request. And never serialize a raw device pointer across a process boundary — device residency does not survive the trip, so ship owned data envelopes and rebuild device buffers at the worker.

How this lands depending on where you came from

The GPU hardware is the same regardless of your background; what differs is the habit you bring to its boundary. Each of these is a mental-model shift rather than an API mapping — the trap that the language you know best sets for you when you reach for a device.

Production patterns

These are the habits that keep a GPU lane healthy once it is more than a benchmark. They share a theme: state the budget and the ownership before the device ever runs, and measure the whole path afterward, so the accelerator stays a deliberate decision rather than an assumption.

Pitfalls and tradeoffs

Almost every failure mode below is a systems mistake wearing a performance costume. The kernel is rarely the problem; the trouble is what surrounds it — too many small launches, too much copying, a borrowed buffer that outlived its request, or one shared device with no admission control.

The most common CUDA mistake in service code is not a wrong kernel intrinsic. It is a wrong systems model: too many tiny launches, too much copying, or too little admission control around one scarce device.

Examples

Example 1: a safe Rust wrapper around a kernel launch

This is the safe-gate pattern from earlier, made runnable. Read it in the order of the flow diagram above: `launch_vec_add` calls `build_launch_config` first (which rejects empty input and zero thread counts), allocates the three `DeviceBuffer` owners, and only then enters the unsafe block. Notice the `SAFETY` comment that states the invariants the raw launch relies on. The unsafe region is a handful of lines; everything that could go wrong has been checked before it.

Example 37-1. `safe_kernel_launch_wrapper.rs` — *Launch wrapper*

```
#[derive(Debug, Clone, Copy)]
struct LaunchConfig {
    threads_per_block: u32,
    blocks: u32,
}

#[derive(Debug)]
struct DeviceBuffer {
    len: usize,
    bytes: usize,
}

impl DeviceBuffer {
```

```

    fn for_f32(len: usize) -> Self {
        Self {
            len,
            bytes: len * std::mem::size_of::<f32>(),
        }
    }
}

fn build_launch_config(len: usize, threads_per_block: u32) -> Result<LaunchConfig, &'static str> {
    if len == 0 {
        return Err("empty input");
    }

    if threads_per_block == 0 {
        return Err("threads_per_block must be > 0");
    }

    let blocks = ((len as u32) + threads_per_block - 1) / threads_per_block;

    Ok(LaunchConfig {
        threads_per_block,
        blocks,
    })
}

unsafe fn raw_launch_vec_add(
    config: LaunchConfig,
    a: &DeviceBuffer,
    b: &DeviceBuffer,
    out: &mut DeviceBuffer,
) -> Result<(), &'static str> {
    if a.len != b.len || a.len != out.len {
        return Err("shape mismatch");
    }

    if config.blocks == 0 || config.threads_per_block == 0 {
        return Err("invalid launch");
    }

    Ok(())
}

fn launch_vec_add(len: usize, threads_per_block: u32) -> Result<(LaunchConfig, usize), &'static str> {
    let config = build_launch_config(len, threads_per_block)?;
    let a = DeviceBuffer::for_f32(len);
    let b = DeviceBuffer::for_f32(len);
    let mut out = DeviceBuffer::for_f32(len);

    unsafe {
        // SAFETY:
        // - this wrapper creates all three device buffers with the same logical length.
        // - build_launch_config guarantees nonzero block and thread counts.
        // - the raw launch does not outlive these local buffers in this demo.
        raw_launch_vec_add(config, &a, &b, &mut out)?;
    }
}

```

```
Ok((config, a.bytes + b.bytes + out.bytes))
}

fn main() {
    let (config, device_bytes) = launch_vec_add(4_096, 256).unwrap();

    println!("blocks = {}", config.blocks);
    println!("threads = {}", config.threads_per_block);
    println!("device bytes = {}", device_bytes);
}
```

OUTPUT

```
blocks = 16
threads = 256
device bytes = 49152
```

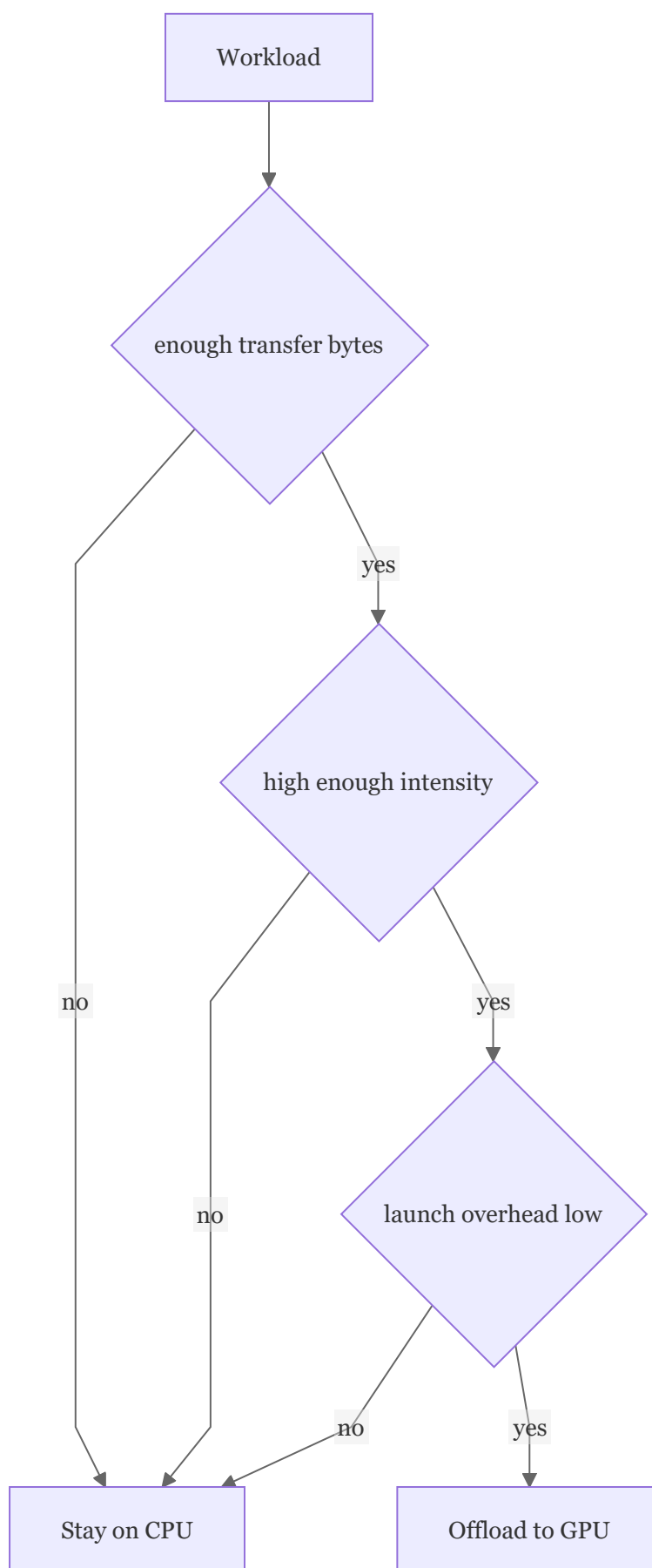
Grid and block shape are checked before the raw launch becomes even possible.

Device buffers are explicit owners. The host wrapper decides when they exist and when they disappear.

The unsafe block is small enough that another engineer can re-derive the invariants quickly.

Example 2: estimate transfer and launch budget before offload

The first GPU decision is a budgeting question, and this code makes it an explicit one. The function to watch is `should_use_gpu`: it does not offload just because the work exists, it requires three conditions to hold at once — enough transfer bytes, high enough arithmetic intensity, and low enough launch overhead. The diagram traces that gate; if any branch fails, the honest answer is to stay on the CPU.



should_use_gpu offloads only when all three thresholds pass; any single failure routes back to the CPU.

Example 37-2. `transfer_budget_estimator.rs` — *Transfer budget*

```
#[derive(Debug, Clone, Copy)]
struct Workload {
    elements: usize,
    flops_per_element: u64,
    input_buffers: usize,
    output_buffers: usize,
}

fn transfer_bytes(work: Workload) -> usize {
    work.elements * std::mem::size_of::<f32>() * (work.input_buffers + work.output_buffers)
}

fn arithmetic_intensity(work: Workload) -> f64 {
    work.flops_per_element as f64
    / (std::mem::size_of::<f32>() as f64 * (work.input_buffers + work.output_buffers) as f64)
}

fn should_use_gpu(work: Workload, launch_us: u64) -> bool {
    let bytes = transfer_bytes(work);
    let intensity = arithmetic_intensity(work);

    bytes >= 8_000_000 && intensity >= 4.0 && launch_us <= 50
}

fn main() {
    let work = Workload {
        elements: 1_000_000,
        flops_per_element: 64,
        input_buffers: 2,
        output_buffers: 1,
    };
    let launch_us = 25_u64;

    println!("transfer bytes = {}", transfer_bytes(work));
    println!("intensity = {:.2}", arithmetic_intensity(work));
    println!("gpu faster = {}", should_use_gpu(work, launch_us));
}
```

OUTPUT

```
transfer bytes = 12000000
intensity = 5.33
gpu faster = true
```

Bytes on the bus are a first-class cost. Count them before you trust the kernel.

More math per byte usually improves the odds that the GPU boundary pays for itself.

A GPU is often the right tool only after batch size, intensity, and launch cost are already visible.

The repository also includes standalone Rust source under `examples/ch37_cuda_and_gpu_acceleration/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

CHAPTER 38 · MERKLE TREE GAMES AND CHALLENGES

Merkle Tree Games and Challenges

Hash trees, ownership-aware construction, inclusion proofs, persistent variants, parallel construction, serialization, and production challenges around content-addressed storage and distributed verification

Merkle-based systems support integrity checks, replay resistance, and distributed verification. This chapter applies hash trees to production audit, challenge, and synchronization workflows.

Builds on Chapters 19, 26, 31, and 37

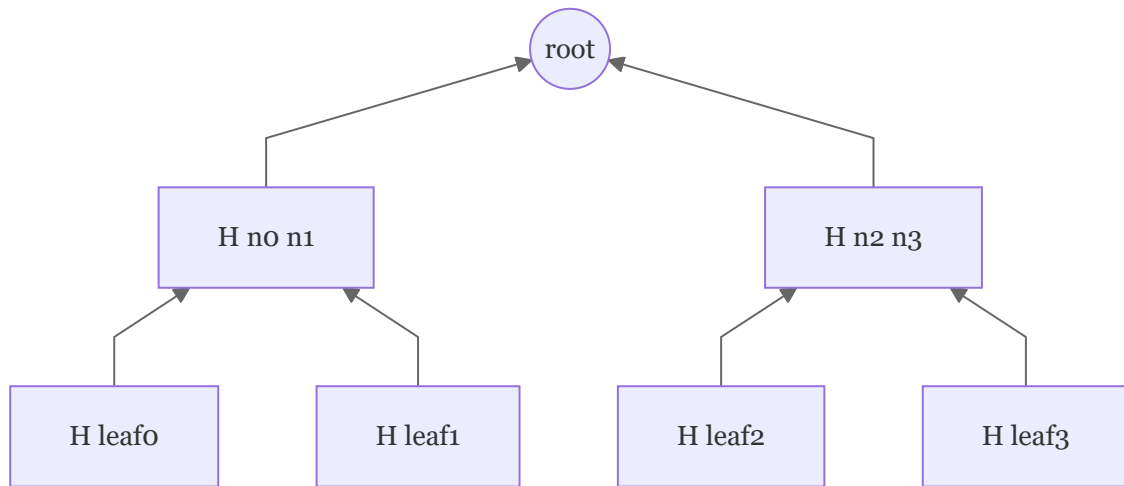
Chapter 19 covered versioned serialization. Chapter 26 covered bounded parallel execution. Chapter 31 covered distributed task replay and trace lineage. Chapter 37 covered expensive specialist workers and explicit boundary budgeting. Merkle systems touch all four.

Opening scenario

Three teams arrive at the same primitive from different directions. A patch service wants to prove that the 120 MB update a client just downloaded is bit-for-bit the file the build server published, without re-sending the whole thing. A game backend wants tamper-evident match history, so a disputed score can be checked against a checkpoint the server signed hours earlier. A distributed verifier wants to confirm that a peer holds a specific record without trusting the peer or pulling its entire dataset. All three need the same thing: a small, fixed-size commitment to a large body of data, plus a way to prove that one piece really belongs to it.

A Merkle tree gives you exactly that. You hash each piece of data into a leaf, hash leaves together in pairs to form a level above, and keep climbing until a single hash remains. That top hash — the root — is the commitment. Change any byte of any leaf and the root changes. The business requirement, then, is not really “use a hash tree.” It is one reproducible commitment policy that all three teams agree on: canonical leaf encoding, explicit domain separation, versioned odd-leaf handling, and portable proof envelopes. Get the policy wrong and two correct-looking implementations will compute different roots from identical data.

Read the tree below from the bottom up. The four leaves are hashes of the raw data; each pair is hashed into the node above it; the two nodes meet at the root. To prove that leaf2 belongs to this root, you do not ship the whole tree. You ship just the siblings a verifier needs to climb back up — here that is $H(\text{leaf}_3)$ and $H(\text{node}_1)$ — and the verifier recomputes the root for itself.



Leaves hash upward in pairs until one root remains. Any change to any leaf changes the root.

At a glance

- Commitment systems stay interoperable only when leaf encoding, internal-node hashing, and odd-leaf policy are explicit.
- Flat levels usually beat pointer-rich trees for proof generation, serialization, and parallel construction.
- Proof verification needs sibling orientation as well as sibling values.
- Persistent snapshots, distributed verification, and content-addressed storage are workflow choices layered on top of the same commitment idea.

Questions to ask in review

- Could another service reproduce the same root from the same leaves without guessing?
- Is the proof envelope versioned strongly enough for cross-process or cross-language use?
- Does the storage model fit the real workload: rebuild, append, snapshot, or distributed verification?
- If the build is parallel, where does it stop being worth the scheduling overhead?

Mental model

Three ideas carry most of the weight in this chapter. The first is what a Merkle tree actually buys you — a small commitment plus cheap inclusion proofs. The second is a Rust-specific representation choice that surprises people coming from object-oriented backgrounds. The third is the realization that most of the bugs live in policy, not in the hash call. Hold these three in mind and the rest of the chapter reads as elaboration.

Core concepts

Hash trees and integrity proofs

The whole value of a Merkle tree comes from one property: the root is a deterministic function of every leaf. Because each internal node hashes its children, and the root hashes the level below it, a single flipped bit

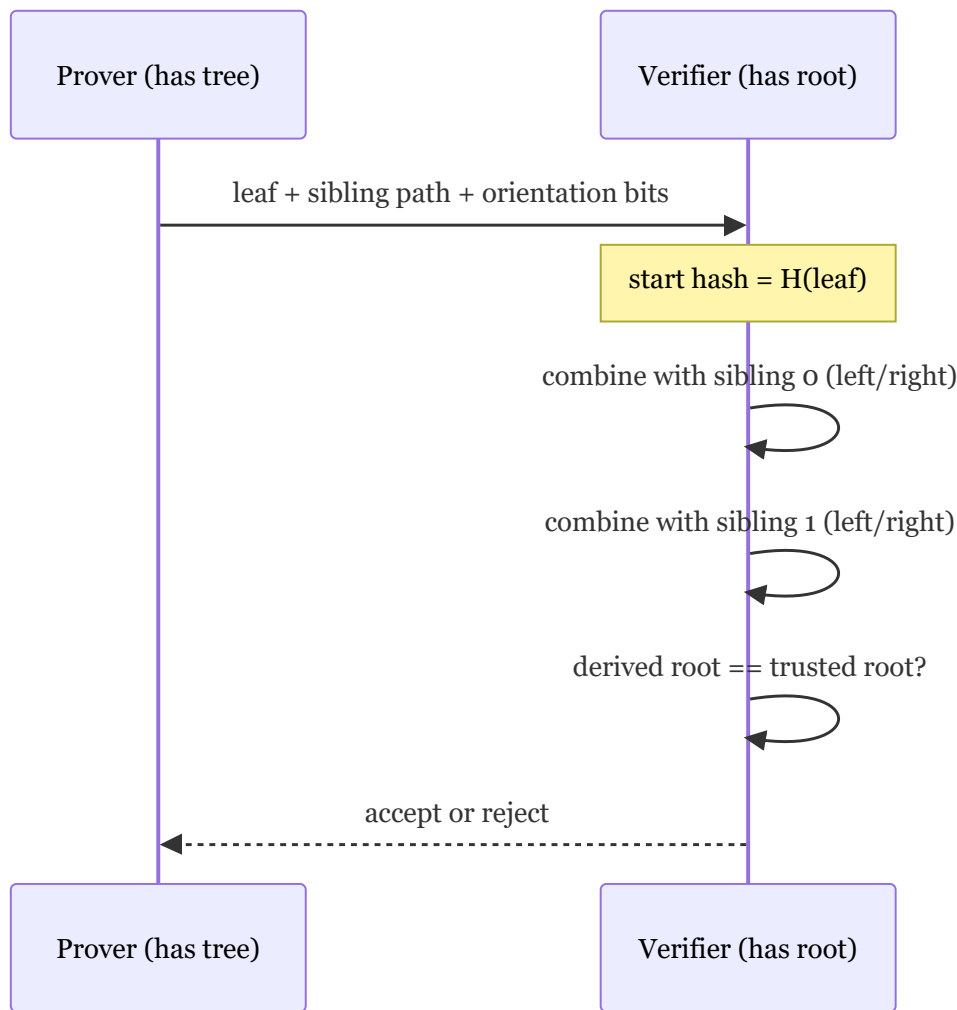
anywhere at the bottom propagates all the way up. That is what makes the root a compact integrity witness — a 32-byte value that stands in for an arbitrarily large dataset. The choices below are about how to store that structure in Rust and how to keep the commitment reproducible, which matters far more than the specific pair-hash function you pick.

A Merkle proof is not a re-serialization of the whole tree. It is the minimum sibling path needed to rebuild the same root from one leaf and one policy set.

Proof generation and verification

An inclusion proof answers a narrow question: “does this leaf belong under this root?” The prover, who holds the whole tree, walks from the leaf up to the root and collects the sibling hash at each level — and, crucially, whether that sibling sat on the left or the right. The verifier holds only the leaf, the proof, and the trusted root. It re-hashes its way up using the siblings and re-derives a root; if that root matches the one it trusts, the leaf is genuine. Notice what the verifier never needs: the rest of the data. The proof is $\log_2(n)$ hashes, not the tree.

The trace below is the part people get wrong. At each step the verifier must combine its running hash with the sibling in the correct order. If the sibling was on the right, the order is $H(\text{running} \parallel \text{sibling})$; if it was on the left, it is $H(\text{sibling} \parallel \text{running})$. Drop the orientation bit and verification becomes ambiguous — sometimes passing, sometimes failing, on data that is actually correct.



The verifier rebuilds the root from one leaf and a short sibling path. Orientation decides the hash order at each step.

Persistent tree variants

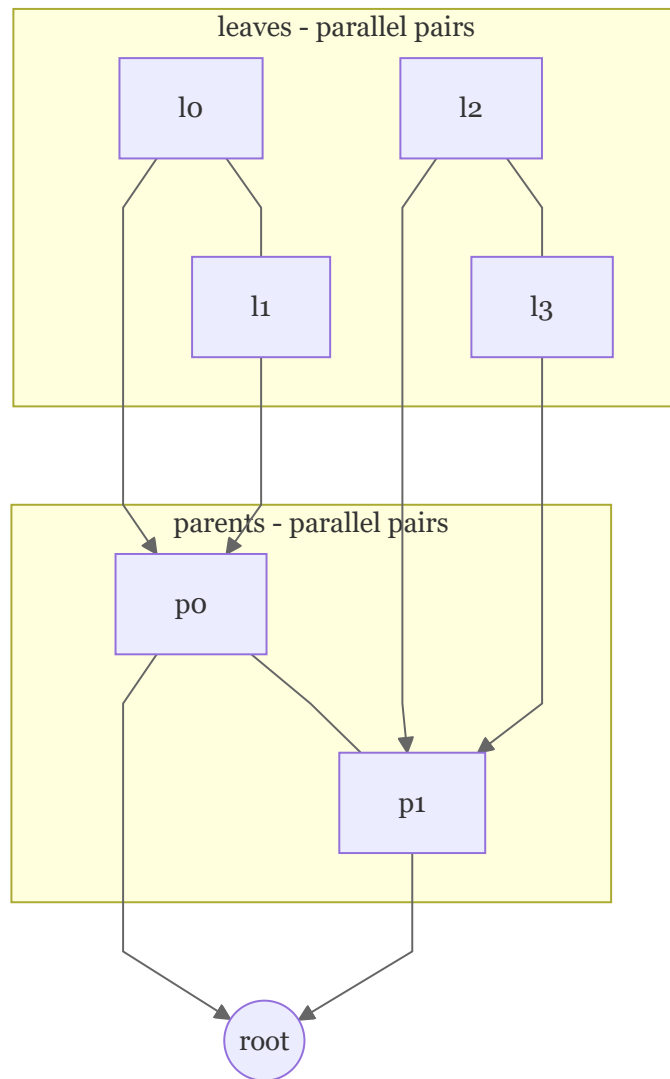
Rebuilding a million-leaf tree because one leaf changed is wasteful, and most real systems never do it. When a single leaf changes, only the hashes on the path from that leaf to the root are affected — roughly $\log_2(n)$ nodes. A persistent tree exploits this: it shares the untouched subtrees with the previous version and allocates fresh nodes only along the changed path. This is the same copy-on-write idea behind persistent data structures generally, and it is where Rust’s ownership model pays off — read-only snapshots can be shared with Arc precisely because nothing can mutate them out from under a holder.

Parallel Merkle tree construction

Building a Merkle tree is a textbook fan-in. Within a single level every parent depends only on its own pair of children and on nothing else at that level, so all the pair-hashes in one level can run at the same time. The dependency is purely vertical: level $k+1$ cannot start until level k is finished, because its inputs are the outputs of the level below. That gives you a clean shape — parallelize within a level, synchronize between levels — and it holds whether the workers are CPU threads, Rayon tasks, GPU lanes, or remote nodes.

In the snippet below, look at the boundary, not the scheduler. `par_chunks(2)` splits one level into independent pairs, the map hashes each pair in parallel, and collect gathers the next level in order before the

outer loop advances. The odd-pair fallback (`if pair.len() == 2`) is the odd-leaf policy showing up again — here it duplicates the lone node, which must match what every other implementation does.



Pairs within a level hash in parallel; each level is a synchronization barrier before the next begins.

Serialization formats

A root by itself is just 32 bytes with no self-description, and that is the trap. The moment a proof leaves the process that built the tree, the receiver has to reproduce the producer’s exact rules, and those rules are not visible in the hash. So the unit you actually ship is not the root and not the bare sibling list — it is an envelope that carries enough metadata to recompute the commitment from scratch. Two facts about leaf bytes deserve special care: they must be canonical (pin endianness and field order; never let a debug format leak into the wire contract), and they must be domain-separated from internal-node bytes so a leaf can never be mistaken for a node.

In the struct below, the body of the proof is siblings, but the fields that make it portable are the metadata: `hash_alg` and `tree_version` tell the verifier which policy produced the root, and `leaf_index` with `leaf_count` let it reconstruct the orientation at each level. Drop any of those and you have a proof that only the original process can interpret.

Games and challenge patterns

These three patterns are the reason the chapter pairs “games” with “challenges.” Each one uses the same commitment-plus-proof core, but applies it to a different trust problem. Content-addressed storage turns the root into a deduplication key and a fetch validator. Tamper-evident game state turns periodic roots into checkpoints you can later use to settle a dispute about a single move. Distributed verification turns proofs into messages that travel between peers who do not trust each other. Read them as variations on one theme: commit once, prove cheaply, verify without the bulk data.

How this maps to what you already know

The Merkle algorithm is language-neutral; what changes is the instinct you bring to representing it. The cards below are about the mental-model shift, not about which crate replaces which library. The recurring theme: the commitment is an owned, immutable byte structure, the proof is a small self-contained value, and the policy that ties them together is an explicit contract rather than something hidden in a class or helper.

Production patterns

Pitfalls and tradeoffs

The most common Merkle bug is not a bad hash function call. It is two systems quietly disagreeing on leaf encoding, node domain separation, or odd-leaf policy while both sides still look locally “correct.”

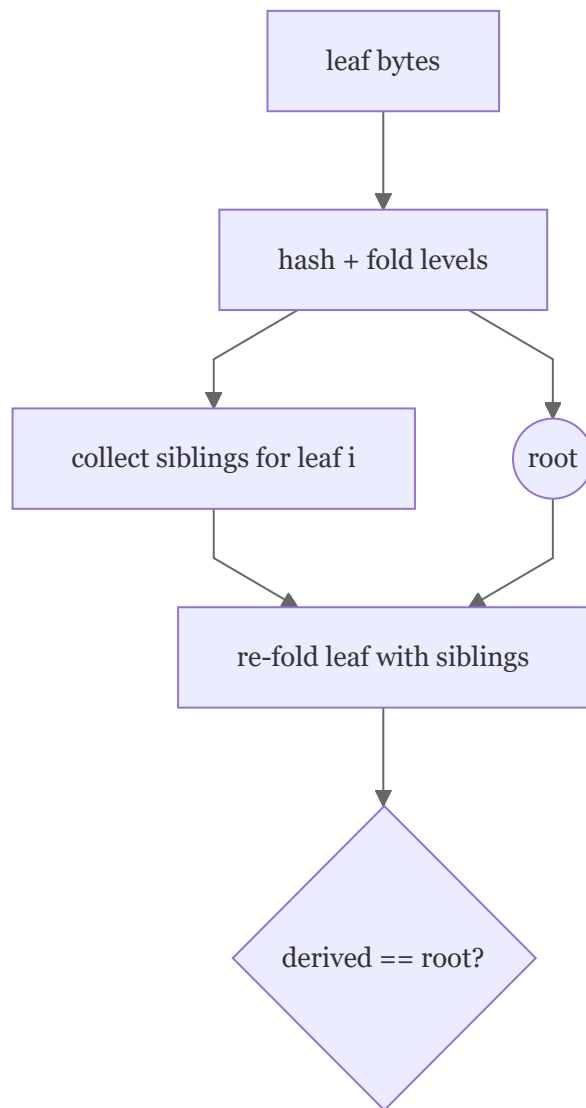
Examples

The browser examples use a tiny deterministic 32-bit demo hash so the tree logic stays runnable without external crates. In production, swap the hash function for a real cryptographic primitive and keep the envelope policy explicit.

Example 1: build a tree, emit a proof, verify one leaf

One owner keeps all levels. The proof is just a sibling path plus orientation bits. No pointer graph is needed.

Before reading the code, follow the three stages it moves through. First it builds the tree by hashing leaves and folding each level into the next until one root remains. Then it emits a proof for one chosen leaf by collecting the sibling at each level along the way to the root. Finally it verifies by re-folding the leaf with those siblings and comparing the result to the root. Watch how the same fold logic is reused in build and verify, and how each proof step records both a hash and a side.



Build once to get the root, emit a sibling path for one leaf, then verify by re-deriving the root from that leaf alone.

Example 38-1. `basic_merkle_tree.rs` — Basic proof

```

type Hash = u32;

#[derive(Debug, Clone, Copy)]
struct ProofStep {
    sibling: Hash,
    sibling_is_left: bool,
}

fn hash_bytes(tag: u8, bytes: &[u8]) -> Hash {
    let mut hash = 2_166_136_261_u32 ^ tag as u32;

    for &byte in bytes {
        hash ^= byte as u32;
        hash = hash.wrapping_mul(16_777_619);
    }

    hash
}

```

```

fn hash_leaf(text: &str) -> Hash {
    hash_bytes(0, text.as_bytes())
}

fn hash_node(left: Hash, right: Hash) -> Hash {
    let mut bytes = [0_u8; 8];
    bytes[..4].copy_from_slice(&left.to_le_bytes());
    bytes[4..].copy_from_slice(&right.to_le_bytes());
    hash_bytes(1, &bytes)
}

#[derive(Debug)]
struct MerkleTree {
    levels: Vec<Vec<Hash>>,
}

impl MerkleTree {
    fn from_leaves(leaves: &[&str]) -> Self {
        assert!(!leaves.is_empty(), "a Merkle tree needs at least one leaf");

        let mut levels = vec![leaves.iter().map(|leaf| hash_leaf(leaf)).collect:::<Vec<_>>()];

        while levels.last().unwrap().len() > 1 {
            let current = levels.last().unwrap();
            let mut next = Vec::with_capacity((current.len() + 1) / 2);

            for pair in current.chunks(2) {
                let left = pair[0];
                let right = if pair.len() == 2 { pair[1] } else { pair[0] };
                next.push(hash_node(left, right));
            }

            levels.push(next);
        }

        Self { levels }
    }

    fn root(&self) -> Hash {
        self.levels.last().unwrap()[0]
    }

    fn proof(&self, mut index: usize) -> Vec<ProofStep> {
        let mut proof = Vec::new();

        for level in &self.levels[..self.levels.len() - 1] {
            let sibling_index = if index % 2 == 0 {
                (index + 1).min(level.len() - 1)
            } else {
                index - 1
            };

            proof.push(ProofStep {
                sibling: level[sibling_index],
                sibling_is_left: sibling_index < index,
            });
        }
    }
}

```

```

    });

    index /= 2;
}

proof
}
}

fn verify_proof(leaf: &str, proof: &[ProofStep], expected_root: Hash) -> bool {
    let mut acc = hash_leaf(leaf);

    for step in proof {
        acc = if step.sibling_is_left {
            hash_node(step.sibling, acc)
        } else {
            hash_node(acc, step.sibling)
        };
    }

    acc == expected_root
}

fn main() {
    let leaves = ["alpha", "beta", "gamma", "delta"];
    let tree = MerkleTree::from_leaves(&leaves);
    let proof = tree.proof(2);

    println!("leaf count = {}", tree.levels[0].len());
    println!("proof len = {}", proof.len());
    println!("verified = {}", verify_proof("gamma", &proof, tree.root()));
}

```

OUTPUT

```

leaf count = 4
proof len = 2
verified = true

```

Each level is a flat vector. The root is just the last level's first element.

Verification needs sibling order as well as sibling value. That direction bit is part of the contract.

The tree owns its levels. Proof steps are copyable values that move easily across process boundaries.

Example 2: parallelize one level at a time

The example uses scoped threads to keep the model crate-free in the browser. The important point is the level boundary, not one specific scheduler choice.

This is the level-by-level shape from the parallel-construction diagram above, made runnable. The key lines build each level in parallel and then synchronize before moving up, and the program proves it got the same answer by computing the root a second time sequentially and asserting roots match = true. That equality check is the whole point: parallelism must never change the commitment. If a refactor ever breaks the match, the parallel path and the sequential path have drifted on ordering or odd-leaf handling.

Example 38-2. `parallel_merkle_levels.rs` — *Parallel levels*

```

type Hash = u32;

fn hash_bytes(tag: u8, bytes: &[u8]) -> Hash {
    let mut hash = 2_166_136_261_u32 ^ tag as u32;

    for &byte in bytes {
        hash ^= byte as u32;
        hash = hash.wrapping_mul(16_777_619);
    }

    hash
}

fn hash_leaf(text: &str) -> Hash {
    hash_bytes(0, text.as_bytes())
}

fn hash_node(left: Hash, right: Hash) -> Hash {
    let mut bytes = [0_u8; 8];
    bytes[..4].copy_from_slice(&left.to_le_bytes());
    bytes[4..].copy_from_slice(&right.to_le_bytes());
    hash_bytes(1, &bytes)
}

fn next_level(level: &[Hash]) -> Vec<Hash> {
    let mut next = Vec::with_capacity((level.len() + 1) / 2);

    for pair in level.chunks(2) {
        let left = pair[0];
        let right = if pair.len() == 2 { pair[1] } else { pair[0] };
        next.push(hash_node(left, right));
    }

    next
}

fn next_level_parallel(level: &[Hash]) -> Vec<Hash> {
    if level.len() <= 2 {
        return next_level(level);
    }

    let pair_count = (level.len() + 1) / 2;
    let left_pairs = pair_count / 2;
    let split = (left_pairs * 2).max(2).min(level.len());

    std::thread::scope(|scope| {
        let left = &level[..split];
        let right = &level[split..];

        let left_handle = scope.spawn(move || next_level(left));
        let right_handle = scope.spawn(move || next_level(right));

        let mut out = left_handle.join().unwrap();
        out.extend(right_handle.join().unwrap());
    });
}

```



```

        out
    })
}

fn build_root(leaves: &[&str], reducer: fn(&[Hash]) -> Vec<Hash>) -> (Hash, usize) {
    let mut level: Vec<Hash> = leaves.iter().map(|leaf| hash_leaf(leaf)).collect();
    let mut levels = 1;

    while level.len() > 1 {
        level = reducer(&level);
        levels += 1;
    }

    (level[0], levels)
}

fn main() {
    let leaves = ["a", "b", "c", "d", "e", "f", "g", "h"];

    let (serial_root, levels) = build_root(&leaves, next_level);
    let (parallel_root, _) = build_root(&leaves, next_level_parallel);

    println!("leaves = {}", leaves.len());
    println!("levels = {}", levels);
    println!("roots match = {}", serial_root == parallel_root);
}

```

OUTPUT

```

leaves = 8
levels = 4
roots match = true

```

Pair hashing inside one level is independent work. The next level boundary is the synchronization point.

Near the root, parallel overhead can dominate. Production code should stop parallelizing once the level gets small.

This same level-wise shape is easy to batch inside a GPU lane or a distributed verification worker later.

The repository also includes standalone Rust source under `examples/ch38_merkle_tree_games_and_challenges/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

CHAPTER 39 · GRAPH SEARCH GAMES

Graph Search Games

Graph representations in Rust, BFS and DFS, Dijkstra and A, ownership-friendly graph models, arena-backed graphs, parallel traversal, and challenge tracks for mazes, dependency resolution, and distributed search*

Graph search is where the algorithm you already know meets the storage decision Rust forces you to make first. This chapter treats BFS, DFS, Dijkstra, and A* as production data-processing components: how the nodes are owned, what an edge holds, which traversal fits the cost model, and only then whether the frontier should go parallel or distributed.

Builds on Chapters 10, 11, 13, 22, and 26

Chapter 10 covered contiguous storage and slice-first APIs. Chapter 11 covered hash-based visited sets and maps. Chapter 13 explained arena-backed ownership. Chapter 22 covered threads and shared-state tradeoffs. Chapter 26 covered bounded work orchestration. This chapter uses all of them.

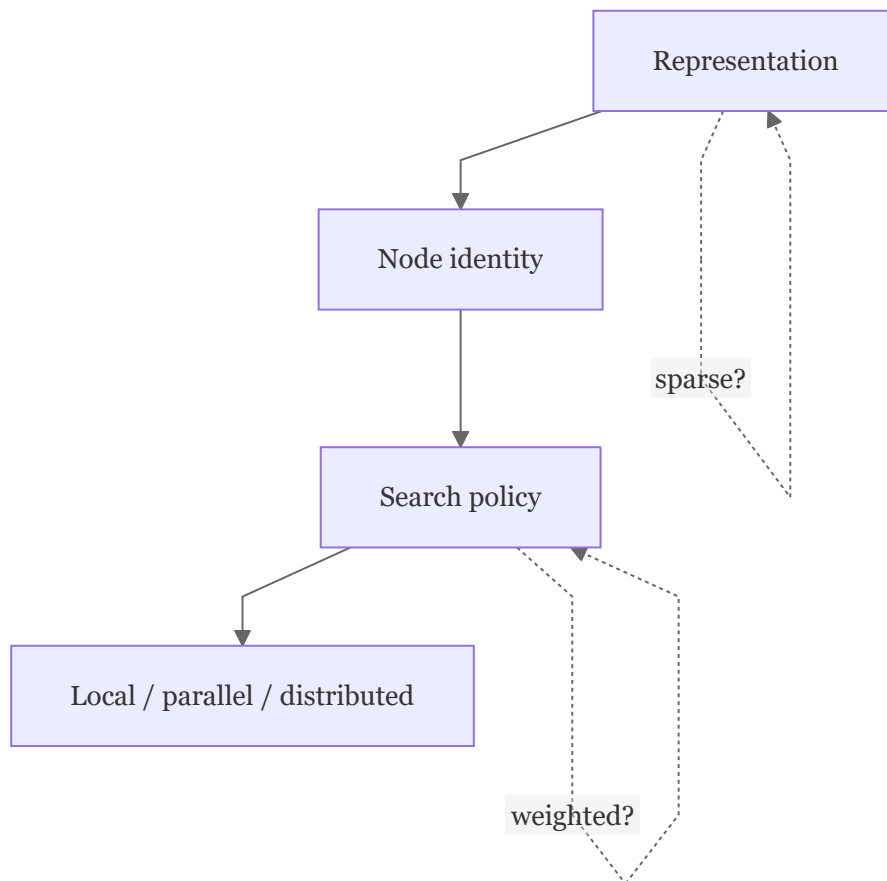
Opening scenario

A platform team is building a maze solver, a dependency resolver, and a distributed topology search. The business requirement is to choose graph ownership, representation, and traversal policy from the workload before adding parallelism or distributed execution.

A practical decision order

The order matters because each decision constrains the next. Representation determines how cheaply you can read neighbors; the identity model determines whether the borrow checker will let your search loop coexist with mutation; the cost structure determines which algorithm is even correct; and concurrency is a question you should only ask once the single-threaded version is right.

1. Choose the graph representation from sparsity and mutation shape.
2. Choose the node identity model from ownership: stable indices or explicit shared owners.
3. Choose the search from cost structure: unweighted, weighted, heuristic, or structural.
4. Only then decide whether the frontier should stay local, go parallel, or go distributed.



Each choice constrains the next. Settle storage and identity before picking an algorithm, and concurrency last of all.

At a glance

- Choose representation from sparsity and mutation shape before choosing an algorithm.
- Prefer one owner plus stable handles when the graph grows, serializes, or crosses queue boundaries.
- Use BFS for unweighted hops, DFS for structure, Dijkstra for weighted shortest paths, and A* only when the heuristic is worth defending.
- Parallel or distributed frontiers add duplicate-suppression and merge-cost questions immediately.

Design questions

- Is this graph sparse, dense, grid-shaped, or arena-friendly?
- What should an edge store: a stable handle, a weight, or a queued message?
- Which search policy matches the real cost model rather than the most advanced-looking algorithm?
- If the frontier widens, where do queue bounds, visited-state contention, and trace lineage become operational concerns?

Mental model

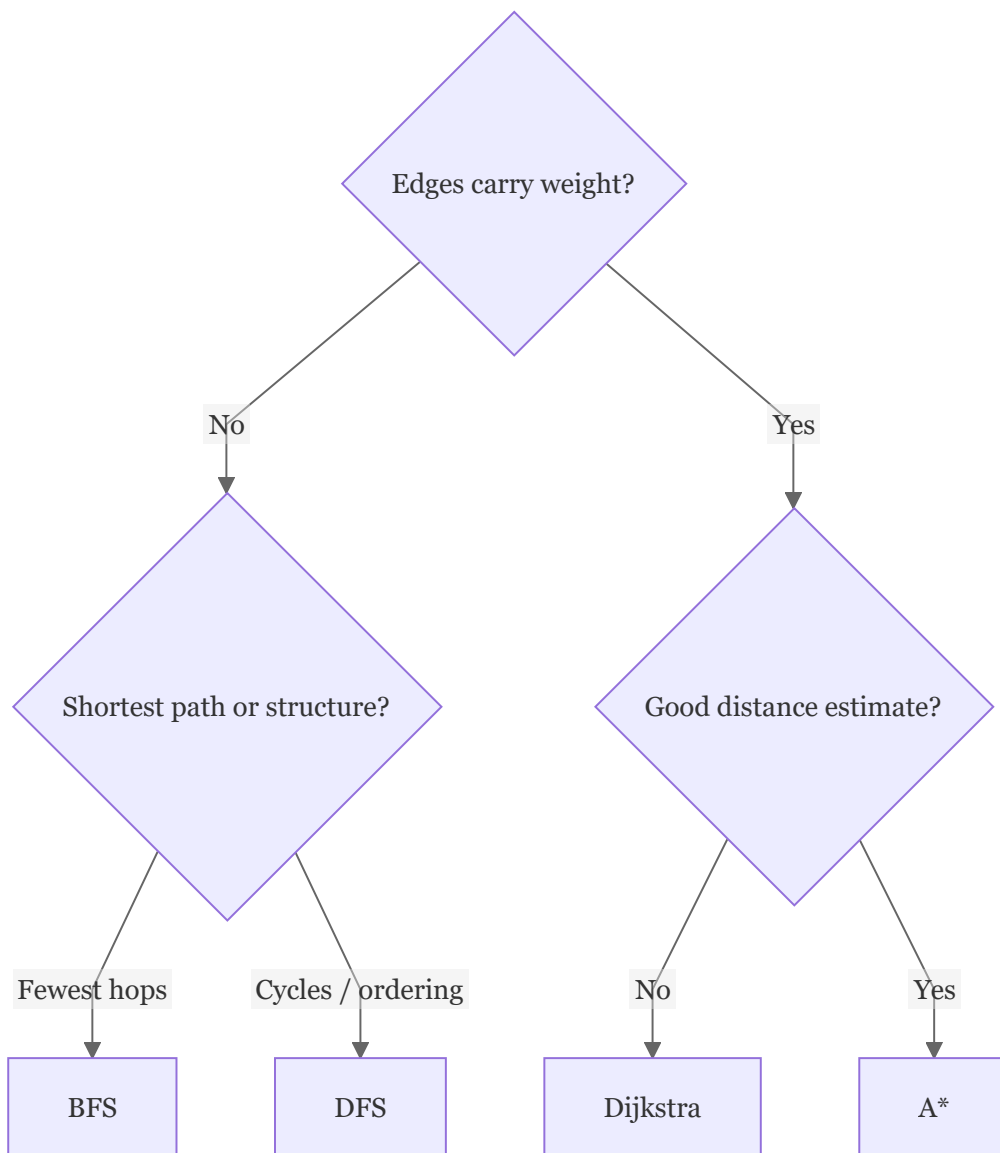
Core concepts

Graph representations in Rust

For most service, maze, and dependency workloads, adjacency lists win because the graph is sparse and edge iteration dominates. Adjacency matrices only become calm when the graph is dense enough or fixed enough that the quadratic storage buys a real lookup advantage.

Choosing among BFS, DFS, Dijkstra, and A*

These four traversals are not a ranking from simple to advanced; they are answers to different questions. The decision tree below collapses the choice to two questions you can answer about almost any graph: do the edges carry weight, and do you have a usable estimate of the distance to the goal?



Two questions decide the search: are edges weighted, and is there a usable heuristic toward the goal?

A common correction: do not use a more general weighted search when the workload is unweighted. BFS often gives the simplest correct answer and the simplest implementation.

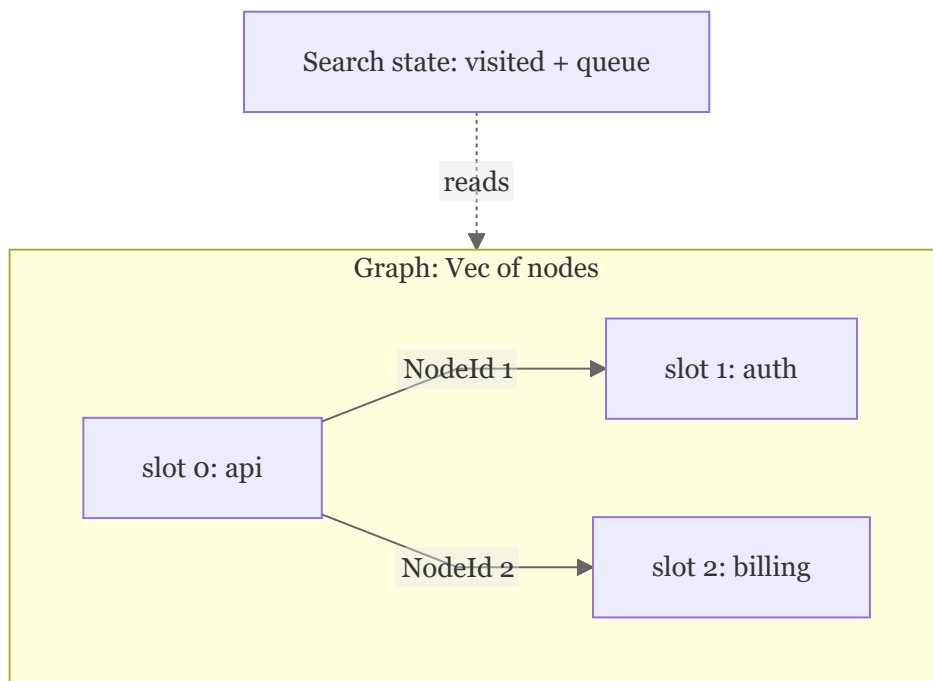
Ownership-friendly graph models

This is one of the main places Rust differs from many C++ or C# designs: the simpler graph is often one owner plus handles, not a web of objects each acting as an independent owner.

Arena-backed graphs

An arena-backed graph fits the common lifecycle exactly: the graph is built once as a batch, searched many times, and then dropped as a batch. That describes mazes, abstract syntax trees, dependency plans, route graphs, and many game worlds. The pattern is just a `Vec<Node>` that owns every node plus a `NodeId` newtype that names a slot in it. The newtype is the load-bearing detail: it gives each node a stable logical identity that survives reallocation and serialization, and it stops you from confusing a node index with any other usize floating around the program.

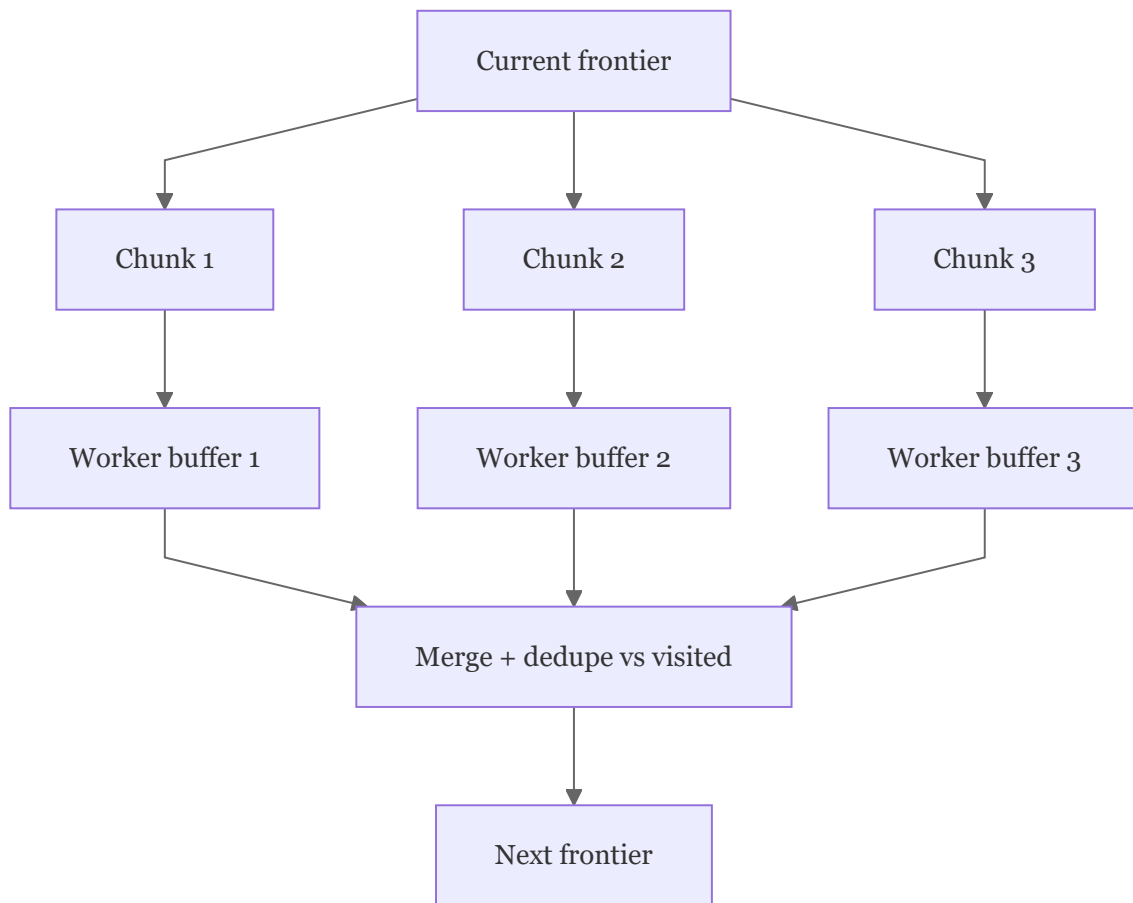
The diagram shows why this stays calm under mutation: edges point at slots, not at addresses, so growing the table never invalidates an edge. Compare that to a pointer graph, where one reallocation would dangle every stored neighbor pointer.



One Vec owns every node; edges and search state hold NodeId handles into it, never references.

Parallel graph traversal

The safe way to parallelize a search is to keep BFS's layer structure and only parallelize within a layer. Split the current frontier into chunks, expand each chunk on its own worker into a private next-frontier buffer, and merge the buffers (deduplicating against the visited set once) before the next level begins. The merge is a barrier, so the speedup is real only when each level is wide enough that the parallel expansion dwarfs the synchronization cost.



Level-synchronous BFS: expand frontier chunks in parallel, merge and dedupe once per level.

The repository also includes a standalone example under `examples/ch39_graph_search_games/parallel_frontier_levels.rs` that shows one level-synchronous frontier expansion shape outside the in-browser editor.

How to think about this coming from another language

Every language gives you a default mental picture of a graph, and most of those defaults assume a runtime that tracks who points at whom. Rust does not. The single biggest adjustment, regardless of where you come from, is to stop modeling a graph as a web of objects holding references and start modeling it as one owning table plus small handles. Here is the specific shift, and the specific trap, for each background.

Practice projects

Production patterns

Pitfalls and tradeoffs

The most common graph-design mistake in Rust is trying to preserve a reference-rich object model from another language when a stable-index arena model would have made search, mutation, and serialization

much easier.

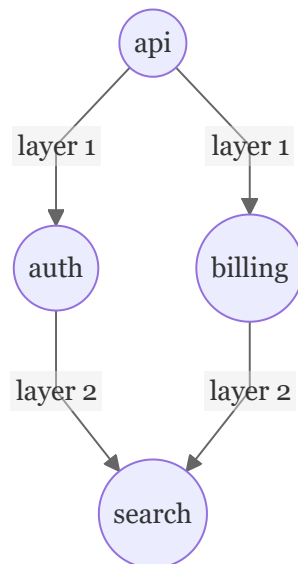
Examples

Both examples use the same storage idea from the mental model: one owning node table and edges that hold a `NodeId` handle. Run each listing first to confirm the baseline output, then edit the graph or the goal and watch how the answer changes. Each listing is introduced by what to look at and a diagram of the flow before the code itself.

Example 1: BFS over an adjacency list with stable handles

The graph is owned in one node table. Edges are `NodeId` handles, BFS uses a queue, and the shortest unweighted route is counted in hops.

What to look at: the loop in `shortest_hops` marks each node with a distance the first time it is reached and never revisits it. Because the queue is FIFO, nodes come out in layer order, so the distance recorded for the goal is the minimum number of hops. The diagram traces that expansion over the sample graph: `api` is layer 0, its direct neighbors are layer 1, and `search` is first reached at layer 2.



BFS expands in layers; search is first reached at hop 2 from api, which is the shortest path.

Example 39-1. `bfs_adjacency_handles.rs` — *Bfs handles*

```

use std::collections::VecDeque;

#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
struct NodeId(usize);

#[derive(Debug)]
struct Node {
    name: &'static str,
    edges: Vec<NodeId>,
}

#[derive(Default, Debug)]

```

```

struct Graph {
    nodes: Vec<Node>,
}

impl Graph {
    fn add_node(&mut self, name: &'static str) -> NodeId {
        let id = NodeId(self.nodes.len());
        self.nodes.push(Node {
            name,
            edges: Vec::new(),
        });
        id
    }

    fn add_edge(&mut self, from: NodeId, to: NodeId) {
        self.nodes[from.0].edges.push(to);
    }

    fn name(&self, id: NodeId) -> &'static str {
        self.nodes[id.0].name
    }

    fn names(&self, ids: &[NodeId]) -> Vec<&'static str> {
        ids.iter().map(|&id| self.name(id)).collect()
    }

    fn bfs_order(&self, start: NodeId) -> Vec<NodeId> {
        let mut visited = vec![false; self.nodes.len()];
        let mut queue = VecDeque::new();
        let mut order = Vec::new();

        visited[start.0] = true;
        queue.push_back(start);

        while let Some(id) = queue.pop_front() {
            order.push(id);

            for &next in &self.nodes[id.0].edges {
                if !visited[next.0] {
                    visited[next.0] = true;
                    queue.push_back(next);
                }
            }
        }

        order
    }

    fn shortest_hops(&self, start: NodeId, goal: NodeId) -> Option<usize> {
        let mut dist = vec![usize::MAX; self.nodes.len()];
        let mut queue = VecDeque::new();

        dist[start.0] = 0;
        queue.push_back(start);

        while let Some(id) = queue.pop_front() {

```



```

        if id == goal {
            return Some(dist[id.0]);
        }

        for &next in &self.nodes[id.0].edges {
            if dist[next.0] == usize::MAX {
                dist[next.0] = dist[id.0] + 1;
                queue.push_back(next);
            }
        }
    }
}

None
}

fn main() {
    let mut graph = Graph::default();

    let api = graph.add_node("api");
    let auth = graph.add_node("auth");
    let billing = graph.add_node("billing");
    let search = graph.add_node("search");

    graph.add_edge(api, auth);
    graph.add_edge(api, billing);
    graph.add_edge(auth, search);
    graph.add_edge(billing, search);

    let order = graph.bfs_order(api);
    let hops = graph.shortest_hops(api, search).unwrap_or(0);

    println!("bfs = {}", graph.names(&order).join(", "));
    println!("path hops = {}", hops);
}

```

OUTPUT

```

bfs = api,auth,billing,search
path hops = 2

```

One `Vec<Node>` plus `NodeId` keeps ownership stable and makes edges cheap values.

BFS explores by layers, so the first time the goal is reached is the shortest unweighted hop count.

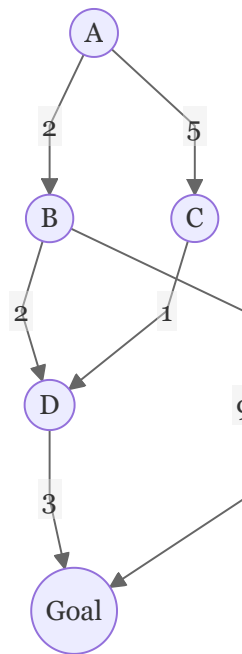
This is the calm shape for dependency graphs, service-route graphs, and maze steps when costs are uniform.

Example 2: weighted shortest path with Dijkstra and A*

The graph keeps coordinates only because A* needs a heuristic. Dijkstra uses exact cost only; A* adds a guess about the remaining distance. Both must agree on the answer.

What to look at: both methods share the same State on a BinaryHeap, and the inverted Ord makes that max-heap behave as a min-heap on the priority field. The only real difference is the priority: Dijkstra orders by cost so far, while A* orders by cost so far plus the Manhattan heuristic to the goal. On the sample graph the

tempting direct edge B to Goal costs 9, but routing through D is cheaper, so both report the optimal cost of 7. The diagram shows the two candidate routes and why the indirect one wins.



A to B to D to Goal costs $2+2+3 = 7$, beating the direct B to Goal edge of 9. Dijkstra and A both find 7.*

Example 39-2. `weighted_graph_dijkstra_astar.rs` — *Dijkstra astar*

```

use std::cmp::Ordering;
use std::collections::BinaryHeap;

#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash, PartialOrd, Ord)]
struct NodeId(usize);

#[derive(Clone, Copy, Debug)]
struct Edge {
    to: NodeId,
    cost: u32,
}

#[derive(Debug)]
struct Node {
    name: &'static str,
    x: i32,
    y: i32,
    edges: Vec<Edge>,
}

#[derive(Default)]
struct WeightedGraph {
    nodes: Vec<Node>,
}

#[derive(Clone, Copy, Debug, PartialEq, Eq)]
struct State {
    cost: u32,

```

```

    node: NodeId,
    estimate: u32,
}

impl Ord for State {
    fn cmp(&self, other: &Self) -> Ordering {
        other
            .estimate
            .cmp(&self.estimate)
            .then_with(|| other.cost.cmp(&self.cost))
            .then_with(|| other.node.cmp(&self.node))
    }
}

impl PartialOrd for State {
    fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
        Some(self.cmp(other))
    }
}

impl WeightedGraph {
    fn add_node(&mut self, name: &'static str, x: i32, y: i32) -> NodeId {
        let id = NodeId(self.nodes.len());
        self.nodes.push(Node {
            name,
            x,
            y,
            edges: Vec::new(),
        });
        id
    }

    fn add_edge(&mut self, from: NodeId, to: NodeId, cost: u32) {
        self.nodes[from.0].edges.push(Edge { to, cost });
    }

    fn heuristic(&self, left: NodeId, right: NodeId) -> u32 {
        self.nodes[left.0].x.abs_diff(self.nodes[right.0].x)
            + self.nodes[left.0].y.abs_diff(self.nodes[right.0].y)
    }

    fn dijkstra_cost(&self, start: NodeId, goal: NodeId) -> Option<u32> {
        let mut dist = vec![u32::MAX; self.nodes.len()];
        let mut heap = BinaryHeap::new();

        dist[start.0] = 0;
        heap.push(State {
            cost: 0,
            node: start,
            estimate: 0,
        });

        while let Some(State { cost, node, .. }) = heap.pop() {
            if node == goal {
                return Some(cost);
            }
        }
    }
}

```

```

        if cost > dist[node.0] {
            continue;
        }

        for edge in &self.nodes[node.0].edges {
            let next_cost = cost + edge.cost;
            if next_cost < dist[edge.to.0] {
                dist[edge.to.0] = next_cost;
                heap.push(State {
                    cost: next_cost,
                    node: edge.to,
                    estimate: next_cost,
                });
            }
        }
    }
}

None
}

fn a_star_cost(&self, start: NodeId, goal: NodeId) -> Option<u32> {
    let mut best = vec![u32::MAX; self.nodes.len()];
    let mut heap = BinaryHeap::new();

    best[start.0] = 0;
    heap.push(State {
        cost: 0,
        node: start,
        estimate: self.heuristic(start, goal),
    });

    while let Some(State { cost, node, .. }) = heap.pop() {
        if node == goal {
            return Some(cost);
        }

        if cost > best[node.0] {
            continue;
        }

        for edge in &self.nodes[node.0].edges {
            let next_cost = cost + edge.cost;
            if next_cost < best[edge.to.0] {
                best[edge.to.0] = next_cost;
                heap.push(State {
                    cost: next_cost,
                    node: edge.to,
                    estimate: next_cost + self.heuristic(edge.to, goal),
                });
            }
        }
    }
}

None
}

```

```

}

fn main() {
    let mut graph = WeightedGraph::default();

    let a = graph.add_node("A", 0, 0);
    let b = graph.add_node("B", 1, 0);
    let c = graph.add_node("C", 0, 1);
    let d = graph.add_node("D", 1, 1);
    let goal = graph.add_node("Goal", 2, 1);

    graph.add_edge(a, b, 2);
    graph.add_edge(a, c, 5);
    graph.add_edge(b, d, 2);
    graph.add_edge(c, d, 1);
    graph.add_edge(d, goal, 3);
    graph.add_edge(b, goal, 9);

    let dijkstra = graph.dijkstra_cost(a, goal).unwrap_or(0);
    let a_star = graph.a_star_cost(a, goal).unwrap_or(0);

    println!("dijkstra cost = {}", dijkstra);
    println!("a_star cost = {}", a_star);
}

```

OUTPUT

```

dijkstra cost = 7
a_star cost = 7

```

Dijkstra is the correct baseline whenever edge costs matter and remain non-negative.

A* becomes useful only because the heuristic is explicit and tied to node coordinates.

A BinaryHeap frontier makes the weighted next-best expansion policy explicit and efficient enough for most host-side work.

The repository also includes standalone Rust source under `examples/ch39_graph_search_games/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

CHAPTER 40 · MATRIX OPTIMIZATION GAMES

Matrix Optimization Games

Matrix storage layouts, cache-aware multiplication, SIMD opportunities, sparse vs dense tradeoffs, blocking and tiling, parallel matrix operations, GPU offload, and optimization challenge tracks

Matrix optimization depends on layout, tiling, sparsity, SIMD, thread budgets, and accelerator boundaries. This chapter turns those factors into measurable Rust implementation choices.

Builds on Chapters 12, 22, 26, 32, 33, and 37

Chapter 12 established dense matrix layout and views. Chapter 22 covered threads and ownership across them. Chapter 26 covered pools and bounded work orchestration. Chapter 32 covered MPI and HPC partitioning. Chapter 33 covered performance-oriented Rust more broadly. Chapter 37 covered GPU boundaries and transfer budgeting.

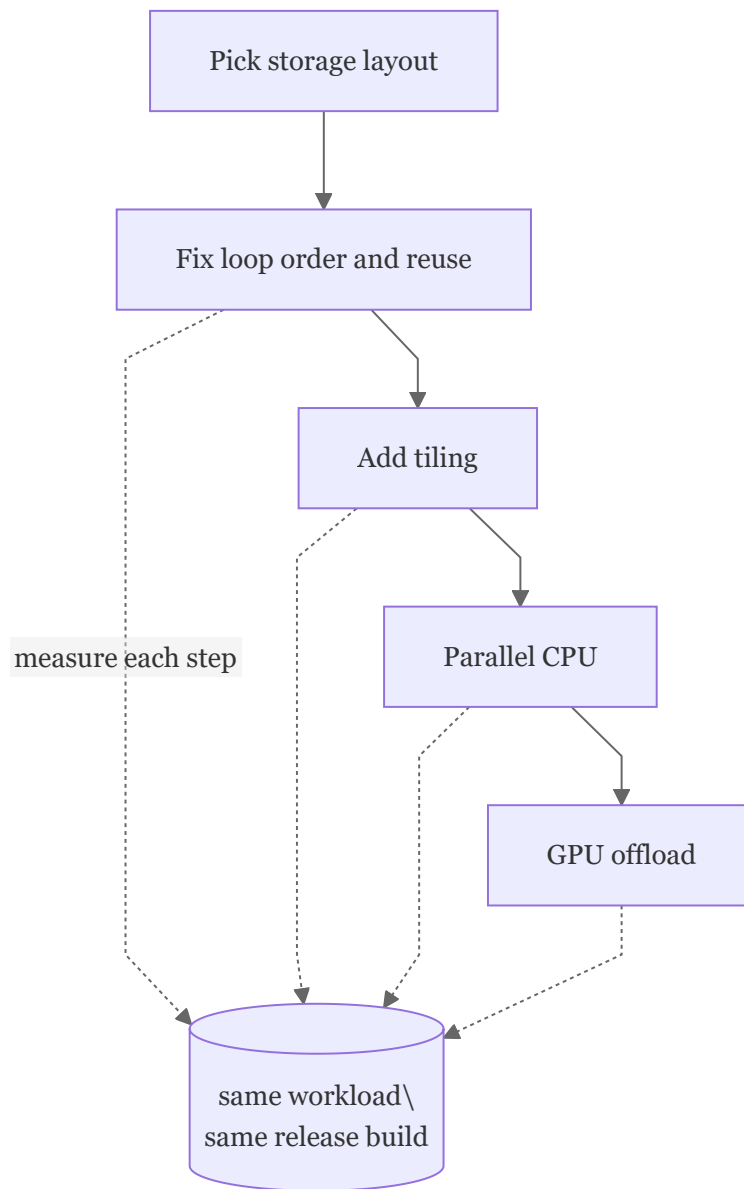
Opening scenario

Three teams land on the same desk in the same week. One owns a batch scoring service whose hot path is a dense matrix multiply. One owns a route planner whose graph is mostly empty space. One owns a research spike that wants to put everything on a GPU. Each team has a story about why their workload is slow and what would make it fast, and each story is plausible. The work here is to replace those stories with measurements, in an order that does not waste effort.

That order matters more than any single trick. The mistake repeated across all three teams is reaching for the most exciting lever first: SIMD intrinsics, a thread pool, a GPU. Those levers can all help, but only after the storage layout and the loop order are honest. A clever kernel built on a cache-hostile layout just makes the wrong memory pattern run faster. So the chapter walks the levers in the order that actually pays off, and insists that every step prove itself against the same workload and the same release build.

A practical decision order

Read the pipeline below from left to right. Each stage is cheaper to get right than the one after it, and each later stage assumes the earlier ones are already settled. You almost never need to reach the GPU stage, and when you do, you want to arrive there with the layout question already answered.



Optimize in cost order: layout first, accelerator last, and measure every stage against one fixed workload.

1. Choose dense row-major, column-oriented, blocked, or sparse storage from actual access pattern.
2. Fix loop order and reuse before widening the execution model.
3. Introduce tiles before threads, and bounded work before a shared accelerator.
4. Compare CPU, parallel CPU, and GPU on one benchmark plan instead of on three stories.

At a glance

- Start with storage layout and loop order before you talk about threads, SIMD, or GPUs.
- Dense and sparse matrices are different workload models, not merely different optimization flavors.
- Tiling is both a cache policy and an ownership policy for parallel work.
- A fair CPU-versus-GPU comparison publishes transfer and launch costs, not only kernel time.

Optimization order

- What is the real inner-loop access pattern?
- Does the workload want row-major dense storage, blocked storage, or a sparse format?
- Can each worker own an output tile without a hot shared merge?
- Which signal proves the next change mattered: wall time, cache misses, bytes moved, or queue wait?

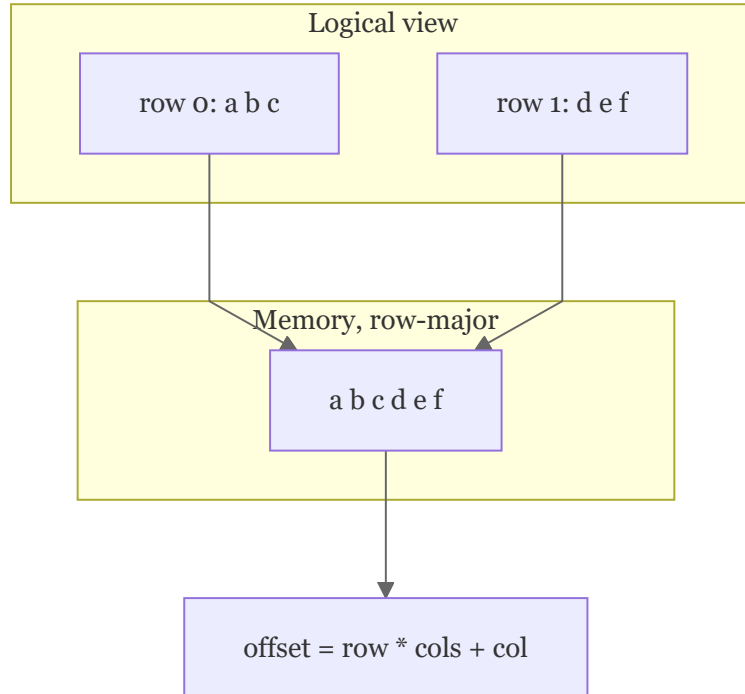
Mental model

Before any specific technique, three ideas frame the whole chapter. They explain why the optimization order is what it is: speed lives in how memory is laid out and reused, dense and sparse are genuinely different problems, and the wide execution models only help once the local boundary is already honest.

Core concepts

Matrix storage layouts

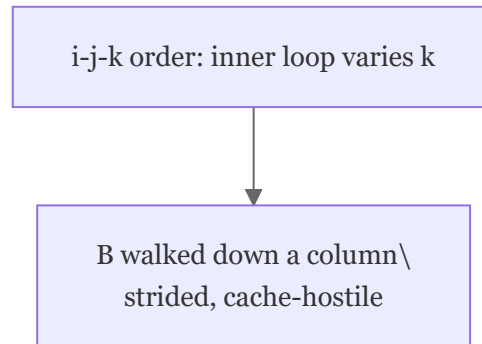
A matrix is a two-dimensional idea, but memory is one-dimensional. Every layout is really a rule for flattening rows and columns into a single contiguous run of bytes, and the rule you pick decides which traversals are cheap. Row-major storage lays each row down end to end, so walking a row is a straight, cache-friendly scan and walking a column jumps by a full row width on every step. The diagram below shows that flattening for a small matrix; keep it in mind whenever you reason about which loop is the fast one.



Row-major flattens rows back to back; the index formula is the bridge between the 2D view and the 1D buffer.

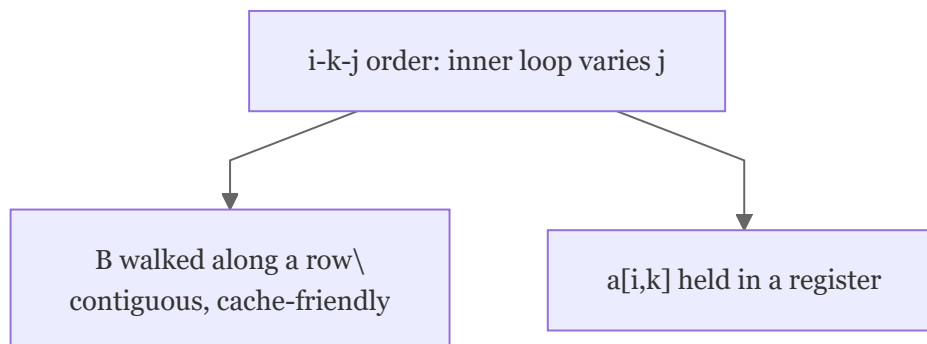
Cache-aware multiplication

Matrix multiply does the same arithmetic no matter how you nest the loops, but the three loop indices i , j , and k can be ordered six ways, and the order decides how memory is touched. With row-major storage the classic i - j - k order walks B down a column in the inner loop, which strides across whole rows and defeats the cache. Swapping to i - k - j makes the inner loop sweep a row of B contiguously while a single scalar from A stays hot. Same result, far fewer cache misses. The diagram contrasts the two inner-loop access patterns.



Slow order: the i - j - k nesting walks B down a column in the inner loop, striding across whole rows.

Reordering to i - k - j keeps the same arithmetic but changes which way the inner loop walks B :



Fast order: the i - k - j nesting sweeps a row of B contiguously while a scalar of A stays hot in a register.

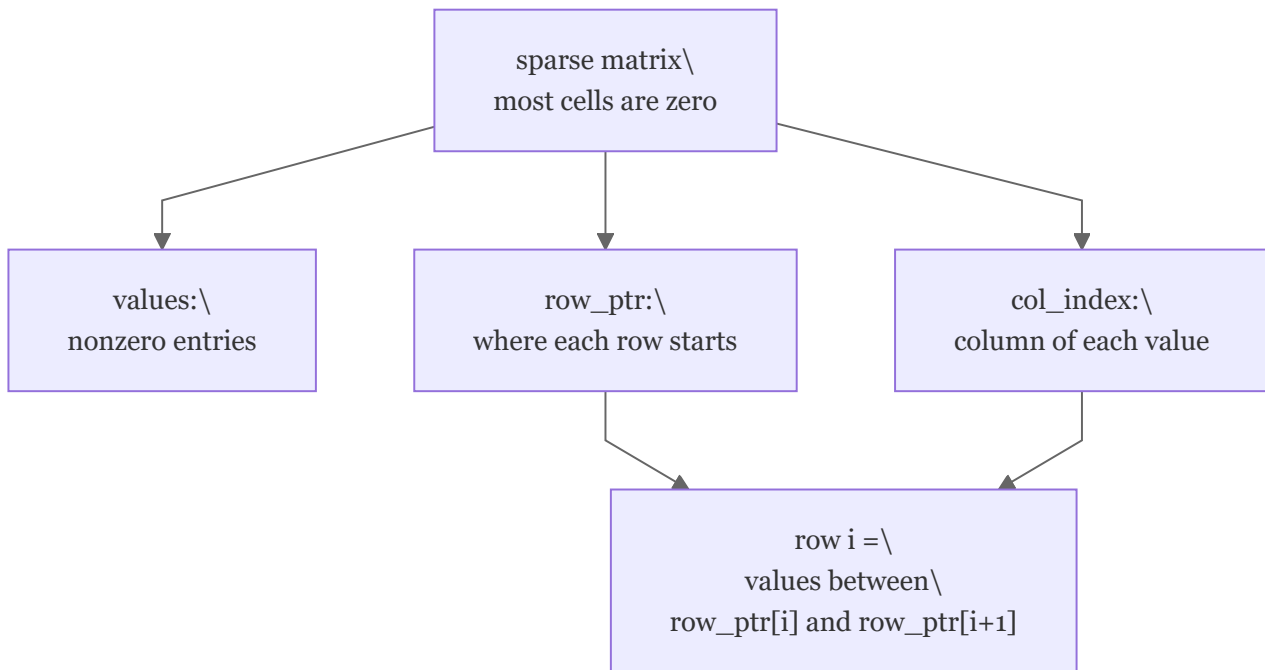
A reliable first upgrade is often embarrassingly simple: stop walking columns of a row-major right-hand matrix in the innermost loop. Even before tiling, that change can move the cache story in your favor.

SIMD opportunities

SIMD lets one instruction multiply or add several values at once, which is exactly the shape of the inner dot-product loop. But the compiler can only vectorize what it can prove is safe and regular, and you can only reach for intrinsics productively once the data is already contiguous. So treat SIMD as the last layer, not the first. In most kernels the auto-vectorizer does the work for free if you feed it a flat buffer and a tight loop; explicit intrinsics earn their complexity only after profiling shows the loop is genuinely arithmetic-bound. The snippet below is the kind of inner loop that vectorizes well: contiguous loads, a fixed trip count, and no branchy noise.

Sparse vs dense matrices

Dense and sparse are not two tuning settings on the same matrix; they are two different data models that answer different questions. A dense matrix stores every cell, including the zeros, and pays for that with predictable contiguous traversal. A sparse matrix stores only the nonzero values plus enough index metadata to find them, trading random-access simplicity for the ability to skip empty space entirely. The most common sparse format, compressed sparse row (CSR), encodes the whole matrix as three flat arrays: the nonzero values, the column index of each value, and a per-row pointer that says where each row begins in those arrays. The diagram shows how those three arrays relate, and why iterating one row is just slicing between two row pointers.



CSR is three flat arrays; a row is the slice of values and columns between two consecutive row pointers.

Blocking and tiling

Blocking turns one large multiply into many smaller reuse-friendly subproblems. The common host-side shape is tile loops over *ii*, *kk*, and *jj*, then short inner loops that keep one tile of *A* hot while updating one tile of *C*.

Better cache reuse, fewer cold misses on reused blocks, and a natural parallel work unit.

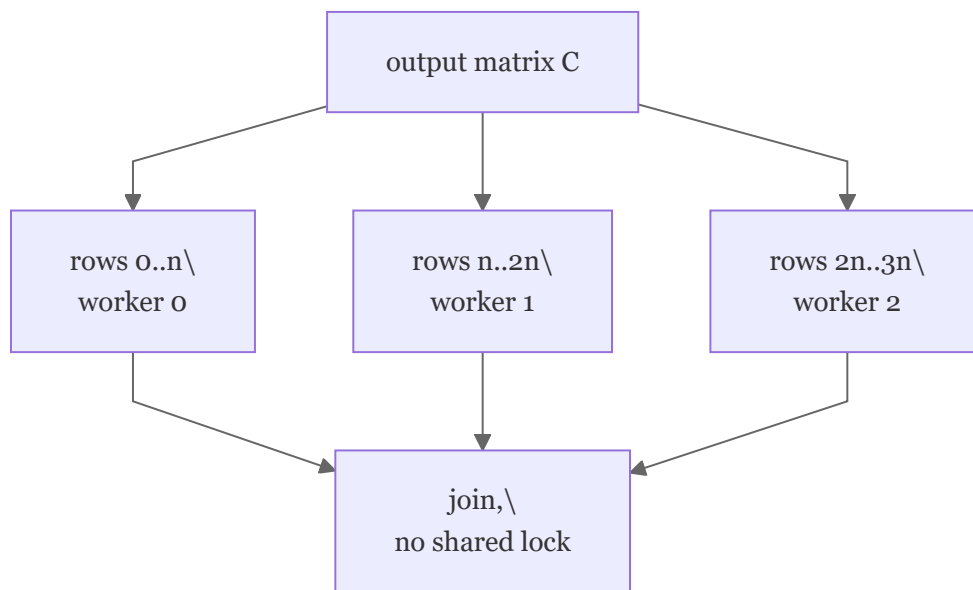
More indexing logic, more tuning surface, and more sensitivity to matrix size and cache shape.

If the matrices are tiny or the service is dominated by queueing or serialization, tiling may not move the real wall-clock story enough to matter.

Parallel matrix operations

Matrix multiply parallelizes well because the output cells are independent: nothing in computing $C[i, j]$ depends on any other cell of *C*. That independence is the whole opportunity, and the way to keep it is to partition the output so each worker owns a disjoint set of rows or tiles and writes only there. The moment workers share a write target through a lock or a contended accumulator, you serialize the gain back out. In

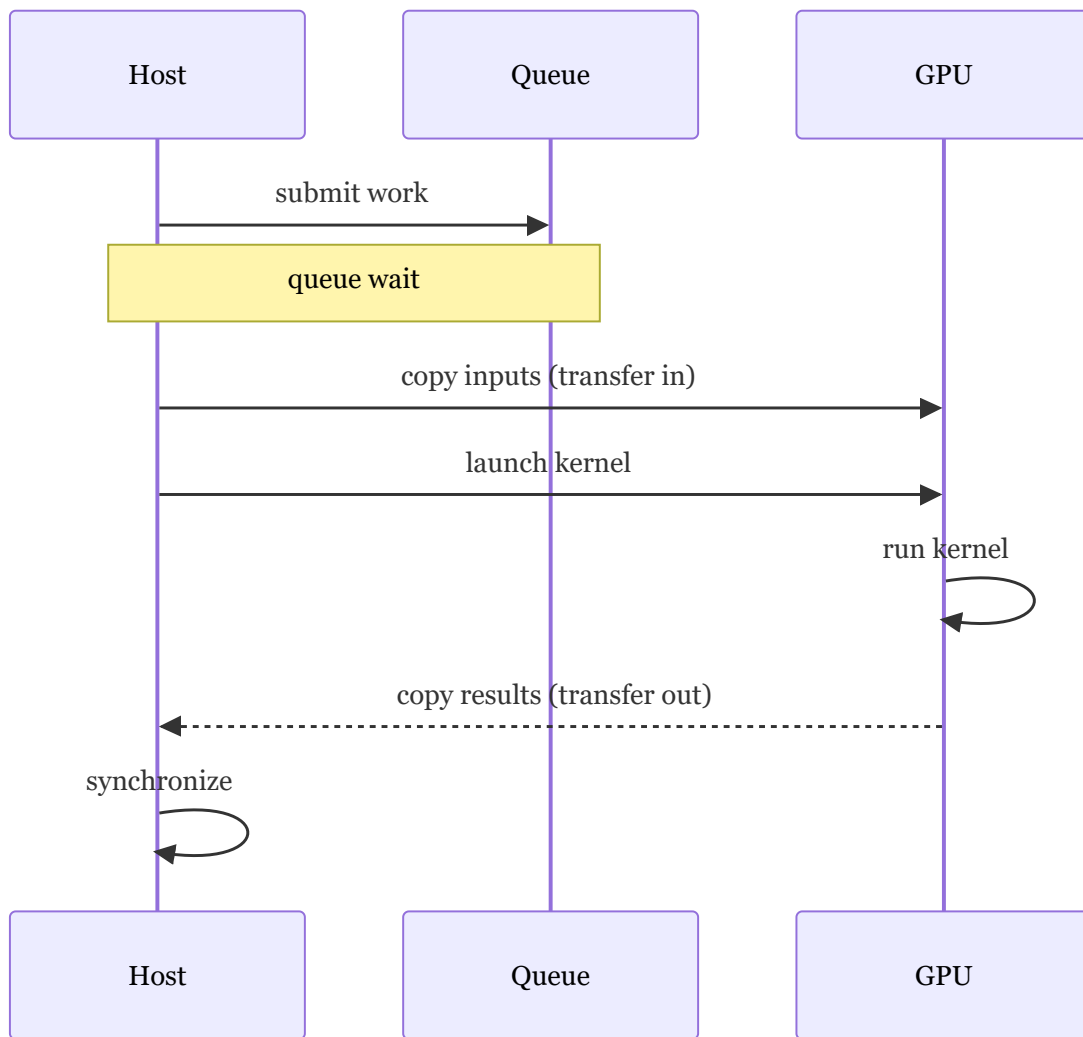
Rust this is the same ownership question from the earlier chapters, just applied to a buffer: hand each worker its own slice of the output and the merge cost disappears. The diagram shows the clean split.



Partition the output by rows so each worker owns its slice; joining needs no shared lock or contended accumulator.

GPU offload

A GPU does not run your kernel in isolation; it sits at the far end of a queue with a copy bus in between. The end-to-end cost of an offload is the sum of waiting for the device, copying inputs across the bus, launching the kernel, running it, and copying results back. A 2 ms kernel wrapped in 18 ms of transfer and synchronization is an 18 ms feature, not a 2 ms one. That is why a kernel benchmark in isolation is so misleading and why the only honest comparison measures each segment separately. The sequence below traces a single offload so you can see where the time actually goes.



Kernel time is one slice of the path; queue wait, transfers, and synchronization often dominate the real latency.

The fastest matrix kernel can still lose the product decision if the path is admission-bound or transfer-bound. The break-even size is usually a service fact, not a whiteboard fact.

How to think about this coming from another language

Every language you have used already made a layout decision for matrices, usually without telling you. The useful question is not which crate replaces which library. It is what mental habit each background brings, which of those habits help here, and which one quietly produces the wrong memory layout if you let it run on autopilot.

Optimization games and challenge tracks

Each track is scored, because the point is not to chase one fastest number but to defend a decision with evidence. The scoring weights reward the parts that engineers most often skip: a clear cache story, an honest memory budget, and benchmark accounting that separates queue wait from transfer from kernel time. Treat the extension lists as ways to make the comparison fairer, not as feature checklists.

Production patterns

Pitfalls and tradeoffs

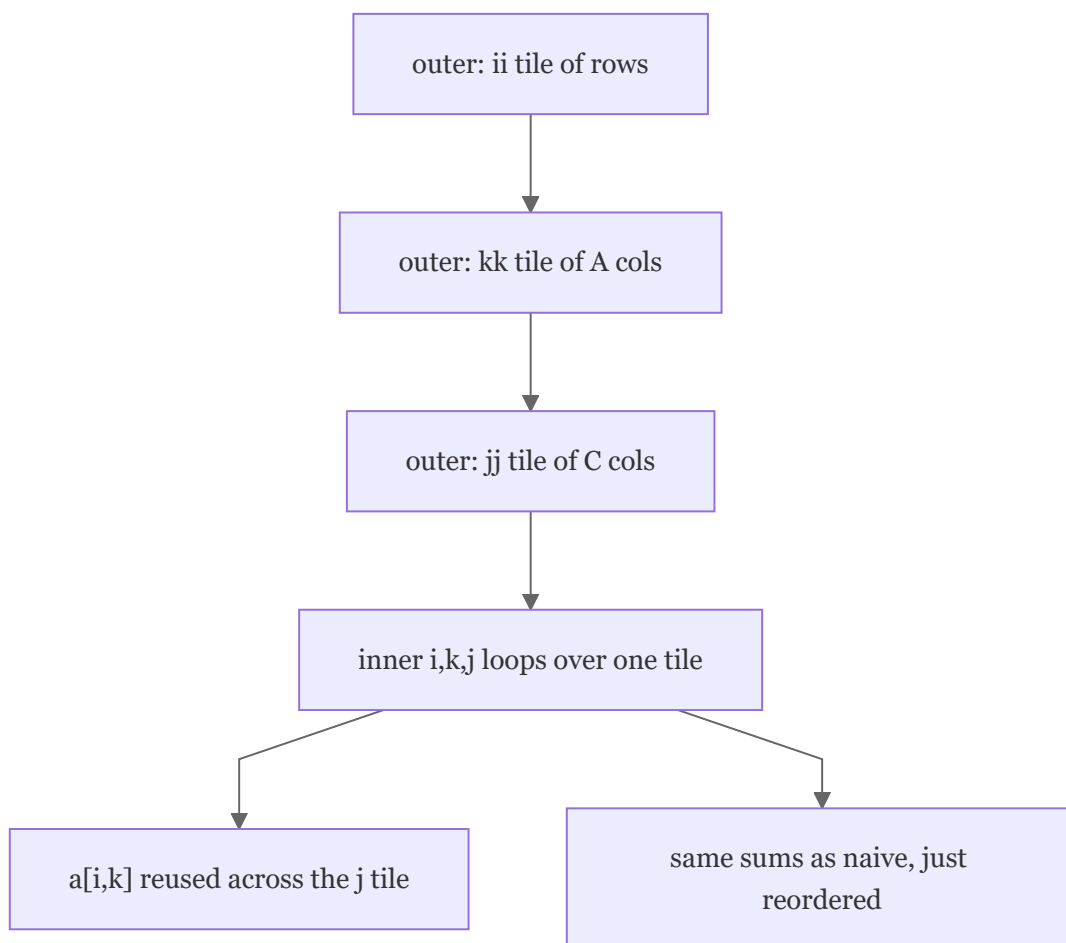
In matrix work, the most expensive mistake is often skipping the layout review and jumping straight to the accelerator review.

Examples

Example 1: row-major dense multiplication with tiling

The example keeps the owner flat, compares one naive multiply against one tiled multiply, and prints one deterministic cell plus a checksum so correctness stays visible while tuning.

What to look at: the Matrix struct stores one flat Vec<f32> with the $\text{offset} = \text{row} * \text{cols} + \text{col}$ formula doing all the 2D-to-1D translation. Then compare the two kernels. `matmul_naive` is the plain triple loop. `matmul_tiled` wraps the same arithmetic in outer `ii / kk / jj` tile loops, then runs short inner loops inside each tile so a block of data stays hot in cache. The diagram shows that nesting; the key check is that `approx_eq` still prints true, meaning the tiling changed locality without changing the result.



Tiling adds three outer loops that carve the work into cache-sized blocks; the inner loops do the identical arithmetic.

Example 40-1. row_major_tiled_matmul.rs — *Tiled matmul*

```

#[derive(Debug, Clone, PartialEq)]
struct Matrix {
    rows: usize,
    cols: usize,
    data: Vec<f32>,
}

impl Matrix {
    fn from_vec(rows: usize, cols: usize, data: Vec<f32>) -> Self {
        assert_eq!(data.len(), rows * cols);
        Self { rows, cols, data }
    }

    fn zeros(rows: usize, cols: usize) -> Self {
        Self {
            rows,
            cols,
            data: vec![0.0; rows * cols],
        }
    }

    fn offset(&self, row: usize, col: usize) -> usize {
        row * self.cols + col
    }

    fn get(&self, row: usize, col: usize) -> f32 {
        self.data[self.offset(row, col)]
    }

    fn set(&mut self, row: usize, col: usize, value: f32) {
        let index = self.offset(row, col);
        self.data[index] = value;
    }
}

fn matmul_naive(a: &Matrix, b: &Matrix) -> Matrix {
    assert_eq!(a.cols, b.rows);
    let mut out = Matrix::zeros(a.rows, b.cols);

    for i in 0..a.rows {
        for j in 0..b.cols {
            let mut acc = 0.0_f32;
            for k in 0..a.cols {
                acc += a.get(i, k) * b.get(k, j);
            }
            out.set(i, j, acc);
        }
    }

    out
}

fn matmul_tiled(a: &Matrix, b: &Matrix, tile: usize) -> Matrix {
    assert_eq!(a.cols, b.rows);

```

```

let mut out = Matrix::zeros(a.rows, b.cols);
let tile = tile.max(1);

let mut ii = 0;
while ii < a.rows {
    let mut kk = 0;
    while kk < a.cols {
        let mut jj = 0;
        while jj < b.cols {
            let i_end = (ii + tile).min(a.rows);
            let k_end = (kk + tile).min(a.cols);
            let j_end = (jj + tile).min(b.cols);

            for i in ii..i_end {
                for k in kk..k_end {
                    let a_ik = a.get(i, k);
                    for j in jj..j_end {
                        let current = out.get(i, j);
                        out.set(i, j, current + a_ik * b.get(k, j));
                    }
                }
            }

            jj += tile;
        }
        kk += tile;
    }
    ii += tile;
}

out
}

fn approx_eq(left: &Matrix, right: &Matrix) -> bool {
    left.rows == right.rows
    && left.cols == right.cols
    && left
        .data
        .iter()
        .zip(&right.data)
        .all(|(l, r)| (l - r).abs() < 0.001)
}

fn checksum(matrix: &Matrix) -> f32 {
    matrix.data.iter().copied().sum()
}

fn main() {
    let a = Matrix::from_vec(
        3,
        3,
        vec![
            1.0, 2.0, 0.0,
            0.0, 1.0, 3.0,
            2.0, 0.0, 1.0,
        ],
    ),

```

```

);

let b = Matrix::from_vec(
    3,
    3,
    vec![
        3.0, 1.0, 2.0,
        2.0, 1.0, 0.0,
        1.0, 4.0, 2.0,
    ],
);

let naive = matmul_naive(&a, &b);
let tiled = matmul_tiled(&a, &b, 2);

println!("naive == tiled = {}", approx_eq(&naive, &tiled));
println!("c[1,2] = {:.2}", tiled.get(1, 2));
println!("checksum = {:.2}", checksum(&tiled));
}

```

OUTPUT

```

naive == tiled = true
c[1,2] = 6.00
checksum = 55.00

```

One flat row-major owner keeps cache reasoning and serialization simple.

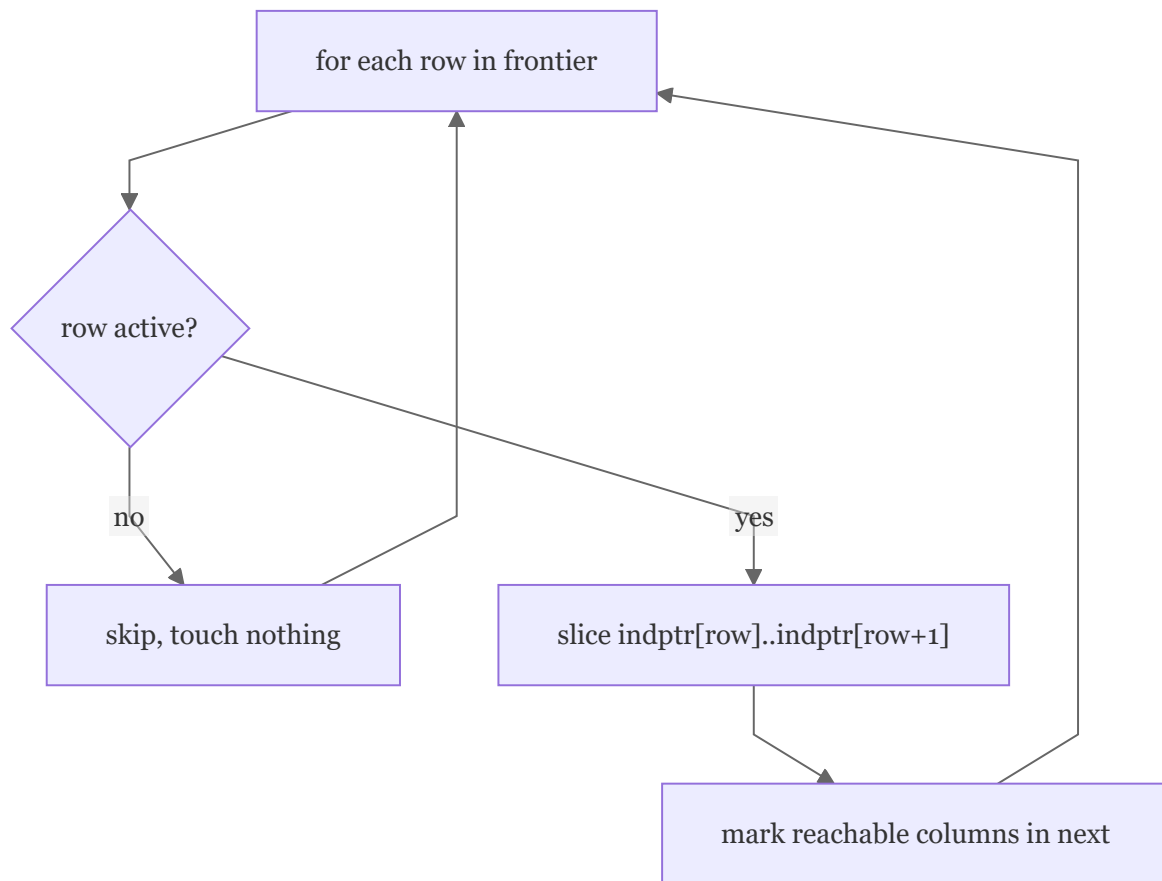
Tile loops turn a large multiply into reuse-friendly subproblems without changing correctness.

Printing equality and checksum keeps optimization grounded in identical results.

Example 2: sparse frontier expansion with CSR

This turns a pathfinding-style frontier step into explicit sparse matrix traversal and prints the dense byte budget for comparison.

What to look at: `advance_frontier` is one breadth-first expansion step expressed as a sparse matrix-vector product. It walks only the rows that are currently active in the frontier, and for each one it reads the slice `indptr[row]..indptr[row + 1]` to find that row's outgoing edges, marking each reachable column in the next frontier. Crucially it never touches a zero cell, so the cost scales with the number of edges, not with rows times columns. The `dense_bytes` print exists to make the tradeoff concrete: the CSR payload here is a handful of values while the equivalent dense matrix would reserve every cell. The diagram traces that per-row skip-and-expand loop.



The frontier step visits only active rows and only their stored edges, so work scales with edges rather than total cells.

Example 40-2. sparse_frontier_csr.rs — Sparse frontier

```

#[derive(Debug)]
struct CsrMatrix {
    rows: usize,
    cols: usize,
    indptr: Vec<usize>,
    indices: Vec<usize>,
    data: Vec<f32>,
}

impl CsrMatrix {
    fn nnz(&self) -> usize {
        self.data.len()
    }

    fn dense_bytes(&self) -> usize {
        self.rows * self.cols * std::mem::size_of::<f32>()
    }
}

fn advance_frontier(csr: &CsrMatrix, frontier: &[f32]) -> Vec<u8> {
    assert_eq!(frontier.len(), csr.rows);

    let mut next = vec![0_u8; csr.cols];

```

```

    for row in 0..csr.rows {
        if frontier[row] == 0.0 {
            continue;
        }

        let start = csr.indptr[row];
        let end = csr.indptr[row + 1];

        for edge in start..end {
            let col = csr.indices[edge];
            if csr.data[edge] != 0.0 {
                next[col] = 1;
            }
        }
    }

    next
}

fn render(bits: &[u8]) -> String {
    bits.iter()
        .map(|bit| bit.to_string())
        .collect::<Vec<_>>()
        .join(",")
}

fn main() {
    let graph = CsrMatrix {
        rows: 5,
        cols: 5,
        indptr: vec![0, 2, 3, 5, 6, 6],
        indices: vec![1, 2, 3, 3, 4, 4],
        data: vec![1.0, 1.0, 1.0, 1.0, 1.0, 1.0],
    };

    let frontier = vec![1.0, 0.0, 1.0, 0.0, 0.0];
    let next = advance_frontier(&graph, &frontier);

    println!("nnz = {}", graph.nnz());
    println!("next frontier = {}", render(&next));
    println!("dense bytes = {}", graph.dense_bytes());
}

```

OUTPUT

```

nnz = 6
next frontier = 0,1,1,1,1
dense bytes = 100

```

CSR pays metadata to skip zeros and to touch only active outgoing edges.

One frontier step over an adjacency matrix is often really sparse pathfinding work in disguise.

Printing dense bytes beside nonzero count makes the representation tradeoff reviewable quickly.

The repository also includes standalone Rust source under `examples/ch40_matrix_optimization_games/` including a small CPU-versus-GPU tournament scoreboard sketch alongside the two in-browser worked

examples.

Summary

PART VI

Engineering Systems for Production

Error strategy at scale, testing, observability, packaging, and a capstone that ties it together.

-
- 41** Error Handling in Large Systems

 - 42** Testing Advanced Rust Systems

 - 43** Observability

 - 44** Packaging and Deployment

 - 45** Capstone: A Distributed Rust System

CHAPTER 41 · ERROR HANDLING IN LARGE SYSTEMS

Error Handling in Large Systems

Result and Option, typed error enums, thiserror and anyhow, context propagation, async and FFI boundaries, logging discipline, and error-contract design for production systems

In a system with many layers, error handling stops being about formatting one message and becomes interface design. This chapter treats Rust errors as production contracts: how to keep domain failures, infrastructure failures, retry policy, and operator context distinct, and how to translate between them only where the meaning actually changes.

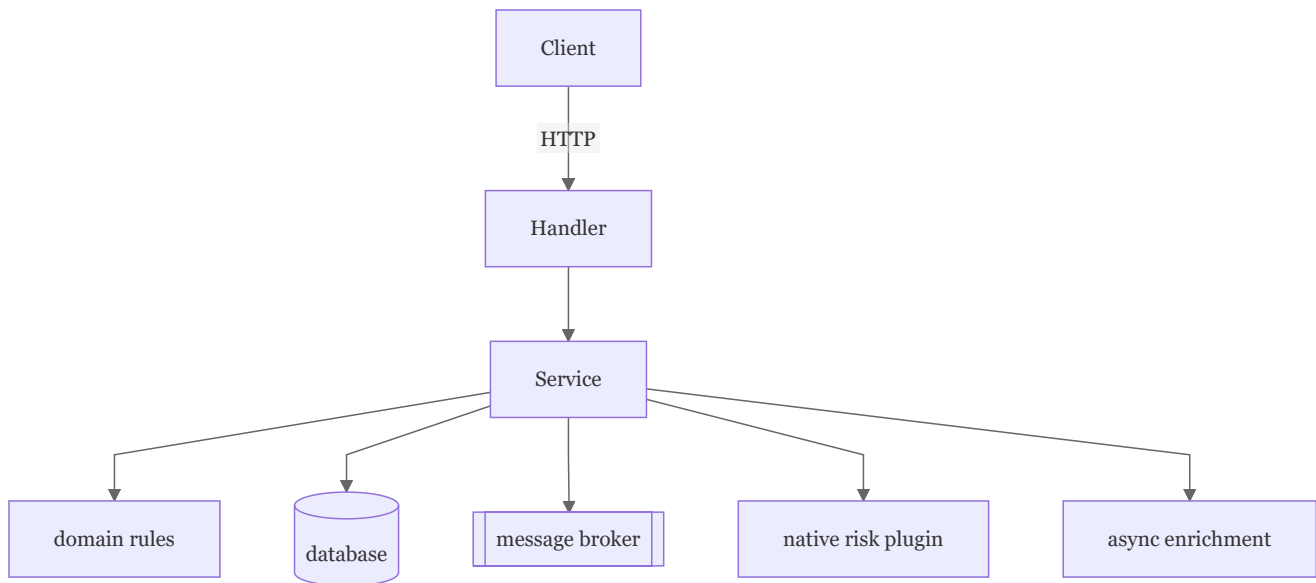
Builds on Chapters 16, 17, 24, 28, 30, and 40

Chapter 16 established domain invariants and domain errors. Chapter 17 introduced Result-first refactoring. Chapter 24 explained async state machines and await boundaries. Chapter 28 covered FFI translation. Chapter 30 covered retries and dead-letter policy at broker boundaries. Chapter 40 reminded us that even specialist subsystems still need clear operational contracts when they fail.

Opening scenario

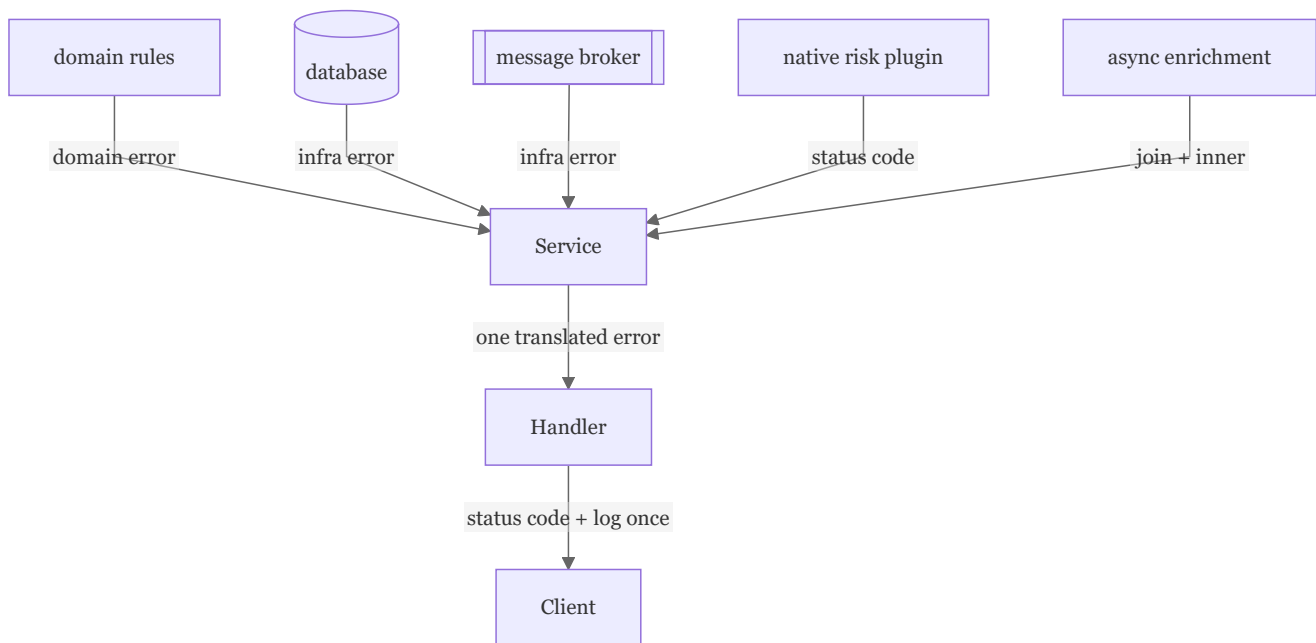
Picture a multi-tenant billing platform. A request arrives over HTTP, the service publishes some internal work to a broker, it calls a native risk-scoring plugin over a C ABI, and it fans out a handful of async enrichment tasks before answering. A single charge can therefore fail in five different vocabularies at once: a business rule can reject it, a database row can be missing, the broker can be unreachable, the native plugin can return a status code, and a spawned task can panic. The job of this chapter is to keep those vocabularies from bleeding into each other.

The requirement the team agrees on is one error policy per boundary. Domain errors stay in domain language and never mention SQL or TCP. Infrastructure errors stay actionable so a caller can retry or dead-letter. Retries are classified explicitly rather than guessed from a message. And every failure is logged exactly once, at the layer where it becomes operationally meaningful, with stable identifiers an operator can correlate later. The diagram below shows where those translation boundaries sit.



Request path: one charge fans out from the service into five dependencies, each speaking its own vocabulary.

Each of those five dependencies can fail differently. The return path collapses those failures back into a single caller-facing contract:



Return path: the service is the single seam where five vocabularies are translated into one caller-facing contract and logged once.

A practical decision order

1. Ask what the caller can still do: retry, reject, translate, or crash.
2. Keep Option local unless absence itself has become a contract violation.
3. Keep typed errors close to the layer that still understands them.
4. Add context at IO, task, queue, and FFI boundaries where meaning changes.
5. Log once, with structure, at the boundary that makes the event operationally relevant.

The in-browser editors simulate output so you can focus on contract shape. In a real Cargo project, the worked examples use ecosystem crates such as `thiserror`, `anyhow`, and `tokio`.

At a glance

- Choose `Option`, `Result`, or `panic` from the caller's next action, not from style preference.
- Keep domain errors, infrastructure errors, and transport-facing errors distinct until the contract truly changes.
- Add context where meaning changes: IO, task joins, queues, FFI, and process boundaries.
- Log once at the boundary that makes the event operationally meaningful.

Caller-action checklist

- Should the caller `retry`, `reject`, `translate`, `dead-letter`, or `crash`?
- Does the next layer still need typed classification, or is rich top-level context enough?
- Is this failure local absence, recoverable error, or broken invariant?
- Which stable IDs must travel with the error so an operator can reconstruct the incident later?

How to think about this coming from another language

Most senior engineers do not arrive at Rust error handling with a blank slate; they arrive with an exception habit, a runtime habit, or an `err-string` habit. The shift that matters is not which crate to import. It is where failure becomes visible and who is responsible for deciding what happens next. Find your background below and read the trap as much as the rule.

Mental model

Core concepts

Choosing between `Option`, `Result`, and a typed enum

These three are not interchangeable styles; they encode three different statements about a failure. An `Option` says a value may simply be absent and that absence is unremarkable. A `Result` says the operation can fail in a way the caller is expected to handle. A typed error enum goes one step further and says the caller may need to tell those failures apart. The common repair in real code is to keep an `Option` local and promote it into a typed `Result` at exactly the boundary where missing data stops being routine and becomes a contract violation.

A reliable large-system rule is simple: if the caller needs classification, use a typed error. If the caller only needs one local branch on absence, `Option` is often enough. Promote absence into a typed `Result` exactly where the contract becomes meaningful.

When to reach for `thiserror` and when to reach for `anyhow`

These two crates are not competitors; they answer different questions and most production systems use both. The deciding factor is whether a downstream layer still needs to branch on the error. If it does, you

want a named enum with named variants, and this error generates the tedious Display and From boilerplate while keeping the structure intact. If the only remaining job is to attach context and surface or log the failure, a single opaque anyhow::Error with a rich context chain is lighter and just as honest.

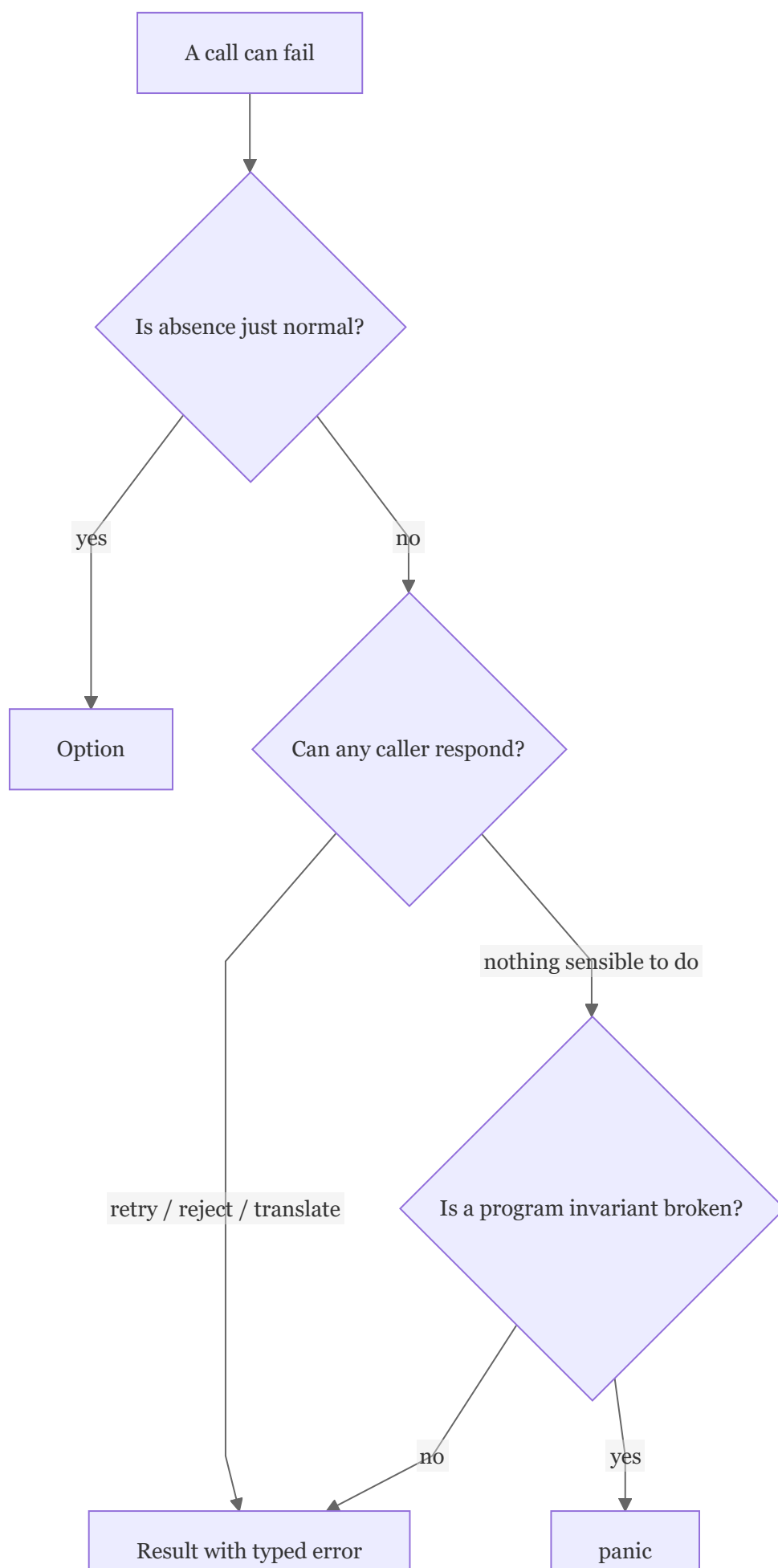
This is the usual operational split: typed enums when a downstream layer still branches on the error, rich context wrappers when the boundary is already top-level orchestration and the next action is mostly logging, surfacing, or exiting.

Adding context without burying the source

Context is the answer to the question an operator will ask at 3 a.m.: what was the system trying to do when this failed? A low-level error like "file not found" is true but useless on its own; wrapped with "loading worker config" it becomes diagnosable. The discipline is to add context at the moments where meaning changes and to add it once, never re-stating the same low-level text at every frame on the way up.

Recoverable versus unrecoverable failures

The dividing line is not how serious the failure feels; it is whether any caller, anywhere up the stack, has a sensible response. Bad input, a rejected business action, or one flaky dependency all have responses, so they belong in a Result. A violated invariant has no sensible response because the program is now in a state it was designed never to reach, and that is the narrow place a panic is correct. The decision usually flows like this.

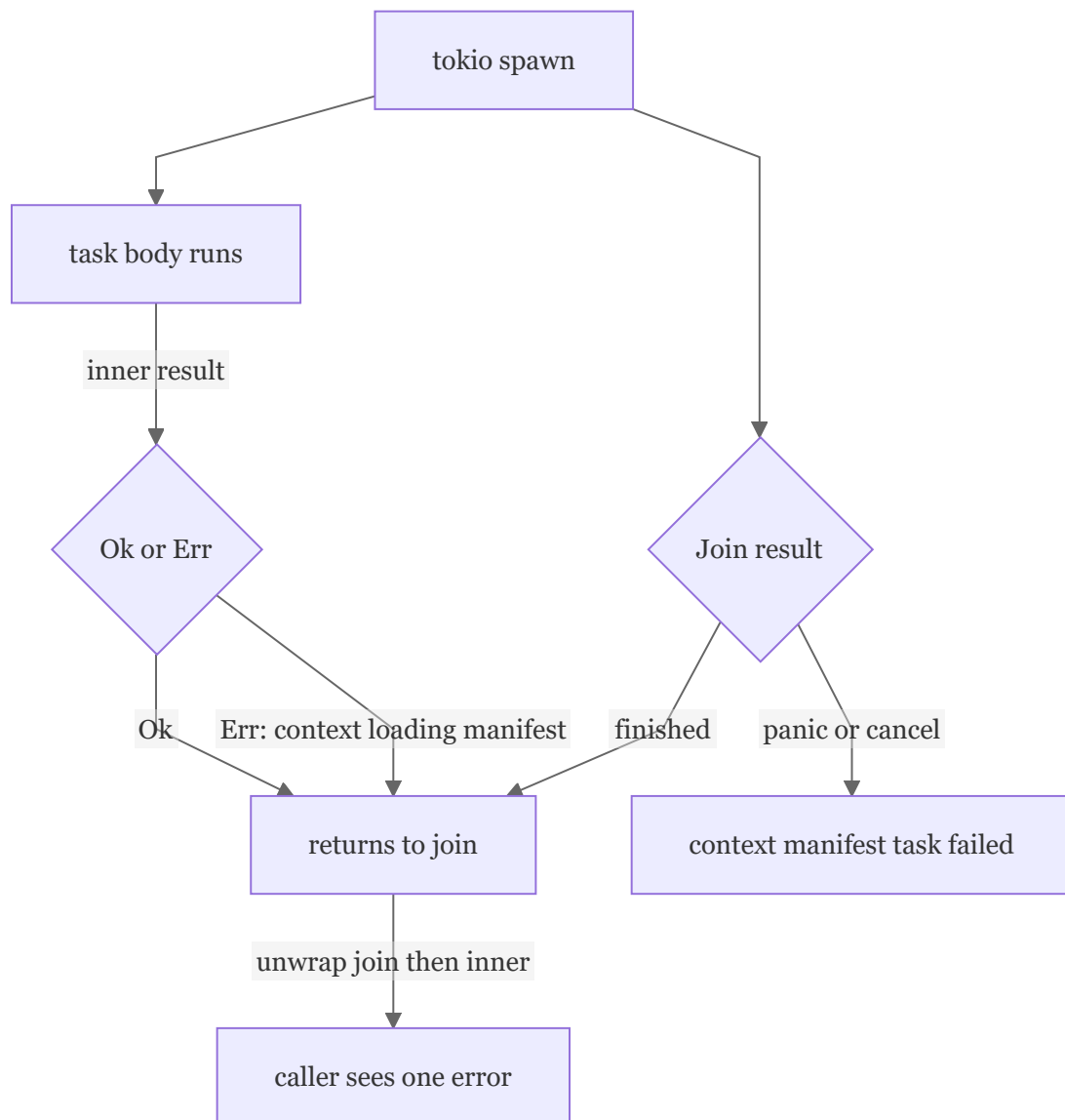


Route by the caller's options, not by how alarming the failure sounds. Panic is the dead end reserved for broken invariants.

A panic is not Rust's version of a normal business exception. It is the language saying the program's own assumptions are broken badly enough that continuing may be wrong.

Errors that cross an async task boundary

When you spawn a task, you create two stacked failure layers, and a lot of bugs come from forgetting the outer one. The inner layer is whatever the task body returns: an IO error, a domain rejection, an `anyhow::Error`. The outer layer is the join result, which tells you whether the task even finished normally or instead panicked or was cancelled. A panicked task is a different operational event from a missing file, and collapsing them into one string hides exactly the distinction an operator needs. The snippet below is worth reading carefully: notice the two separate `.context(...)` calls and the double `??` at the end, which unwraps the join layer first and then the inner layer.

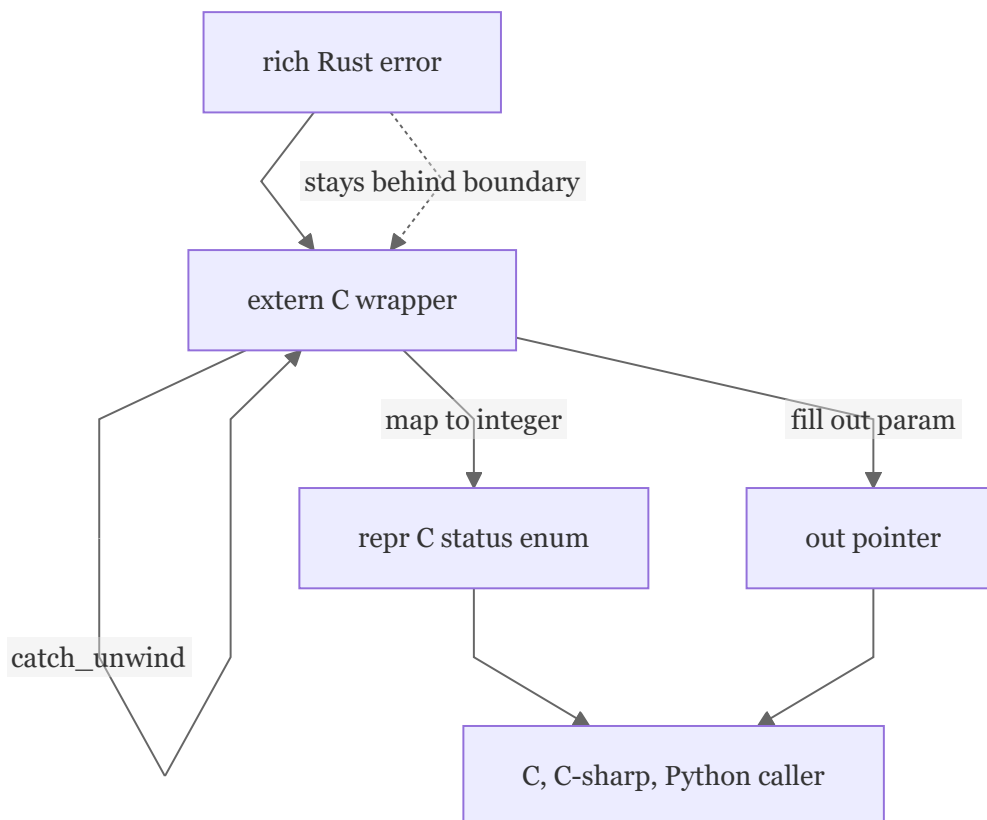


A spawned task has a join layer and an inner layer; the double ?? unwraps both, and each gets its own context.

The key lesson is that async failures usually have a task layer and an operation layer. Handle both deliberately instead of collapsing them into one generic timeout or one generic string.

Errors that cross a C ABI boundary

A C ABI cannot carry a Rust enum, an `anyhow::Error`, or the unwinding that a panic implies; letting any of those escape across the boundary is undefined behavior, not a missed opportunity for richness. So the boundary becomes a deliberate narrowing: the rich error stays on the Rust side, and the foreign caller receives the smallest contract it can actually honor, usually a small `#[repr(C)]` status enum plus out-parameters. In the code below, look at the Status enum: each variant is an integer the other runtime can switch on, and the detailed Rust reason never leaves the wrapper.



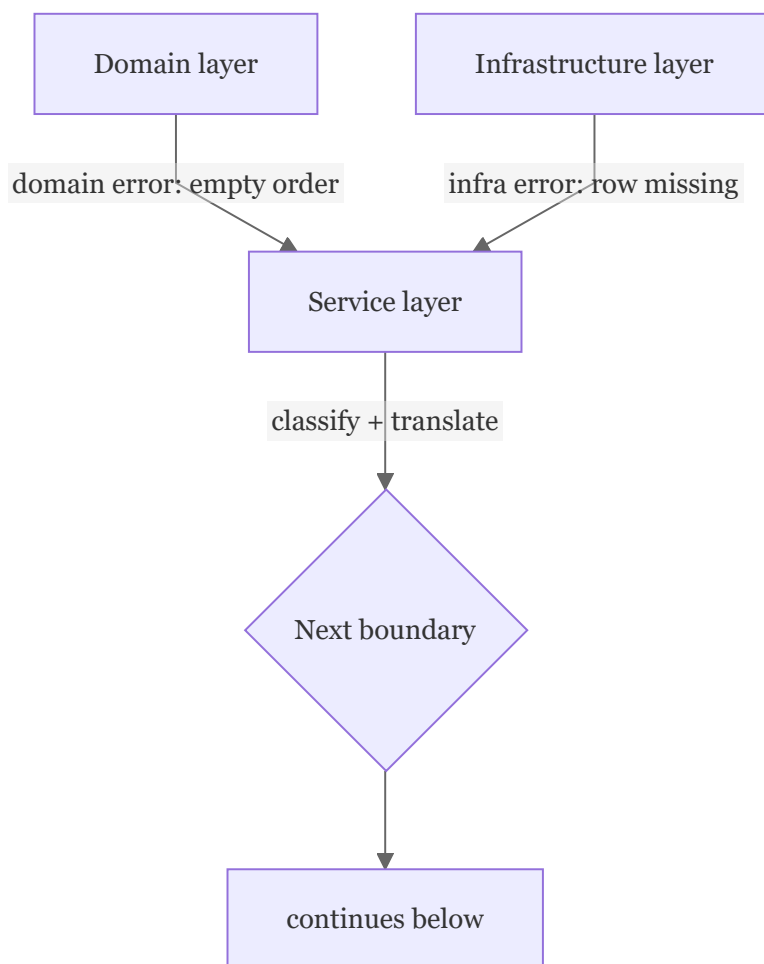
The detailed error stays on the Rust side; the foreign caller sees only a status code and out-parameters.

This is deliberately boring. That is a virtue. The richer Rust-side error chain should stay behind the wrapper where another Rust layer can still use it safely.

The `#[unsafe(no_mangle)]` attribute-wrapper syntax assumes the Rust 2024 edition (rustc 1.82 or newer). On older toolchains, write the bare `#[no_mangle]` attribute instead.

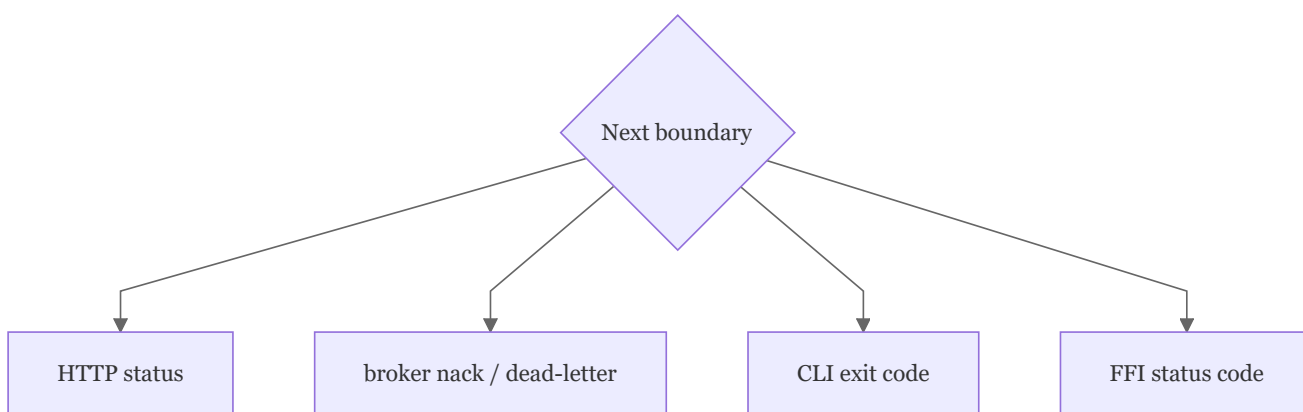
Keeping domain errors and infrastructure errors apart

This is the single distinction that keeps a large error model from rotting. A domain error speaks the language of the business: an order is empty, a transition is invalid, a quota is exhausted. An infrastructure error speaks the language of the machinery: a row is missing, a connection was refused, a task join failed. The two stay separate as they travel up, and the service layer is the one place that combines them and decides how the next boundary should see the result, whether that is an HTTP status, a broker nack, or a CLI exit code. The layering looks like this.



Domain and infrastructure errors stay distinct until the service layer, where classification becomes operational policy.

From that single classify-and-translate point, the service shapes the failure for whichever boundary comes next:



The one classification point fans out into a per-boundary contract: HTTP status, broker nack, CLI exit code, or FFI status code.

Logging a failure once, at the right layer

The most common logging mistake in a layered system is not too little logging; it is the same failure logged five times on its way up, each line slightly reworded, until the incident channel is a hall of mirrors. The cure

is to pick the one layer where the failure becomes operationally meaningful, log it there with structured fields, and let every layer below simply return the error. Severity should track the caller's reality too: a rejected business command is not an error-level event just because it came back as a Result.

Designing the error contract you expose to callers

A public error contract is an API surface, and it ages like one. The discipline is to expose only what the caller can act on, to keep the retryable-versus-terminal distinction visible somewhere other than a substring of a message, and to resist the catch-all error bag that means nothing in particular. When the contract has to stay stable across teams or languages, translate your internal crates and variants at the boundary rather than leaking them, because once another runtime depends on a variant you can no longer freely rename it.

Production patterns

Pitfalls and tradeoffs

The hardest production mistake is often not one missing match arm. It is a failure contract that never said which layer owned retry, which layer owned translation, and which layer should have logged the incident only once.

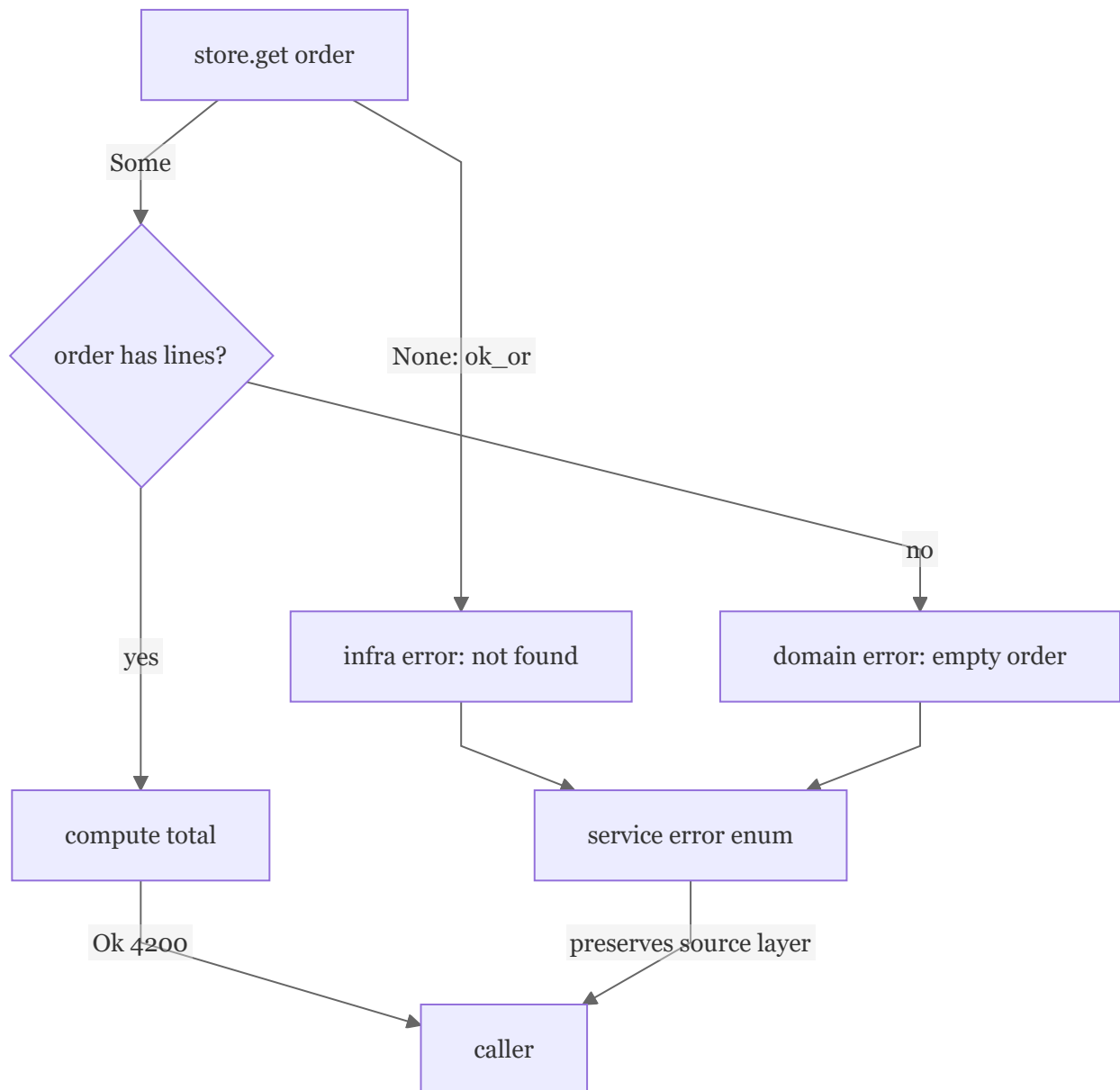
Examples

Both editors simulate their output so you can focus on the shape of the contract rather than on toolchain setup. Read the short pointer and the diagram above each listing first, then run the code and try changing the inputs to see how the error path reshapes.

Example 1: separate domain and infrastructure errors with typed contracts

The service boundary preserves domain language for invalid business state and infrastructure language for missing storage state, then translates both into one service-facing error enum.

What to look at: trace one charge through the function. The store lookup starts life as an Option, gets promoted into an infrastructure error when the row is missing, while the empty-order check raises a domain error in business language. Both reach the service layer, which folds them into a single service-facing enum without erasing which layer actually failed. The flow is below; the code follows.



Absence becomes an infra error; an invalid order becomes a domain error; the service enum carries both without losing which layer failed.

Example 41-1. domain_and_infrastructure_errors.rs — Typed contracts

```

use std::collections::HashMap;
use thiserror::Error;

#[derive(Debug, Clone)]
struct Order {
    id: &'static str,
    lines: Vec<u64>,
}

#[derive(Debug, Error)]
enum DomainError {
    #[error("order {order_id} has no lines")]
    EmptyOrder { order_id: &'static str },
}

```

```

#[derive(Debug, Error)]
enum InfraError {
    #[error("order {order_id} not found in store")]
    MissingOrder { order_id: &'static str },
}

#[derive(Debug, Error)]
enum ServiceError {
    #[error(transparent)]
    Domain(#[from] DomainError),
    #[error(transparent)]
    Infra(#[from] InfraError),
}

fn load_order<'a>(
    store: &'a HashMap<'static str, Order>,
    order_id: &'static str,
) -> Result<'a Order, InfraError> {
    store
        .get(order_id)
        .ok_or(InfraError::MissingOrder { order_id })
}

fn total_cents(order: &Order) -> Result<u64, DomainError> {
    if order.lines.is_empty() {
        return Err(DomainError::EmptyOrder {
            order_id: order.id,
        });
    }

    Ok(order.lines.iter().copied().sum())
}

fn bill_order(
    store: &HashMap<'static str, Order>,
    order_id: &'static str,
) -> Result<u64, ServiceError> {
    let order = load_order(store, order_id)?;
    Ok(total_cents(order)?)
}

fn main() {
    let mut store = HashMap::new();
    store.insert(
        "ord-ok",
        Order {
            id: "ord-ok",
            lines: vec![1200_u64, 3000],
        },
    );
    store.insert(
        "ord-empty",
        Order {
            id: "ord-empty",
            lines: vec![],
        },
    );
}

```

```
);

println!("ok = {}", bill_order(&store, "ord-ok").unwrap());
println!("missing = {}", bill_order(&store, "ord-missing").unwrap_err());
println!("empty = {}", bill_order(&store, "ord-empty").unwrap_err());
}
```

OUTPUT

```
ok = 4200
missing = order ord-missing not found in store
empty = order ord-empty has no lines
```

The store lookup is locally an `Option`, then promoted into a typed infrastructure error at the service boundary.

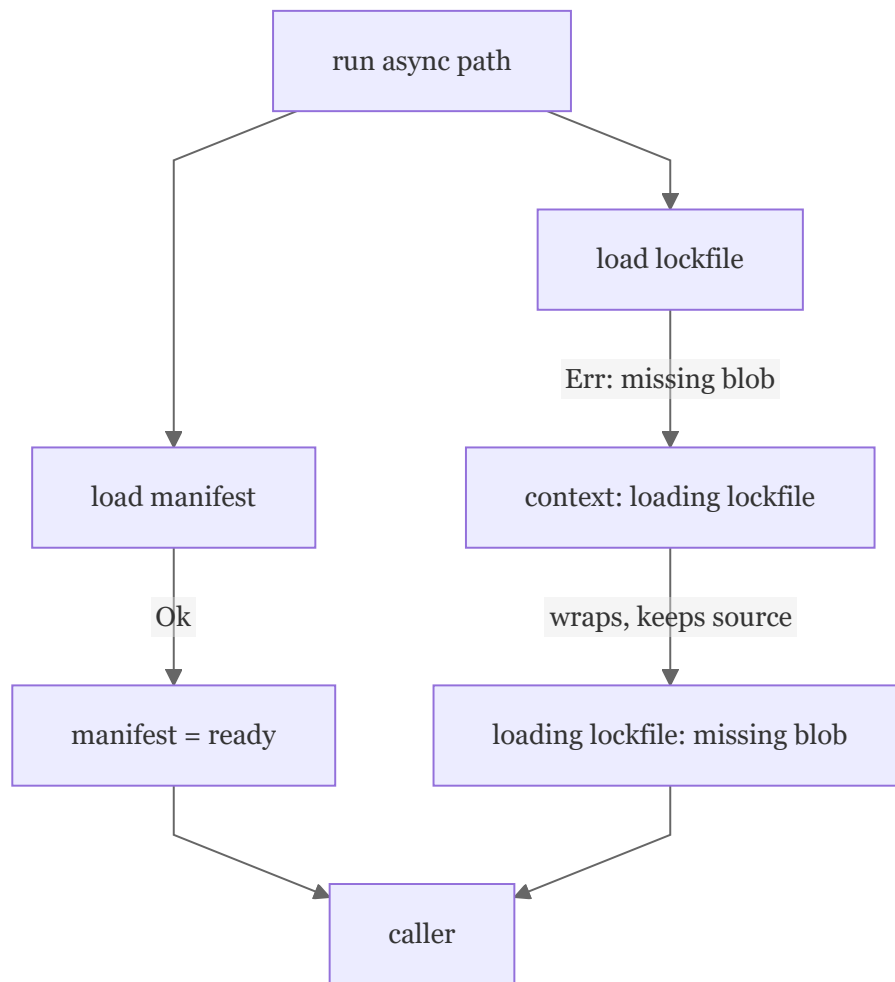
Domain and infrastructure failures stay distinct enough that the next layer can still map or retry them honestly.

The caller sees one service error type, but the error chain still preserves which layer actually failed.

Example 2: add context across async boundaries without losing the source error

The top-level async path uses `anyhow` for orchestration context, while the underlying IO error still survives inside the chain. The spawned task boundary is handled explicitly.

What to look at: there are two paths through this code. The manifest path succeeds and returns `ready`. The lockfile path fails at the IO layer with missing blob, and the `.context("loading lockfile")` call wraps it so the final message reads as a chain rather than a bare cause. Notice the original IO reason is still visible at the tail; context adds meaning, it does not replace the source.



The success path returns a value; the failure path attaches operation context while the underlying IO reason survives in the chain.

Example 41-2. `async_context_propagation.rs` — Async context

```

use anyhow::{Context, Result};
use std::io::{self, ErrorKind};

async fn read_artifact(name: &'static str) -> Result<&'static str> {
    match name {
        "manifest" => Ok("ready"),
        _ => Err(io::Error::new(ErrorKind::NotFound, "missing blob").into()),
    }
}

#[tokio::main]
async fn main() -> Result<>> {
    let manifest = tokio::spawn(async {
        read_artifact("manifest")
            .await
            .context("loading manifest")
    })
    .await
    .context("manifest task failed")??;

    let failed = read_artifact("lockfile")

```

```
.await
.context("loading lockfile")
.err()
.map(|err| format!("{:?}", err))
.unwrap_or_else(|| String::from("none"));

println!("manifest = {}", manifest);
println!("failed = {}", failed);

Ok(())
}
```

OUTPUT

```
manifest = ready
failed = loading lockfile: missing blob
```

The application shell can keep one rich error channel because the next action is mostly context, logging, and exit or translation.

The join layer and the inner operation layer are both visible, so a panicked task would not be confused with a missing artifact.

The context tells operators which operation failed without replacing the underlying IO reason.

The repository also includes standalone Rust source under `examples/ch41_error_handling_in_large_systems/` including a small FFI status-boundary example alongside the two in-browser worked examples.

Summary

CHAPTER 42 · TESTING ADVANCED RUST SYSTEMS

Testing Advanced Rust Systems

Unit tests, integration tests, property testing, fuzzing, unsafe and async testing, distributed-system harnesses, golden files, snapshot testing, and benchmark regression budgets

Testing an advanced Rust system is less about test count and more about defending the right invariants at the cheapest layer that can still prove them — across parsers, concurrency, IO, time, and failure. This chapter organizes a test suite around production risk and determinism rather than line or command coverage.

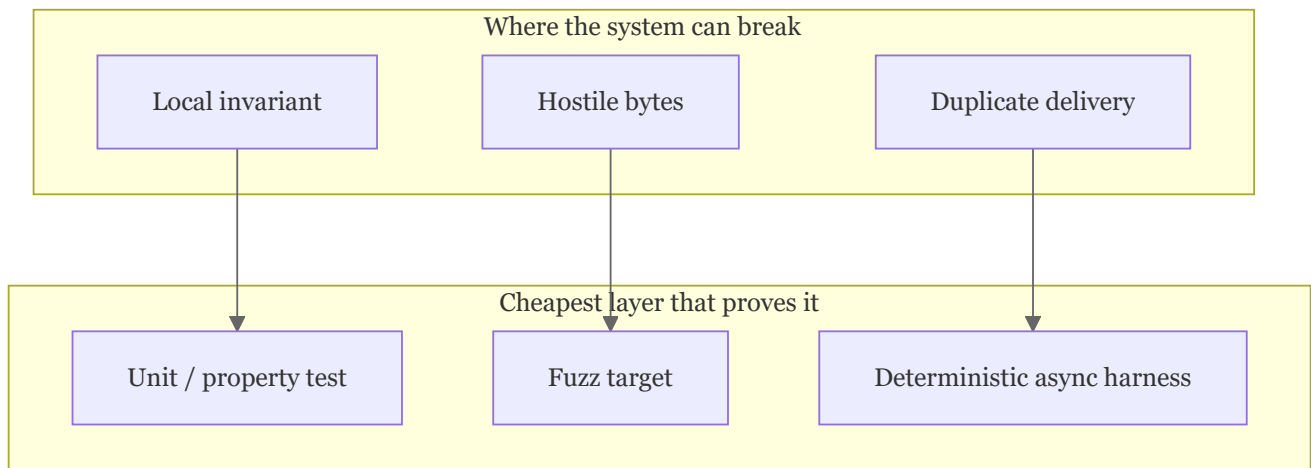
Builds on Chapters 08, 19, 24, 25, 31, and 41

Chapter 08 covered unsafe invariants and FFI boundaries. Chapter 19 covered stable wire contracts and fixture-friendly DTOs. Chapters 24 and 25 covered async and Tokio. Chapter 31 covered replay, leases, and distributed work. Chapter 41 separated domain, infrastructure, and transport error contracts. This chapter uses all of them.

Opening scenario

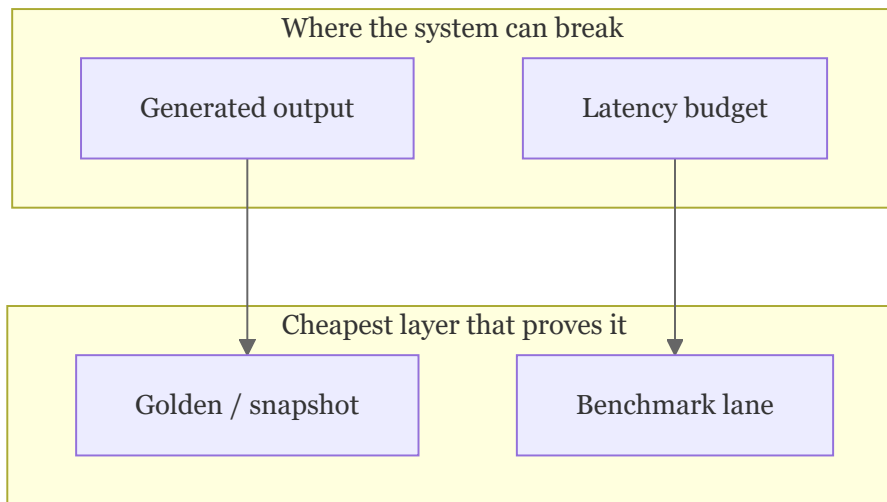
Picture one service that has accumulated every hard-to-test ingredient at once. It has an unsafe parser fast path that walks raw bytes, async workers that process those frames, distributed replay where the same message can arrive twice, generated reports that another team reviews byte-for-byte, and a latency budget that the product has promised to customers. A single end-to-end suite that boots the whole thing and pokes it from the outside would be slow, flaky, and almost useless when it fails: a red build would tell you something broke, not which contract broke.

The job for the rest of this chapter is to take that one tangled service and break its risks apart into separate, mostly deterministic claims — local invariants, hostile-input handling, retry safety, output stability, and performance budgets — and then prove each claim at the cheapest layer where it can be proven reliably. The diagram below shows that mapping: a single risk in the system on the left, the test layer that owns it on the right.



Part 1 — invariants, hostile input, and duplicate delivery each push down to a cheap, deterministic layer.

The same mapping continues for output stability and performance:



Part 2 — generated output and latency budgets move to snapshot review and an isolated benchmark lane, instead of being tested only through one slow end-to-end path.

In a real Cargo repository this layering is not abstract — it shows up as concrete files and crates: ordinary unit-test modules next to the code, an `tests/` directory for integration suites, a property-testing crate such as `proptest`, fuzz targets driven by `cargo-fuzz`, snapshot tooling such as `insta`, and a benchmark lane kept well away from the correctness suite.

At a glance

- Choose the cheapest test layer that can prove the real claim.
- Use property tests for invariants, fuzzing for hostile inputs, and integration tests for public seams.
- Make time, retries, cancellation, and queueing deterministic before calling async or distributed tests “done.”

- Keep snapshots, golden files, and benchmark budgets disciplined enough that reviewers can still trust them.

Proof questions for the suite

- Which invariant can be proven below the transport layer?
- Which boundary is hostile enough to deserve fuzzing or unsafe-edge tests?
- What nondeterminism must be removed before the result is reviewable?
- Which claims belong in performance lanes rather than in correctness assertions?

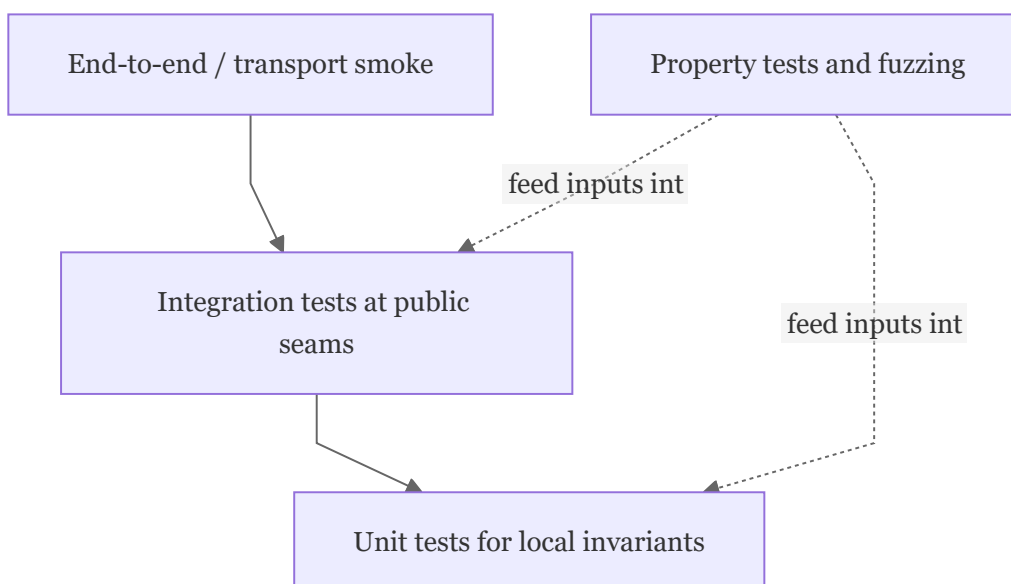
Mental model

Three ideas carry most of the weight in this chapter. Name the claim each test defends, prove it at the cheapest layer that can, and engineer determinism instead of hoping for it. Everything else — which crate, which macro, which directory — is downstream of getting these three right.

Core concepts

The shape of a layered suite

The familiar “test pyramid” is really a statement about cost and stability. Cheap, fast, deterministic tests live at the bottom and there are many of them; expensive, slower, more fragile tests live at the top and there are few. Property tests and fuzzing sit off to the side: they are not a separate altitude so much as a different way of generating inputs for the bottom and middle layers. Read the next diagram as a budget — most of your assertions should be near the base, and each step up should be justified by a claim that genuinely needs that much machinery to be true.



Most assertions live at the cheap base; property tests and fuzzing feed generated inputs into the lower layers rather than forming a separate top tier.

Unit tests

Unit tests are the right layer for local invariants, pure helpers, edge conditions, and typed domain rules. They should prove things like “quota never exceeds limit,” “state transition rejects duplicate submission,” or “parser header length is checked before decoding.” This is usually the cheapest place to keep a failure precise: when a unit test goes red, the blast radius is one function or one type, so the message points almost directly at the broken line instead of at incidental wiring.

In the snippet below, look at the two assertions on the same ID. The first insert must succeed and the second must fail — that pair is the entire contract of a duplicate-rejecting index, stated without booting anything around it.

Integration tests

Integration tests should exercise public seams the way another crate or another deployment unit does: HTTP handlers driven through a test app, an adapter and its store doing a real round-trip, or a queue-consumer shell wrapped around an owned envelope. The discipline that keeps them useful is restraint. They are not the place to re-prove every tiny invariant from scratch — that work belongs in the cheap layers below. An integration test should assert the things that only become true once the pieces are wired together, such as “a successful checkout returns 202 and the outbox now contains the order.”

In the example, the interesting lines are the two assertions after the request. The status code proves the public HTTP contract; the `outbox_contains` check proves the side effect crossed the seam. Everything before them is just enough setup to make those two claims meaningful.

Property-based testing

A property test inverts how you normally write assertions. Instead of supplying a specific input and checking a specific output, you state a rule that must hold for every input in some range, and the framework generates hundreds of cases trying to break it. Properties shine when the important claim is general: no negative balance, output always sorted, a parser round-trip is lossless, resource accounting never exceeds the limit, or retry bookkeeping stays idempotent under repeated input.

The random data is not the valuable part on its own — thirty random runs that all pass prove little. The valuable part is shrinking: when a property fails, `proptest` automatically reduces the failing case to a minimal counterexample, so you get the smallest input that violates the rule rather than a 4 KB blob you still have to debug. In the snippet, the loop body is the whole test: after every reservation, used quota must stay at or below the limit, no matter which sequence of quantities the generator picked.

Fuzzing

Fuzzing is property testing pointed at openly hostile input. It is the right tool for parsers, decoders, compression wrappers, protocol frames, unsafe byte walkers, and FFI boundaries — anywhere arbitrary bytes from outside cross into your code. A coverage-guided fuzzer such as `cargo-fuzz` mutates inputs and watches which branches they reach, steadily steering toward the paths your hand-written cases never thought to exercise. The implicit property is usually the simplest one possible: “whatever bytes arrive, this function must not panic, hang, or trigger undefined behavior.”

Two habits make fuzzing pay off. Target the smallest boundary that still reproduces the failure, so a crash points at one parser rather than the whole service. And save crashing inputs into a corpus so each fix comes with a permanent regression case. The harness below is deliberately tiny — that minimalism is the point, because a boring target keeps the crash cause diagnosable.

Testing unsafe code

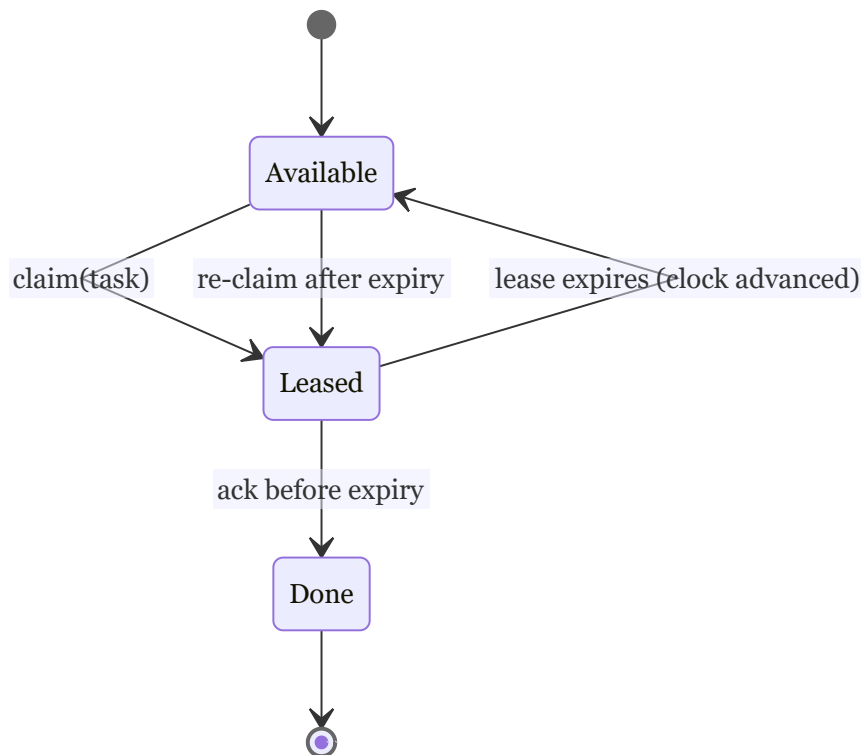
Unsafe code is different because the bug you fear may not be observable from a passing test. A use of an invalid pointer or a violated aliasing rule is undefined behavior: the program might return the right answer today and corrupt memory next week after the optimizer makes a different choice. So testing unsafe code is not about throwing more inputs at it — it is about pinning down the invariant the unsafe block relies on, and then checking that invariant with tools that can see undefined behavior even when ordinary assertions cannot. The checklist below is the order that tends to work: write the invariant first, then keep a safe reference, then hammer the boundaries, then bring in Miri and sanitizers.

Unsafe code is not “well tested” because one happy-path integration test passed. It is well tested only when the test suite covers the invariant edges where UB risk actually lives.

Testing async code

The recurring temptation in async tests is to express “after some time, X should happen” with a real sleep. That makes the test slow and, worse, makes it depend on scheduler luck, so it passes on a fast laptop and flakes in CI. The fix is to take time out of the operating system’s hands and put it in the test’s hands. Tokio’s `start_paused = true` freezes the clock so it only advances when you advance it, which turns “wait 31 seconds for a lease to expire” into an instantaneous, exact, repeatable step. The same instinct applies to queueing, cancellation, and task ownership: control them deliberately rather than hoping the runtime cooperates. For genuinely concurrent state machines, loom can later explore interleavings, but a deterministic boundary comes first.

The test below exercises a lease that should expire and let the task be reclaimed. The behavior under test is a small state machine, so it is worth seeing the states before the code. Watch how the test never sleeps for real: it claims the task, advances the fake clock past the lease, and then claims again, expecting the same task to come back.

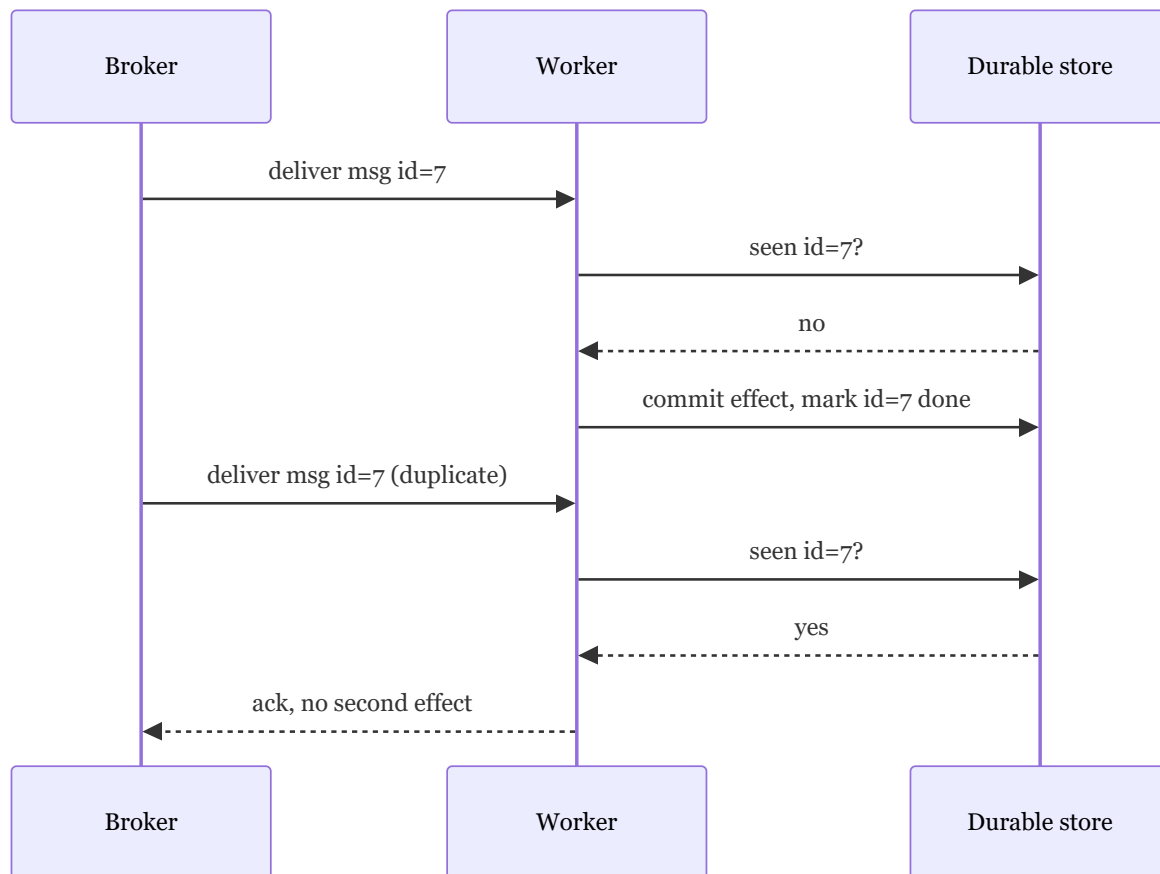


A leased task returns to Available when the (fake) clock passes the lease deadline, so a re-claim redelivers the same task — the exact path the test drives.

Testing distributed systems

In a distributed system the connection opening is the boring part. What actually keeps you up at night is everything that happens because most brokers deliver at least once: the same message can arrive twice, a lease can expire and redeliver mid-flight, a retry can re-run work that already half-succeeded. So the claims worth testing are replay safety, duplicate suppression, retry classification, queue age, and durable completion — not “a broker connection opened.” Keep the transport reality in one small top layer if the repo already supports it, and prove the hard behavior below it with fake time and owned envelopes, where it is deterministic.

The single most important property is usually idempotency under duplicate delivery. The sequence below is the one your test should drive directly: the same message ID is delivered twice, but the durable effect happens exactly once because the worker checks a completion record before acting. If your harness can produce this flow and assert “one effect, one duplicate suppressed,” you have tested the contract that matters.



At-least-once delivery means id=7 can arrive twice; the completion check in the durable store makes the second delivery a no-op. This is the flow a distributed test should reproduce.

Golden files

A golden-file test compares the program’s output against a known-good artifact checked into the repository. It shines when the output is an external contract someone actually cares about byte-for-byte: a protocol frame, a normalized CLI dump, compiler-style diagnostics, or a rendered document another team consumes. The point is that a change to that output should be a deliberate, reviewable event — a diff against the golden file in a pull request — rather than something that drifts silently.

The snippet is the whole pattern in three lines: render the artifact, read the committed golden bytes, compare. The only subtlety lives in what you render — anything nondeterministic (timestamps, UUIDs, paths) must be normalized before the comparison, or the test becomes noise the team learns to ignore.

Snapshot testing

Snapshot testing is the ergonomic cousin of golden files. A crate such as `insta` captures the output on the first run, stores it next to the test, and on later runs shows you a diff and lets you accept or reject the change with a review command. It is the right tool when the output is too large to hand-write assertions for but still small enough that a human can eyeball the diff: CLI reports, error trees, generated code, formatted documents.

The one-line call in the snippet hides the workflow, not the risk. A snapshot proves “the output looks like this,” which is weaker than “the output is correct.” Treat it as a review aid for shape and pair it with a real semantic assertion for any load-bearing invariant — accepting a snapshot diff without thinking is how a genuine regression slips through.

Benchmark regression tests

Benchmark regressions belong in their own lane. They are less deterministic than correctness tests, more sensitive to hardware and build profile, and only useful when the budget itself is meaningful to the product. Keep them coarse, stable, and obviously tied to one service budget. When a budget guard is expressed as an ordinary `#[test]`, mark it `#[ignore]` (or gate it behind a feature flag) so it runs in an isolated performance job on a dedicated runner, not in the shared correctness suite. A Criterion or `[[bench]]` harness is the cleaner home once the budget earns one.

In the snippet, the two things to notice are the `#[ignore]` attribute that keeps this out of the normal correctness run, and the comparison against a single coarse budget (`p95_us <= 250`) rather than an exact timing. That looseness is intentional: it is a guard against regressions large enough to matter operationally, not a precise measurement.

How testing thinking shifts by background

The hardest part of testing advanced Rust is rarely the syntax of `#[test]` — it is unlearning the substitution and isolation habits your previous language made cheap. Each card below names the mental-model shift, not a crate-for-library mapping.

Production patterns

Pitfalls and tradeoffs

The most expensive advanced-testing mistake is usually not “too few tests.” It is too much of the wrong test layer and too little control over time, replay, unsafe invariants, and output stability.

Examples

Both runnable examples below are deliberately deterministic so they execute in the browser, but each is shaped like the production test it stands in for. The first is a quota invariant — the seed of a property test. The second is a duplicate-delivery harness — the seed of a distributed idempotency test. Run each once to see the baseline output, then change an input and watch which assertion moves.

Example 1: deterministic invariant harness you can later lift into a property test

The browser example stays deterministic, but the shape is the same one you would lift into a property-testing crate: many generated scenarios, one invariant, zero ambiguity about the contract.

Example 42-1. `property_invariant_harness.rs` — *Invariant harness*

```
#[derive(Debug, Clone, Copy)]
struct Quota {
    limit: u32,
    used: u32,
}

impl Quota {
    fn try_reserve(&mut self, qty: u32) -> bool {
        if self.used + qty > self.limit {
            return false;
        }
    }
}
```

```

    }

    self.used += qty;
    true
}
}

fn invariant_holds(limit: u32, ops: &[u32]) -> bool {
    let mut quota = Quota { limit, used: 0 };

    for &qty in ops {
        let _accepted = quota.try_reserve(qty);
        if quota.used > quota.limit {
            return false;
        }
    }

    true
}

fn main() {
    let limit = 8_u32;
    let scenarios: &[&[u32]] = &[
        &[1_u32, 1, 1],
        &[4_u32, 4],
        &[5_u32, 4],
        &[2_u32, 2, 2, 2],
        &[8_u32],
    ];

    let all_valid = scenarios
        .iter()
        .all(|ops| invariant_holds(limit, ops));

    println!("cases = {}", scenarios.len());
    println!("all valid = {}", all_valid);
    println!("limit = {}", limit);
}

```

OUTPUT

```

cases = 5
all valid = true
limit = 8

```

The claim is simple and load-bearing: used quota never exceeds limit, even when some reservations are rejected.

This stays below the transport and runtime layers, so failures point straight at the state rule rather than at incidental integration noise.

In a real project, the same invariant becomes a property test by replacing the fixed scenarios with a generator and a shrinker.

Example 2: deterministic async idempotency harness for a service boundary

The important property is not “Tokio works.” The important property is that duplicate deliveries are harmless and the async boundary keeps that rule explicit.

Example 42-2. `async_idempotent_delivery_harness.rs` — *Delivery harness*

```
use std::collections::HashSet;
use std::sync::Arc;

use tokio::sync::Mutex;

#[derive(Default)]
struct FakeStore {
    applied: Mutex<HashSet<'static str>>,
    duplicate_count: Mutex<usize>,
}

impl FakeStore {
    async fn apply_once(&self, event_id: &'static str) -> bool {
        let mut applied = self.applied.lock().await;
        if !applied.insert(event_id) {
            drop(applied);
            let mut duplicates = self.duplicate_count.lock().await;
            *duplicates += 1;
            return false;
        }

        true
    }

    async fn processed(&self) -> usize {
        self.applied.lock().await.len()
    }

    async fn duplicates(&self) -> usize {
        *self.duplicate_count.lock().await
    }
}

#[tokio::main]
async fn main() {
    let store = Arc::new(FakeStore::default());

    for event_id in ["evt-1", "evt-1", "evt-2"] {
        let _ = store.apply_once(event_id).await;
    }

    println!("processed = {}", store.processed().await);
    println!("duplicates = {}", store.duplicates().await);
    println!("deliveries = {}", 3);
}
```

OUTPUT

```
processed = 2
duplicates = 1
```

```
deliveries = 3
```

The state is still one owned set behind an async lock, so the harness tests the real contract instead of a mock-only approximation.

Duplicate delivery is part of the design center. The harness counts it and proves it does not duplicate the durable effect.

The same pattern scales into a queue worker or lease-based consumer once the event ID becomes a durable checkpoint key.

The repository also includes standalone Rust source under `examples/ch42_testing_advanced_rust_systems/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

CHAPTER 43 · OBSERVABILITY

Observability

Logging, metrics, tracing, structured logs, OpenTelemetry, async and distributed tracing, profiling in production, alerting, SLOs, and observability-driven refactoring

Observability is what lets you explain a production Rust service after the fact, without redeploying with more print statements. This chapter walks through the four signals an operator actually reaches for — structured logs, metrics, traces, and profiles — and ties them to service-level objectives and the boundaries Rust already makes visible in your code.

Builds on Chapters 25, 30, 31, 35, 36, and 42

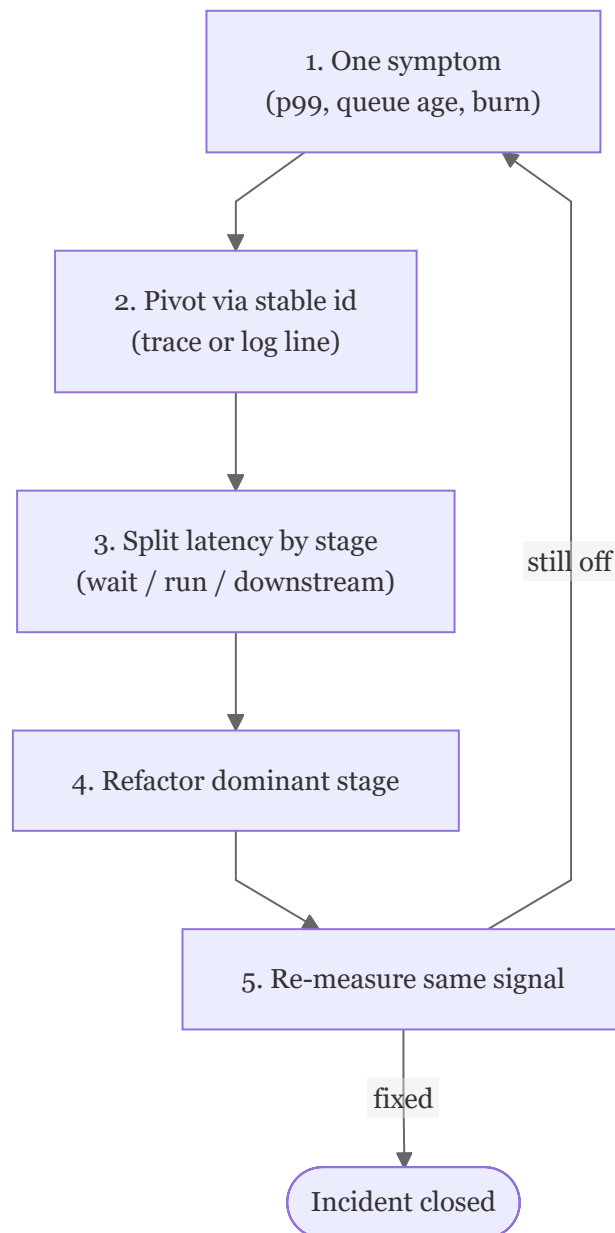
Chapter 25 covered Tokio task boundaries. Chapter 30 covered broker delivery and retry signals. Chapter 31 made queue wait and replay part of distributed work. Chapters 35 and 36 separated profiling, tracing, and distributed latency analysis. Chapter 42 added deterministic test harnesses so observability signals can be asserted instead of admired.

Opening scenario

A queue-backed scoring service misses its latency SLO right after a rollout, yet CPU usage and the handler’s median runtime both look perfectly healthy. Nothing in the obvious dashboard is red. This is the everyday shape of a real incident: the symptom is visible but the cause is hidden behind a boundary you have not instrumented yet. The job for the rest of this chapter is to instrument the ownership and execution boundaries that can actually explain it — queue age, retry rate, worker saturation, trace propagation across the spawn, and where CPU time really goes.

A practical incident loop

Good debugging under pressure follows a loop rather than a hunch. Notice the shape below: you always start from one symptom, pivot to a single trace or log line through a stable id, decompose the latency by stage, fix the stage that dominates, and then re-measure the same signal to prove the fix mattered. The loop is what stops you from optimizing the cheapest code to edit instead of the boundary that hurts.



The incident loop: start from a symptom, prove the fix on the same signal you started from.

1. Start from one symptom: queue age, retry rate, p99 latency, or error-budget burn.
2. Pivot to one trace or structured log line using a stable ID.
3. Separate queue wait, handler run time, downstream wait, and completion time.
4. Refactor the dominant boundary, then re-measure the same signal to prove the fix actually mattered.

At a glance

- Use logs for events, metrics for aggregate health, traces for causality, and profiles for cost.
- Keep stable IDs flowing across tasks, queues, retries, and durable completion records.
- Instrument queue wait, handler time, publish time, and failure paths separately.
- Turn telemetry into refactoring evidence by tying it to one operator-facing question at a time.

Signal-selection checklist

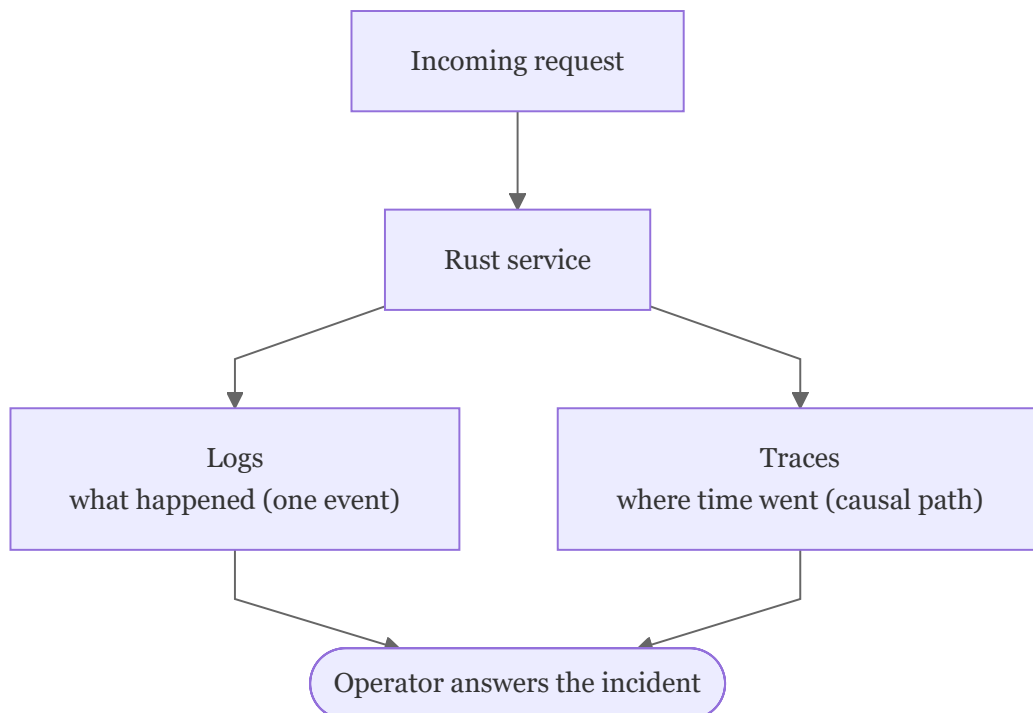
- Is the question about an event, an aggregate, a causal path, or a cost hotspot?
- Which IDs belong in logs and traces, and which labels must stay out of metrics?
- What should page first: burn rate, queue age, saturation, or one failure class?
- Which observable would prove the next refactor fixed the dominant boundary instead of the easiest code to edit?

Mental model

Core concepts

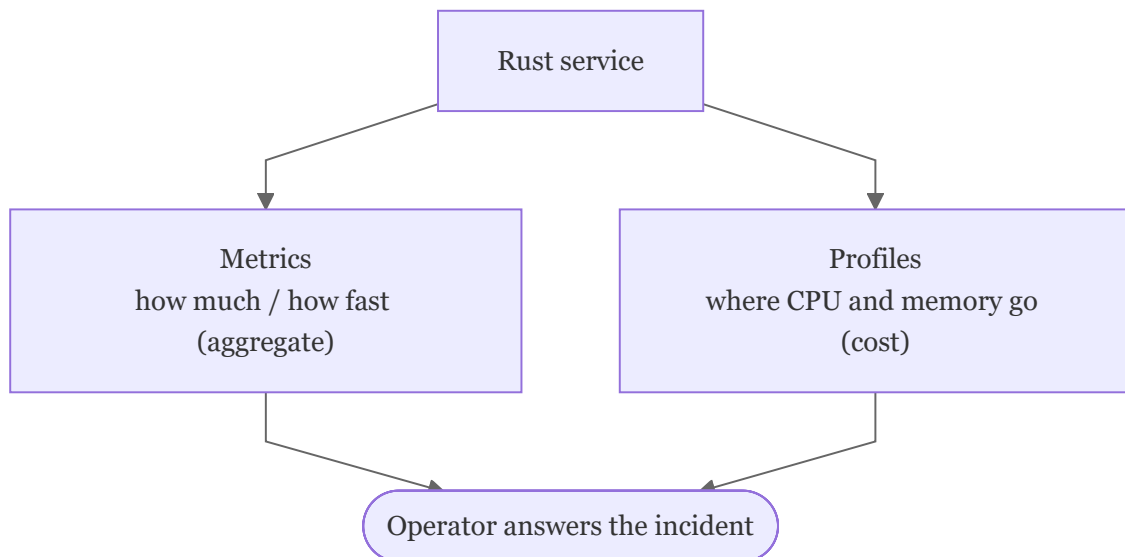
The four signals and the questions they answer

Before the individual tools, hold the whole picture in view. One request enters the system and produces all four kinds of signal as it flows through. They are not redundant — each answers a question the others cannot. When you look at the diagram, notice that logs and traces are emitted per request (high detail, high cost), while metrics are aggregated across all requests (low detail, cheap to keep), and profiles sample the process as a whole. Matching the question to the signal is the entire skill.



First half: the per-request signals. Logs answer what happened; traces answer where the time went.

The other two signals are not emitted per request — metrics aggregate across every request and profiles sample the process as a whole — yet they feed the same operator answering the same incident:



Second half: the fleet-wide signals. Metrics answer how much and how fast; profiles answer where the cost goes.

Logging and structured logs

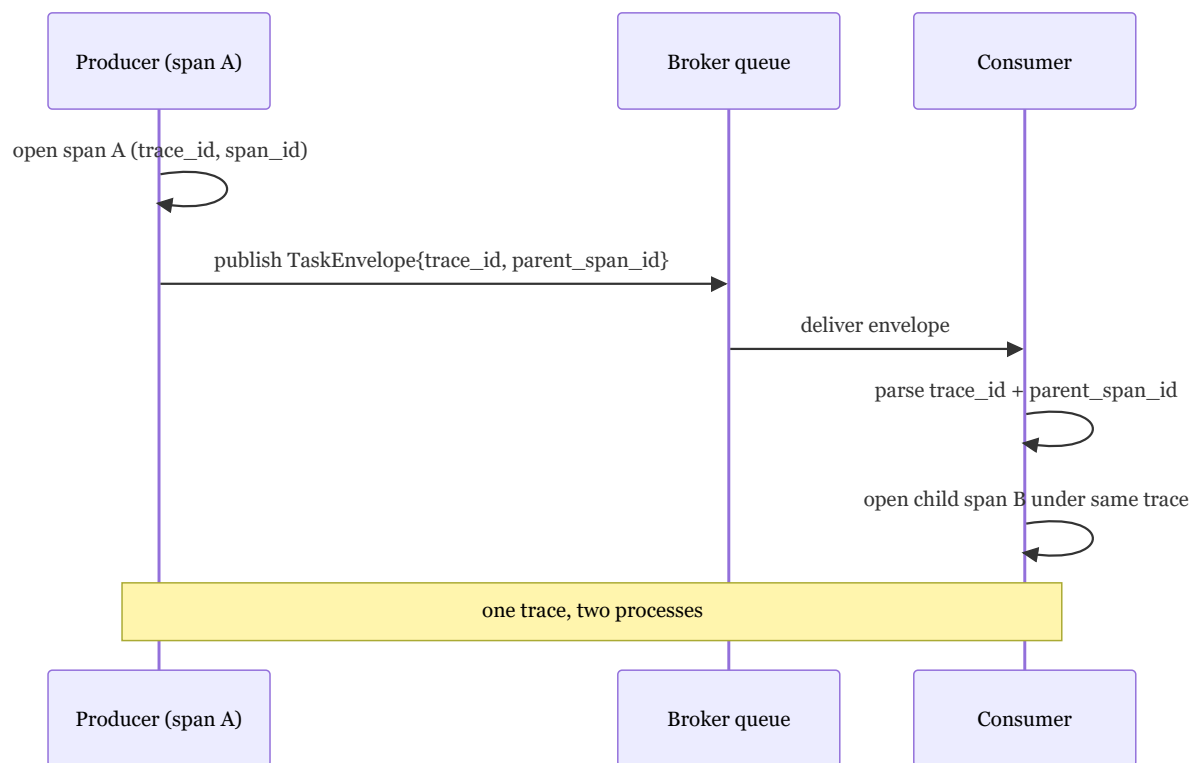
Metrics

A useful default split is counter + gauge + histogram: counters for accepted and retried work, gauges for queue depth and busy workers, histograms for queue wait, run time, and end-to-end latency.

Tracing, async tracing, and distributed tracing

OpenTelemetry and crossing the wire

What to look at first: the struct below is the whole point. A producer finishes its span, copies the live `trace_id` and `parent_span_id` into a `TaskEnvelope`, and serializes it onto the queue. The consumer parses the envelope and re-opens a child span from those fields. Without this hand-off the consumer would start a brand new, unrelated trace and the timeline would split in two exactly where the work crossed the broker. The diagram shows that hand-off; the struct is the contract that makes it possible.



The envelope carries trace context across the broker so the consumer continues the same trace instead of starting a new one.

Profiling in production

Profiling belongs in an observability chapter because hot CPU, queue wait, lock contention, and serialization cost are often all part of the same incident. The best production profile is the one you can line up next to one trace and one SLO graph.

Alerting and SLOs

Observability-driven refactoring

How this maps from your background

Every one of these languages can be observed well — the concepts of logs, metrics, traces, and profiles are universal. What changes in Rust is mostly how much is automatic versus how much you wire on purpose. The recurring theme below is the same: there is less ambient context magic, so propagation and span boundaries become things you state explicitly rather than things the runtime threads for you. That is more work at the keyboard and far less mystery during an incident.

Production patterns

These are the habits that pay off before an incident rather than during one. The common thread is doing the cheap, deliberate work up front — naming fields, publishing the boring backlog metrics, propagating context across every handoff — so that when something breaks the data is already there to explain it.

Pitfalls and tradeoffs

Almost every entry below is a case of the right idea applied to the wrong signal: detail where you needed aggregation, aggregation where you needed identity, or instrumentation that stopped exactly at the boundary that mattered. Read them as the failure modes of the four signals, not as a list of unrelated mistakes.

The most common observability failure is not “too little telemetry.” It is the wrong telemetry at the wrong boundary: unstructured logs, invisible queue wait, or metrics so high-cardinality that nobody can query them when the incident actually starts.

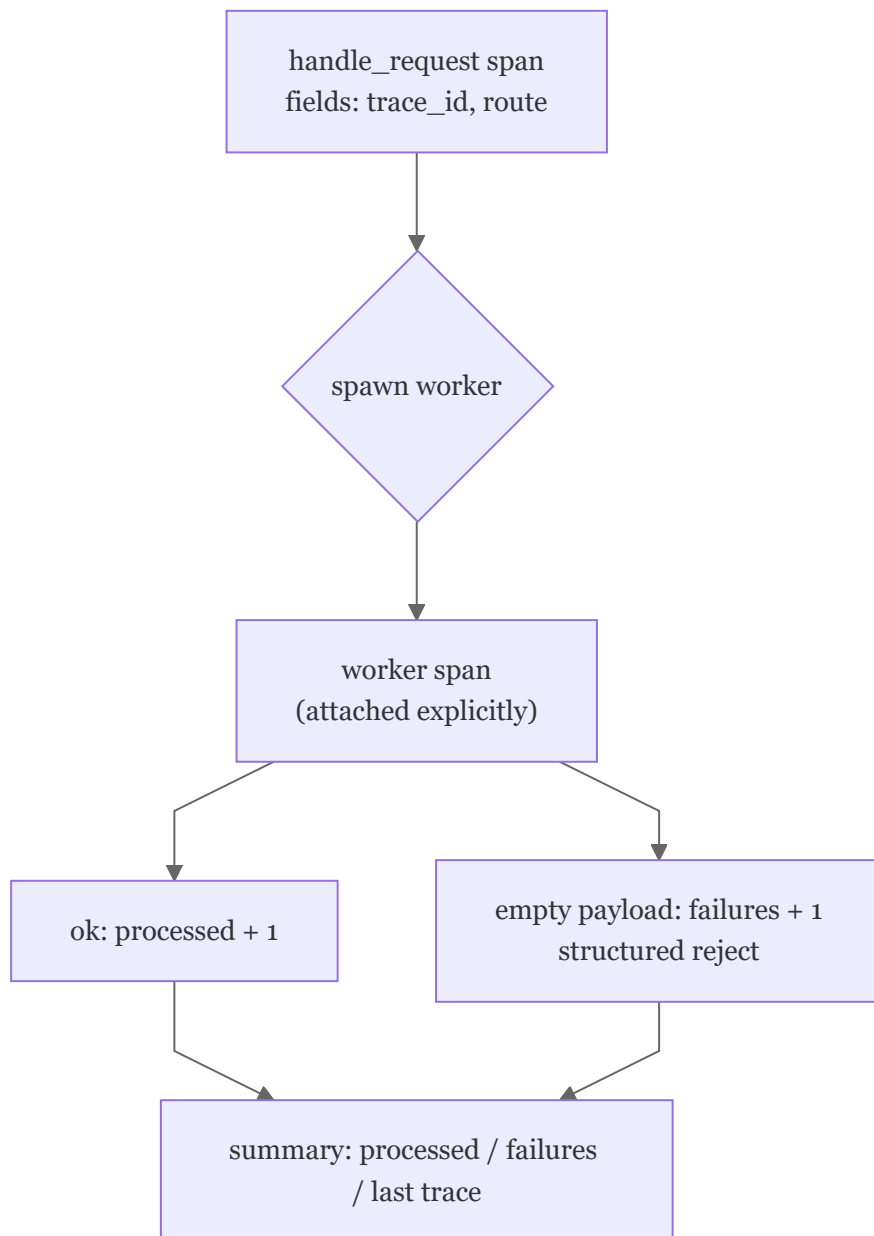
Examples

The browser runner summarizes instrumentation so output stays deterministic. In a real Cargo service, the same code would normally pair these spans with a subscriber or an OpenTelemetry bridge.

Example 1: instrument a Tokio worker with structured spans

The request handler records structured fields, the worker adds a parent span, and the summary makes success, failure, and last-seen trace identity explicit.

What to look at first: follow one request id through the nesting below before reading the code. The request span is the parent; when work is handed to the worker, the worker span is attached explicitly so it nests under the right request instead of running detached. The success and failure arms both stay inside that span, which is why the final summary can report processed, failures, and the last trace id with no ambiguity about which request each event belonged to. The code is the literal implementation of this tree.



The worker span nests under the request span on purpose, so every event stays attributable to one trace id.

Example 43-1. tracing_tokio_worker.rs — Tokio spans

```

use tokio::sync::mpsc;
use tracing::{error, info, info_span, instrument, Instrument};

#[derive(Debug, Clone)]
struct Request {
    trace_id: &'static str,
    route: &'static str,
    bytes: usize,
}

#[instrument(
    name = "handle_request",
    skip(request),
    fields(trace_id = %request.trace_id, route = request.route, bytes = request.bytes)

```

```

    })
    async fn handle_request(request: Request) -> Result<usize, &'static str> {
        if request.bytes == 0 {
            error!(status = "reject", "empty payload");
            return Err("empty payload");
        }

        info!(status = "ok", "request handled");
        Ok(request.bytes / 10)
    }

#[tokio::main]
async fn main() {
    let (tx, mut rx) = mpsc::channel::<Request>(4);

    tx.send(Request {
        trace_id: "req-7",
        route: "/score",
        bytes: 640,
    })
    .await
    .unwrap();

    tx.send(Request {
        trace_id: "req-8",
        route: "/score",
        bytes: 0,
    })
    .await
    .unwrap();

    tx.send(Request {
        trace_id: "req-9",
        route: "/health",
        bytes: 120,
    })
    .await
    .unwrap();

    drop(tx);

    let worker = info_span!("worker", worker = "ingest-a");
    let mut processed = 0_usize;
    let mut failures = 0_usize;
    let mut last_trace = "none";

    while let Some(request) = rx.recv().await {
        last_trace = request.trace_id;

        match handle_request(request).instrument(worker.clone()).await {
            Ok(_units) => processed += 1,
            Err(_error) => failures += 1,
        }
    }

    println!("instrumented = {}", true);
}

```

```
println!("processed = {}", processed);
println!("failures = {}", failures);
println!("last trace = {}", last_trace);
}
```

OUTPUT

```
instrumented = true
processed = 2
failures = 1
last trace = req-9
```

The handler emits fields that are filterable later instead of burying `trace_id` and `route` inside prose.

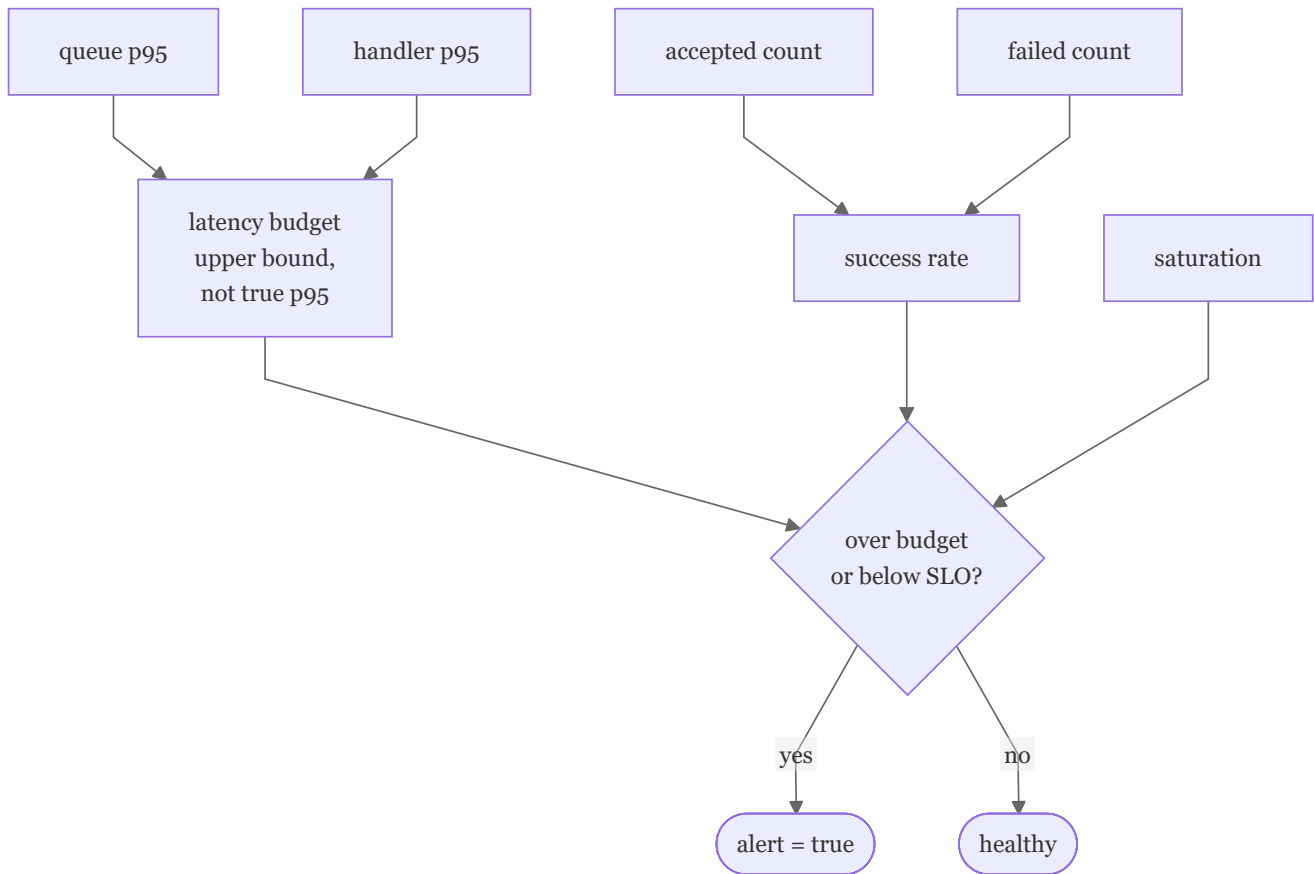
The worker span is explicit, so a later queue or task analysis still knows which runtime boundary owned the request.

One empty payload becomes one structured reject instead of one mysterious count drop somewhere in a dashboard.

Example 2: summarize latency, success rate, and alert state from one SLO window

This is the kind of aggregate view an operator or an alert rule wants: a conservative latency budget, success budget, and whether the window already looks unhealthy. Note that $p_{95}(\text{queue}) + p_{95}(\text{handler})$ is an upper bound, not the true end-to-end p_{95} — percentiles are not additive.

What to look at first: this is a small pipeline, not a clever algorithm. Two separate latency histograms (queue and handler) feed a conservative latency budget; accepted and failed counts feed a success rate; both results plus a saturation check feed one boolean alert. Trace the arrows in the diagram and you have already read the program — the code just turns each box into a few lines. The honest separation of queue p_{95} and handler p_{95} is the part worth dwelling on, because it is what lets you blame the right stage later.



The SLO window is a pipeline: separate latency stages and counts fold into one alert decision.

Example 43-2. `metrics_slo_window.rs` — *Slo window*

```

#[derive(Debug, Clone, Copy)]
struct WindowStats {
    requests: u64,
    errors: u64,
    queue_p95_ms: u64,
    handler_p95_ms: u64,
    busy_workers: u32,
    total_workers: u32,
}

fn success_rate(stats: &WindowStats) -> f64 {
    1.0 - stats.errors as f64 / stats.requests as f64
}

fn latency_budget_ms(stats: &WindowStats) -> u64 {
    stats.queue_p95_ms + stats.handler_p95_ms
}

fn saturation(stats: &WindowStats) -> f64 {
    stats.busy_workers as f64 / stats.total_workers as f64
}

fn burn_alert(stats: &WindowStats) -> bool {
    latency_budget_ms(stats) > 350 || success_rate(stats) < 0.995 || saturation(stats) > 0.90
}

```

```
fn main() {
    let stats = WindowStats {
        requests: 5_000,
        errors: 40,
        queue_p95_ms: 180,
        handler_p95_ms: 210,
        busy_workers: 19,
        total_workers: 20,
    };

    println!("latency budget = {}", latency_budget_ms(&stats));
    println!("success rate = {:.4}", success_rate(&stats));
    println!("alert = {}", burn_alert(&stats));
}
```

OUTPUT

```
latency budget = 390
success rate = 0.9920
alert = true
```

Queue p95 and handler p95 are kept separate so end-to-end latency can be decomposed honestly.

One burn-style rule can combine latency, success rate, and saturation instead of paging on one raw error counter.

A window like this tells you whether the next fix belongs in worker capacity, admission control, or handler code.

The repository also includes standalone Rust source under `examples/ch43_observability/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

CHAPTER 44 · PACKAGING AND DEPLOYMENT

Packaging and Deployment

Static binaries, cross-compilation, Docker images, minimal runtime containers, wasm and native-library packaging, CI/CD, supply-chain security, feature-gated builds, and release engineering

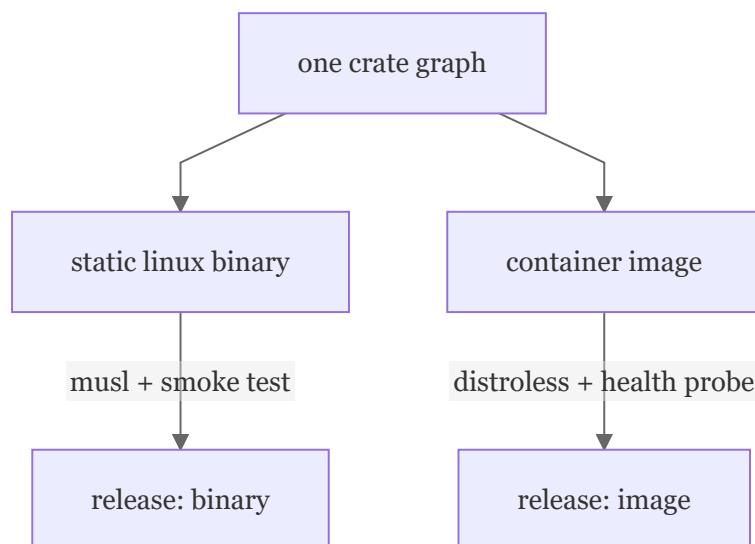
Deployment readiness requires reproducible builds, minimal runtime assumptions, artifact provenance, and rollback policy. This chapter covers Rust packaging as an operations contract.

Builds on Chapters 03, 19, 25, 28, 29, 41, and 43

Chapter 03 established Cargo, features, and workspace structure. Chapter 19 covered wire contracts. Chapter 25 covered service runtime shape. Chapters 28 and 29 covered native and wasm boundaries. Chapter 41 covered translated error contracts, and Chapter 43 covered health, telemetry, and operator signals. Packaging and deployment sit across all of them.

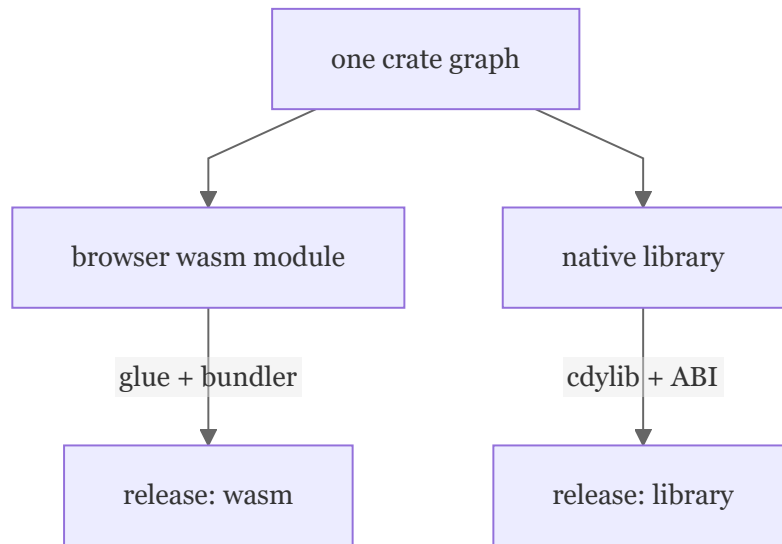
Opening scenario

One Rust codebase has to ship four different ways: as a static Linux binary for the fleet, as a container image for the orchestrator, as a browser Wasm module for the dashboard, and as a native library for a C++ host that embeds the risk engine. They share source, but they do not share a runtime contract. The business requirement is to treat each artifact as its own promise, with its own target triple and feature set, its own smoke test, and its own provenance, signatures, and rollback instructions. The diagram below shows how one crate graph fans out into four distinct delivery shapes that must each be packaged and verified separately.



Half one: the same crate graph fans out to a static Linux binary and a container image, each with its own target and verification.

The same crate graph also feeds two more delivery shapes:



Half two: the browser wasm module and the native library each pick a distinct packaging shell before they count as releasable.

A practical decision order

1. Choose the runtime contract first: binary, image, wasm module, or native library.
2. Choose the target triple and feature set second.
3. Choose the packaging shell third: filesystem image, loader metadata, or ABI surface.
4. Attach smoke tests, checksums, signatures, and rollback rules before calling it releasable.

At a glance

- Treat every artifact shape as a runtime contract: binary, image, wasm module, or native library.
- Target triples, feature flags, and crate types are release decisions, not only compiler settings.
- Smoke-test the final packaged artifact on a target-like environment, not only the crate before packaging.
- Checksums, signatures, SBOMs, and rollback instructions belong to the release story from the start.

Release-engineering questions

- Who runs this artifact, and what runtime assumptions does it make?
- Which feature set is the smallest honest release for this consumer?
- What trust data ships with the final artifact, and which CI lane proves it was produced correctly?
- Could another engineer roll back this release without rebuilding from memory during an incident?

Mental model

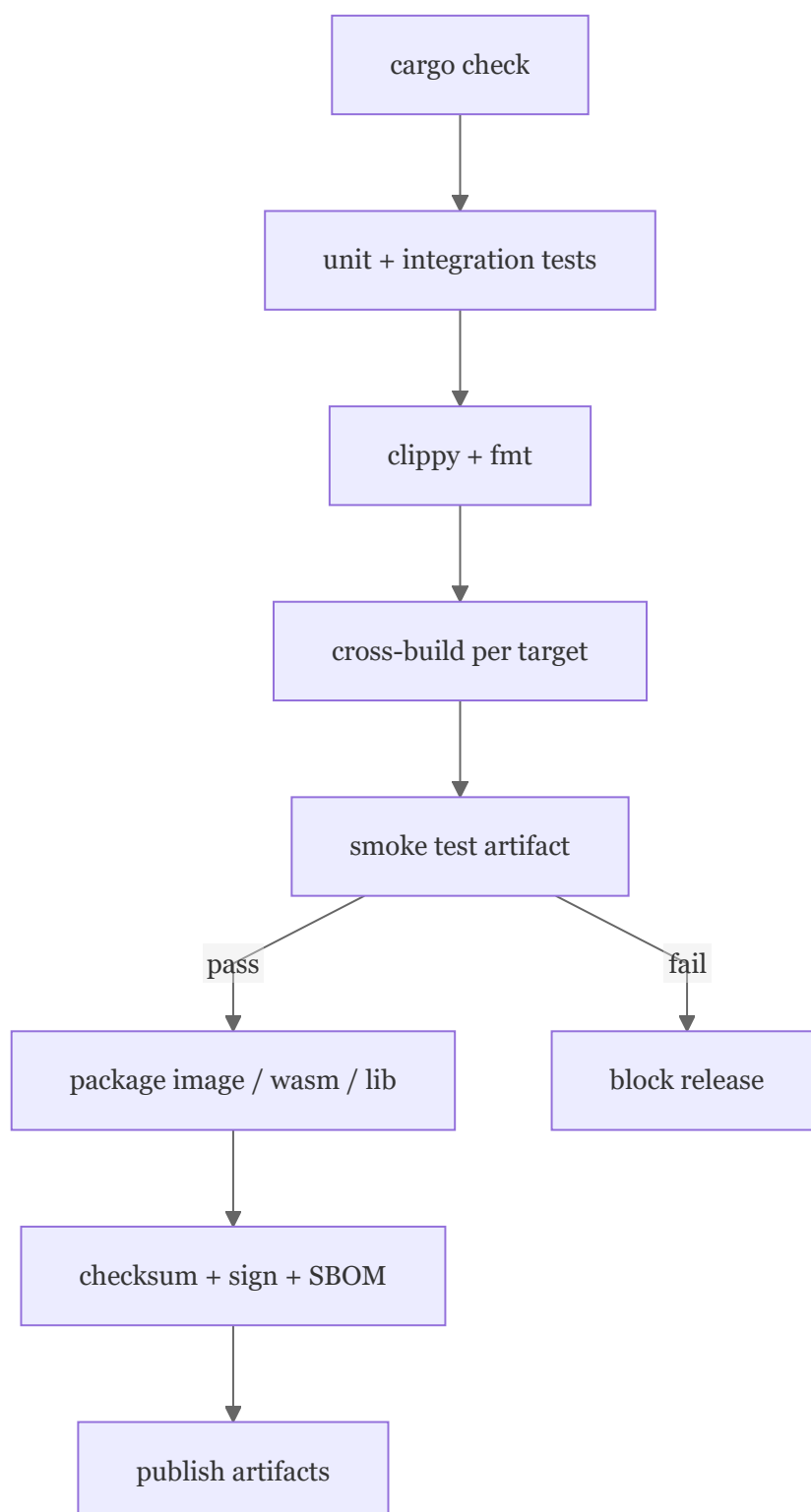
Core concepts

How to think about deployment coming from C++, C#, Go, or Python

The useful comparison here is not 'which crate replaces which library.' It is where each background expects the runtime to come from, and how that expectation changes when the artifact has to stand on its own. Find your starting language and notice the shift it asks for.

The release pipeline as a gate sequence

It helps to picture the pipeline as a chain of gates rather than a script. Each stage only runs if the one before it passed, and the artifact only earns a signature and publication after the smoke test confirms it actually runs on a target-like environment. The key transition to watch is the one from build to smoke test: that is where a release stops being 'it compiled' and becomes 'it runs.'



*Build proves the code compiles; the smoke test proves the packaged artifact runs.
Only then do trust data and publication follow.*

Production patterns

Pitfalls and tradeoffs

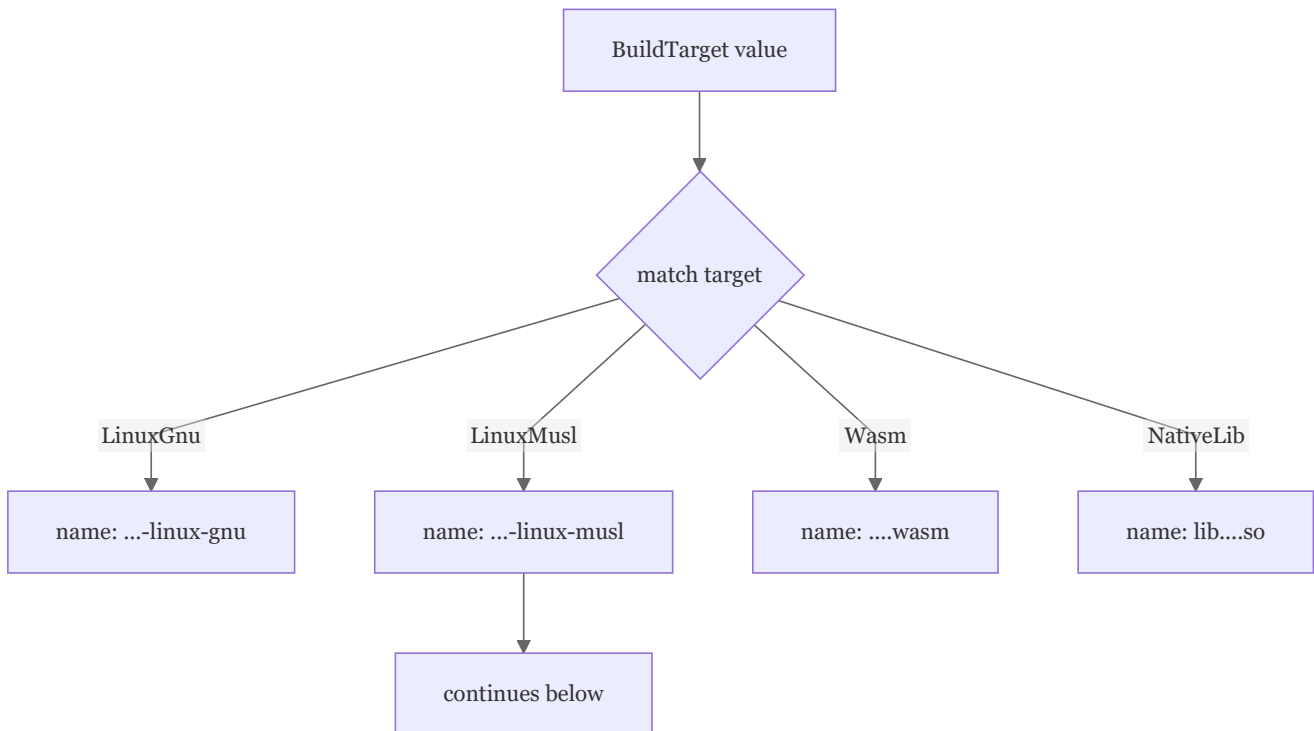
The most expensive deployment bug is often not a compiler failure. It is a “successful” release whose final artifact was never tested in the real runtime shape that production actually executes.

Examples

Example 1: target-aware artifact naming

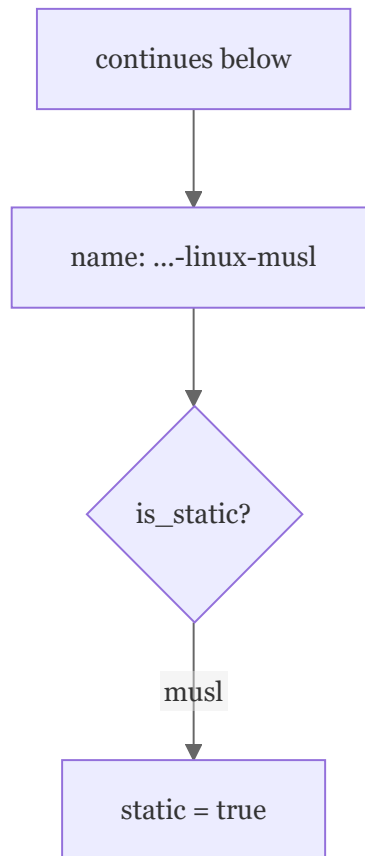
The point is not string formatting. The point is that each target class becomes an explicit artifact contract instead of an implicit convention hidden in release scripts.

What to look at: the match in `artifact_name` is the whole idea. One `BuildTarget` value routes to exactly one naming rule, and `is_static` records intent as data rather than as a comment in a release script. Follow how a single target flows through both functions to a fully described artifact.



First the match selects the artifact name: one enum variant routes to exactly one naming rule.

From the musl name, the same target value then decides static intent:



The musl target also records static intent as data: the enum, not a comment, decides the build is static.

Example 44-1. target_matrix.rs — Target matrix

```

#[derive(Debug, Clone, Copy)]
enum BuildTarget {
    LinuxGnu,
    LinuxMusl,
    Wasm,
    NativeLib,
}

fn artifact_name(service: &str, target: BuildTarget) -> String {
    match target {
        BuildTarget::LinuxGnu => format!("{}", x86_64-unknown-linux-gnu", service),
        BuildTarget::LinuxMusl => format!("{}", x86_64-unknown-linux-musl", service),
        BuildTarget::Wasm => format!("{}", wasm32-unknown-unknown.wasm", service),
        BuildTarget::NativeLib => format!("lib{}.so", service),
    }
}

fn is_static(target: BuildTarget) -> bool {
    matches!(target, BuildTarget::LinuxMusl)
}

fn main() {
    let service = "payments";

    println!("musl = {}", artifact_name(service, BuildTarget::LinuxMusl));
}

```

```
println!("wasm = {}", artifact_name(service, BuildTarget::Wasm));
println!("static = {}", is_static(BuildTarget::LinuxMusl));
}
```

OUTPUT

```
musl = payments-x86_64-unknown-linux-musl
wasm = payments-wasm32-unknown-unknown.wasm
static = true
```

Artifact naming should make the target obvious to humans and automation.

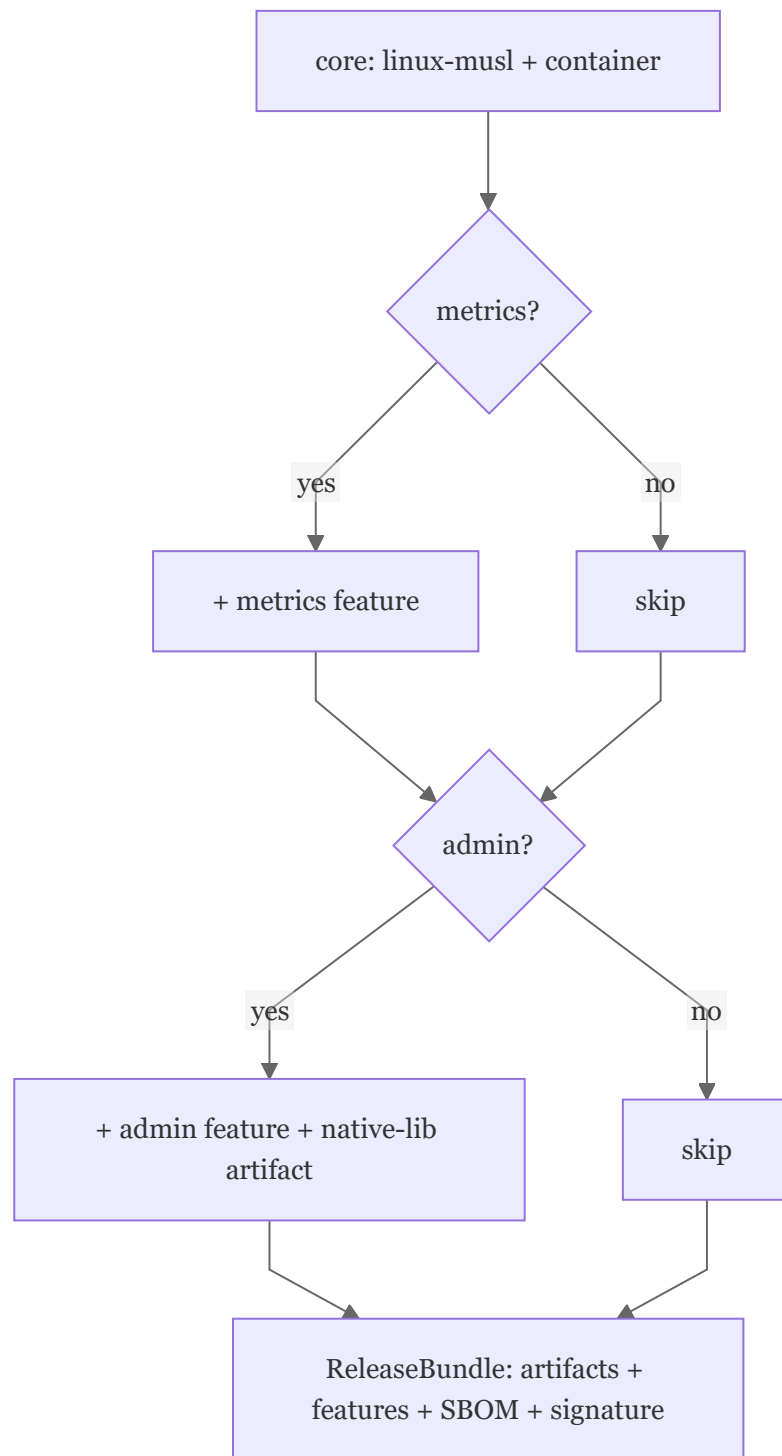
Whether a build is intended to be static is a first-class release property, not tribal knowledge.

One codebase can legitimately emit binaries, wasm modules, and native libraries without pretending they are the same delivery artifact.

Example 2: release bundle planning

A release is more than one executable. Feature selection, artifact list, and supply-chain outputs should all be visible in one reviewable bundle plan.

What to look at: watch how `build_bundle` starts from a core artifact-and-feature set and then conditionally accumulates more based on the metrics and admin flags. Each flag is a real packaging decision: admin does not just add a feature, it adds a whole new artifact (the native library). The diagram traces that branching before you read the code.



Flags do not just flip booleans; they grow the artifact inventory and feature list that the release bundle must account for.

Example 44-2. `release_bundle.rs` — Release bundle

```

#[derive(Debug)]
struct ReleaseBundle {
    profile: &'static str,
    artifacts: Vec<&'static str>,
    features: Vec<&'static str>,
    sbom: bool,
    signature: bool,
}

```



```

}

fn build_bundle(profile: &'static str, metrics: bool, admin: bool) -> ReleaseBundle {
    let mut artifacts = vec!["linux-musl", "container"];
    let mut features = vec!["core"];

    if metrics {
        features.push("metrics");
    }

    if admin {
        artifacts.push("native-lib");
        features.push("admin");
    }

    ReleaseBundle {
        profile,
        artifacts,
        features,
        sbom: true,
        signature: true,
    }
}

fn main() {
    let bundle = build_bundle("release", true, false);

    println!("profile = {}", bundle.profile);
    println!("artifacts = {}", bundle.artifacts.join(","));
    println!("features = {}", bundle.features.join(","));
    println!("signed = {}", bundle.sbom && bundle.signature);
}

```

OUTPUT

```

profile = release
artifacts = linux-musl,container
features = core,metrics
signed = true

```

Feature selection changes the release artifact and should therefore be explicit in the release plan.

Operators should know which shapes were built and published for a given version.

Checksums, signatures, and SBOM outputs belong to the bundle story, not only to the CI implementation.

The repository also includes standalone Rust source under `examples/ch44_packaging_and_deployment/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

CHAPTER 45 · CAPSTONE: A DISTRIBUTED RUST SYSTEM

Capstone: A Distributed Rust System

Compose an end-to-end distributed Rust system: typed task envelopes, brokered work, replay-safe completion, Merkle verification, specialized graph and matrix lanes, optional acceleration, and rollout discipline

The capstone assembles a distributed Rust service with bounded ingress, brokered work, replay-safe workers, verifiable payloads, compute executors, and observability. The business case is an auditable system that can be operated under load.

Builds on Chapters 25, 30, 31, 32, 36, 37, 38, 39, 40, 43, and 44

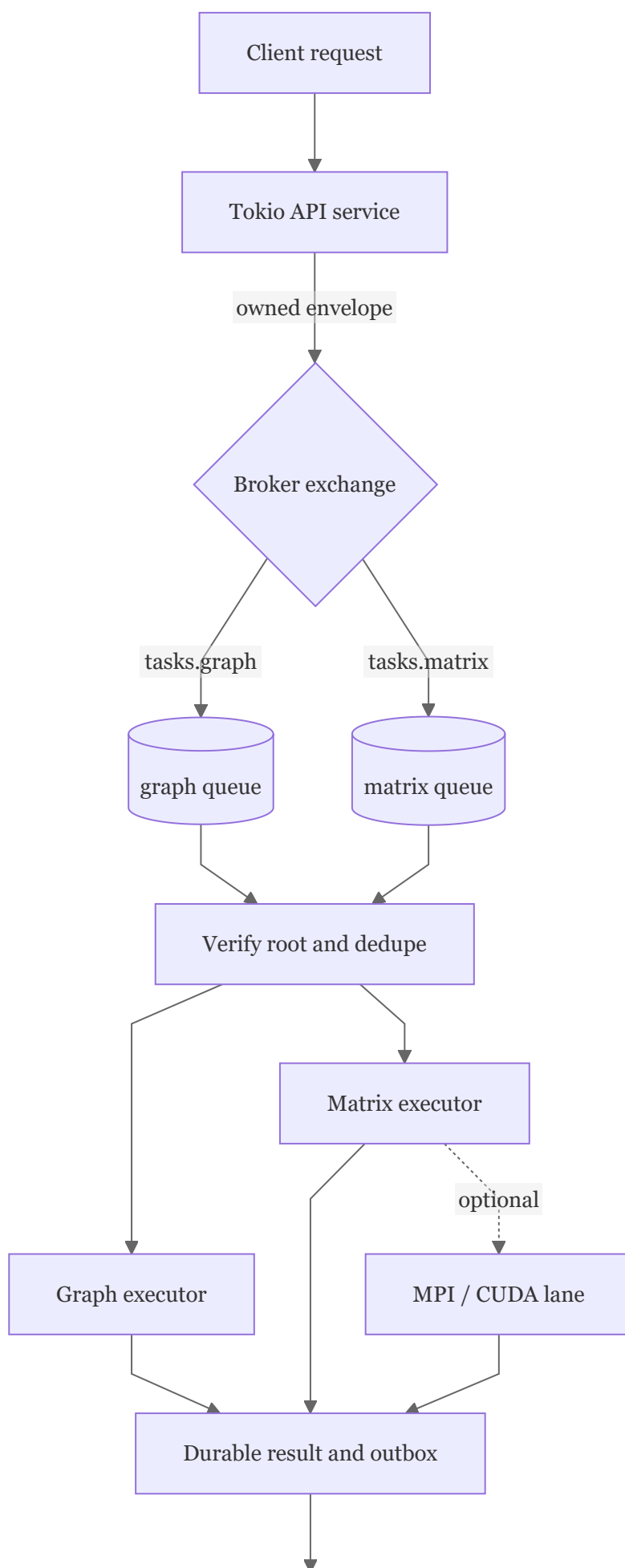
This chapter deliberately recombines the previous async, broker, distributed-task, Merkle, graph, matrix, profiling, observability, and deployment material into one coherent system. Treat it as an architecture review, not as a new disconnected topic.

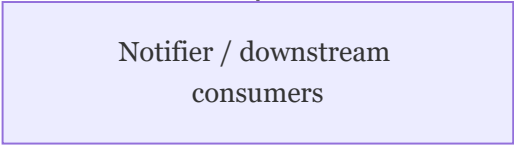
Opening scenario

A platform service accepts work over HTTP, routes it through a broker, verifies worker inputs, executes graph or matrix workloads, and optionally dispatches dense batches to MPI or CUDA lanes. The business requirement is a replay-safe, observable, and deployable architecture with explicit ownership, queue budgets, verification, and completion semantics.

Every chapter up to this point handed you one piece of that sentence in isolation. The job now is to make the pieces agree with each other. The diagram below is the trip a single unit of work takes from a client request to a durable result. As you read the rest of the chapter, keep returning to it and ask the same question at each arrow: who owns the data on this side of the boundary, and what has to be true before it crosses?

One possible capstone topology





Notifier / downstream
consumers

One unit of work, traced end to end. The route splits fast graph jobs from heavy matrix jobs at the broker; verification happens before any expensive executor runs; durable completion is the single point that makes the work safe to forget.

At a glance

- The capstone is one composed system: API shell, durable envelope, queue policy, worker lanes, verification, execution, and completion.
- Correctness starts with owned task boundaries and replay-safe finish-once state.
- Graph, matrix, MPI, and CUDA paths are execution lanes, not excuses to widen the outer contract.
- Rollout readiness requires profiling, observability, packaging, canary rules, and rollback artifacts alongside the code.

Capstone review questions

- Where does request-local data first become an owned durable envelope?
- What event makes work safe to forget: broker ack, durable result write, or finish-once checkpoint?
- Which workload classes deserve separate queues, worker pools, or specialist acceleration budgets?
- Which metric will tell you the real bottleneck is queueing, verification, execution, retry, or rollout policy?

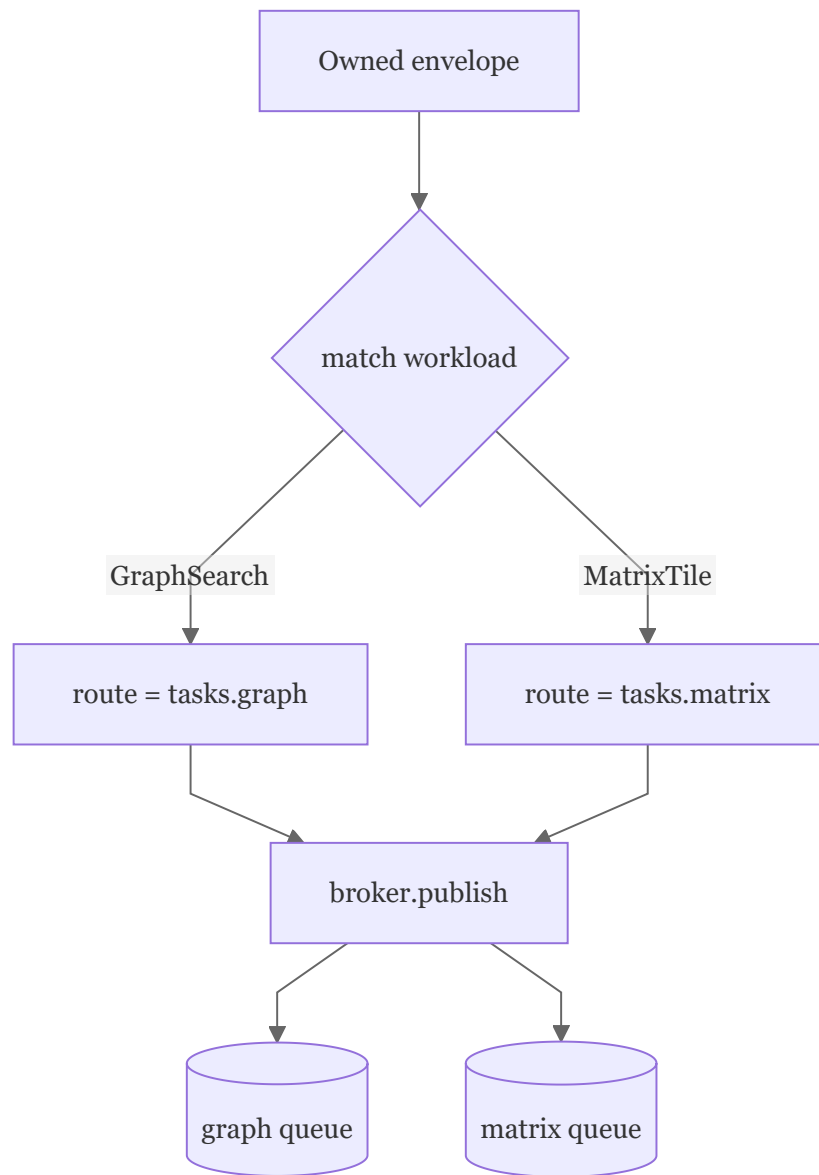
Mental model

Core concepts

The sections below walk the same pipeline left to right: the domain model that defines the unit of work, the broker that moves it, the Tokio shell that admits it, the worker pool that runs it, and the verification step that gates the expensive part. Each one is a boundary in the topology diagram above, and each one answers the ownership question for the data crossing it.

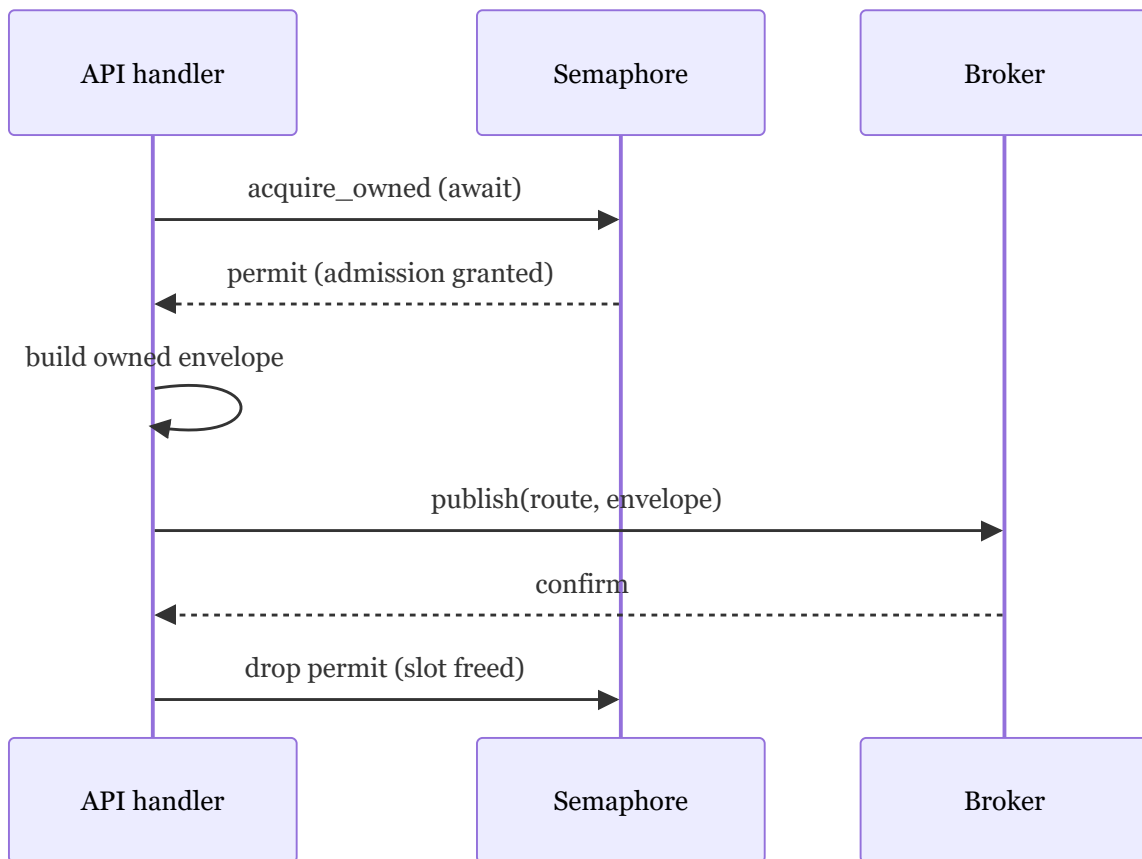
The important ownership rule is simple: the queue or broker carries an owned envelope. It does not carry a borrow into API-local request state. That single design choice removes a large amount of async and retry confusion immediately.

The whole routing decision is the match on the workload variant: a graph job and a matrix job leave the same publisher but land in different queues. That one branch is what lets a slow matrix backlog grow without ever stalling a fast graph job behind it. Read the shape first, then the code.



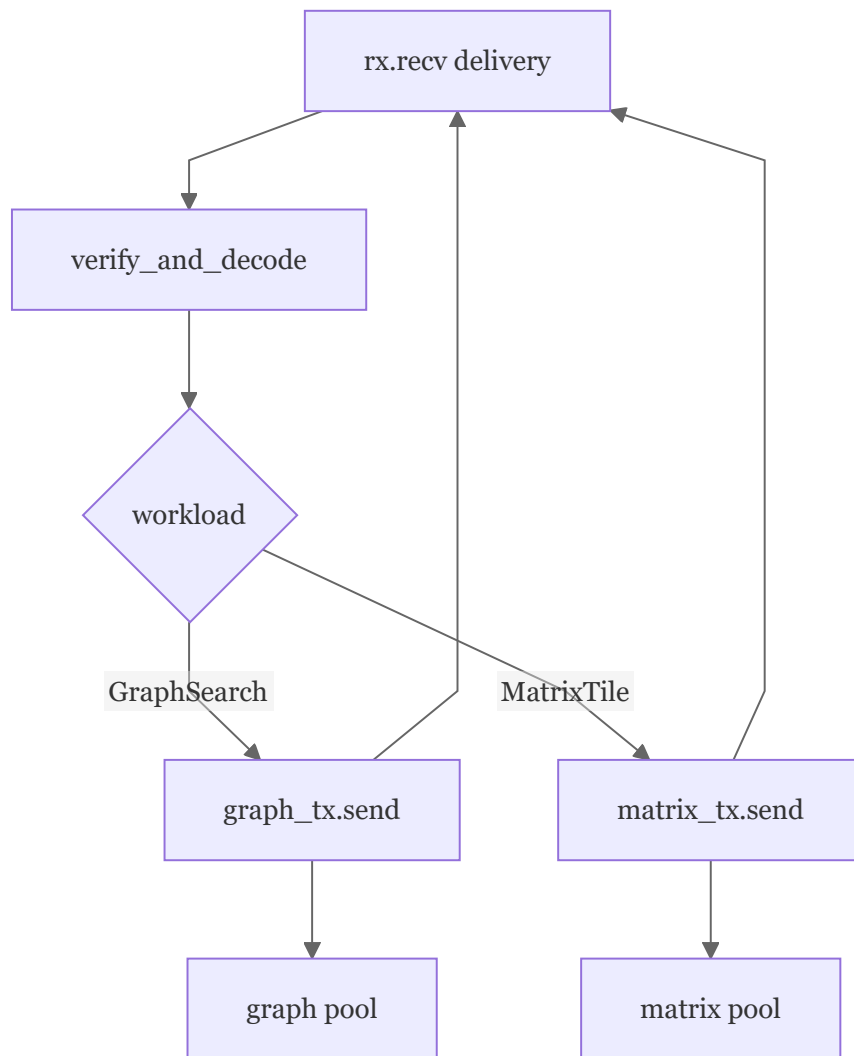
The workload variant chooses the route, and the route chooses the queue. Separate queues mean a slow class never blocks a fast one.

The order of these four lines is the whole point. The handler waits for an admission permit before it builds anything, so backpressure is applied at the door rather than at the broker. It builds an owned envelope, not a borrow into the request, so the published work can outlive the handler. Trace the sequence below, then read the code: each step is one line.



Admission first, owned envelope second, publish third, release last. A bounded semaphore turns a burst into a wait instead of an overload.

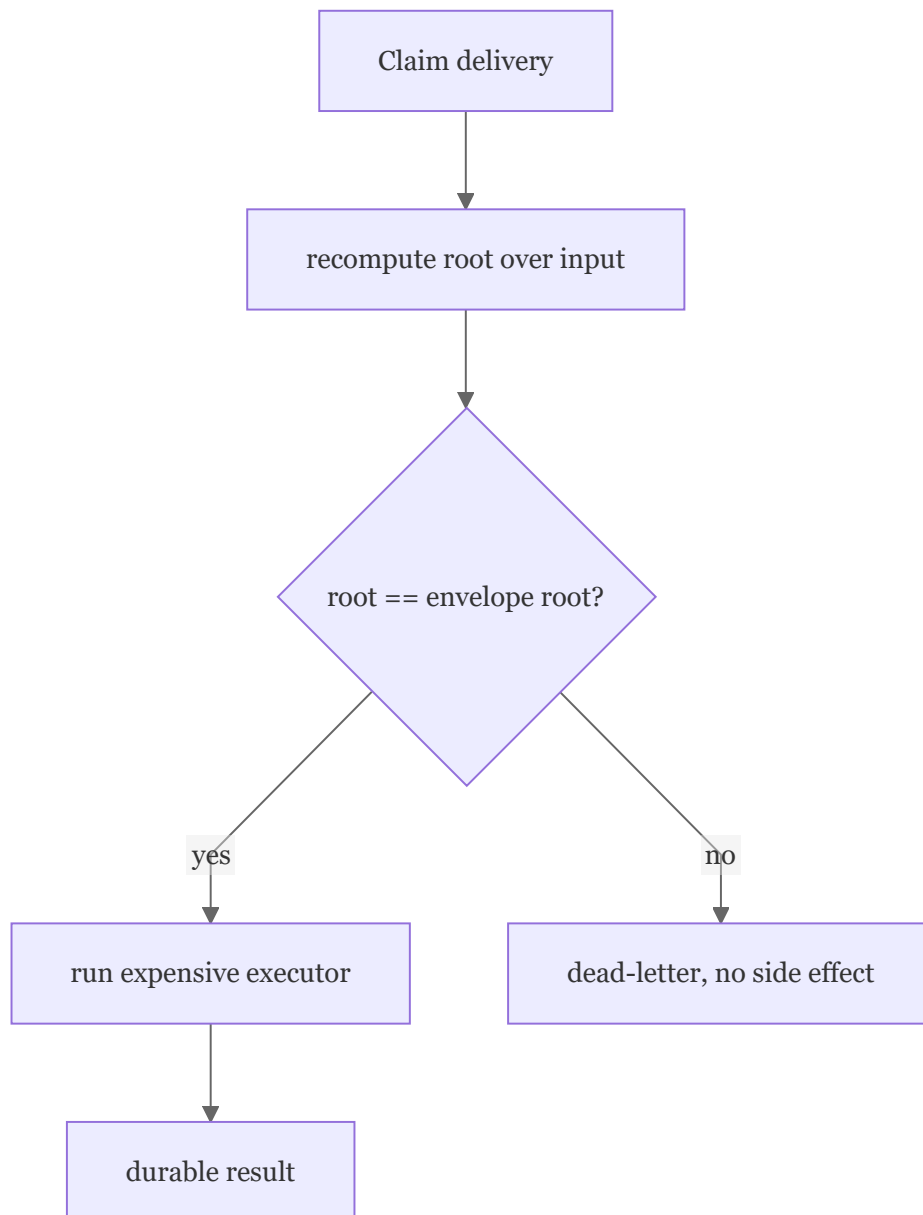
Notice what the dispatch loop does and does not do. It pulls one delivery, verifies and decodes it once, then sends the decoded spec down a workload-specific channel. It does not execute the graph or matrix work inline. That separation is what keeps the dispatcher thin and lets each lane have its own bounded pool. The diagram is the loop body; the code is the same thing written out.



The consumer verifies once, then hands each workload to its own lane. The dispatcher never blocks on execution; the lanes absorb the cost.

A common production repair is exactly this: keep the dispatcher thin and the execution lanes specialized. Fast graph jobs and slow matrix or GPU jobs usually deserve separate budgets once the p99 latency matters, because a single shared pool lets the slow class quietly consume every worker the fast class needed.

The single rule that makes this worth doing is the position of the gate: recompute the root over the canonical input and compare it before the worker runs the expensive side effect, never after. If the roots disagree, the work stops and is dead-lettered instead of executing on input that drifted. The struct below is just the metadata you carry so that check is possible and auditable; the diagram is where it sits in the flow.



Verify before effect. A mismatch ends the trip with zero side effects; only a matching root reaches the executor.

This chapter deliberately ends with rollout and packaging because distributed correctness does not stop at code. The queue budget, telemetry surface, artifact shape, and rollback plan are part of the final system.

How this lands depending on where you came from

You have almost certainly built something shaped like this capstone before: an ingress, a queue, a pool of workers, and some compute at the end. What changes in Rust is not the architecture but where the language forces a decision. The cards below name the mental-model shift, and the specific trap, for each background.

Production patterns

Pitfalls and tradeoffs

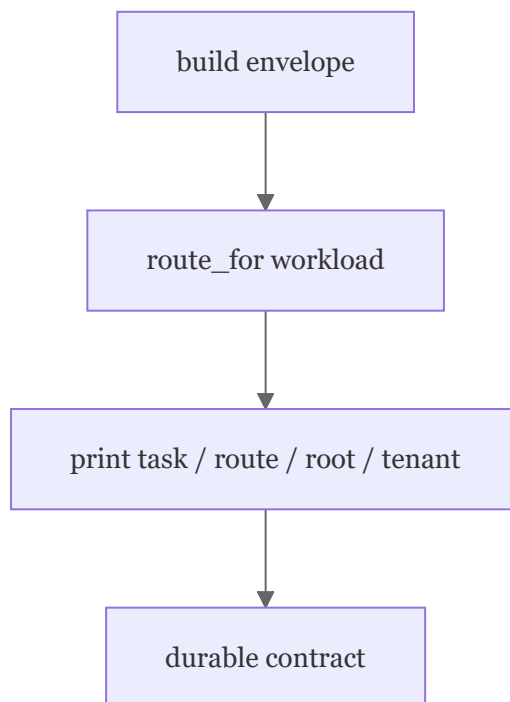
The capstone fails in production when the boundaries are optimized independently but specified poorly together. The root cause is usually contract drift: replay without idempotency, acceleration without queue budgets, or telemetry that observes only one layer of the path.

Examples

Example 1: task envelope, broker route, and verification root

The API or submission shell emits one owned envelope with a workload-specific route and one explicit verification root. This is the durable contract other processes will see.

Watch how the four printed lines come straight off one owned value. The example builds a `TaskEnvelope`, derives the route from its workload variant, and reads the task id, route, root, and tenant back out. Run it to confirm the baseline, then change the workload to a matrix variant and watch the route flip to `tasks.matrix` without touching any other line. That is the whole submission contract in miniature.



One owned envelope in, one routed contract out. The route is a pure function of the workload variant.

Example 45-1. `task_envelope_and_routing.rs` — Envelope routing

```

#[derive(Debug, Clone)]
enum Workload {
    GraphSearch { start: u32, goal: u32 },
    MatrixTile { rows: usize, cols: usize, tile: usize },
}
  
```

```

#[derive(Debug, Clone)]
struct TaskEnvelope {
    task_id: &'static str,
    tenant: &'static str,
    attempt: u32,
    verification_root: u32,
    workload: Workload,
}

fn routing_key(workload: &Workload) -> &'static str {
    match workload {
        Workload::GraphSearch { .. } => "tasks.graph",
        Workload::MatrixTile { .. } => "tasks.matrix",
    }
}

fn main() {
    let task = TaskEnvelope {
        task_id: "task-7",
        tenant: "acme",
        attempt: 1,
        verification_root: 4242,
        workload: Workload::GraphSearch { start: 4, goal: 19 },
    };

    println!("task = {}", task.task_id);
    println!("route = {}", routing_key(&task.workload));
    println!("root = {}", task.verification_root);
    println!("tenant = {}", task.tenant);
}

```

OUTPUT

```

task = task-7
route = tasks.graph
root = 4242
tenant = acme

```

The queue receives one owned task contract rather than borrowed request-local state.

The route names the workload class, which makes queue splits and observability calmer later.

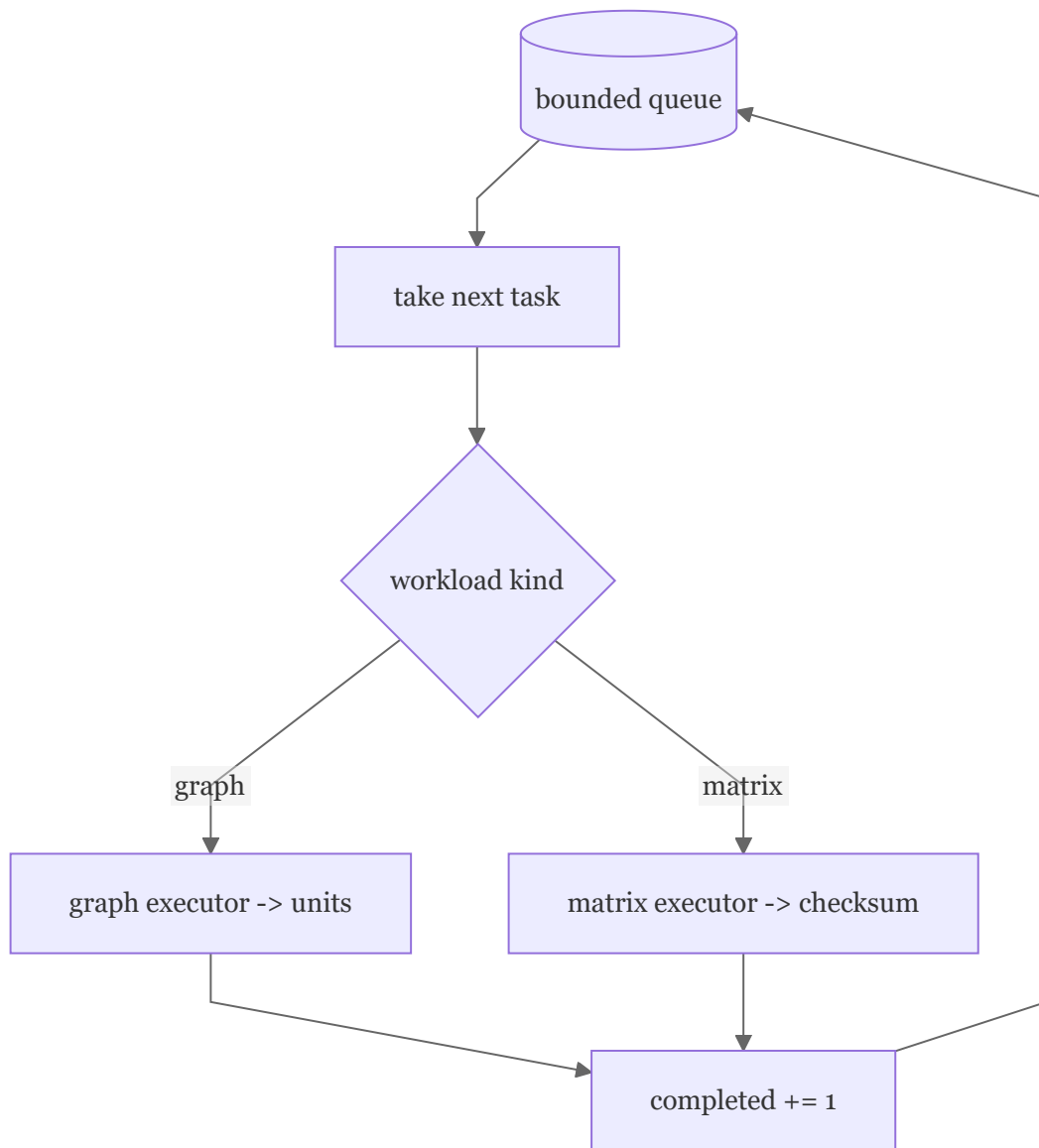
The root is carried explicitly so the worker can reject drift before executing expensive side effects.

Tenant or shard identity belongs on the envelope too, not only in one local API handler variable.

Example 2: worker pool executing graph and matrix workloads

The worker pool owns a bounded queue, then dispatches two different workload classes without losing one clear completion contract.

The three output lines map onto three facts: the pool completed every queued item, the graph lane produced a unit count, and the matrix lane produced a checksum. Follow the diagram one item at a time: each task is drained from the shared queue, dispatched to its lane, executed, and counted as complete. Run it, then add another graph or matrix task to the queue and watch only the matching tally and the completion count move.



One owner drains the queue and routes each item to a lane. Completed count, graph units, and matrix checksum are the three running tallies you see printed.

Example 45-2. worker_pool_graph_and_matrix.rs — Pool workloads

```

use std::collections::VecDeque;

#[derive(Debug, Clone)]
enum Workload {
    GraphSearch { frontier: Vec<u32> },
    MatrixTile { left: [f32; 4], right: [f32; 4] },
}

#[derive(Debug, Clone)]
struct Job {
    id: &'static str,
    workload: Workload,
}

fn graph_units(frontier: &[u32]) -> u32 {
    frontier.iter().copied().sum()
  }

```

```

}

fn matrix_checksum(left: [f32; 4], right: [f32; 4]) -> f32 {
    left.iter().zip(right).map(|(l, r)| *l * r).sum()
}

fn main() {
    let mut queue = VecDeque::from([
        Job {
            id: "graph-1",
            workload: Workload::GraphSearch {
                frontier: vec![2_u32, 3, 5],
            },
        },
        Job {
            id: "matrix-1",
            workload: Workload::MatrixTile {
                left: [1.0_f32, 2.0, 3.0, 4.0],
                right: [2.0_f32, 3.0, 4.0, 5.0],
            },
        },
    ]);

    let mut completed = 0_u32;
    let mut graph_total = 0_u32;
    let mut matrix_total = 0.0_f32;

    while let Some(job) = queue.pop_front() {
        match job.workload {
            Workload::GraphSearch { frontier } => {
                graph_total += graph_units(&frontier);
            }
            Workload::MatrixTile { left, right } => {
                matrix_total += matrix_checksum(left, right);
            }
        }
        completed += 1;
    }

    println!("completed = {}", completed);
    println!("graph units = {}", graph_total);
    println!("matrix checksum = {:.1}", matrix_total);
}

```

OUTPUT

```

completed = 2
graph units = 10
matrix checksum = 40.0

```

The worker owns the next runnable item instead of scattering mutable completion state everywhere.

Graph execution can keep stable handles and frontier metrics separate from dense numeric work.

Matrix work can later swap CPU, MPI, or CUDA executors without changing the outer task contract.

The capstone should still record one durable finish-once checkpoint after these local results exist.

The repository also includes standalone Rust source under `examples/ch45_capstone_distributed_rust_system/` including the two editor-backed examples plus one Tokio worker-topology sketch that shows how the API shell and bounded worker lane fit together outside the in-browser editor.

Summary

PART VII

Networked Services and Secure Boundaries

Build the service layer — REST/OpenAPI, gRPC, WebSockets, TLS, and peer-to-peer — that exposes your system to the world.

-
- 46 FastAPI-Style Web Apps, Swagger, and OpenAPI Codegen
 - 47 gRPC Services with Protobuf and Service API Codegen
 - 48 WebSockets and Long-Lived Connections
 - 49 HTTPS, TLS, and Secure Service Boundaries
 - 50 libp2p and Peer-to-Peer Rust Systems
-

FastAPI-Style Web Apps, Swagger, and OpenAPI Codegen

Design typed Rust web APIs with extractors, state, OpenAPI and Swagger workflows, generated clients, auth, idempotency, and graceful shutdown—without leaking HTTP into the domain

A FastAPI-style developer experience in Rust comes from typed handlers, a documented OpenAPI contract, generated transport code, and domain services kept clear of HTTP concerns. This chapter shows where each of those boundaries belongs and how to keep the contract from drifting away from the code.

Builds on Chapters 19, 25, 41, 42, 43, and 45

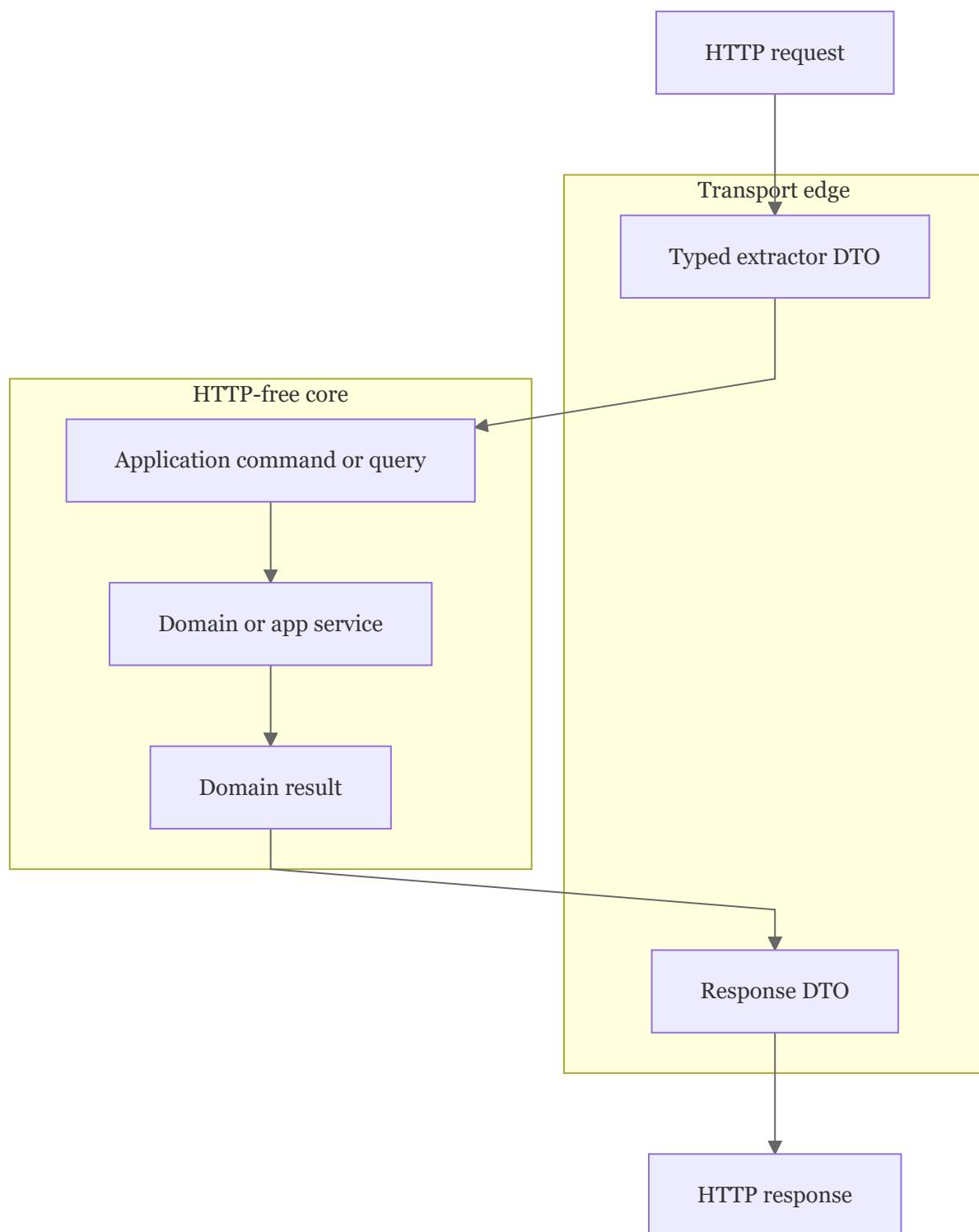
Chapter 19 covered stable wire DTOs and serialization contracts. Chapter 25 covered Tokio and service runtime boundaries. Chapter 41 covered typed errors and translation layers. Chapter 42 covered contract and integration tests. Chapter 43 covered tracing, metrics, and SLOs. Chapter 45 tied those ideas together in one distributed system. This chapter narrows that same discipline to HTTP APIs and OpenAPI contracts.

Opening scenario

A team has a working API prototype, written in something that made it easy to ship, and now needs to move it to Rust. The motivation is the usual set of production pressures: tighter and more predictable latency, real control over concurrency, and explicit handling of the failure cases that kept paging the on-call engineer. What the team does not want to lose is the part of the prototype that made it pleasant to build, namely typed handlers and documentation that other teams can actually read.

The requirement, stated plainly, is a single boundary rule. HTTP DTOs, the generated OpenAPI artifact, and the framework extractors all live at the transport edge. The domain services behind that edge stay independent of HTTP, so they remain easy to test, easy to reuse from a non-HTTP caller, and able to survive a framework change. Most of this chapter is about where to draw that line and how to keep it from eroding.

The shape to hold in your head is a one-way flow that crosses one boundary exactly once. A request enters as transport bytes, is decoded into an HTTP-facing DTO, is converted into a single application command or query, and only then reaches the service. The result travels back out the same way. The conversion happens at the handler and nowhere deeper, which is what keeps the service free of transport types.



Transport types live only at the edge; the command, the service, and the result never see HTTP.

Mental model

Three ideas carry most of the weight in this chapter, and they are easier to hold onto if you state them before reaching for any framework. HTTP is a translation boundary rather than your domain model. The FastAPI-style feel you want comes from types and seams, not from runtime magic. And OpenAPI is an artifact you

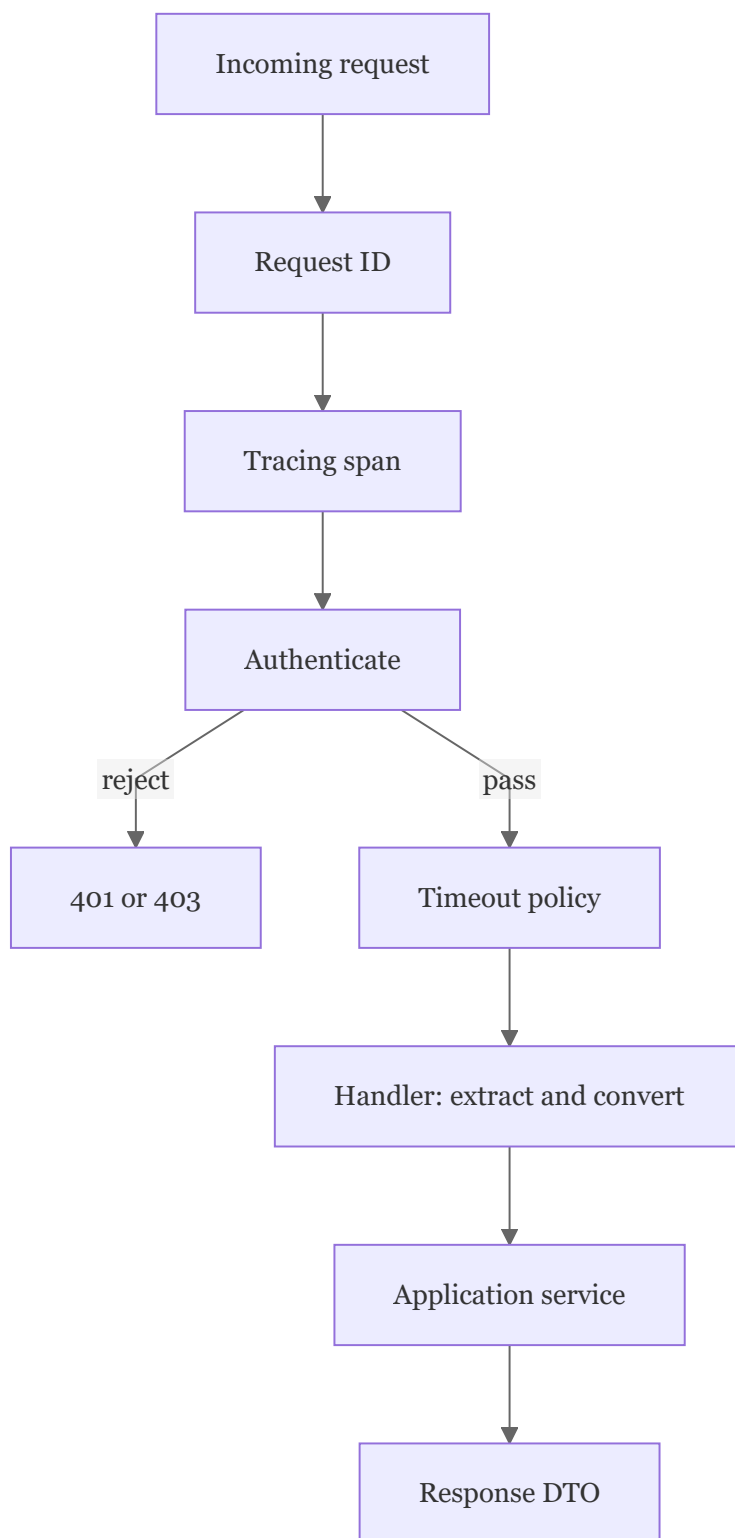
own and review, not a screenshot the build happens to produce. The three cards below say each of these more precisely; the rest of the chapter is mostly consequences of them.

Core concepts

What gives Rust a FastAPI-like feel: routers, extractors, typed state, middleware, and dependency boundaries

The productivity of a FastAPI-style stack is not one feature; it is five small pieces that line up. The router maps an HTTP method and path to one handler and nothing more. Extractors turn the messy parts of a request, such as the path, the query string, the JSON body, and the auth header, into ordinary typed function arguments. Typed state hands the handler its dependencies. Middleware runs the cross-cutting work that every route needs. And the dependency boundary is the single conversion from transport DTOs into an application call. The five cards below take each piece in turn.

Those pieces compose into a request lifecycle with a fixed order. Middleware runs first and outermost, wrapping the handler so that a request ID and a tracing span exist before authentication runs, the timeout policy is in force before the handler starts real work, and an early rejection, such as a failed auth check, short-circuits before the body is ever decoded. Only when the cross-cutting layers pass does the router reach the handler, where extractors decode the typed inputs and the one conversion into an application call happens.



Middleware wraps the handler in a fixed order, so an early rejection never reaches the body or the service.

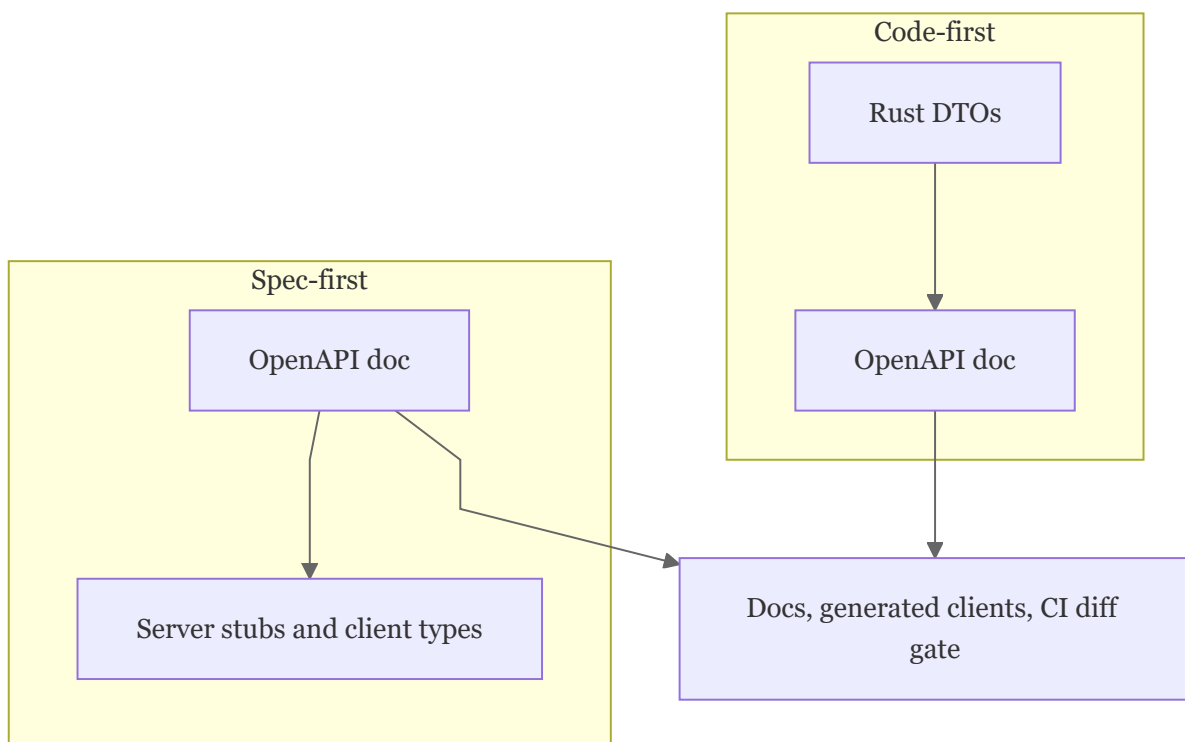
The main difference from Python FastAPI is operational, not stylistic. Rust will not let you pretend a spawned task still borrows request-local state or that an auth claim magically exists everywhere. The signature is part of the ownership model: if a handler needs the caller's identity, it takes a typed caller context as an argument, and if a background task needs request data, that data is moved into it.

Choosing between Rust web frameworks without coupling the domain model to HTTP

Framework choice in Rust attracts more debate than it deserves, because the decision matters far less when the domain is properly insulated. The useful test is not which framework has the nicest extractor syntax; it is how much code would change if you swapped frameworks next year. If the answer is “a handful of handler signatures and the router”, the framework is doing its job as an outer shell. If the answer is “the service layer too”, the transport edge has leaked inward and the framework now owns your architecture. The cards below describe what to insist on regardless of which one you pick.

OpenAPI as a contract: code-first, spec-first, and mixed workflows

The deciding question is which artifact is the source of truth, because every other thing, the docs, the generated clients, and the server, is downstream of it. Code-first makes the Rust DTOs authoritative and derives the spec from them, which keeps spec and server mechanically aligned but means the spec only exists after you write code. Spec-first makes the OpenAPI document authoritative and generates server stubs and client types from it, which lets several teams agree on the contract before any server exists. The diagram shows the two directions of flow through the same OpenAPI document.



*Code-first derives the spec from DTOs; spec-first generates code from the spec.
Either way, one document feeds docs, clients, and the CI diff gate.*

The best workflow is the one that keeps drift reviewable, not the one that is purest in theory. If client teams live off generated SDKs, the spec probably deserves its own checked-in artifact and a diff gate even when Rust DTOs still generate most of it. The failure mode to design against is the same in every workflow: a contract that quietly stops describing the server it claims to document.

Swagger UI integration and documentation ergonomics

Swagger UI is worth treating as a real user interface, because for the teams consuming your API it is the product. It is where another engineer figures out how to call you without reading your handlers, and it is the

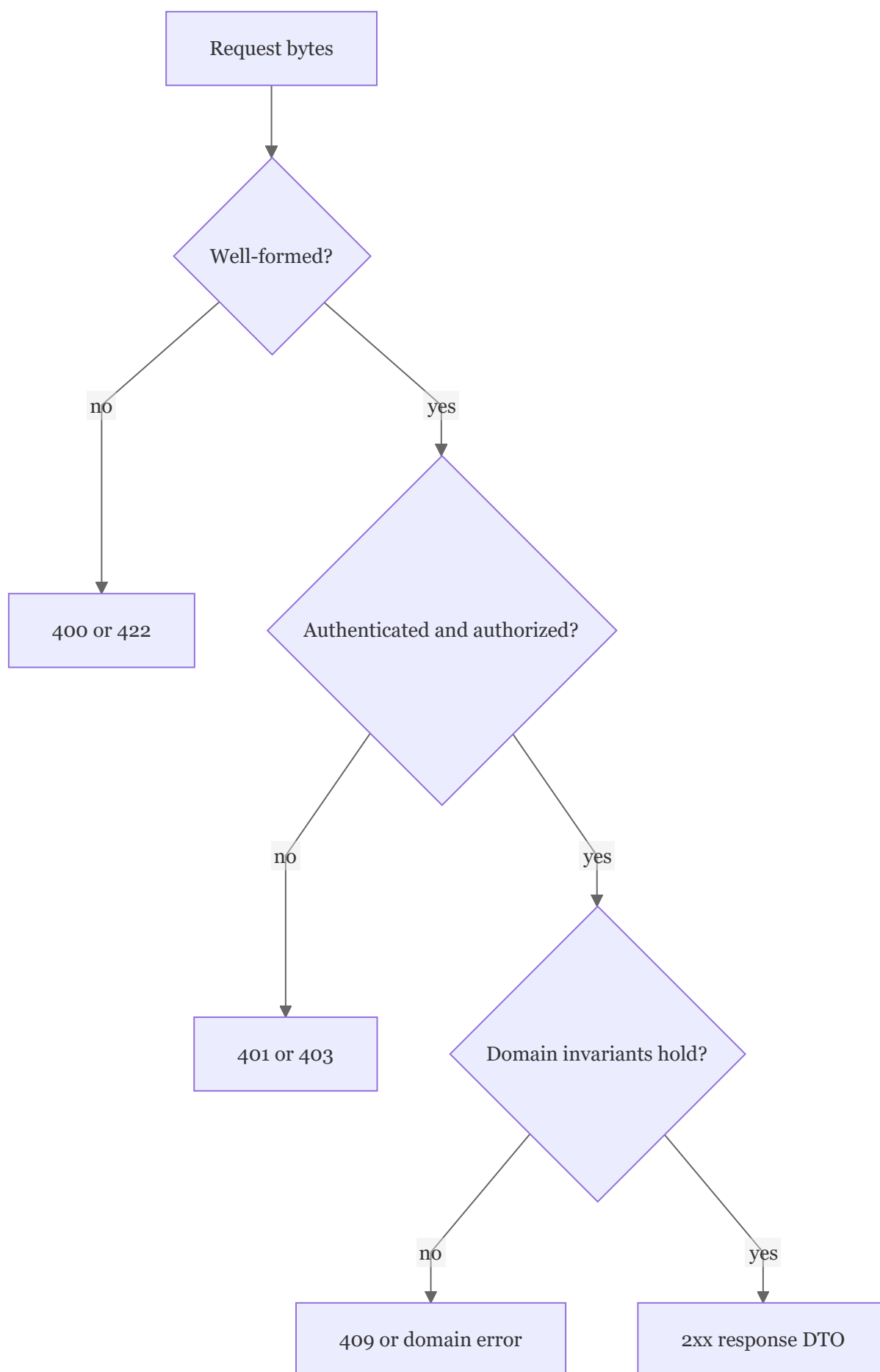
first thing a new teammate opens. That framing changes what “good docs” means: realistic example payloads, documented error responses, clearly marked auth requirements, and stable operation IDs matter far more than prose descriptions. The cards below cover the parts that make the page trustworthy rather than merely present.

Client and server code generation from OpenAPI

Generated code is convenient precisely because it is mechanical, and that is also its risk. A generator encodes its own opinions about naming, optionality, and error shape, and those opinions will not match your domain’s. The rule that keeps this healthy is to let generated types live in the transport layer and never deeper. A generated client gets wrapped behind a small trait so the rest of the system depends on your names, not the generator’s; a generated server stub stays thin and hands off to one application service quickly. Treated that way, regenerating after a spec change is a routine rebuild rather than a refactor that ripples through the domain.

Request validation, typed errors, authentication hooks, and response schemas

It helps to separate two questions that often get blurred. The first is whether the request is well-formed: are the required fields present, is the page size within bounds, does the enum decode, did the path parameter parse? That is transport validation, and it belongs at the edge in the extractor. The second is whether the requested operation is allowed and consistent with domain rules, which can only be answered by the domain itself. Authentication sits between the two: it produces one typed caller context that the handler uses to authorize explicitly. The diagram traces a request through both checks and shows where each failure becomes which status code.



Transport validation, then auth, then domain invariants. Each gate maps to a distinct, reviewable status class.

A common migration mistake is validating only at the HTTP layer and then pretending the domain can never see bad state again. Transport validation and domain invariants are complementary, not interchangeable. The edge keeps malformed input out; the domain keeps invalid operations out. A service that trusts the edge to enforce business rules is one non-HTTP caller away from a bug.

Production API concerns: versioning, pagination, idempotency, tracing, metrics, and graceful shutdown

These are the concerns that do not show up in the first demo and dominate the second year. Each is a promise the contract makes to clients over time. Versioning is the promise that a field's meaning will not change under them. Pagination is the promise that a large result set will not arrive as one unbounded response. Idempotency is the promise that a retried create will not charge a customer twice. Tracing and metrics are the promise that when something goes wrong you can see where, and graceful shutdown is the promise that a deploy will not drop in-flight work. None of these are framework features you switch on; they are design decisions that belong in the contract from the start.

Testing API contracts and preventing documentation drift

Documentation drift is rarely a missing description. It is a handler and an OpenAPI document that slowly stopped describing the same system, with no failing test to announce it. The fix is to make the contract part of the build rather than part of code review. Generate the spec in CI and fail when the checked-in copy drifts; rebuild and compile any published clients against the new contract; and run request and response fixtures against the actual server boundary so serialization, auth hooks, and status mapping stay honest. The four cards below are the layers of that safety net, from cheapest to most thorough.

What changes by background

Most engineers arriving at Rust web work already know how to build an API somewhere else, so the useful framing is not “here is routing” but “here is what to unlearn.” The patterns below describe the one mental shift each background tends to need and the trap that comes from carrying old habits across unchanged. The recurring theme is that Rust makes explicit what your previous stack handled for you: ownership of state, where conversions happen, and where bad input is rejected.

Production patterns

Pitfalls and tradeoffs

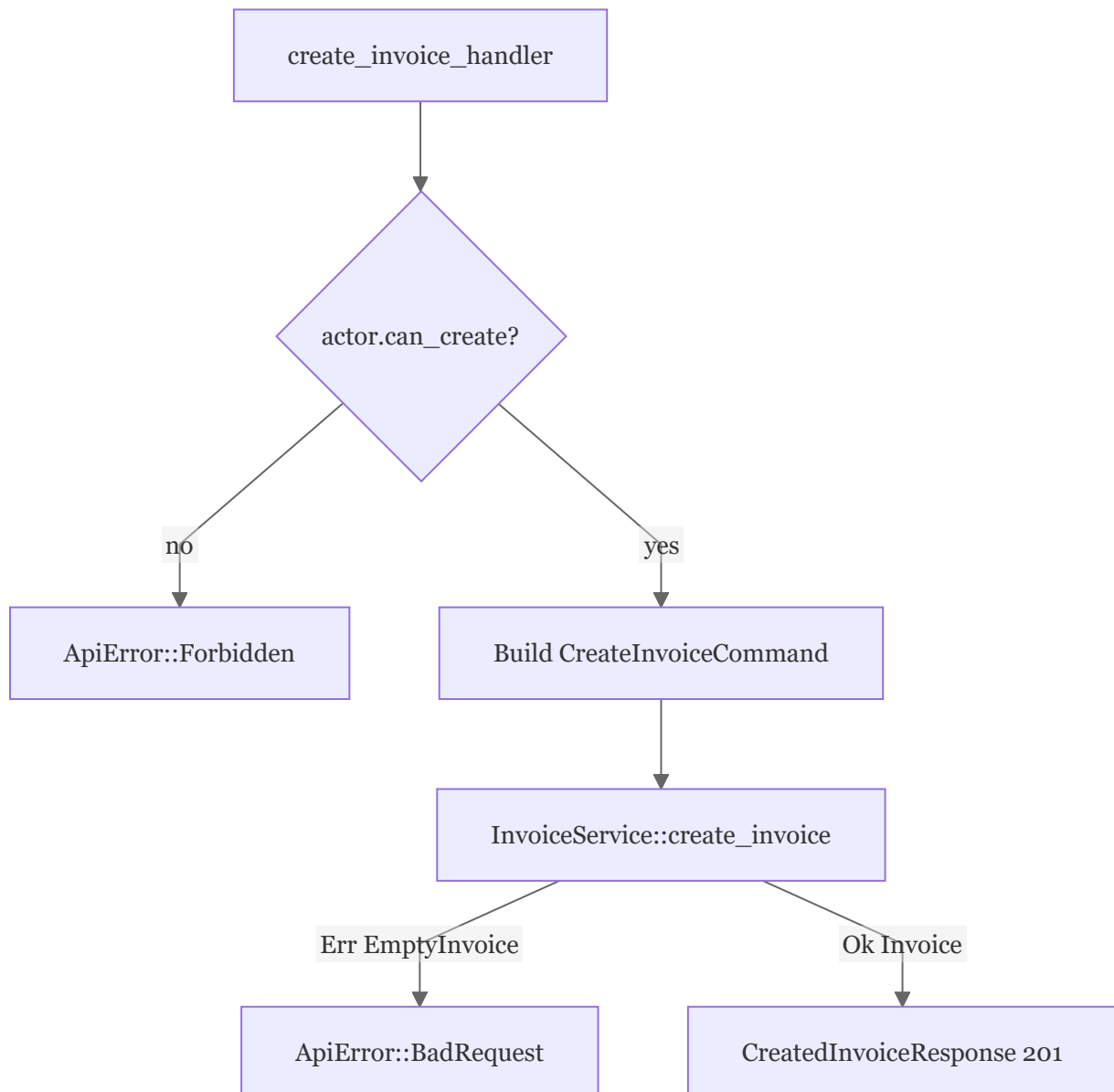
The biggest documentation bug is often not a missing description string. It is a handler and an OpenAPI contract that silently stopped describing the same system.

Examples

Example 1: keep the HTTP handler separate from the application service

The handler owns transport translation and auth. The service owns domain behavior. That seam survives framework swaps much better than a direct framework-shaped domain API.

Read `create_invoice_handler` top to bottom and watch the boundary work: it authorizes using the typed `AuthenticatedUser`, builds exactly one `CreateInvoiceCommand` from the request DTO, calls the service once, and maps the domain's `DomainError` to an `ApiError` only on the way out. The service itself never sees a request type, an auth header, or a status code. The diagram is that handler as a flow, including the two ways it can reject before any work happens.



Authorize, build one command, call the service once, map errors at the edge. The service stays HTTP-free.

Example 46-1. handler_service_boundary.rs — *Service boundary*

```

#![allow(dead_code)]

#[derive(Debug, Clone)]
struct CreateInvoiceRequest {
    customer_id: String,
    line_totals: Vec<u64>,
}

#[derive(Debug, Clone)]
struct CreateInvoiceCommand {
    tenant: String,
    customer_id: String,
    line_totals: Vec<u64>,
}

#[derive(Debug, Clone)]
struct Invoice {
    id: String,
    tenant: String,
    total_cents: u64,
}

#[derive(Debug)]
enum DomainError {
    EmptyInvoice,
    Unauthorized,
}

#[derive(Debug)]
enum ApiError {
    BadRequest(&'static str),
    Forbidden,
}

struct InvoiceService;

impl InvoiceService {
    fn create_invoice(&self, cmd: CreateInvoiceCommand) -> Result<Invoice, DomainError> {
        if cmd.line_totals.is_empty() {
            return Err(DomainError::EmptyInvoice);
        }

        let total_cents: u64 = cmd.line_totals.iter().copied().sum();

        Ok(Invoice {
            id: format!("inv-{}", cmd.customer_id),
            tenant: cmd.tenant,
            total_cents,
        })
    }
}

struct ApiState {
    invoices: InvoiceService,
}

```

```

    service_name: String,
}

#[derive(Debug)]
struct AuthenticatedUser {
    tenant: String,
    can_create: bool,
}

#[derive(Debug)]
struct CreatedInvoiceResponse {
    status: u16,
    request_id: String,
    invoice_id: String,
    tenant: String,
    total_cents: u64,
}

fn create_invoice_handler(
    state: &ApiState,
    actor: AuthenticatedUser,
    request_id: &str,
    body: CreateInvoiceRequest,
) -> Result<CreatedInvoiceResponse, ApiError> {
    if !actor.can_create {
        return Err(ApiError::Forbidden);
    }

    let cmd = CreateInvoiceCommand {
        tenant: actor.tenant,
        customer_id: body.customer_id,
        line_totals: body.line_totals,
    };

    let created = state
        .invoices
        .create_invoice(cmd)
        .map_err(|err| match err {
            DomainError::EmptyInvoice => ApiError::BadRequest("invoice must contain at least one line")
            DomainError::Unauthorized => ApiError::Forbidden,
        })?;

    Ok(CreatedInvoiceResponse {
        status: 201,
        request_id: request_id.to_string(),
        invoice_id: created.id,
        tenant: created.tenant,
        total_cents: created.total_cents,
    })
}

fn main() {
    let state = ApiState {
        invoices: InvoiceService,
        service_name: String::from("billing-api"),
    };

```

```

let actor = AuthenticatedUser {
    tenant: String::from("acme"),
    can_create: true,
};
let request = CreateInvoiceRequest {
    customer_id: String::from("42"),
    line_totals: vec![1_200_u64, 3_000],
};

let created = create_invoice_handler(&state, actor, "req-7", request).unwrap();

println!("status = {}", created.status);
println!("invoice = {}", created.invoice_id);
println!("total cents = {}", created.total_cents);
println!("request id = {}", created.request_id);
}

```

OUTPUT

```

status = 201
invoice = inv-42
total cents = 4200
request id = req-7

```

CreateInvoiceRequest is HTTP-facing shape, not the domain-facing service contract.

The handler builds one command and the service works only from that command.

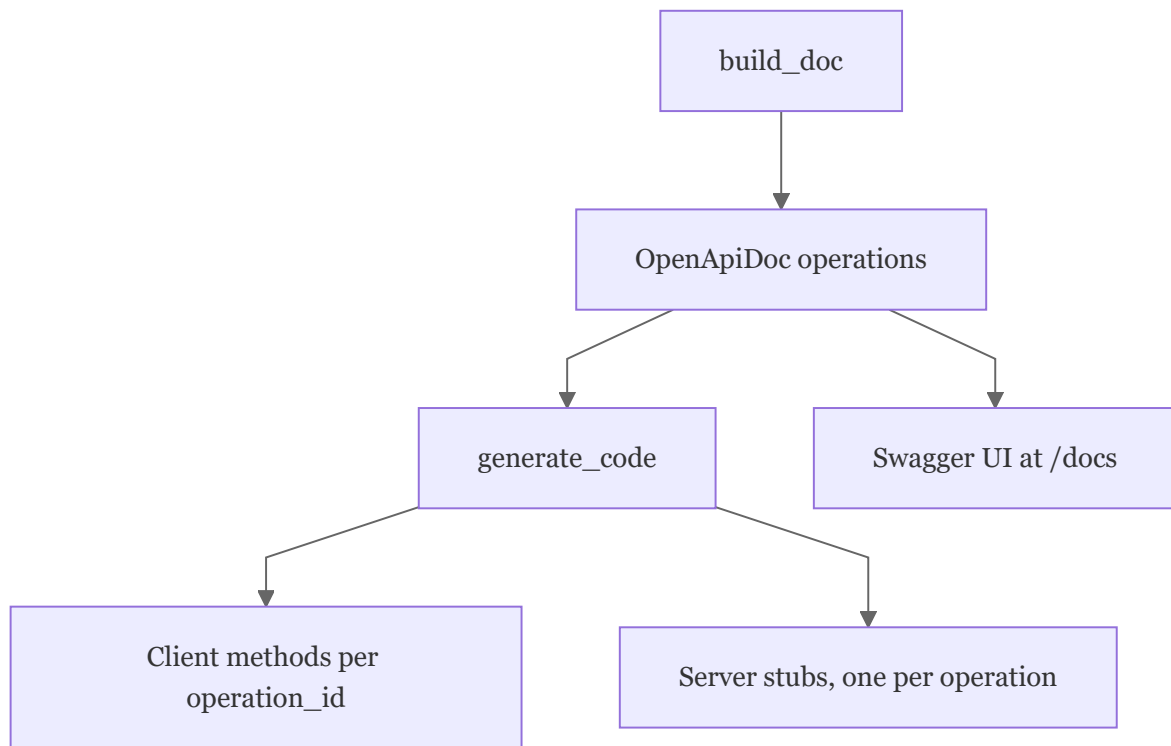
Auth becomes one caller context object, not a global ambient assumption.

Domain rejection is translated to an API-facing error only at the edge.

Example 2: treat OpenAPI and Swagger as contract artifacts, not side effects

The example stays self-contained, but the structure matches real code-first or mixed workflows: define operations once, emit docs once, and generate thin transport code from the same artifact.

The shape to notice is that one value, the OpenApiDoc returned by `build_doc`, is the single source of truth. `generate_code` only reads it, deriving one client method per `operation_id` and counting one server stub per operation. Add an operation to the doc and both the generated client and the stub count follow automatically; nothing is maintained by hand on the side. That is the whole point of treating OpenAPI as an artifact rather than a side effect. The diagram traces that fan-out.



One OpenApiDoc feeds the Swagger surface and the codegen; clients and stubs are derived, never hand-kept.

Example 46-2. openapi_contract_codegen_scaffold.rs — Codegen scaffold

```

#![allow(dead_code)]

#[derive(Debug, Clone)]
struct ApiSchema {
    name: &'static str,
    required_fields: &'static [&'static str],
}

#[derive(Debug, Clone)]
struct ApiOperation {
    method: &'static str,
    path: &'static str,
    operation_id: &'static str,
    request: Option<ApiSchema>,
    response: ApiSchema,
    auth: bool,
}

#[derive(Debug)]
struct OpenApiDoc {
    title: &'static str,
    version: &'static str,
    swagger_ui_path: &'static str,
    operations: Vec<ApiOperation>,
}

#[derive(Debug)]
struct CodegenPlan {

```

```

    client_methods: Vec<&'static str>,
    server_stubs: usize,
}

fn build_doc() -> OpenApiDoc {
    OpenApiDoc {
        title: "Billing API",
        version: "2026-04",
        swagger_ui_path: "/docs",
        operations: vec![
            ApiOperation {
                method: "POST",
                path: "/v1/invoices",
                operation_id: "create_invoice",
                request: Some(ApiSchema {
                    name: "CreateInvoiceRequest",
                    required_fields: &["customer_id", "line_totals"],
                }),
                response: ApiSchema {
                    name: "CreatedInvoiceResponse",
                    required_fields: &["invoice_id", "total_cents"],
                },
                auth: true,
            },
            ApiOperation {
                method: "GET",
                path: "/v1/invoices/{id}",
                operation_id: "get_invoice",
                request: None,
                response: ApiSchema {
                    name: "InvoiceResponse",
                    required_fields: &["invoice_id", "total_cents"],
                },
                auth: true,
            },
        ],
    }
}

fn generate_code(doc: &OpenApiDoc) -> CodegenPlan {
    let client_methods = doc
        .operations
        .iter()
        .map(|op| op.operation_id)
        .collect::<Vec<_>>();

    CodegenPlan {
        server_stubs: doc.operations.len(),
        client_methods,
    }
}

fn main() {
    let doc = build_doc();
    let plan = generate_code(&doc);
}

```

```
println!("operations = {}", doc.operations.len());
println!("swagger = {}", doc.swagger_ui_path);
println!("client methods = {}", plan.client_methods.join(", "));
println!("server stubs = {}", plan.server_stubs);
}
```

OUTPUT

```
operations = 2
swagger = /docs
client methods = create_invoice,get_invoice
server stubs = 2
```

Stable operation IDs are the anchor for generated clients and docs review.

Mount docs intentionally and keep the path and audience explicit.

Generated methods belong to the transport layer, not directly to the domain model.

Generated stubs are helpful only if they terminate quickly at one application service boundary.

The repository also includes standalone Rust source under `examples/ch46_fastapi_style_web_apps_swagger_openapi_codegen/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor, including one extra typed-error and idempotency scaffold.

Summary

gRPC Services with Protobuf and Service API Codegen

Treat protobuf as the contract: unary and streaming gRPC shapes, generated code, transport-to-domain mapping, deadlines, retries, metadata, and polyglot compatibility

A gRPC service is really two things at once: a protobuf schema that several languages share, and a pile of generated Rust code that turns that schema into types, clients, and a server trait. This chapter is about keeping those two things in their place — letting codegen remove boilerplate while you stay in charge of where domain rules, deadlines, retries, metadata, and streaming backpressure actually live.

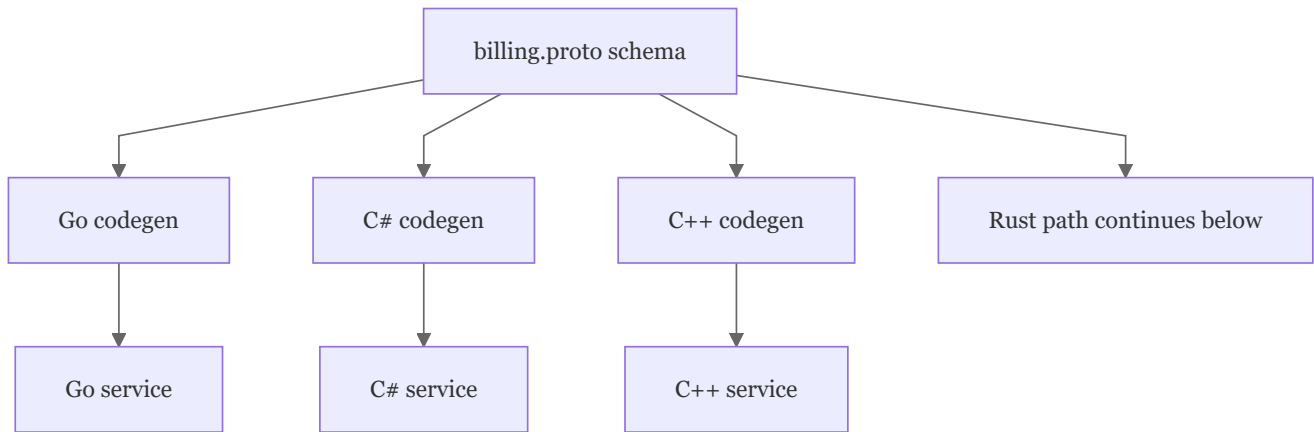
Builds on Chapters 19, 25, 41, 43, and 46

Chapter 19 established transport DTO discipline. Chapter 25 covered async runtime boundaries. Chapter 41 separated domain, infrastructure, and transport errors. Chapter 43 covered tracing and metrics. Chapter 46 established OpenAPI and generated-API workflows on the HTTP side. This chapter applies the same thinking to protobuf and gRPC.

Opening scenario

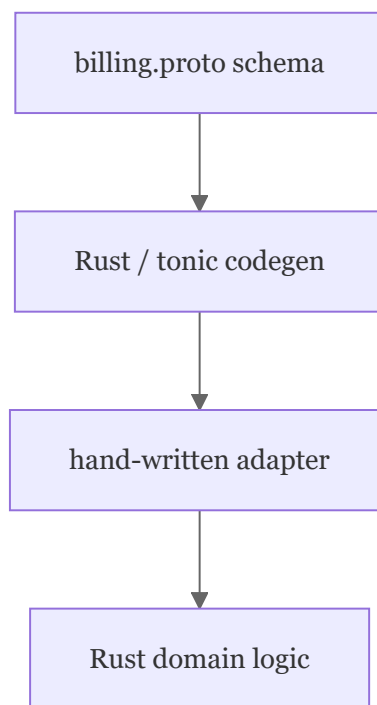
A new Rust service is joining a gRPC fleet that already has Go, C#, and C++ services talking to each other through shared protobuf contracts. Nobody is asking you to invent the contract — it exists, other teams generate from it, and breaking it breaks them. The job is narrower and harder than greenfield work: preserve wire compatibility, expose a generated client and server adapter that feels native in Rust, keep domain rules from seeping into generated transport types, and make deadlines, retries, metadata, and streaming backpressure explicit enough to survive review.

The diagram below is the shape worth holding onto for the rest of the chapter. One .proto file is the single source of truth; each language runs its own code generator and gets its own local API. Rust is just one of those consumers, and the part you control is the thin adapter between the generated surface and your domain.



The same schema feeds every language's code generator. Go, C#, and C++ each get their own local surface.

The Rust branch off that same schema is the part you actually own — it runs one more step than the others:



The part you own in Rust is the adapter between generated transport types and your domain logic.

Mental model

Core concepts

The protobuf contract: messages, fields, services, and streaming shapes

gRPC design starts in the schema, not in handler code, and the schema carries two distinct kinds of decision. The message blocks describe data: each field has a name, a type, and — most importantly — a number. Those numbers, not the names, are what actually go on the wire, which is why they become the unit of long-term

compatibility. The service block describes behavior: which methods exist and, for each one, whether either side streams. The four RPC shapes you can spell here are the same four every gRPC language generates from.

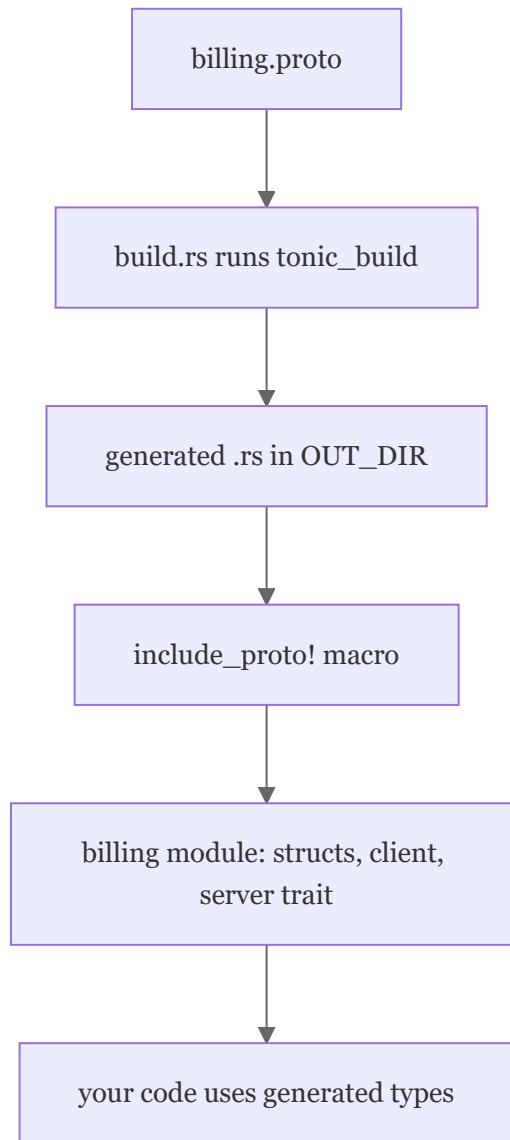
When you read the schema below, look at two things before anything else. First, the field numbers (`= 1`, `= 2`) — those are the durable identity you must not reuse. Second, the four `rpc` lines and where `stream` appears, because that one keyword decides whether a method is a request-response call or a long-lived pipe.

Each `rpc` line in that service maps to exactly one of four communication shapes, and the position of the `stream` keyword is the whole difference. The cards below name them; the rest of the chapter keeps coming back to the operational cost each one carries.

How a .proto file becomes Rust types: the build-time codegen pipeline

In Rust the standard tool for this is `tonic` together with its companion build crate. The mechanism is a `Cargo build.rs` script that runs before your crate compiles: it reads the `.proto` files, generates Rust source into Cargo's output directory, and a small `include_proto!` macro pulls that generated source into a module of your choosing. The result is that the generated code is never checked into the repository as Rust — it is reproduced from the schema on every build, which is exactly what keeps the two from drifting apart.

The flow below is worth tracing once end to end. The thing to notice is the `rerun-if-changed` line in the build script: it tells Cargo to re-run codegen whenever the schema changes, so editing the `.proto` is enough to regenerate everything downstream. You never hand-edit generated output, and you never let it go stale.

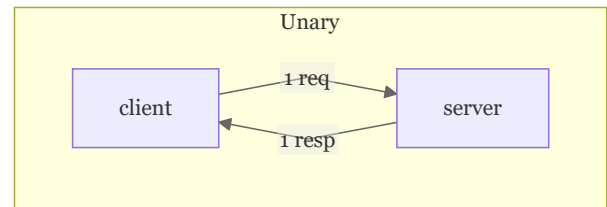
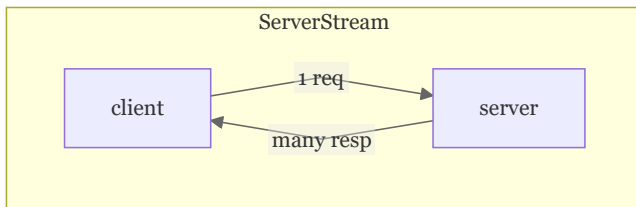


Editing the schema re-triggers build.rs, which regenerates the module your code depends on.

In practice, many Rust teams keep generated protobuf and tonic surfaces in a transport crate or module, then make the application crate depend on a smaller hand-written interface. That keeps protobuf churn from looking like domain churn.

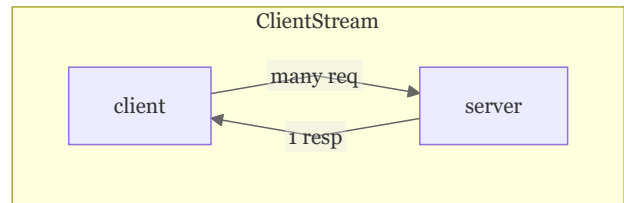
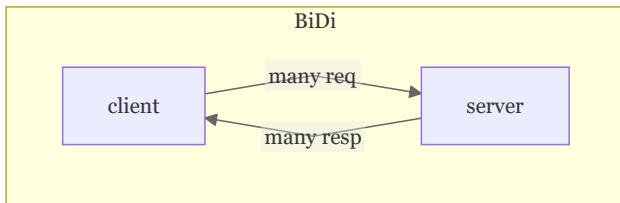
Reading the four streaming shapes off the generated server trait

The four rpc shapes from the schema turn directly into a Rust trait, and you can read the operational contract straight off each method signature. The diagram below shows what each shape looks like as message flow between client and server — one message each way, a fan-out of responses, a fan-in of requests, or a full duplex conversation.



The two simplest shapes: one message each way, or one request that fans out into many responses.

The two streaming shapes that flip the fan-in direction round out the set:



Many requests collapsing into one response, or both sides streaming over one long-lived channel.

Now read the trait against that picture. Codegen produces the trait below; your server type writes the impl block that fills in real bodies. The detail that catches people new to tonic is the two associated type declarations: a server-streaming method does not return a concrete stream, it returns `Self::StreamInvoicesStream`, and you must name a concrete type that satisfies that bound. That associated type is where the buffering and ownership decisions for the stream actually get pinned down.

Streaming APIs are rarely “just like unary but more advanced.” They are pacing systems. If message rate, buffer size, and cancellation rules are vague, the streaming design is still unfinished.

Mapping generated transport types into domain models

This is the single most consequential habit in the chapter, and it is easy to skip because the generated types are right there and already own their data. The temptation is to pass a `CreateInvoiceRequest` straight into business logic. Resist it. A generated message is a data transfer object shaped by wire concerns: every field is optional-ish, names follow protobuf conventions, and the type knows nothing about your invariants. The moment domain code starts pattern-matching on transport structs, a schema change becomes a domain change, and a Go team's additive field edit can ripple into your business layer. The fix is a single translation seam at the edge — convert the request into a typed domain command on entry, translate the domain result back into a response or a gRPC status on exit, and let everything in between speak only your own vocabulary.

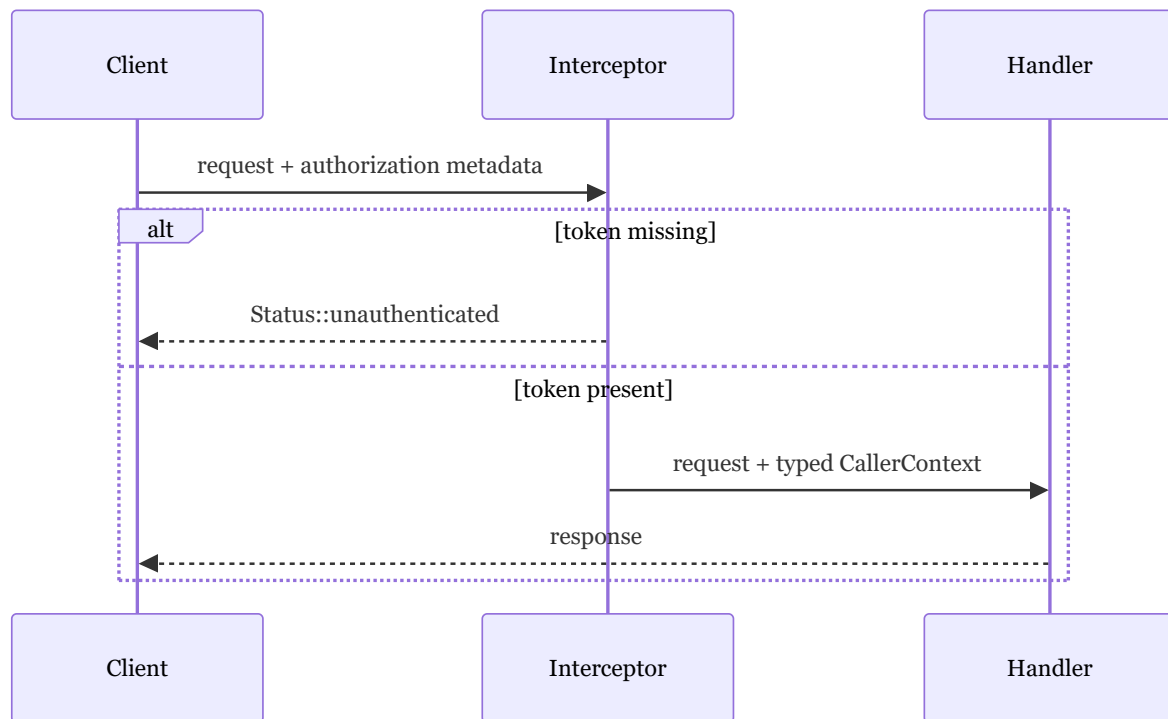
Error models: status codes, rich errors, retries, deadlines, and cancellation

gRPC's error vocabulary is a small set of status codes, and the discipline is choosing them by what the caller should do next rather than by what went wrong internally. A code like `InvalidArgument` tells the caller “fix your request and stop retrying”; `Unavailable` tells it “I am temporarily down, try again.” Flattening every failure into one generic code throws that signal away. The two control-flow concepts that ride alongside status codes — deadlines and cancellation — are just as load-bearing: a caller can attach a deadline, and the server is expected to stop working once it passes, because results nobody is waiting for are pure waste. The cards below separate the four pieces that usually get tangled together.

Interceptors, authentication metadata, tracing, and observability

gRPC carries side-band data — auth tokens, trace IDs, tenant identifiers — in per-call metadata, the protobuf equivalent of HTTP headers. An interceptor is the seam that runs before your handler, reads that metadata, and either rejects the call or enriches it. The pattern that keeps handlers clean is to do the untyped work once in the interceptor and hand the handler a typed value instead: validate the token, then attach a `CallerContext` to the request extensions so the handler reads identity as a struct, not as raw header strings.

The sketch below is exactly that flow: pull the authorization metadata, turn a missing token into an `Unauthenticated` status before any handler runs, and otherwise insert a typed caller context. Read it against this sequence:



The interceptor turns raw metadata into a typed context or a rejection before the handler ever runs.

Staying compatible across Go, C#, C++, Python, and the rest of the fleet

In a polyglot fleet the schema is a shared interface, and the rule of thumb is that protobuf evolution must be additive and boring. Field numbers are the wire identity, so the safe moves are narrow: add new fields with new numbers, mark removed fields as reserved so nobody recycles them, and never change a tag's meaning in place. Everything else — Rust generating async traits, Go generating interfaces, C# generating async bases — is a local cosmetic difference that the shared schema papers over, as long as the schema itself stays stable.

Go, C#, C++, and Rust will each generate a different local API shape from the same protobuf. Review the schema as the contract and treat generated language surfaces as adapters around that contract.

Testing generated APIs and preventing breaking protobuf changes

Testing splits cleanly into two questions: does your adapter behave, and is the schema still safe for everyone else. The first is ordinary unit testing of the mapping seam — cheap, fast, and the place most real bugs hide. The second needs tooling the compiler cannot provide, because a local cargo build passing tells you nothing

about whether a partner team's stale client still parses your messages. That is what descriptor-diff and breaking-change checks in CI are for. The items below cover both halves.

The most useful breaking-change gate is the one that runs before a partner team regenerates from a stale contract. Treat schema drift as a build failure, not as a post-rollout surprise.

What changes by background

Every language in this list generates from the same protobuf, so the wire is not where you will be surprised. The surprise is what each ecosystem used to do for you automatically that Rust now asks you to state. Read your own row as the habit you most need to unlearn.

Production patterns

Pitfalls and tradeoffs

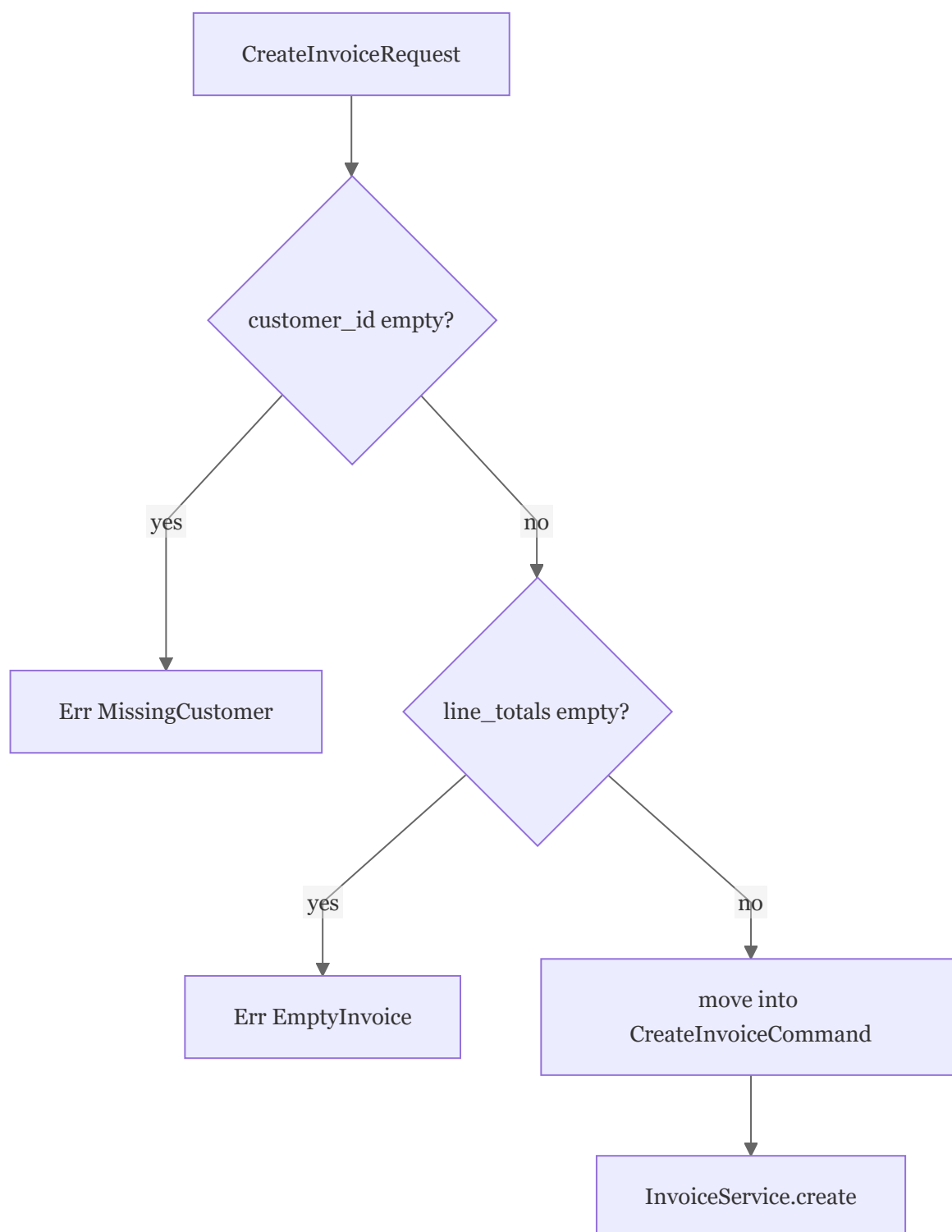
The most expensive gRPC mistake is often not a tonic call-site bug. It is one contract that kept compiling locally while schema evolution, retries, or streaming pace drifted apart across services.

Examples

Example 1: map a generated request into a domain command

The transport request owns strings and vectors already. The adapter converts it into one application command, and the service never learns about gRPC metadata or generated transport wrappers.

What to look at: `into_command` is the entire boundary. It runs two validation checks, and only on success does it move the request's owned strings and vectors into a `CreateInvoiceCommand`. A missing customer or an empty invoice never reaches `InvoiceService`; it returns a `TransportError` at the edge. Trace that path before reading the code:



Validation gates first; only a clean request is moved into a domain command.

Example 47-1. `transport_mapping.rs` — Domain mapping

```

#[derive(Debug, Clone)]
struct CreateInvoiceRequest {
    customer_id: String,
    line_totals: Vec<u64>,
}

#[derive(Debug, Clone)]

```

```

struct CreateInvoiceCommand {
    tenant: String,
    customer_id: String,
    line_totals: Vec<u64>,
}

#[derive(Debug)]
enum TransportError {
    MissingCustomer,
    EmptyInvoice,
}

fn into_command(
    request: CreateInvoiceRequest,
    tenant: &str,
) -> Result<CreateInvoiceCommand, TransportError> {
    if request.customer_id.trim().is_empty() {
        return Err(TransportError::MissingCustomer);
    }

    if request.line_totals.is_empty() {
        return Err(TransportError::EmptyInvoice);
    }

    Ok(CreateInvoiceCommand {
        tenant: tenant.to_string(),
        customer_id: request.customer_id,
        line_totals: request.line_totals,
    })
}

struct InvoiceService;

impl InvoiceService {
    fn create(&self, command: &CreateInvoiceCommand) -> String {
        format!("inv-{}", command.customer_id)
    }
}

fn main() {
    let request = CreateInvoiceRequest {
        customer_id: String::from("cust-7"),
        line_totals: vec![1_200_u64, 3_000],
    };

    let command = into_command(request, "acme").unwrap();
    let total_cents: u64 = command.line_totals.iter().copied().sum();
    let service = InvoiceService;
    let invoice_id = service.create(&command);

    println!("tenant = {}", command.tenant);
    println!("customer = {}", command.customer_id);
    println!("total cents = {}", total_cents);
    println!("invoice = {}", invoice_id);
}

```

OUTPUT

```
tenant = acme
customer = cust-7
total cents = 4200
invoice = inv-cust-7
```

The request type is transport-owned input, not the application service contract itself.

into_command is where request validation and naming translation stay explicit.

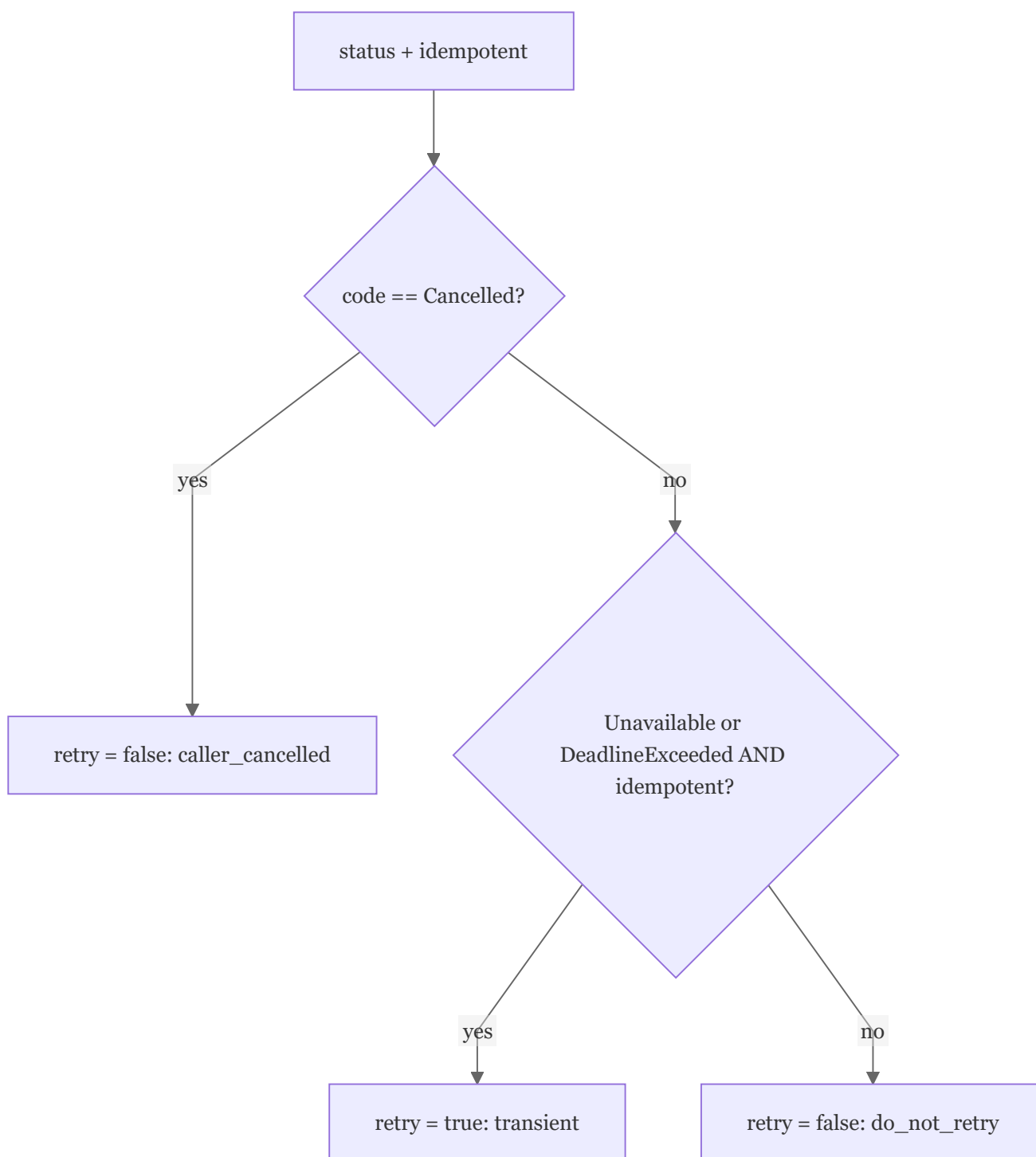
Strings and vectors are owned once at the transport edge, then moved into the application command.

This seam is cheap to unit test without a live server or generated client.

Example 2: classify retry and cancellation decisions explicitly

The transport boundary decides whether a status is transient and whether the work should abort after cancellation or deadline expiry. That policy should not be hidden in ad hoc client code.

What to look at: classify is a single match that turns a status code plus an idempotency flag into a yes/no retry decision, and the order of arms matters. Only Unavailable and DeadlineExceeded on an idempotent call are transient; Cancelled is its own answer; everything else falls through to "do not retry." The separate should_abort function is the cancellation/deadline check. Follow the decision tree, then read the match:



One policy function, read top to bottom: cancellation, then transient-and-idempotent, then reject.

Example 47-2. `status_deadline_retry.rs` — *Deadline retry*

```

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
enum GrpcCode {
    InvalidArgument,
    DeadlineExceeded,
    Unavailable,
    Cancelled,
}

```

```

#[derive(Debug, Clone, Copy)]
struct GrpcStatus {
    code: GrpcCode,
    message: &'static str,
}

#[derive(Debug, Clone, Copy)]
struct RetryDecision {
    retry: bool,
    reason: &'static str,
}

fn classify(status: GrpcStatus, idempotent: bool) -> RetryDecision {
    match status.code {
        GrpcCode::Unavailable | GrpcCode::DeadlineExceeded if idempotent => RetryDecision {
            retry: true,
            reason: "transient",
        },
        GrpcCode::Cancelled => RetryDecision {
            retry: false,
            reason: "caller_cancelled",
        },
        _ => RetryDecision {
            retry: false,
            reason: "do_not_retry",
        },
    }
}

fn should_abort(deadline_ms: u64, elapsed_ms: u64, cancelled: bool) -> bool {
    cancelled || elapsed_ms >= deadline_ms
}

fn main() {
    let unavailable = GrpcStatus {
        code: GrpcCode::Unavailable,
        message: "peer restarting",
    };
    let invalid = GrpcStatus {
        code: GrpcCode::InvalidArgument,
        message: "line_totals must not be empty",
    };

    let retry_unavailable = classify(unavailable, true);
    let retry_invalid = classify(invalid, true);

    println!("retry unavailable = {}", retry_unavailable.retry);
    println!("retry invalid = {}", retry_invalid.retry);
    println!("cancelled = {}", should_abort(150, 120, true));
}

```

OUTPUT

```

retry unavailable = true
retry invalid = false
cancelled = true

```

Unavailable and deadline failure are only retryable when the operation and caller semantics allow it.

Abort logic belongs in the service boundary before useless work keeps running after the caller is gone.

One explicit policy function is easier to test and document than many scattered retry branches.

The repository also includes standalone example assets under `examples/ch47_grpc_services_protobuf_service_api_codegen/` including a small protobuf contract, a build-script sketch, and the two transport-focused Rust examples used by this chapter.

Summary

CHAPTER 48 · WEBSOCKETS AND LONG-LIVED CONNECTIONS

WebSockets and Long-Lived Connections

Model long-lived connection systems with explicit read/write ownership, bounded fan-out, versioned envelopes, reconnect policy, and graceful shutdown

A request handler lives for milliseconds; a WebSocket lives for hours. That single change of timescale is what this chapter is really about. Once a connection sticks around, it acquires a lifecycle, its own memory and tasks, a pace it can be pushed faster than, and a way it has to shut down. We treat each connection as a small long-lived subsystem and work through who owns what, how to keep slow clients from sinking the server, and how to retire a socket cleanly.

Builds on Chapters 23, 24, 25, 30, 42, 43, and 47

Chapter 23 covered channels and synchronization choices. Chapter 24 explained futures, Pin, and cancellation. Chapter 25 covered Tokio runtime behavior. Chapter 30 covered broker-based fan-out and retry pressure. Chapter 42 covered deterministic failure-mode testing. Chapter 43 covered tracing and queue-age metrics. Chapter 47 compared streaming transport options on the gRPC side.

Opening scenario

An operations team runs an admin console that shows live task progress and pushes operator notifications. The naive version is an afternoon of work: open a socket, loop over events, write them out. It demos beautifully. Then real usage arrives. Operators leave the console open in five tabs across two monitors. A deploy bounces the load balancer and every browser reconnects in the same second. One operator is on hotel Wi-Fi and reads messages slower than the server produces them. A new release adds a field to an event and older tabs start choking on it.

None of those are exotic failures. They are the ordinary weather of long-lived connections, and the afternoon prototype has no answer for any of them. What the team actually needs is a connection model that survives many tabs, reconnect storms, slow consumers, bounded fan-out, protocol versioning, and graceful shutdown. The rest of this chapter builds that model one decision at a time.

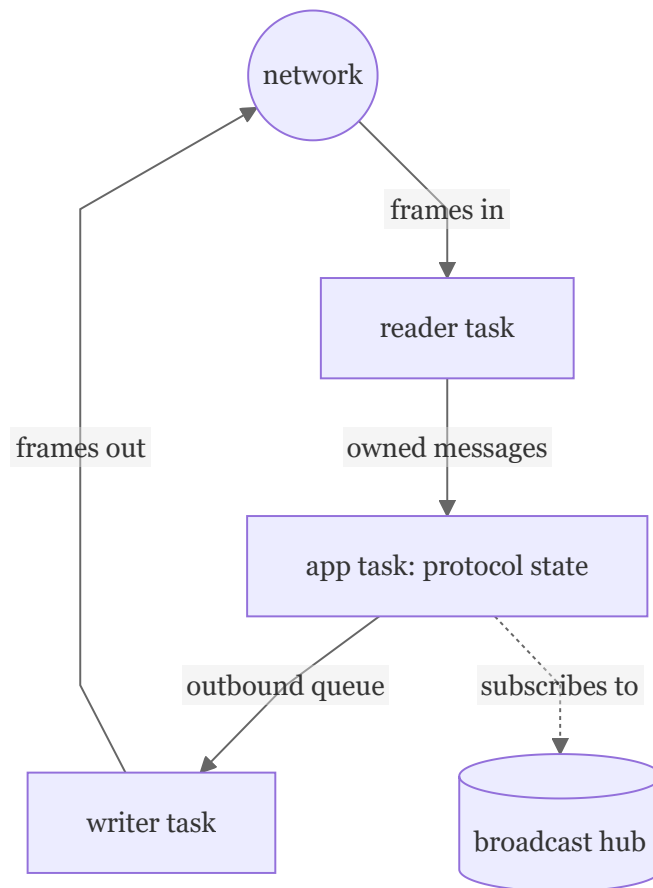
A useful design order

1. Decide who owns read IO, write IO, and protocol state before you write handler code.
2. Put a bound on outbound buffering before the first slow client appears.
3. Version the message envelope before mixed client releases force the issue.
4. Write reconnect, shutdown, and observability policy as part of the transport design, not after it.

Mental model

Before the details, fix the picture. A WebSocket connection is not a function that returns; it is a small running machine with three internal jobs. One job reads inbound frames off the wire. One job writes

outbound frames to the wire. A third job in the middle holds the application state and decides what to send. The two transport jobs are the only code that ever touches the socket, and they talk to the application through bounded channels rather than shared locks. Keep that shape in mind; almost every recommendation in this chapter is a consequence of it.



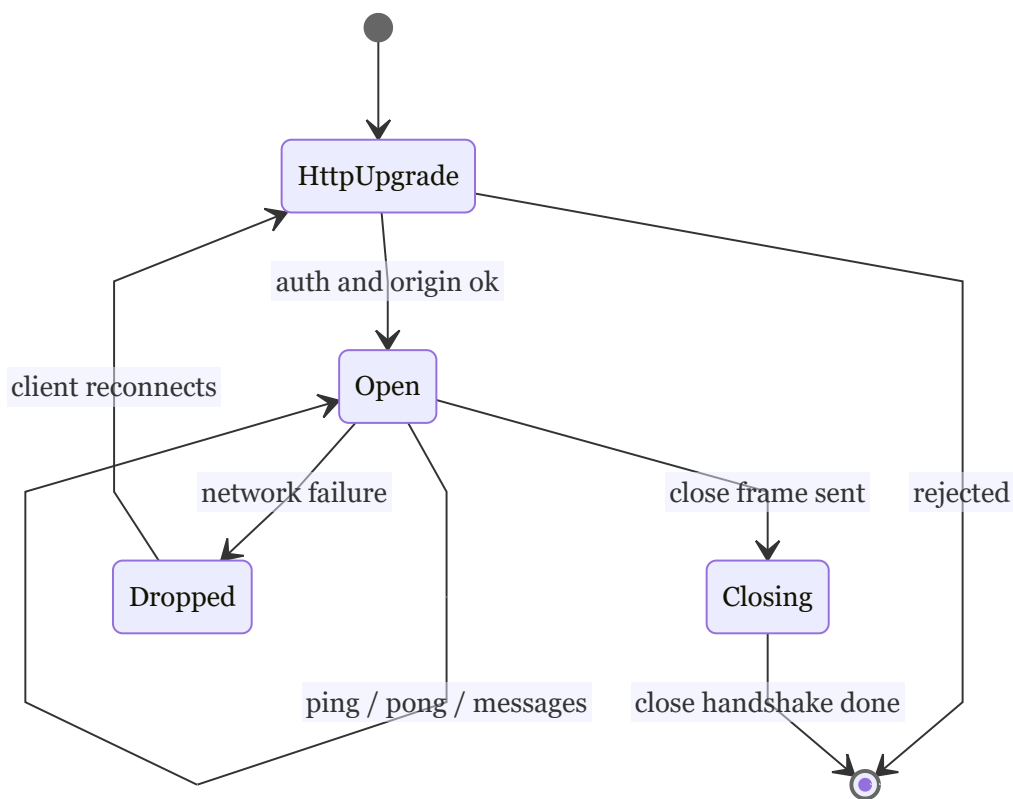
One reader, one application owner, one writer. Only the reader and writer touch the socket; everything else flows through owned messages on bounded channels.

Core concepts

The work divides into five recurring concerns: the lifecycle the connection moves through, who owns which part of it, how the server protects itself when a client falls behind, how the message protocol survives being changed under live traffic, and how to authenticate and shut down without leaving sockets dangling. We take them in that order, because each one assumes the previous one is already settled.

The connection lifecycle: upgrade, split, heartbeat, close, reconnect

A connection moves through a small set of phases, and each phase has different rules about what is allowed. The thing to watch in the diagram below is where ordinary HTTP discipline ends and long-lived state begins: everything to the left of the split, authentication, origin checks, headers, rate limits, is normal request handling, and it must finish before the socket starts carrying application traffic. After the split, you are in a steady state that only two events can leave: an explicit close handshake, or a drop that the client answers with a reconnect.



The upgrade is the last moment normal HTTP auth applies. Once Open, only a close handshake or a drop-then-reconnect leaves the steady state.

One practical correction is worth keeping in mind: ping or pong is transport liveness. Resume cursors, application acknowledgments, or last-seen sequence numbers are product liveness. Keep both, and do not let one pretend to be the other.

Who owns the read side, the write side, and the protocol state

In most async Rust WebSocket stacks the first thing you do after the upgrade is split the connection into a read half and a write half. That split is not an implementation detail; it is the ownership boundary the rest of the design hangs on. The read half goes to exactly one task, the write half goes to exactly one task, and any application logic that needs to react to messages or push updates sits in a third owner that never touches the socket directly. The cards below assign each responsibility to a single owner; the reason this matters is that the alternative, many tasks sharing the write half behind a lock, makes ordering, close, and error handling into a race that Rust would rather you not write in the first place.

The calm default is usually reader -> app -> writer. A broad shared writer lock often looks simpler at first and becomes harder to cancel, order, and test later.

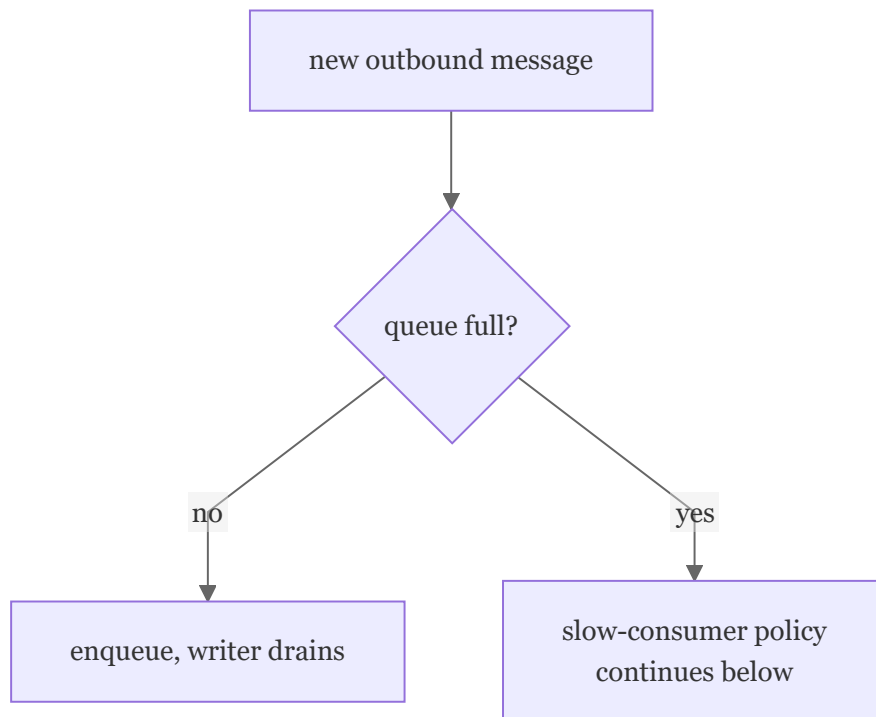
How this lands if you come from another language

The one-reader, one-writer, one-application-owner shape is the same in any language. What differs is the habit you arrive with, and which of those habits Rust quietly refuses to let you keep. Each card below is the mental-model shift, not an API mapping.

Keeping a slow client from sinking the server

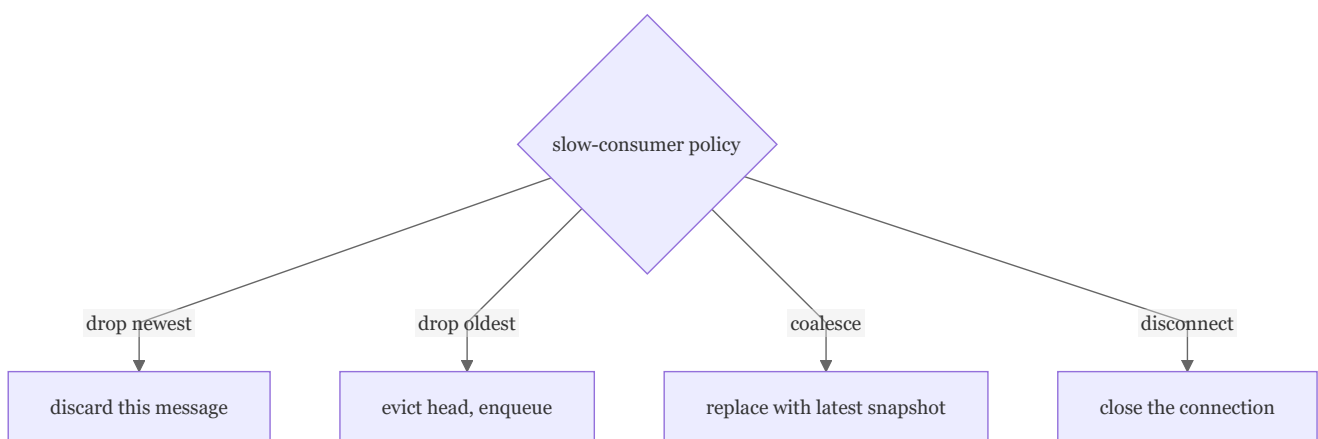
Backpressure is the question of what happens when you produce messages faster than a client can read them. The honest framing is that you are always answering it, even when you never wrote the answer down:

an unbounded queue is a policy that says grow memory until the process dies. The diagram traces the decision every push makes. The fork that matters is the one in the middle, when the per-connection queue is full, you are forced to choose between dropping data, coalescing it, or dropping the client, and that choice is a product decision, not a transport detail.



The entry fork: with room in the queue, enqueue and the writer drains it. When the queue is full, control passes to the slow-consumer policy.

When the queue is full, the policy you chose ahead of time decides which of four things to sacrifice:



The four slow-consumer outcomes. Each one sacrifices something different: the new data, the old data, freshness across updates, or the client itself.

An unbounded broadcast path is not “more reliable.” It simply delays the moment you discover which client or tenant is too slow for the current policy.

Designing a message protocol that survives being changed

A WebSocket protocol drifts exactly the way an HTTP API or a broker envelope drifts, and for the same reason: old clients and new servers run side by side for as long as someone keeps a tab open. The defense is to make the contract explicit in the envelope rather than implicit in the deploy. Two ideas carry most of the weight. First, every message carries a version, a kind, and a stable identity, so a mixed fleet stays debuggable. Second, transport-level messages, ping, pong, close, subscribe, ack, are a different layer from domain events like `invoice_created`, and keeping them separate in the type system keeps them separate in your head.

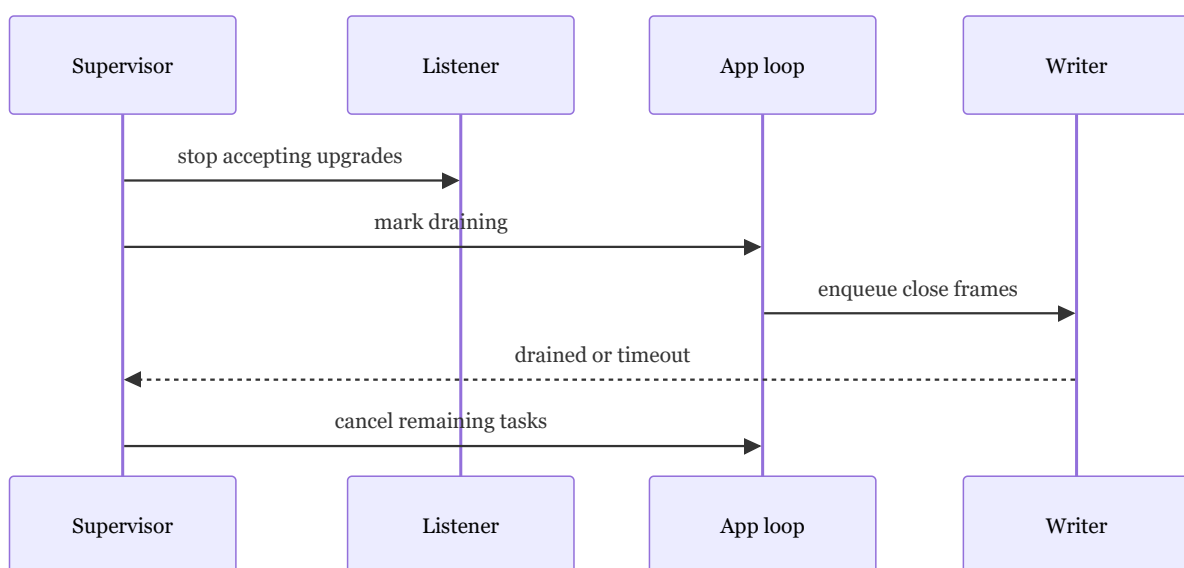
In the sketch below, look at the two derives. The `#[serde(tag = "type")]` on the client message turns the enum into a tagged union on the wire, so adding a new variant is additive and old clients simply never send it. The generic `Envelope<T>` wraps any payload with the version and trace fields so routing and logging never have to parse the body first.

Authentication, authorization, origin checks, and rate limiting

The dangerous assumption with a long-lived connection is that proving who you are once, at the upgrade, also settles what you are allowed to do for the next several hours. It does not. Authentication happens at the handshake, where you still have ordinary HTTP cookies, headers, and origin to work with. Authorization happens on every message, because a connection can be perfectly authenticated and still have no business subscribing to a particular stream or issuing a particular command. Keep those two as separate checks, and remember that flooding handshakes and flooding messages are different attacks that need separate rate limits.

Shutting down without dropping sockets on the floor

Graceful shutdown is an ordering problem, and the order is the same one TCP services and brokers use: stop admitting new work, drain what is already in flight, then cancel the rest with a deadline. The sequence below shows why the order is not negotiable. If you cancel the connection tasks first, the close frames never go out and clients see an abrupt drop; the entire close policy you wrote simply did not run. The signal flows from a supervisor through a watch or cancellation channel that the writer and application loop are already watching, so shutdown is something they observe rather than something done to them.



Stop admission, drain through the writer, then cancel with a timeout. Cancelling first would skip the close frames entirely.

The important sequencing rule is the same as in TCP services and brokers: stop admission first, drain or close second, then cancel the rest with a timeout. If you drop the runtime underneath active sockets, your close policy never actually ran.

Testing WebSocket protocols and failure modes

When to reach for something other than a WebSocket

Everything above is the cost of running a WebSocket well, which is the best argument for checking whether you need one. The deciding question is almost always direction. If the client only ever listens, a server-sent-events stream gives you push over plain HTTP with none of the bidirectional ceremony. If updates are rare, polling is genuinely simpler infrastructure. gRPC streaming wins for typed service-to-service traffic, and a message broker wins when producers and consumers should be decoupled in time. Pick a WebSocket when both sides truly need to talk over one long-lived session, not by reflex.

Production patterns

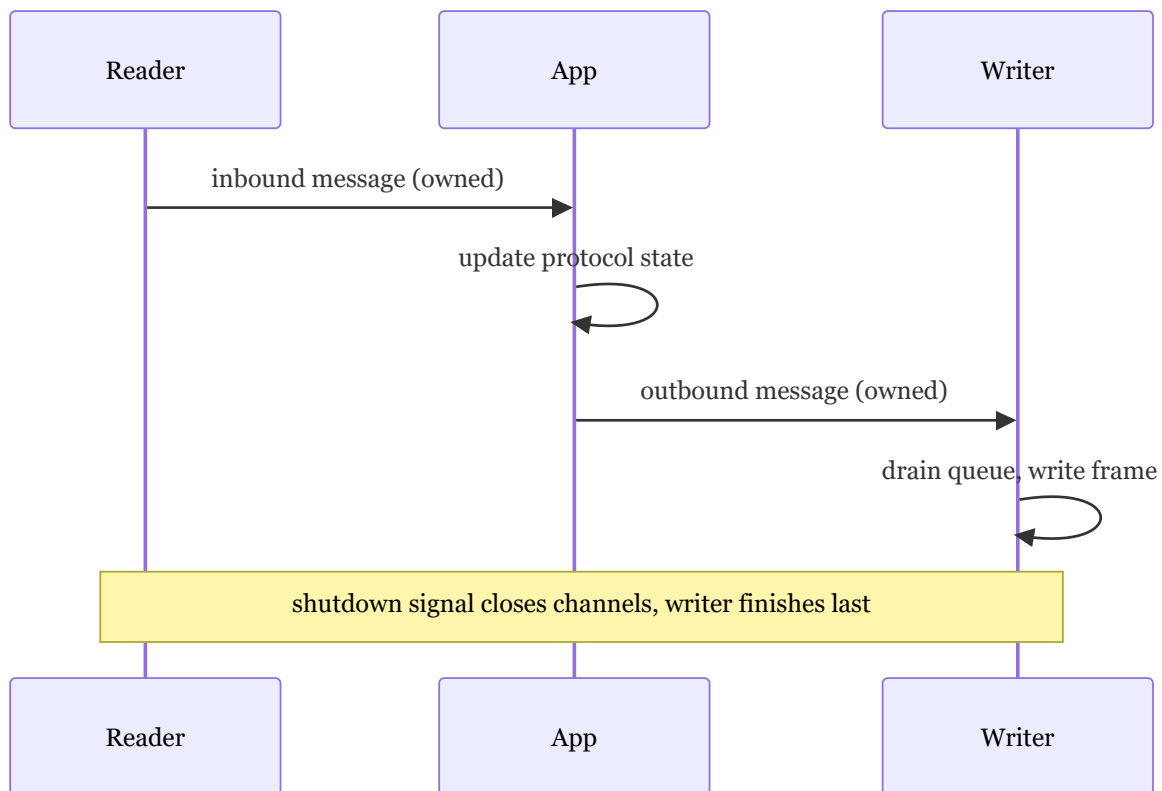
Pitfalls and tradeoffs

The most expensive WebSocket bug is often not a parser failure. It is one connection model that never stated who owns writes, how slow consumers are handled, or what happens when thousands of clients reconnect at once.

Examples

Example 1: separate reader, writer, and application logic with channels

This example models the whole connection with Tokio channels and a watch shutdown signal: one reader task, one app task, one writer task. A real WebSocket crate hands you the transport halves instead of channels, but the ownership shape is identical, and the shape is the lesson. As you read it, watch the direction of data: nothing ever reaches the writer except by way of the app task, and the reader never writes. Follow one inbound message through the diagram first, then find the same hops in the code.



Inbound flows reader to app to writer; outbound never skips the app. On shutdown the channels close in order so the writer drains before the connection ends.

Example 48-1. `channel_separated_connection_loop.rs` — *lo split*

```

use tokio::sync::{mpsc, watch};

#[derive(Debug)]
enum InboundFrame {
    Text(String),
    Ping(u64),
    Close,
}

#[derive(Debug)]
enum OutboundFrame {
    Text(String),
    Pong(u64),
    Close,
}

#[tokio::main]
async fn main() {
    let (inbound_tx, mut inbound_rx) = mpsc::channel::<InboundFrame>(4);
    let (outbound_tx, mut outbound_rx) = mpsc::channel::<OutboundFrame>(4);
    let (shutdown_tx, mut shutdown_rx) = watch::channel(false);

    let reader = tokio::spawn(async move {
        inbound_tx
            .send(InboundFrame::Text(String::from("ready")))
            .await
            .unwrap();
    });
  
```

```

inbound_tx.send(InboundFrame::Ping(7)).await.unwrap();
inbound_tx.send(InboundFrame::Close).await.unwrap();
});

let app = tokio::spawn(async move {
    let mut inbound = 0_u32;

    while let Some(frame) = inbound_rx.recv().await {
        inbound += 1;

        match frame {
            InboundFrame::Text(text) => {
                outbound_tx
                    .send(OutboundFrame::Text(format!("echo:{}", text)))
                    .await
                    .unwrap();
            }
            InboundFrame::Ping(nonce) => {
                outbound_tx.send(OutboundFrame::Pong(nonce)).await.unwrap();
            }
            InboundFrame::Close => {
                let _ = outbound_tx.send(OutboundFrame::Close).await;
                shutdown_tx.send(true).unwrap();
                break;
            }
        }
    }

    inbound
});

let writer = tokio::spawn(async move {
    let mut outbound = 0_u32;

    loop {
        tokio::select! {
            biased;
            Some(frame) = outbound_rx.recv() => {
                match frame {
                    OutboundFrame::Text(_) | OutboundFrame::Pong(_) | OutboundFrame::Close => {
                        outbound += 1;
                    }
                }
            }
            _ = shutdown_rx.changed() => {
                if *shutdown_rx.borrow() {
                    while let Ok(frame) = outbound_rx.try_recv() {
                        match frame {
                            OutboundFrame::Text(_) | OutboundFrame::Pong(_) | OutboundFrame::Close
                                => outbound += 1;
                        }
                    }
                }
                break outbound;
            }
        }
    }
});

```

```

    }
  }
});

reader.await.unwrap();
let inbound = app.await.unwrap();
let outbound = writer.await.unwrap();

println!("inbound = {}", inbound);
println!("outbound = {}", outbound);
println!("closed = {}", true);
}

```

OUTPUT

```

inbound = 3
outbound = 3
closed = true

```

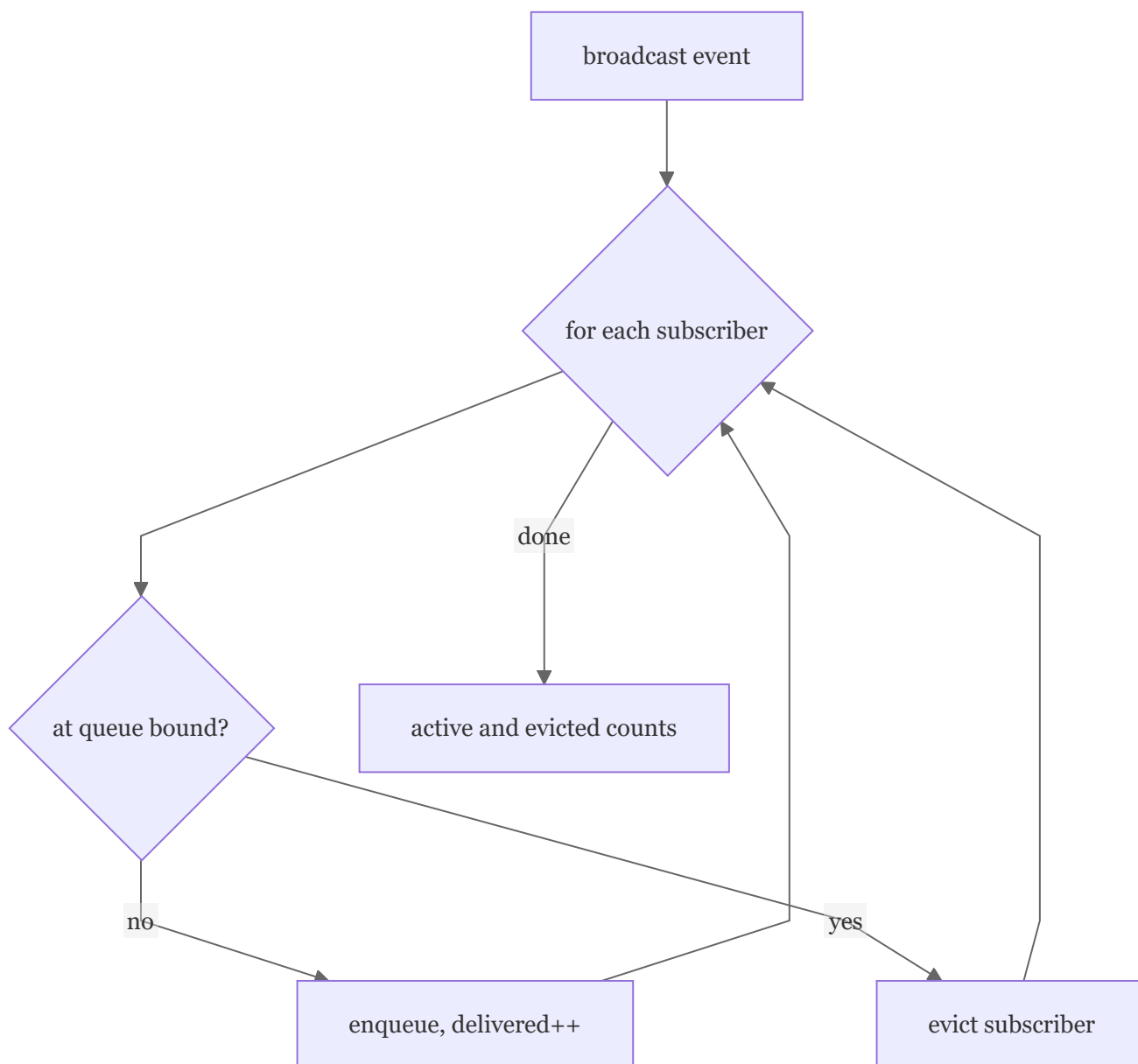
Only the reader task consumes inbound frames from the transport boundary.

Only the writer task drains outbound messages, which keeps close and pong ordering explicit.

The app task owns protocol reactions without needing broad shared socket access.

Example 2: bounded fan-out with slow-consumer eviction

This is the slow-consumer policy from the diagram above, made concrete and runnable. It keeps the queue logic crate-light with a `VecDeque` per client, but the decision is the same one a bounded channel makes in a real async service. The line to focus on is the capacity check before each enqueue: the broadcast walks every subscriber, and any client already at its bound is evicted rather than allowed to grow memory. The diagram shows that fork from the broadcast's point of view; in the run, client beta is the one over its limit, so it is the one that disappears.



One broadcast visits every subscriber. Those with room receive the event; those already at the bound are evicted, which is why the run reports active = 2 and evicted = beta.

Example 48-2. bounded_fanout_slow_consumers.rs — Slow consumers

```

use std::collections::{HashMap, VecDeque};

#[derive(Debug, Default)]
struct Client {
    pending: VecDeque<String>,
}

#[derive(Debug)]
struct Hub {
    clients: HashMap<'static str, Client>,
    max_pending: usize,
    delivered: usize,
    evicted: Vec<'static str>,

```

```

}

impl Hub {
    fn new(max_pending: usize) -> Self {
        Self {
            clients: HashMap::new(),
            max_pending,
            delivered: 0,
            evicted: Vec::new(),
        }
    }

    fn add_client(&mut self, id: &'static str, pending: usize) {
        let mut queue = VecDeque::new();
        for _ in 0..pending {
            queue.push_back(String::from("old"));
        }

        self.clients.insert(id, Client { pending: queue });
    }

    fn broadcast(&mut self, payload: &str) {
        let ids: Vec<_> = self.clients.keys().copied().collect();

        for id in ids {
            let at_limit = self
                .clients
                .get(id)
                .map(|client| client.pending.len() >= self.max_pending)
                .unwrap_or(false);

            if at_limit {
                self.clients.remove(id);
                self.evicted.push(id);
                continue;
            }

            if let Some(client) = self.clients.get_mut(id) {
                client.pending.push_back(payload.to_string());
                self.delivered += 1;
            }
        }
    }
}

fn main() {
    let mut hub = Hub::new(2);
    hub.add_client("alpha", 0);
    hub.add_client("beta", 2);
    hub.add_client("gamma", 1);

    hub.broadcast("update");

    println!("active = {}", hub.clients.len());
}

```

```
println!("evicted = {}", hub.evicted.join(","));
println!("delivered = {}", hub.delivered);
}
```

OUTPUT

```
active = 2
evicted = beta
delivered = 2
```

One max pending value makes the slow-consumer rule mechanically testable.

The server notices the client at the limit before the next push grows memory again.

In a real room or tenant hub, this policy combines with metrics, reconnect advice, and optional snapshot coalescing.

The repository also includes standalone Rust source under `examples/ch48_websockets_long_lived_connections/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor, including a small versioned envelope sketch alongside the two runnable examples.

Summary

CHAPTER 49 · HTTPS, TLS, AND SECURE SERVICE BOUNDARIES

HTTPS, TLS, and Secure Service Boundaries

Engineer secure service edges: TLS termination, rustls vs platform TLS, mTLS, certificate rotation, hardened cookies/CORS/HSTS, and safe debugging

A secure service boundary is not one library call. It is a set of decisions about where TLS terminates, which identities you accept, how certificates rotate, which proxy headers you believe, and what your HTTP defaults are. This chapter treats HTTPS and TLS as those decisions made explicit, so that the boundary is something a reviewer can read rather than something the runtime quietly assumes.

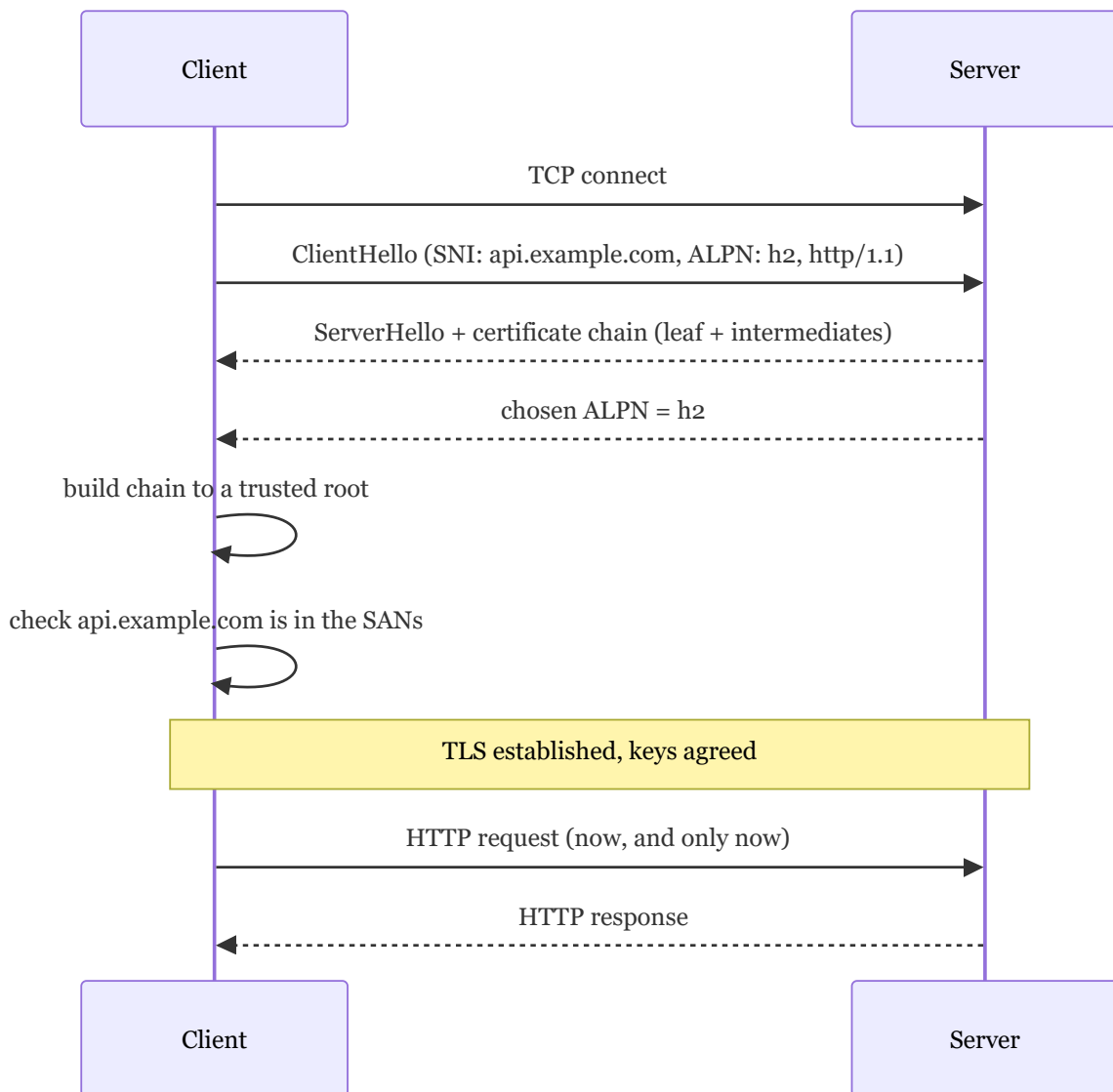
Builds on Chapters 19, 25, 30, 43, 46, 47, and 48

Chapter 19 established contract-first serialization, Chapter 25 covered runtime boundaries, Chapter 30 covered durable transport edges, Chapter 43 covered observability, Chapter 46 covered HTTP APIs, Chapter 47 covered gRPC transport policy, and Chapter 48 covered long-lived connections. HTTPS and TLS sit across all of those surfaces.

Opening scenario

A single Rust API serves three very different audiences at once: browsers loading a web UI, other services calling it machine-to-machine, and long-lived administrative sessions. Some of that traffic crosses the public internet and some stays inside a private network. The team's job is to define one coherent boundary that answers every security question for all three audiences: where TLS terminates, who holds the certificates, how peer identity is established and authorized, which proxy headers the application is allowed to believe, what the secure HTTP defaults are, how local and CI tests obtain trust, and how certificates rotate without an outage. None of those answers come from a framework. They are decisions, and this chapter is about making them on purpose and writing them down where a reviewer can see them.

Before any of that policy makes sense, it helps to see exactly what happens on the wire. Notice that every security check below, certificate validation, hostname matching, and protocol selection, finishes before the first HTTP byte is sent. The handshake is where trust is established; HTTP is just what rides on top afterward.



HTTPS is HTTP that only begins after the handshake. The client validates the chain, matches the hostname against the SANs, and confirms the negotiated protocol before sending a single HTTP byte.

Mental model

Three ideas carry most of this chapter. The first is about what HTTPS actually is at the byte level. The second is about where the encryption boundary sits in your system. The third is about treating the boundary as policy you encode rather than behavior you hope for. Hold these three in mind and the rest of the chapter is mostly detail filling them in.

Core concepts

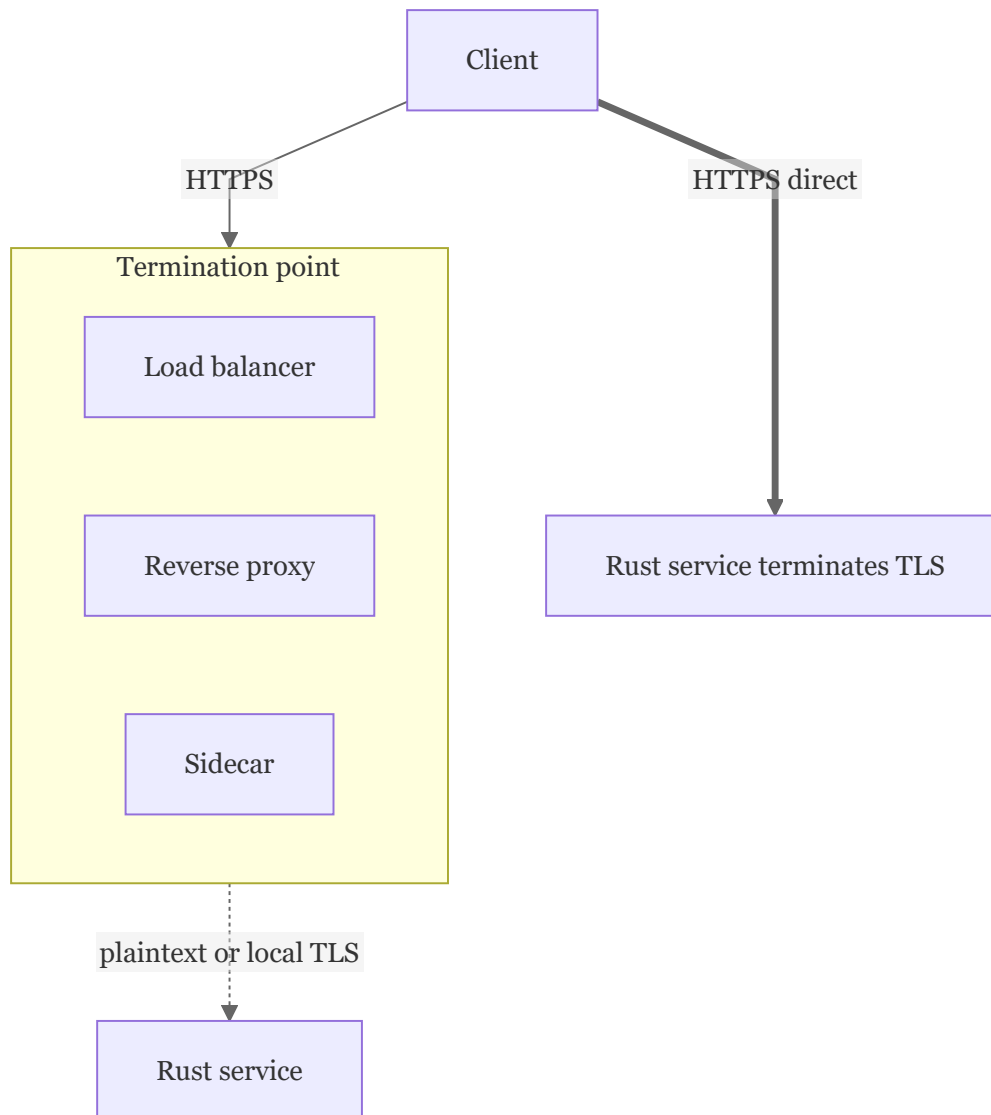
What HTTPS actually checks: chains, hostnames, ALPN, and whose trust store

When a TLS connection fails, it almost always fails for one of four reasons, and they map cleanly onto four questions. Could the client build a chain to a root it trusts? Did the certificate cover the hostname the client

asked for? Did the two sides agree on a protocol? And, crucially, whose trust store are we even talking about? Most production TLS debugging is just figuring out which of these four boxes is the one that failed.

Where TLS ends: load balancer, reverse proxy, sidecar, or your own process

The single most consequential question in this chapter is where the public TLS connection actually ends. Follow the dotted plaintext segment in the diagram below: wherever it appears, that is a hop your application does not see as TLS, and therefore a hop where the original scheme and client identity survive only as headers you choose to trust. The further the termination point is from your code, the more you rely on the network path and on honest forwarding; the closer it is, the more certificate and trust logic your own process owns.



Solid lines are encrypted; the dotted line is the plaintext (or short local-TLS) hop the edge introduces. Terminating in-process removes that hop, at the cost of owning certificate and reload logic yourself.

The most common mistake here is to treat proxy headers as trustworthy simply because TLS terminated somewhere upstream. A header such as X-Forwarded-Proto is just bytes; any client can send it. Believe it only when it arrives through the specific proxy hops you operate, and strip or overwrite it at the edge so no external caller can forge it.

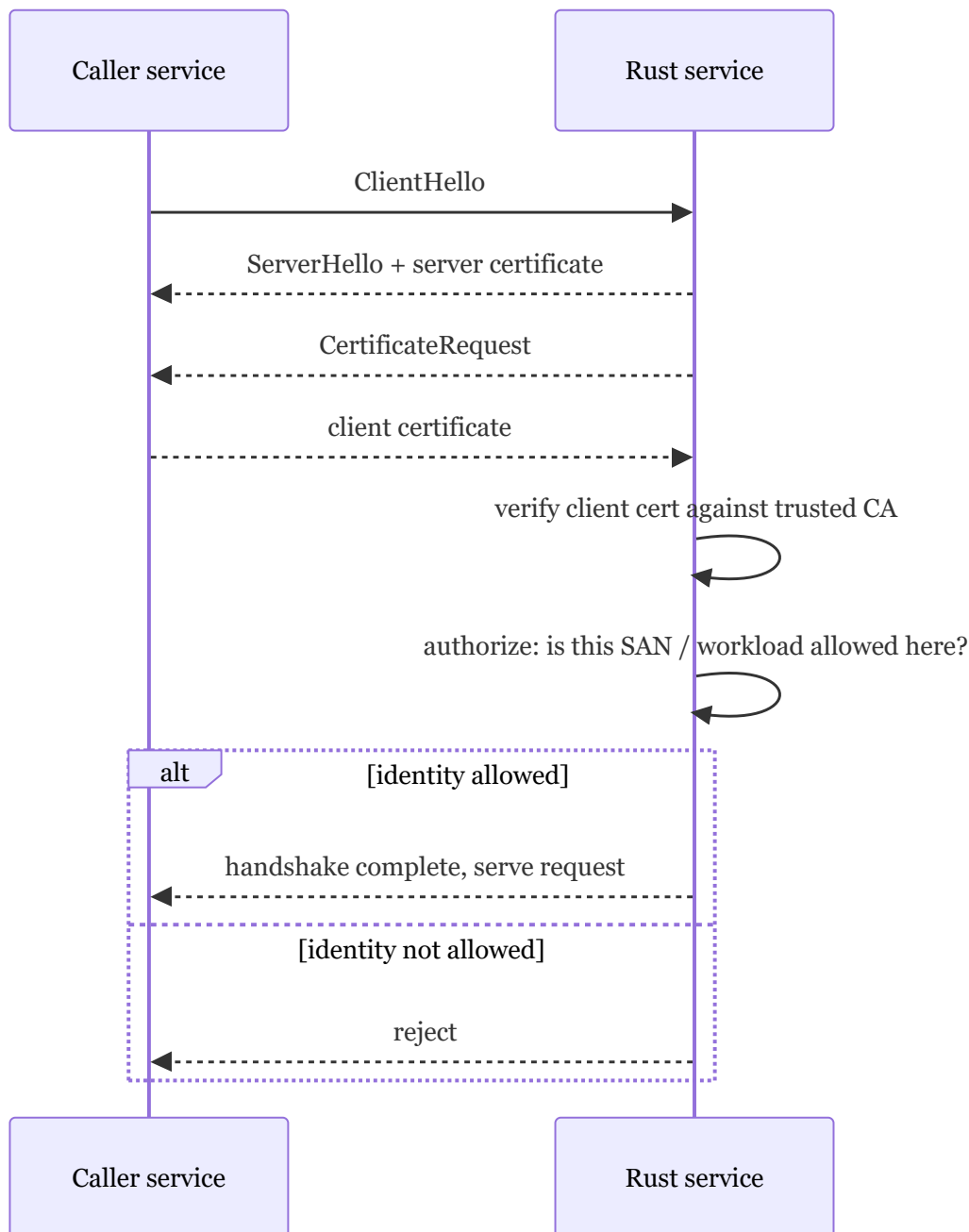
Picking a TLS library: Rust-native or the platform's stack

If your service terminates TLS itself, you choose a TLS implementation, and the two honest options pull in opposite directions. A Rust-native stack carries its own trust and behaves identically everywhere. The platform stack inherits the operating system's trust and certificate lifecycle. The right answer depends less on which API is shorter and more on whether you want to own your trust material or defer to the environment that already manages it.

A good senior-level decision is usually “which runtime trust-store and packaging contract do we want?” long before it is “which exact client or server API looks shortest in code?”

mTLS: authenticating the caller, not just the server

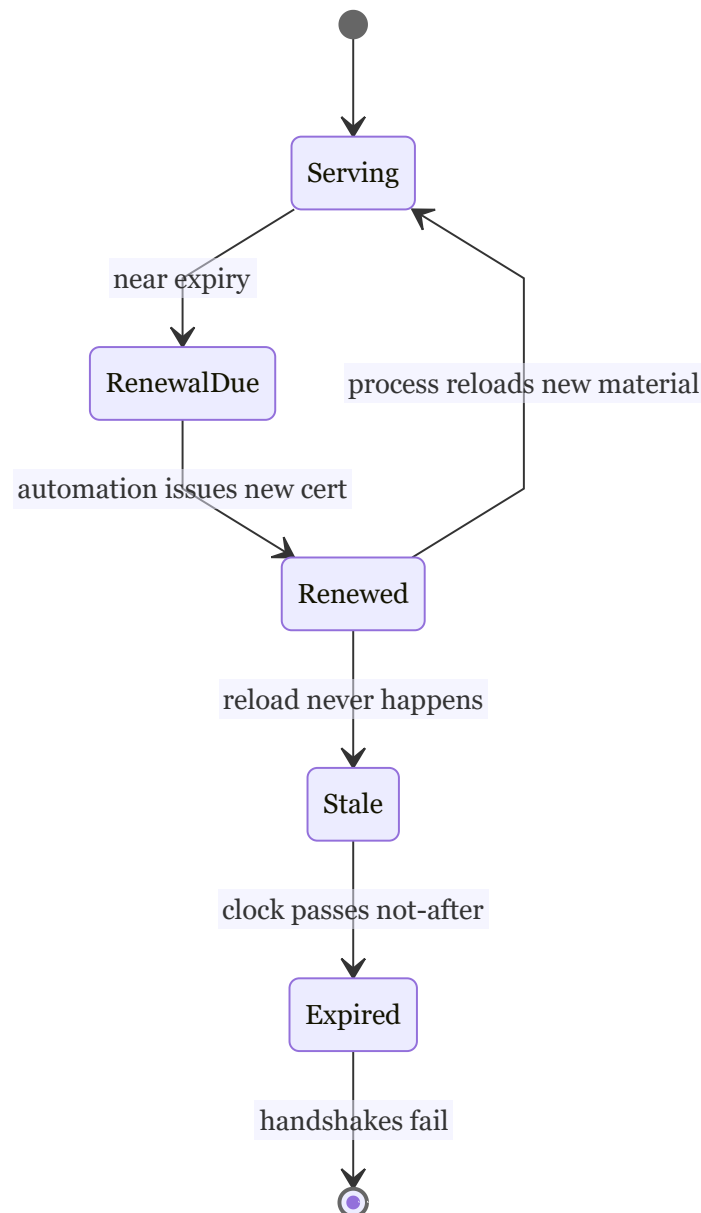
For service-to-service traffic, you often want both ends to prove who they are. The diagram below shows the key addition over ordinary HTTPS: the server requests a certificate from the client, the client presents one, and then the server runs a policy check on the verified identity. Watch where that policy check lands. The handshake only proves the client holds a certificate your CA signed; deciding whether this identity may reach this route is a separate, deliberate step.



mTLS adds a client certificate to the handshake. Verifying it proves the caller is signed by a trusted CA; the separate authorization step decides whether that specific identity is permitted on this route.

Certificate rotation as a lifecycle, not a one-off task

A certificate is a secret with an expiry date, which makes rotation an ongoing state machine rather than a thing you do once. The transition that catches teams out is the one drawn in bold below: renewing the material on disk and actually reloading it into the running process are two different events, and if the second one never fires, you have a renewed certificate and a live outage at the same time. Read the diagram as the happy path you must automate, with the dashed edge as the failure you must alert on.



The dangerous edge is Renewed to Stale: the certificate on disk is fresh but the process is still serving the old one. Automate the reload, and alert when a renewed certificate has not been picked up.

The HTTP defaults that make a TLS boundary actually secure

A correct TLS handshake protects bytes in transit, but it does not by itself stop a session cookie from leaking, a cross-origin page from reading your responses, or a browser from quietly falling back to plaintext. Those are HTTP-level concerns, and each one is a small policy you set deliberately. The five below are the defaults worth treating as non-negotiable on a public boundary, with exceptions made consciously rather than by omission.

Debugging TLS failures without weakening security

TLS failures feel opaque because the error often arrives as a generic 'handshake failed,' but the underlying cause is almost always one of a short list. Work through this checklist in order, from the most common cause (a name mismatch) to the more subtle ones (ALPN, proxy trust, mTLS policy). The discipline that matters

most is the last item: never reach for 'disable verification' as a fix, because that converts a diagnosable configuration problem into a silent, permanent vulnerability.

A debugging checklist that ends with “turn verification off” is not a production checklist. Use a local CA or explicit test trust bundle instead.

Testing HTTPS locally and in CI

The whole point of this section is to test the secure path, not a weakened stand-in for it. The reliable approach is to run a small certificate authority you control, in development and in CI, so your tests exercise the same chain assembly and hostname verification a real client will. That keeps verification on everywhere and lets your tests assert the policy itself: redirects, cookie flags, HSTS, origin rules, and mTLS accept/reject behavior.

How this lands depending on where you come from

The TLS facts in this chapter are the same in every language. What differs is the habit you arrive with and the assumption you most need to drop. These cards are about the mental-model shift, not about which crate maps to which library.

Production patterns

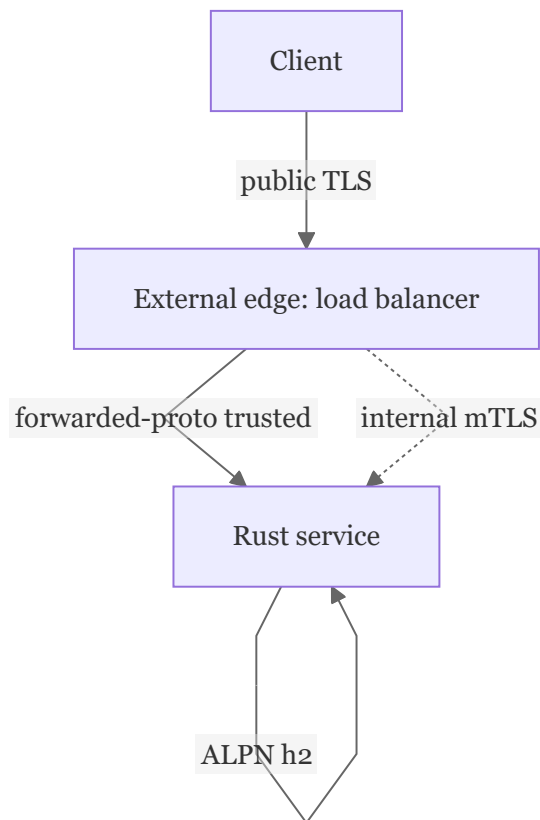
Pitfalls and tradeoffs

Examples

Example 1: choose a TLS termination and internal peer-auth strategy deliberately

The point is not framework syntax. The point is that external termination, forwarded-proto trust, ALPN, and internal mTLS policy are one explicit configuration surface.

Before reading the code, picture the boundary it describes. The request crosses an external edge that terminates public TLS, then reaches the service over an internal hop that uses mTLS, and the service only believes the forwarded scheme because it came through that known edge. The struct in the listing is just this picture turned into typed fields, so a reviewer can confirm each decision in a diff.



The example encodes exactly this: external termination at the edge, mTLS on the internal hop, forwarded scheme trusted only from that edge, and ALPN pinned to h2.

Example 49-1. `tls_termination_topology.rs` — *Topology policy*

```

#[derive(Debug, Clone, Copy)]
enum TlsTermination {
    LoadBalancer,
    ReverseProxy,
    InProcess,
}

#[derive(Debug, Clone, Copy)]
enum PeerAuth {
    Off,
    Mtls,
}

#[derive(Debug)]
struct ServiceEdge {
    external_tls: TlsTermination,
    internal_peer_auth: PeerAuth,
    trusted_proxy_hops: u8,
    app_protocol: &'static str,
}

impl ServiceEdge {
    fn external_mode(&self) -> &'static str {
        match self.external_tls {
            TlsTermination::LoadBalancer => "load-balancer",

```

```

        TlsTermination::ReverseProxy => "reverse-proxy",
        TlsTermination::InProcess => "in-process",
    }
}

fn internal_mode(&self) -> &'static str {
    match self.internal_peer_auth {
        PeerAuth::Off => "tls-only",
        PeerAuth::Mtls => "mtls",
    }
}

fn forwarded_proto_policy(&self) -> &'static str {
    if self.trusted_proxy_hops > 0 {
        "trusted-proxy-only"
    } else {
        "ignore-forwarded-proto"
    }
}

fn alpn(&self) -> &'static str {
    self.app_protocol
}

fn main() {
    let edge = ServiceEdge {
        external_tls: TlsTermination::LoadBalancer,
        internal_peer_auth: PeerAuth::Mtls,
        trusted_proxy_hops: 1,
        app_protocol: "h2",
    };

    println!("external = {}", edge.external_mode());
    println!("internal = {}", edge.internal_mode());
    println!("forwarded proto = {}", edge.forwarded_proto_policy());
    println!("alpn = {}", edge.alpn());
}

```

OUTPUT

```

external = load-balancer
internal = mtls
forwarded proto = trusted-proxy-only
alpn = h2

```

One enum value makes the termination owner visible instead of implicit.

mTLS is modeled as peer-auth policy, not as a vague “secure internal” claim.

Forwarded scheme data is trusted only when a known proxy hop exists.

HTTP protocol selection belongs in the same boundary model as certificates and hostnames.

Example 2: encode secure cookie, CORS, HSTS, and rotation defaults explicitly

This is a configuration-oriented example for the application edge. The values are boring on purpose, because boring secure defaults survive production review.

Read each field as a policy statement. The cookie name prefix, the CORS mode, the HSTS flag, and the rotation window each represent one decision that a reviewer can evaluate in isolation and a test can assert against directly.

Example 49-2. `security_defaults_and_rotation.rs` — *Security defaults*

```
#[derive(Debug)]
struct SessionCookiePolicy {
    name: &'static str,
    secure: bool,
    http_only: bool,
    same_site: &'static str,
}

impl SessionCookiePolicy {
    fn is_hardened(&self) -> bool {
        self.secure && self.http_only && self.same_site != "None"
    }
}

#[derive(Debug)]
struct HttpSecurityPolicy {
    redirect_http: bool,
    hsts_max_age_secs: u64,
    allowed_origin: &'static str,
    renew_before_days: u16,
}

impl HttpSecurityPolicy {
    fn cors_mode(&self) -> &'static str {
        if self.allowed_origin == "*" {
            "wildcard"
        } else {
            "locked-down"
        }
    }

    fn hsts_enabled(&self) -> bool {
        self.hsts_max_age_secs > 0
    }

    fn rotate_now(&self, days_left: u16) -> bool {
        days_left <= self.renew_before_days
    }
}

fn main() {
    let cookie = SessionCookiePolicy {
        name: "__Host-session",
        secure: true,
        http_only: true,
```

```
        same_site: "Lax",
    };

    let policy = HttpSecurityPolicy {
        redirect_http: true,
        hsts_max_age_secs: 63_072_000,
        allowed_origin: "https://app.example.com",
        renew_before_days: 14,
    };

    println!("cookie = {}", cookie.name);
    println!("cookie hardened = {}", cookie.is_hardened());
    println!("cors = {}", policy.cors_mode());
    println!("hsts = {}", policy.hsts_enabled());
    println!("rotate now = {}", policy.rotate_now(21));
}
```

OUTPUT

```
cookie = __Host-session
cookie hardened = true
cors = locked-down
hsts = true
rotate now = false
```

Secure, HttpOnly, and SameSite stay visible instead of buried in one framework default.

Exact origin policy makes credentialed browser flows reviewable.

HTTPS redirects and HSTS should reinforce each other, not contradict each other.

Renewal windows are deployment policy, not a comment on a certificate ticket.

A typed config surface is much easier to diff and test in CI than scattered string settings.

The repository also includes standalone Rust source under `examples/ch49_https_tls_secure_service_boundaries/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

CHAPTER 50 · LIBP2P AND PEER-TO-PEER RUST SYSTEMS

libp2p and Peer-to-Peer Rust Systems

Compose peer identity, discovery, request-response/pub-sub, relay policy, sync, and observability into a reviewable Rust P2P architecture

Peer-to-peer systems need one controlled swarm loop, explicit protocols, connectivity policy, identity management, and convergence rules. This chapter covers libp2p from those production requirements.

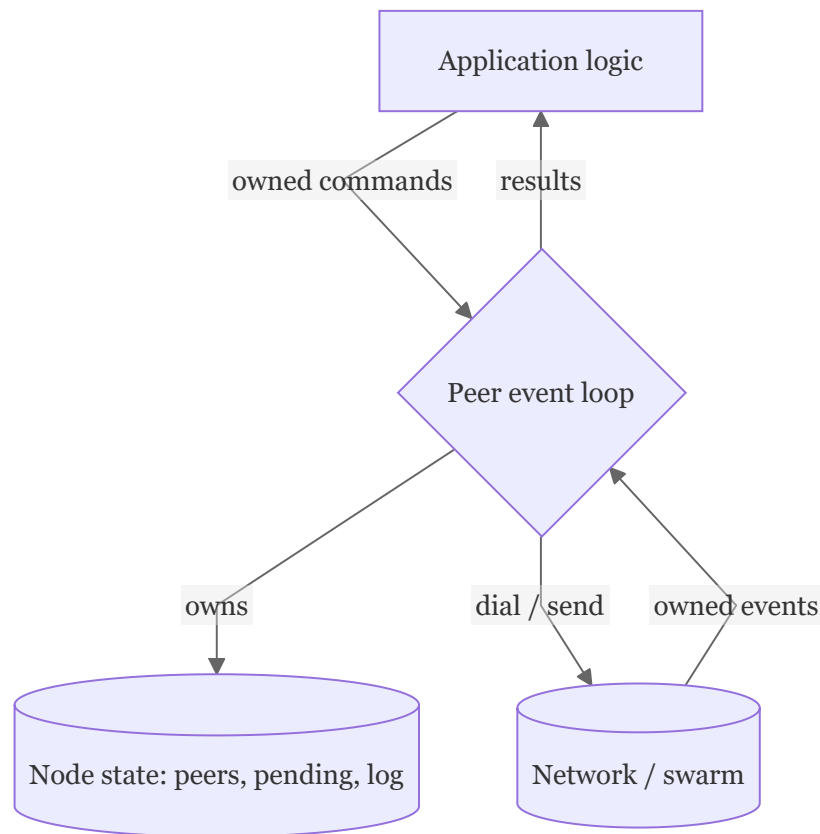
Builds on Chapters 25, 30, 31, 38, 43, 47, 48, and 49

Chapter 25 covered Tokio task boundaries. Chapter 30 covered brokered delivery and replay. Chapter 31 covered distributed task execution. Chapter 38 covered Merkle-based verification. Chapter 43 covered observability. Chapter 47 covered gRPC transport contracts. Chapter 48 covered long-lived connections. Chapter 49 covered secure boundaries and TLS. Peer-to-peer systems sit across all of them.

Opening scenario

A field-deployed coordination system must keep exchanging state when nodes lose their link to a central server or sit behind restrictive home and carrier networks. There is no broker to lean on and no single endpoint everyone dials. Each node has to find peers on its own, prove who it is, open an encrypted channel, and then run several conversations at once: discovery, request-response fetches, and pub-sub gossip. The business requirement is to compose identity, transport, secure channels, discovery, relay policy, request-response, pub-sub, and state convergence inside one bounded peer event loop that an on-call engineer can reason about at three in the morning.

That last phrase is the design constraint that matters. The temptation is to spread network state across many objects and let callbacks mutate it from everywhere. The calmer shape is a single loop: it waits on two sources — commands coming down from your application, and events coming up from the network — and it owns the state both sides touch. Everything below is built around that one picture. Read the diagram first, then the code sketch, which is the same loop written in Rust-shaped pseudocode.



One loop waits on commands from above and events from below, and it alone owns the node state both sides read and write.

The select! below is the loop in the diagram. The two arms are the two arrows into the loop: one drains application commands, the other drains network events. Both hand their value to a function that takes &mut node.state, so there is exactly one owner of the state and no shared-mutable web to keep coherent.

The exact libp2p Rust APIs evolve over time, and crate method names will not match this sketch line for line. The architecture is far more stable than the names: one swarm owns the network edge, behaviours own protocol state, and your application translates owned commands and owned events through that loop.

Mental model

Before any code, three corrections clear up most early confusion. The word “peer-to-peer” suggests a single clever socket that magically connects everyone; it is really a stack of separate protocols you assemble on purpose. The word “libp2p” suggests one crate you import and call; it is really a way to compose small protocol state machines. And the demo — two laptops on one Wi-Fi network — hides the work that actually fills your sprint board, which is connectivity, abuse control, and getting divergent state back into agreement.

How this lands by background

P2P work pulls hard on instincts you already have, and some of those instincts help while others quietly mislead. The useful question is not which crate maps to which library; it is where each language taught you to put trust, and how that placement has to move in Rust. Read the card for your background before the rest of the chapter, then notice when the text is correcting exactly the habit it names.

Core concepts

The layers a peer connection is built from

A single connection to a peer is not one thing; it is a short stack of concerns, each solving a different problem, layered in a fixed order. You establish who the peer is, you pick a path for the bytes, you upgrade that path to an encrypted channel, and only then do you split it into many logical streams so that discovery, request-response, and gossip can all run over the one connection at once. Discovery sits beside the stack because it answers a prior question — which peers exist and how to reach them — that you must answer before there is a connection to build at all. The cards below expand each layer in that order.

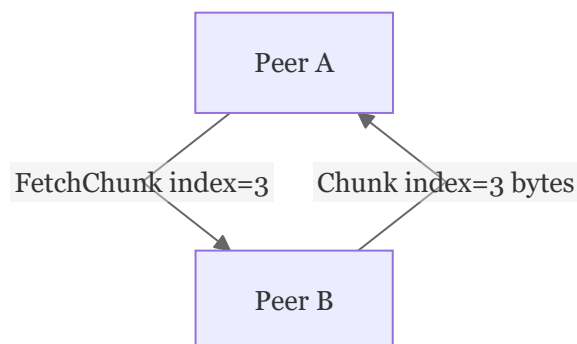
How a libp2p node is wired in Rust: swarm, behaviours, and events

The Rust libp2p design has one organising idea worth getting right early. The swarm is the outer machine that owns connections and does the polling; it does not understand your protocols. Each protocol — identify, discovery, request-response, pub-sub — lives in its own behaviour, which is a small state machine. The swarm drives every behaviour and bubbles up what they produce as a stream of events. Your job is to turn each event into an owned value and act on it in the application layer, and to push application decisions back down as commands. The cards expand each box in that path.

One useful correction for engineers coming from framework-heavy networking is this: the swarm is not your business object, and a behaviour is not your whole node. Keep network orchestration and domain state separate so tests can drive protocol state machines without real sockets first.

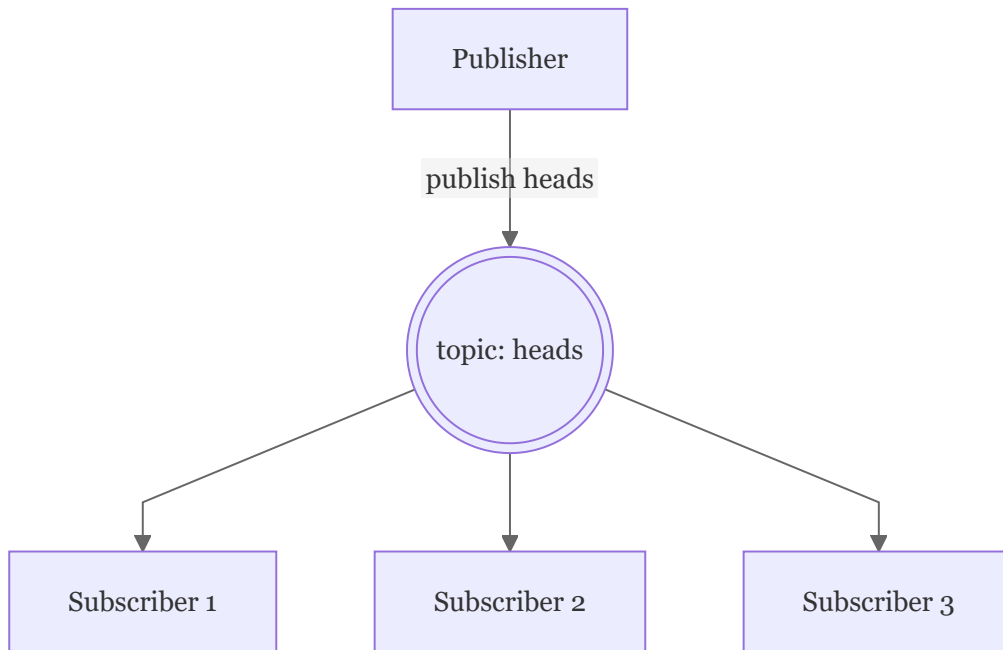
Two protocol shapes: a question to one peer, or news for many

Almost every peer interaction is one of two shapes, and they have different contracts. Request-response is a question put to one named peer that expects exactly one bounded answer: fetch a state summary, ask for a chunk, pull one slice of work. Its contract is correlation (matching the reply to the request), a timeout, and a size cap. Publish-subscribe is news emitted to a topic that any number of peers may have subscribed to, where eventual propagation matters more than a precise per-peer reply. Its contract is topic naming, who may publish, fan-out policy, validation cost, and tolerating duplicates. Picture the difference before reading the message types.



Request-response: Peer A asks one named peer for one bounded answer, and Peer B returns exactly that reply correlated to the request.

Side by side, the other shape sends one message to many peers at once:

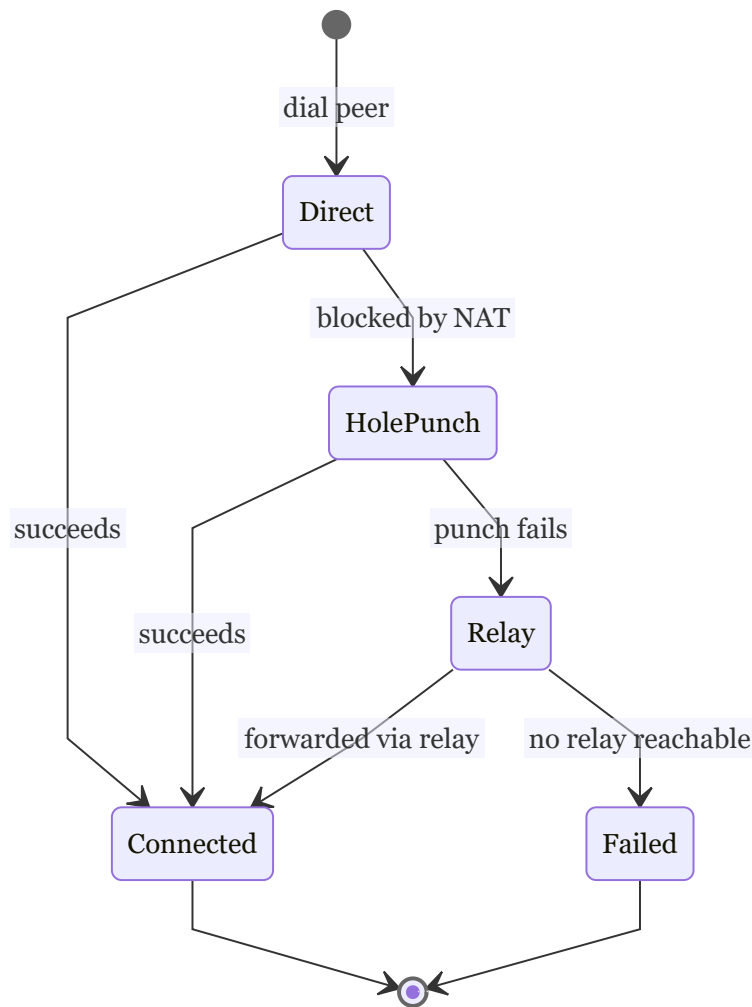


Pub-sub: a publisher emits to a topic and every peer subscribed to that topic receives the message, fanning out to all subscribers.

The request-response message types below are deliberately small. Notice that the request carries only what the peer on the wire needs to act — a known version, a root hash, an index — and that `Reject` is a first-class response, not an exception. A transport message should answer only what the remote peer needs; your domain model can stay richer internally, and you map between the two explicitly.

Reaching a peer: discovery, NAT, and the relay fallback ladder

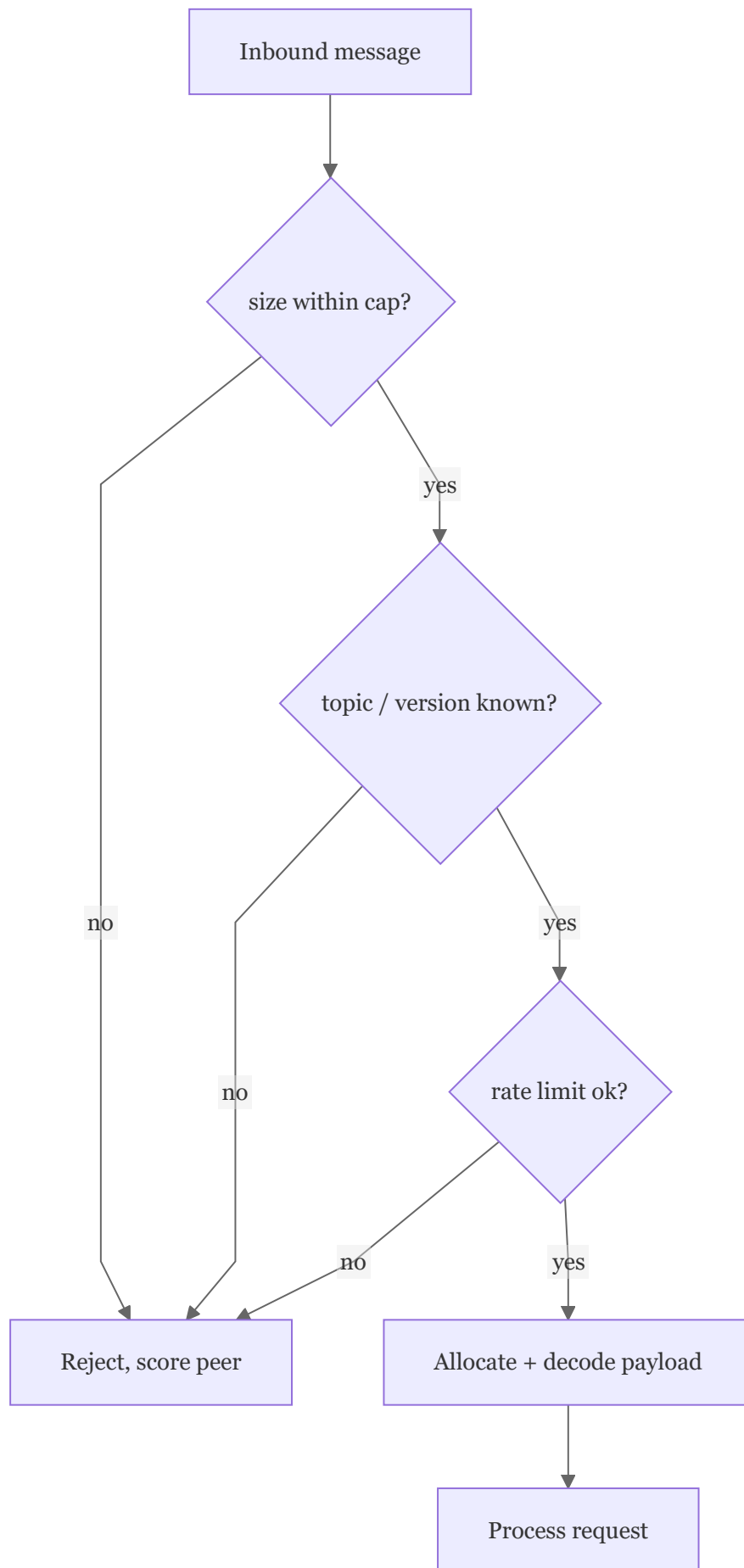
On a laptop demo, dialing a peer is one step that always succeeds. On real networks it is a ladder of attempts, because most users sit behind NAT that refuses unsolicited inbound connections. A node first tries to dial the peer directly. If that fails, it can attempt hole punching, where both peers coordinate through a third party to open matching ports at the same instant. If hole punching also fails, traffic falls back to a relay that forwards bytes for both sides — which works, but adds latency, costs bandwidth, and becomes an abuse target. The diagram is the decision you are really implementing; treat each transition as something you measure, not something that just happens.



Direct dial, then hole punch, then relay. Each rung is a metric: dial success, punch success, and how much traffic falls through to relays.

Security: knowing the peer is not the same as trusting the request

In a P2P system the attacker is on the inside: any peer can connect, and a valid cryptographic identity only tells you who is sending bytes, not that the bytes are safe to act on. Identity, signed records, and channel encryption each solve one narrow problem and none of them validate a request. The defensive posture that holds up is to do work in increasing order of cost, rejecting as early as possible. Check the cheap things first — size, topic, version, rate — and only allocate buffers or decode heavy payloads once those pass. The pipeline below is the shape of an inbound handler that an adversary cannot easily turn into free compute or memory pressure.



Cheap checks gate expensive work: size, topic, and rate are verified before any buffer is allocated or any payload decoded.

“Authenticated channel” and “safe application request” are separate claims. Keep size checks, rate limits, and protocol authorization even when the remote peer is cryptographically identified.

State synchronization and conflict handling

Observability, testing, and simulation for P2P networks

When to choose P2P vs client-server, message brokers, or distributed task queues

The right answer is often hybrid. A system can use HTTP or gRPC for control plane, a broker for durable work and replay, and libp2p only where peer discovery, direct data exchange, or partial decentralization are real product requirements. The point is to choose from workload shape, not from ideology.

Production patterns

Pitfalls and tradeoffs

The most expensive P2P bug is often not a bad packet decode. It is an under-specified operational model: no NAT fallback, no bounded queues, no replay-safe sync semantics, or no way to explain a failure once the network becomes partially partitioned.

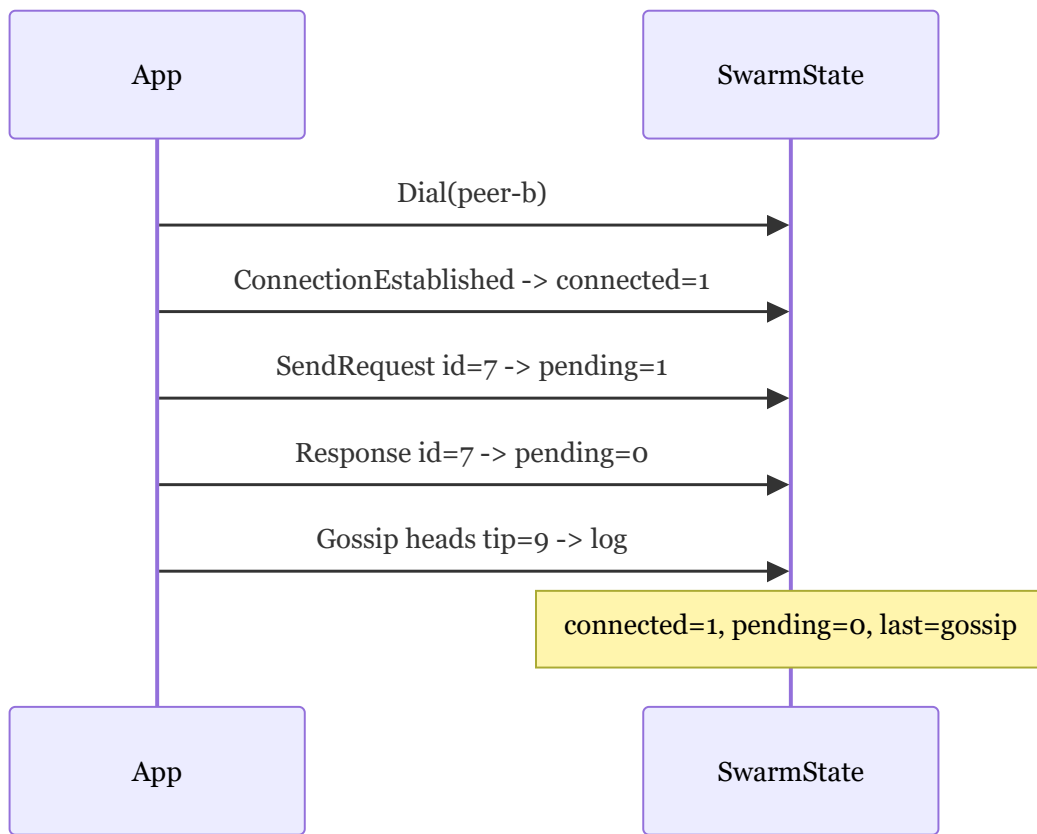
Examples

The interactive examples stay self-contained and compile as ordinary Rust without a live network. That keeps the ownership model stable even if libp2p crate APIs evolve. Treat them as libp2p-style protocol loops and state machines rather than as copy-paste setup code.

Example 1: the swarm loop as one owner with two entry points

One owner, `SwarmState`, keeps all protocol state. Commands and inbound network events are owned enums that can cross async or channel boundaries.

What to look at: the state machine has exactly two doors, `on_command` and `on_event`, and both take `&mut self` — the same single-owner shape as the loop at the top of the chapter. Follow one request through: `SendRequest` records the request id in `pending_requests`, and the matching `Response` event is the only thing that removes it. That insert-then-remove pair is why the final `pending = 0`. The sequence below is the exact run in `main`.



Commands and events arrive in order; pending requests rise on SendRequest and fall on the matching Response, ending at zero.

Example 50-1. swarm_event_loop.rs — State machine

```

use std::collections::{BTreeSet, HashMap, VecDeque};

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
struct PeerId(&'static str);

#[derive(Debug, Clone)]
enum LocalCommand {
    Dial(PeerId),
    SendRequest {
        peer: PeerId,
        request_id: u64,
        body: String,
    },
    Publish {
        topic: &'static str,
        body: String,
    },
}

#[derive(Debug, Clone)]
enum NetworkEvent {
    ConnectionEstablished(PeerId),
    Response {
        peer: PeerId,
        request_id: u64,
    },
}

```

```

        body: String,
    },
    Gossip {
        peer: PeerId,
        topic: &'static str,
        body: String,
    },
}

#[derive(Debug, Default)]
struct SwarmState {
    connected: BTreeSet<PeerId>,
    pending_requests: HashMap<u64, PeerId>,
    log: VecDeque<String>,
}

impl SwarmState {
    fn on_command(&mut self, command: LocalCommand) {
        match command {
            LocalCommand::Dial(peer) => {
                self.log.push_back(format!("dial {}", peer.0));
            }
            LocalCommand::SendRequest {
                peer,
                request_id,
                body,
            } => {
                self.pending_requests.insert(request_id, peer);
                self.log.push_back(format!("request {} {}", peer.0, body));
            }
            LocalCommand::Publish { topic, body } => {
                self.log.push_back(format!("publish {} {}", topic, body));
            }
        }
    }

    fn on_event(&mut self, event: NetworkEvent) {
        match event {
            NetworkEvent::ConnectionEstablished(peer) => {
                self.connected.insert(peer);
                self.log.push_back(format!("connected {}", peer.0));
            }
            NetworkEvent::Response {
                peer,
                request_id,
                body,
            } => {
                self.pending_requests.remove(&request_id);
                self.log.push_back(format!("response {} {}", peer.0, body));
            }
            NetworkEvent::Gossip { peer, topic, body } => {
                self.log
                    .push_back(format!("gossip {} {} {}", topic, peer.0, body));
            }
        }
    }
}

```

```

}

fn main() {
    let mut swarm = SwarmState::default();
    let peer = PeerId("peer-b");

    swarm.on_command(LocalCommand::Dial(peer));
    swarm.on_event(NetworkEvent::ConnectionEstablished(peer));
    swarm.on_command(LocalCommand::SendRequest {
        peer,
        request_id: 7,
        body: String::from("want-state"),
    });
    swarm.on_event(NetworkEvent::Response {
        peer,
        request_id: 7,
        body: String::from("state-v3"),
    });
    swarm.on_event(NetworkEvent::Gossip {
        peer,
        topic: "heads",
        body: String::from("tip=9"),
    });

    println!("connected = {}", swarm.connected.len());
    println!("pending = {}", swarm.pending_requests.len());
    println!(
        "last = {}",
        swarm.log.back().map(String::as_str).unwrap_or("none")
    );
}

```

OUTPUT

```

connected = 1
pending = 0
last = gossip heads peer-b tip=9

```

The swarm-state owner holds the maps and queues. Commands and events move in as owned values.

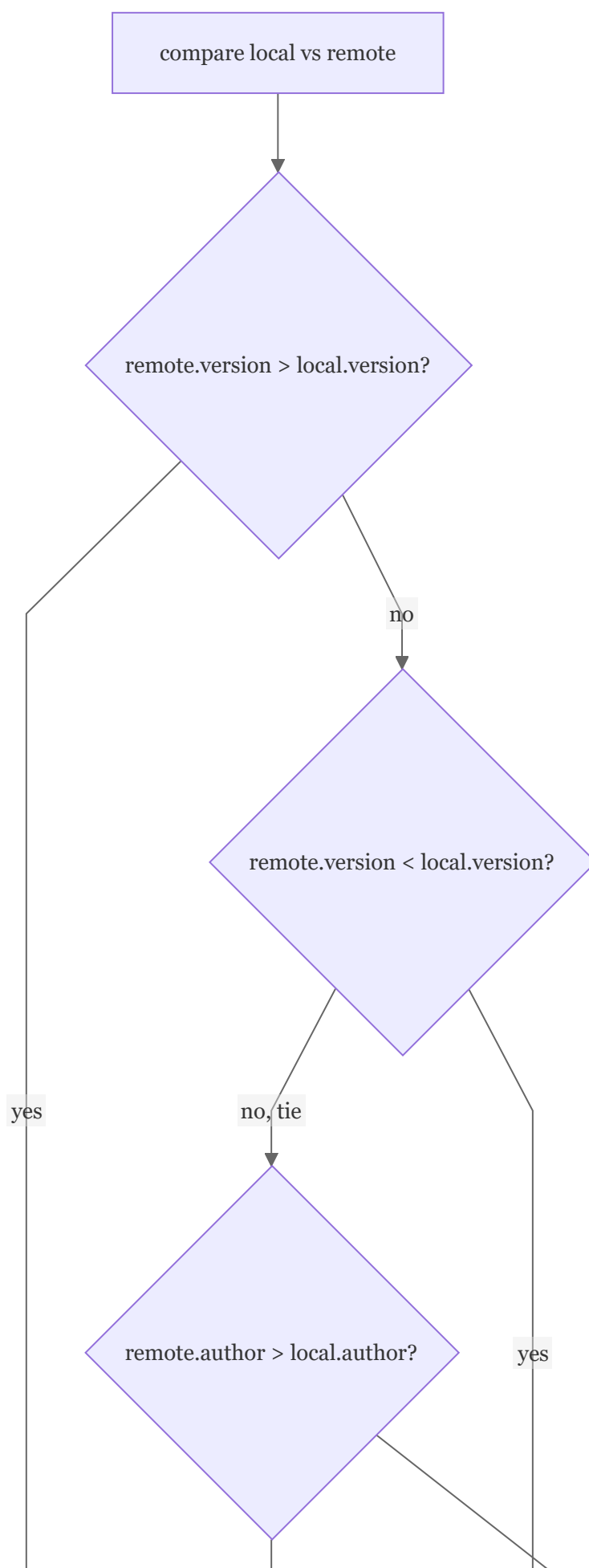
Pending requests live in one map and are removed only when the matching response arrives.

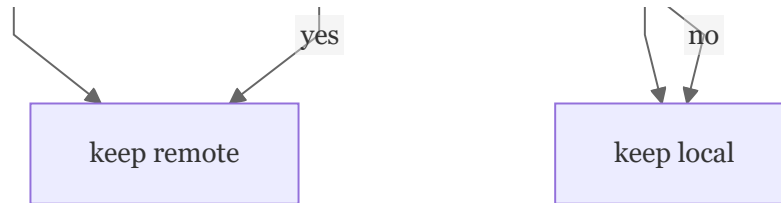
The loop records network events in one application log without requiring a live transport to teach the idea.

Example 2: merging conflicting updates with one deterministic rule

This example is intentionally modest: pick the highest version, then break ties deterministically by author. In real systems you may need stronger CRDT, log, or Merkle-based semantics.

What to look at: `merge_peer_updates` folds the updates through `pick_newer`, so the whole merge is just that one comparison applied repeatedly. The comparison is the contract every peer must share: higher version wins; on an equal version, the larger author string wins. Trace the inputs — current is version 3, then two version-4 updates from peer-b and peer-c. Both beat version 3, and the tie between them goes to peer-c. The flow below is the body of `pick_newer`.





Version decides first; only an exact tie falls through to the author comparison, which makes the merge deterministic across peers.

Example 50-2. `state_sync_conflict_resolution.rs` — Sync conflicts

```

#[derive(Debug, Clone, PartialEq, Eq)]
struct VersionedValue {
    version: u64,
    author: &'static str,
    value: String,
}

fn pick_newer(local: VersionedValue, remote: VersionedValue) -> VersionedValue {
    if remote.version > local.version {
        remote
    } else if remote.version < local.version {
        local
    } else if remote.author > local.author {
        remote
    } else {
        local
    }
}

fn merge_peer_updates(
    current: VersionedValue,
    updates: Vec<VersionedValue>,
) -> VersionedValue {
    updates.into_iter().fold(current, pick_newer)
}

fn main() {
    let current = VersionedValue {
        version: 3,
        author: "peer-a",
        value: String::from("allow-read"),
    };

    let merged = merge_peer_updates(
        current,
        vec![
            VersionedValue {
                version: 4,
                author: "peer-b",
                value: String::from("allow-write"),
            },
            VersionedValue {
                version: 4,
                author: "peer-c",
            }
        ]
    );
}

```

```
        value: String::from("allow-write+audit"),
    },
],
);

println!("version = {}", merged.version);
println!("author = {}", merged.author);
println!("value = {}", merged.value);
}
```

OUTPUT

```
version = 4
author = peer-c
value = allow-write+audit
```

A tie-break rule is part of the sync contract. Two peers need the same merge rule to converge.

Scalar versions and peer-ID ordering are simple and often insufficient once true concurrent edits matter.

This is the right place to ask whether your real workload wants logs, CRDTs, or Merkle diffing instead.

The `repository` also includes standalone Rust source under `examples/ch50_libp2p_peer_to_peer_rust_systems/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

PART VIII

Frontier Specializations

Dip in on demand: zero-knowledge proofs, verifiable and portable ML inference, on-chain smart contracts, and no-std for constrained targets.

-
- 51** Zero-Knowledge Proofs for Rust Engineers
 - 52** ZoKrates Workflows and Ethereum Verifiers
 - 53** EZKL, Verifiable LLM Inference, and GPU-Aware ZKML
 - 54** ONNX Model Inference in Rust
 - 55** Smart Contracts in Rust: Solana, Sei, and Beyond
 - 56** no-std Rust and Constrained Runtime Derivatives
-

CHAPTER 51 · ZERO-KNOWLEDGE PROOFS FOR RUST ENGINEERS

Zero-Knowledge Proofs for Rust Engineers

Separate statements, witnesses, constraints, transcripts, keys, proofs, and service boundaries so ZK systems stay reviewable in Rust

A zero-knowledge proof lets one party convince another that a statement is true without revealing why it is true. For a Rust engineer the cryptography is rarely the hard part to ship. The hard part is the systems shape: a public statement, private witness custody, versioned artifacts, expensive proving capacity, and a small verifier API. This chapter treats those as boundaries you can name and review, not as one opaque call to a crate.

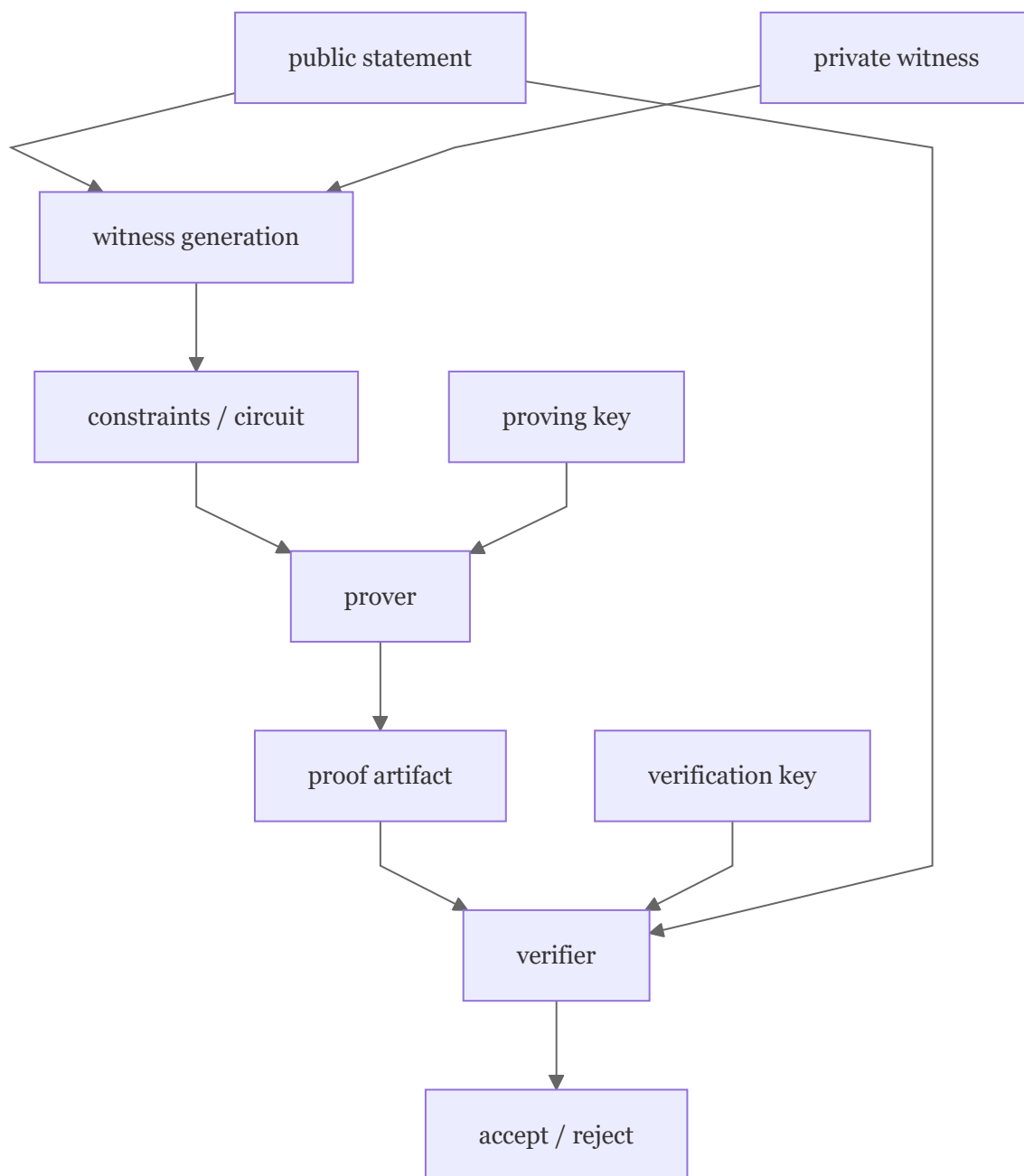
Builds on Chapters 19, 37, 38, and 43

Chapter 19 established canonical serialization and transport DTO discipline. Chapter 37 covered heavy accelerator lanes and their integration costs. Chapter 38 covered commitments, Merkle trees, and proof artifacts. Chapter 43 covered observability for specialized worker boundaries. This chapter recombines those ideas for proof-backed systems.

Opening scenario

Picture a service that has to vouch for a number it is not allowed to disclose. A lender wants to confirm that a customer's committed order total stays under their published credit limit without seeing the line items. A risk team wants to prove a score was computed from an approved model without exposing the model inputs. In both cases the business requirement is the same shape: let the verifier learn one specific claim and nothing else. To deliver that, you have to keep public statements, private witnesses, constraints, proof artifacts, verifier metadata, and proving capacity as separate, named things rather than letting them collapse into a single function that "does the proof."

The pipeline below is the spine of the whole chapter. Read it left to right and notice where the boundary flips: everything up to and including the prover holds the secret witness, and from the proof artifact onward only public data crosses. That single handoff is what every design decision in this chapter is protecting.



The proving side owns the witness; only the proof artifact and public statement cross to the verifier.

Mental model

Before any crate names or APIs, three ideas reorganize how you reason about this topic. Each one is a place where the obvious intuition from ordinary backend work quietly leads you wrong, and each one shows up later as a concrete type or service boundary.

Core concepts

The seven things a proof system actually traffics in

When people say "the ZK part" they usually mean a tangle of seven distinct objects. The reason integrations rot is that teams keep them implicit and let two of them merge. The fastest way to stay sane is to give each one a name and a home in your type system, exactly as you would for the inputs, outputs, and configuration of any other pipeline. The diagram earlier is just these seven objects wired together; the cards below define each one on its own terms.

What zero knowledge does and does not guarantee

It is worth being blunt about the boundaries of the guarantee, because "zero knowledge" sounds like it solves more than it does. The math protects one thing well: a verifier can be convinced of a statement without learning the witness, provided the proving system is sound and the circuit is correct. Everything outside that sentence is still your problem. The two columns below separate the promise from the things teams keep wishing it covered.

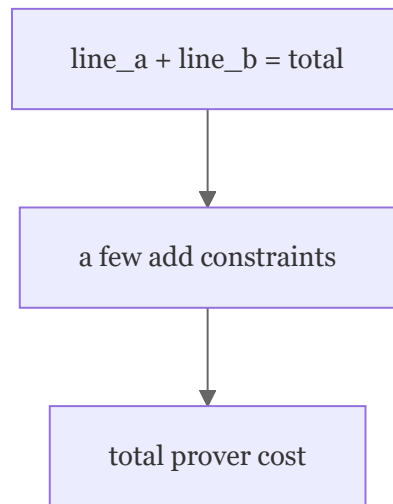
SNARKs, STARKs, commitments, hashes, and circuits at a pragmatic level

You do not need to derive these systems to integrate them well, but you do need a working sense of the tradeoffs they push onto your service. The choice between a SNARK and a STARK is rarely about elegance; it shows up in your design as proof size on the wire, verifier cost, and whether you inherit a trusted-setup ceremony you now have to document and defend. The summaries below are deliberately operational rather than mathematical.

Encoding computations as constraints

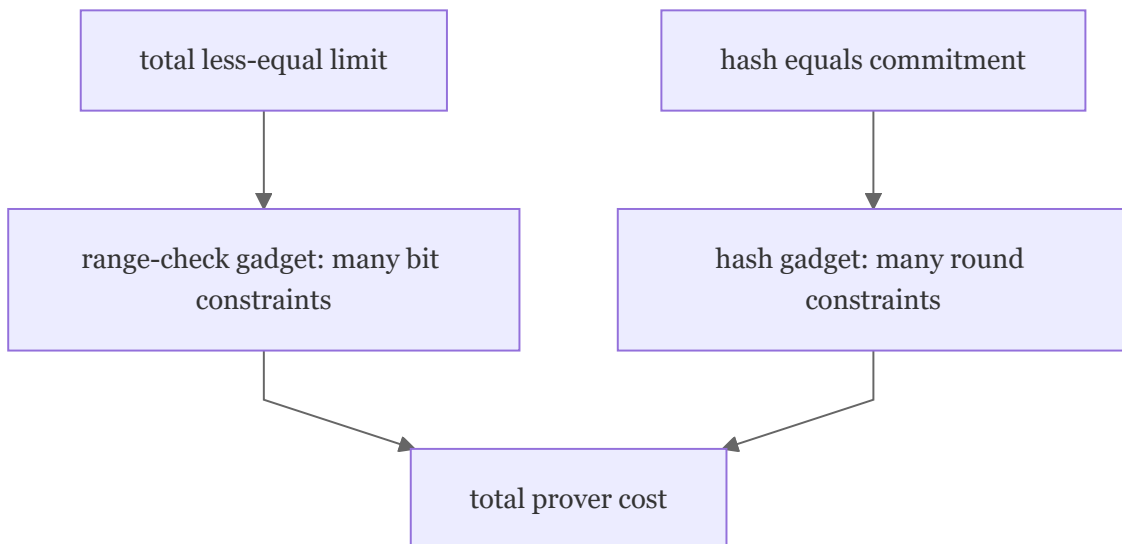
Engineers coming from C++, C#, or Go usually expect the cost model of the original function to carry over. It does not. A simple sum is cheap in constraints because it maps almost directly onto field arithmetic. Range checks, hashes, Merkle paths, and non-native field arithmetic are often far more expensive, because each one expands into a gadget made of many low-level constraints. The lesson is uncomfortable but useful: a one-line operation in your source can be the dominant cost in the circuit, so constraint review is an architecture review, not only a math exercise.

Look at the small example below before reading the listing. It proves that two private amounts sum to a public total, stay under a limit, and match a public commitment. Watch the right-hand column of the diagram: the addition stays cheap, but the innocent-looking $\text{total} \leq \text{limit}$ and the hash each blow up into a multi-constraint gadget. That asymmetry is the whole point.



The cheap row: a sum maps almost directly onto field arithmetic.

The other two rows look just as innocent in source, but each expands into a gadget that dominates the same total cost:

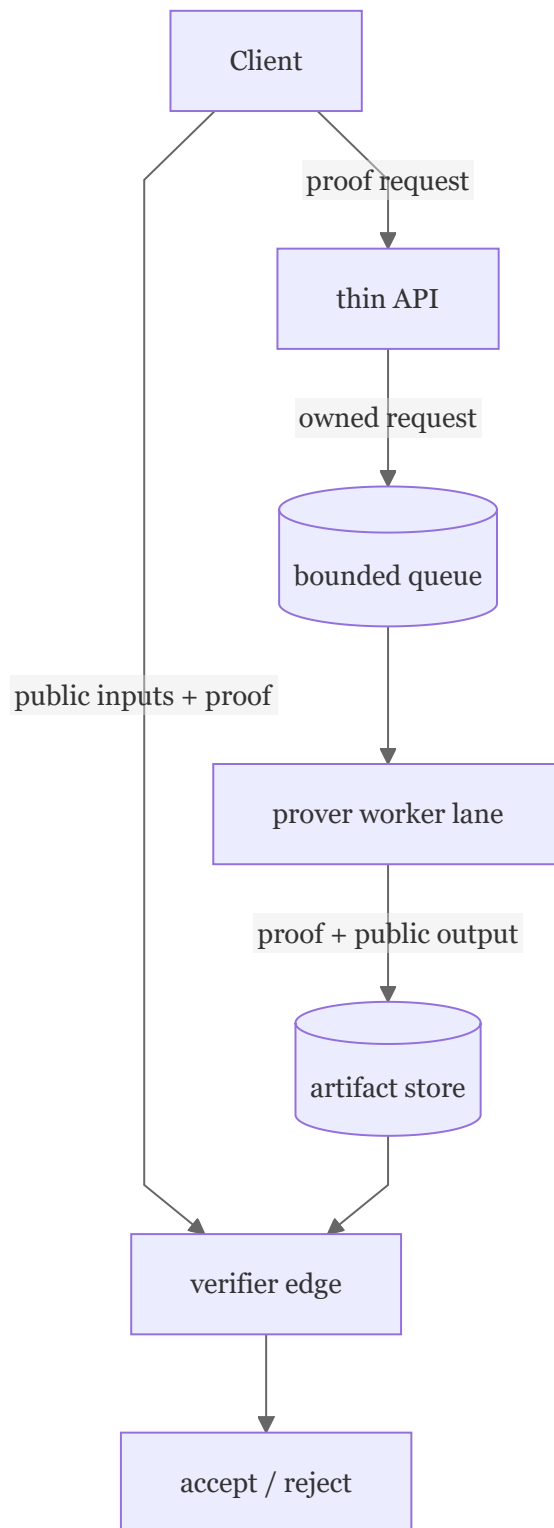


The expensive rows: a comparison and a hash each blow up into many constraints.

The subtlety is intentional. Inequality and hashing are written as a single line each, but neither is one cheap primitive inside most proving systems. They expand into gadgets with real prover and verifier cost, which is exactly why estimating circuit size from the source listing alone will mislead you.

Rust integration patterns for proof generation and verification boundaries

The single most important architectural decision is to stop treating proving and verifying as two ends of one function. They want to live on different machines, scale on different curves, and fail in different ways. The diagram shows the topology that keeps working whether the underlying prover is pure Rust, FFI-backed, GPU-backed, or a separate toolchain process: a thin API that enqueues owned proof requests, a bounded prover lane that holds all the heavy state, and a small verifier edge that only ever sees public data. The cards then break down the boundaries that diagram implies.



Heavy proving lives behind a queue; the verifier edge stays small and never touches the witness.

The load-bearing design rule is simple: proving and verification are different service shapes. The prover should own witness generation and heavy compute. The verifier should own public inputs and a small proof check. That split stays valuable even if the underlying ecosystem is pure Rust, FFI-backed, GPU-backed, or toolchain-driven.

Serialization, transcript handling, deterministic inputs, and domain separation

Most non-cryptographic proof failures trace back to this section. Prover and verifier must agree, byte for byte, on what was hashed and in what order, and they must derive their challenges from the same transcript. A transcript is just the running record of public values and messages that both sides absorb to derive challenges; if one side serializes a map in a different iteration order, or shares challenge space between two unrelated protocols, the proof becomes unverifiable even though the math is correct. These rules are protocol requirements, not cleanup tasks, and they belong in tests from the first commit.

Performance, memory, and hardware considerations

Teams new to this work tend to budget for the prover and forget everything around it. In practice the cost is spread across the pipeline, and the surprise is usually that witness preparation, not the prover itself, is the first thing to fall over. Read these four notes as a profiling checklist: find where wall time and peak memory actually go before you reach for a GPU, because hardware acceleration only pays off for some backends and some batch shapes.

Security review checklist and production caveats

A proof-backed system has two review surfaces that are easy to confuse, and confusing them is how sound cryptography still ships an insecure service. One surface is the cryptography: soundness, the trusted-setup or transparency story, circuit correctness, transcript construction. The other is the ordinary Rust systems surface: secret handling, canonical serialization, resource budgets, unsafe FFI wrappers, log redaction. A sound proving system put behind a leaky, non-deterministic integration is still a broken system, so both columns below need an owner.

How this lands by background

The cryptography is the same regardless of where you came from, but the instinct that misleads you is not. Each of these backgrounds brings one habit that helps here and one that quietly works against you. Read the card for your starting language as a mental-model adjustment, not a list of crate equivalents.

Production patterns

None of these patterns are exotic. They are the same separation-of-concerns and versioning discipline you would apply to any pipeline that crosses a trust boundary, applied here to the specific artifacts a proof system produces. The thread running through all of them is to keep the four moving parts, witness, proof, verifier request, and circuit version, individually nameable and testable.

Pitfalls and tradeoffs

Each of these is a real way teams have shipped a proof-backed system that looked correct and was not. They cluster around two illusions: that a proof equals privacy, and that a passing verification equals a correct circuit. Hold both of those suspect.

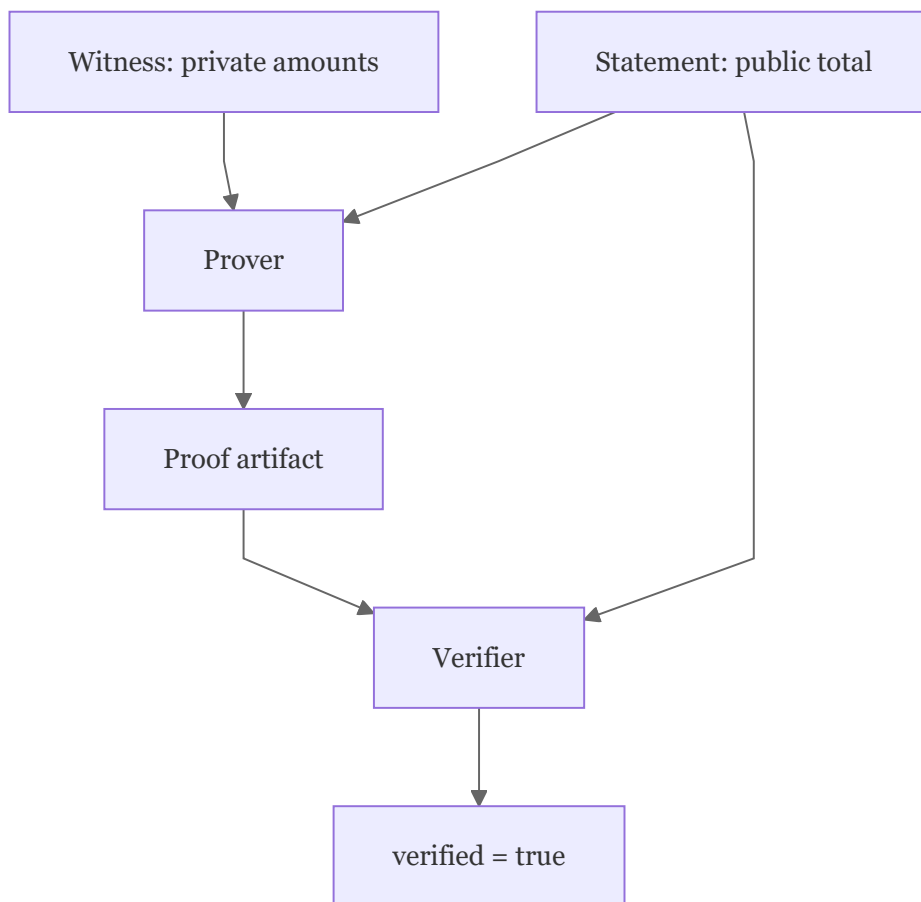
The biggest production mistake is often not “the prover was slow.” It is that the team never made witness generation, transcript policy, proving artifacts, and verifier integration reviewable as separate things.

Examples

The runnable examples model proof boundaries and transcript discipline without depending on one specific proving crate. They are intentionally about systems shape rather than real cryptography, so a browser demo cannot mislead you into thinking it produced a sound proof. What they do show faithfully is how the types keep the witness on one side of the boundary and how both sides agree on a transcript.

Example 1: keep statement, witness, and proof artifact separate

Before you read the code, fix the one rule it enforces in your head: the witness never appears in any type the verifier touches. Follow the data in the diagram below. The witness flows only into the prover; the verifier is handed a statement and a proof and nothing else. If you ever find the witness type imported on the verifier side, the design has already drifted.



The witness type reaches the prover only; the verifier sees statement and proof.

Example 51-1. proof_boundary_types.rs — Witness proof

```

#[derive(Debug, Clone, Copy)]
struct Statement {
    public_total: u64,
    public_limit: u64,
}

#[derive(Debug, Clone, Copy)]
struct Witness {
    left: u64,

```

```

        right: u64,
    }

    #[derive(Debug, Clone, Copy)]
    struct ProofArtifact {
        public_total: u64,
        public_limit: u64,
        proof_bytes_len: usize,
    }

    fn prove(statement: Statement, witness: Witness) -> Result<ProofArtifact, &'static str> {
        if witness.left + witness.right != statement.public_total {
            return Err("sum constraint failed");
        }

        if statement.public_total > statement.public_limit {
            return Err("public statement not admissible");
        }

        Ok(ProofArtifact {
            public_total: statement.public_total,
            public_limit: statement.public_limit,
            proof_bytes_len: 96,
        })
    }

    fn verify(statement: Statement, proof: ProofArtifact) -> bool {
        statement.public_total == proof.public_total
        && statement.public_limit == proof.public_limit
        && proof.proof_bytes_len > 0
    }

    fn main() {
        let statement = Statement {
            public_total: 45,
            public_limit: 50,
        };
        let witness = Witness { left: 20, right: 25 };
        let proof = prove(statement, witness).unwrap();

        println!("public total = {}", statement.public_total);
        println!("proof bytes = {}", proof.proof_bytes_len);
        println!("verified = {}", verify(statement, proof));
    }

```

OUTPUT

```

public total = 45
proof bytes = 96
verified = true

```

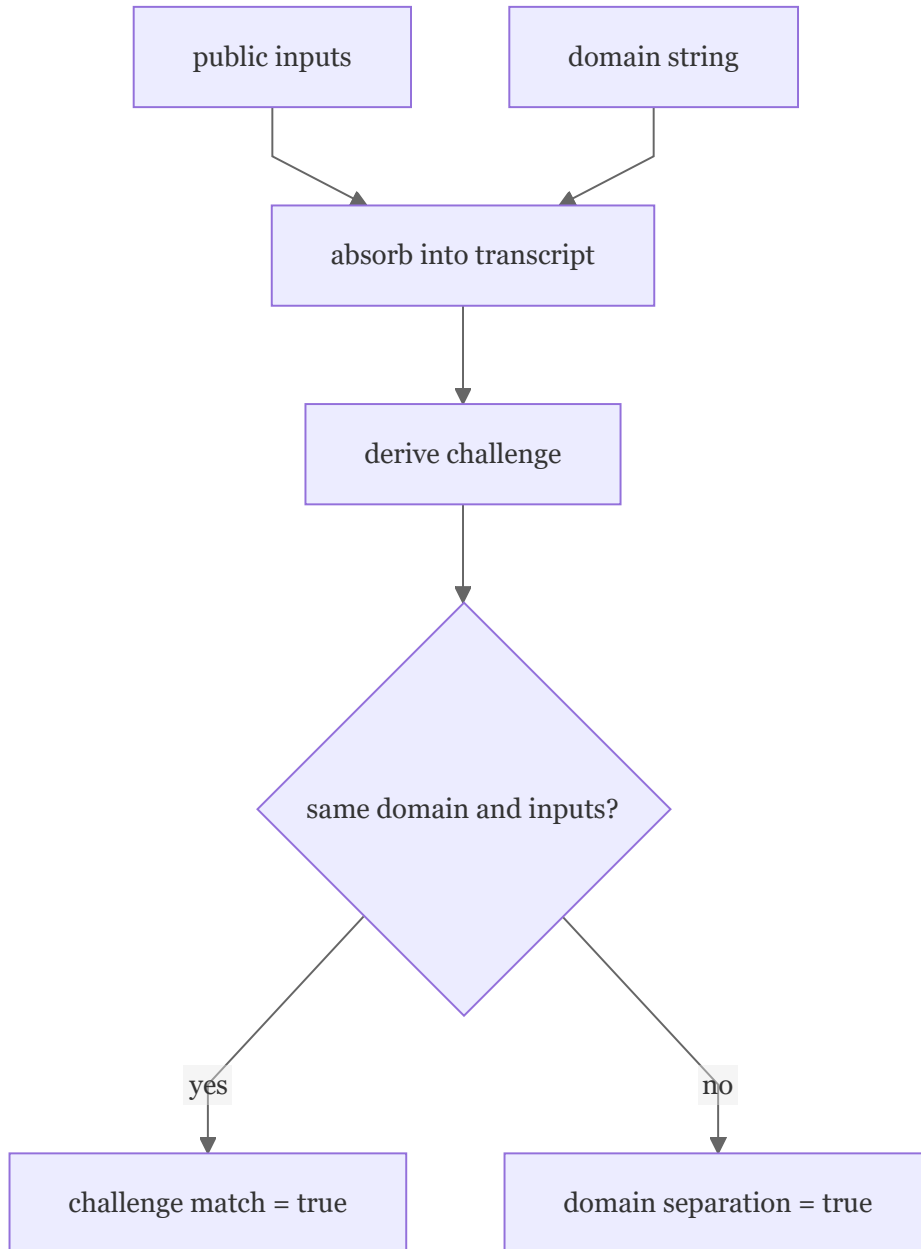
Public values belong in the verifier-facing boundary and can be logged or routed more broadly.

Private inputs stay proving-side only. If they appear in your verification DTOs, the design already drifted.

The proof is its own typed artifact. Treat it like versioned transport data, not like a hidden field on a witness struct.

Example 2: domain-separated transcripts and deterministic public inputs

The transcript here is intentionally simple and not cryptographic. The behavior to watch is the branching shown below: the same public inputs under the same domain string must converge on one challenge, while a different domain must land on a different one. The first property gives prover and verifier a shared challenge; the second keeps a billing proof from ever colliding with an inventory proof in the same challenge space.



Same domain and inputs converge on one challenge; a different domain diverges by design.

Example 51-2. transcript_domain_separation.rs — Domain separation

```

#[derive(Debug)]
struct Transcript {
    domain: &'static str,
    bytes: Vec<u8>,
}
  
```

```

impl Transcript {
    fn new(domain: &'static str) -> Self {
        Self {
            domain,
            bytes: domain.as_bytes().to_vec(),
        }
    }

    fn append_u64(&mut self, label: &str, value: u64) {
        self.bytes.extend_from_slice(label.as_bytes());
        self.bytes.extend_from_slice(&value.to_le_bytes());
    }

    fn challenge(&self) -> u64 {
        self.bytes.iter().fold(1_469_598_103_934_665_603_u64, |acc, byte| {
            acc.wrapping_mul(1_099_511_628_211).wrapping_add(*byte as u64)
        })
    }
}

fn challenge_for(domain: &'static str, public_total: u64, public_commitment: u64) -> u64 {
    let mut transcript = Transcript::new(domain);
    transcript.append_u64("public_total", public_total);
    transcript.append_u64("public_commitment", public_commitment);
    transcript.challenge()
}

fn main() {
    let domain = "billing-proof:v1";
    let prover_challenge = challenge_for(domain, 45, 9_001);
    let verifier_challenge = challenge_for(domain, 45, 9_001);
    let other_domain_challenge = challenge_for("inventory-proof:v1", 45, 9_001);

    println!("domain = {}", domain);
    println!("challenge match = {}", prover_challenge == verifier_challenge);
    println!("domain separation = {}", prover_challenge != other_domain_challenge);
}

```

OUTPUT

```

domain = billing-proof:v1
challenge match = true
domain separation = true

```

The same public inputs and same domain should yield the same verifier-visible transcript state.

Different protocol domains should not accidentally share challenge space or artifact interpretation.

Public input ordering and transcript rules are protocol compatibility decisions between prover and verifier.

The repository also includes standalone Rust source under `examples/ch51_zero_knowledge_proofs_rust_engineers/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

ZoKrates Workflows and Ethereum Verifiers

Run ZoKrates like a toolchain: compile/setup/witness/proof/export/verify stages, artifact custody, Ethereum verifier integration, and upgrade discipline

Proof workflows need controlled source programs, setup material, witness custody, generated verifiers, and deployment tracking. This chapter covers ZoKrates and Ethereum verification as staged production artifacts.

Builds on Chapters 44, 46, 49, and 51

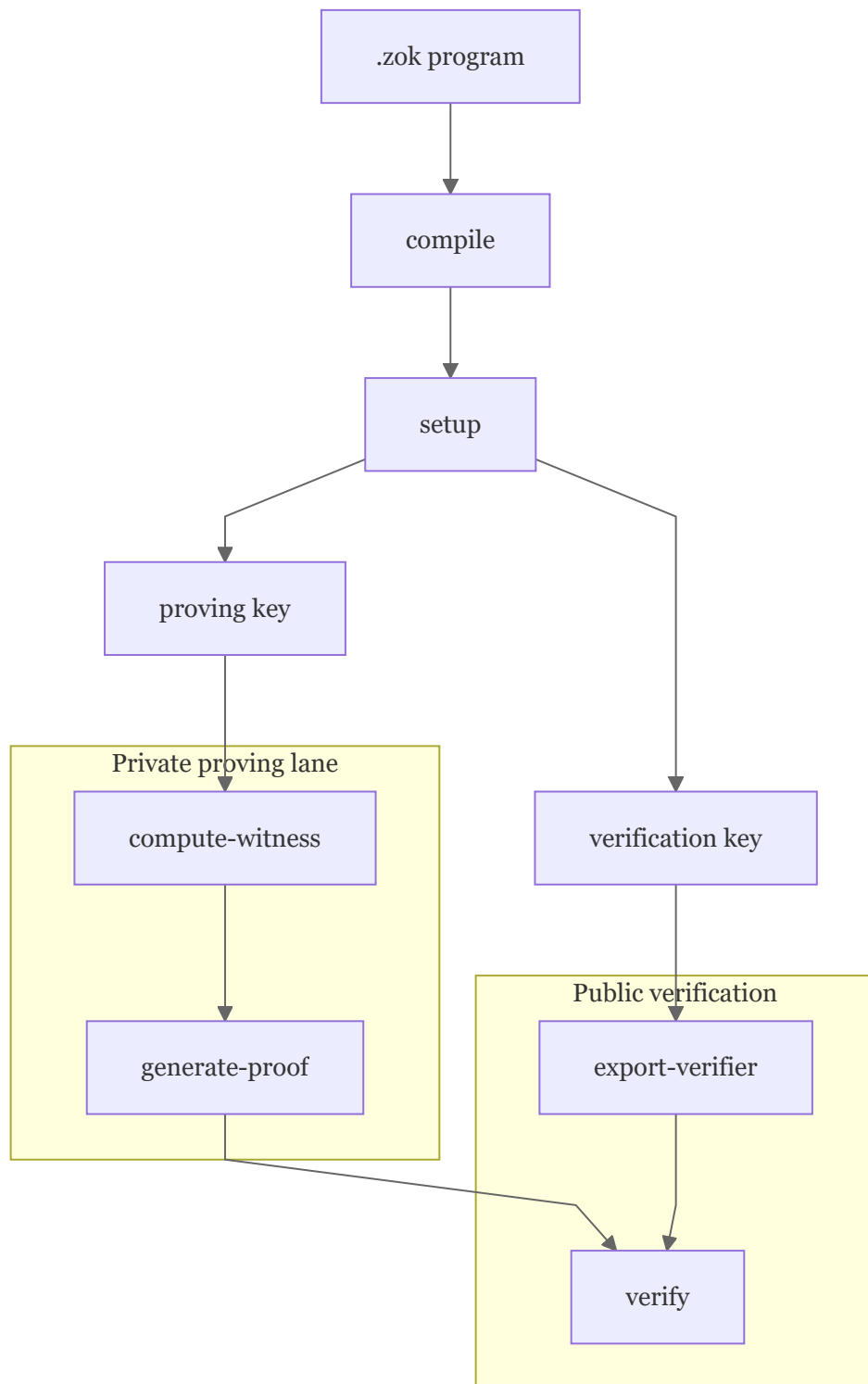
Chapter 51 established the general ZKP mental model: statements, witnesses, constraints, transcripts, and proof artifacts. Chapter 44 covered packaging and custody of build outputs. Chapter 46 covered transport-facing contract boundaries, and Chapter 49 covered secure service boundaries. This chapter narrows that material to the ZoKrates workflow and Ethereum-oriented verifier integration.

Opening scenario

Picture a compliance service that has to prove a fact about a customer to a smart contract without ever revealing the underlying data. The classic example is an age check: the contract needs to be convinced that a person is over eighteen, but it must never see the birth date that establishes it. That is exactly the kind of statement a zero-knowledge proof can carry, and ZoKrates is the toolchain we will use to turn the statement into a circuit, a proof, and a verifier that Ethereum can call.

The work is less about cryptography than it looks. The cryptography lives inside ZoKrates. Your job, in Rust, is to manage everything around it: shaping proof-friendly inputs, driving the compile and setup stages, keeping witness data in custody, generating proofs in a worker, deploying the generated verifier contract, and sending versioned verification requests on-chain. Each of those stages produces an artifact with its own secrecy and ownership rules, and the cost of getting the workflow wrong is almost never a single failed proof. It is a system nobody else can operate safely.

The shape to hold in your head is a one-directional pipeline. The source program is compiled once, a setup step derives a matched pair of keys, and then each proof flows from a private witness on the left toward a public verification on the right. Read the diagram left to right and notice the two lanes that must never cross: the witness stays in the private proving lane, and only the proof plus public inputs reach the verifier.



One source program, one setup, then every proof flows from a private witness to a public verifier. The witness never crosses into the public lane.

How this differs from Chapter 51: that chapter explained the proof-system concepts themselves, the statement, witness, constraints, and transcript. This chapter is concrete and operational. It treats ZoKrates as a tooling workflow and Rust as the orchestration, custody, and integration layer wrapped around it.

Mental model

Three ideas keep this work tractable. ZoKrates is a boundary you orchestrate rather than a library you embed. Proving and verifying are different shapes of service with different costs. And Ethereum integration is mostly artifact management plus a small amount of transport translation. Hold these and most design questions in the chapter answer themselves.

Core concepts

The seven stages: write, compile, setup, compute witness, generate proof, export verifier, verify

Every ZoKrates project moves through the same seven stages, and the most useful thing you can do as the Rust engineer is to notice that they do not all belong in the same place. Writing and compiling the circuit are source-control concerns. Setup is a one-time, trust-sensitive ceremony. Computing the witness and generating the proof are the heavy lifting that belongs in a worker. Exporting the verifier is a release step, and verification is the only stage that an external caller, including a smart contract, ever touches directly. The table below names each stage, the artifact it produces, who should own it, and the one operational fact worth remembering about it.

Stages assume default filenames unless overridden: out for the compiled program, proving.key and verification.key from setup, witness from compute-witness, and proof.json from generate-proof. Pass -i and -o when you want explicit per-circuit paths instead.

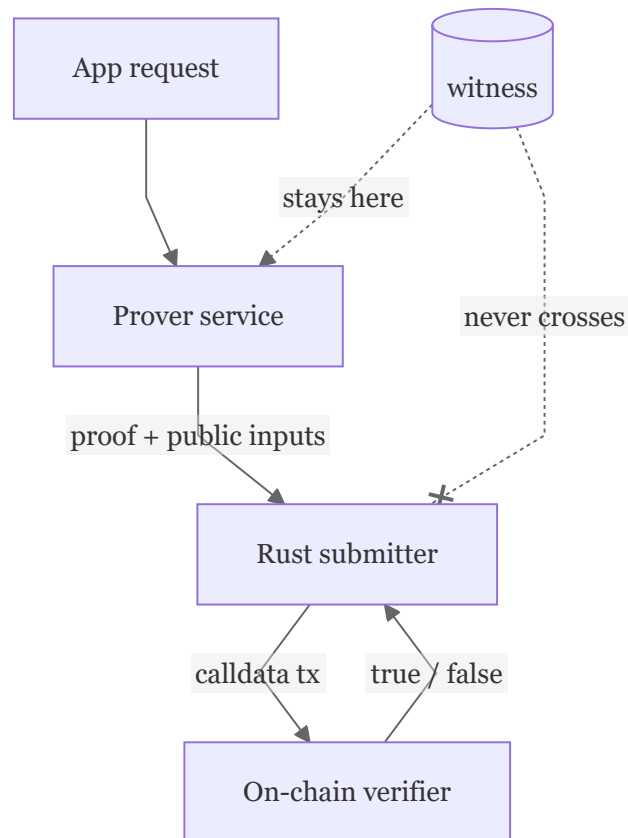
Shaping inputs so the proof program stays small

A circuit is not a general-purpose program. Every operation it performs becomes constraints, and constraints are what make proving expensive, so the data you feed a ZoKrates program should already be numeric, bounded, and canonical. The work of turning a messy HTTP body or a JSON document into clean field elements belongs in Rust, before witness generation ever starts. The four guidelines below all push in the same direction: keep the proof program lean and let Rust absorb the irregularity of the outside world.

A common modeling repair is to let Rust do transport parsing, range normalization, hashing or commitment preparation, and only then hand canonical numeric or committed values into the ZoKrates program.

Connecting a generated verifier to an Ethereum system

When a smart contract is the verifier, the boundary between your Rust services and the chain becomes the place where most mistakes happen. The cleanest layout keeps three responsibilities physically separate: a prover service that does the heavy work and emits a proof, a thin Rust submitter that encodes public inputs and sends the transaction, and the on-chain verifier contract that returns true or false. The submitter is the only component that talks to Ethereum, and it only ever sees public inputs and a proof, never a witness. Hold that split in mind as you read the four rules that follow.

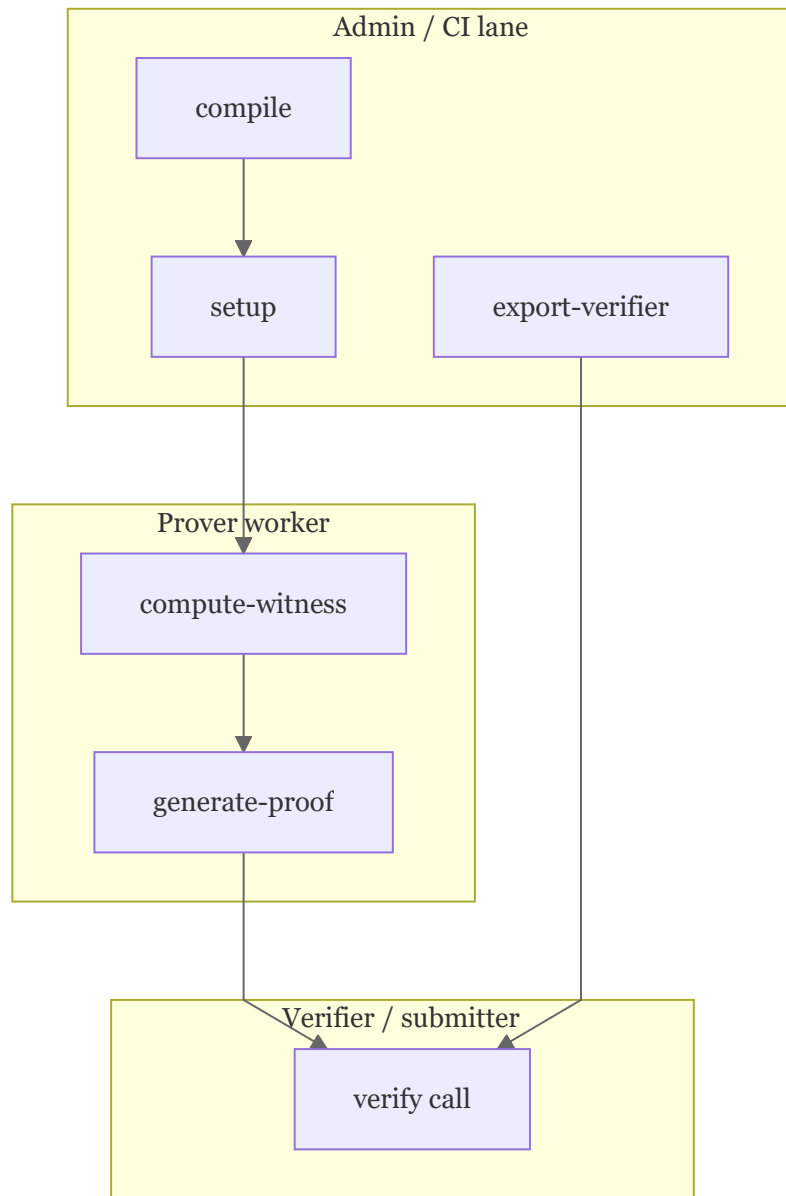


The submitter is the only path to Ethereum, and it carries proof plus public inputs only. The witness stays with the prover.

In practice Rust owns the contract address map, the public-input encoder, proof-blob transport, and the on-chain submission policy, including gas, confirmation, and retry. The Solidity verifier itself stays a generated contract artifact with its own toolchain and deployment review path. Owning the calldata is not the same as owning the verifier logic, and conflating the two is how teams end up shipping a contract nobody reviewed.

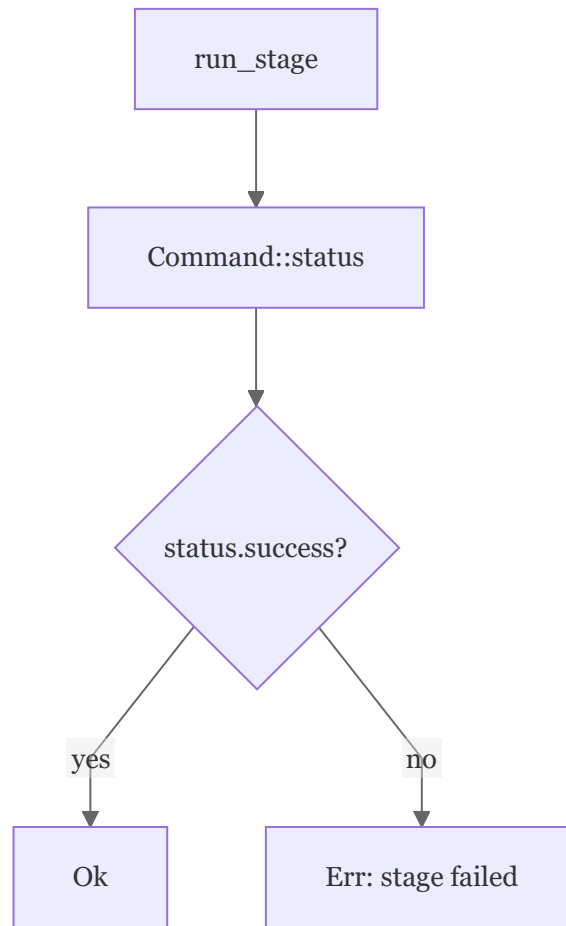
Where each stage runs: admin lane, prover worker, and verifier

A common early mistake is to wire the whole workflow into one place, often a request handler or a build.rs step, because the prototype was small. The stages have genuinely different operational profiles, so they want different homes. Compile, setup, and export-verifier are infrequent and trust-sensitive and belong in a controlled CLI or CI lane. Witness generation and proof creation are heavy and belong in a bounded worker. Verification is light and belongs in a thin service or a transaction submitter. The diagram shows that separation before the cards describe each lane.



Three lanes, three operational profiles: infrequent and trusted, heavy and bounded, light and exposed.

Whatever lane a stage runs in, Rust ultimately shells out to the ZoKrates binary. The pattern is small but worth getting right: spawn the process, wait for it to finish, and translate a non-zero exit status into a proper Rust error instead of letting it pass silently. The helper below does exactly that, returning `Ok()` only when the stage actually succeeded. Read it as the success-or-error branch a worker would call once per stage.



Every stage invocation collapses to one decision: did the process exit cleanly, or does Rust raise an error?

The rule that follows from all of this: do not hide setup or proof generation inside a routine `build.rs` or a request-handler path. Those steps are too heavy, too stateful, and too security-sensitive to behave like ordinary compilation or a synchronous validation check.

Who owns each artifact, and how long it should live

The workflow leaves a trail of files behind it, and each one has a different secrecy level and a different custodian. The proving key is sensitive and access-controlled; the verification key is public and ships inside the deployed contract; the witness is the secret you went to all this trouble to protect and should barely exist on disk at all. The single discipline that ties them together is version alignment: a proof, a key, and a verifier contract only mean anything as a matched set. The table is your checklist for keeping that set straight.

Testing the workflow in layers

Proof failures are notoriously opaque: a verifier returning false tells you nothing about which of a dozen moving parts drifted. The cure is to test in layers and to make failures specific. Unit-test the Rust input normalization on its own. Contract-test local verification against known fixtures. Then keep the Ethereum compile-and-deploy path in its own lane. Above all, write negative tests that fail for named reasons, so an incident points at the wrong key or the wrong input ordering instead of a generic mystery.

Where the time and cost actually go

Most teams assume proof generation is the expensive part and stop measuring there. It is expensive, but the full picture is more interesting. Witness generation, especially when Rust is hashing large payloads or building Merkle paths first, can dominate before the prover even starts. Verification is cheap by comparison but never free: on-chain it burns gas, off-chain it still parses, checks versions, and may retry. The cards below mark the hot lanes worth isolating and measuring.

In production, queue age for proof generation, witness build time, proof size, verifier-call latency, and contract or RPC failure rates matter at least as much as the cryptographic happy path.

What to plan for before calling it production-ready

A few concerns sit outside the happy path but decide whether the system is safe to run. If the proving scheme relies on a trusted setup, the provenance of that setup is part of your security model, not a footnote. An upgrade is rarely a single switch: a new circuit usually means new keys, new proofs, and possibly a new contract, so plan for a window where two versions run side by side. And keep the audit surfaces distinct, because circuit correctness, Rust secret handling, and Solidity deployment safety are different reviews that should never collapse into one vague claim that the proof system is secure.

Generated verifier contracts and setup artifacts are not production-safe merely because they came from a ZK toolchain. They still need ordinary release engineering, provenance, review, and rollback planning.

How this lands by background

The hard part of this chapter is not the cryptography; it is resisting the instinct to fold a multi-stage, artifact-heavy pipeline into one tidy abstraction. What that instinct looks like depends on where you are coming from, so here is the mental-model shift for each starting point.

Production patterns

Pitfalls and tradeoffs

The most expensive ZoKrates integration mistake is usually not one failed proof. It is a system that never made circuit version, key provenance, witness secrecy, and verifier deployment boundaries explicit enough for another engineer to operate safely.

Examples

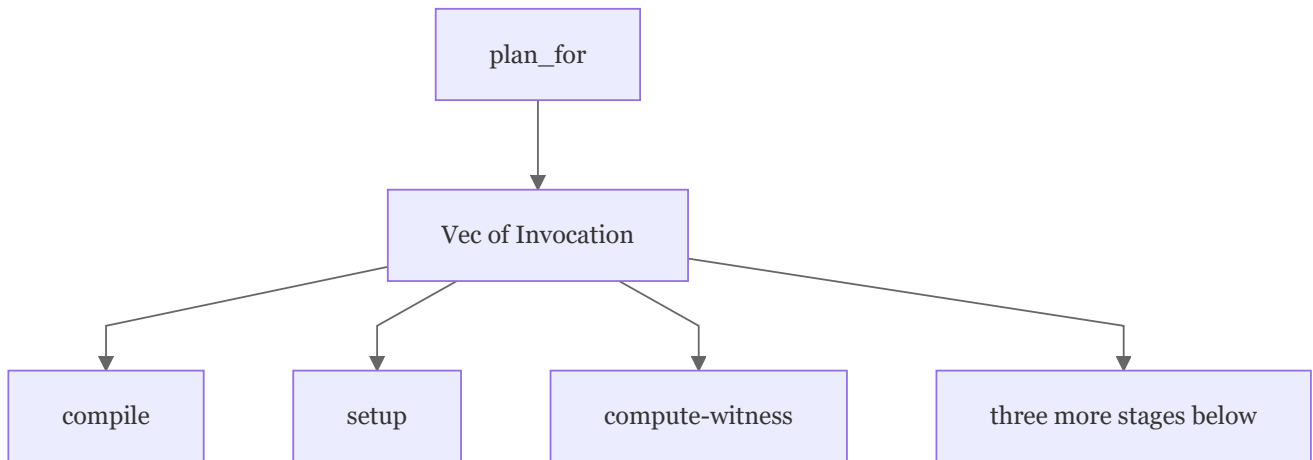
These examples are intentionally Rust-facing orchestration and boundary examples. They do not pretend the verifier contract is handwritten Rust or that the witness belongs in every request type.

Example 1: build a Rust orchestration plan around ZoKrates CLI stages

The point is not to run the commands in-browser. The point is that Rust can version, render, and own the pipeline explicitly before a CLI runner or service worker executes it.

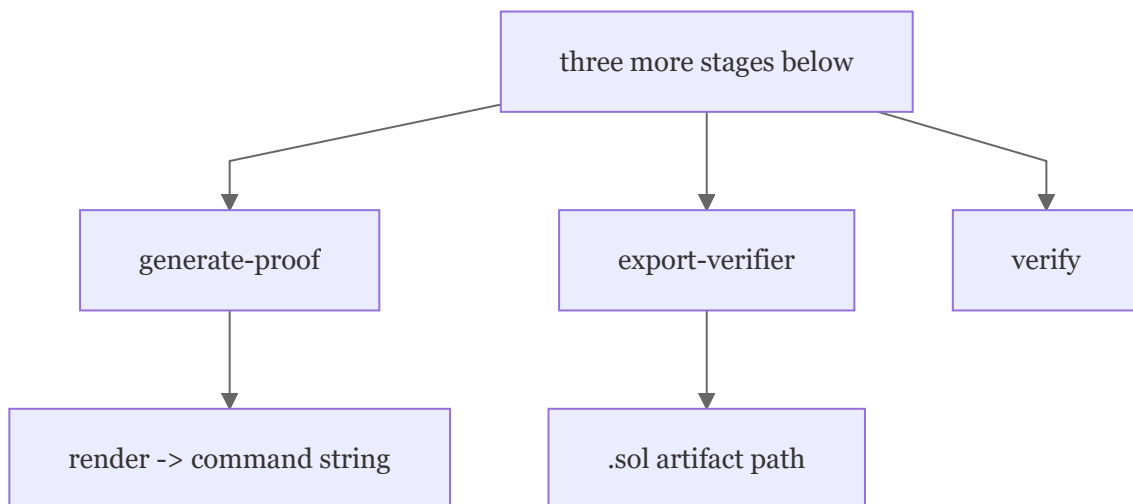
What to look at: the code does not run any ZoKrates command. It builds a list of Invocation values, one per stage, each carrying its program, its arguments, and the artifact path it will produce. Follow `plan_for`: it

derives a single artifacts/<circuit> base path and threads it through compile, setup, witness, proof, export, and verify so every stage agrees on where the previous one wrote. The diagram shows that the plan is just data, ready for a runner to execute later.



First half: plan_for returns data, not side effects. It builds a typed list of invocations, one per stage.

The remaining stages round out the list, and two of them produce the concrete outputs a worker reads back:



Second half: generate-proof renders to a command string and export-verifier yields the .sol artifact path a runner can execute in order.

Example 52-1. workflow_orchestration_plan.rs — Command plan

```

use std::path::PathBuf;

#[derive(Debug, Clone)]
struct Invocation {
    stage: &'static str,
    program: &'static str,
    args: Vec<String>,
    artifact: PathBuf,
}

fn plan_for(circuit: &str, witness_args: &[&str]) -> Vec<Invocation> {

```

```

let compiled = format!("artifacts/{}", circuit);
vec![
    Invocation {
        stage: "compile",
        program: "zokrates",
        args: vec![
            "compile".into(),
            "-i".into(),
            format!("{}", circuit),
            "-o".into(),
            compiled.clone(),
        ],
        artifact: PathBuf::from(&compiled),
    },
    Invocation {
        stage: "setup",
        program: "zokrates",
        args: vec!["setup".into(), "-i".into(), compiled.clone()],
        artifact: PathBuf::from("artifacts/proving.key"),
    },
    Invocation {
        stage: "compute-witness",
        program: "zokrates",
        args: {
            let mut args = vec![
                "compute-witness".into(),
                "-i".into(),
                compiled.clone(),
                "-a".into(),
            ];
            args.extend(witness_args.iter().map(|value| value.to_string()));
            args
        },
        artifact: PathBuf::from("artifacts/witness"),
    },
    Invocation {
        stage: "generate-proof",
        program: "zokrates",
        args: vec!["generate-proof".into(), "-i".into(), compiled.clone()],
        artifact: PathBuf::from("artifacts/proof.json"),
    },
    Invocation {
        stage: "export-verifier",
        program: "zokrates",
        args: vec!["export-verifier".into(), "-i".into(), compiled.clone()],
        artifact: PathBuf::from("artifacts/AggCheckVerifier.sol"),
    },
    Invocation {
        stage: "verify",
        program: "zokrates",
        args: vec!["verify".into(), "-i".into(), compiled],
        artifact: PathBuf::from("artifacts/verify.log"),
    },
]
}

```

```
fn render(invocation: &Invocation) -> String {
    format!("{}", invocation.program, invocation.args.join(" "))
}

fn main() {
    let plan = plan_for("age_check", &["18", "21"]);

    println!("steps = {}", plan.len());
    println!("compile = {}", render(&plan[0]));
    println!("proof = {}", render(&plan[3]));
    println!("contract = {}", plan[4].artifact.display());
}
```

OUTPUT

```
steps = 6
compile = zokrates compile -i age_check.zok -o artifacts/age_check
proof = zokrates generate-proof -i artifacts/age_check
contract = artifacts/AgeCheckVerifier.sol
```

Rust owns the sequence, arguments, and artifact paths even if ZoKrates performs the cryptographic work.

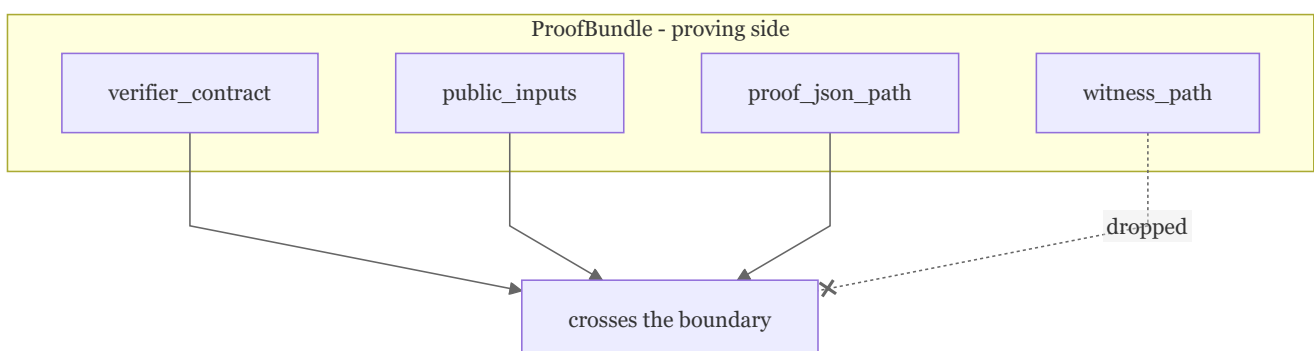
Compile, setup, proof, and verifier export are explicit stages instead of being hidden in one ad hoc shell script or request handler.

The generated verifier contract is already visible as a deployable file, not as a side effect nobody versions later.

Example 2: convert a proof bundle into an Ethereum verifier call boundary

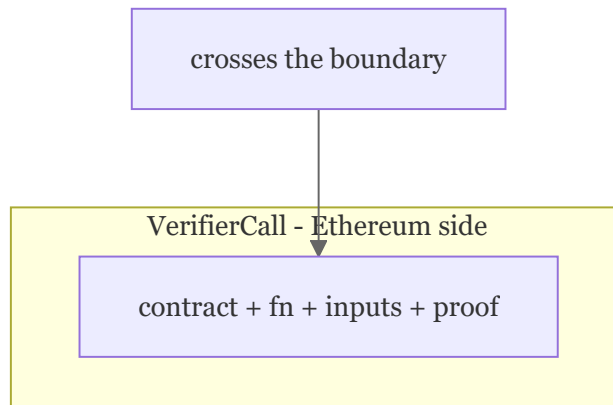
The verifier call boundary keeps witness data out and exposes only what an Ethereum-facing contract call should need: contract location, verifier function, public inputs, and proof source.

What to look at: there are two structs, and the gap between them is the whole point. The ProofBundle is the proving-side record and it includes a witness_path. The VerifierCall has no such field. Watch to_verifier_call copy across the contract, function, public-input count, and proof source, and simply leave the witness behind. The diagram traces which fields cross the boundary and which one is dropped on purpose.



First half: the proving-side ProofBundle. Three public fields cross the boundary; the witness path is dropped on purpose.

Only the fields that crossed become the Ethereum-side request:



Second half: `to_verifier_call` assembles the `VerifierCall` from the public fields only, so the witness can never reach the chain.

Example 52-2. `ethereum_verifier_boundary.rs` — *Verifier boundary*

```

#[derive(Debug, Clone)]
struct ProofBundle {
    circuit_id: String,
    verifier_contract: String,
    public_inputs: Vec<String>,
    proof_json_path: String,
    witness_path: String,
}

#[derive(Debug, Clone)]
struct VerifierCall {
    contract: String,
    function: &'static str,
    public_inputs_len: usize,
    proof_source: String,
}

fn to_verifier_call(bundle: &ProofBundle) -> VerifierCall {
    VerifierCall {
        contract: bundle.verifier_contract.clone(),
        function: "verifyTx",
        public_inputs_len: bundle.public_inputs.len(),
        proof_source: bundle.proof_json_path.clone(),
    }
}

fn main() {
    let bundle = ProofBundle {
        circuit_id: String::from("age-check:v2"),
        verifier_contract: String::from("contracts/AgeCheckVerifier.sol"),
        public_inputs: vec![String::from("45"), String::from("50")],
        proof_json_path: String::from("artifacts/proof.json"),
        witness_path: String::from("artifacts/witness"),
    };

    let call = to_verifier_call(&bundle);
    let witness_leaked = call.proof_source.contains("witness");
  
```

```
println!("contract = {}", call.contract);
println!("verifier fn = {}", call.function);
println!("public inputs = {}", call.public_inputs_len);
println!("witness leaked = {}", witness_leaked);
}
```

OUTPUT

```
contract = contracts/AgeCheckVerifier.sol
verifier fn = verifyTx
public inputs = 2
witness leaked = false
```

The Rust boundary models what the contract call actually needs instead of reusing the proving-side type.

Witness-only fields remain proving-side custody, not Ethereum verifier input.

Contract path and circuit ID belong in the proof bundle because generated verifier code is tied to one proof system configuration.

Rust can own calldata preparation, retry policy, and contract address mapping without pretending it owns the Solidity verifier logic itself.

The repository also includes standalone Rust source under `examples/ch52_zokrates_workflows_ethereum_verifiers/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

EZKL, Verifiable LLM Inference, and GPU-Aware ZKML

Productize verifiable inference with explicit model artifacts, tokenization and quantization policy, bounded proving lanes, GPU-aware profiling, and verifier-safe APIs

Verifiable inference lets a server prove it ran a specific model on specific inputs, so a third party can check the result without rerunning the model or seeing the private data. Doing that for an LLM, on a GPU, in production, turns into an artifact pipeline: frozen model graphs, input commitments, witness generation, proving capacity, accelerator budgets, and a thin verifier. This chapter treats EZKL-style ZKML not as a math demo but as a service architecture you have to own and operate.

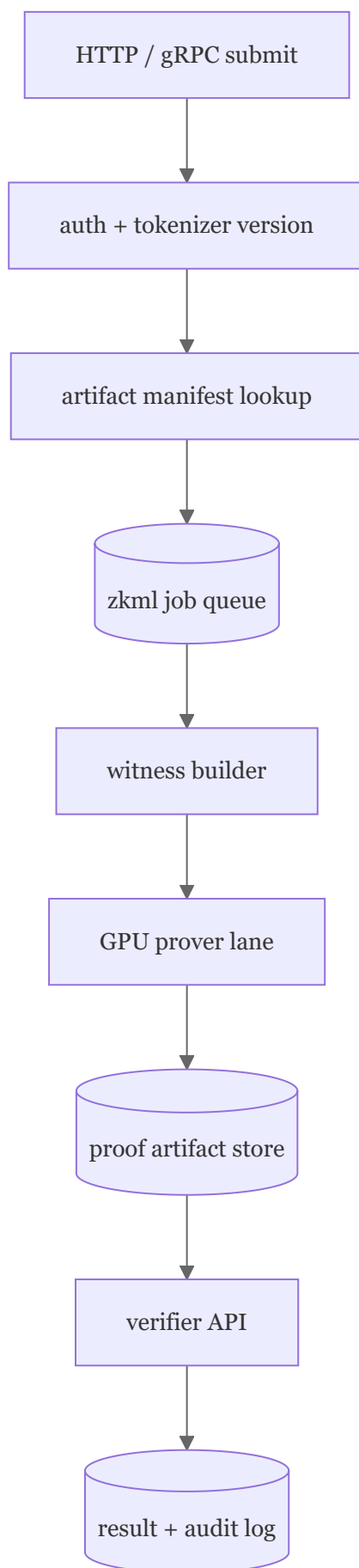
Builds on Chapters 37, 43, 47, 51, and 52

Chapter 37 covered GPU cost models and safe host-side wrappers. Chapter 43 covered observability for heavy asynchronous systems. Chapter 47 covered typed service and transport boundaries. Chapter 51 covered statements, witnesses, transcripts, and proof artifacts. Chapter 52 covered proof workflow orchestration and verifier artifact custody. This chapter applies those ideas to verifiable ML and LLM-serving workflows.

Opening scenario

You run a multi-tenant inference gateway. A regulated customer asks a fair question: how do they know the model you advertised, with the inference policy you published, is actually the model that produced their result? They cannot rerun it because they do not have the weights, and you cannot hand them the private prompts of other tenants. Verifiable inference resolves the standoff. The server attaches a cryptographic proof that a named computation, over committed public inputs, produced the claimed output, and the customer checks that proof in milliseconds without seeing anything private.

That single product promise expands into a surprising amount of infrastructure. You need a frozen model graph, a versioned tokenization and quantization policy, a witness builder that assembles the private execution trace, bounded proving lanes that may reach for a GPU, a small verifier API, and telemetry across every stage. The rest of this chapter walks that pipeline. Before the prose, here is the shape of the request as it moves from an HTTP submitter all the way to an audit log.



The heavy lane (witness, prover) sits behind a queue; the verifier on the right is the small, fast lane.

The same topology as a service sketch

Read the diagram top to bottom in code form. The indentation shows which stores and manifests each stage reaches into. The submit edge and the verifier API are deliberately thin; everything expensive hides behind the queue.

Hold onto the one boundary that beginners always blur: verifiable inference proves a computation claim under chosen artifacts. It does not automatically prove that the model is truthful, fair, safe, aligned, or even useful. A perfectly valid proof can wrap a terrible answer.

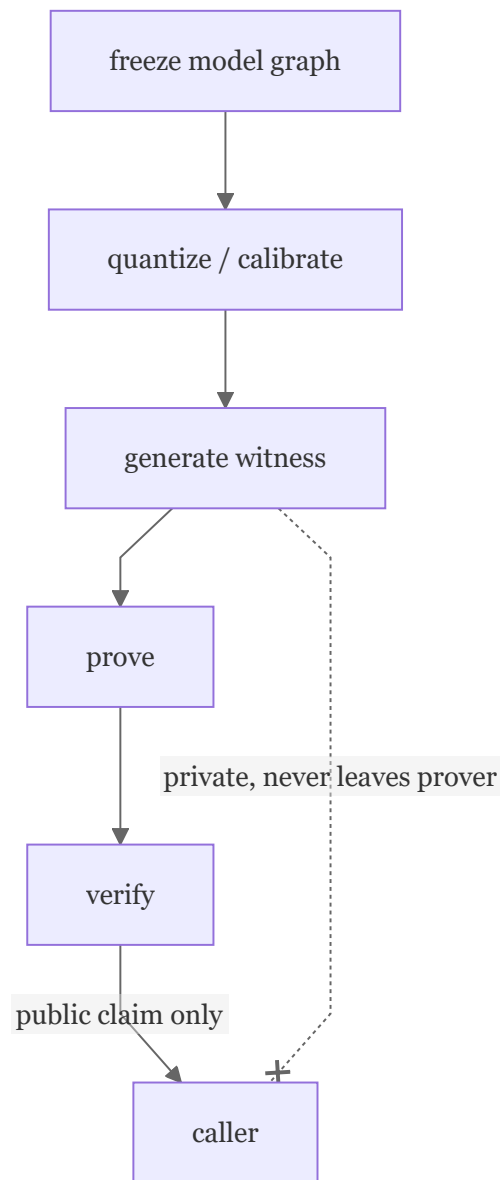
Mental model

If you remember three things from this chapter, make them these. ZKML is an artifact pipeline before it is a model call, the cost is wildly asymmetric between proving and verifying, and any claim about GPU acceleration has to be earned with a measurement. The three cards below unpack each one. They are the lens you should hold while reading every later section, because almost every production mistake in this space is really a violation of one of them.

Core concepts

The five stages every ZKML pipeline shares

Strip away the tooling and every verifiable-inference flow is the same five steps in the same order. You freeze the computation into a graph, you pin a numeric representation so the proof system can reason about it, you build the witness from the actual run, you prove, and you verify. Each arrow in the diagram below is a place where a version mismatch can silently change what the proof means, which is why the cards that follow treat each stage as a contract rather than a convenience.



One direction, five stages. The witness is the only stage that must never cross to the verifier side.

Where EZKL fits, and what to keep out of the request path

EZKL is one mature option for compiling a model graph into a proof-oriented circuit and driving the export-quantize-witness-prove-verify cycle. Its exact commands and file layouts change between releases, so do not memorize them. Memorize the boundary instead: compilation, trusted setup, and heavy proof generation belong in CI, admin tooling, or dedicated workers, while the API edge does almost nothing but select artifacts, enqueue work, and hand back a receipt. The four cards spell out what that discipline buys you.

Why LLMs are the hard case

Everything above applies to any model, but LLMs stress every part of it at once. The tokenizer becomes part of the signed contract, attention makes proving cost scale brutally with sequence length, fixed-point encoding has to stand in for the floats you took for granted, batching reshapes both your accelerator math and your replay size, and open-ended sampling adds nondeterminism a proof cannot tolerate. The practical consequence, which the last card states plainly, is that mature systems rarely prove free-form generation. They prove a bounded, deterministic slice and are honest about it.

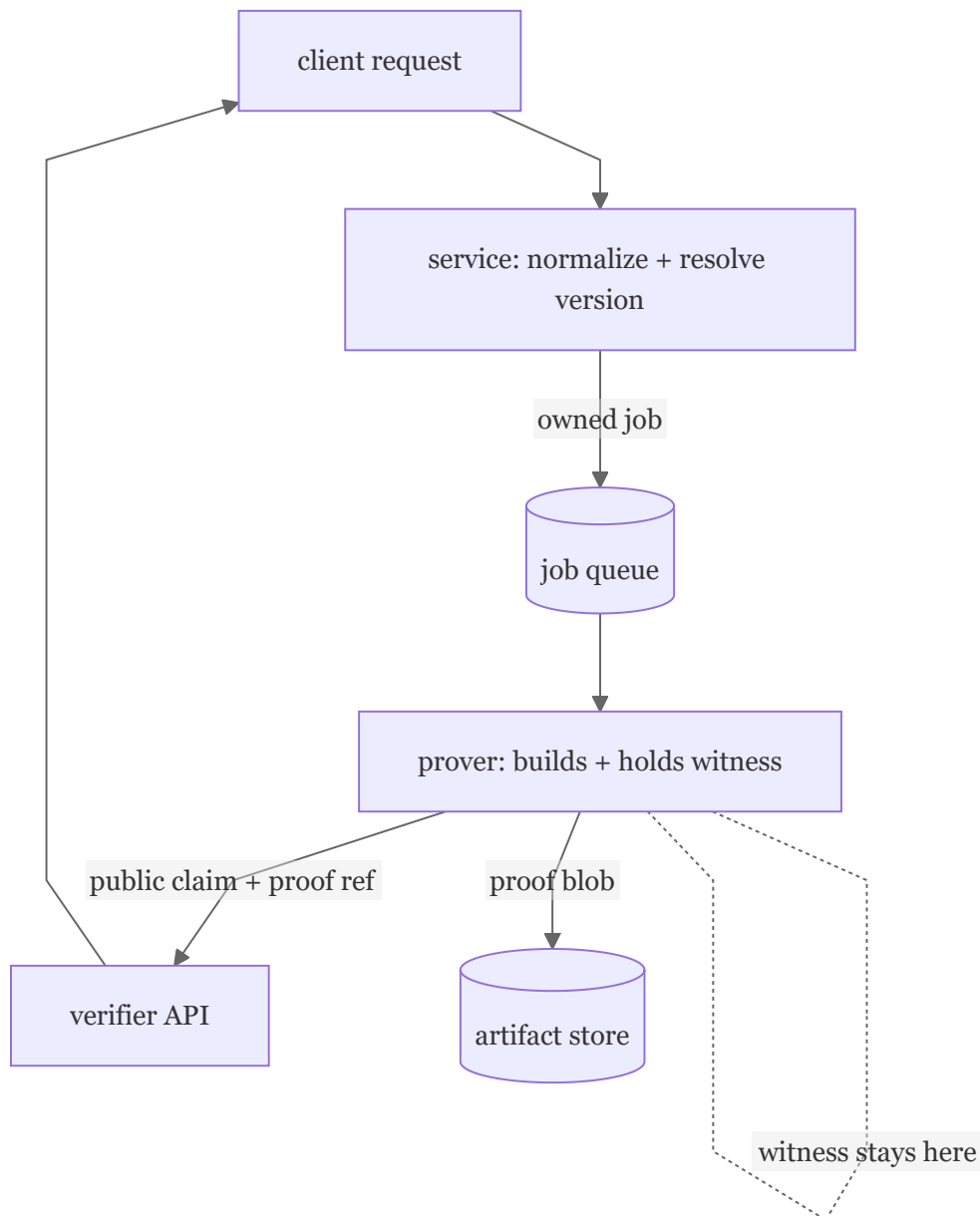
GPU-aware execution: opportunities, transfer costs, and failure modes

The temptation is to assume the GPU is always faster and move on. Chapter 37 already warned against that, and ZKML makes the warning sharper because the proving kernel is only one slice of wall time. A blazing kernel still loses if host-to-device copies, queue wait, or artifact serialization dominate, and a GPU lane brings its own incident class: out-of-memory, driver resets, and CPU/GPU artifact mismatches. The cards below frame the accelerator as a workload-dependent option you justify with telemetry, not a default.

GPU usage in ZKML should stay measurement-driven. The same proof backend may shift from CPU-favorable to GPU-favorable or back again as batch size, witness shape, and artifact transfer cost change.

The four boundaries Rust should keep separate

This is where Rust earns its place. The job is mostly about who is allowed to hold what, and the type system is the cheapest enforcement you will ever get. Watch the witness in the diagram: it is born in the prover lane and must never appear on the verifier edge. The service layer takes a request and hands off one owned job; the prover keeps private custody of the witness; the verifier sees only public claims plus a proof reference; and storage decides, deliberately, whether witness files are ephemeral or auditable.



Ownership handoff: the witness never crosses to the verifier or the client side.

What a proof actually promises, and what it does not

It is worth being precise about the guarantee, because the marketing around zero-knowledge tends to oversell it. A passing proof says the chosen computation was followed over the chosen public inputs under the chosen proving assumptions. It says nothing about whether the model is good, and the privacy it offers is only as tight as your public boundary: timing, artifact sizes, and metadata can all leak. Reproducibility belongs in this section too, because a prover and verifier that disagree on any frozen input are simply two different systems that happen to share a name.

Profiling the whole lane, not just the prover binary

When you go to optimize, the prover binary is the obvious suspect and frequently the wrong one. Witness generation, with its canonicalization and commitment building, can be hotter than the proof engine itself, and queue wait can dwarf both under load. Measure each stage separately, give the small verifier lane its own SLOs rather than waving it off as cheap, and give any GPU lane its own telemetry so the accelerator does not become a black box the moment an incident starts.

Turning the pipeline into a product you can operate

Everything converges here. A shippable ZKML service exposes submit, status, and verify surfaces that behave like a job system with receipts, not a synchronous toy endpoint; it bounds its proving queues and separates CPU from GPU lanes; it moves model hash, circuit ID, verifier key, quantization manifest, and tokenizer version together as one record; and it emits the metrics that let you reason about all of the above. The final card is the reality check: full LLM-scale proving is usually too expensive to ship naively, so most production systems prove bounded subgraphs or audit-oriented outputs instead.

How this lands depending on where you came from

The hard part of ZKML is rarely the cryptography itself; it is unlearning the habits your previous language encouraged. Each background brings a different instinct to the table, and each instinct trips on a different part of this pipeline. Find the card that matches your history and notice the specific mental shift it asks for.

Production patterns

These are the habits that separate a demo from a service you can keep running. None of them are exotic; they are the same discipline applied at every boundary the diagrams above traced. Treat each as a default you deviate from only with a reason.

Pitfalls and tradeoffs

Most of these are the mental-model violations made flesh. They are easy to commit because each one looks like a harmless shortcut at the time: inline the prover just this once, treat a tokenizer bump as a refactor, assume the GPU helped. The callout at the end names the single most expensive one.

The most expensive ZKML mistake is often contract drift: one tokenizer change, one quantization tweak, one graph export difference, or one verifier-key mismatch that was never treated as a versioned systems boundary.

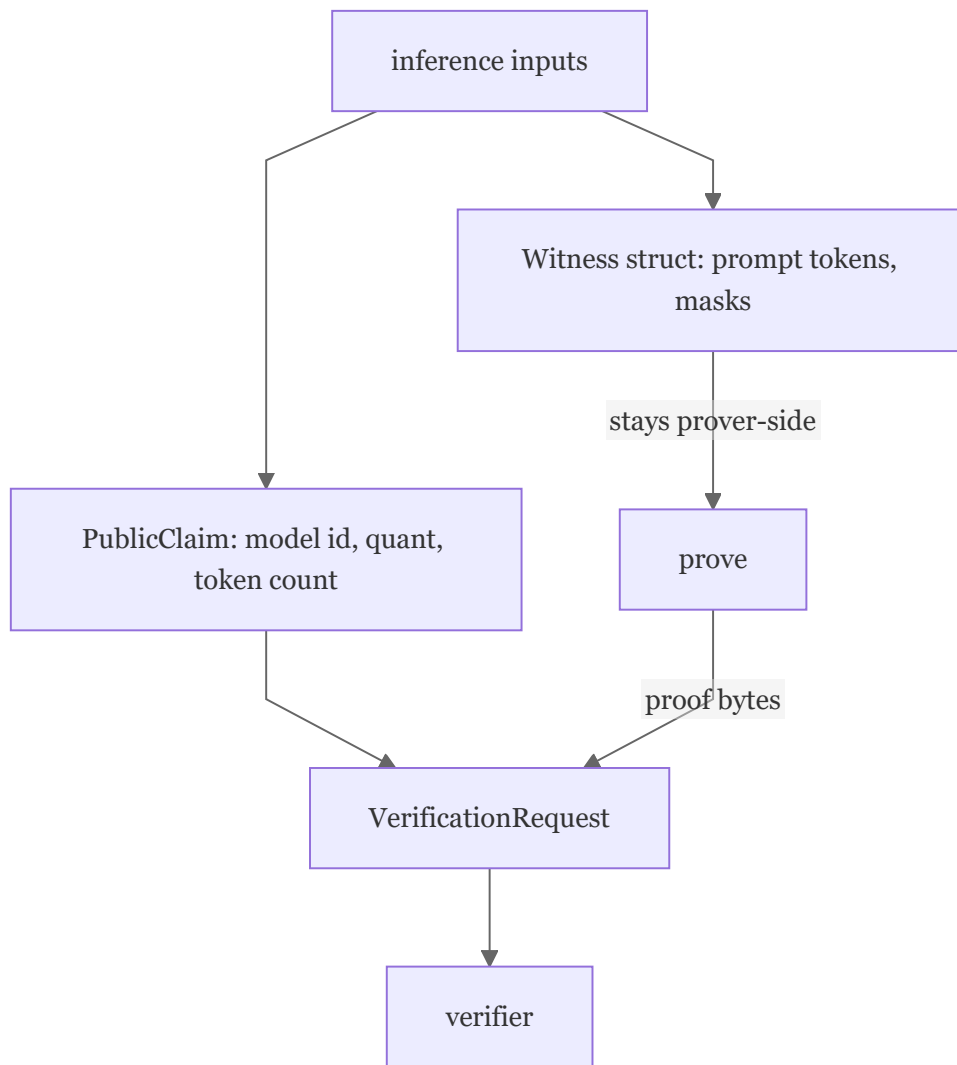
Examples

Two runnable examples make the abstractions concrete. The first shows how to encode the public/private boundary in the type system so a leak becomes a compile error rather than an audit finding. The second shows a proving job routed through a queue and profiled by stage. Read the short orientation and diagram above each listing first, then run it and change a value to see the behavior move.

Example 1: keep public claims, witness material, and verification requests separate

The main lesson is boundary honesty. The verifier-facing request carries the public claim and proof artifact only. Witness material remains proving-side custody.

What to look at: notice that the witness fields and the public-claim fields live in separate structs, and the verification request only ever holds the public type plus a proof reference. The diagram traces which field is allowed to reach the verifier. If you tried to put a prompt token into the verification request, it simply would not type-check.



Two structs in, one verification request out. The witness has no path to the verifier.

Example 53-1. `verification_boundary_types.rs` — *Boundary types*

```

#[derive(Debug, Clone)]
struct PublicClaim {
    model_id: String,
    quantization_bits: u8,
    token_count: usize,
    output_commitment: String,
}

#[derive(Debug, Clone)]
struct WitnessMaterial {
    prompt_tokens: Vec<u32>,
    attention_mask: Vec<u8>,
}

#[derive(Debug, Clone)]
struct ProofArtifact {
    circuit_id: String,
    proof_bytes_len: usize,
    verifier_key_id: String,
}
  
```

```

#[derive(Debug, Clone)]
struct VerificationRequest {
    claim: PublicClaim,
    proof: ProofArtifact,
}

fn build_verification_request(claim: PublicClaim, proof: ProofArtifact) -> VerificationRequest {
    VerificationRequest { claim, proof }
}

fn main() {
    let claim = PublicClaim {
        model_id: String::from("llm-int8:v3"),
        quantization_bits: 8,
        token_count: 128,
        output_commitment: String::from("out:9af1"),
    };

    let _witness = WitnessMaterial {
        prompt_tokens: vec![101_u32, 202, 303],
        attention_mask: vec![1_u8, 1, 1],
    };

    let proof = ProofArtifact {
        circuit_id: String::from("ezkl-circuit:v3"),
        proof_bytes_len: 2_048,
        verifier_key_id: String::from("vk:2026-04"),
    };

    let request = build_verification_request(claim, proof);

    println!("model = {}", request.claim.model_id);
    println!("public tokens = {}", request.claim.token_count);
    println!("proof bytes = {}", request.proof.proof_bytes_len);
    println!("witness kept private = {}", true);
}

```

OUTPUT

```

model = llm-int8:v3
public tokens = 128
proof bytes = 2048
witness kept private = true

```

Model ID, quantization choice, and public token count are verifier-facing contract fields here.

Prompt tokens and masks stay proving-side. If they leak into verification DTOs, the boundary already widened.

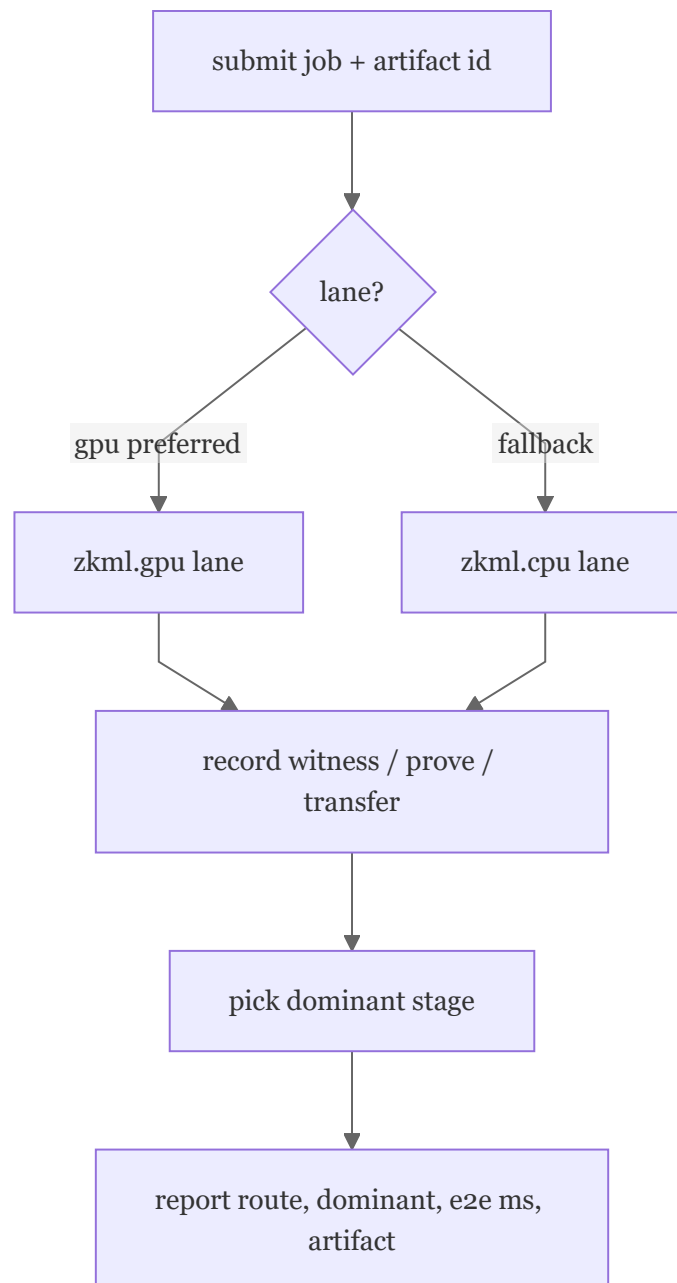
Circuit and verifier key identifiers make mixed deployments and failures attributable.

This is the shape to keep whether the prover is a local binary, a GPU lane, or an external worker service.

Example 2: route heavyweight proving through one queue and profile the slow stage explicitly

The route, dominant stage, and end-to-end timing are all explicit. That is a more production-honest starting point than “call prove() somewhere inside the handler.”

What to look at: the job picks a lane by routing policy, runs through the witness, prove, and transfer stages while recording each one, then reports which stage dominated and the end-to-end total. The diagram is that control flow. The point of the example is that the dominant stage and the artifact ID are first-class outputs, not values you have to reconstruct from logs after an incident.



Routing, per-stage timing, and a dominant-stage verdict are explicit, not hidden inside a handler.

Example 53-2. proving_queue_profile.rs — Queue profile

```

#[derive(Debug, Clone)]
struct ZkmlJob {
    artifact_id: String,
    gpu_requested: bool,
    batch_size: usize,
}

#[derive(Debug, Clone, Copy)]
struct StageTimes {
    witness_ms: u64,
    prove_ms: u64,
    verify_ms: u64,
    transfer_ms: u64,
}

fn queue_name(job: &ZkmlJob) -> &'static str {
    if job.gpu_requested {
        "zkml.gpu"
    } else {
        "zkml.cpu"
    }
}

fn dominant_stage(times: &StageTimes) -> &'static str {
    [
        ("witness", times.witness_ms),
        ("prove", times.prove_ms),
        ("verify", times.verify_ms),
        ("transfer", times.transfer_ms),
    ]
    .into_iter()
    .max_by_key(|(_, ms)| *ms)
    .map(|(name, _)| name)
    .unwrap()
}

fn end_to_end_ms(times: &StageTimes) -> u64 {
    times.witness_ms + times.prove_ms + times.verify_ms + times.transfer_ms
}

fn main() {
    let job = ZkmlJob {
        artifact_id: String::from("model-sha256:abc123"),
        gpu_requested: true,
        batch_size: 4,
    };

    let times = StageTimes {
        witness_ms: 320,
        prove_ms: 1_280,
        verify_ms: 40,
        transfer_ms: 240,
    };
}

```

```
println!("route = {}", queue_name(&job));
println!("dominant = {}", dominant_stage(&times));
println!("e2e ms = {}", end_to_end_ms(&times));
println!("artifact = {}", job.artifact_id);
}
```

OUTPUT

```
route = zkml.gpu
dominant = prove
e2e ms = 1880
artifact = model-sha256:abc123
```

GPU preference becomes one routing policy, not one invisible local branch inside business logic.

Prove time is the largest contributor here, but witness and transfer are still material and measurable.

The job carries one artifact ID so proof results can be tied back to one exact model snapshot.

Large LLM proofs often need narrower subgraphs or different product promises than small analytics proofs do.

The repository also includes standalone Rust source under `examples/ch53_ezkl_verifiable_llm_inference_gpu_zkml/` so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

ONNX Model Inference in Rust

Run portable ONNX models from Rust: the ort runtime versus pure-Rust tract, tensors and shapes, execution providers, batching, and dependency-light deployment

A trained model is a frozen graph of arithmetic. This chapter is about running that graph from Rust: loading an ONNX file into a session, feeding it correctly shaped tensors, reading the result, and choosing between the native-runtime and pure-Rust paths. The math is the easy part; the contract at the edges is where the work is.

Opening scenario

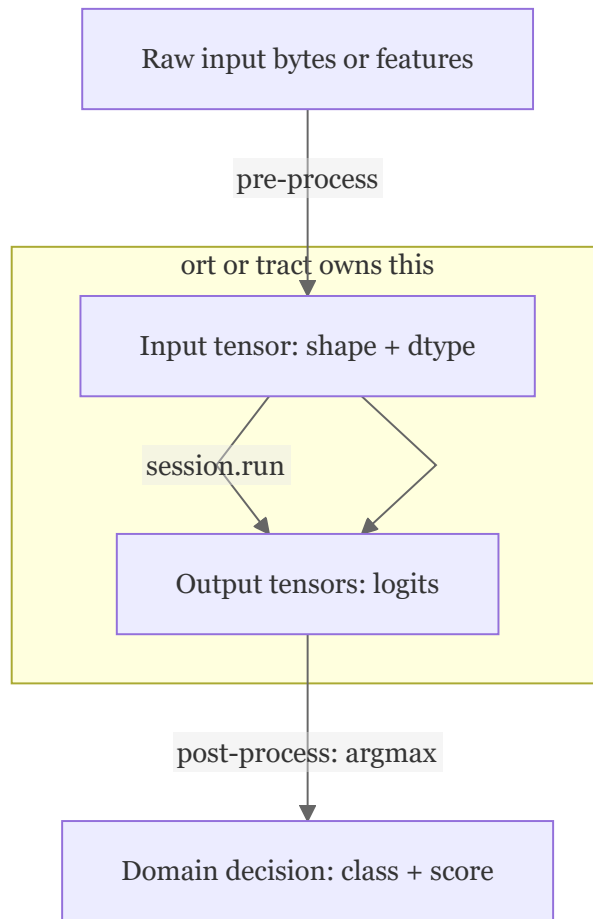
A team has trained a classifier in Python and wants to serve it inside an existing Rust service: no separate Python process, no model server to operate, just one binary that loads the model at startup and answers requests. The model has been exported to ONNX. The job now is to load it once, turn each request into a tensor of the exact shape the graph expects, run a forward pass, and turn the output back into a labeled decision.

A practical decision order

1. Choose the runtime first: ort for coverage and GPU, tract for a clean pure-Rust build.
2. Build the session once at startup and keep it alive for the life of the process.
3. Pin the input contract: exact names, shapes, and dtypes, checked rather than assumed.
4. Keep pre- and post-processing in tested functions that move owned data across the boundary.

Mental model

The shape worth memorizing is a three-stage pipeline with the session in the middle. Raw input becomes a tensor, the session turns that tensor into output tensors, and post-processing turns those into a domain decision. The runtime owns only the middle box; the two ends are ordinary Rust code that you write, own, and test.

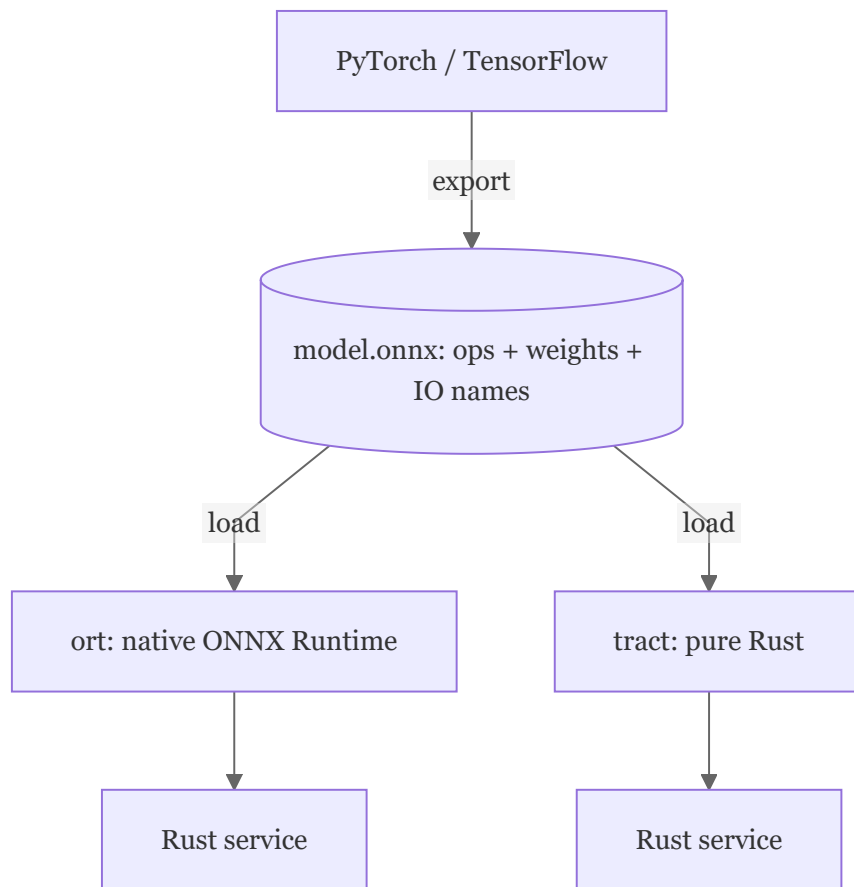


The runtime owns only the forward pass. Pre-processing into the input contract and post-processing into a decision are code you own.

Core concepts

ONNX as a portable graph format

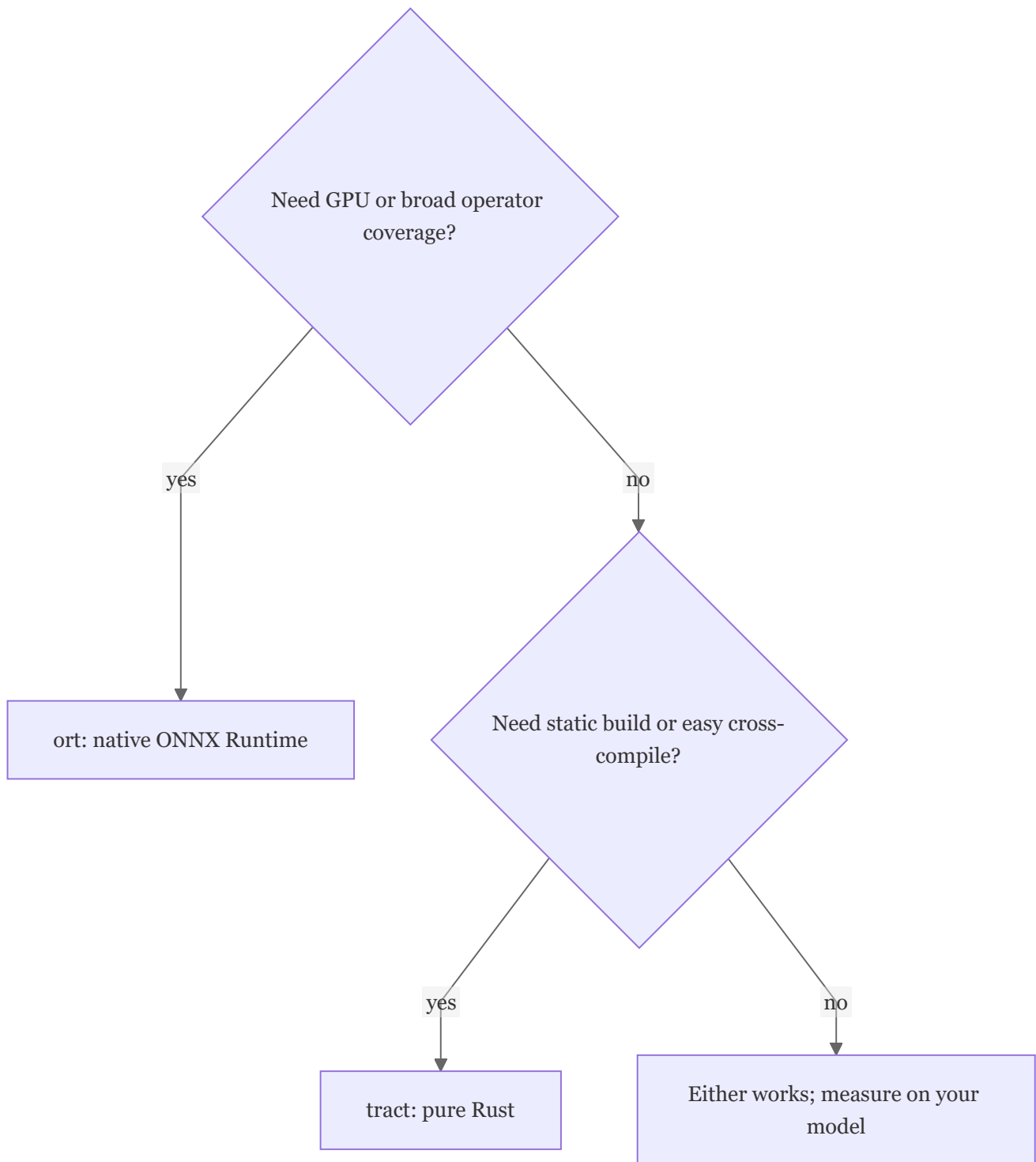
The file is the contract. A producer trains in any framework and exports to ONNX; a consumer loads that one file into whichever runtime fits its deployment. Because operators and tensor declarations are standardized, the same graph runs unchanged across runtimes, which is what lets training and serving live in different languages without a fragile re-export step in between.



One exported graph, many runtimes. The .onnx file is the stable handoff between training and serving.

The ort versus tract tradeoff

The decision is rarely about raw speed on one model. It is about what your build and deploy story can accept: a native runtime dependency and GPU access, or a single pure-Rust binary that cross-compile cleanly. Decide it early, because it shapes everything downstream from image size to which operators your model is allowed to use.



A starting heuristic. GPU and operator breadth pull toward ort; clean static builds pull toward tract.

Sessions: build once, run many

Loading a model is the expensive step. Building a session parses the file, optimizes the graph, selects execution providers, and allocates working memory. Running the session afterward is comparatively cheap. The operational consequence is the most important habit in this chapter: construct the session at startup, store it as a long-lived shared resource, and run it per request. The real ort call below is what that startup step looks like; the runnable listing further down stands in for the per-request run.

Input and output tensors with ndarray

A tensor is a flat buffer plus a shape and a dtype. In Rust the usual way to build one is with the ndarray crate, which plays the role NumPy plays in Python: it owns the buffer and tracks the shape so the runtime can interpret the bytes correctly. The graph names each input and output and fixes its rank, so the data you build must match [batch, features] exactly, including element type. A buffer that is the right length but the wrong shape is a different tensor as far as the model is concerned.

The most common inference bug is not a wrong model. It is a tensor whose shape or dtype does not match the graph, which either errors at runtime or, worse, runs and returns confident nonsense.

Execution providers: CPU, CUDA, TensorRT

An execution provider decides where the operators actually run. With ort you can register providers in priority order, and the runtime assigns each operator to the first provider that supports it, falling back to CPU for the rest. This is a deployment and latency tradeoff, not a free speedup: each step up the ladder buys throughput at the cost of heavier dependencies and tighter hardware assumptions.

Pre- and post-processing ownership

Tensors borrow their backing buffers, so the borrow checker is doing real work here: a tensor that views a Vec must not outlive that buffer. The calm pattern is to build owned inputs, hand the session a tensor that borrows them for the run, copy or map the outputs into owned domain values, and let the scratch buffers drop at the end of the scope.

Deployment: binary size and no Python runtime

How this lands by background

Almost everyone arrives at ONNX inference with a runtime they already know from another ecosystem. The useful thing is to see which part of that knowledge transfers cleanly and which part Rust changes. The session and named-tensor shape is shared across all of these; what shifts is who owns tensor lifetimes and how the model gets deployed.

Production patterns

Pitfalls and tradeoffs

The runtime makes the math correct. It does nothing to make your shapes, dtypes, and label maps correct. Those stay your responsibility, and they are where inference quietly goes wrong.

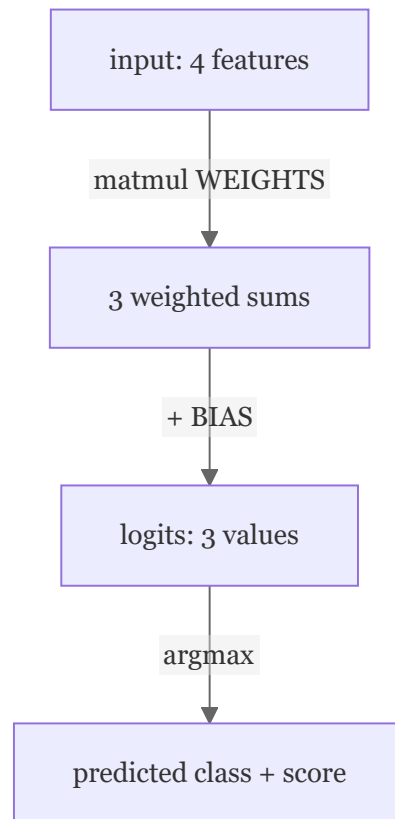
Examples

The in-browser runner cannot link ONNX Runtime, so both listings are deterministic pure-Rust forward passes over one dense layer: a matmul against a fixed weight matrix, a bias add, then argmax. That is exactly the arithmetic a one-layer ONNX model performs, so the shapes, the logits, and the predicted class are the same numbers a real session would return for these weights. Read each one as the body of the per-request run that would otherwise call session.run.

Example 1: load-and-run a one-layer classifier

One input vector flows through a dense layer and an argmax, mirroring a forward pass through a one-layer ONNX graph and printing the predicted class and its score.

What to look at: forward is the matmul plus bias add that a single Gemm operator would run inside the graph, and argmax is the post-processing you own. Trace one input through the diagram below, then read the same path in code: four input features, three output logits, and the largest logit chosen as the class.



A four-feature input becomes three logits via matmul and bias, and argmax picks the winning class. This is one dense layer of an ONNX graph.

Example 54-1. onnx_session_load_run.rs — Load run

```

// A real service would load an .onnx graph and run it through a session:
//
// use ort::{Session, inputs};
// let session = Session::builder()?
//     .with_optimization_level(GraphOptimizationLevel::Level3)?
//     .commit_from_file("classifier.onnx")?;
// let outputs = session.run(inputs!["input" => input_tensor])?;
//
// The in-browser runner cannot call into ONNX Runtime, so this listing is a
// deterministic pure-Rust forward pass that mirrors what one dense layer of
// that graph does: a matmul against a weight matrix, a bias add, then argmax.
// The numbers, shapes, and the "predicted class" are exactly what a one-layer
// ONNX model with the same weights would produce.

const IN: usize = 4;
const OUT: usize = 3;

```



```

// Weights stored row-major: one row of IN values per output class.
const WEIGHTS: [[f32; IN]; OUT] = [
    [0.2, 0.8, -0.5, 0.1],
    [-0.3, 0.5, 0.9, 0.4],
    [0.6, -0.2, 0.3, -0.7],
];

const BIAS: [f32; OUT] = [0.1, -0.2, 0.05];

// One fixed input "tensor" of shape [IN]. In ort/trtact this would be an
// ndarray Array passed in as the named input of the session.
fn input_tensor() -> [f32; IN] {
    [0.5, -1.0, 2.0, 0.25]
}

// Dense layer: logits[o] = bias[o] + sum_c weights[o][c] * input[c].
fn forward(input: &[f32; IN]) -> [f32; OUT] {
    let mut logits = [0.0f32; OUT];
    for o in 0..OUT {
        let mut sum = BIAS[o];
        for c in 0..IN {
            sum += WEIGHTS[o][c] * input[c];
        }
        logits[o] = sum;
    }
    logits
}

// Post-processing the caller owns: turn raw logits into a class index.
fn argmax(logits: &[f32; OUT]) -> usize {
    let mut best = 0;
    for i in 1..OUT {
        if logits[i] > logits[best] {
            best = i;
        }
    }
    best
}

fn main() {
    let input = input_tensor();
    let logits = forward(&input);
    let class = argmax(&logits);

    println!("input dims = {}", IN);
    println!("classes = {}", OUT);
    println!(
        "logits = [{:.4}, {:.4}, {:.4}]",
        logits[0], logits[1], logits[2]
    );
    println!("predicted class = {}", class);
    println!("score = {:.4}", logits[class]);
}

```

OUTPUT

```

input dims = 4
classes = 3

```

```
logits = [-1.5750, 1.0500, 0.9750]
predicted class = 1
score = 1.0500
```

A fixed four-element vector stands in for a named graph input of shape [4].

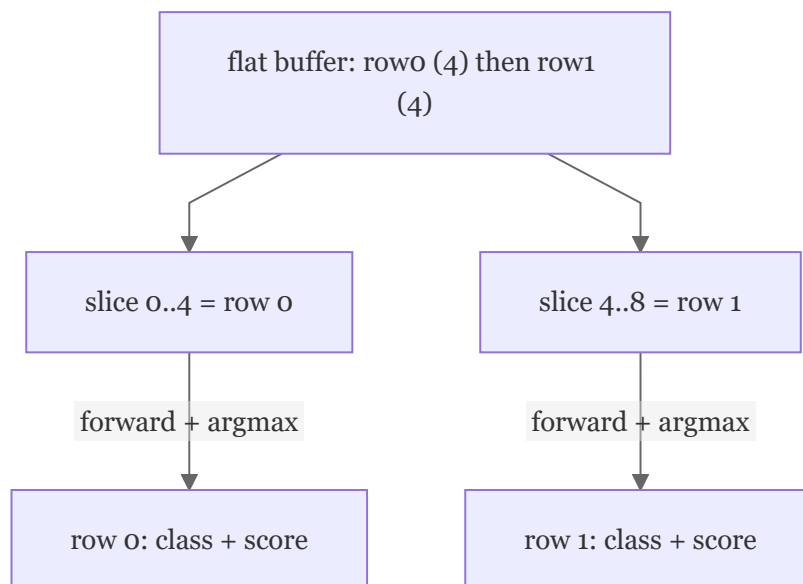
Matmul plus bias is the arithmetic the runtime would run for one dense layer.

Argmax turns raw logits into a class, which is code you own, not the model.

Example 2: batched tensor IO with per-row argmax

A two-row batch is packed into one row-major flat buffer, the same dense layer runs over each row, and the per-row argmax is printed, mirroring an input tensor of shape [2, 4].

What to look at: the batch is one flat Vec laid out row-major, exactly how a runtime stores a [rows, features] tensor in memory. The loop slices one row at a time with $r * \text{IN}..(r + 1) * \text{IN}$ and runs the same layer per row, which is why one session run over a batch beats running the model once per row. The diagram shows the layout; the output is one line per row.



A row-major flat buffer holds the whole batch; each row is sliced out and run through the same layer to its own argmax.

Example 54-2. onnx_tensor_batch_argmax.rs — Batch argmax

```
// Batched inference: the same one-layer model run over more than one row at
// once. A real session takes an input tensor of shape [rows, features] and
// returns logits of shape [rows, classes]; here we keep the batch as a single
// row-major flat Vec and slice one row at a time, with no external crates.

const IN: usize = 4;
const OUT: usize = 3;

const WEIGHTS: [[f32; IN]; OUT] = [
    [0.2, 0.8, -0.5, 0.1],
    [-0.3, 0.5, 0.9, 0.4],
    [0.6, -0.2, 0.3, -0.7],
];
```

```

const BIAS: [f32; OUT] = [0.1, -0.2, 0.05];

// Two input rows packed row-major into one flat buffer: [row0.., row1..].
fn build_batch() -> Vec<f32> {
    vec![
        0.5, -1.0, 2.0, 0.25, // row 0
        1.5, 0.5, -0.5, 1.0, // row 1
    ]
}

// Run the dense layer over one feature row and return its logits.
fn forward_row(row: &[f32]) -> [f32; OUT] {
    let mut logits = [0.0f32; OUT];
    for o in 0..OUT {
        let mut sum = BIAS[o];
        for c in 0..IN {
            sum += WEIGHTS[o][c] * row[c];
        }
        logits[o] = sum;
    }
    logits
}

fn argmax(logits: &[f32; OUT]) -> usize {
    let mut best = 0;
    for i in 1..OUT {
        if logits[i] > logits[best] {
            best = i;
        }
    }
    best
}

fn main() {
    let batch = build_batch();
    let rows = batch.len() / IN;

    println!("batch rows = {}", rows);
    println!("features per row = {}", IN);

    for r in 0..rows {
        let row = &batch[r * IN..(r + 1) * IN];
        let logits = forward_row(row);
        let class = argmax(&logits);
        println!("row {} argmax = {} score = {:.4}", r, class, logits[class]);
    }
}

```

OUTPUT

```

batch rows = 2
features per row = 4
row 0 argmax = 1 score = 1.0500
row 1 argmax = 0 score = 1.1500

```

The whole batch is one contiguous buffer, the layout a real tensor uses for [2, 4].

Slicing by stride recovers each row without copying the underlying buffer.

One run over N rows amortizes session overhead better than N single-row runs.

Next chapter

Inference puts a deterministic, self-contained binary at the edge of a service. Chapter 55 takes determinism in a different direction: smart contracts on Solana and Sei, where Rust runs in a constrained on-chain environment and every byte of state and compute is paid for and verified. The same instincts, explicit ownership and a tight binary, carry straight over.

Summary

Smart Contracts in Rust: Solana, Sei, and Beyond

Write on-chain Rust across execution models: Solana's account and program split, Sei and CosmWasm wasm actors, and how both compare with EVM, NEAR, and ink!

A smart contract is just sandboxed, deterministic, gas-metered code over shared state. This chapter looks at how Solana, Sei (through CosmWasm), and the EVM answer the same handful of questions differently, and why the Rust you write to satisfy each one is far more alike than the platforms make it appear.

Builds on Chapter 19

Chapter 19 covered serialization and versioned data contracts, which is exactly what a contract's boundary format is. That lesson is load-bearing here, because a chain's boundary format is the contract's real public API. Chapter 56, the finale, takes the sandboxed, no-ambient-I/O discipline a contract runs under all the way down to `no_std` and constrained runtimes.

Opening scenario

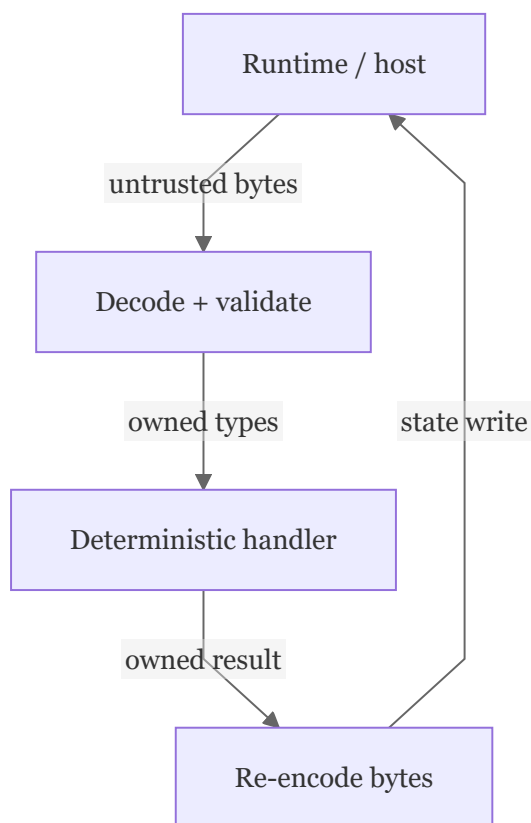
A team that already ships a Solana program is asked to deploy comparable logic to a Sei (CosmWasm) chain, and to keep an eye on an EVM deployment a partner maintains. The question is not "rewrite everything" but "what actually changes." The answer turns out to be small and specific: the boundary format and the entry-point shape change; the owned-domain core does not.

The four questions to ask of any chain

1. Where does contract state live, with the code or in separate accounts?
2. What does the code execute as, SBF, wasm, or EVM bytecode?
3. How is data serialized across the host boundary, Borsh, JSON, or ABI?
4. What shape is the entry point the runtime calls?

Mental model

The shape worth memorizing is a thin shell around a pure core. Untrusted bytes enter, get decoded into owned domain types, a deterministic handler runs, and owned results are re-encoded into bytes the runtime persists. Everything chain-specific lives in the shell; the handler is ordinary Rust.



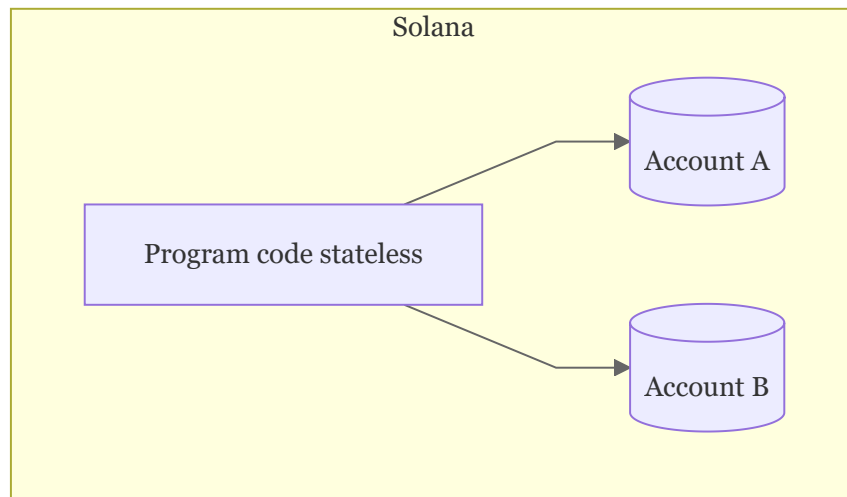
Every chain shares this shape: decode untrusted bytes into owned types, run a deterministic handler, encode results back. Only the edges change per platform.

How the platforms compare

This is a cross-platform comparison, not a cross-language one: the user asked specifically about Solana and Sei, with the EVM included for contrast. Read the first three cards as three answers to the same four questions, then read the fourth card as the Rust ideas that stay constant no matter which chain you land on.

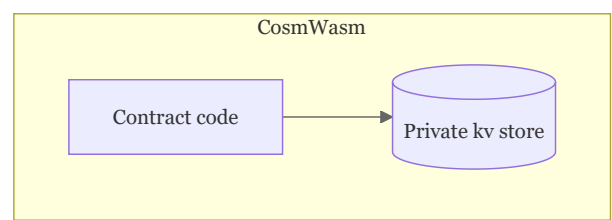
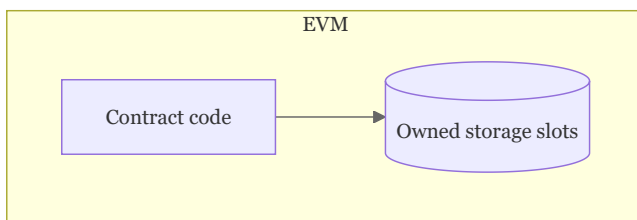
The four answers, side by side

The diagram below makes the structural difference concrete: Solana keeps code and state apart, while CosmWasm and the EVM bind each contract to its own storage.



Solana keeps stateless code apart from the external accounts the runtime passes in; one program can act on many accounts.

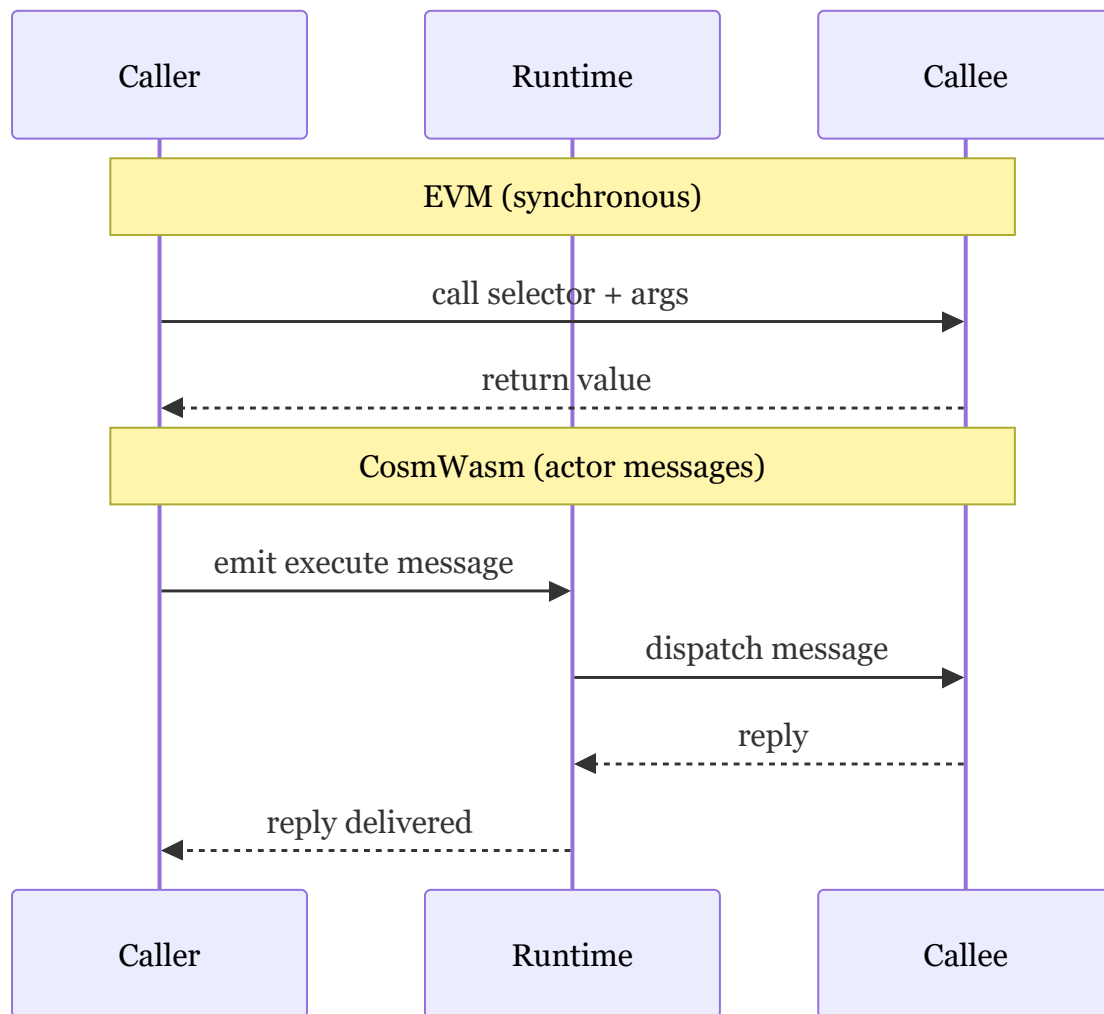
CosmWasm and the EVM take the opposite approach: each contract is bound to its own private storage rather than reaching into external accounts.



CosmWasm and the EVM each bundle a contract with its own private storage, so code and state share one address.

Differences and similarities, made explicit

How a contract composes with another contract is where the platforms diverge most sharply, and it is worth tracing once. The EVM call is synchronous and in-line; CosmWasm hands a message to the runtime and may receive a reply; Solana invokes another program against accounts it explicitly passes along.



The EVM composes by synchronous in-line calls; CosmWasm composes by emitting messages the runtime dispatches and replies to. Reentrancy and atomicity differ as a result.

Core concepts

Ownership at the (de)serialization boundary

The single most important habit in contract code is the same one Chapter 19 taught for any wire format, with a hostile counterparty assumed: never let an undecoded or unvalidated value reach state. Borrow the input, parse it into owned types, validate, and only then mutate.

Determinism and metered execution

A contract runs inside a sandbox with no ambient I/O, every operation costs a metered resource, and every validator re-executes it expecting identical results. These three facts rule out a surprising amount of ordinary Rust, and the discipline they demand is close to the embedded and `no_std` work Chapter 56 takes up next.

The broader Rust-on-chain landscape

Solana, Sei, and the EVM are the focus here, but Rust reaches several other chains, each varying the edge format while keeping the same core habits. Three are worth knowing by name.

Production patterns

Pitfalls and tradeoffs

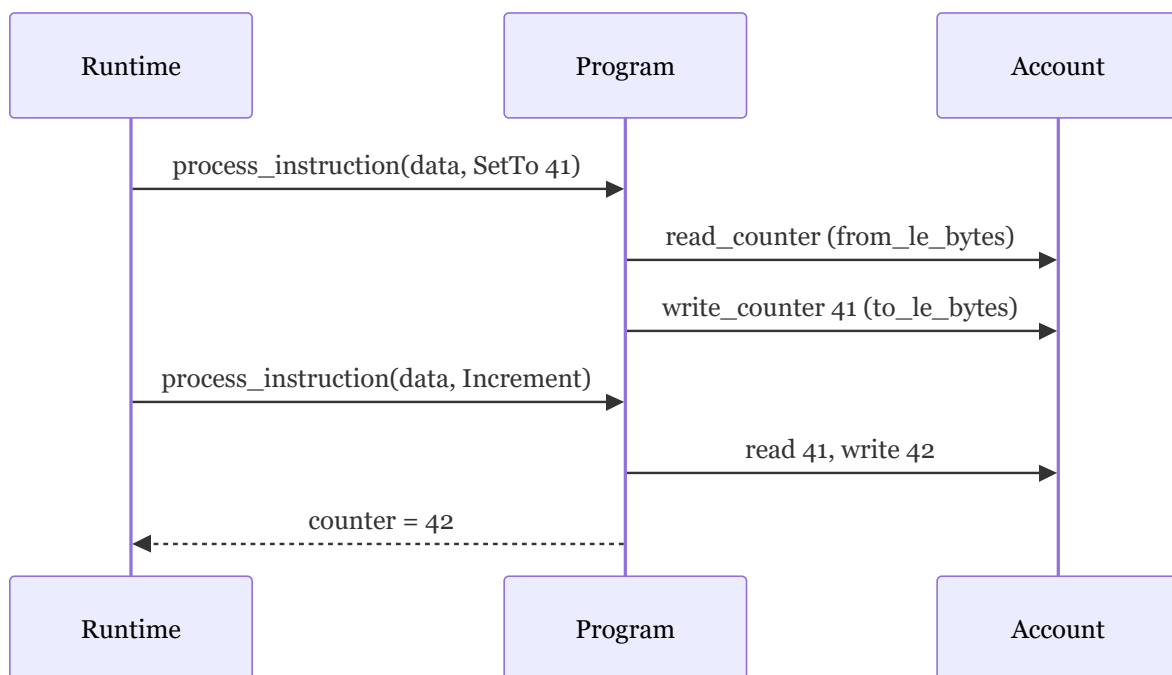
The chain will not protect you from a contract that trusts its input. Validation, determinism, and a versioned boundary format are your job, not the runtime's.

Examples

Example 1: Solana's account and instruction model

A self-contained, crate-free model of how a Solana program works on external account data: stateless code, a mutable byte buffer, a hand-rolled Borsh-style counter, and an instruction that drives it.

What to look at: the program holds no state. The Account owns the data buffer, and `process_instruction` borrows it mutably, decodes a little-endian u64 by hand the way Borsh would, applies the instruction, and writes the bytes back. Trace one call through the diagram, then read the same path in code; try changing the `SetTo` value and watch the final counter follow.



The runtime hands the program a mutable account buffer per instruction. The program decodes, applies, and re-encodes; the runtime persists the bytes.

Example 55-1. `solana_counter_program.rs` — Solana counter

```
// A hand-built model of Solana's account + instruction model, no crates.
//
// On Solana a "program" is stateless code. The state it works on lives in
// separate "accounts" that the runtime passes in as mutable byte buffers.
// An instruction tells the program which operation to run and with what data.
//
// Here we model exactly that: an Account owns a data buffer and an owner
```

```

// field, process_instruction borrows the buffer mutably, decodes a u64
// counter by hand (the way Borsh would, little-endian), applies the op, and
// writes the counter back. main() drives a couple of instructions.

#[derive(Debug)]
struct Account {
    owner: &'static str,
    data: [u8; 8],
}

#[derive(Debug)]
enum Instruction {
    Increment,
    SetTo(u64),
}

#[derive(Debug)]
enum ProgramError {
    DataTooSmall,
}

// Decode the counter the way a by-hand Borsh reader would: a little-endian u64.
fn read_counter(data: &[u8]) -> Result<u64, ProgramError> {
    if data.len() < 8 {
        return Err(ProgramError::DataTooSmall);
    }
    let mut bytes = [0u8; 8];
    bytes.copy_from_slice(&data[..8]);
    Ok(u64::from_le_bytes(bytes))
}

// Re-encode the counter back into the account buffer, little-endian.
fn write_counter(data: &mut [u8], value: u64) -> Result<(), ProgramError> {
    if data.len() < 8 {
        return Err(ProgramError::DataTooSmall);
    }
    data[..8].copy_from_slice(&value.to_le_bytes());
    Ok(())
}

// The "program entry point": stateless code over a mutable account buffer.
fn process_instruction(data: &mut [u8], instruction: &Instruction) -> Result<u64, ProgramError> {
    let current = read_counter(data)?;
    let next = match instruction {
        Instruction::Increment => current + 1,
        Instruction::SetTo(value) => *value,
    };
    write_counter(data, next)?;
    Ok(next)
}

fn main() {
    let mut account = Account {
        owner: "CounterProgram1111",
        data: [0u8; 8],
    };

```

```
// The runtime hands the program a mutable view of account data each call.
let after_set = process_instruction(&mut account.data, &Instruction::SetTo(41)).unwrap();
let after_inc = process_instruction(&mut account.data, &Instruction::Increment).unwrap();

let final_counter = read_counter(&account.data).unwrap();

println!("after set = {}", after_set);
println!("after increment = {}", after_inc);
println!("counter = {}", final_counter);
println!("owner = {}", account.owner);
}
```

OUTPUT

```
after set = 41
after increment = 42
counter = 42
owner = CounterProgram1111
```

The program owns no storage; state lives in the account buffer passed in.

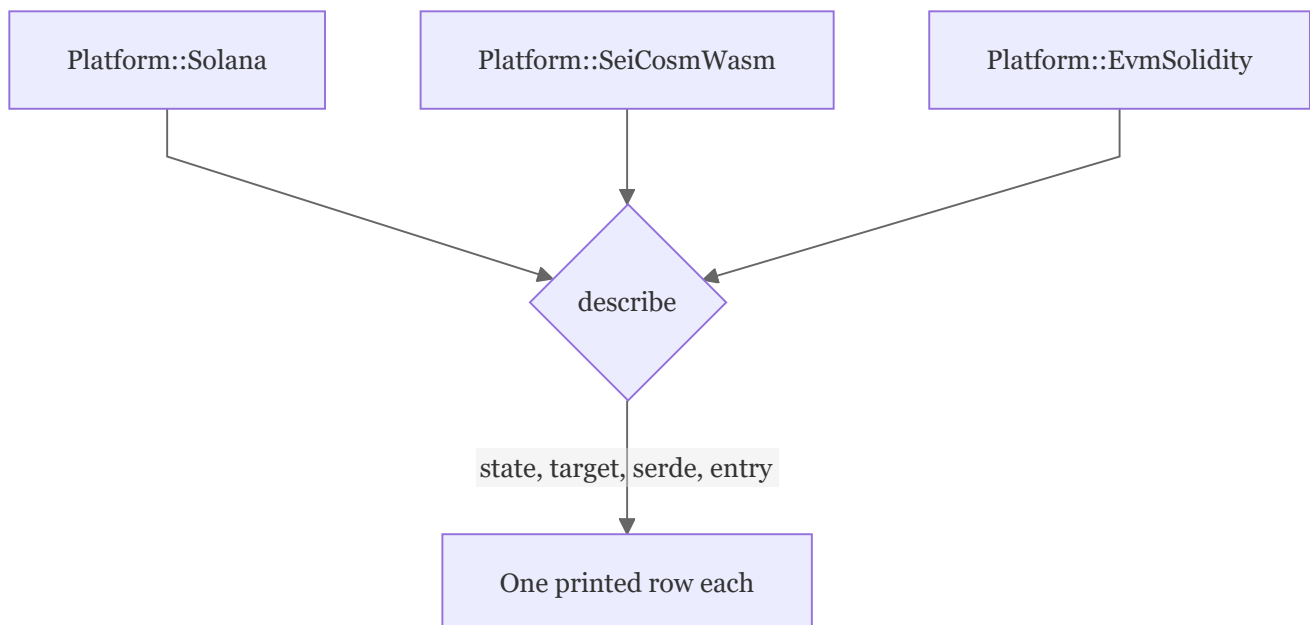
from_le_bytes and to_le_bytes stand in for a real codec.

Each account records its owning program, the field real validation checks first.

Example 2: the platform mental model as data

The Solana / Sei (CosmWasm) / EVM comparison encoded as an enum with four fields per variant, printed as one row per platform so the contrasts line up.

What to look at: each platform is an enum variant, and describe maps it to the four facts, state model, execution target, serialization, and entry shape. The loop prints one line per platform. This is the comparison table from earlier turned into runnable data; editing a field in one arm changes exactly one line of output.



Each variant carries the same four facts; describe maps a platform to its row, and main prints one line per platform.

Example 55-2. platform_compare.rs — Platform compare

```
// The Solana vs Sei(CosmWasm) vs EVM mental model, encoded as data.
//
// Every on-chain platform answers the same four questions differently:
// 1. Where does contract state live?
// 2. What does the code actually run as (the execution target)?
// 3. How is data serialized across the host boundary?
// 4. What shape is the entry point the runtime calls?
//
// We model each platform as an enum variant, attach those four facts, and
// print one line per platform so the contrasts sit side by side.

#[derive(Debug, Clone, Copy)]
enum Platform {
    Solana,
    SeiCosmWasm,
    EvmSolidity,
}

struct PlatformModel {
    name: &'static str,
    state_model: &'static str,
    execution_target: &'static str,
    serialization: &'static str,
    entry_shape: &'static str,
}

fn describe(platform: Platform) -> PlatformModel {
    match platform {
        Platform::Solana => PlatformModel {
            name: "Solana",
            state_model: "external accounts",
            execution_target: "SBF",
            serialization: "Borsh",
            entry_shape: "process_instruction",
        },
        Platform::SeiCosmWasm => PlatformModel {
            name: "Sei/CosmWasm",
            state_model: "contract-owned kv",
            execution_target: "wasm",
            serialization: "JSON/serde",
            entry_shape: "instantiate/execute/query",
        },
        Platform::EvmSolidity => PlatformModel {
            name: "EVM/Solidity",
            state_model: "contract storage",
            execution_target: "EVM bytecode",
            serialization: "ABI",
            entry_shape: "selector dispatch",
        },
    }
}

fn main() {
    let platforms = [Platform::Solana, Platform::SeiCosmWasm, Platform::EvmSolidity];
}
```

```

for platform in platforms {
    let model = describe(platform);
    println!(
        "{} | state={} | target={} | serde={} | entry={}",
        model.name,
        model.state_model,
        model.execution_target,
        model.serialization,
        model.entry_shape
    );
}

```

OUTPUT

```

Solana | state=external accounts | target=SBF | serde=Borsh | entry=process_instruction
Sei/CosmWasm | state=contract-owned kv | target=wasm | serde=JSON/serde | entry=instantiate/execute/query
EVM/Solidity | state=contract storage | target=EVM bytecode | serde=ABI | entry=selector dispatch

```

External accounts on Solana versus contract-owned storage on Sei and the EVM.

SBF, wasm, and EVM bytecode are three sandboxes with the same determinism demand.

Borsh, JSON, and the ABI are each a versioned public contract, not an internal detail.

Next chapter

A contract runs in a sandbox with no ambient I/O and a metered budget, which is the same world the book's finale lives in. Chapter 56 carries that constrained-runtime thread all the way down into `no_std` and embedded targets, where you give up the standard library and make every runtime service explicit. If you want to go deeper on the cryptographic side of on-chain systems instead, Chapter 51 on zero-knowledge proofs is the natural companion: proving facts about state without revealing it is increasingly part of how contracts scale and stay private.

Summary

no-std Rust and Constrained Runtime Derivatives

Design portable Rust surfaces across core, alloc, and std: fixed-capacity memory, DMA/MMIO ownership, embedded layering, and host-testable no_std crates

Constrained targets need explicit runtime surfaces for allocation, panic handling, IO, startup, and host calls. This chapter covers no_std Rust as a portability and assurance strategy.

Builds on Chapters 03, 08, 28, 29, and 44

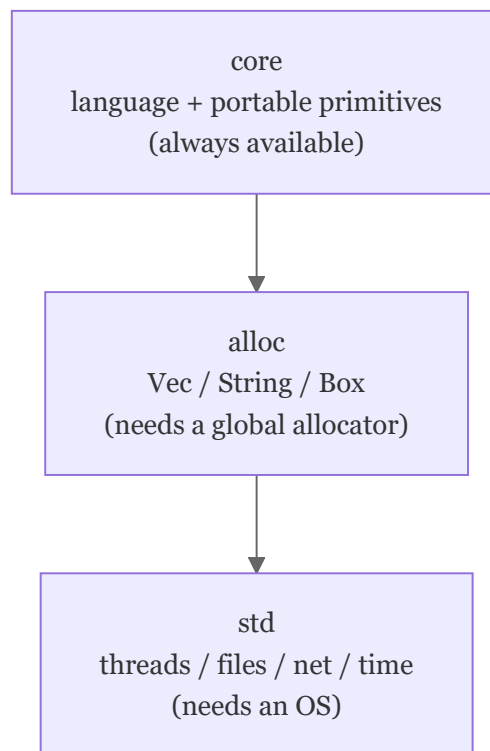
Chapter 03 covered feature flags and dependency hygiene. Chapter 08 established how to audit small unsafe surfaces. Chapters 28 and 29 covered C/C++ and wasm host boundaries. Chapter 44 covered cross-target build and release engineering. This chapter recombines those ideas for `no_std` and constrained-runtime Rust.

Opening scenario

Imagine one team that owns a packet-framing and checksum library and is asked to ship it in four very different homes: the firmware on a sensor board with no operating system, the kernel-adjacent startup code of a small hypervisor, a sandboxed wasm guest that talks to a host over a narrow ABI, and an ordinary Linux service that logs to stdout. The same parsing logic has to run unchanged in all four. The only thing that may differ per home is which runtime services exist underneath it.

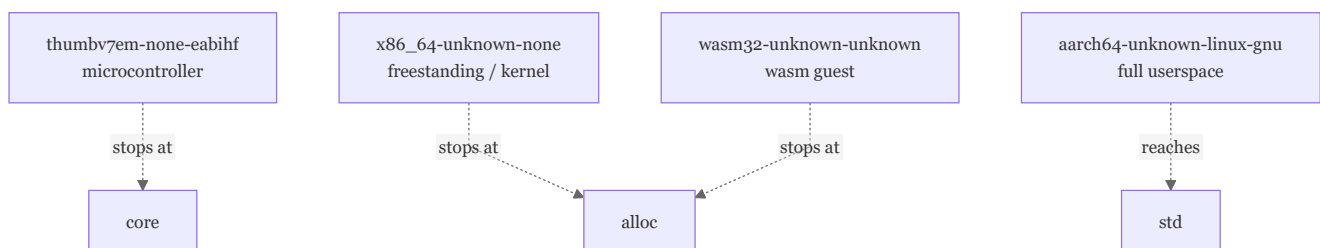
That is the whole job of this chapter: decide, per layer, the smallest runtime surface that is actually needed. Pure logic that needs nothing but the language stays in core. Helpers that own a growable buffer get gated behind alloc. Anything that touches the OS gets gated behind std. The target triple you compile for then decides which of those three surfaces is even allowed.

The diagram below is the picture to hold in your head for the rest of the chapter. Read it as three concentric capability rings (core is always there, alloc needs a heap, std needs an OS) and four target triples that each stop at a different ring.



The three capability rings: core is always available, alloc needs a heap, std needs an OS.

Each target triple stops at a different ring. Here is how the four homes from the scenario map onto the stack above:



The target triple decides how far up the stack you may reach: firmware stops at core, kernel and wasm reach alloc, a Linux userspace reaches std.

Mental model

Three ideas do most of the work in this chapter, and getting them straight up front prevents nearly all of the confusion newcomers have with `no_std`. The first is that you are not learning a different language. The second is that `no_std` and `no_heap` are separate decisions. The third is that the target triple, not your preference, sets the ceiling on what you may use.

Core concepts

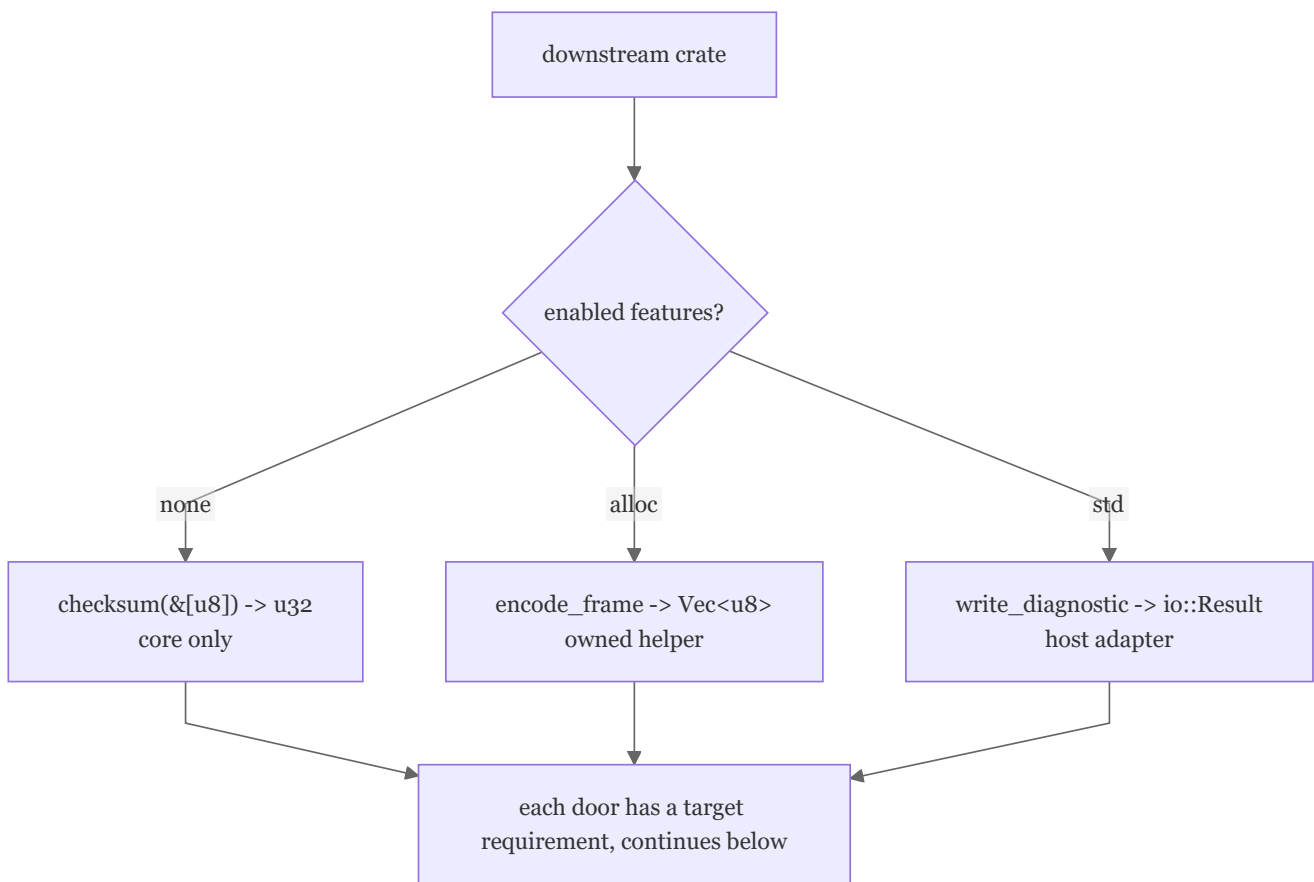
With the three-ring picture in place, the rest of the chapter walks the concrete decisions a senior engineer makes: how to lay out a crate so each function lives at the right ring, when `no_std` is the right call at all, how to manage memory without a heap, how ownership changes when DMA and registers enter the picture, how

the embedded ecosystem is layered, what happens at FFI and wasm boundaries, and how to test code that may never run on your laptop. Each subsection pairs the reasoning with a small, runnable shape.

The three surfaces in practice: core, alloc, std, and what the target triple decides

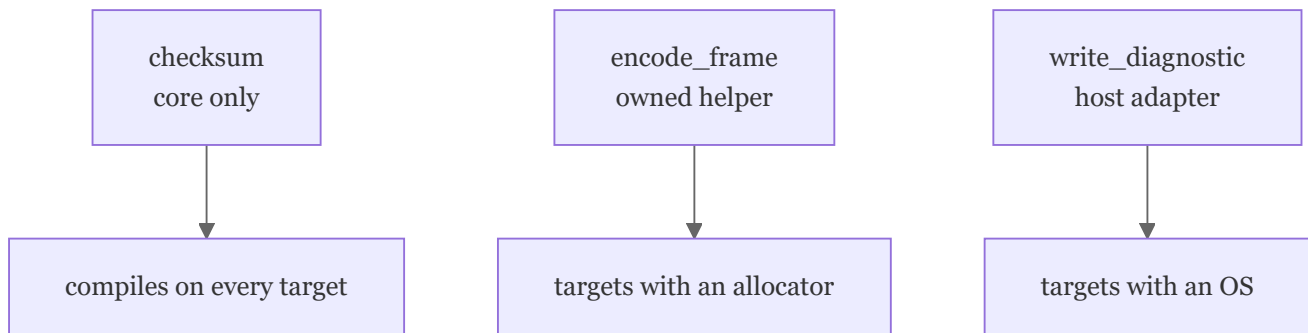
The standard library you reach for on a server is really three layers stacked on top of each other, and `no_std` lets you opt out of the upper ones. Knowing exactly what lives in each layer is what lets you place a function at the lowest ring it can tolerate. The cards below name the contents of each surface; the code that follows shows how one crate routes its public API across all three at once.

Look at the listing below as three doors, one per surface. The crate is `no_std` unless the caller turns on the `std` feature. `checksum` takes only a slice and so compiles everywhere. `encode_frame` returns an owned `Vec<u8>` and is therefore behind the `alloc` door. `write_diagnostic` touches the OS and lives behind the `std` door. A firmware build enables nothing extra and sees only the first function; a server build enables `std` and sees all three. The diagram traces those three paths before you read the code.



One crate, three feature-gated doors. The smallest contract (`checksum`) is the only one every target must support.

Each of those three doors carries a different target requirement:



The core-only door reaches every target; the alloc door needs a heap; the std door needs an OS.

In a hosted program `std` supplies a panic handler for free: it prints a message and unwinds or aborts. In a freestanding binary there is no `std` to do that, so the compiler requires you to provide a `#[panic_handler]` yourself. The one below simply spins forever, which is a deliberate, defined failure state for a microcontroller.

The lesson is that panic strategy, startup glue, and linker scripts are not background details on these targets; they are first-class items you own or import explicitly. Some boards rely on a platform crate to provide that layer, others want it hand-written, and there is no single universal boot story to assume.

When to reach for `no_std` (and when not to)

It is easy to over-apply `no_std` out of a sense of minimalism. The honest test is whether the code must run somewhere that does not have, or does not want, the standard library. There are four recurring situations where that is genuinely the case. Outside them, a normal `std` crate is usually the better engineering choice; `no_std` has real costs in ergonomics and ecosystem reach, and you should pay them on purpose.

The correction worth internalizing: `no_std` is not a synonym for "microcontroller only." It is a way to make runtime assumptions explicit enough that one body of logic can survive several constrained hosts honestly. If your code will only ever run on a normal OS, do not go `no_std` just to feel lean; you will trade away large parts of the ecosystem for a portability you are not using.

Designing a crate that stays portable

Portability is not something you bolt on later; it is a property of how the crate is shaped from the first commit. The pattern is the same one the crate-root listing showed: keep the baseline API in core, layer owned conveniences behind an `alloc` feature, and reserve `std` for the outermost integration. The four practices below are what keep that layering from quietly eroding as the crate grows and acquires dependencies.

The senior rule of thumb is to design the smallest truthful API first. If borrowed slices and fixed-capacity callers are enough, keep the surface in core and let it compile everywhere. Add owned convenience only when the product boundary genuinely benefits from `alloc` or `std`, and gate it so the addition never becomes mandatory for callers that did not ask for it. The dependency point deserves emphasis: a single transitive crate that pulls in `std` with default features will, by itself, end your portability story without any warning. Audit with `default-features = false` and a `no_std` target in CI, not by hope.

Managing memory when there is no std heap

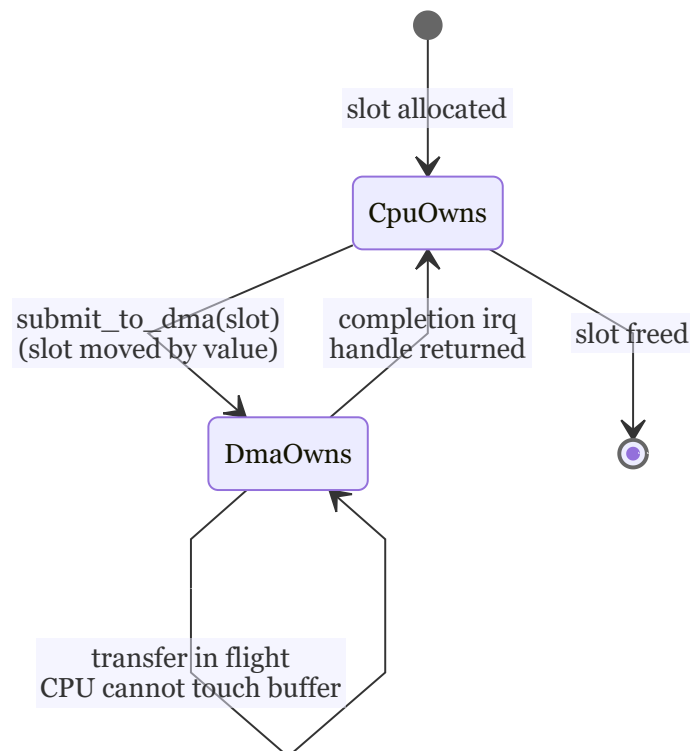
The biggest habit to unlearn from hosted programming is the assumption that allocation always succeeds. On a constrained target, memory is a fixed budget and growth can fail, so the design question shifts from "how do I allocate?" to "what happens when I cannot?" The four strategies below answer that question at increasing levels of dynamism, from no heap at all to a heap whose failure mode you control.

The difference is one method. Where a hosted version would call `push` or `extend_from_slice` and let an allocation failure abort the process, the version below first calls `try_reserve` and returns the error to the caller. The buffer only grows once the capacity is secured, so the out-of-memory case becomes an ordinary `Result` you can test.

Ownership and data layout next to the hardware

Close to the metal, Rust's ownership model stops being a discipline about cleanup and becomes a tool for reasoning about who is physically allowed to touch a piece of memory at a given instant. Two concurrent actors you cannot see from ordinary code (a DMA engine and a hardware register that changes itself) are exactly the cases where a careless `borrow` turns into a real fault. The three ideas below are how you keep that honest.

The key idea in the next listing is that `submit_to_dma` takes the slot by value. Once you call it, you no longer have the slot, so the compiler will reject any attempt to read or write that buffer while the transfer is in flight. The peripheral conceptually owns the memory until it signals completion and the handle is returned. The diagram shows that round trip of authority first, so the by-value parameter reads as deliberate rather than incidental.



The slot handle is the authority token. While DMA owns it, the borrow checker forbids any CPU access to the buffer.

Here the entire unsafe footprint is two one-line functions. Wrapping the raw pointer in a `Register` type means the rest of the driver works with a safe API, and any audit of MMIO correctness reduces to reviewing

these two reads and writes plus the validity of the address that produced the pointer.

Embedded Rust layering: PAC, HAL, BSP, RTIC or Embassy-style execution, interrupts, and critical sections

Interrupt safety usually combines ownership discipline with either atomics or a critical-section model. Do not assume a `Mutex` from `std` exists, and do not assume an interrupt boundary behaves like a thread boundary.

FFI and ABI boundaries for ``no_std`` code: C, C++, kernels, hypervisors, and wasm hosts

Keep panic and allocation policy explicit at the ABI seam. A freestanding or host-constrained caller cannot safely consume an implicit unwind or an unexpected heap contract.

Testing, simulation, debugging, fuzzing, and profiling ``no_std`` crates from a host-side `std` environment

Safety boundaries: unsafe, atomics, MMIO, interrupt concurrency, panic handlers, and audit checklists

- State the invariant before the unsafe block: valid register address, valid buffer slot, or valid FFI pointer contract.
- Use atomics only when the target supports them and the interrupt or multi-core memory model is actually understood.
- Never treat memory-mapped registers like normal Rust references with ordinary aliasing and optimization rules.
- Keep panic behavior explicit: abort, custom panic handler, or host-defined trap semantics.
- Audit custom allocators, linker scripts, startup objects, and boundary layout attributes as part of the release surface.

Production tradeoffs: portability, specialization, observability, binary size, linker scripts, and release engineering

Comparison callout

Production patterns

Keep the smallest cross-target logic in `core`, then add `alloc` or `std` at explicit adapter boundaries instead of letting the runtime surface spread inward accidentally.

Prefer fixed-capacity or fallible-memory paths where bounds are real, and make buffer ownership transfer explicit for peripherals, DMA, or host ABI handoff.

Separate host-side tests from target-specific tests so the portable logic stays fast to validate even when device or guest runtime harnesses are slower.

Treat linker scripts, panic mode, startup, allocator choice, and target runners as part of the release contract, not only as build-system trivia.

Pitfalls and tradeoffs

The common senior mistake is not unsafe syntax. It is an unspoken runtime assumption: assuming an allocator, assuming logs exist, assuming unwind exists, or assuming one dependency will behave the same across all targets.

Examples

Example 1: a portable crate surface split across core, alloc, and std

The smallest API stays slice-based. Owned helpers are alloc-gated. Host diagnostics stay std-gated. This is the shape to prefer before a library accidentally commits to a wider runtime than it needs.

Example 56-1. portable_surface.rs — *Portable surface*

```
#[cfg(feature = "alloc")]
extern crate alloc;

#[cfg(feature = "alloc")]
use alloc::vec::Vec;

fn checksum(bytes: &[u8]) -> u32 {
    bytes.iter().map(|&byte| byte as u32).sum()
}

#[cfg(feature = "alloc")]
fn encode_frame(tag: u8, payload: &[u8]) -> Vec<u8> {
    let mut out = Vec::with_capacity(payload.len() + 1);
    out.push(tag);
    out.extend_from_slice(payload);
    out
}

#[cfg(feature = "std")]
fn write_diagnostic(service: &str, checksum: u32) -> String {
    format!("{:}:{:}", service, checksum)
}

fn main() {
    let payload = [1_u8, 2, 3];
    let sum = checksum(&payload);

    println!("portable modes = core|alloc|std");
    println!("checksum = {}", sum);
    println!("alloc-gated api = encode_frame");
}
```

OUTPUT

```
portable modes = core|alloc|std
checksum = 6
alloc-gated api = encode_frame
```

`checksum` depends only on slices and integers, so it stays portable across firmware, wasm, kernels, and servers.

`encode_frame` is an owned convenience API, not the baseline contract every target must support.

Host formatting or IO belongs at the edge so constrained targets do not inherit it accidentally.

Example 2: fixed-capacity buffer ownership with a DMA-style slot handle

This example uses no heap at all. Allocation is fallible, slots are explicit, and release is visible. That is often calmer than pretending a constrained runtime can always grow another queue.

Example 56-2. `fixed_capacity_dma.rs` — *Capacity dma*

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
struct SlotId(usize);

struct Pool<const SLOTS: usize, const BYTES: usize> {
    used: [bool; SLOTS],
    lens: [usize; SLOTS],
    data: [[u8; BYTES]; SLOTS],
}

impl<const SLOTS: usize, const BYTES: usize> Pool<SLOTS, BYTES> {
    const fn new() -> Self {
        Self {
            used: [false; SLOTS],
            lens: [0; SLOTS],
            data: [[0; BYTES]; SLOTS],
        }
    }

    fn alloc_copy(&mut self, bytes: &[u8]) -> Option<SlotId> {
        if bytes.len() > BYTES {
            return None;
        }

        let mut index = 0;
        while index < SLOTS {
            if !self.used[index] {
                self.used[index] = true;
                self.lens[index] = bytes.len();
                self.data[index][..bytes.len()].copy_from_slice(bytes);
                return Some(SlotId(index));
            }
            index += 1;
        }

        None
    }

    fn as_slice(&self, id: SlotId) -> &[u8] {
        &self.data[id.0][..self.lens[id.0]]
    }

    fn release(&mut self, id: SlotId) {
        self.used[id.0] = false;
    }
}
```

```

        self.lens[id.0] = 0;
    }

    fn in_use(&self) -> usize {
        self.used.iter().filter(|&&used| *used).count()
    }
}

fn main() {
    let mut pool = Pool::new(2, 8);

    let first = pool.alloc_copy(b"abc").unwrap();
    let _second = pool.alloc_copy(b"rust").unwrap();
    let overflow = pool.alloc_copy(b"more").is_none();
    let sent_bytes = pool.as_slice(first).len();

    pool.release(first);

    println!("in_use = {}", pool.in_use());
    println!("overflow = {}", overflow);
    println!("sent bytes = {}", sent_bytes);
}

```

OUTPUT

```

in_use = 1
overflow = true
sent bytes = 3

```

Capacity is part of the type, not a loose operational hope.

Option<SlotId> makes exhaustion explicit and easy to test under load or device stress.

The slot handle is the authority token. That is the same pattern many DMA, queue, and driver APIs want.

The repository also includes standalone Rust source under examples/ch56_no_std_rust_constrained_runtime_derivatives/ so the chapter examples can be reviewed as ordinary files outside the in-browser editor.

Summary

END OF BOOK

Advanced Rust

O. Iakushkin

Set in Georgia and Helvetica Neue; code in Consolas.
Syntax highlighting by Pygments. Rendered with headless Chrome and
PyMuPDF.

Thank you for reading.